

Google Cloud Platform & DevOps

Scenario-based interview questions and troubleshooting scenarios, each with a flow diagram and a short explanation

400 questions · 15 sections · Prepared for Sai Rathish · September 2026

DIAGRAM LEGEND

Component / step

Key component / outcome

Error / fault response

Client or backend system

Numbered dark arrows = request/response steps (explained under each diagram) · red dashed arrows = error path · grey dashed arrows = background/config traffic · coloured containers = zones (grey users / external, blue networking, purple compute / GKE / serverless, green data / storage, teal CI/CD, orange security / IAM, slate observability, amber Google Cloud, sand on-premises).

Contents

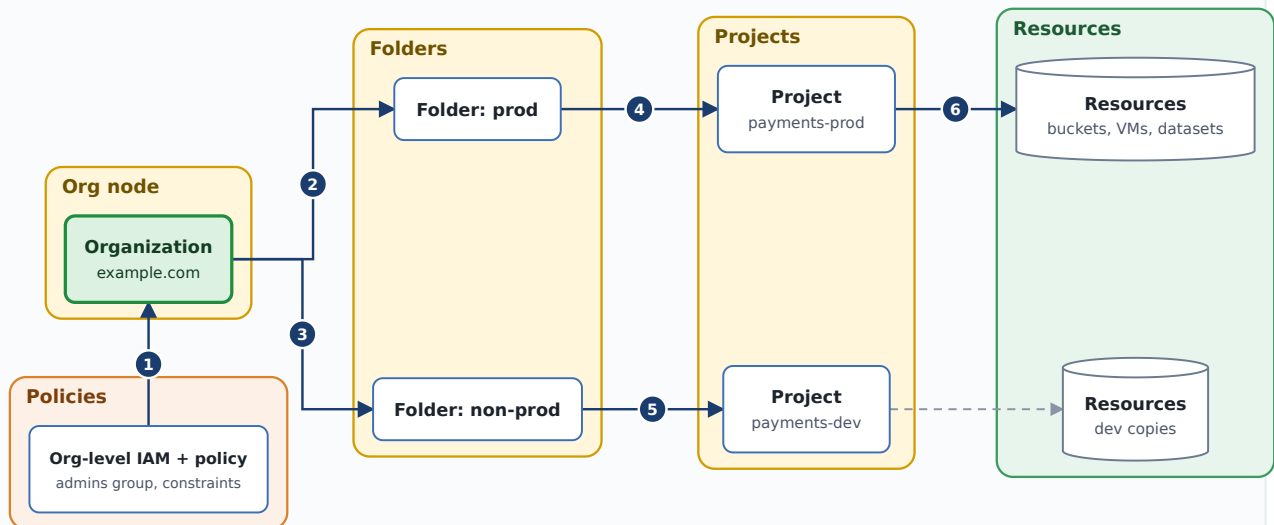
1. GCP Fundamentals: Resource Hierarchy, Projects & Billing	Q1-Q23
2. IAM, Identity & Security	Q24-Q53
3. Networking: VPC, Load Balancing & Hybrid Connectivity	Q54-Q91
4. Compute Engine & Managed Instance Groups	Q92-Q115
5. Google Kubernetes Engine (GKE)	Q116-Q150
6. Serverless & Eventing: Cloud Run, Functions & Pub/Sub	Q151-Q177
7. Storage & Databases	Q178-Q210
8. Observability & SRE	Q211-Q233
9. Terraform & Infrastructure as Code	Q234-Q267
10. CI/CD: Cloud Build, Artifact Registry, Cloud Deploy & Pipelines	Q268-Q299
11. Docker & Containers	Q300-Q319
12. Kubernetes Core Concepts	Q320-Q351
13. Git, DevOps Practices & DevSecOps	Q352-Q370
14. Troubleshooting Scenarios	Q371-Q386
15. Cost Optimization, DR & Architecture Design	Q387-Q400

1. GCP Fundamentals: Resource Hierarchy, Projects & Billing

How organizations, folders and projects are laid out, how policy and cost flow through that tree, and the day-one guardrails every platform team is asked about.

Q1 A company with 40 product teams is moving to Google Cloud. How do you structure the organization, folders and projects, and how do IAM and organization policies inherit down that tree?

FLOW DIAGRAM



- 1 IAM allow bindings and org policies attached once at the org node
- 2 Folders inherit the org's bindings and constraints
- 4 Projects inherit from their folder; lower levels can add access, not remove it
- 6 Resources get the union of all allow bindings above them
- Same rules apply in non-prod, with looser folder-level grants

EXPLANATION

The organization node maps to your Cloud Identity or Workspace domain and sits at the top. Under it, create a shallow folder layer (for example `prod`, `non-prod`, `shared`, then business units if needed) and one project per workload per environment. IAM allow policies are additive: a role granted on a folder applies to every project and resource below it and cannot be revoked lower down, so grant broad roles high only to small admin groups. Organization policies also inherit, but a lower level can override them where the constraint allows it (merge, replace or reset). If you need to take a permission away below a grant, use an IAM deny policy rather than trying to remove it at the project.

KEY POINTS

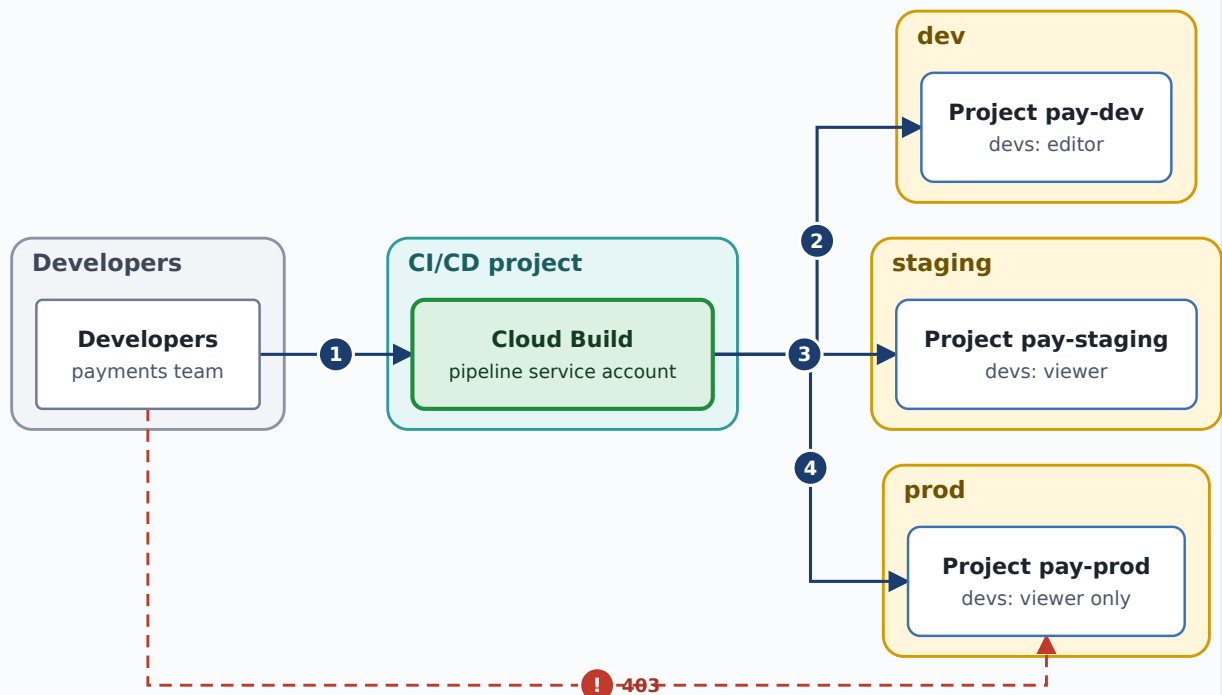
- Keep the hierarchy 2-3 levels deep; deep trees make effective access hard to reason about.
- Bind roles to groups at folder level, not to individuals on hundreds of projects.

CONFIGURATION EXAMPLE

```
gcloud resource-manager folders create --display-name=prod \
  --organization=123456789012
gcloud projects create payments-prod-7f3a --folder=456789012345
gcloud resource-manager folders add-iam-policy-binding 456789012345 \
  --member=group:payments-admins@example.com --role=roles/editor
```

Q2 A team currently runs dev, staging and prod in one project and someone deleted a prod bucket while testing. How should projects be split, and how do you keep humans out of prod?

FLOW DIAGRAM



- 1 Push code; merge to main starts the pipeline
- 2 Deploy on every merge
- 3 Deploy tagged releases for testing
- 4 Deploy after approval; only the pipeline SA can write to prod
- ! 403 humans hold viewer roles in prod, no delete rights

EXPLANATION

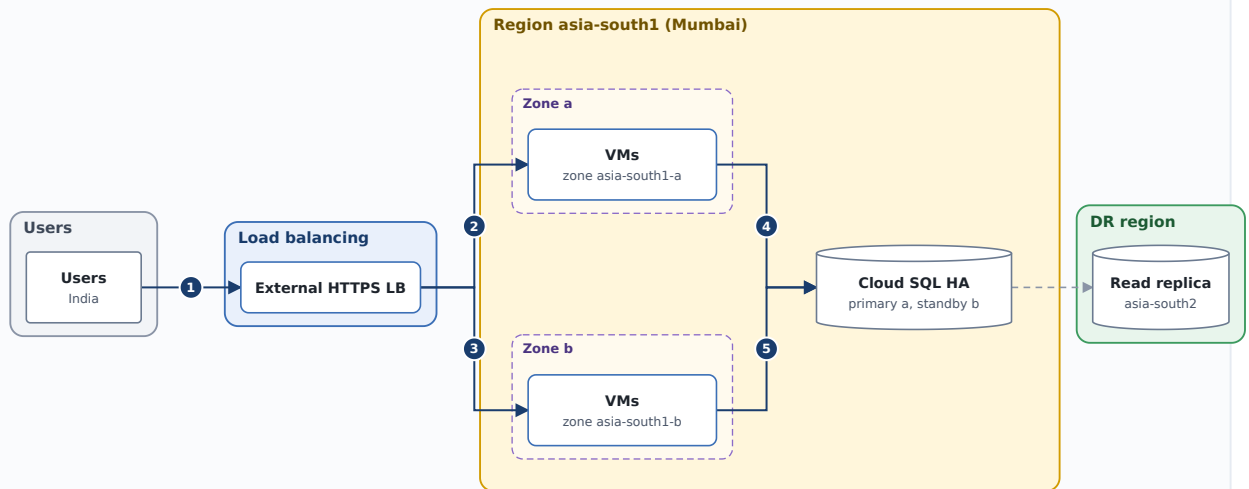
Use one project per application per environment, because a project is the boundary for IAM, quotas, API enablement, billing and many resource names. Developers get broad roles in dev, read-only in staging and prod, and changes reach prod only through a pipeline service account that lives in a separate CI/CD project. Emergency write access to prod goes through a time-bound break-glass group rather than standing roles. Split further (for example a separate data project) only when a team boundary, compliance scope or quota needs it; too many tiny projects add overhead without adding safety.

KEY POINTS

- Project = blast-radius, IAM and quota boundary.
- Production writes come from automation identities, not people.

Q3 You are deploying a latency-sensitive app for users in India that must survive a zone failure and have a DR option. How do you choose regions and zones?

FLOW DIAGRAM



- 1 Users reach the nearest Google edge
- 2 Traffic spread across two zones
- 3 If one zone fails, the other keeps serving
- 4 Regional HA database with synchronous standby in another zone
- 5 Asynchronous cross-region replica, promoted only in a regional disaster

EXPLANATION

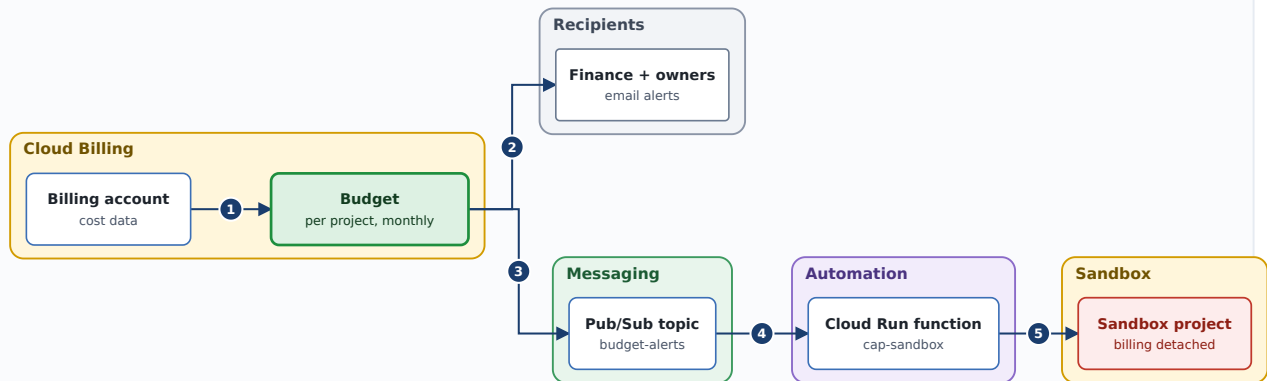
A region is an independent geographic area (for example `asia-south1` in Mumbai) and a zone is a failure domain inside it (`asia-south1-a`, `-b`, `-c`). Pick the region closest to users and data-residency rules that offers every service and machine type you need, then run across at least two zones for zone-level HA using a regional MIG or regional GKE cluster and a regional database. For DR, pair it with a second region (`asia-south2` in Delhi) holding asynchronous replicas and backups. Also compare regional prices and check that quotas and GPU or machine family availability exist in both regions before committing.

KEY POINTS

- Zones protect against a data-centre failure; regions protect against a regional outage.
- Traffic between zones and regions is billed, so keep chatty tiers in the same zone set.

Q4 Finance wants alerts at 50%, 90% and 100% of each team's monthly budget, and sandbox projects must stop spending when they overrun. How do you implement it?

FLOW DIAGRAM



- 1 Spend is compared with the budget several times a day
- 2 Emails at 50%, 90% and 100% thresholds
- 3 Every evaluation also publishes JSON with cost and budget amount
- 4 Function reads costAmount vs budgetAmount
- 5 Over 100%: unlink billing, which stops paid resources

EXPLANATION

Create a Cloud Billing budget per team or project with threshold rules at 0.5, 0.9 and 1.0, on actual or forecasted spend, and send emails to billing admins and a Monitoring notification channel. Budgets only notify; they never cap spend on their own. For sandboxes, connect the budget to a Pub/Sub topic and run a small function that disables billing on the project when spend passes 100%; this stops resources and can delete some data, so use it only where that is acceptable. Cost data can lag by several hours, so set thresholds with some headroom.

KEY POINTS

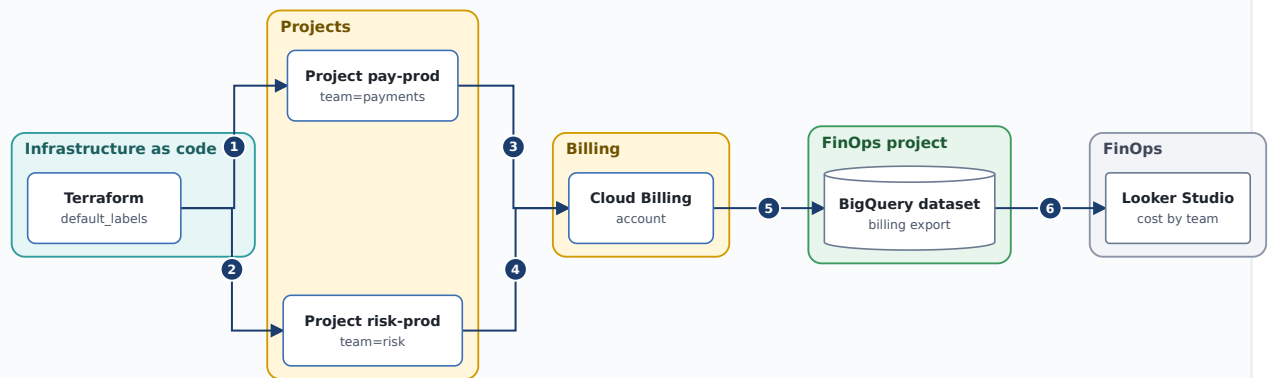
- Budgets alert; only automation (or quotas) can actually stop spend.
- Forecasted-spend thresholds warn earlier than actual-spend ones.

CONFIGURATION EXAMPLE

```
gcloud billing budgets create --billing-account=0X0X0X-0X0X0X-0X0X0X \
  --display-name="sandbox-42 monthly" --budget-amount=500USD \
  --filter-projects=projects/sandbox-42 \
  --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0 \
  --notifications-rule-pubsub-topic=projects/finops/topics/budget-alerts
```

Q5 Half the monthly bill shows as unallocated because resources have no team label, and FinOps wants a daily cost-per-team dashboard. What do you put in place?

FLOW DIAGRAM



- 1 Required labels on the project and on every resource
- 3 Usage rows carry resource labels and project labels
- 5 Standard + detailed usage cost export
- 6 Group by label team, include credits

EXPLANATION

First agree a small label schema (team , env , cost-center , app) and apply it from Terraform with provider default_labels , plus labels on each project, since project labels appear on every billing row as project.labels . Enable Cloud Billing export to a BigQuery dataset in a FinOps project (standard and detailed usage cost), and enable it early because history before enablement is not guaranteed. Build a view or scheduled query that sums cost plus credits by label and feed Looker Studio. Catch gaps with a CI policy check on Terraform plans and a periodic Asset Inventory search for resources missing the label.

KEY POINTS

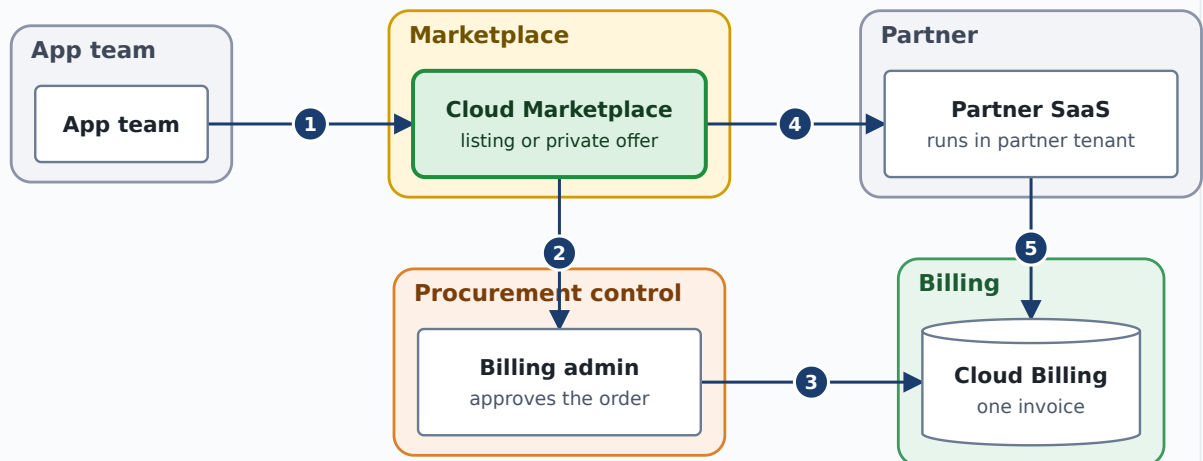
- Label the project as well as resources; project labels cover services that do not support resource labels.
- Net cost = cost + SUM(credits.amount); credits are negative.

CONFIGURATION EXAMPLE

```
SELECT l.value AS team,
       SUM(cost) + SUM(IFNULL((SELECT SUM(c.amount) FROM UNNEST(credits) c), 0)) AS net_cost
FROM `finops.billing.gcp_billing_export_v1_XXXXXX`, UNNEST(project.labels) AS l
WHERE l.key = 'team' AND invoice.month = '202609'
GROUP BY team ORDER BY net_cost DESC
```

Q6 A team wants to buy a partner product (for example a managed database or security tool) through Google Cloud Marketplace. What are the benefits, and how do you keep purchasing under control?

FLOW DIAGRAM



- ❶ Pick a product or accept a negotiated private offer
- ❷ Purchase needs a billing or procurement admin role
- ❸ Order is attached to the Cloud Billing account
- ❹ Partner provisions the service and links your account
- ❺ Usage is reported to Google and billed with GCP

EXPLANATION

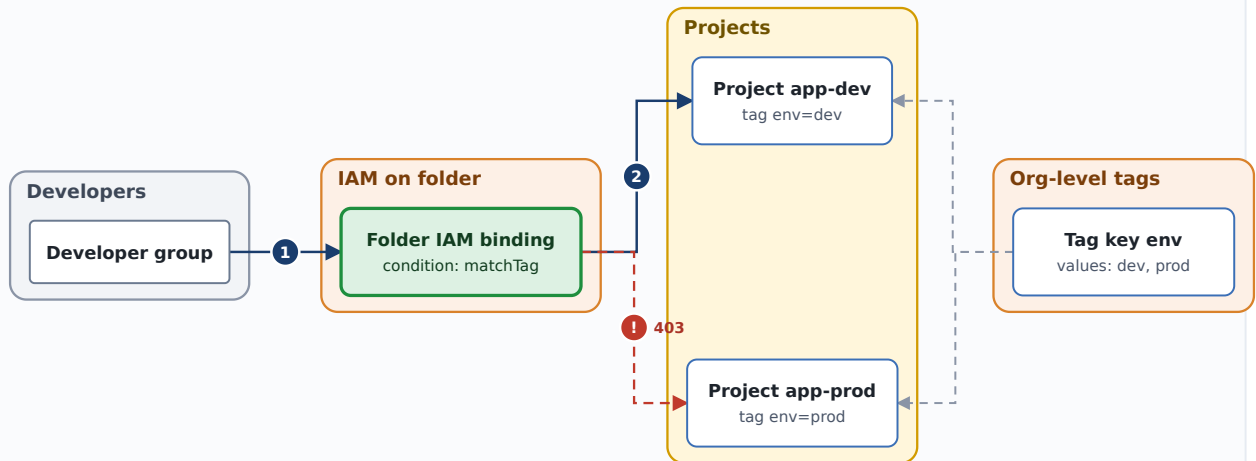
Marketplace products are billed through your Cloud Billing account, so there is no separate vendor contract to set up, one invoice, and eligible purchases can count toward a Google Cloud spend commitment. Private offers let procurement negotiate price and terms with the partner and accept them in the console. Control spend by limiting who holds Billing Account Administrator or procurement order roles, so other users can only request a purchase for approval. Check each listing's deployment model: some deploy VMs or GKE apps into your project, while SaaS listings run in the partner's environment and connect over the internet or Private Service Connect.

KEY POINTS

- Billing-account roles, not project roles, gate Marketplace purchases.
- SaaS data leaves your projects; review it like any third-party vendor.

Q7 Security wants developers to have instance-admin rights only on resources marked as dev, across many projects. Why is a label not enough, and how do tags solve it?

FLOW DIAGRAM



- 1 Start a VM: `compute.instances.start`
- 2 Condition true for `env=dev`, access granted
- ! 403 condition false for `env=prod`
- Tag value bound to the project, inherited by its resources
- Labels are free-form metadata; IAM and org policy conditions cannot read them.

EXPLANATION

Labels are key-value metadata on individual resources, used for billing breakdowns and filtering; anyone with update rights on the resource can change them and IAM does not evaluate them. Resource Manager tags are separate resources defined at the org or project level, have their own IAM (who can create values and who can bind them), are inherited down the hierarchy, and can be referenced in IAM Conditions and organization policy conditions with `resource.matchTag()`. So create a tag key `env` with values `dev` and `prod`, bind values on folders or projects, and grant the role on a folder with a condition that matches `env=dev`. Restrict `tagUser` rights so developers cannot re-tag prod as dev.

KEY POINTS

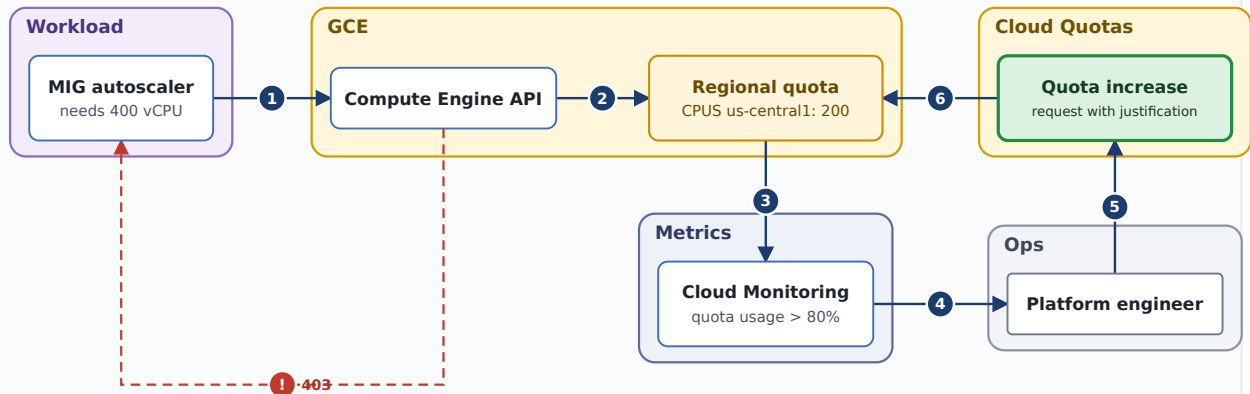
- Tags = access and policy control; labels = cost and inventory metadata.
- Network firewall policies can also target secure tags on VMs.

CONFIGURATION EXAMPLE

```
gcloud resource-manager tags keys create env --parent=organizations/123456789012
gcloud resource-manager tags values create dev --parent=123456789012/env
gcloud resource-manager tags bindings create --tag-value=123456789012/env/dev \
  --parent=//cloudresourcemanager.googleapis.com/projects/app-dev
# IAM condition expression:
# resource.matchTag('123456789012/env', 'dev')
```

Q8 A product launch needs 400 vCPUs in us-central1, but the project's regional CPU quota is 200 and scale-out fails. How do you handle it now and avoid it next time?

FLOW DIAGRAM



- 1 Create more instances
- 2 Check regional CPU quota
- ! 403 QUOTA_EXCEEDED: CPUS in us-central1
- 3 Usage vs limit exported as metrics
- 4 Alert before the limit is hit
- 5 Request the new value days before launch
- 6 Approved value becomes the new limit

EXPLANATION

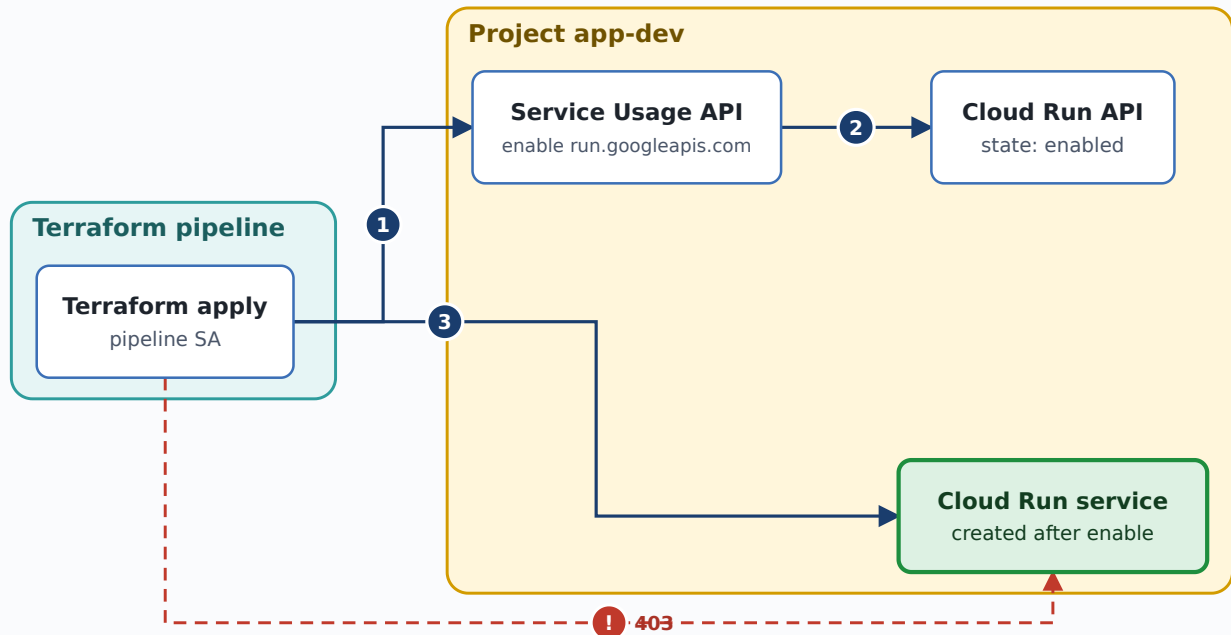
Quotas are per project (often per region and per machine family) and protect both you and Google capacity; system limits are fixed and cannot be raised. Check current values in the console under IAM & Admin > Quotas & System Limits or with `gcloud compute regions describe`, then request an increase there or through the Cloud Quotas API, giving the value, region and a short justification; large jumps can take longer and may need billing history. For the launch, also check family-specific quotas (for example N2 CPUs), IP addresses and persistent disk. Prevent repeats with Monitoring alerts on quota usage and a pre-launch capacity checklist.

KEY POINTS

- Raise quotas days before a launch, not during the incident.
- Quota is not a capacity reservation; use reservations when you need guaranteed capacity.

Q9 A Terraform run in a new project fails with 'Cloud Run Admin API has not been used in project ... or it is disabled'. How do you manage API enablement at scale?

FLOW DIAGRAM



- 1 google_project_service resource runs first
- 2 API turned on; can take a minute to propagate
- 3 Service created via depends_on
- ! 403 SERVICE_DISABLED when the enable step is missing

EXPLANATION

Every Google API must be enabled per project before use, and the caller needs `serviceusage.services.enable` (for example Service Usage Admin). Declare required APIs in Terraform with `google_project_service` (or the project factory's `activate_apis` list), make dependent resources `depends_on` them, and set `disable_on_destroy = false` so a refactor does not turn an API off under running workloads. Allow for propagation delay after enabling. Keep the API list in the project template so every new project gets the same baseline, and review which APIs are enabled with `gcloud services list --enabled`.

KEY POINTS

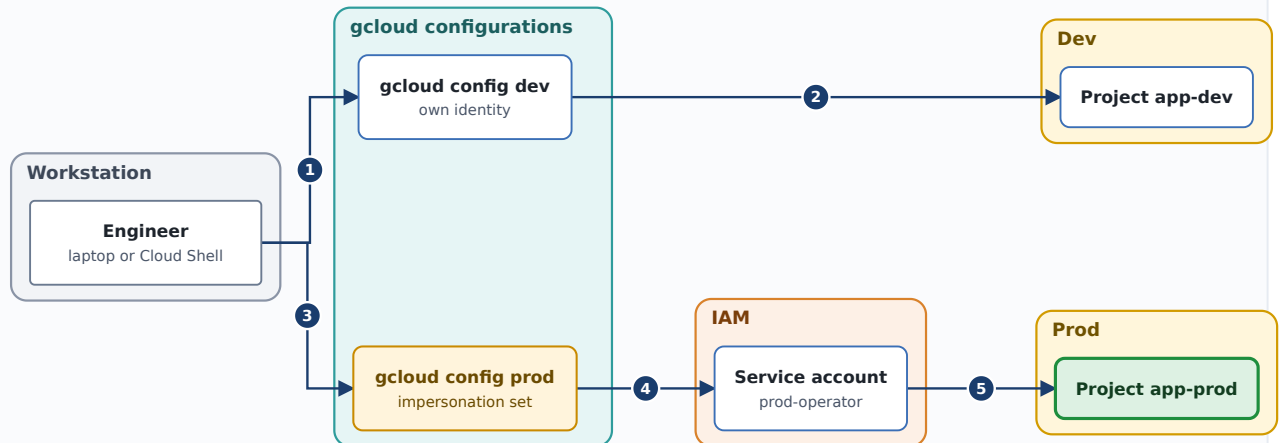
- Enabling an API can create service agents and default grants; plan for them in IAM reviews.
- Disabling an API can break or delete dependent resources.

CONFIGURATION EXAMPLE

```
resource "google_project_service" "run" {
  project      = "app-dev-01"
  service      = "run.googleapis.com"
  disable_on_destroy = false
}
# resource "google_cloud_run_v2_service" "api" { ... depends_on = [google_project_service.run] }
```

Q10 An engineer who works across dev and prod ran a destructive gcloud command in prod by mistake. How do you set up gcloud and Cloud Shell to make that much harder?

FLOW DIAGRAM



- 1 Default configuration for daily work
- 2 User credentials, dev roles only
- 3 Activated explicitly for one shell
- 4 Needs Token Creator on the SA, logged in audit logs
- 5 Short-lived token for prod actions

EXPLANATION

Create named gcloud configurations (`dev` , `prod`) that each set account, project and default region, keep `dev` as the default, and activate `prod` only for a single shell with `CLOUDSDK_ACTIVE_CONFIG_NAME=prod` . Do not give humans standing write roles in prod; set `auth/impersonate_service_account` in the prod configuration so prod actions need Service Account Token Creator, which you can grant just in time. Show the active project in the shell prompt, and in scripts always pass `--project` explicitly. Cloud Shell gives a temporary VM with a 5 GB persistent home directory and preinstalled tools, which is handy for break-glass work without local credentials.

KEY POINTS

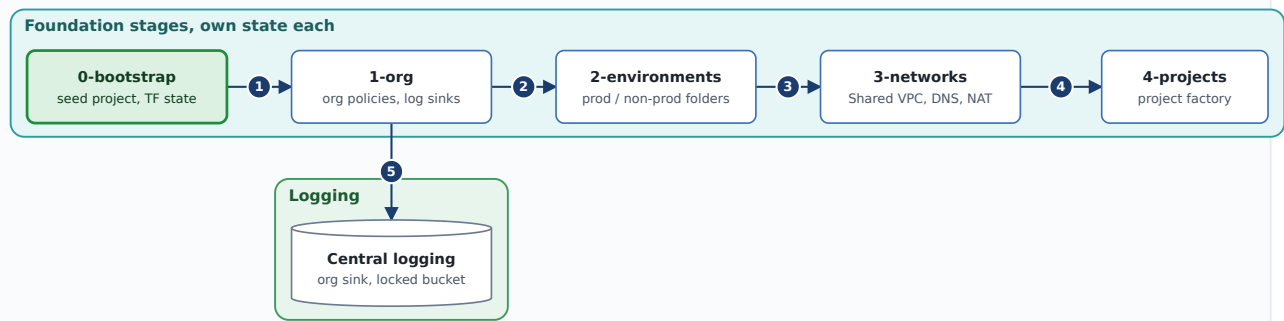
- Impersonation leaves an audit trail and needs no downloaded keys.
- Scripts should never rely on whatever project is currently set.

CONFIGURATION EXAMPLE

```
gcloud config configurations create prod
gcloud config set project app-prod-01
gcloud config set compute/region europe-west2
gcloud config set auth/impersonate_service_account \
  prod-operator@app-prod-01.iam.gserviceaccount.com
gcloud config configurations activate dev
```


Q11 You are asked to build a Google Cloud landing zone for a regulated company before any workloads move. What goes into it and how do you deliver it?

FLOW DIAGRAM



- ❶ Seed project holds the state bucket and CI service accounts
- ❷ Org-wide policies, SCC, audit and billing exports
- ❸ Folder per environment with its own IAM
- ❹ Hub networks, hybrid links, firewall policies
- ❺ Aggregated sink sends all audit logs to a central project

EXPLANATION

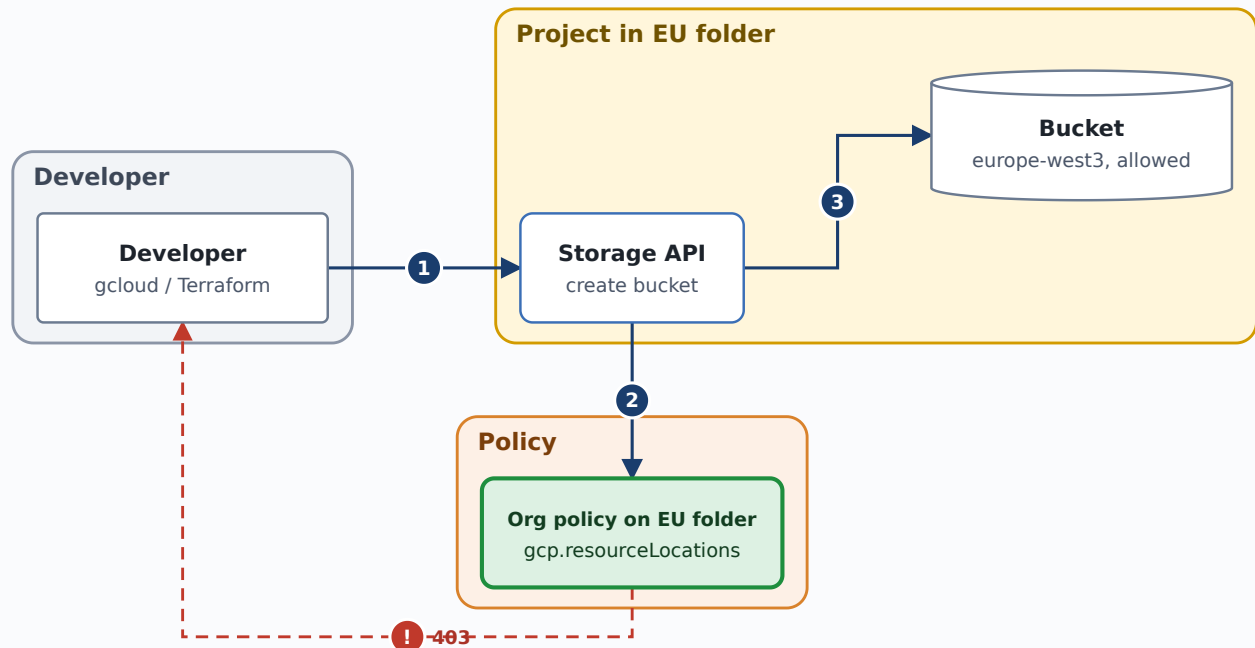
A landing zone is the baseline every project inherits: resource hierarchy, identity and groups, organization policies, centralized audit logging, networking (Shared VPC, DNS, hybrid connectivity, egress), security tooling (Security Command Center, KMS), billing export and a project factory. Deliver it as Terraform in stages, for example Google's terraform-example-foundation or Cloud Foundation Fabric: bootstrap, org, environments, networks, projects, each with its own state and a least-privilege service account. Run it from a pipeline with plan review, so the org is never changed by hand. Agree the design (IP plan, naming, folder layout, who owns what) before writing code, because these are hard to change later.

KEY POINTS

- Separate state and service account per stage limits blast radius.
- Centralize logs and billing export on day one; history cannot be recreated.

Q12 Regulation says customer data must stay in the EU. How do you stop anyone creating resources in us-central1 and what does the control not cover?

FLOW DIAGRAM



- 1 Request a bucket in us-central1, then in europe-west3
- 2 Location checked against the effective policy
- ! 403 violates constraint gcp.resourceLocations
- 3 EU location matches in:eu-locations, created

EXPLANATION

Set the organization policy constraint `gcp.resourceLocations` on the org or the EU folder with the value group `in:eu-locations`, so supported services reject creation of regional or zonal resources elsewhere. It is checked only at creation time: existing resources are not moved or deleted, so find them with Cloud Asset Inventory. It covers supported services only, and some global resources or multi-region choices behave differently, so read the service list. Location rules do not stop data being copied out by a user with read access; combine with VPC Service Controls and IAM for exfiltration risk.

KEY POINTS

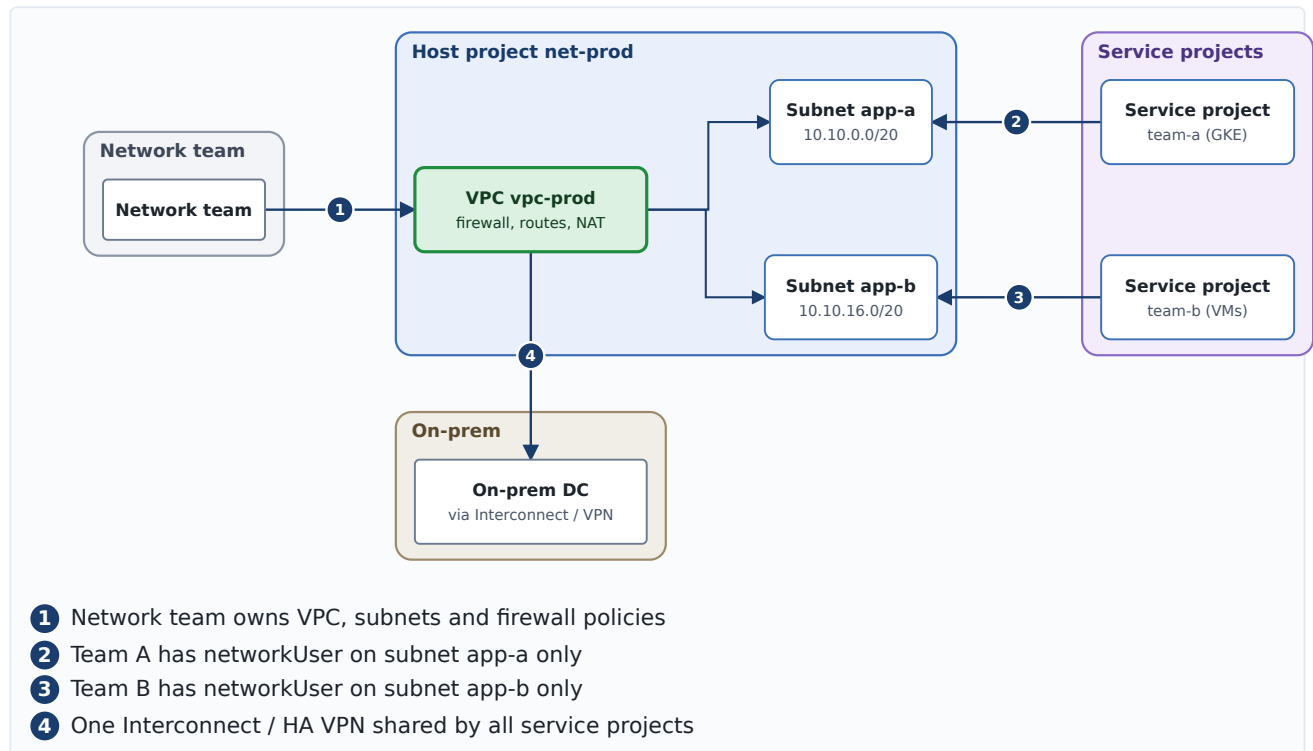
- Org policies restrict configuration; IAM decides who can act.
- Test new constraints on a sandbox folder before applying org-wide.

CONFIGURATION EXAMPLE

```
# policy.yaml, applied with: gcloud org-policies set-policy policy.yaml
name: folders/456789012345/policies/gcp.resourceLocations
spec:
  rules:
    - values:
        allowedValues:
          - in:eu-locations
```

Q13 Several app teams need private connectivity to on-prem, but the network team wants central control of subnets and firewalls. How does Shared VPC fit, at a high level?

FLOW DIAGRAM



EXPLANATION

Shared VPC lets a host project own one VPC network while attached service projects create VMs, GKE nodes and other resources in its subnets. The network team keeps control of IP ranges, routes, firewall rules and hybrid links, while app teams keep their own projects for IAM, billing and quotas. Grant `roles/compute.networkUser` on individual subnets to each team's group and service accounts, instead of on the whole host project. A service project attaches to exactly one host project, and a host project cannot also be a service project; typical designs use one host project per environment.

KEY POINTS

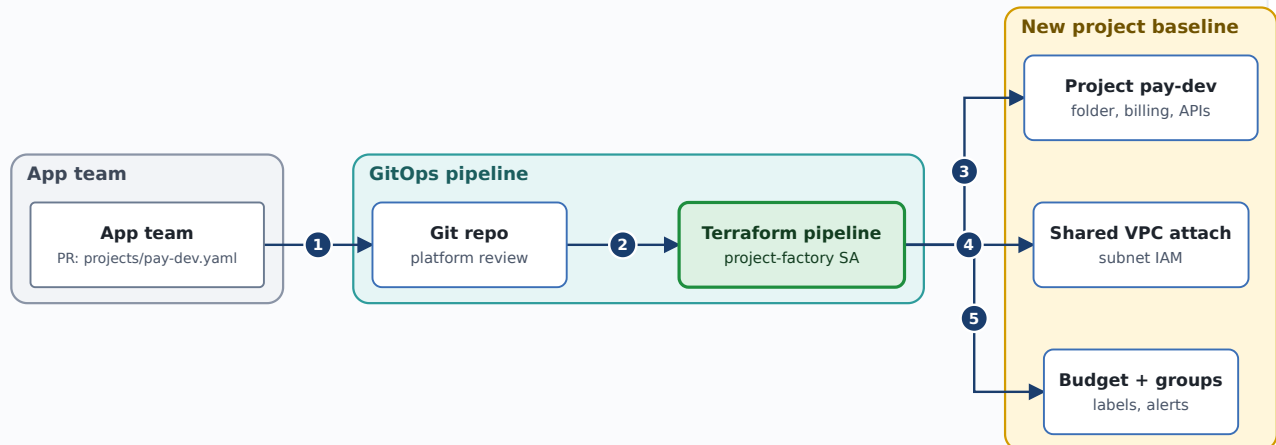
- Separation of duties: network admins in the host, app admins in service projects.
- Costs of VMs land in service projects; network egress is billed where the resource runs.

CONFIGURATION EXAMPLE

```
gcloud compute shared-vpc enable net-prod
gcloud compute shared-vpc associated-projects add team-a-prod --host-project=net-prod
gcloud compute networks subnets add-iam-policy-binding app-a \
  --project=net-prod --region=europe-west1 \
  --member=group:team-a-devs@example.com --role=roles/compute.networkUser
```

Q14 Teams open tickets for new projects and wait a week; each project ends up configured differently. How do you build a self-service project factory with guardrails?

FLOW DIAGRAM



- ❶ Request is a small YAML file in a pull request
- ❷ Merge after review starts plan and apply
- ❸ Create in the right folder, link billing, enable APIs
- ❹ Attach to the environment's host project
- ❺ Budget, IAM groups and required labels

EXPLANATION

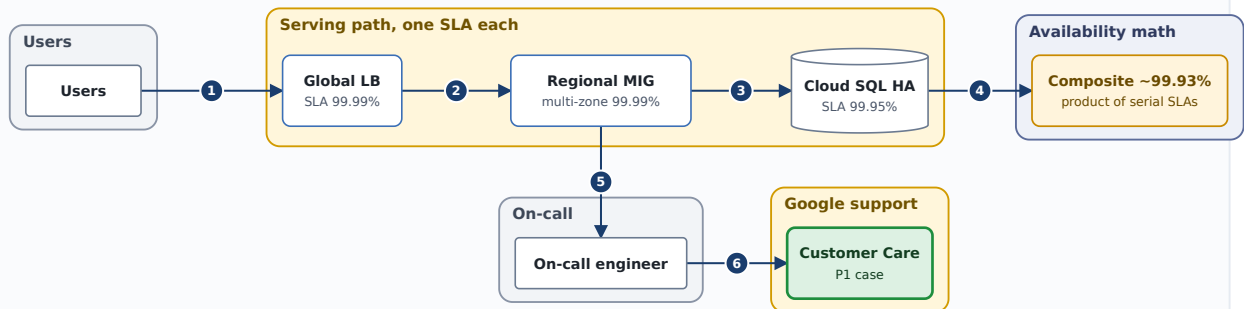
Keep one YAML file per project in Git describing name, folder, environment, labels, APIs, network and budget. A pipeline running Terraform (for example the `terraform-google-modules/project-factory` module or Fabric's project factory) creates the project in the right folder, links billing, enables APIs, attaches it to Shared VPC, sets group-based IAM, creates a budget and removes default service account grants. Platform engineers review the pull request, and the pipeline service account is the only identity with project-creator and billing-user rights. Teams get a project in minutes, and every project has the same baseline.

KEY POINTS

- The factory SA holds Project Creator and Billing Account User; people do not.
- Set `iam.automaticIamGrantsForDefaultServiceAccounts` so default SAs are not Editor.

Q15 Before a production launch, leadership asks what availability you can promise and what support you get from Google if something breaks. How do you answer?

FLOW DIAGRAM



- ④ Serial dependencies multiply: $0.9999 \times 0.9999 \times 0.9995$
- ⑤ Monitoring alert pages on-call
- ⑥ Open P1 with project ID, impact and timeline

EXPLANATION

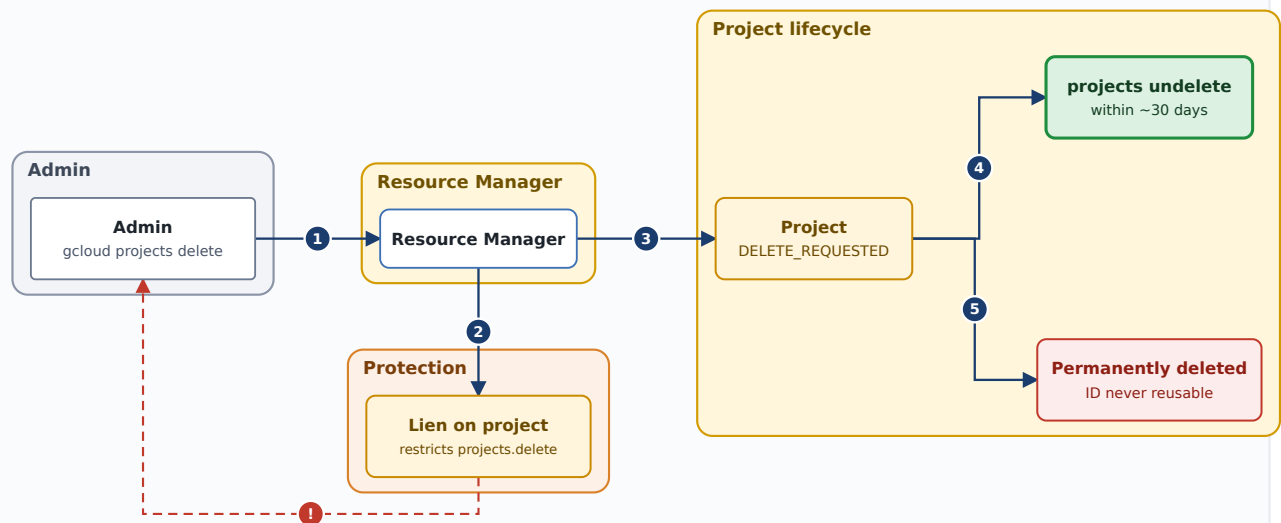
Each service publishes its own SLA (check the current SLA pages), and serial dependencies multiply, so the whole path is lower than any single component: three services at 99.99%, 99.99% and 99.95% give about 99.93%. SLAs give financial credits, not a guarantee, and usually require a supported setup such as multi-zone deployment. For support, Basic covers billing only; Standard, Enhanced and Premium add technical cases with faster response targets, and Enhanced or Premium are the usual choice for production P1 cover, with Premium adding a technical account manager. Set your own SLO a bit below the composite figure and design redundancy for the parts you control.

KEY POINTS

- Your SLO should be tighter than what you promise customers and looser than the composite SLA.
- Put project IDs, times and error samples in the case to cut response time.

Q16 An engineer nearly deleted the Shared VPC host project. How do you protect critical projects, and what actually happens when a project is deleted?

FLOW DIAGRAM



- 1 Delete request
- 2 Liens are checked first
- ! FAILED_PRECONDITION while a lien exists
- 3 Lien removed: project shut down, resources stop
- 4 Undelete restores the project in the grace period
- 5 After the period, resources are purged

EXPLANATION

Deleting a project first marks it `DELETE_REQUESTED`: billing and resources stop, and an owner can restore it with `gcloud projects undelete` for about 30 days, after which it is permanently removed and the project ID can never be reused. Protect critical projects with a lien on `resourcemanager.projects.delete` (Shared VPC host projects get one automatically), grant `resourcemanager.projects.delete` only to a few admins, and manage projects through Terraform with `prevent_destroy`. Before deleting any project, check dependencies such as Shared VPC attachments, KMS keys, log sinks and service accounts used elsewhere, and export any data you must keep.

KEY POINTS

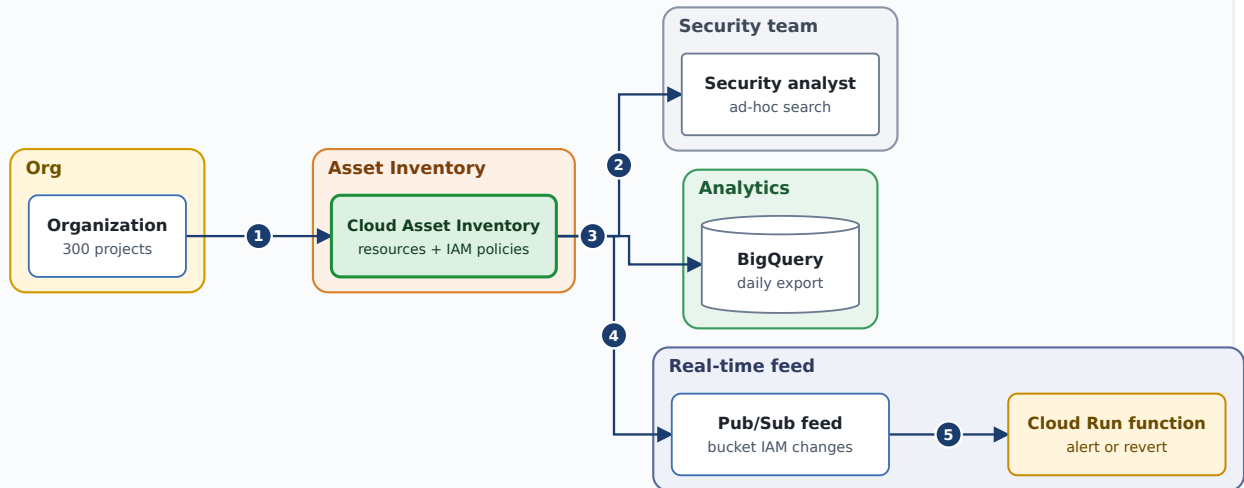
- Some resources (for example data in some services) may not be recoverable even if the project is restored.
- A lien blocks deletion until someone with lien permissions removes it.

CONFIGURATION EXAMPLE

```
gcloud alpha resource-manager liens create --project=net-prod \
  --restrictions=resourcemanager.projects.delete \
  --reason="Shared VPC host - do not delete" --origin=platform-team
gcloud projects undelete app-old-01
```

Q17 Security asks which storage buckets across 300 projects are public and wants to be alerted when any bucket becomes public. How do you answer and automate it?

FLOW DIAGRAM



- 1 Metadata indexed org-wide, with 35 days of history
- 2 search-all-iam-policies for allUsers bindings
- 3 Scheduled export for SQL reporting
- 4 Feed fires when a bucket IAM policy changes
- 5 Notify the owner or remove the binding

EXPLANATION

Cloud Asset Inventory indexes resource metadata and IAM policies across the organization, with change history for the last 35 days. Use `gcloud asset search-all-iam-policies` at org scope to find `allUsers` or `allAuthenticatedUsers` bindings, and `search-all-resources` to list buckets by location or label. For ongoing reporting export snapshots to BigQuery, and for real-time alerts create an asset feed on bucket IAM changes to Pub/Sub that triggers a function. The long-term fix is preventive: enforce `storage.publicAccessPrevention` and uniform bucket-level access by organization policy.

KEY POINTS

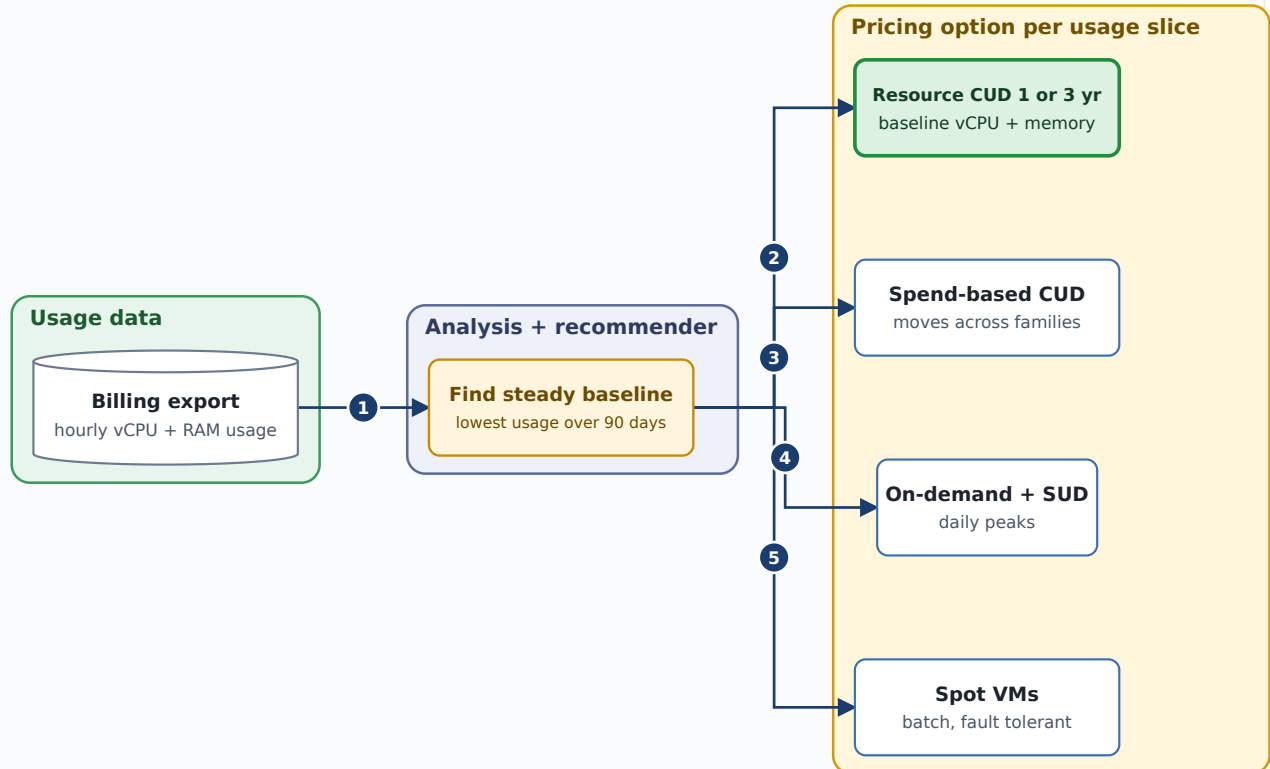
- Search APIs need Cloud Asset Viewer at the scope you query.
- Detection (feeds) plus prevention (org policy) beats either alone.

CONFIGURATION EXAMPLE

```
gcloud asset search-all-iam-policies --scope=organizations/123456789012 \
  --query="policy:allUsers OR policy:allAuthenticatedUsers" \
  --asset-types=storage.googleapis.com/Bucket
gcloud asset feeds create public-buckets --organization=123456789012 \
  --asset-types=storage.googleapis.com/Bucket --content-type=iam-policy \
  --pubsub-topic=projects/sec-ops/topics/asset-changes
```

Q18 Compute spend is about \$80k a month and most of it is steady. How do you use sustained use discounts, committed use discounts and Spot to cut the bill without losing flexibility?

FLOW DIAGRAM



- 1 Look at the always-on minimum per region and family
- 2 Always-on floor: biggest discount, fixed region + family
- 3 Steady spend that may change machine types
- 4 Variable peaks; SUD applies automatically on eligible families
- 5 Interruptible work at a large discount

EXPLANATION

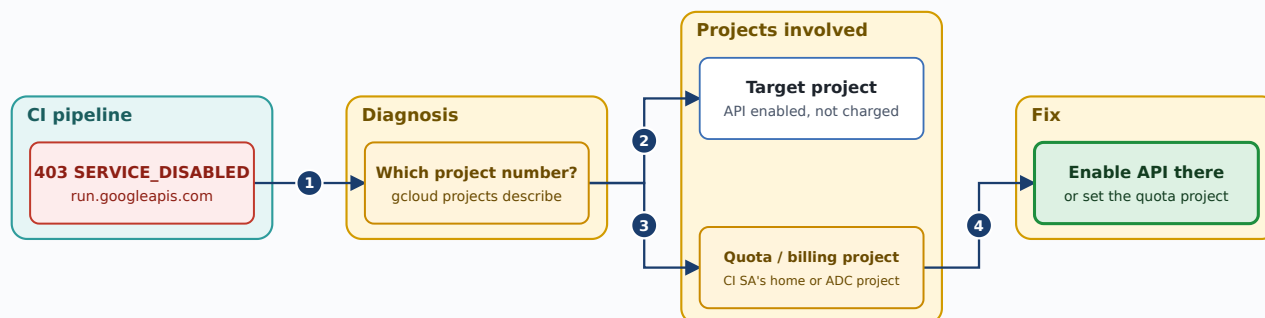
Sustained use discounts apply automatically to eligible machine families (such as N1 and N2, not E2) when a VM runs for a large part of the month. Committed use discounts trade a 1- or 3-year commitment for a larger discount: resource-based CUDs commit vCPU and memory for a machine family in a region, while spend-based (flexible) CUDs commit an hourly spend and move across families and some services. Cover only the steady floor you are sure of, using billing export data and the Recommender's CUD suggestions, leave peaks on demand, and move batch and fault-tolerant work to Spot VMs. Turn on commitment sharing at the billing account so one project's unused commitment covers another's usage.

KEY POINTS

- Commitments are billed whether or not you use them.
- Right-size first, then commit; committing to waste locks it in.

Q19**Troubleshooting**

A CI job fails with `SERVICE_DISABLED`: Cloud Run Admin API has not been used in project 948213... before or it is disabled, but the API is enabled in the target project. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Map the number in the error to a project ID
- 2 Not the target: API state there is irrelevant
- 3 Usually the SA's own project or the ADC quota project
- 4 Enable the API in that project or change it

EXPLANATION

The error always names the project whose API state was checked, and that is the quota or billing project of the call, not always the project you deploy into. With user credentials (ADC) the quota project comes from `gcloud auth application-default set-quota-project` or `GOOGLE_CLOUD_QUOTA_PROJECT`; for some client-based APIs it is the project that owns the service account. First map the number to a project ID, then either enable the API there or point the quota project at the target project. If the API was enabled a minute ago, wait a few minutes and retry, because enabling propagates asynchronously.

KEY POINTS

- Enable APIs in Terraform (`google_project_service`) with `disable_on_destroy = false` so every project has the same set.
- Set the quota project explicitly in CI instead of relying on defaults.

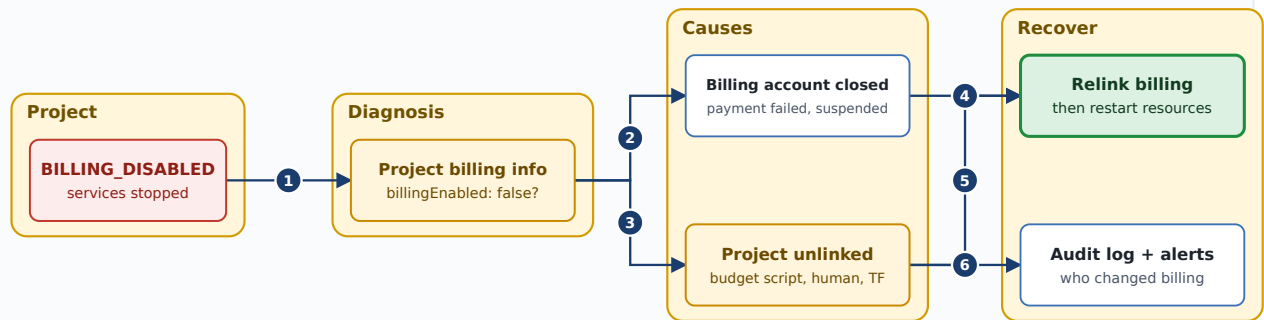
CONFIGURATION EXAMPLE

```

gcloud projects list --filter='projectNumber=948213000123' --format='value(projectId)'
gcloud services list --enabled --project=ci-tools | grep run.googleapis.com
gcloud services enable run.googleapis.com --project=ci-tools
# or charge the call to the target project
gcloud auth application-default set-quota-project prod-app
gcloud config set billing/quota_project prod-app
  
```

Q20**Troubleshooting**

Overnight a project's Cloud Run services return errors, VMs are stopped and API calls fail with `BILLING_DISABLED`: This API method requires billing to be enabled. How do you find the cause and recover?

FLOW DIAGRAM

- 1 Check which billing account the project uses
- 2 Account itself is closed or payment overdue
- 3 Project was detached from the account
- 4 Fix payment in Billing console, or link another account
- 6 Find who unlinked it in the audit logs

EXPLANATION

`gcloud billing projects describe` shows whether billing is enabled and which account is linked. If the billing account is closed or suspended (failed payment, expired card, reseller issue), fix it in the Billing console or link the project to another open account. If the project was unlinked, look in the audit logs for `UpdateProjectBillingInfo`: a budget-cap Cloud Function, a Terraform change or a person may have done it. After relinking, APIs work again within minutes, but stopped VMs and some resources must be started again, and resources deleted after a long unbilled period cannot come back.

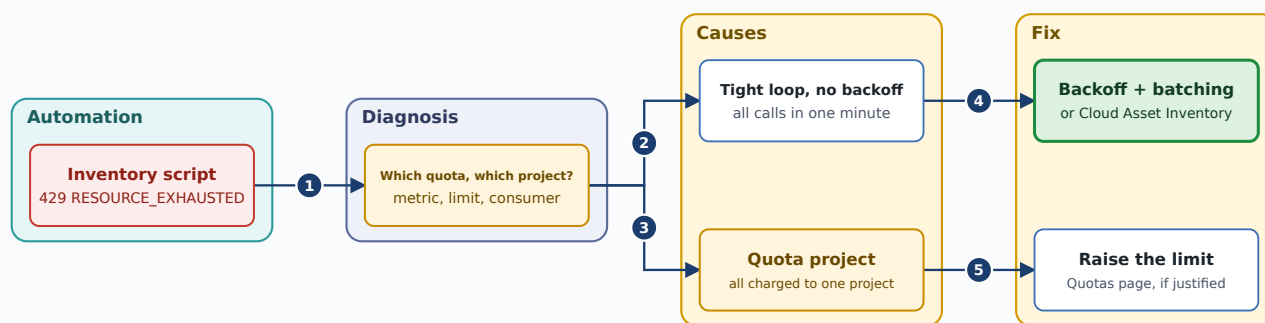
KEY POINTS

- Only use automatic billing disablement on sandbox projects, never on prod.
- Restrict `billing.resourceAssociations.delete` and `roles/billing.admin` to a small group.

CONFIGURATION EXAMPLE

```

gcloud billing projects describe prod-app
gcloud billing accounts describe 01A2B3-C4D5E6-F7G8H9 --format='value(open)'
gcloud logging read 'protoPayload.methodName:"UpdateProjectBillingInfo"' \
  --project=prod-app --freshness=2d --format='value(protoPayload.authenticationInfo.principalEmail)'
gcloud billing projects link prod-app --billing-account=01A2B3-C4D5E6-F7G8H9
gcloud compute instances start web-1 web-2 --zone=asia-south1-a
  
```

Q21**Troubleshooting****A nightly inventory script that lists instances across 300****projects fails with 429 RESOURCE_EXHAUSTED: Quota exceeded for quota****metric 'Read requests' and limit 'Read requests per minute'. How do you fix it?****FLOW DIAGRAM**

- ❶ Error names service, metric and per-minute limit
- ❷ Calls burst far above the per-minute rate
- ❸ Every call counts against the quota project
- ❹ Exponential backoff with jitter, fewer calls
- ❺ Request increase only after fixing the pattern

EXPLANATION

This is a rate quota on API requests, not a resource quota like CPUs: the limit resets every minute and is counted against the consumer (quota) project of the caller. The usual cause is a script firing thousands of list calls in parallel without retries, so fix the client first: exponential backoff with jitter on 429, lower concurrency and page sizes that reduce calls. For inventory across many projects, one Cloud Asset Inventory search or export replaces hundreds of per-project list calls. If the need is still real, request a higher rate limit on the Quotas page for that API.

KEY POINTS

- Google client libraries retry 429 with backoff when retries are enabled; do not disable them.
- Watch `serviceruntime.googleapis.com/quota/rate/net_usage` to see how close you run.

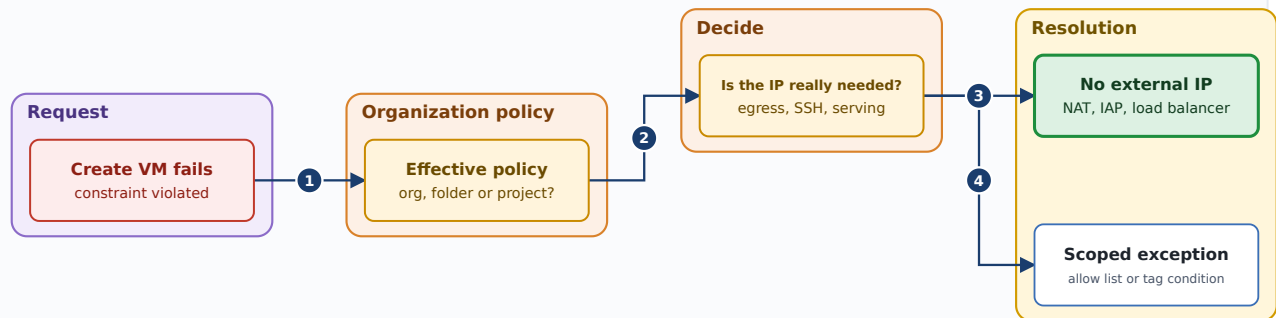
CONFIGURATION EXAMPLE

```

gcloud logging read 'protoPayload.status.code=8' --project=ops-tools --freshness=1d --limit=5
# one call instead of one per project
gcloud asset search-all-resources --scope=organizations/123456789012 \
  --asset-types=compute.googleapis.com/Instance \
  --format='table(name.basename(),location,project)'
```

Q22**Troubleshooting****Creating a VM fails with Constraint**

`constraints/compute.vmExternalIpAccess` violated for project `prod-web`. The team says the VM needs a public IP. How do you troubleshoot and resolve it?

FLOW DIAGRAM

- 1 Find where the policy is set and inherited from
- 2 Ask what the IP is for
- 3 Usual answer: Cloud NAT, IAP TCP or an LB
- 4 Rare case: allow this VM only, with approval

EXPLANATION

The error names the constraint, so read the effective policy on the project with `gcloud org-policies describe --effective` and see at which level it is set. `compute.vmExternalIpAccess` is a list constraint that allows external IPs only for VMs listed in it, so the safe fix is usually to avoid the public IP: Cloud NAT for outbound traffic, IAP TCP forwarding for SSH, and a load balancer for serving. If an exception is truly required, the policy admin adds that single VM's full resource path to the allowed values on the project (or uses a tag-based condition), with a ticket and review. Developers cannot bypass it themselves because they lack `orgpolicy.policyAdmin`.

KEY POINTS

- Test new constraints with dry-run policies and Policy Simulator before enforcing.
- Document the approved exceptions process so teams do not ask for the whole policy to be removed.

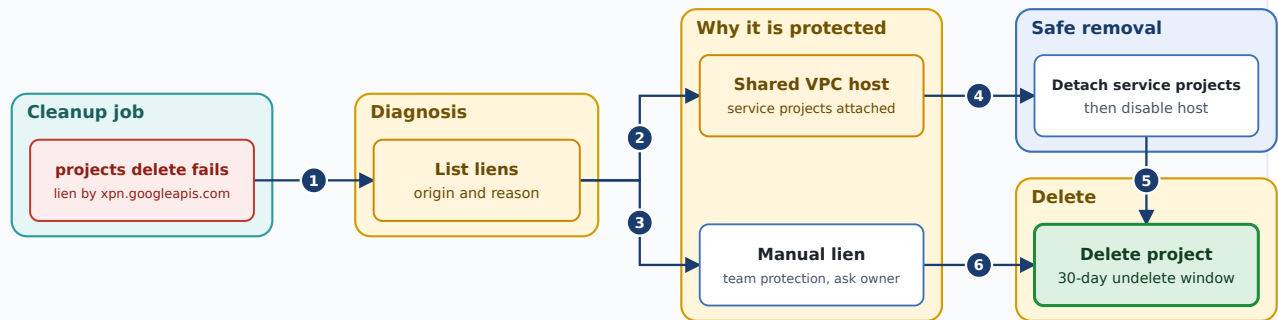
CONFIGURATION EXAMPLE

```

gcloud org-policies describe compute.vmExternalIpAccess --project=prod-web --effective
gcloud org-policies list --project=prod-web
# exception for one VM (policy admin only)
cat > ip.yaml <<'EOF'
name: projects/prod-web/policies/compute.vmExternalIpAccess
spec:
  rules:
    - values:
        allowedValues: ["projects/prod-web/zones/us-central1-a/instances/sftp-1"]
EOF
gcloud org-policies set-policy ip.yaml
  
```

Q23**Troubleshooting****A cleanup job fails to delete an old project with**

FAILED_PRECONDITION: A lien to prevent deletion was placed on the project by [xpn.googleapis.com] . What does this mean and how do you delete it safely?

FLOW DIAGRAM

- 1 The error names who placed the lien
- 2 xpn lien is added when the project is a host
- 3 Other origin: a person protected it on purpose
- 4 Check nobody still uses its subnets
- 5 Lien disappears when hosting is disabled
- 6 Remove lien only with owner approval

EXPLANATION

A lien blocks project deletion until it is removed, and the origin in the error tells you who placed it.

`xpn.googleapis.com` adds one automatically when a project becomes a Shared VPC host, so the project is still serving networks to service projects; deleting it would break them. Check the associated projects and whether any VMs or GKE clusters use its subnets, detach them, then disable Shared VPC hosting, which removes the lien. For liens placed by a person, confirm with the owner before `liens delete`, which needs `resourcemanager.projects.updateLiens`; after deletion the project can still be restored for about 30 days.

KEY POINTS

- Put your own liens on critical projects (host, logging, KMS) as a guard against accidents.
- Give `resourcemanager.projects.updateliens` to very few people.

CONFIGURATION EXAMPLE

```

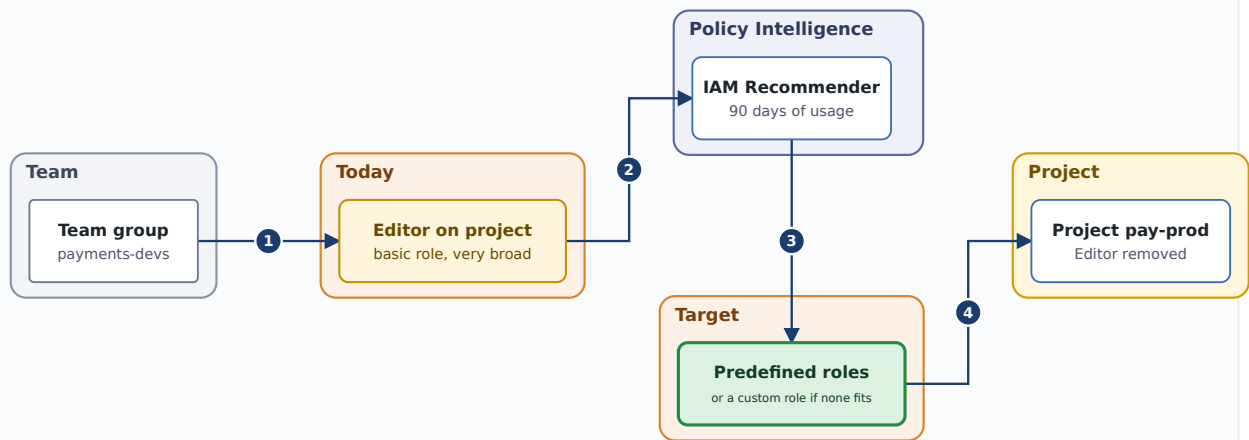
gcloud alpha resource-manager liens list --project=old-net-host
gcloud compute shared-vpc associated-projects list old-net-host
gcloud compute shared-vpc associated-projects remove old-svc-1 --host-project=old-net-host
gcloud compute shared-vpc disable old-net-host
gcloud projects delete old-net-host
  
```

2. IAM, Identity & Security

Who can do what, how workloads authenticate without keys, and the controls that stop data leaving: the security questions most GCP interviews spend the longest on.

Q24 An app team was given Editor on the prod project 'to get things working'. How do you move them to least privilege without breaking their work?

FLOW DIAGRAM



- 1 Group holds Editor on the whole project
- 2 Granted vs used permissions compared from logs
- 3 Suggests smaller roles, e.g. `run.developer`
- 4 Grant new roles to the group, then remove Editor

EXPLANATION

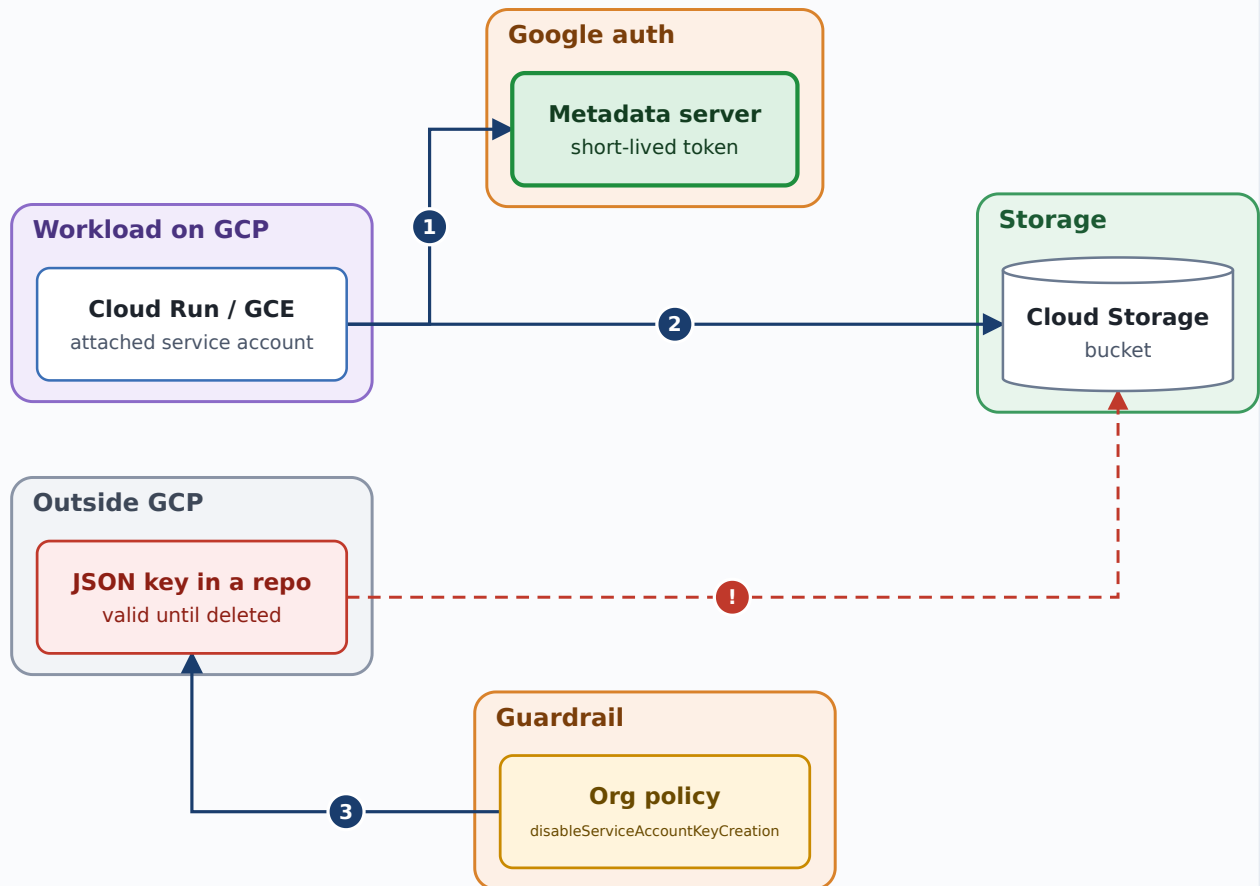
Basic roles (Owner, Editor, Viewer) predate IAM and cover most services in the project, so they break least privilege. Replace them with predefined roles per service (for example `roles/run.developer`, `roles/storage.objectViewer`), granted to groups at the lowest level that works. Use the IAM role recommender, which compares granted permissions with what was actually used over the last 90 days, to find the smallest set. Create a custom role only when no predefined role fits; custom roles do not pick up new permissions automatically, so someone must maintain them.

KEY POINTS

- Add the new roles first, watch for denials, then remove Editor.
- Keep Owner for a tiny admin group, never for daily work.

Q25 An audit found dozens of downloaded service account JSON keys in repos and on laptops. How do you get rid of keys and stop new ones?

FLOW DIAGRAM



- 1 Workload asks the metadata server for a token; no file on disk
- 2 Calls the API with a token that expires in about an hour
- ! Leaked key works from anywhere until someone deletes it
- 3 Constraint blocks creation of new user-managed keys

EXPLANATION

Workloads on Google Cloud should use an attached service account (on the VM, Cloud Run service or, for GKE, through Workload Identity Federation for GKE) and get short-lived tokens from the metadata server. Workloads outside Google Cloud should use Workload Identity Federation, and humans should use their own identity or service account impersonation. List existing keys with `gcloud iam service-accounts keys list --managed-by=user`, check key usage with Policy Intelligence (activity analysis), move each consumer to a keyless method, then disable and delete the key. Enforce `iam.disableServiceAccountKeyCreation` at the org and allow exceptions only on specific projects with a documented reason.

KEY POINTS

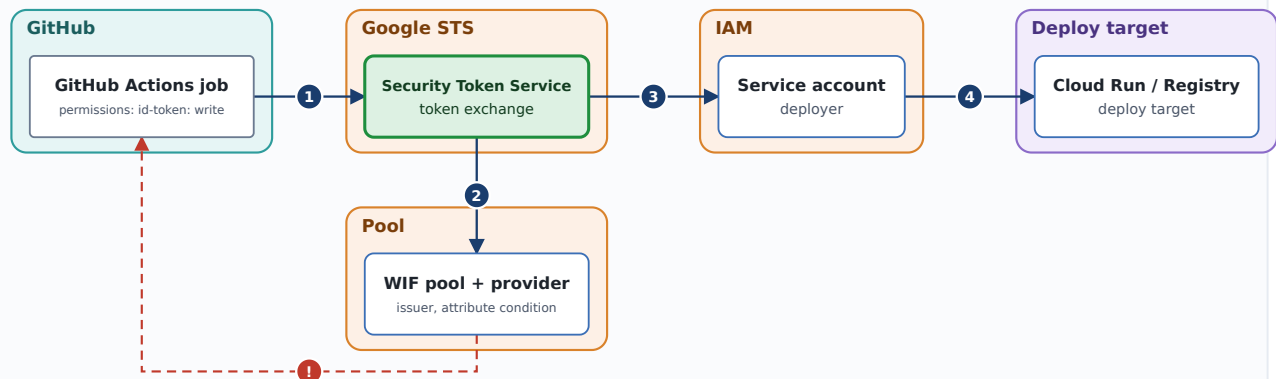
- Disable a key first and wait for errors before deleting it.
- Secret scanning in repos catches keys that already leaked.

CONFIGURATION EXAMPLE

```
gcloud iam service-accounts keys list \
  --iam-account=app@pay-prod.iam.gserviceaccount.com --managed-by=user
gcloud iam service-accounts keys disable KEY_ID \
  --iam-account=app@pay-prod.iam.gserviceaccount.com
gcloud resource-manager org-policies enable-enforce \
  iam.disableServiceAccountKeyCreation --organization=123456789012
```

Q26 GitHub Actions deploys to GCP using a service account key stored as a repo secret. How do you replace it with keyless authentication, and what changes for a workload on AWS?

FLOW DIAGRAM



- 1 Sends GitHub's OIDC token for this run
- 2 Checks issuer, audience and repo condition
- ! Rejected: token from a repo outside the allowed org
- 3 Federated token impersonates SA (workloadIdentityUser)
- 4 Short-lived access token used to deploy

EXPLANATION

Create a workload identity pool with an OIDC provider for `https://token.actions.githubusercontent.com`, map `google.subject` and attributes such as `attribute.repository`, and add an attribute condition like `assertion.repository_owner == 'my-org'` so other GitHub orgs cannot use it. Grant `roles/iam.workloadIdentityUser` on the deployer service account to a `principalSet` for the specific repository (or grant roles directly to the federated principal). In the workflow, request `id-token: write` and use `google-github-actions/auth`; the job gets tokens that last about an hour and there is no secret to leak. For AWS, add an AWS provider with the account ID; the workload signs a `GetCallerIdentity` request with its IAM role and exchanges it through the same STS flow using a credential configuration file.

KEY POINTS

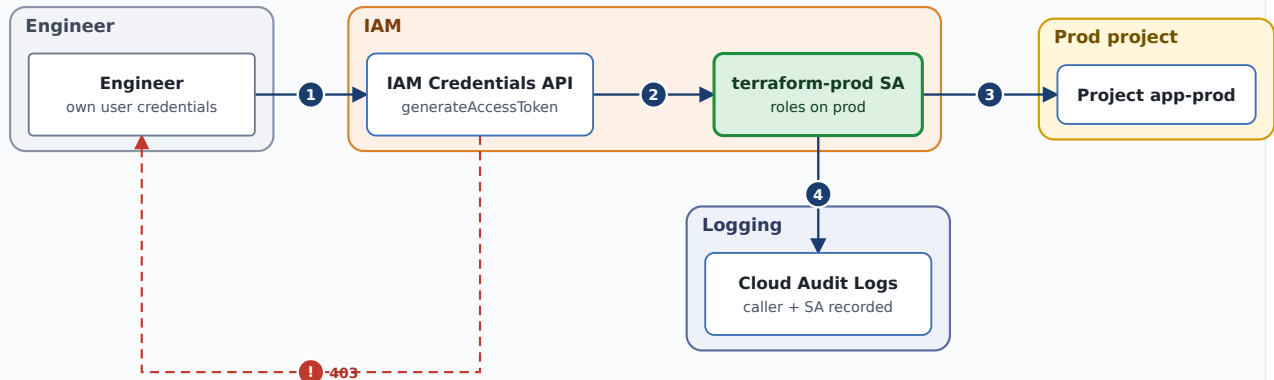
- Always set an attribute condition; without it any GitHub repo could try the provider.
- Scope the `principalSet` to a repo, and ideally a branch or environment.

CONFIGURATION EXAMPLE

```
permissions:
  id-token: write
  contents: read
steps:
  - uses: google-github-actions/auth@v2
    with:
      workload_identity_provider:
        projects/123456789/locations/global/workloadIdentityPools/github/providers/my-org
        service_account: deployer@pay-prod.iam.gserviceaccount.com
```


Q27 Platform engineers occasionally need to run Terraform against prod from their laptops, but security does not want them holding prod roles. How do you set this up?

FLOW DIAGRAM



- 1 Ask for a token for the Terraform service account
- 2 Allowed only if caller has Token Creator on this SA
- 3 Short-lived token used for all API calls
- 4 Logs show the SA and the human who impersonated it
- ! 403 caller lacks iam.serviceAccounts.getAccessToken

EXPLANATION

Give the prod roles only to a Terraform service account, and grant engineers `roles/iam.serviceAccountTokenCreator` on that one service account, not on the project. They then run `gcloud` with `--impersonate-service-account`, or set `impersonate_service_account` in the Terraform provider, and get tokens that last about an hour; no key is ever downloaded. Audit logs record both the service account and the original caller in the delegation info, so actions stay attributable. For tighter control make the Token Creator grant time-bound with an IAM condition or through Privileged Access Manager.

KEY POINTS

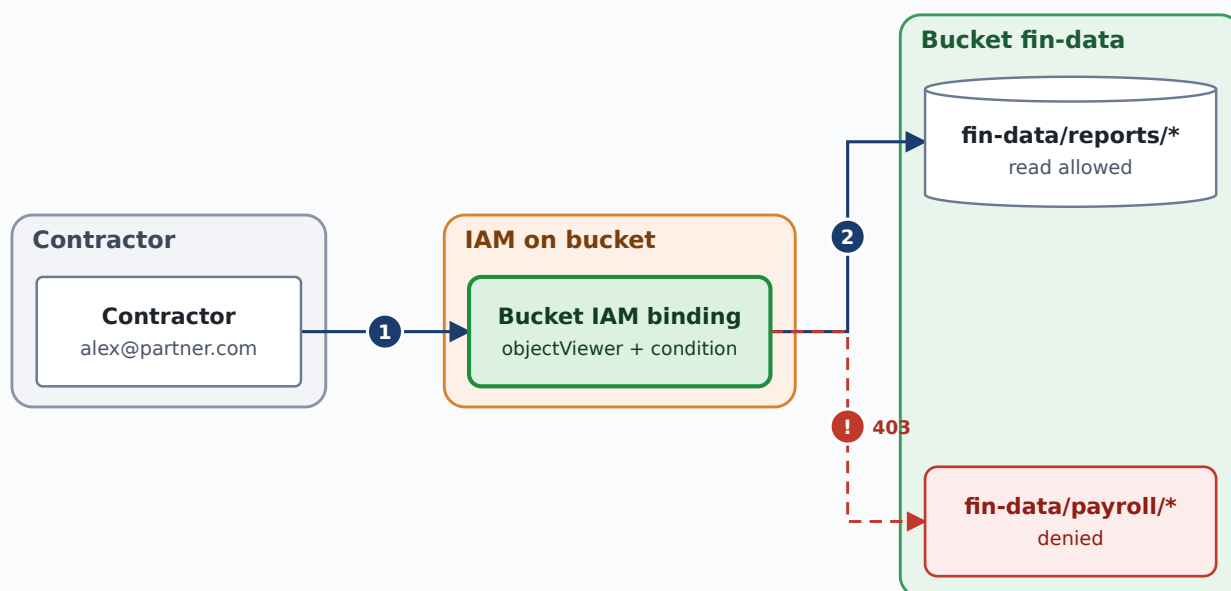
- Token Creator on a project lets you impersonate every SA in it; grant it per SA.
- The same pattern works for CI jobs that need to switch between environments.

CONFIGURATION EXAMPLE

```
gcloud iam service-accounts add-iam-policy-binding \
  terraform-prod@app-prod.iam.gserviceaccount.com \
  --member=group:platform-oncall@example.com \
  --role=roles/iam.serviceAccountTokenCreator
# provider "google" {
#   impersonate_service_account = "terraform-prod@app-prod.iam.gserviceaccount.com"
# }
```

Q28 A contractor needs read access only to objects under reports/ in one bucket, and only until the end of the quarter. How do you grant exactly that?

FLOW DIAGRAM



- 1 Read an object
- 2 Name starts with reports/ and date is before expiry
- ! 403 condition false for any other prefix
- After the timestamp the condition is always false, so access ends without anyone removing the binding.

EXPLANATION

Grant `roles/storage.objectViewer` on the bucket with an IAM Condition that combines a resource-name prefix and an expiry: `resource.name.startsWith('projects/_/buckets/fin-data/objects/reports/') && request.time < timestamp('2027-01-01T00:00:00Z')`. The bucket must use uniform bucket-level access for IAM conditions to apply. Listing is evaluated against the bucket, not objects, so a prefix condition also blocks `objects.list`; if the contractor must list, grant list separately or give them exact object paths. Conditions cannot be attached to basic roles, and not every resource type supports resource-name attributes, so check support per service.

KEY POINTS

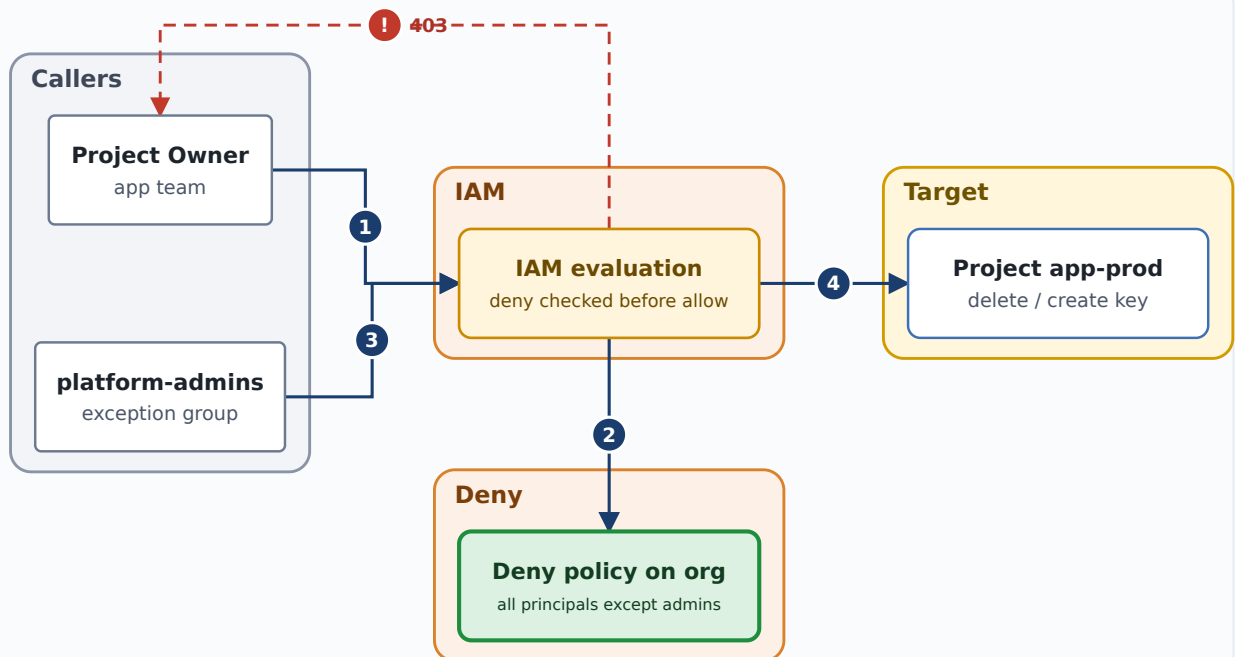
- Expiring conditions remove the risk of forgotten temporary access.
- Put a clear title on each condition; it shows up in reviews.

CONFIGURATION EXAMPLE

```
gcloud storage buckets add-iam-policy-binding gs://fin-data \
  --member=user:alex@partner.com --role=roles/storage.objectViewer \
  --condition='title=reports-until-q4,expression=resource.name.startsWith("projects/_/buckets/fin-data/objects/reports/") && request.time < timestamp("2027-01-01T00:00:00Z")'
```

Q29 Security wants to guarantee that nobody except the platform admins can delete projects or create service account keys, even users who are project Owner. How?

FLOW DIAGRAM



- ❶ projects.delete called by an Owner
- ❷ Deny rule matches this permission
- ❸ 403 denied by deny policy, whatever allow says
- ❹ Exception principal skips the deny rule
- ❺ Allow policy then decides as normal

EXPLANATION

Use an IAM deny policy attached at the organization or a folder. Deny rules are evaluated before allow policies, so a matching deny blocks the call even for Owner, and it is inherited by everything below. Set `deniedPrincipals` to all principals, `exceptionPrincipals` to the platform-admins group, and list permissions in the v2 format such as `resourcemanager.googleapis.com/projects.delete` and `iam.googleapis.com/serviceAccountKeys.create`. Not every permission is supported in deny policies, so check the supported list, and test on a folder first.

KEY POINTS

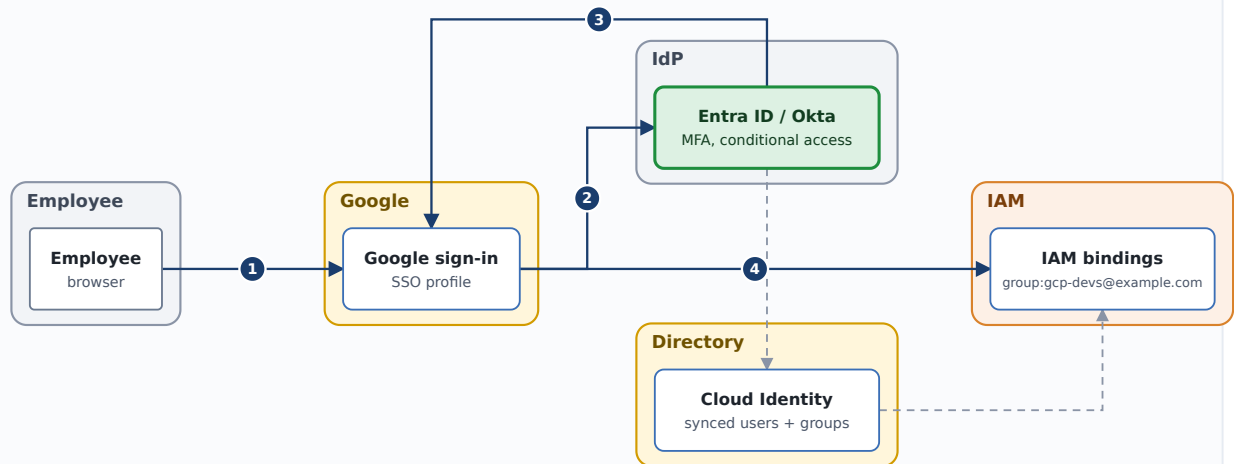
- Allow policies can only add access; deny policies are how you take it away below a grant.
- Managing deny policies needs the Deny Admin role, keep it tightly held.

CONFIGURATION EXAMPLE

```
gcloud iam policies create protect-core --kind=denypolicies --policy-file=deny.json \
  --attachment-point=cloudresourcemanager.googleapis.com/organizations/123456789012
# deny.json:
{"rules": [{"denyRule": {
  "deniedPrincipals": ["principalSet://goog/public:all"],
  "exceptionPrincipals": ["principalSet://goog/group/platform-admins@example.com"],
  "deniedPermissions": ["resourcemanager.googleapis.com/projects.delete",
    "iam.googleapis.com/serviceAccountKeys.create"]}}]}
```

Q30 The company uses Microsoft Entra ID (or Okta) for everything. How do employees sign in to Google Cloud, and how do you manage their access through groups?

FLOW DIAGRAM



- 1 Open the console, enter work email
- 2 Redirect to the IdP over SAML (or OIDC)
- 3 Signed assertion after MFA
- 4 Access decided by group memberships
- Provisioning connector (or GCDS) syncs users and groups
- IAM bindings reference the synced groups

EXPLANATION

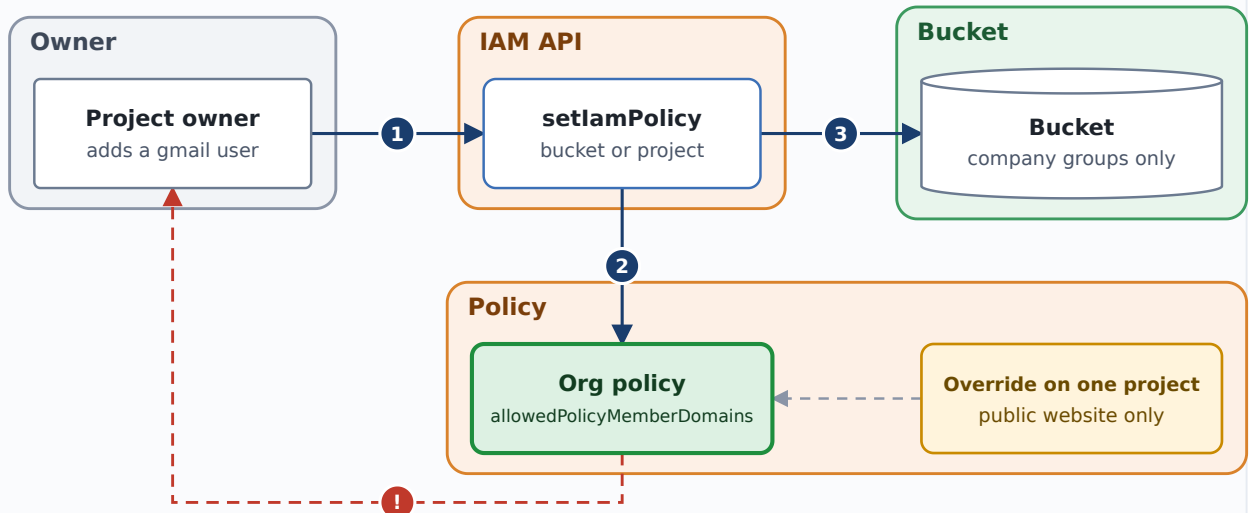
The usual pattern is Cloud Identity (the free edition is enough) as the Google-side directory, with SAML single sign-on to Entra ID or Okta so passwords and MFA stay in the corporate IdP. Users and groups are provisioned automatically by the IdP's Google Cloud provisioning connector, or by Google Cloud Directory Sync for on-prem Active Directory, and leavers disabled in the IdP lose access. Grant IAM roles only to groups such as `gcp-payments-devs@example.com` and manage membership in the IdP. Keep a couple of super admin accounts outside SSO for emergencies. An alternative is Workforce Identity Federation, which lets IdP users access Google Cloud without syncing accounts into Cloud Identity, using attribute-based principal sets.

KEY POINTS

- Groups in IAM, people in groups, groups owned by the IdP.
- Enforce MFA in the IdP, since Google defers authentication to it under SSO.

Q31 Someone shared a production bucket with a personal gmail.com account. How do you prevent sharing outside the company across the whole organization?

FLOW DIAGRAM



- 1 Try to add user:someone@gmail.com
- 2 Members checked against allowed customer IDs
- ! Rejected: member not in a permitted customer
- 3 Company users and groups are accepted
- Documented exception where allUsers is required

EXPLANATION

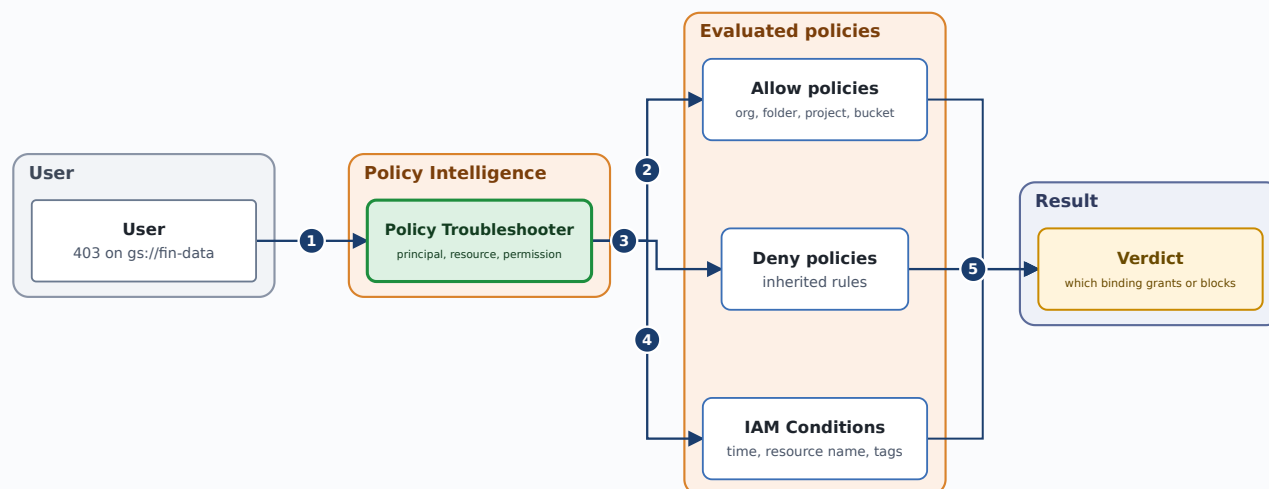
Set the organization policy `iam.allowedPolicyMemberDomains` (domain restricted sharing) with your Cloud Identity or Workspace customer ID, so IAM policies can only include members from that directory. It blocks new bindings but does not remove existing ones, so find those first with Cloud Asset Inventory. It also blocks `allUsers`, so public resources such as a static website need a scoped override on that project. Pair it with other baseline constraints: `iam.disableServiceAccountKeyCreation`, `storage.publicAccessPrevention`, `storage.uniformBucketLevelAccess`, `compute.vmExternalIpAccess` and `iam.automaticIamGrantsForDefaultServiceAccounts`.

KEY POINTS

- Constraints limit configuration; they complement IAM rather than replace it.
- Newer managed constraints (`iam.managed.*`) offer similar controls; check which your org uses.

Q32 A user gets 403 on a bucket even though their group seems to have the right role. How do you find out why?

FLOW DIAGRAM



- ❶ Enter who, which resource, which permission
 - ❷ Walks every allow binding, including group membership
 - ❸ Checks deny rules that match
 - ❹ Evaluates conditions on each binding
 - ❺ Shows the binding that decides the outcome
- If IAM says access is granted, look at the error for a VPC Service Controls violation or an org policy.

EXPLANATION

Run Policy Troubleshooter for the exact principal, full resource name and permission, which walks allow policies, deny policies and conditions and shows which binding grants or blocks access. Common causes are a deny policy, a condition that evaluates false (wrong prefix or expired), a role that does not contain the permission, the user not being in the group they think, or a recent change that has not propagated yet (usually a couple of minutes). If IAM says granted but calls still fail, check VPC Service Controls (the error includes a violation ID) and organization policies. Use Policy Analyzer (`gcloud asset analyze-iam-policy`) for the reverse question: who has this permission on this resource.

KEY POINTS

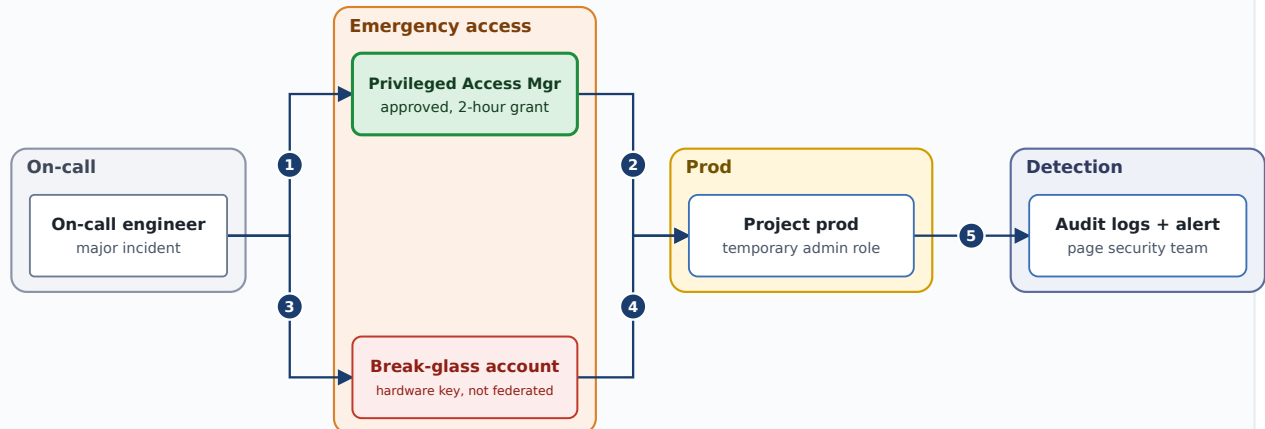
- Always reproduce with the exact permission, e.g. `storage.objects.get`, not the role.
- Troubleshooter needs permission to read the policies it evaluates.

CONFIGURATION EXAMPLE

```
gcloud policy-intelligence troubleshoot-policy iam \
  //storage.googleapis.com/projects/_/buckets/fin-data \
  --principal-email=alex@example.com --permission=storage.objects.get
gcloud asset analyze-iam-policy --organization=123456789012 \
  --full-resource-name="//storage.googleapis.com/projects/_/buckets/fin-data \
  --permissions=storage.objects.get
```

Q33 Nobody has standing admin rights in prod. How do you design emergency (break-glass) access for an outage, including the case where your IdP itself is down?

FLOW DIAGRAM



- ❶ Request entitlement with justification; peer approves
- ❷ Role granted, removed automatically at expiry
- ❸ Only if SSO or the normal path is broken
- ❹ Sealed credentials, used by two people
- ❺ Every action logged; log-based alert fires on use

EXPLANATION

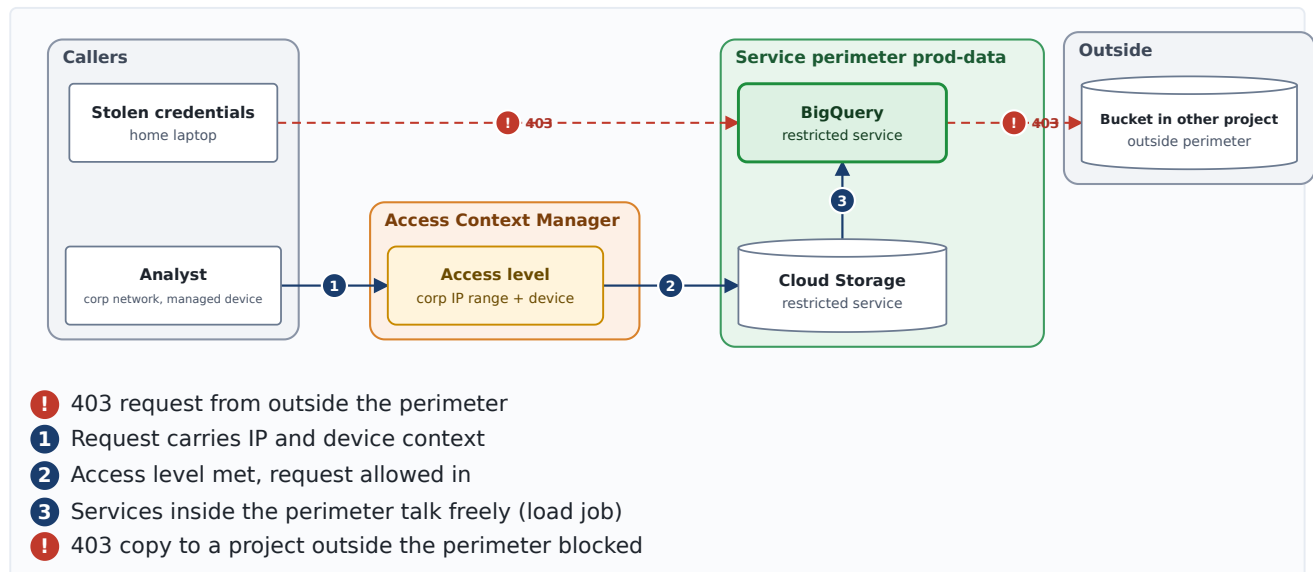
Use two layers. The normal emergency path is just-in-time access, for example Privileged Access Manager entitlements or a time-bound IAM condition, where an engineer requests a role with a justification, a peer approves, and the grant expires on its own. For the case where SSO or IAM automation is broken, keep one or two break-glass accounts in Cloud Identity that are excluded from SSO, protected with hardware security keys, with credentials stored sealed and split between people. Alert on any sign-in or activity from those accounts with log-based alerts, review each use afterwards, and test the procedure a couple of times a year.

KEY POINTS

- Break-glass that is never tested will fail during the real incident.
- Alerting on use matters as much as protecting the credentials.

Q34 Regulated projects hold BigQuery and Cloud Storage data. How do you stop data being copied out even by someone with valid credentials?

FLOW DIAGRAM



EXPLANATION

Put the data projects in a VPC Service Controls perimeter and restrict services such as `bigquery.googleapis.com` and `storage.googleapis.com`. API calls that cross the perimeter boundary are blocked even with valid IAM, so an attacker with stolen credentials outside the perimeter, or an insider copying to a personal project, is stopped. Allow legitimate access with access levels (corporate IPs, managed devices) and narrow ingress and egress rules for specific identities and projects. Roll out in dry-run mode first and read the violation logs, and route VMs to Google APIs through `restricted.googleapis.com` with Private Google Access.

KEY POINTS

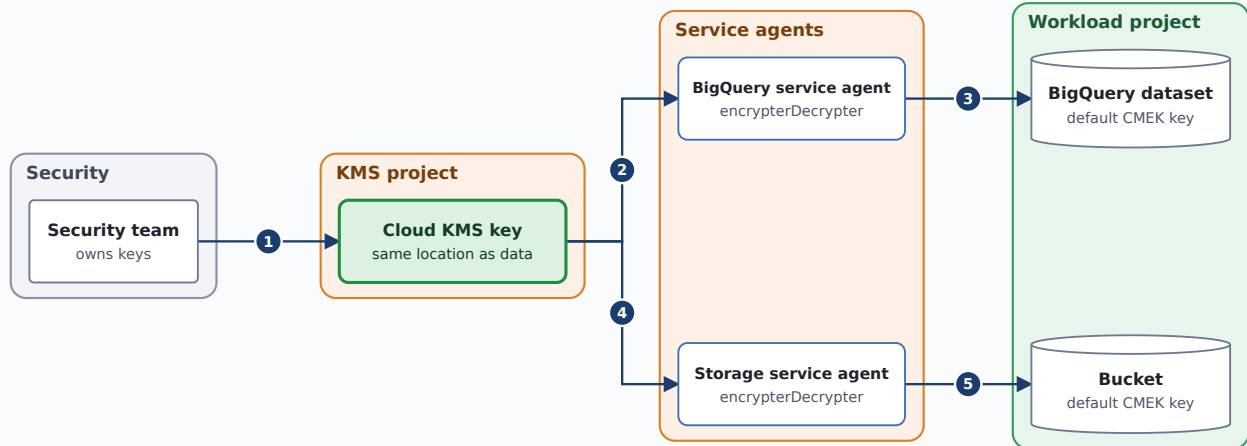
- VPC SC complements IAM: IAM decides who, the perimeter decides from where and to where.
- Dry-run for a few weeks; perimeters break pipelines you forgot about.

CONFIGURATION EXAMPLE

```
gcloud access-context-manager perimeters create prod_data \
  --policy=POLICY_ID --title="prod-data" \
  --resources=projects/11111111111,projects/22222222222 \
  --restricted-services=bigquery.googleapis.com,storage.googleapis.com \
  --access-levels=corp_network
```


Q35 Compliance requires that the company controls the encryption keys for BigQuery and Cloud Storage data, with yearly rotation. How do you implement CMEK and what are the risks?

FLOW DIAGRAM



- 1 Key ring in the data's location, rotation period 365d
- 2 Grant key use to each service's agent
- 3 New tables encrypted with the key
- 5 New objects encrypted with the key
- Rotation adds a new primary version; old versions stay to decrypt existing data.

EXPLANATION

By default Google encrypts all data with Google-managed keys; CMEK means the data is protected by a Cloud KMS key you control. Create the key ring in a separate KMS project, in the same location as the data (for example `europa-west1` or the `eu` multi-region), with a rotation period, and grant `roles/cloudkms.cryptoKeyEncrypterDecrypter` to each service agent that uses it. Set the key as the default on datasets and buckets, or use Cloud KMS Autokey to create keys per resource. Rotation creates a new primary version for new writes but does not re-encrypt existing data. The main risk is availability: disabling or destroying a key version makes the data it protects unreadable, and destruction happens after a scheduled waiting period, so restrict `cloudkms.cryptoKeyVersions.destroy`.

KEY POINTS

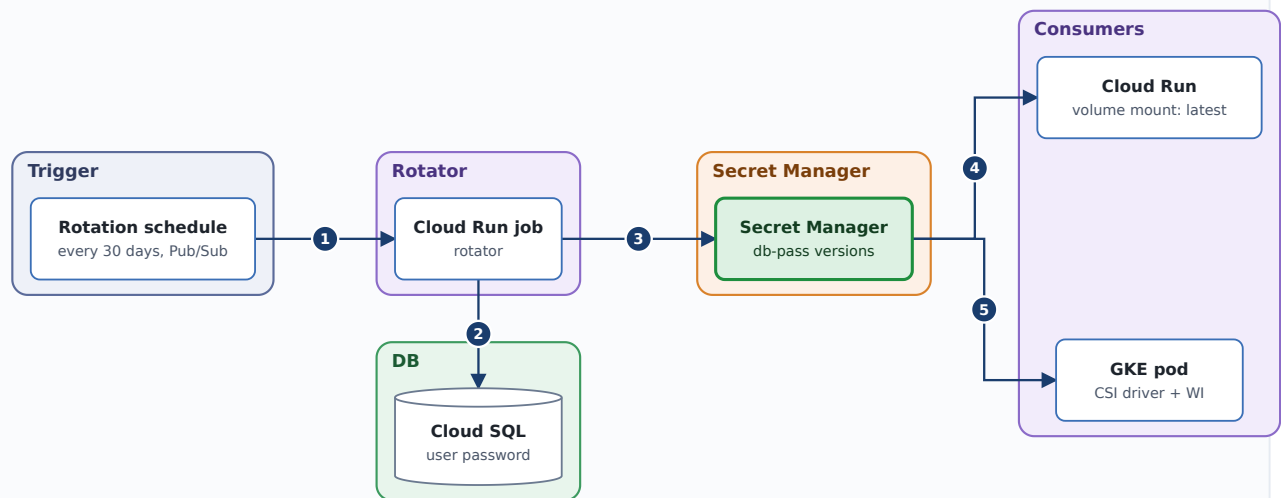
- Keep KMS admins separate from people who can use keys.
- Key location must match the resource location or creation fails.

CONFIGURATION EXAMPLE

```
gcloud kms keyrings create data-eu --location=europe-west1 --project=kms-prod
gcloud kms keys create data-key --keyring=data-eu --location=europe-west1 \
  --project=kms-prod --purpose=encryption \
  --rotation-period=365d --next-rotation-time=2026-10-01T00:00:00Z
gcloud kms keys add-iam-policy-binding data-key --keyring=data-eu \
  --location=europe-west1 --project=kms-prod \
  --member=serviceAccount:service-123456789@gs-project-accounts.iam.gserviceaccount.com \
  --role=roles/cloudkms.cryptoKeyEncrypterDecrypter
```

Q36 Services on Cloud Run and GKE need a database password that is rotated every 30 days without redeploying. How do you use Secret Manager for this?

FLOW DIAGRAM



- 1 Rotation message on the secret's topic
- 2 Set a new password on the DB user
- 3 Add new version; keep the previous one enabled for a while
- 4 Mounted file reads get the new version
- 5 CSI driver or add-on syncs the new version

EXPLANATION

Store the password as a Secret Manager secret; each change is a new immutable version and `latest` points at the newest. Configure a rotation schedule with a Pub/Sub topic: Secret Manager only sends the notification, so a rotator job must change the database password and add the new version. On Cloud Run, mount the secret as a volume with `latest` so new instances and file reads see new versions (environment-variable secrets are read only at instance start); on GKE use the Secret Manager add-on or CSI driver with Workload Identity Federation for GKE. Keep the old version enabled until every client has switched, then disable it. Grant `roles/secretmanager.secretAccessor` per secret to each workload's service account, not project-wide.

KEY POINTS

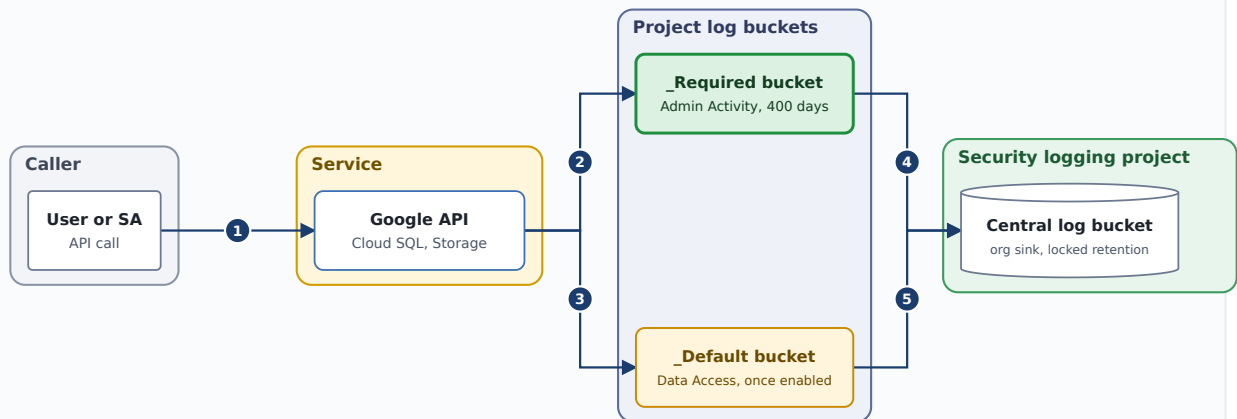
- Pin a version number if you need controlled rollouts; use `latest` for automatic rotation.
- Access to secret values is logged in Data Access audit logs when enabled.

CONFIGURATION EXAMPLE

```
gcloud secrets create db-pass --replication-policy=automatic \
  --topics=projects/app-prod/topics/rotate-db-pass \
  --rotation-period=2592000s --next-rotation-time=2026-10-01T00:00:00Z
gcloud run services update api --region=europe-west1 \
  --update-secrets=/secrets/db-pass=db-pass:latest
```

Q37 Someone deleted a Cloud SQL instance, and separately you cannot tell who downloaded objects from a sensitive bucket. How do audit logs work and what do you change?

FLOW DIAGRAM



- 1 Delete instance / read object
- 2 Config changes always logged, cannot be disabled
- 3 Reads are logged only if Data Access logs are on
- 4 Aggregated org sink copies all audit logs

EXPLANATION

Admin Activity audit logs (config changes like `cloudsql.instances.delete`) are always on, free, and kept 400 days in the `_Required` bucket, so the deletion is found with a Logs Explorer query on `protoPayload.methodName`. Data Access logs (reads and data writes) are off by default for most services, with BigQuery a notable exception, so enable them in the IAM audit config for Cloud Storage on sensitive projects; they can be high volume and are billed. System Event and Policy Denied logs are also recorded. Send all audit logs to a central logging project with an aggregated org sink and a locked retention bucket, so project owners cannot delete the evidence.

KEY POINTS

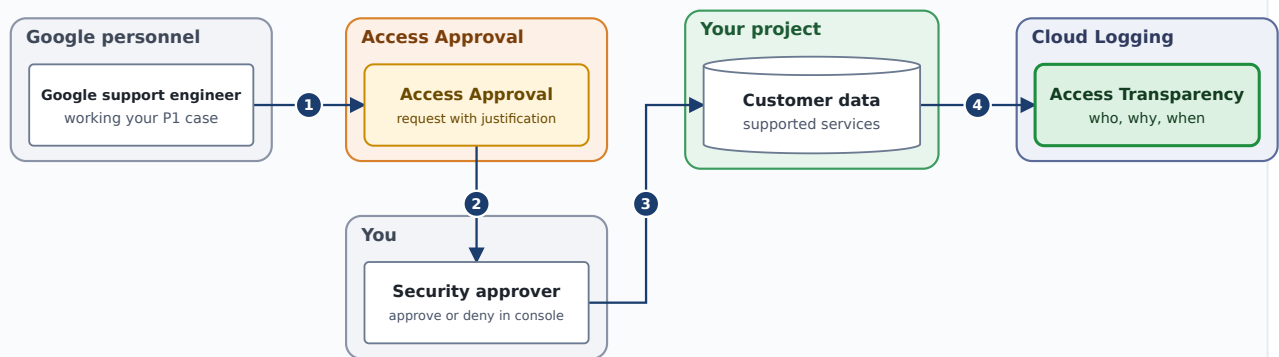
- Enable Data Access logs selectively; turning them on everywhere is expensive.
- `_Default` retention is 30 days unless you change it.

CONFIGURATION EXAMPLE

```
gcloud logging sinks create org-audit \
  logging.googleapis.com/projects/sec-logs/locations/europe-west1/buckets/org-audit \
  --organization=123456789012 --include-children \
  --log-filter='logName:"cloudaudit.googleapis.com"'
gcloud logging read 'protoPayload.methodName="cloudsql.instances.delete"' \
  --project=app-prod --freshness=30d
```

Q38 A regulator asks how you know when Google staff access your data, and whether you can require approval first. What do you tell them?

FLOW DIAGRAM



- 1 Access needed, with reason and case reference
- 2 Notification by email or Pub/Sub
- 3 Approval allows time-limited access
- 4 Access written to Access Transparency logs

EXPLANATION

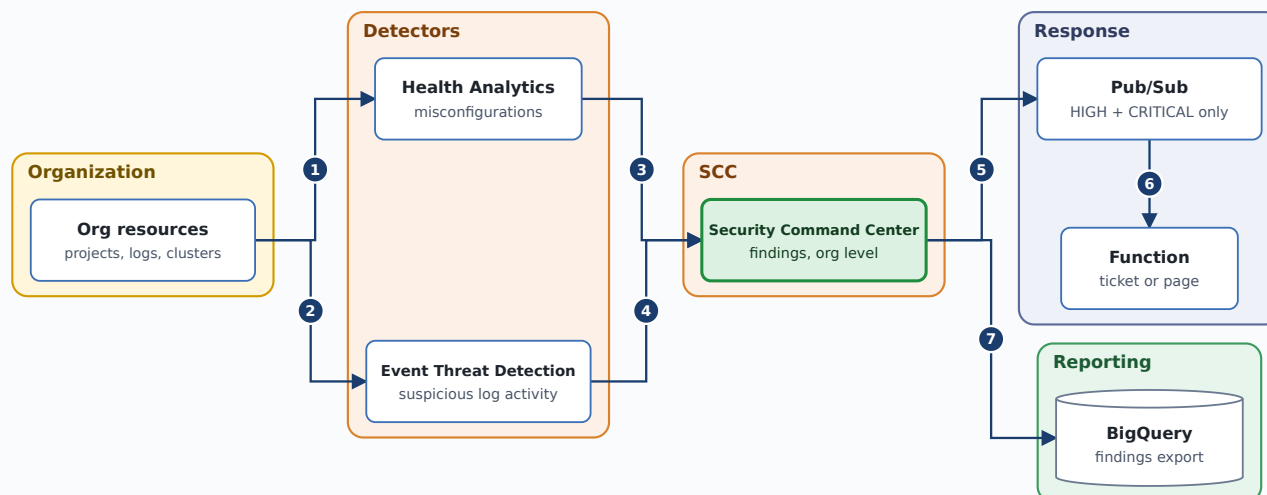
Access Transparency writes near real-time logs when Google personnel access customer content, including the justification (for example a support case you opened), the resource and the time. Access Approval goes further for supported services: Google must ask and wait for your explicit approval before accessing content, with some documented exceptions such as legal obligations. Both are enabled at the organization level; Access Transparency needs an eligible support or product tier, so check current requirements. Export these logs to your SIEM with the other audit logs, and for stronger key control combine with Cloud EKM and Key Access Justifications.

KEY POINTS

- Most Google access is tied to a support case you opened.
- These are evidence controls for auditors, not replacements for your own IAM.

Q39 The CISO wants one place to see misconfigurations and threats across the whole organization, with a ticket raised for every high-severity finding. How do you set it up?

FLOW DIAGRAM



- 1 Scans asset configurations
- 2 Analyses audit, DNS and flow logs
- 4 Findings from all detectors in one place
- 5 Notification config filters by severity
- 6 Opens a ticket or pages the owner
- 7 Continuous export for trends and SLAs

EXPLANATION

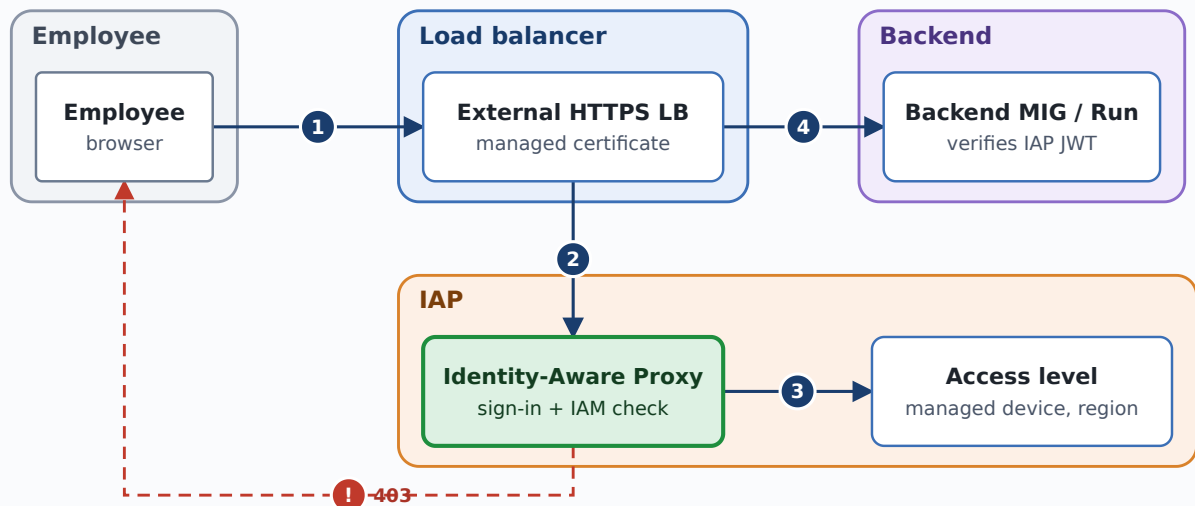
Activate Security Command Center at the organization level so every project is covered. Security Health Analytics reports misconfigurations such as public buckets, open firewall rules or keys that are not rotated; in the Premium or Enterprise tiers, Event Threat Detection, Container Threat Detection and VM Threat Detection add threat findings from logs and runtime signals. Create a notification config that publishes HIGH and CRITICAL findings to Pub/Sub, where a function opens tickets or pages the owning team, and export findings to BigQuery for trend reporting. Use mute rules for accepted risks so the queue stays meaningful.

KEY POINTS

- Route findings to the owning team using project labels, not to one central inbox.
- Fix root causes with org policies so the same finding stops appearing.

Q40 An internal admin web app must be reachable only by employees in the ops group, from anywhere, without a VPN. How do you protect it with IAP?

FLOW DIAGRAM



- 1 Request admin.example.com
- 2 IAP enabled on the backend service
- 3 Optional context check
- ! 403 not in the allowed group
- 4 Only allowed requests reach the app, with a signed header

EXPLANATION

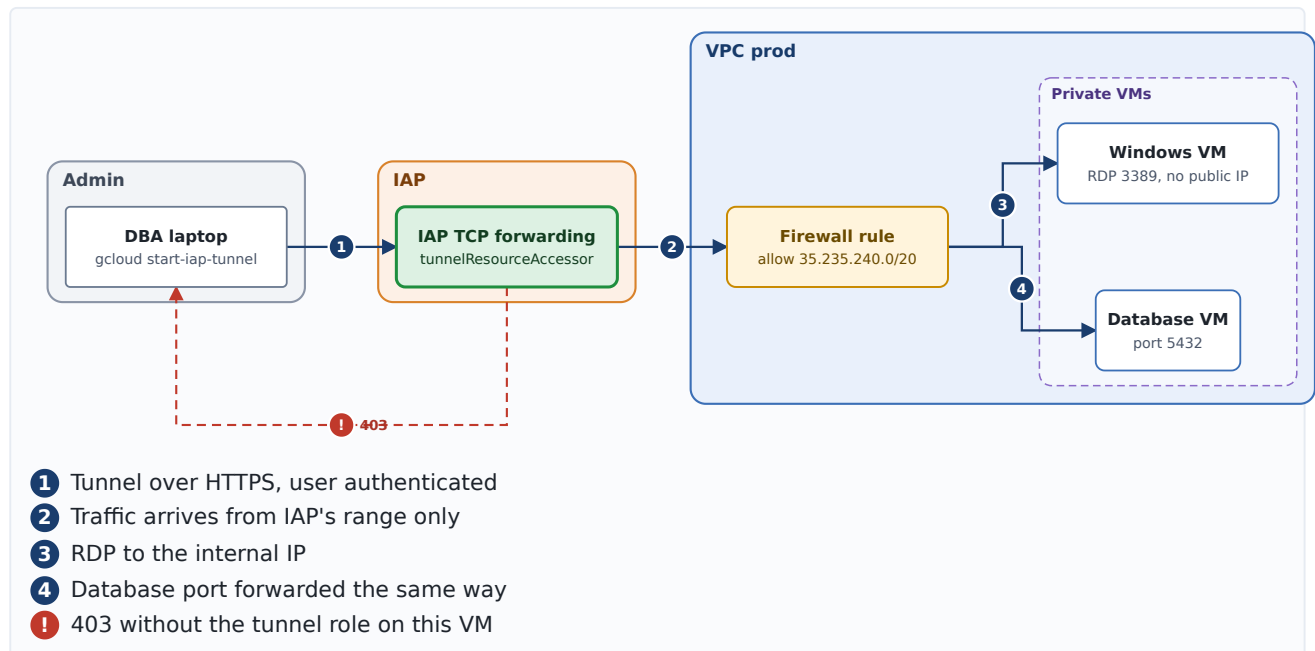
Put the app behind an external Application Load Balancer and enable IAP on its backend service (Cloud Run can also use IAP directly). IAP signs users in with their Google identity (including SSO users) and checks `roles/iap.httpsResourceAccessor` on the resource, which you grant to the ops group; access levels can add device or location requirements. The backend should allow traffic only from the load balancer ranges (`130.211.0.0/22` , `35.191.0.0/16`) and verify the `x-goog-iap-jwt-assertion` header so nobody can bypass IAP. This replaces VPN access for web apps and gives per-request audit logs.

KEY POINTS

- Validate the IAP JWT in the app; firewall rules alone can be misconfigured.
- Grant the accessor role on the backend service, not the whole project, when apps differ.

Q41 DBAs need RDP to a Windows VM and access to a self-managed database port, but VMs have no public IPs and you do not want a bastion host. How?

FLOW DIAGRAM



EXPLANATION

Use IAP TCP forwarding. Create a firewall rule allowing ingress from `35.235.240.0/20` (IAP's range) only on the ports needed, grant `roles/iap.tunnelResourceAccessor` to the DBA group (optionally per instance or with conditions), and have users run `gcloud compute start-iap-tunnel` to forward a local port to the VM's internal IP. IAP authenticates each connection with the user's Google identity and logs it, so there is no public IP, no bastion to patch and no VPN. For Linux SSH, `gcloud compute ssh --tunnel-through-iap` does the same, combined with OS Login.

KEY POINTS

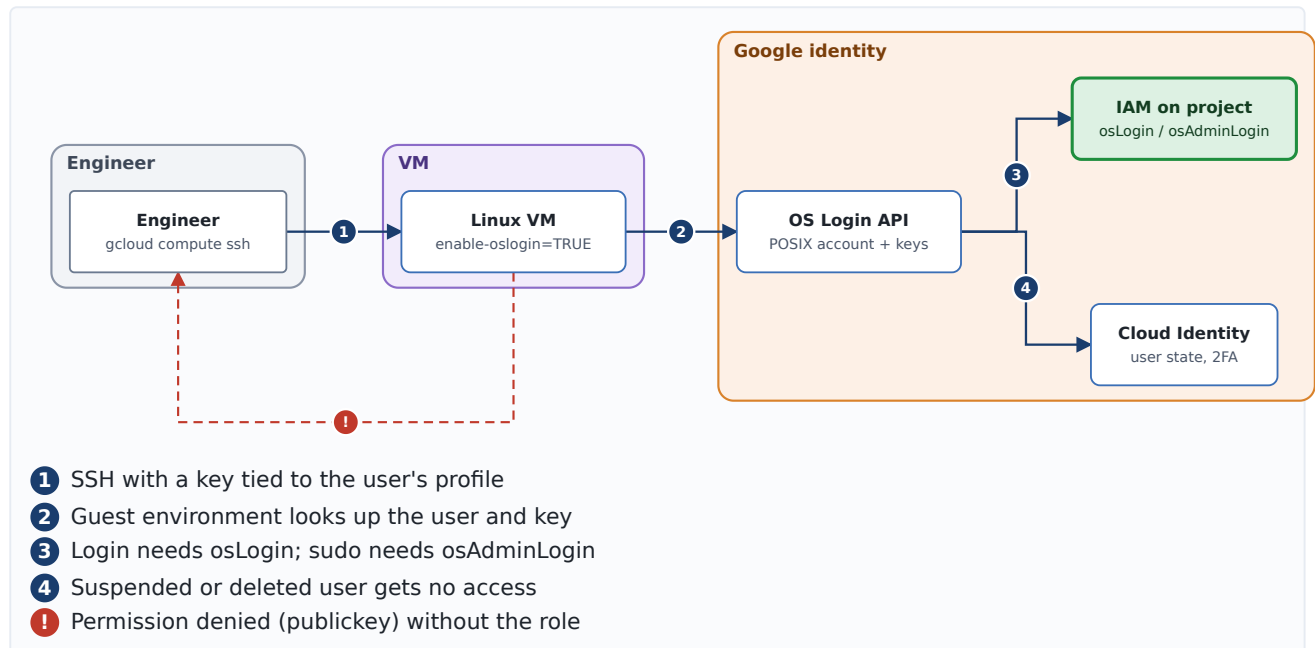
- Allow only the ports each group needs from the IAP range.
- IAP tunnels suit admin access; they are not for bulk data transfer.

CONFIGURATION EXAMPLE

```
gcloud compute firewall-rules create allow-iap-admin --network=prod-vpc \
  --direction=INGRESS --allow=tcp:3389,tcp:5432 \
  --source-ranges=35.235.240.0/20 --target-tags=admin-access
gcloud compute start-iap-tunnel win-01 3389 \
  --local-host-port=localhost:3389 --zone=europe-west1-b
```

Q42 You manage SSH access for 200 engineers across many Linux VMs. Leavers must lose access immediately and only SREs may use sudo. How does OS Login help?

FLOW DIAGRAM



EXPLANATION

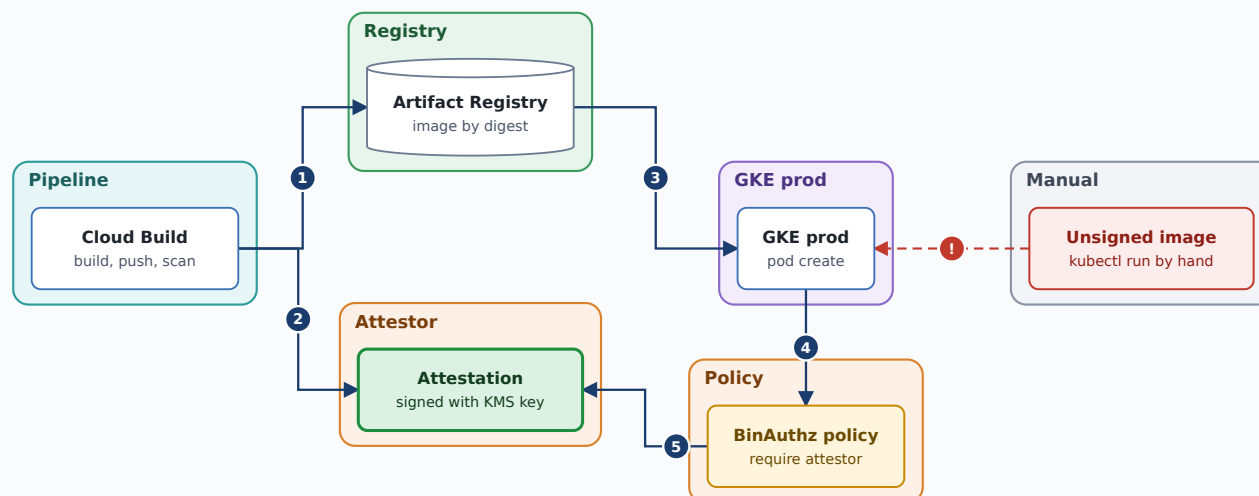
Enable OS Login with the metadata key `enable-oslogin=TRUE` at project level, or enforce it with the org policy `compute.requireOsLogin`, so Linux user accounts and SSH keys come from Google identities instead of keys stored in instance metadata. Grant `roles/compute.osLogin` to engineers and `roles/compute.osAdminLogin` (which adds sudo) only to SREs, ideally through groups. Access follows the identity, so suspending a user in the IdP or removing them from the group cuts SSH access without touching VMs. Add `enable-oslogin-2fa=TRUE` for a second factor, and use `roles/compute.osLoginExternalUser` for users from outside the organization.

KEY POINTS

- Block project-wide metadata SSH keys so nobody bypasses OS Login.
- Combine with IAP tunnelling so VMs need no public IPs.

Q43 Only images built and scanned by your pipeline may run on the production GKE cluster. How does Binary Authorization enforce that?

FLOW DIAGRAM



- ❶ Push image; vulnerability scan runs
 - ❷ Sign the digest if the scan passes
 - ❸ Deployment references the image digest
 - ❹ Admission check on every pod
 - ❺ Look for a valid attestation for this digest
- ❗ Denied at admission: no attestation

EXPLANATION

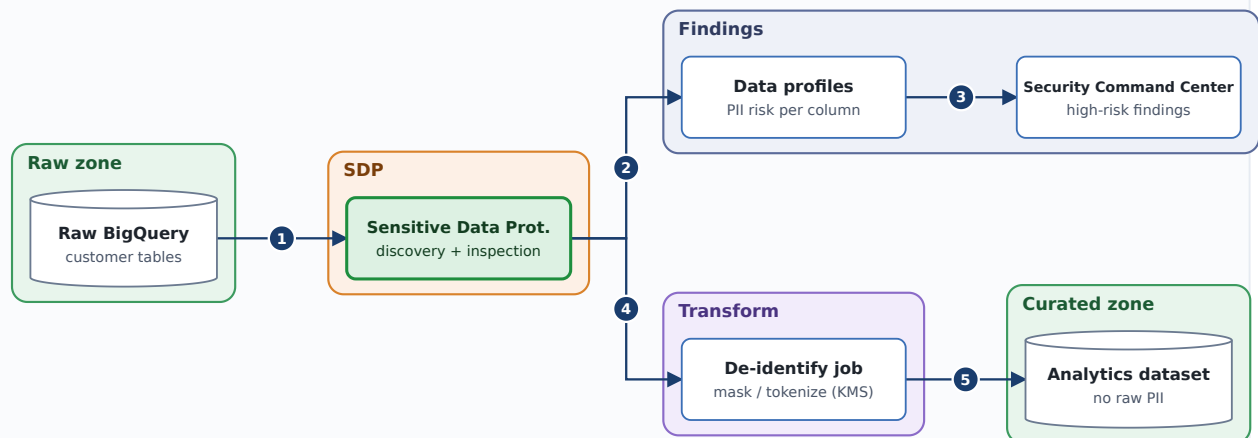
Binary Authorization is an admission control for GKE and Cloud Run that checks a policy before a container starts. The pipeline creates an attestation, a signature over the image digest made with a Cloud KMS key held by an attester, only after tests and vulnerability scanning pass. The policy for the prod cluster requires that attester, so a hand-built or tag-based image is rejected. Run the policy in dry-run mode first, deploy by digest, keep a documented break-glass annotation for emergencies (it is logged), and exempt system images you trust.

KEY POINTS

- Attest the digest, never a mutable tag.
- Different clusters can require different attestors (e.g. QA-passed for prod).

Q44 A data lake in BigQuery and Cloud Storage may contain customer PII. How do you find it and share only de-identified data with the analytics team?

FLOW DIAGRAM



- ❶ Discovery scans tables and buckets on a schedule
- ❷ infoTypes found, e.g. EMAIL_ADDRESS, PHONE_NUMBER
- ❸ High-sensitivity data raised as findings
- ❹ Transform sensitive columns
- ❺ Analysts see only the de-identified copy

EXPLANATION

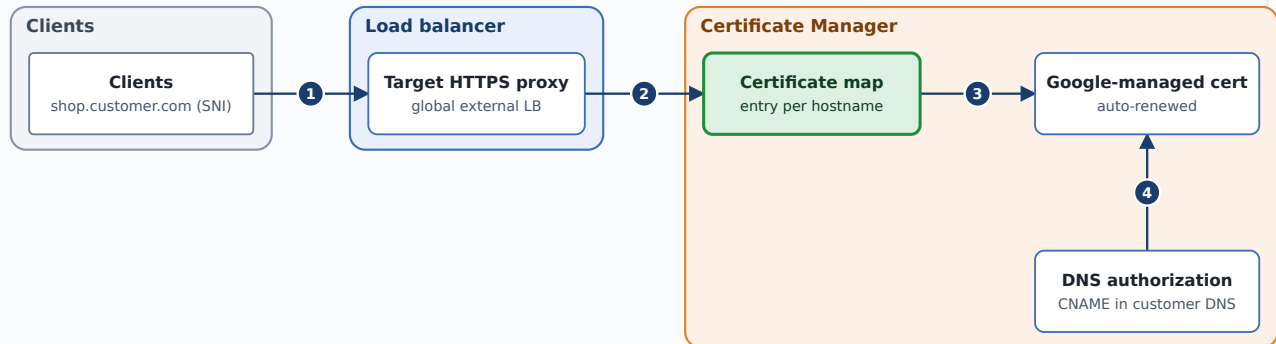
Use Sensitive Data Protection (formerly Cloud DLP). Discovery creates data profiles for BigQuery, Cloud SQL and Cloud Storage across projects, showing which columns or files contain infoTypes such as email, phone or national IDs and how sensitive they are, and can send results to Security Command Center. For sharing, run de-identification: masking or redaction for fields analysts do not need, and deterministic tokenization or format-preserving encryption with a KMS-wrapped key where joins across tables are needed. Write the result to a curated dataset the analytics team can read, and keep the raw zone restricted. BigQuery column-level security with policy tags is an alternative when data should stay in one table.

KEY POINTS

- Deterministic tokens keep joins working without revealing the value.
- Profile first, then decide what to transform; do not guess columns.

Q45 Your SaaS platform serves hundreds of customer domains on one global load balancer, each needing a managed TLS certificate. How do you handle this with Certificate Manager?

FLOW DIAGRAM



- 1 TLS handshake with the hostname in SNI
- 2 Proxy uses the attached certificate map
- 3 Picks the certificate for that hostname
- 4 Domain validated by DNS, even before traffic moves

EXPLANATION

Certificate Manager lets a target HTTPS proxy use a certificate map instead of a short fixed list of certificates, so one load balancer can serve very large numbers of hostnames. For each customer domain, create a DNS authorization (the customer adds a CNAME), a Google-managed certificate using it, and a map entry for the hostname. DNS authorization lets the certificate be issued before traffic points at your load balancer and supports wildcards, so cut-overs have no TLS gap. Google renews managed certificates automatically; watch certificate state in monitoring for customers whose DNS record was removed.

KEY POINTS

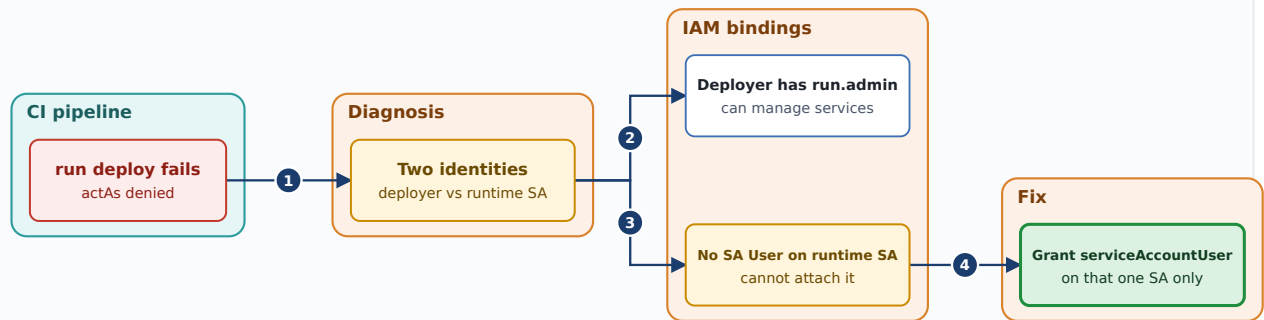
- Load balancer authorization only works once traffic already reaches you; DNS authorization avoids that.
- Classic per-proxy SSL certificate lists have a small limit; maps do not have that problem.

CONFIGURATION EXAMPLE

```
gcloud certificate-manager dns-authorizations create shop-auth --domain=shop.customer.com
gcloud certificate-manager certificates create shop-cert \
  --domains=shop.customer.com --dns-authorizations=shop-auth
gcloud certificate-manager maps create saas-map
gcloud certificate-manager maps entries create shop --map=saas-map \
  --hostname=shop.customer.com --certificates=shop-cert
gcloud compute target-https-proxies update saas-proxy --certificate-map=saas-map
```

Q46**Troubleshooting**

`gcloud run deploy` from the CI service account fails with Permission `'iam.serviceAccounts.actAs'` denied on service account `api-runtime@prod.iam.gserviceaccount.com`, although CI has Cloud Run Admin. How do you fix it?

FLOW DIAGRAM

- ❶ Error names the runtime SA, not the caller
- ❷ Admin on Cloud Run is present and not the issue
- ❸ Attaching an SA to a resource needs `actAs` on it
- ❹ Bind at the SA, not the project

EXPLANATION

Deploying a service that runs as a service account is a way to use that account's permissions, so IAM also requires `iam.serviceAccounts.actAs` on the runtime service account. `roles/run.admin` does not include it; `roles/iam.serviceAccountUser` does. Grant that role to the CI identity on the runtime service account resource itself, not on the whole project, otherwise CI could act as every service account in it. The same error appears for Compute Engine, Cloud Functions, Dataflow and GKE node pools when they are created with a service account.

KEY POINTS

- Project-level Service Account User lets the holder act as every SA in the project; avoid it.
- Use a dedicated runtime SA per service so each grant stays narrow.

CONFIGURATION EXAMPLE

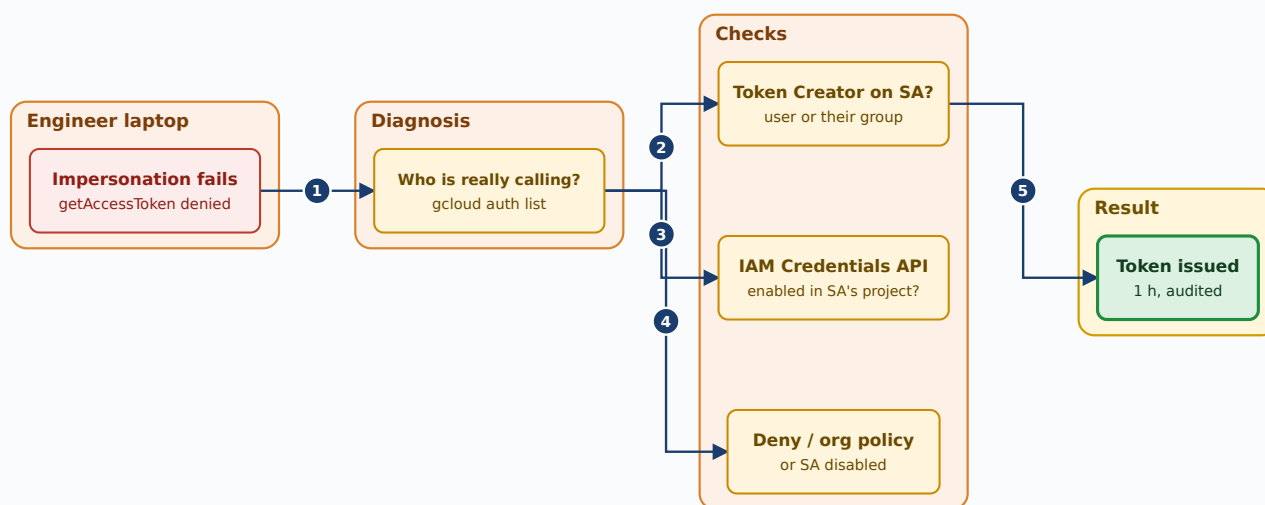
```

gcloud iam service-accounts add-iam-policy-binding \
  api-runtime@prod.iam.gserviceaccount.com \
  --member=serviceAccount:ci-deployer@ci.cd.iam.gserviceaccount.com \
  --role=roles/iam.serviceAccountUser
gcloud iam service-accounts get-iam-policy api-runtime@prod.iam.gserviceaccount.com

```

Q47**Troubleshooting**

An engineer runs `gcloud storage ls --impersonate-service-account=tf-prod@prod.iam.gserviceaccount.com` and gets `Permission 'iam.serviceAccounts.getAccessToken' denied`. How do you troubleshoot impersonation?

FLOW DIAGRAM

- ❶ Wrong account or a stale ADC login is common
- ❷ Needs roles/iam.serviceAccountTokenCreator
- ❸ iamcredentials.googleapis.com must be on
- ❹ Deny policies and a disabled SA also block it
- ❺ Retry after fixing whichever check failed

EXPLANATION

Impersonation calls the IAM Credentials API to mint a short-lived token, and the caller needs `roles/iam.serviceAccountTokenCreator` on the target service account (Owner or Editor on the project is not enough). First confirm the calling identity with `gcloud auth list`: engineers often run as a personal account while the grant is on a group they joined minutes ago, or as a different account in ADC. Then check the binding on the service account, that `iamcredentials.googleapis.com` is enabled in the service account's project, and that the service account is not disabled. Group membership changes and new bindings can take a few minutes to apply.

KEY POINTS

- Grant Token Creator to a group through a PAM entitlement instead of standing access.
- Audit logs in the SA's project record every `GenerateAccessToken` call.

CONFIGURATION EXAMPLE

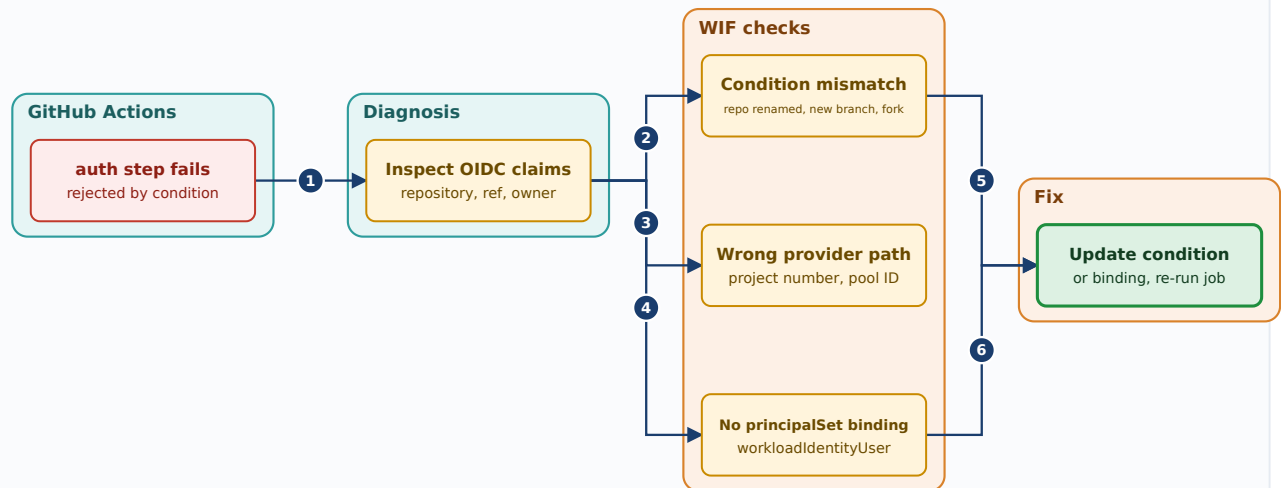
```

gcloud auth list
gcloud iam service-accounts get-iam-policy tf-prod@prod.iam.gserviceaccount.com
gcloud services list --enabled --project=prod | grep iamcredentials
gcloud iam service-accounts describe tf-prod@prod.iam.gserviceaccount.com --format='value(disabled)'
gcloud iam service-accounts add-iam-policy-binding tf-prod@prod.iam.gserviceaccount.com \
  --member=group:platform-eng@example.com --role=roles/iam.serviceAccountTokenCreator

```

Q48**Troubleshooting**

A GitHub Actions job using Workload Identity Federation fails in the auth step with `unauthorized_client: The given credential is rejected by the attribute condition`. It worked last week. How do you fix it?

FLOW DIAGRAM

- ❶ Compare token claims with the provider rules
- ❷ Condition is a CEL check on mapped attributes
- ❸ Provider must use the project NUMBER
- ❹ Exchange works but `getAccessToken` is denied

EXPLANATION

The STS exchange checks the GitHub OIDC token against the provider's attribute mapping and attribute condition; this error means the condition evaluated to false. Typical causes are a renamed repository or organization, a workflow now running from a new branch, tag, environment or fork that the condition does not allow, or someone tightening the condition. Describe the provider, compare its condition with the token's `repository`, `ref` and `repository_owner` claims, and fix whichever is wrong. If the exchange succeeds but you then get `getAccessToken` denied, the principal set lacks `roles/iam.workloadIdentityUser` on the service account.

KEY POINTS

- Match on `repository_id` and `repository_owner_id`; IDs do not change when names change.
- Keep the provider and bindings in Terraform so changes are reviewed.

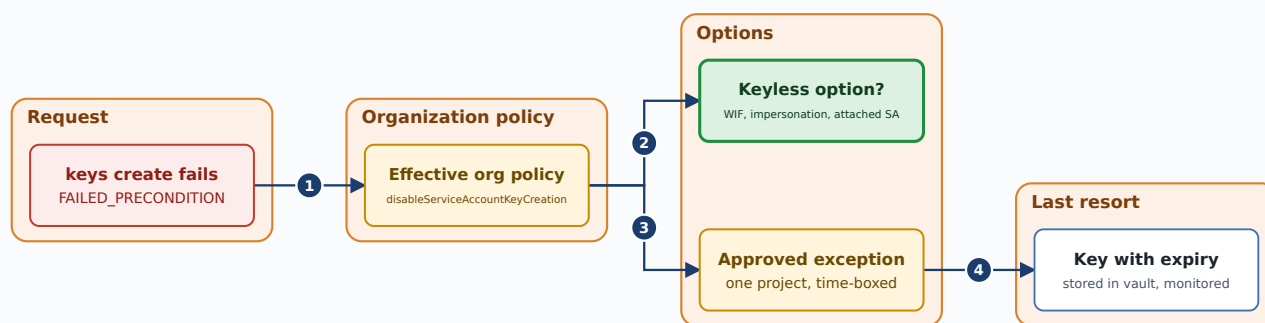
CONFIGURATION EXAMPLE

```

gcloud iam workload-identity-pools providers describe my-repo \
  --workload-identity-pool=github --location=global --project=cicd \
  --format='yaml(attributeCondition,attributeMapping)'
gcloud iam workload-identity-pools providers update-oidc my-repo \
  --workload-identity-pool=github --location=global --project=cicd \
  --attribute-condition="assertion.repository_owner_id=='81234567' && assertion.ref=='refs/heads/main'"
  
```

Q49**Troubleshooting**

A vendor integration needs a service account key, but `gcloud iam service-accounts keys create` fails with `Key creation is not allowed on this service account (FAILED_PRECONDITION)`. How do you handle it?

FLOW DIAGRAM

- 1 Boolean constraint enforced at org or folder
- 2 Most vendors (AWS, Azure, OIDC) support WIF
- 3 Only if no keyless method exists
- 4 Override on one project, create key, re-enforce

EXPLANATION

The org policy `iam.disableServiceAccountKeyCreation` is doing its job, so the answer is rarely to turn it off. First ask whether the vendor can use Workload Identity Federation (it works with AWS, Azure, any OIDC or SAML identity provider) or run inside Google Cloud with an attached service account. If a key is truly required, the policy admin sets a project-level override on an isolated project holding only that service account, creates the key, and ideally re-enforces the policy afterwards; existing keys keep working. Add `iam.serviceAccountKeyExpiryHours`, store the key in the vendor's secret store, and alert on its use in audit logs.

KEY POINTS

- Keep keyed service accounts in a separate project with the narrowest roles.
- Recommender and Activity Analyzer show when a key was last used, so stale keys can be deleted.

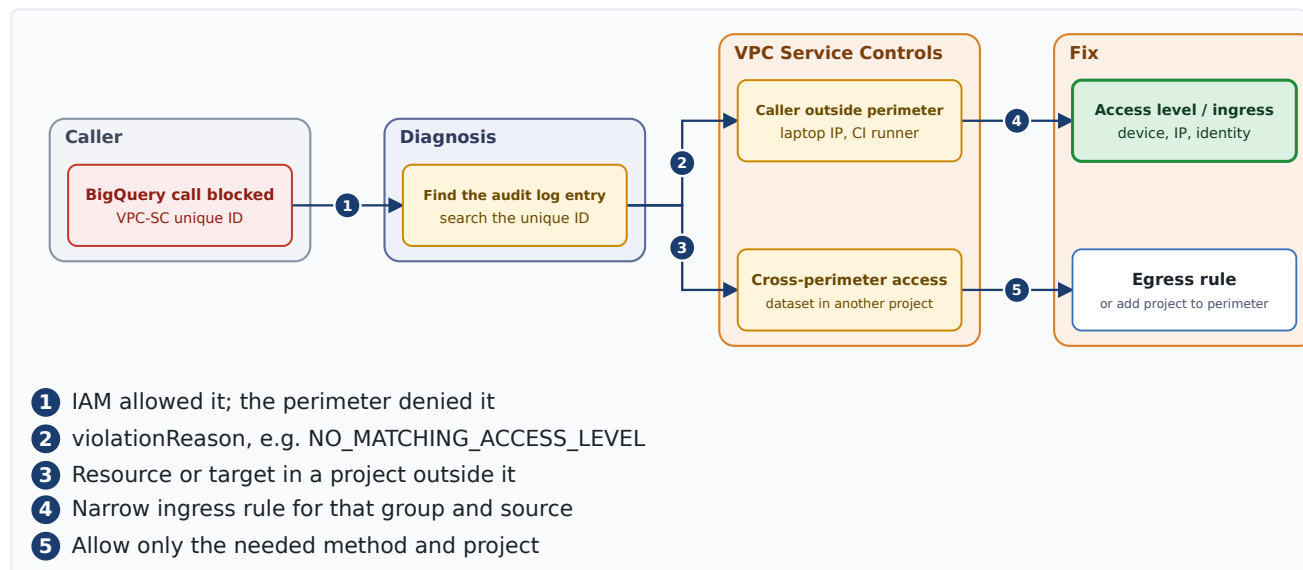
CONFIGURATION EXAMPLE

```

gcloud org-policies describe iam.disableServiceAccountKeyCreation --project=vendor-int --effective
# policy admin: temporary project override
cat > nokey.yaml <<'EOF'
name: projects/vendor-int/policies/iam.disableServiceAccountKeyCreation
spec:
  rules:
    - enforce: false
EOF
gcloud org-policies set-policy nokey.yaml
gcloud iam service-accounts keys create key.json --iam-account=vendor-sync@vendor-int.iam.gserviceaccount.com
  
```

Q50**Troubleshooting**

A data engineer with BigQuery Admin runs a query from a laptop and gets Request is prohibited by organization's policy. vpcServiceControlsUniqueIdentifier: 3xY... How do you troubleshoot it?

FLOW DIAGRAM**EXPLANATION**

VPC Service Controls blocks the request after IAM has allowed it, so adding roles will not help. Search Logs Explorer or the VPC Service Controls troubleshooter for the unique ID: the entry shows the violation reason, the caller, the service and method, and which perimeter blocked it. If the caller is outside the perimeter (a laptop, a SaaS tool or a CI runner), add an access level or an ingress rule for that identity and source; if the call crosses into a project outside the perimeter, add that project or a narrow egress rule. Test changes in dry-run mode first so you do not break other traffic.

KEY POINTS

- Run new perimeters in dry-run for a few weeks and review the violation logs.
- Prefer ingress/egress rules scoped to identities and methods over big access levels.

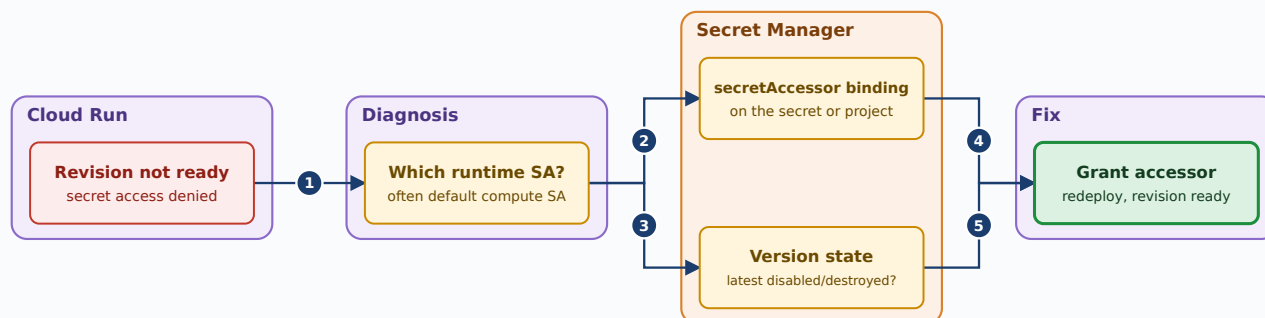
CONFIGURATION EXAMPLE

```

gcloud logging read 'protoPayload.metadata.vpcServiceControlsUniqueId="3xYexample" ' \
  --project=analytics-prod --freshness=1d \
  --
format='yaml(protoPayload.metadata.violationReason,protoPayload.authenticationInfo,protoPayload.methodName)'
gcloud access-context-manager perimeters describe analytics_perimeter --policy=123456789
gcloud access-context-manager perimeters dry-run update analytics_perimeter \
  --policy=123456789 --set-ingress-policies=ingress.yaml
  
```


Q51**Troubleshooting**

A new Cloud Run revision fails to start with `Permission denied on secret: projects/123/secrets/db-password/versions/latest` for Revision service account . How do you fix it?

FLOW DIAGRAM

- 1 The revision reads secrets as its runtime SA
- 2 Grant roles/secretmanager.secretAccessor
- 3 Also check the version is ENABLED
- 5 Enable the version or pin a good one

EXPLANATION

Cloud Run reads secrets with the revision's runtime service account, which is the default compute service account unless `--service-account` was set, so the grant often went to the wrong identity. Check the service account on the revision, then grant `roles/secretmanager.secretAccessor` on that secret (not the whole project) to it. Also confirm the secret is in the project named in the error and that the referenced version is enabled; if `latest` points to a disabled or destroyed version, the revision fails. Deploy again, because the failed revision does not retry by itself.

KEY POINTS

- Pin a version number for env-var secrets; use a mounted volume with `latest` if you need rotation without redeploy.
- Use one runtime SA per service with access only to its own secrets.

CONFIGURATION EXAMPLE

```

gcloud run services describe orders-api --region=europe-west1 \
  --format='value(spec.template.spec.serviceAccountName)'
gcloud secrets versions list db-password --project=prod
gcloud secrets add-iam-policy-binding db-password --project=prod \
  --member=serviceAccount:orders-api@prod.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor
gcloud run deploy orders-api --region=europe-west1 --image=europe-west1-
docker.pkg.dev/prod/app/orders:1.8

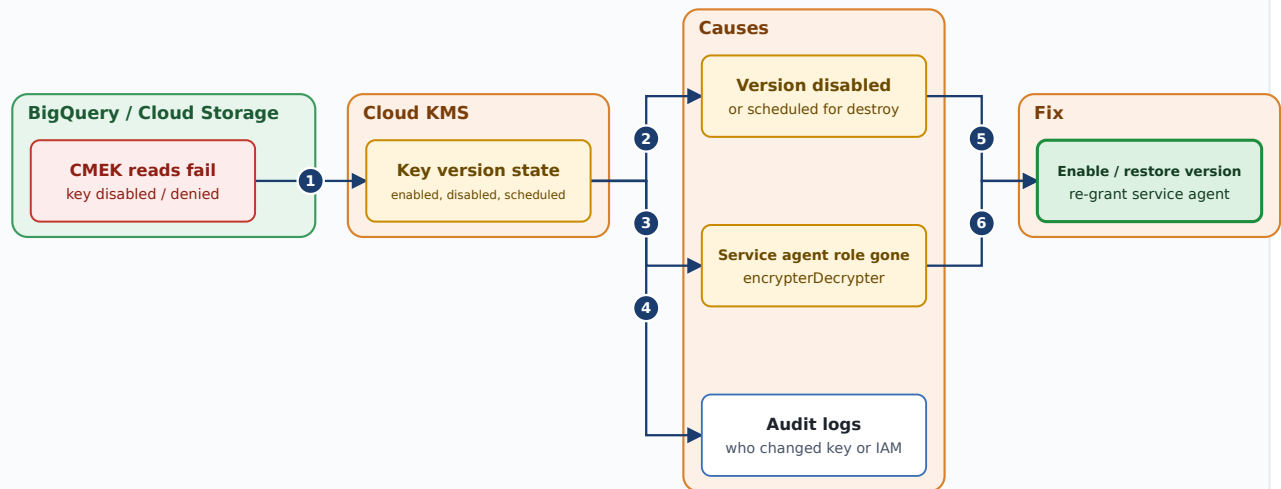
```

Q52

Troubleshooting

BigQuery queries and Cloud Storage reads suddenly fail with errors saying the Cloud KMS key `projects/sec-kms/.../cryptoKeys/data-key` is disabled or cannot be used. How do you restore access?

FLOW DIAGRAM



- 1 Check the primary and older versions
- 2 Disabled data cannot be decrypted
- 3 Each service uses its own service agent
- 4 Find the change and the person
- 5 Restore before the destroy date

EXPLANATION

With CMEK, Google services decrypt data with your key through a per-service service agent, so either a disabled key version or a removed IAM grant stops access. List the key versions: a version in DISABLED state can be re-enabled, and one in DESTROY_SCHEDULED can be restored until its destroy time (24 hours by default, configurable up to 120 days); once destroyed, data encrypted with it is lost. If the versions are fine, check that the BigQuery and Cloud Storage service agents still have `roles/cloudkms.cryptoKeyEncrypterDecrypter` on the key. Audit logs in the KMS project show who disabled the version or changed the policy.

KEY POINTS

- Restrict `cloudkms.cryptoKeyVersions.destroy` and alert on disable or destroy events.
- Set a longer destroy-scheduled duration on keys protecting important data.

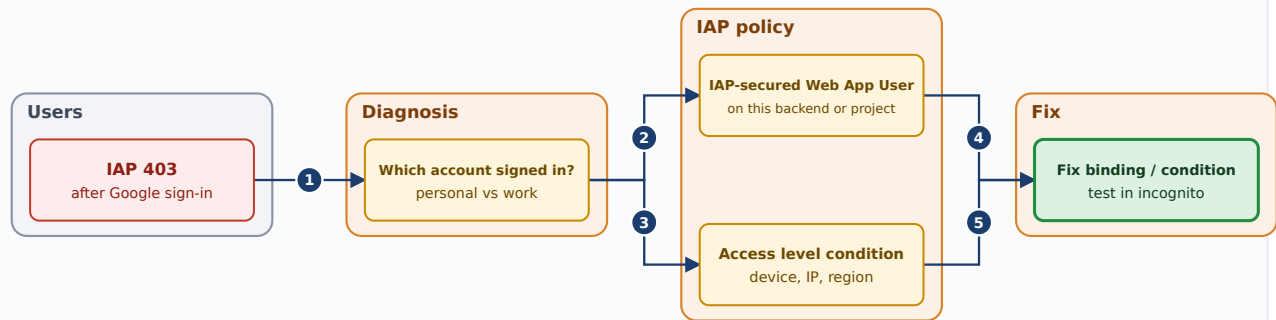
CONFIGURATION EXAMPLE

```

gcloud kms keys versions list --key=data-key --keyring=prod-ring --location=europe --project=sec-kms
gcloud kms keys versions enable 3 --key=data-key --keyring=prod-ring --location=europe --project=sec-kms
gcloud kms keys versions restore 2 --key=data-key --keyring=prod-ring --location=europe --project=sec-kms
gcloud kms keys add-iam-policy-binding data-key --keyring=prod-ring --location=europe --project=sec-kms \
  --member=serviceAccount:service-123456789012@gs-project-accounts.iam.gserviceaccount.com \
  --role=roles/cloudkms.cryptoKeyEncrypterDecrypter
  
```

Q53**Troubleshooting**

Users open an internal app behind IAP and see "You don't have access (403)" after signing in, although they are in the ops group. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Many 403s are the wrong Google account
- ❷ Role must be on the right backend service
- ❸ A context-aware condition may deny them
- ❺ IAP audit logs show which check failed

EXPLANATION

IAP returns 403 after sign-in when the signed-in identity has no `roles/iap.httpsResourceAccessor` for that resource, so first check which account the user actually used, since browsers often pick a personal account. Then check the IAP policy on the specific backend service (or project) and that the ops group is bound, not just granted the role on a different resource; group changes can take a few minutes. If the binding has a condition with an access level (corporate device, IP range), the user may fail it from a home laptop. Enable IAP data access audit logs to see the principal and the reason for each denial.

KEY POINTS

- Grant access to groups, not individual users, on each IAP resource.
- Backends should still verify the `x-goog-iap-jwt-assertion` header.

CONFIGURATION EXAMPLE

```

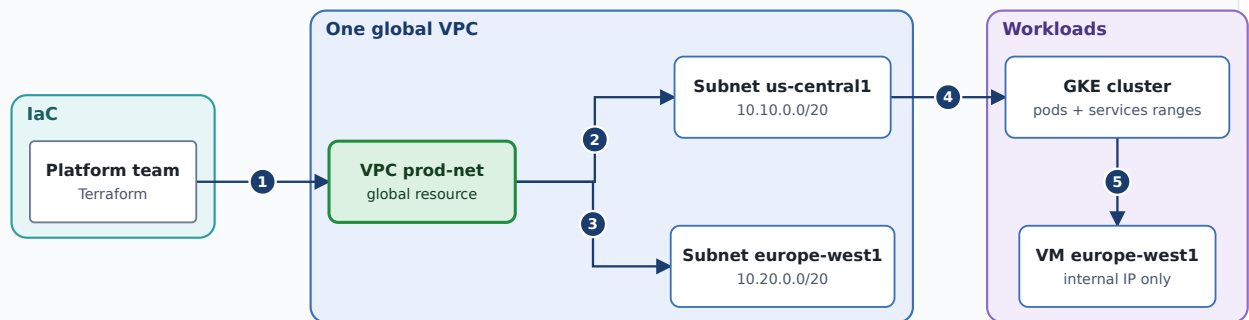
gcloud iap web get-iam-policy --resource-type=backend-services --service=admin-bs --project=ops-prod
gcloud iap web add-iam-policy-binding --resource-type=backend-services --service=admin-bs \
  --project=ops-prod --member=group:ops@example.com --role=roles/iap.httpsResourceAccessor
gcloud logging read 'protoPayload.serviceName="iap.googleapis.com" AND
protoPayload.authorizationInfo.granted=false' \
  --project=ops-prod --freshness=1h --limit=5
  
```

3. Networking: VPC, Load Balancing & Hybrid Connectivity

How Google Cloud networks are built, secured, load balanced and connected to on-premises sites, with the design trade-offs interviewers probe.

Q54 A team is moving from another cloud and plans one VPC per region, joined by VPNs. How would you explain the GCP VPC model and design the network instead?

FLOW DIAGRAM



- 1 Create one custom-mode VPC; the network itself has no region
- 2 Subnets are regional and carry the primary CIDR
- 3 Add a subnet per region you use, no VPN between them
- 4 Secondary ranges give pods and Services their own IPs (VPC-native)
- 5 Pods reach the EU VM over internal IPs on Google's backbone

EXPLANATION

In Google Cloud a VPC network is a global resource and its subnets are regional, so one VPC can span every region you use with no VPN or peering between regions. Routes and firewall rules are also defined at the network level and apply everywhere. Create one custom-mode VPC per environment (or a Shared VPC), add a subnet in each region, and plan non-overlapping CIDRs up front. Give GKE subnets secondary ranges for pods and Services so the cluster is VPC-native and pod IPs are routable across the VPC.

KEY POINTS

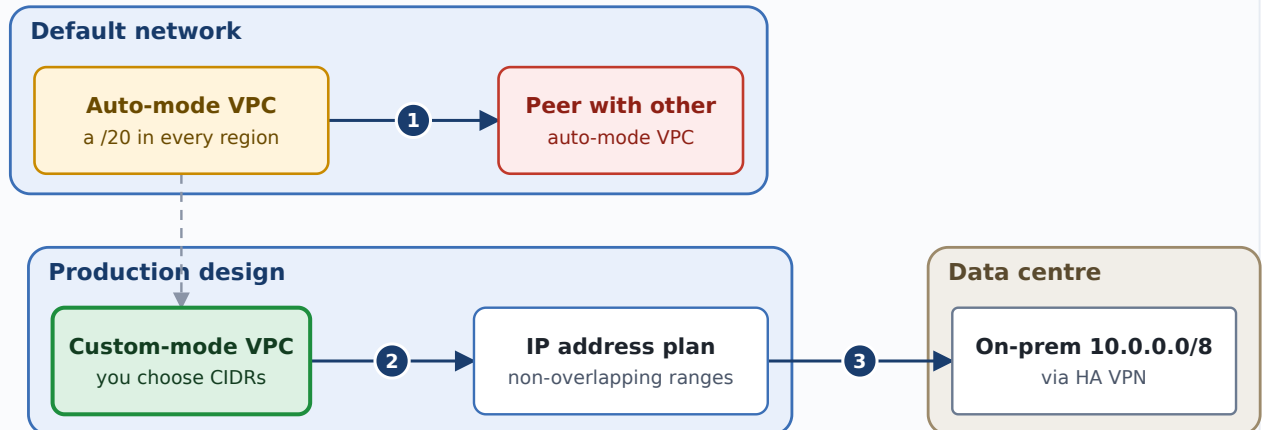
- VPC = global, subnet = regional, VM = zonal.
- Size secondary ranges for pod growth; they are hard to change later.

CONFIGURATION EXAMPLE

```
gcloud compute networks create prod-net --subnet-mode=custom
gcloud compute networks subnets create gke-us \
  --network=prod-net --region=us-central1 --range=10.10.0.0/20 \
  --secondary-range=pods=10.100.0.0/14,services=10.104.0.0/20
```

Q55 A project was created with the default auto-mode network and now needs to peer with another VPC and connect to on-prem 10.0.0.0/8. What goes wrong and what do you recommend?

FLOW DIAGRAM



- ❶ Every auto-mode VPC uses the same 10.128.0.0/9 blocks, so peering fails on overlap
- ❷ Custom mode: create only the subnets you need, from an IP plan
- ❸ Ranges avoid on-prem space, so routes can be exchanged over BGP
- An auto-mode VPC can be switched to custom once (one-way)

EXPLANATION

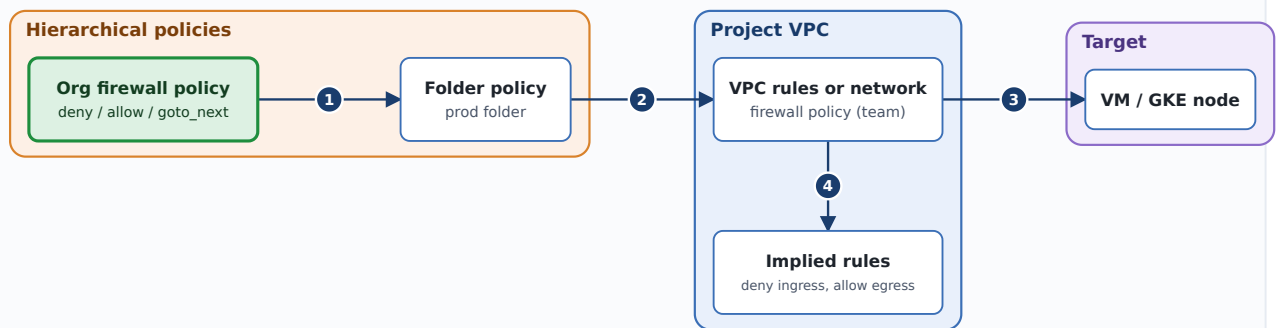
Auto-mode VPCs create a /20 subnet in every region, taken from 10.128.0.0/9. Every auto-mode network uses exactly the same ranges. So two auto-mode networks cannot be peered, and their ranges can clash with corporate address space such as 10.0.0.0/8. New regions also get subnets added automatically, which you may not want. For production use custom-mode VPCs: you create only the subnets you need, with ranges from an enterprise IP plan (per environment and region). Leave room for GKE secondary ranges, proxy-only subnets and PSC subnets. Stop new projects from getting the default network with the org policy `compute.skipDefaultNetworkCreation`.

KEY POINTS

- Auto mode is fine for labs, not for hybrid or peered networks.
- Switching auto to custom is one-way: `gcloud compute networks update NET --switch-to-custom-subnet-mode`.

Q56 Security wants to block SSH and RDP from the internet across every project, while app teams still manage their own rules. How do you implement this?

FLOW DIAGRAM



- ❶ Org rule denies tcp:22,3389 from 0.0.0.0/0; others say goto_next
- ❷ Folder rules evaluated next, then passed down with goto_next
- ❸ Team rules: first match by priority number wins
- ❹ If nothing matches, implied rules apply

EXPLANATION

Use a hierarchical firewall policy attached at the organization (or a folder) with a deny rule for tcp:22 and tcp:3389 from 0.0.0.0/0, and let everything else fall through with `goto_next`. Hierarchical policies are evaluated before the VPC's own firewall rules and network firewall policies, so a project owner cannot override the deny. Teams keep managing VPC rules or network firewall policies for their apps, and admins reach VMs through IAP TCP forwarding (source 35.235.240.0/20) which you allow explicitly. If no rule matches, the implied rules deny ingress and allow egress.

KEY POINTS

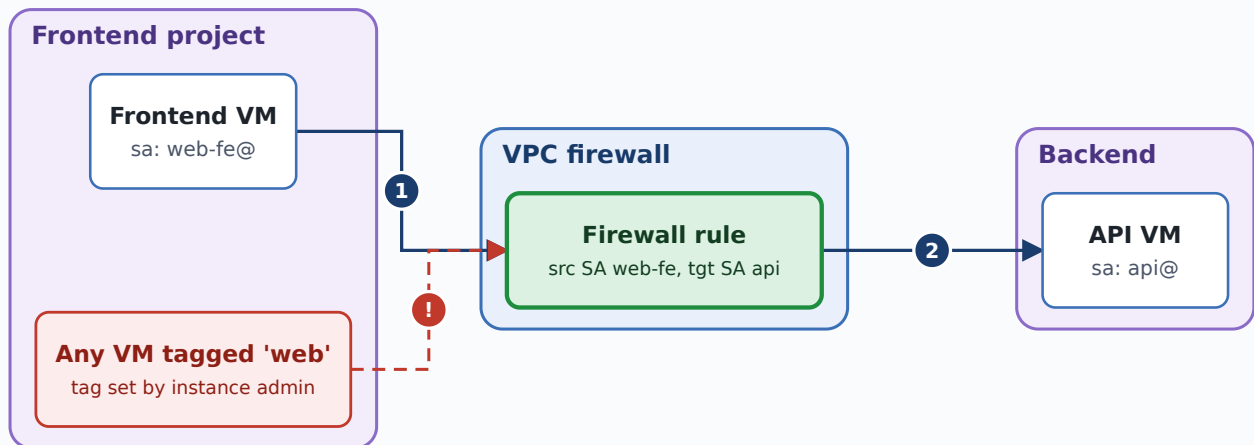
- Order: org policy, folder policy, then VPC rules / network policies, then implied rules.
- `goto_next` delegates; `allow` at org level also cannot be overridden lower down.

CONFIGURATION EXAMPLE

```
gcloud compute firewall-policies create --organization=123456789 \
  --short-name=org-baseline
gcloud compute firewall-policies rules create 1000 \
  --firewall-policy=org-baseline --organization=123456789 \
  --direction=INGRESS --action=deny \
  --src-ip-ranges=0.0.0.0/0 --layer4-configs=tcp:22,tcp:3389
gcloud compute firewall-policies associations create \
  --firewall-policy=org-baseline --organization=123456789
```

Q57 Only the frontend VMs should reach the API VMs on port 8080. Today the rule uses network tags. Why might you switch to service accounts?

FLOW DIAGRAM



- ❶ Packet from a VM running as web-fe@
- ❷ Allowed on tcp:8080 because the source service account matches
- ❗ Denied: a tag alone no longer grants access

EXPLANATION

Network tags are free-form strings that anyone with `compute.instances.setTags` (for example Compute Instance Admin) can add, so a rule keyed on tags can be widened by mistake or on purpose. Service accounts are identities: to run a VM as a service account you need `iam.serviceAccountUser` on it, and changing a VM's service account requires stopping the VM. Firewall rules can target and match source by service account, so `--source-service-accounts` and `--target-service-accounts` express who talks to whom. You cannot mix tags and service accounts in the same rule, and service-account source matching only works for traffic inside the VPC.

KEY POINTS

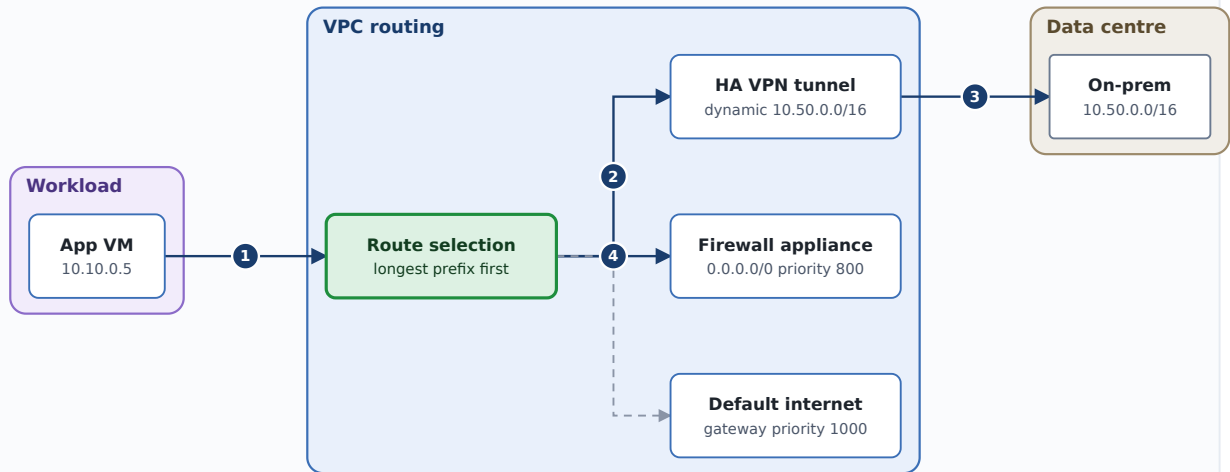
- Tags are labels; service accounts are access-controlled identities.
- Network firewall policies also support IAM-governed secure tags.

CONFIGURATION EXAMPLE

```
gcloud compute firewall-rules create allow-fe-to-api \
  --network=prod-net --direction=INGRESS --action=allow --rules=tcp:8080 \
  --source-service-accounts=web-fe@prod-p.iam.gserviceaccount.com \
  --target-service-accounts=api@prod-p.iam.gserviceaccount.com
```

Q58 A VPC has a default internet route, a 0.0.0.0/0 route to a firewall appliance with priority 800, and a BGP route for 10.50.0.0/16. Which path does each packet take?

FLOW DIAGRAM



- ❶ Packets to 10.50.1.9 and to 8.8.8.8
- ❷ The /16 is more specific than 0.0.0.0/0, so on-prem traffic uses the tunnel
- ❸ Two default routes: the lower priority number (800) wins
- Default internet route is used only if the appliance route is removed

EXPLANATION

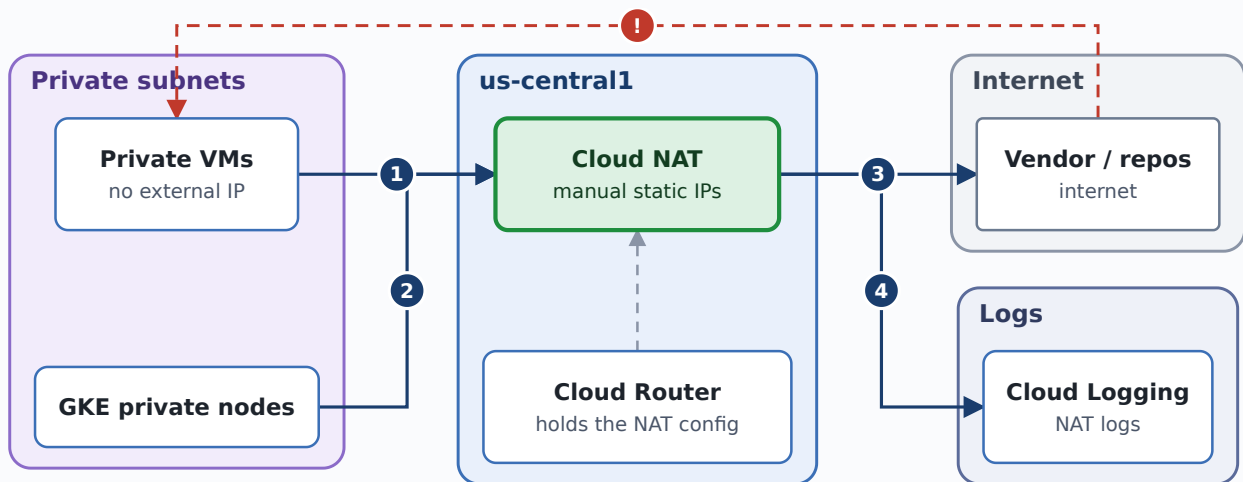
Google Cloud first picks the most specific matching route (longest prefix match). Traffic to 10.50.1.9 therefore follows the 10.50.0.0/16 route learned over BGP, because /16 is more specific than 0.0.0.0/0. Priority only matters when several routes have the same prefix length; the lower number wins. So internet-bound traffic goes to the appliance (priority 800) rather than the default internet gateway (priority 1000). Subnet routes cannot be overridden by less specific custom routes. Static routes can be limited to VMs with certain network tags. An internal passthrough load balancer can be a route's next hop, which lets you run firewall appliances in a highly available pair.

KEY POINTS

- Prefix length first, then priority, then equal-cost multipath.
- Use an internal passthrough NLB as next hop for HA appliances.

Q59 Private VMs and GKE nodes with no external IPs must download OS patches, and a vendor wants a fixed source IP to allowlist. How do you set this up and what can go wrong?

FLOW DIAGRAM



- 1 Outbound connections only
- 3 Source NAT to reserved IPs the vendor can allowlist
- 4 Log errors to spot port exhaustion
- NAT is configured on a Cloud Router; no VM sits in the data path
- ! Unsolicited inbound connections are dropped

EXPLANATION

Create a Cloud Router in the region and a Cloud NAT gateway on it for the relevant subnets, using manually reserved static IPs so the vendor can allowlist them. Cloud NAT is a distributed, software-defined service, so there is no NAT VM to scale or patch, and it only allows return traffic for connections started from inside. The usual failure is source port exhaustion when many connections go to the same destination IP and port, which shows up as dropped packets in NAT logs; fix it with dynamic port allocation, a higher minimum ports per VM, or more NAT IPs. Cloud NAT is regional, so each region needs its own gateway.

KEY POINTS

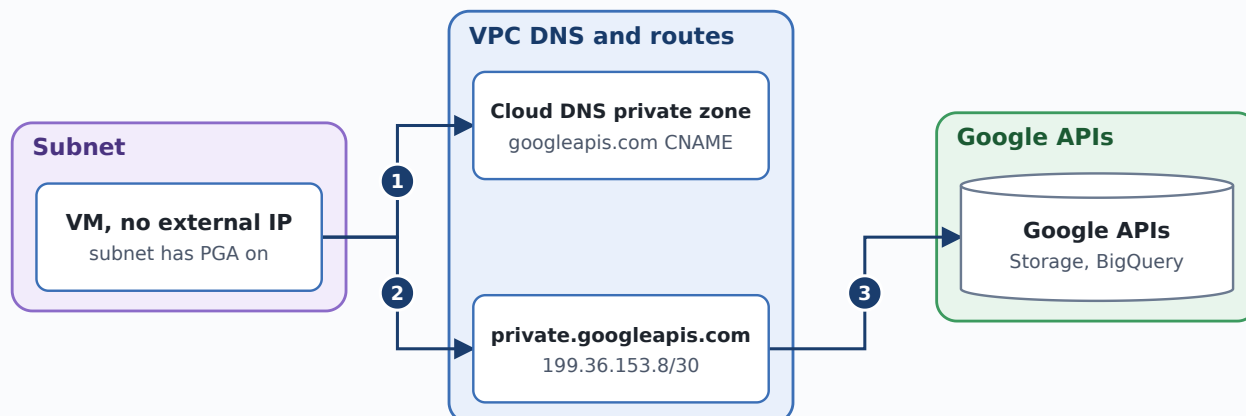
- Each NAT IP gives 64,512 ports to share across VMs.
- Enable logging for errors only to keep costs down.

CONFIGURATION EXAMPLE

```
gcloud compute routers create nat-router --network=prod-net --region=us-central1
gcloud compute addresses create nat-ip-1 --region=us-central1
gcloud compute routers nats create nat-gw --router=nat-router \
  --region=us-central1 --nat-all-subnet-ip-ranges \
  --nat-external-ip-pool=nat-ip-1 --enable-dynamic-port-allocation \
  --enable-logging --log-filter=ERRORS_ONLY
```

Q60 VMs without external IPs need to read from Cloud Storage and BigQuery without going through NAT. How do you enable this, and what changes if VPC Service Controls is in use?

FLOW DIAGRAM



- 1 storage.googleapis.com resolves to the private VIP via CNAME
- 2 Route for the /30 points to the default internet gateway
- 3 Traffic stays on Google's network; no NAT needed
- With VPC Service Controls use restricted.googleapis.com (199.36.153.4/30)

EXPLANATION

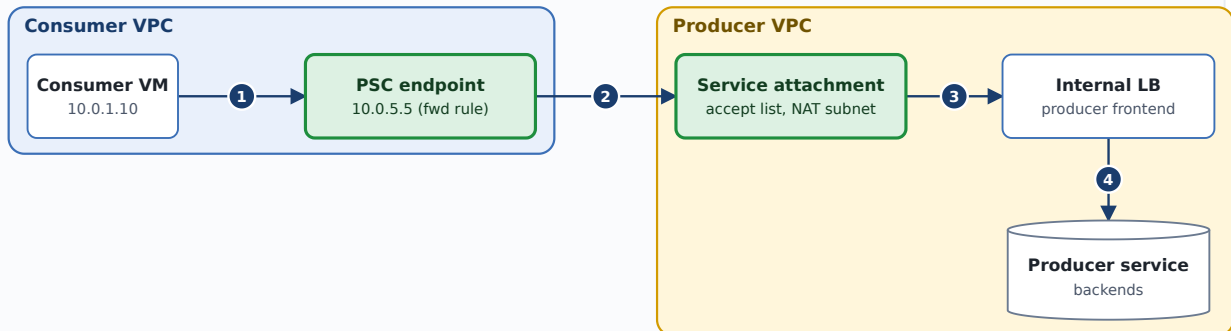
Turn on Private Google Access on the subnet; VMs with only internal IPs can then reach Google APIs using the default domains as long as a route to the default internet gateway exists. For tighter control, create a private Cloud DNS zone for `googleapis.com` that CNAMEs `*.googleapis.com` to `private.googleapis.com` (199.36.153.8/30) and add a route for that range. With VPC Service Controls, use `restricted.googleapis.com` (199.36.153.4/30) instead so only services inside the perimeter are reachable. Private Google Access does not provide general internet access; that still needs Cloud NAT.

KEY POINTS

- PGA is a per-subnet setting.
- Private Service Connect endpoints for Google APIs are an alternative with your own internal IP.

Q61 A SaaS provider and a customer both use 10.0.0.0/16. The customer needs private access to the provider's service. How does Private Service Connect solve it?

FLOW DIAGRAM



- 1 App calls an internal IP in its own VPC
- 2 PSC carries traffic; no peering, so overlapping CIDRs are fine
- 3 Source is NATed from the producer's PSC NAT subnet

EXPLANATION

The producer publishes the service behind an internal load balancer and creates a service attachment with a dedicated PSC NAT subnet and an accept list of consumer projects. The consumer creates a PSC endpoint, a forwarding rule with an internal IP in its own VPC, that targets the service attachment. Traffic is one-way from consumer to producer and is NATed, so address ranges can overlap and neither side sees the other's network. PSC endpoints can also front Google APIs (the all-apis or vpc-sc bundle), and PSC backends let a load balancer in the consumer VPC add its own controls.

KEY POINTS

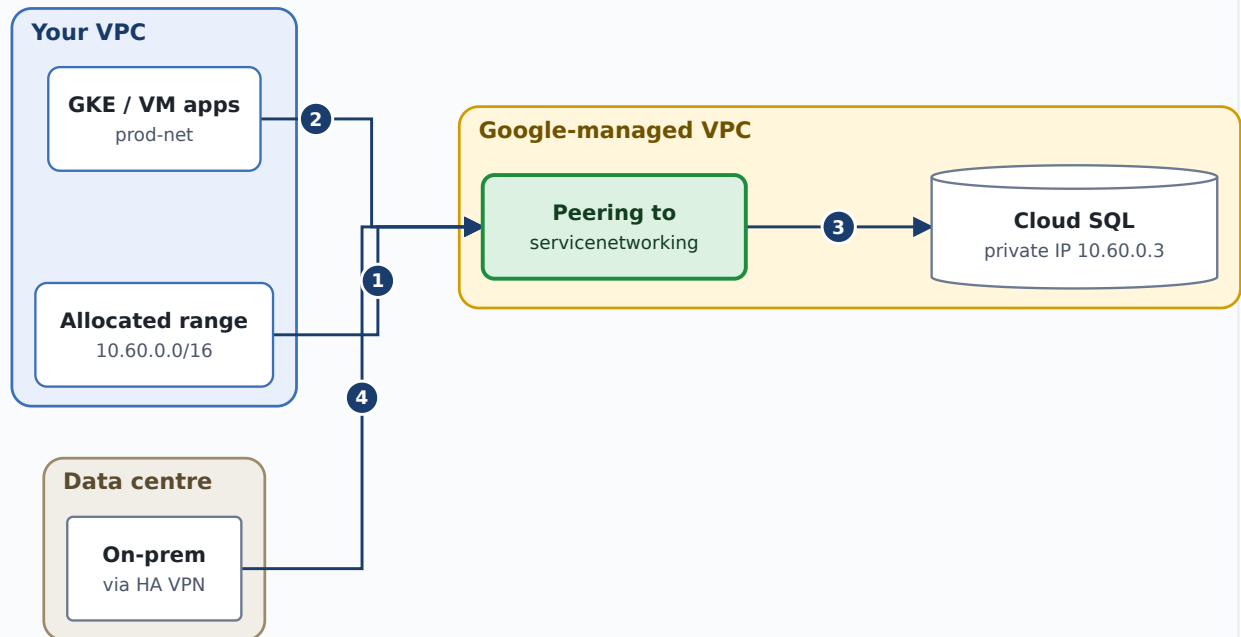
- PSC is service-scoped; VPC peering is network-scoped.
- The producer approves consumers per project or network.

CONFIGURATION EXAMPLE

```
gcloud compute addresses create psc-ip --region=us-central1 \
  --subnet=consumer-subnet --addresses=10.0.5.5
gcloud compute forwarding-rules create psc-ep --region=us-central1 \
  --network=consumer-net --address=psc-ip \
  --target-service-attachment=projects/saas-p/regions/us-central1/serviceAttachments/db-sa
```

Q62 Cloud SQL must have a private IP only, reachable from GKE and from on-prem over VPN. How do you configure private services access, and why can on-prem not reach it at first?

FLOW DIAGRAM



- 1 Reserve a range and connect servicenetworking (VPC peering)
- 2 Apps reach the instance's private IP over the peering
- 4 Works only after exporting custom routes and advertising the range to on-prem

EXPLANATION

First reserve an internal IP range with purpose VPC_PEERING. Then create a private connection to `servicenetworking.googleapis.com`. Cloud SQL instances get IPs from that range inside a Google-managed VPC, which is peered to yours. VPC peering does not pass on routes learned from elsewhere (it is not transitive) by default. So on-prem cannot reach the instance until you do two things: export custom routes on the peering, so the Google side learns the on-prem routes, and advertise the reserved range from Cloud Router to on-prem. Size the range for future instances, because other managed services can share it. Private Service Connect for Cloud SQL is an alternative that avoids peering.

KEY POINTS

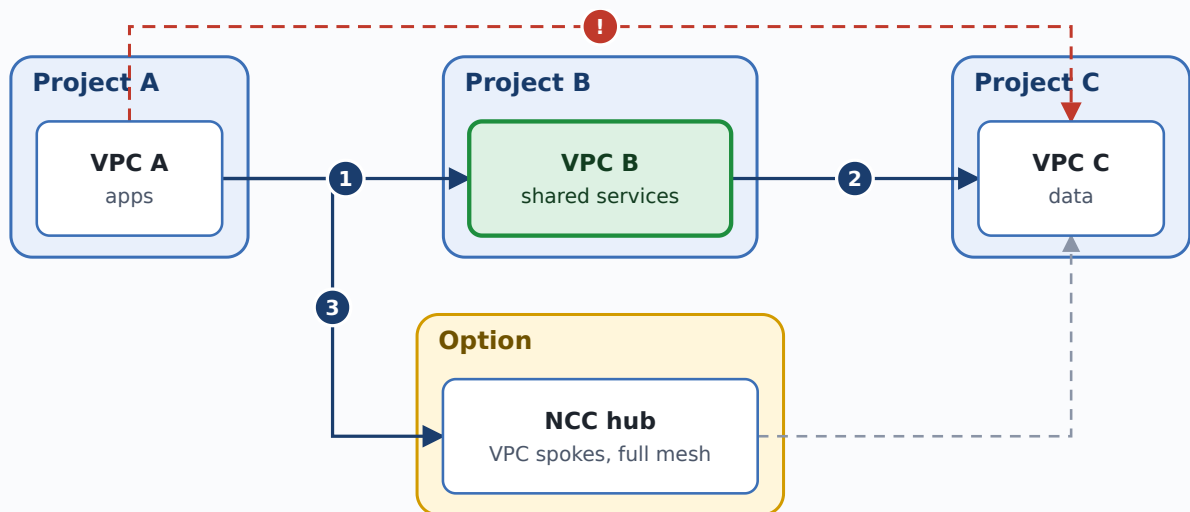
- Two fixes for hybrid: export custom routes, custom BGP advertisement.
- Cloud SQL Auth Proxy still adds IAM auth and TLS.

CONFIGURATION EXAMPLE

```
gcloud compute addresses create google-managed --global \
  --purpose=VPC_PEERING --prefix-length=16 --network=prod-net
gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com \
  --ranges=google-managed --network=prod-net
gcloud compute networks peerings update servicenetworking-googleapis-com \
  --network=prod-net --export-custom-routes
```

Q63 VPC A is peered with B, and B is peered with C. Workloads in A cannot reach C. Why, and what are your options?

FLOW DIAGRAM



- ❶ Peering A-B exchanges subnet routes
- ❷ Peering B-C exchanges subnet routes
- ❗ No route: peering is not transitive
- ❸ Fix: make A and C spokes of one hub

EXPLANATION

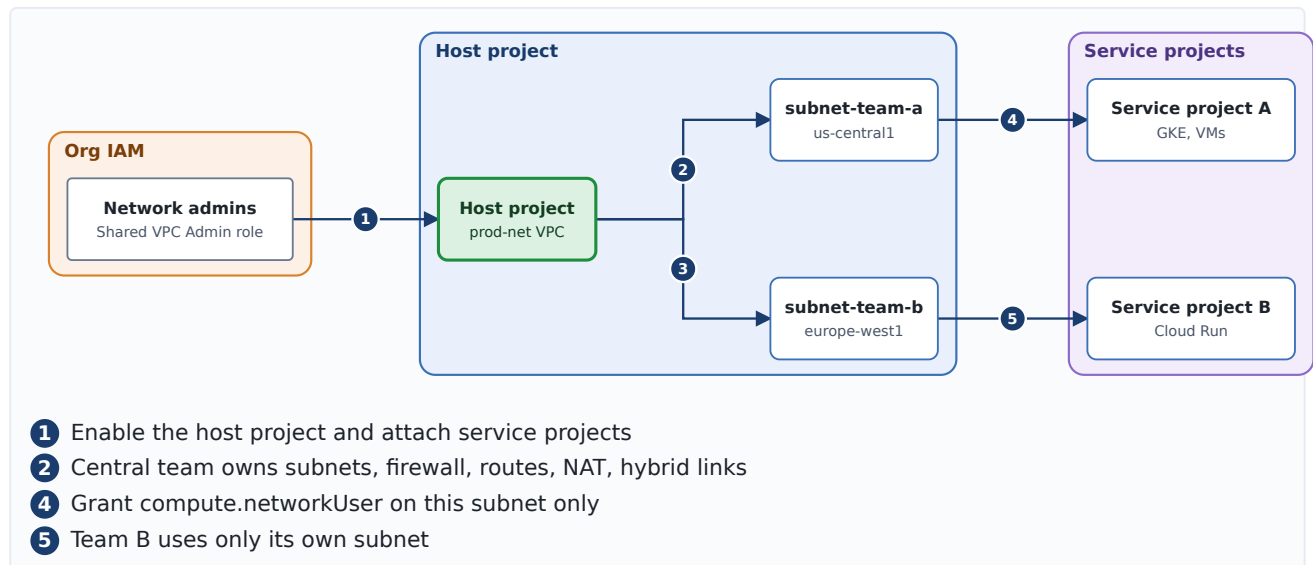
VPC peering only exchanges routes between the two peered networks, so A never learns C's subnets through B. Options are a direct A-C peering (fine for a few VPCs but it grows as a mesh), a Network Connectivity Center hub with VPC spokes that gives full-mesh reachability, HA VPN between VPCs, or consolidating into a Shared VPC. Peering also forbids overlapping subnet ranges and has per-network limits on peers and routes, so check current quotas before building large meshes. Routes learned from on-prem can be shared over peering with import/export of custom routes, but that does not make VPC-to-VPC transit work.

KEY POINTS

- Peering: no overlap, no transit, per-network quotas.
- NCC or Shared VPC scale better than a peering mesh.

Q64 Ten app teams each have their own project, but networking must be controlled centrally. How do you design Shared VPC?

FLOW DIAGRAM



EXPLANATION

Create a host project per environment (for example prod and non-prod) that owns the VPC, subnets, firewall policies, Cloud NAT and hybrid connectivity, and attach each team's project as a service project. Teams keep billing, IAM and resources in their own projects but deploy into subnets they have been granted `roles/compute.networkUser` on, preferably at subnet level rather than project level. The central team keeps `roles/compute.networkAdmin` and security admin on the host. For GKE in service projects, also grant the GKE service agent the Host Service Agent User role on the host project.

KEY POINTS

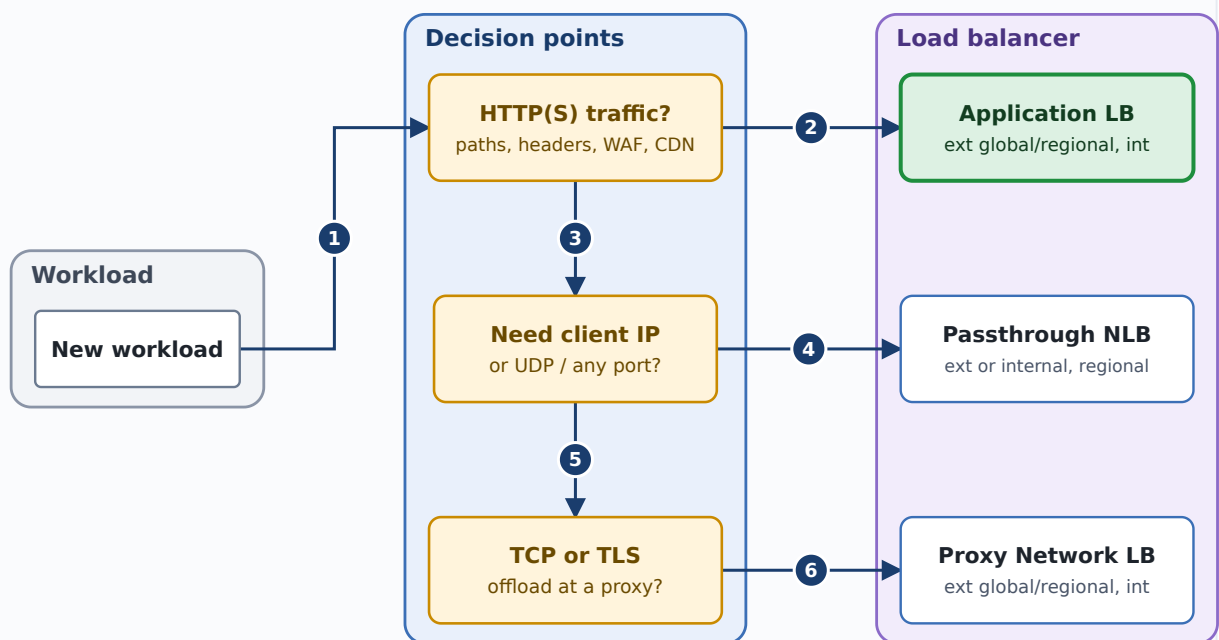
- A service project attaches to exactly one host project.
- Separate hosts per environment keep prod isolated.

CONFIGURATION EXAMPLE

```
gcloud compute shared-vpc enable host-prod
gcloud compute shared-vpc associated-projects add team-a-prod --host-project=host-prod
gcloud compute networks subnets add-iam-policy-binding subnet-team-a \
  --project=host-prod --region=us-central1 \
  --member=group:team-a@example.com --role=roles/compute.networkUser
```

Q65 You are asked to pick a load balancer for a new workload. What questions do you ask, and how do they map to Google Cloud load balancer types?

FLOW DIAGRAM



- 1 Start with the protocol
- 2 Yes: Application LB (Envoy-based, layer 7)
- 3 No
- 4 Yes: passthrough NLB keeps source IP, supports UDP
- 5 No
- 6 Yes: proxy NLB terminates TCP/TLS

EXPLANATION

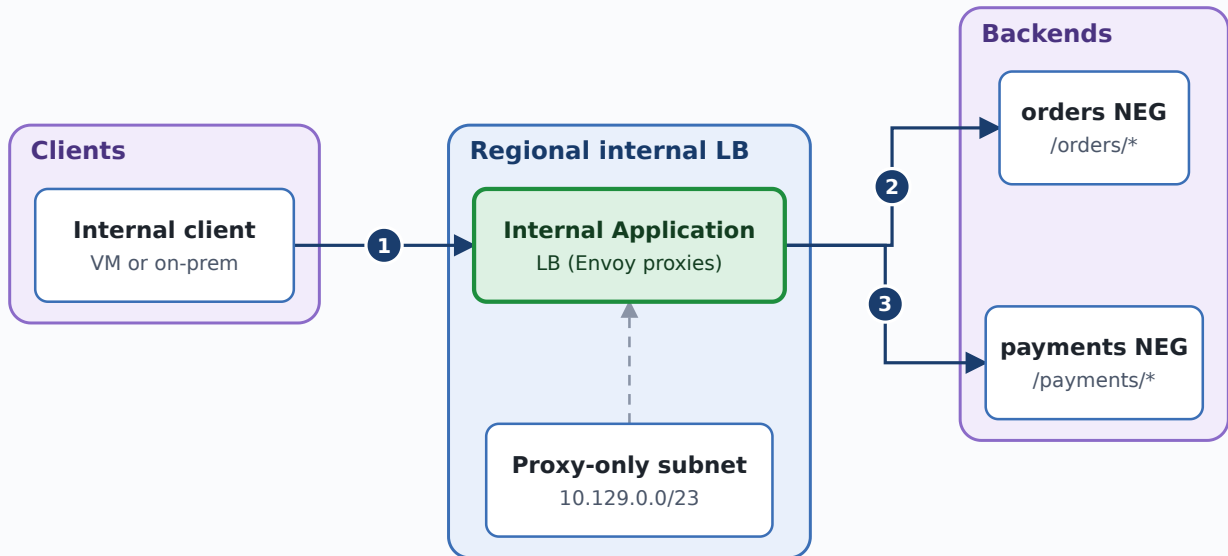
First ask whether the traffic is HTTP(S); if you need host or path routing, header rewrites, Cloud Armor or Cloud CDN, use an Application Load Balancer. If not, ask whether backends must see the client IP, use UDP or arbitrary ports; then use a passthrough Network Load Balancer, which forwards packets unchanged. For other TCP or TLS traffic that benefits from offload or global anycast, use a proxy Network Load Balancer. Then decide external or internal, and global or regional: global for worldwide users on one anycast IP, regional for data residency or Standard network tier.

KEY POINTS

- Proxy LBs hide the client IP (use X-Forwarded-For or PROXY protocol).
- Passthrough LBs are always regional.

Q66 Internal microservices in one region need path-based routing, mTLS options and traffic splitting, but must not be public. Which load balancer do you use and what network prerequisites exist?

FLOW DIAGRAM



- 1 HTTPS to the forwarding rule's internal IP
- 2 URL map sends /orders to one backend service
- 3 Weighted splits allow canary releases
- Proxies take source IPs from the proxy-only subnet; allow it in firewall rules

EXPLANATION

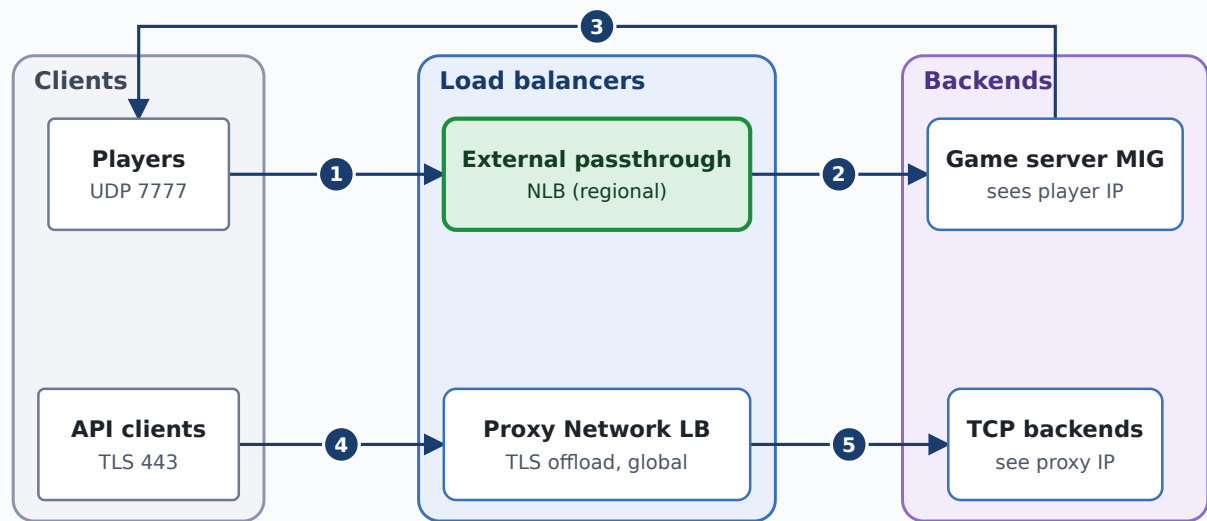
Use a regional internal Application Load Balancer, which runs managed Envoy proxies inside your VPC and supports URL maps, header routing, weighted traffic splitting and Cloud Armor policies for internal traffic. It needs a proxy-only subnet (purpose REGIONAL_MANAGED_PROXY) in each region, and backends must allow ingress from that subnet and from health-check ranges. Enable global access on the forwarding rule if clients in other regions must reach it, or use the cross-region internal Application Load Balancer for multi-region failover. On-prem clients can reach it over VPN or Interconnect like any internal IP.

KEY POINTS

- One proxy-only subnet per region per VPC is shared by all Envoy-based LBs.
- Backends see the proxy IP; the client IP is in X-Forwarded-For.

Q67 A game studio serves UDP game traffic and a TCP/TLS API. Game servers must see player IPs. Which network load balancers do you choose?

FLOW DIAGRAM



- ❶ Packets forwarded unchanged by Maglev
- ❷ Backend receives the original source IP
- ❸ Replies go straight to the player (direct server return)
- ❹ TLS ends at Google's edge
- ❺ New TCP connection; client IP only via PROXY protocol

EXPLANATION

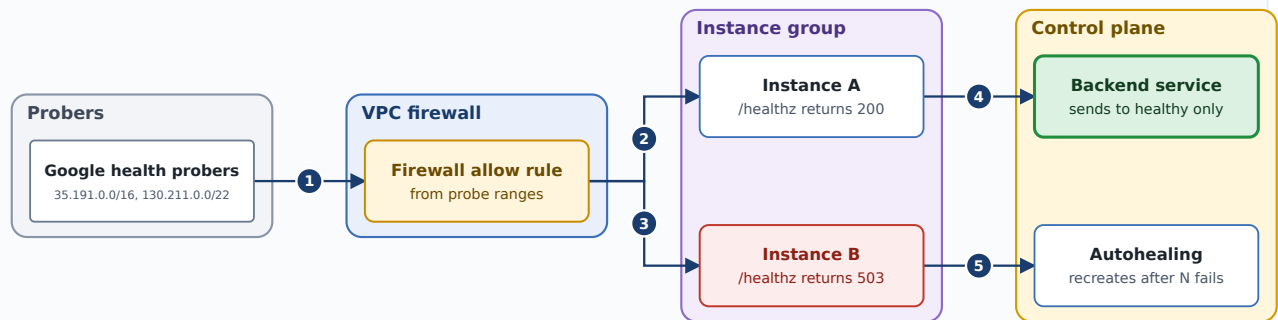
Use an external passthrough Network Load Balancer for the UDP game traffic: it is regional, forwards packets without proxying, keeps the player's source IP and lets replies go directly back from the VM. For the TLS API, a global external proxy Network Load Balancer terminates TLS at Google's edge close to the client and opens new connections to backends, which gives lower latency worldwide but hides the client IP unless you enable PROXY protocol. Firewall rules on passthrough backends must allow the real client ranges, not the proxy ranges. If players are spread globally, run a passthrough NLB per region and steer with DNS.

KEY POINTS

- Passthrough: client IP, UDP, regional.
- Proxy: TLS offload, global anycast, client IP hidden.

Q68 After a deploy, the load balancer marks every backend unhealthy although the app works when you curl it locally. How do you troubleshoot, and how do LB health checks differ from autohealing?

FLOW DIAGRAM



- 1 Probes come from fixed Google ranges
- 2 Missing rule is the most common cause of all-unhealthy
- 4 Healthy backends stay in rotation
- 5 Separate, more tolerant health check recreates the VM

EXPLANATION

Check that a firewall rule allows the health-check source ranges (35.191.0.0/16 and 130.211.0.0/22 for most load balancers) to the health-check port, then confirm the port, path and protocol match what the app serves and that the app listens on the VM's interface, not just localhost. `gcloud compute backend-services get-health` shows per-instance state. LB health checks only take a backend out of rotation; MIG autohealing uses its own health check to recreate VMs, so give it a longer initial delay and higher thresholds to avoid recreate loops during slow starts.

KEY POINTS

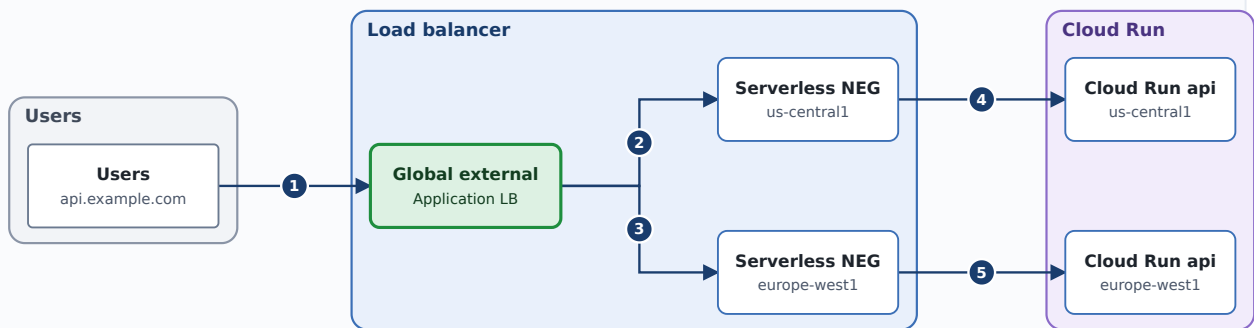
- Health-check path should test the app, not deep dependencies.
- Use separate checks for load balancing and autohealing.

CONFIGURATION EXAMPLE

```
gcloud compute health-checks create http hc-web --port=8080 \
  --request-path=/healthz --check-interval=10s --timeout=5s \
  --healthy-threshold=2 --unhealthy-threshold=3
gcloud compute firewall-rules create allow-hc --network=prod-net \
  --allow=tcp:8080 --source-ranges=35.191.0.0/16,130.211.0.0/22
```

Q69 A Cloud Run API is deployed in two regions. You need one global HTTPS endpoint with a custom domain and Cloud Armor. How do you front it?

FLOW DIAGRAM



- 1 One anycast IP, managed certificate, Cloud Armor policy
- 2 Users are sent to the closest region
- 4 Serverless NEG points at the service by name

EXPLANATION

Create a serverless network endpoint group in each region pointing at the Cloud Run service, add both to one global backend service, and put a global external Application Load Balancer with a Google-managed certificate in front. The LB sends users to the nearest region and lets you attach Cloud Armor and Cloud CDN. Set the Cloud Run ingress to `internal-and-cloud-load-balancing` so the default `run.app` URL cannot bypass the LB. Serverless NEGs do not use health checks, so for regional failover rely on outlier detection on the backend service (check current support) and on Cloud Run's own regional redundancy.

KEY POINTS

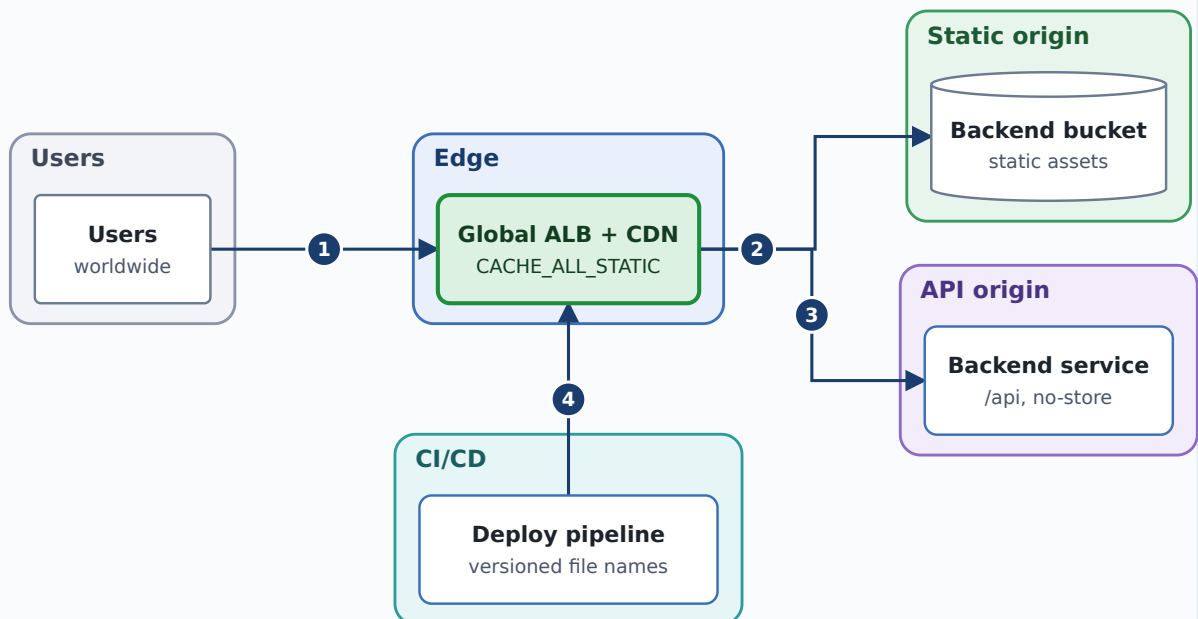
- Serverless NEGs also work for Cloud Functions and App Engine.
- Lock ingress so traffic must pass through the LB.

CONFIGURATION EXAMPLE

```
gcloud compute network-endpoint-groups create api-neg-us --region=us-central1 \
  --network-endpoint-type=serverless --cloud-run-service=api
gcloud compute backend-services add-backend api-bs --global \
  --network-endpoint-group=api-neg-us --network-endpoint-group-region=us-central1
gcloud run services update api --region=us-central1 \
  --ingress=internal-and-cloud-load-balancing
```

Q70 A marketing site on Cloud Storage and an API behind the same load balancer have slow global load times. After releases, users still see old files. How do you use Cloud CDN and handle invalidation?

FLOW DIAGRAM



- 1 Cache hits served from Google's edge
- 2 Miss: fetch from origin, honour Cache-Control max-age
- 3 Dynamic or private responses are not cached
- 4 Upload app.3f9c.js; invalidate only index.html

EXPLANATION

Enable Cloud CDN on the backend bucket with a cache mode such as `CACHE_ALL_STATIC` and sensible TTLs, and keep the API backend uncached or rely on its Cache-Control headers. The stale-file problem is best solved with versioned or hashed asset names, so a new release references new URLs and old cache entries do not matter. Keep HTML short-lived (for example `max-age=60`) and use invalidation only for a few entry-point paths, because invalidations take time to propagate and are rate limited. Tune the cache key (drop unneeded query strings or cookies) to raise the hit ratio.

KEY POINTS

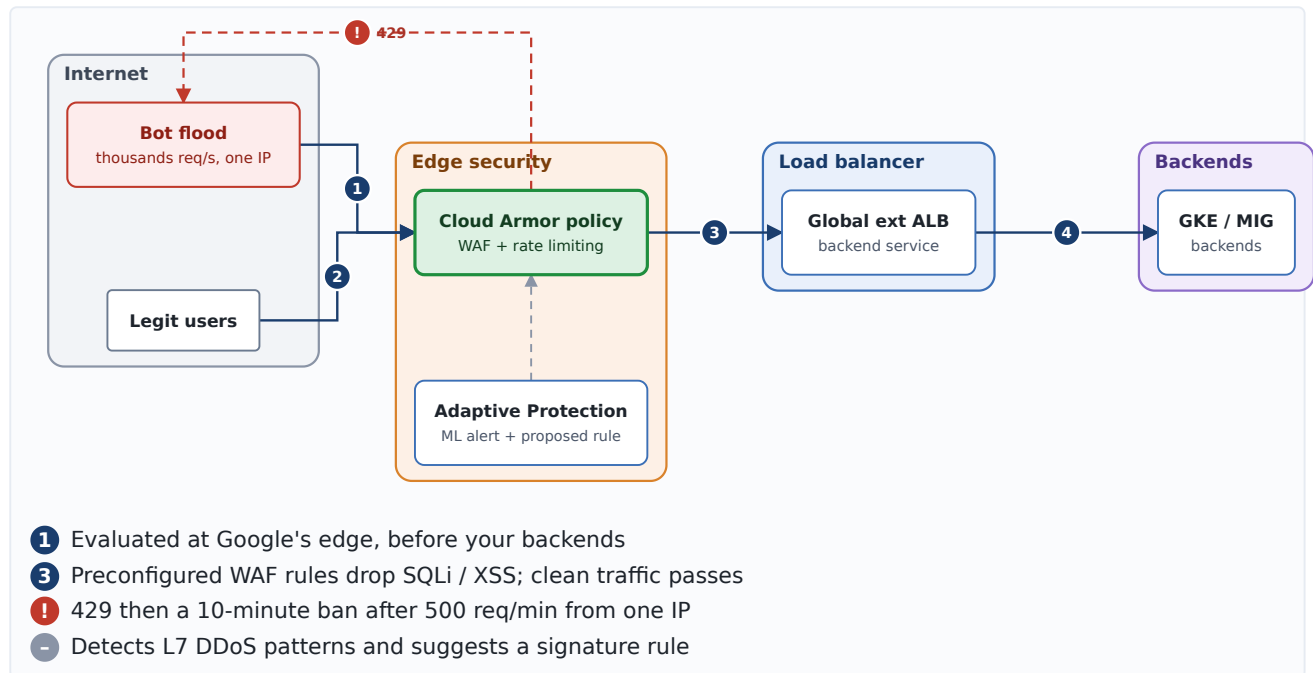
- Versioned URLs beat mass invalidation.
- Watch cache hit ratio in the CDN monitoring dashboard.

CONFIGURATION EXAMPLE

```
gcloud compute backend-buckets update static-bb --enable-cdn \
  --cache-mode=CACHE_ALL_STATIC --default-ttl=3600
gcloud compute url-maps invalidate-cdn-cache web-map \
  --path="/index.html" --async
```

Q71 A public API behind a global load balancer is hit by SQL injection attempts and a bot sending thousands of requests per second. How do you protect it with Cloud Armor?

FLOW DIAGRAM



EXPLANATION

Attach a Cloud Armor security policy to the backend service and add preconfigured WAF rules (for example `sqli-v33-stable`, `xss-v33-stable`) first in preview mode, then enforce once logs show few false positives. Add a rate-based ban rule keyed on client IP (or a header such as an API key) so abusive clients get 429 and are banned for a period. Turn on Adaptive Protection to detect layer-7 DDoS and get suggested rules. Cloud Armor Enterprise adds managed protection and threat intelligence; also use geo and IP allow/deny lists where business rules allow.

KEY POINTS

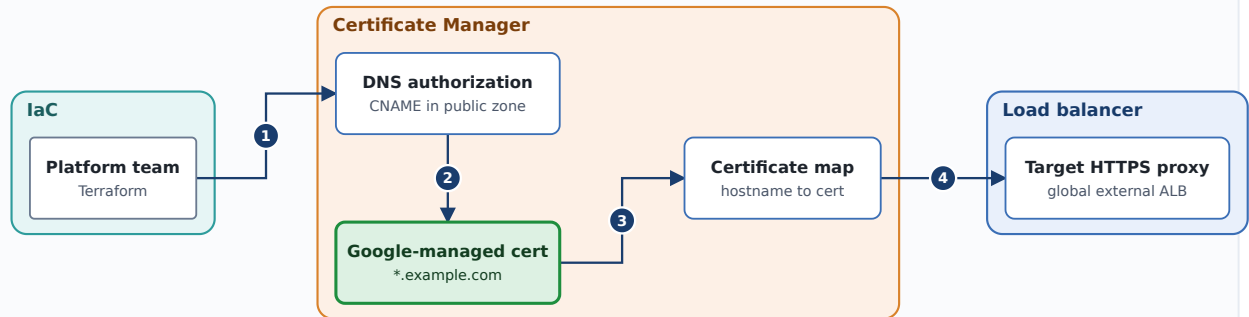
- Deploy WAF rules with `--preview` first and read the logs.
- Rules evaluate by priority; lowest number first.

CONFIGURATION EXAMPLE

```
gcloud compute security-policies rules create 1000 --security-policy=edge-pol \
  --expression="evaluatePreconfiguredWaf('sqli-v33-stable')" --action=deny-403
gcloud compute security-policies rules create 2000 --security-policy=edge-pol \
  --src-ip-ranges="*" --action=rate-based-ban \
  --rate-limit-threshold-count=500 --rate-limit-threshold-interval-sec=60 \
  --ban-duration-sec=600 --conform-action=allow --exceed-action=deny-429 \
  --enforce-on-key=IP
```

Q72 You must serve `example.com` and all its subdomains through one global load balancer, with certificates issued before DNS is cut over. How do you manage the certificates?

FLOW DIAGRAM



- 1 Create a DNS authorization and add its CNAME record
- 2 Wildcard cert issues without pointing traffic at the LB
- 3 Map entries select the cert by hostname
- 4 Attach the map to the target proxy

EXPLANATION

Use Certificate Manager with DNS authorization: you add a CNAME record that proves domain control, so Google-managed certificates, including wildcards, can be issued before the A record points at the load balancer. Put certificates in a certificate map with entries per hostname and attach the map to the target HTTPS proxy, which scales to many certificates. Load-balancer-authorized managed certs (the classic kind) only issue after DNS already points at the LB, which causes downtime during migrations. Self-managed certificates can also be uploaded when a specific CA is required.

KEY POINTS

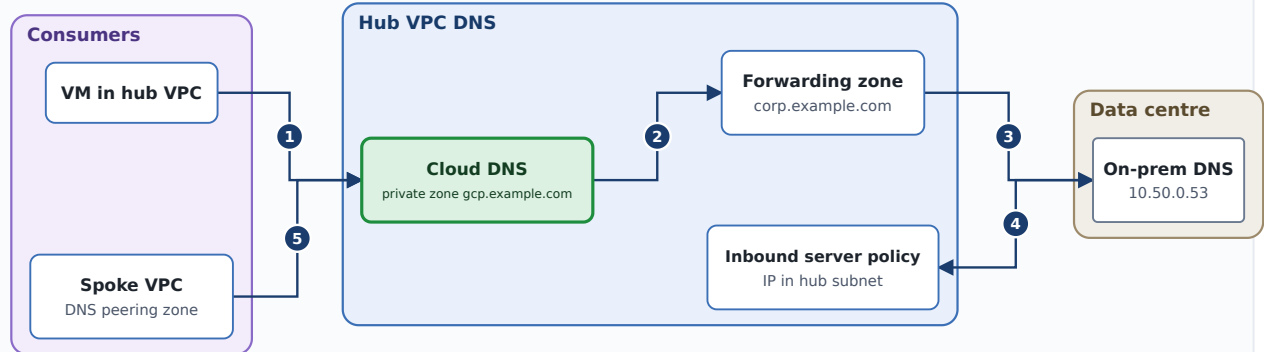
- DNS authorization allows wildcards and zero-downtime migration.
- Certificate maps scale far beyond classic per-proxy cert limits.

CONFIGURATION EXAMPLE

```
gcloud certificate-manager dns-authorizations create ex-auth --domain=example.com
gcloud certificate-manager certificates create ex-cert \
  --domains="*.example.com,example.com" --dns-authorizations=ex-auth
gcloud certificate-manager maps create web-map
gcloud certificate-manager maps entries create wild --map=web-map \
  --hostname="*.example.com" --certificates=ex-cert
gcloud compute target-https-proxies update web-proxy --certificate-map=web-map
```

Q73 GCP VMs must resolve corp.example.com hosted on-prem, on-prem servers must resolve gcp.example.com, and spoke VPCs should reuse the same setup. How do you design Cloud DNS?

FLOW DIAGRAM



- 1 Query db.corp.example.com
- 2 Matches the forwarding zone
- 3 Forwarded over VPN / Interconnect from 35.199.192.0/19
- 4 On-prem conditional forwarder sends gcp.example.com queries here
- 5 DNS peering lets spokes use the hub's zones and forwarding

EXPLANATION

Host GCP records in a private zone (gcp.example.com) visible to the hub VPC, create a forwarding zone for corp.example.com that sends queries to the on-prem resolvers, and add an inbound server policy so Cloud DNS gets an address in the hub subnet for on-prem conditional forwarders. Forwarded queries come from 35.199.192.0/19, so on-prem firewalls must allow it and Cloud Router must advertise a route back for that range. Spoke VPCs use DNS peering zones that point at the hub VPC, so there is one forwarding setup to maintain. Public zones are separate and can use DNSSEC.

KEY POINTS

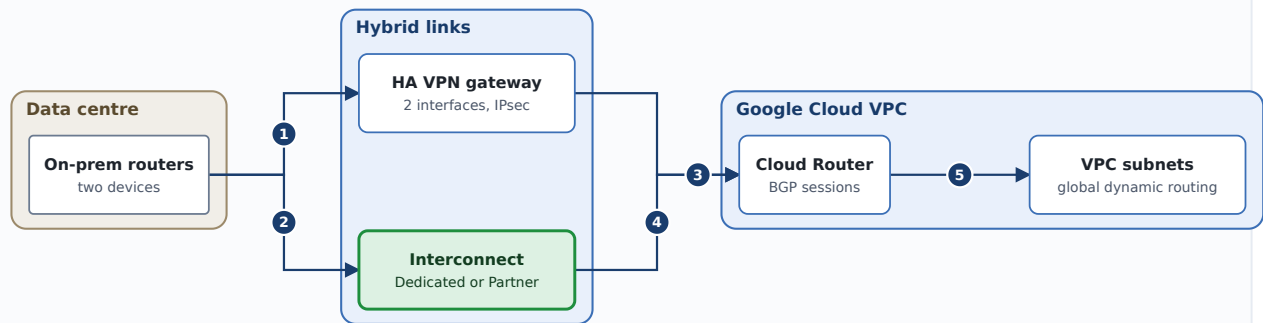
- Forwarding = send queries out; peering = reuse another VPC's resolution.
- Advertise 35.199.192.0/19 to on-prem for forwarding replies.

CONFIGURATION EXAMPLE

```
gcloud dns managed-zones create corp-fwd --dns-name=corp.example.com. \
  --visibility=private --networks=hub-net --forwarding-targets=10.50.0.53 \
  --description="Forward to on-prem"
gcloud dns policies create inbound-pol --networks=hub-net \
  --enable-inbound-forwarding --description="On-prem to GCP"
gcloud dns managed-zones create corp-peer --dns-name=corp.example.com. \
  --visibility=private --networks=spoke-net --target-network=hub-net \
  --target-project=hub-proj --description="Peer to hub"
```

Q74 A company needs private connectivity from its data centre: 1 Gbps now, 20 Gbps in a year, with a 99.99% SLA. Compare HA VPN, Partner and Dedicated Interconnect.

FLOW DIAGRAM



- ❶ Encrypted tunnels over the internet, about 3 Gbps each
- ❷ Private circuits: Dedicated 10/100 Gbps, Partner from 50 Mbps
- ❸ VLAN attachments peer with Cloud Router
- ❹ Learned routes are programmed into the VPC

EXPLANATION

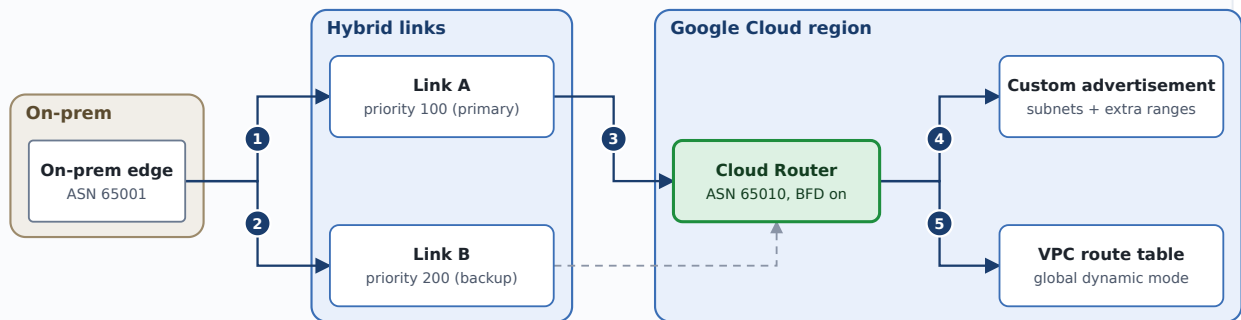
HA VPN is quick to set up, encrypted, and gives a 99.99% SLA with two tunnels to two peer interfaces, but each tunnel carries roughly up to 3 Gbps over the internet, so it fits the 1 Gbps start. Dedicated Interconnect gives 10 or 100 Gbps circuits in a colocation facility; Partner Interconnect goes through a service provider with VLAN attachments from 50 Mbps upward when you are not in a Google colocation site. For 99.99% on Interconnect, use four connections across two metros with Cloud Routers in two regions. A common path is HA VPN now, Interconnect later, with HA VPN over Interconnect or MACsec if encryption is required.

KEY POINTS

- All options use Cloud Router and BGP for dynamic routes.
- Interconnect egress is billed at lower rates than internet egress.

Q75 You have two HA VPN tunnels (or VLAN attachments) and want one to be primary and the other a fast-failover backup. How do you control this with Cloud Router and BGP?

FLOW DIAGRAM



- 1 On-prem prefers link A (MED / AS-path)
- 3 Both BGP sessions up; lower value is preferred
- 4 Advertise subnets plus ranges such as 35.199.192.0/19
- 5 Global dynamic routing shares routes with all regions
- Takes over when BFD detects link A down

EXPLANATION

Cloud Router sends a MED value to each BGP peer, taken from the advertised route priority you set on that peer. Give link A a lower value (100) and link B a higher one (200); on-prem then prefers A for traffic into GCP. For traffic from GCP to on-prem, the on-prem router must advertise its prefixes with a better (lower) MED or a shorter AS path on link A. Enable BFD (Bidirectional Forwarding Detection), a fast heartbeat, so failover takes seconds instead of waiting for the BGP hold timer. Use custom advertisement mode to announce extra ranges such as the private services access range or 199.36.153.8/30. Use global dynamic routing so routes learned in one region reach VMs in all regions.

KEY POINTS

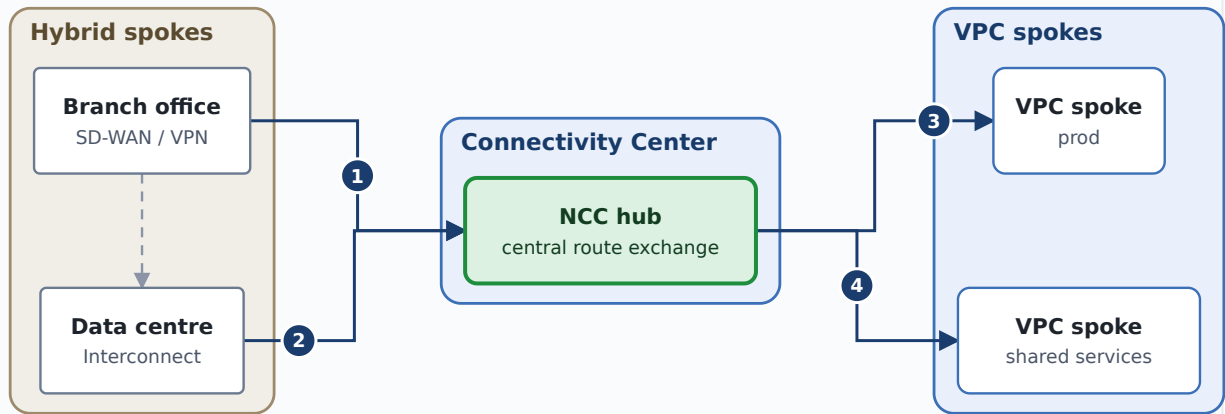
- Advertised route priority becomes MED to the peer.
- BFD gives sub-second to seconds failure detection.

CONFIGURATION EXAMPLE

```
gcloud compute routers update-bgp-peer hyb-rtr --region=us-central1 \
  --peer-name=peer-b --advertised-route-priority=200 \
  --bfd-session-initialization-mode=ACTIVE
gcloud compute routers update hyb-rtr --region=us-central1 \
  --advertisement-mode=custom --set-advertisement-groups=all_subnets \
  --set-advertisement-ranges=35.199.192.0/19
```

Q76 A company has 12 VPCs and several on-prem sites. Peering meshes are getting unmanageable. How does Network Connectivity Center help?

FLOW DIAGRAM



- ❶ VPN tunnels, VLAN attachments or router appliances become spokes
- ❸ VPC spokes exchange subnet routes through the hub
- ❹ Adding a VPC is one spoke, not N peerings
- Optional site-to-site data transfer over Google's backbone

EXPLANATION

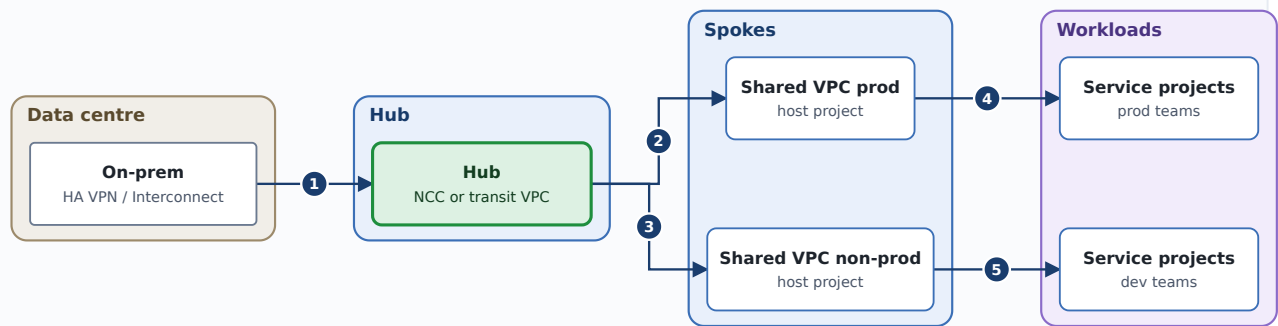
Network Connectivity Center is a hub-and-spoke control plane: VPCs attach as VPC spokes and hybrid links (HA VPN, Interconnect VLAN attachments, router appliance VMs) attach as hybrid spokes. VPC spokes get reachability to each other through the hub, which removes the non-transitive peering problem, and hybrid spokes can share routes with all VPC spokes. You can use a star topology so edge spokes only talk to a center group, and export filters to keep ranges private. Site-to-site data transfer between on-prem locations over Google's network is available in supported regions.

KEY POINTS

- Adding a VPC is one spoke, not a new peering per VPC.
- Check spoke and route limits for large estates.

Q77 Should an enterprise use Shared VPC or a hub-and-spoke model with separate VPCs? Can they be combined?

FLOW DIAGRAM



- ❶ One set of hybrid links and inspection
- ❷ Each environment is a spoke
- ❸ Keep prod and non-prod apart with a star topology or filters
- ❹ Teams deploy into central subnets

EXPLANATION

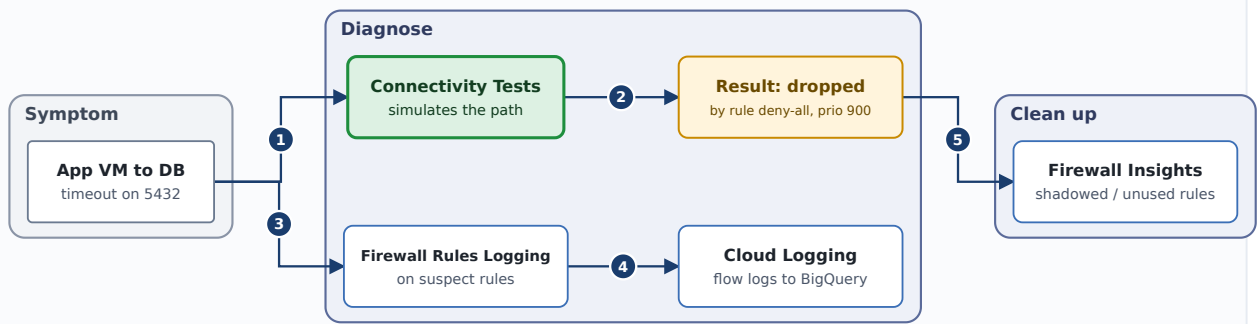
Shared VPC gives one network per environment that a central team controls, which is simple and keeps firewalling and IP planning in one place, but every team shares the same network and quotas. Hub-and-spoke gives each business unit or app its own VPC and autonomy, joined through NCC or a transit VPC with appliances, at the cost of more networks to manage. Most large organisations combine them: a hub for hybrid connectivity and shared services, with a Shared VPC host per environment as a spoke. Choose based on how much autonomy teams need, regulatory isolation, and VPC quotas.

KEY POINTS

- Shared VPC = central control; separate VPCs = autonomy.
- Combining both is the common landing-zone pattern.

Q78 An app VM times out connecting to a database on port 5432 in another subnet.
How do you find the cause with Google Cloud networking tools?

FLOW DIAGRAM



- ❶ Test source VM, destination IP, TCP 5432
- ❷ Config analysis names the blocking rule or missing route
- ❸ Enable logging on the rules involved
- ❹ Logs show rule name and allowed/denied connections
- ❺ Find rules that shadow the allow rule

EXPLANATION

Start with a Connectivity Test from the source VM to the database IP and port; it analyses routes, firewall rules and other configuration and reports where the packet would be dropped. Confirm with Firewall Rules Logging on the suspected rules and VPC Flow Logs on the subnets, which record sampled connections and can be exported to BigQuery. Firewall Insights in Network Intelligence Center finds shadowed, overly broad or unused rules. If the config is correct, check the guest OS firewall and whether the database listens on the right interface.

KEY POINTS

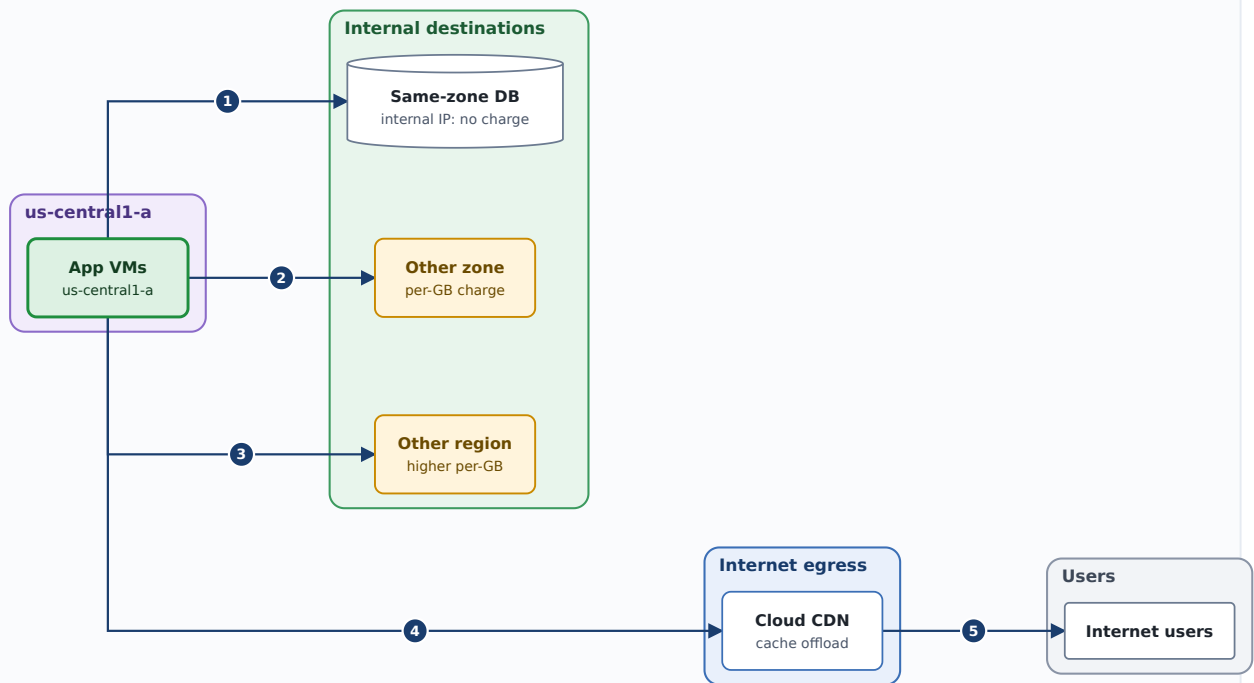
- Connectivity Tests do not send real packets; they analyse config.
- Flow logs are sampled; raise sampling while debugging.

CONFIGURATION EXAMPLE

```
gcloud network-management connectivity-tests create vm-to-db \
  --source-instance=projects/p/zones/us-central1-a/instances/app-1 \
  --destination-ip-address=10.60.0.3 --destination-port=5432 --protocol=TCP
gcloud compute firewall-rules update deny-all --enable-logging
gcloud compute networks subnets update app-subnet --region=us-central1 \
  --enable-flow-logs --logging-flow-sampling=0.5
```

Q79 The network bill is growing faster than compute. Which traffic patterns cost money on Google Cloud and how do you reduce them?

FLOW DIAGRAM



- ❶ Same zone over internal IPs is free
- ❷ Cross-zone traffic in a region is charged
- ❸ Inter-region cost depends on the continents involved
- ❹ Serve static content from the edge
- ❺ Internet egress; Standard tier is cheaper but regional

EXPLANATION

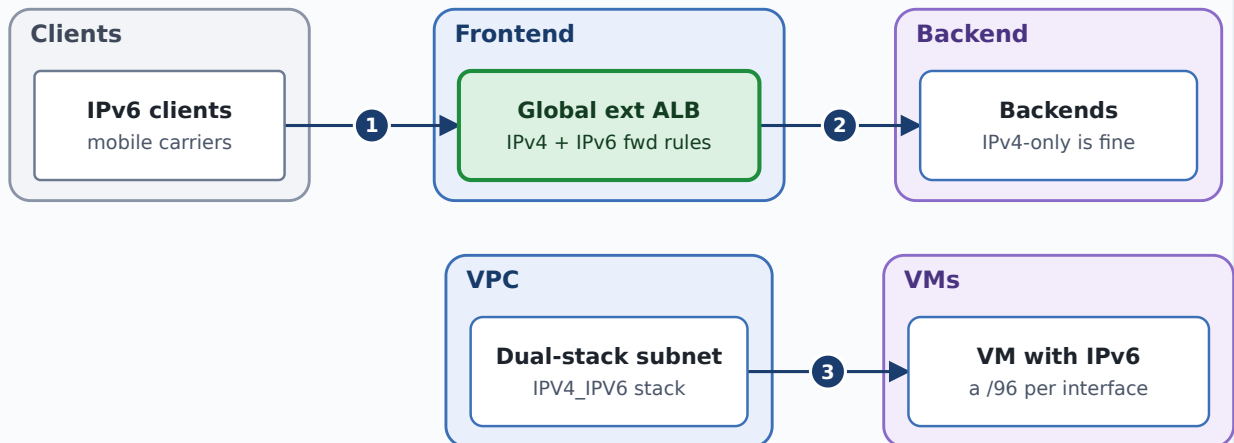
Ingress is free, but egress between zones, between regions and to the internet is charged per GB, with internet egress the most expensive. Reduce it by keeping chatty services in the same zone or region, using internal IPs, compressing payloads, and caching with Cloud CDN. Use Private Google Access or PSC for Google APIs instead of NAT, consider Interconnect for large hybrid volumes because its egress rates are lower, and use Standard network tier where performance allows. Find the top talkers with VPC Flow Logs and billing export to BigQuery grouped by SKU.

KEY POINTS

- Zonal HA has a cost: replicas across zones generate cross-zone traffic.
- Check current pricing; rates vary by region and tier.

Q80 A mobile app's users are increasingly on IPv6-only networks. How do you add IPv6 support on Google Cloud?

FLOW DIAGRAM



- 1 AAAA record points to an IPv6 anycast address
- 2 The proxy terminates IPv6 and connects over IPv4
- 3 Dual-stack VMs get IPv6 addresses from the subnet
- IPv6 needs its own firewall rules (for example `::/0`)

EXPLANATION

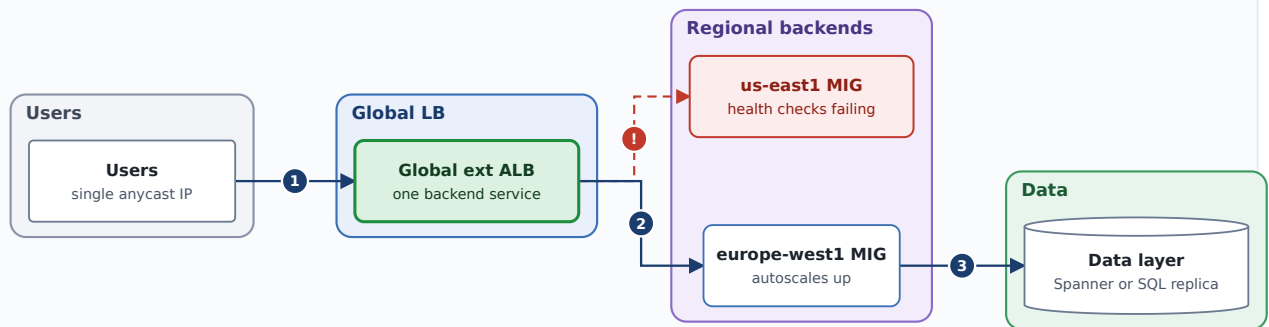
The quickest win is an IPv6 forwarding rule on the global external Application Load Balancer: clients connect over IPv6 and the proxy reaches IPv4-only backends, so the app does not change. For workloads that must use IPv6 end to end, create dual-stack subnets (external or internal IPv6; internal needs ULA enabled on the VPC) and dual-stack VMs, which get a /96 range. Firewall rules must be written for IPv6 ranges separately. IPv6-only subnets and VMs are also supported now; check current feature support per service.

KEY POINTS

- Add an AAAA record alongside the A record.
- IPv6 on the LB frontend needs no backend change.

Q81 An e-commerce site runs in us-east1 behind a global load balancer. The business wants automatic failover to europe-west1 if us-east1 fails. How do you design it?

FLOW DIAGRAM



- 1** DNS never changes; the anycast IP stays the same
- !** No healthy backends in the primary region
- 2** Traffic moves to the next closest healthy region
- 3** Writes go to the surviving writable copy

EXPLANATION

Put MIGs (or GKE NEG) in both regions behind one backend service on a global external Application Load Balancer with health checks. When us-east1 backends fail their health checks, the LB sends users to europe-west1 within seconds without DNS changes; you can also use capacity settings to keep Europe on standby or active. Europe must be able to absorb the full load, so set autoscaling limits and quotas for N+1 capacity. The data layer decides the real RTO and RPO: Spanner multi-region fails over automatically, while a Cloud SQL cross-region replica must be promoted.

KEY POINTS

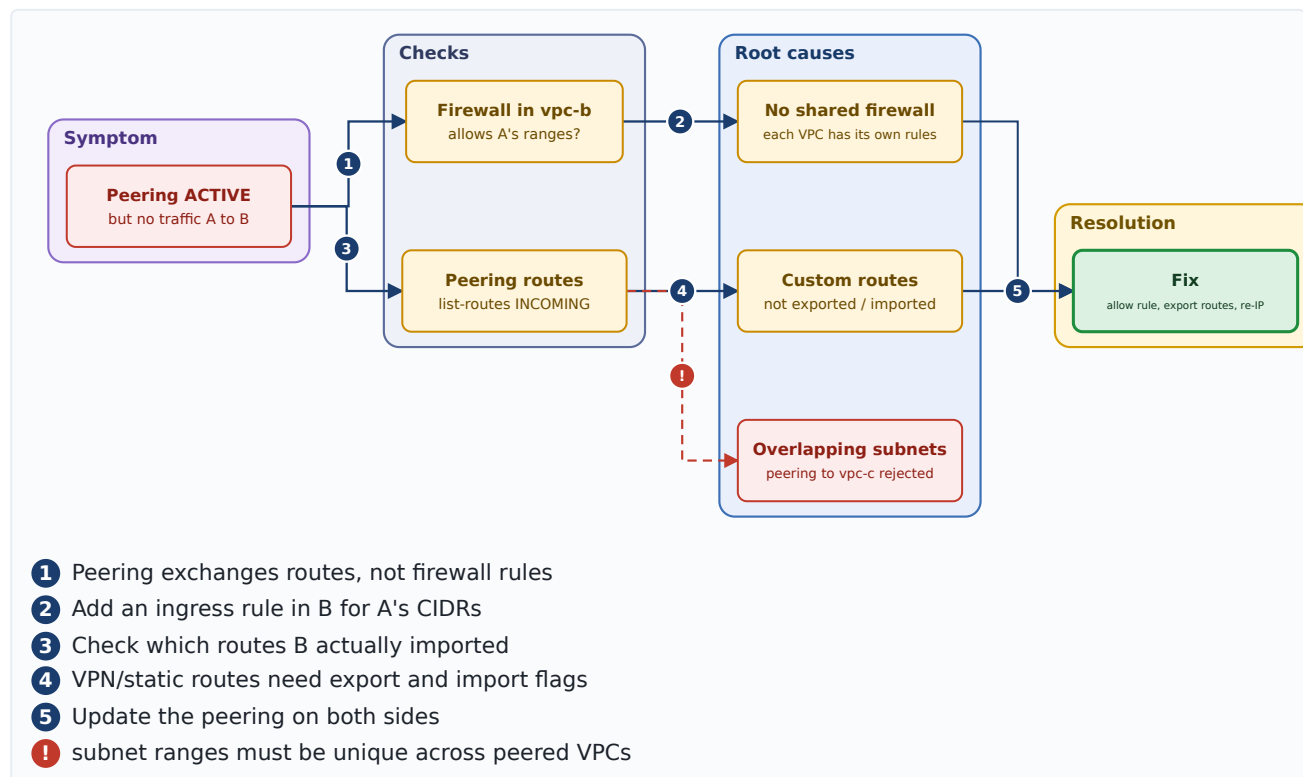
- The LB fails over compute; you must plan data failover.
- Test failover regularly by draining the primary region.

Q82

Troubleshooting

A VPC peering between `vpc-a` and `vpc-b` shows **ACTIVE**, but VMs in A time out reaching VMs in B, and on-prem (connected to A over VPN) cannot reach B at all. A second peering to `vpc-c` fails with 'An IP range in the peer network overlaps with an IP range in the local network'. How do you troubleshoot it?

FLOW DIAGRAM



EXPLANATION

Peering only exchanges subnet routes; each VPC keeps its own firewall, so `vpc-b` needs an ingress rule that allows A's subnet ranges. Routes learned from VPN or static routes in A are not shared by default: A must export custom routes and B must import them, otherwise on-prem traffic has no return path. Use `peerings list-routes` on each side to see what was really exchanged. The `vpc-c` error means a subnet range overlaps, which peering never allows; the only fixes are re-addressing a subnet or using Private Service Connect or NCC instead.

KEY POINTS

- Keep a central IP plan (IPAM) so no two VPCs that may ever peer share a range.
- Peering is not transitive: A to B to C does not work without a hub design.

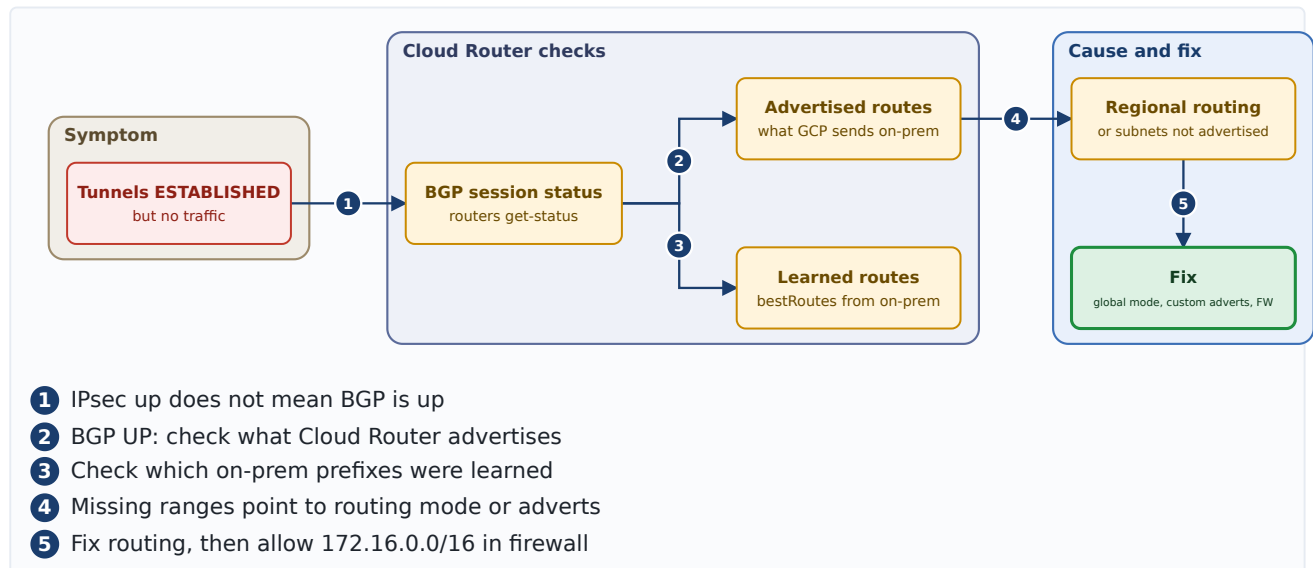
CONFIGURATION EXAMPLE

```

gcloud compute networks peerings list --network=vpc-a
gcloud compute networks peerings list-routes peer-a-b --network=vpc-b \
  --region=us-central1 --direction=INCOMING
gcloud compute networks peerings update peer-a-b --network=vpc-a --export-custom-routes
gcloud compute networks peerings update peer-b-a --network=vpc-b --import-custom-routes
gcloud compute firewall-rules create allow-from-vpc-a --network=vpc-b \
  --direction=INGRESS --rules=tcp:443,tcp:5432 --source-ranges=10.10.0.0/16
  
```


Q83**Troubleshooting**

Both HA VPN tunnels to the data centre show `ESTABLISHED`, but no traffic passes between on-prem `172.16.0.0/16` and the GCP subnets in `us-central1` and `eu-west1`. How do you troubleshoot it?

FLOW DIAGRAM**EXPLANATION**

An `ESTABLISHED` tunnel only proves IPsec is up; routes come from BGP, so first check `bgpPeerStatus` in `routers get-status` (a `DOWN` session usually means wrong `169.254.x.x` link-local IPs or ASN on the peer). If BGP is up, look at `advertisedRoutes`: with regional dynamic routing, the Cloud Router in `us-central1` only advertises and learns routes for its own region, so `eu-west1` is invisible until the VPC uses global routing mode. Ranges such as peered VPCs or Private Service Access are not advertised by default and need custom advertisements. Finally, a VPC firewall rule must allow the on-prem range and the on-prem firewall must allow the GCP ranges.

KEY POINTS

- Alert on the Cloud Router metric for BGP session status, not only on tunnel status.
- Document which ranges are advertised in each direction and keep them in Terraform.

CONFIGURATION EXAMPLE

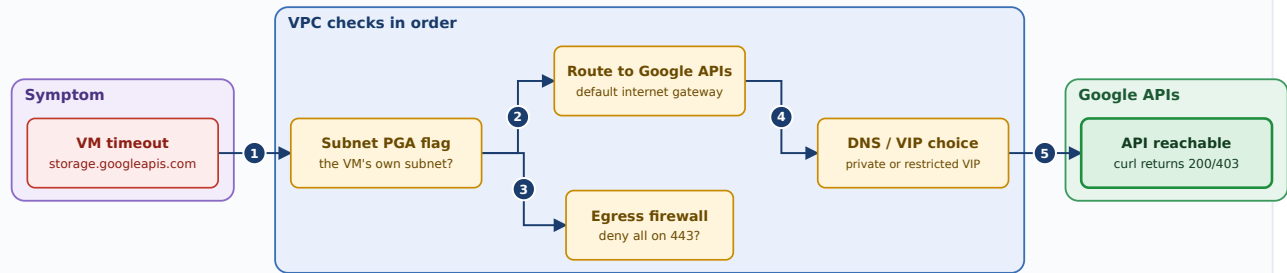
```

gcloud compute vpn-tunnels describe tun-0 --region=us-central1 \
  --format='value(status,detailedStatus)'
gcloud compute routers get-status cr-vpn --region=us-central1 \
  --format='yaml(result.bgpPeerStatus,result.bestRoutes)'
gcloud compute networks update prod-vpc --bgp-routing-mode=global
gcloud compute routers update cr-vpn --region=us-central1 --advertisement-mode=custom \
  --set-advertisement-groups=all_subnets --set-advertisement-ranges=10.200.0.0/20

```

Private Google Access is 'enabled', yet a VM without an external IP still times out on `gsutil ls gs://reports` and `curl https://storage.googleapis.com`. How do you find what is missing?

FLOW DIAGRAM



- 1 PGA is per subnet; check the VM's NIC subnet
- 2 PGA needs a route whose next hop is the internet gateway
- 3 An egress deny to 0.0.0.0/0 also blocks Google APIs
- 4 Custom DNS must map *.googleapis.com to the right VIP
- 5 Test again from the VM

EXPLANATION

Private Google Access is a per-subnet setting, so first confirm `privateIpGoogleAccess` is true on the subnet the VM's interface is actually in. Traffic to Google APIs still needs a route with next hop `default-internet-gateway`, either the default 0.0.0.0/0 route or a route for 199.36.153.8/30 (`private.googleapis.com`) or 199.36.153.4/30 (`restricted.googleapis.com`); teams that deleted the default route for security often forget this. An egress deny-all firewall rule also blocks the traffic, so allow 443 to those ranges. If you use VPC Service Controls, a private DNS zone must CNAME `*.googleapis.com` to `restricted.googleapis.com`, otherwise requests go to public IPs that have no route.

KEY POINTS

- Create the Google APIs route and DNS zone together in Terraform whenever you remove the default route.
- Use Connectivity Tests to `storage.googleapis.com` to see which step drops the packet.

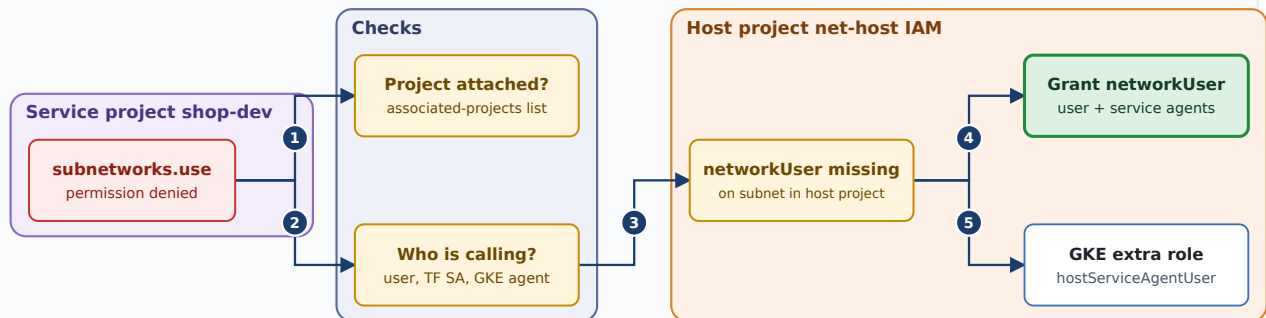
CONFIGURATION EXAMPLE

```

gcloud compute networks subnets describe app-subnet --region=us-central1 \
  --format='value(privateIpGoogleAccess)'
gcloud compute networks subnets update app-subnet --region=us-central1 \
  --enable-private-ip-google-access
gcloud compute routes list --filter="network=prod-vpc AND nextHopGateway=default-internet-gateway"
gcloud compute routes create google-private-apis --network=prod-vpc \
  --destination-range=199.36.153.8/30 --next-hop-gateway=default-internet-gateway
dig +short storage.googleapis.com # run on the VM to see which VIP it uses
  
```

Q85**Troubleshooting**

Deploying a GKE cluster in service project `shop-dev` on a Shared VPC subnet fails with Required 'compute.subnetworks.use' permission for 'projects/net-host/regions/us-central1/subnetworks/gke-subnet'. How do you fix it?

FLOW DIAGRAM

- 1 The service project must be attached to the host
- 2 Find the principal in the error or audit log
- 3 Grants live in the host project, not shop-dev
- 4 Grant on the subnet (or host project)
- 5 GKE service agent also needs this on the host

EXPLANATION

First check that `shop-dev` is attached to the host project; an unattached project cannot use any shared subnet. Then find the principal that is denied: it can be the user or Terraform service account, but for GKE it is often a service agent, because GKE creates resources using `service-PROJECT_NUMBER@container-engine-robot.iam.gserviceaccount.com` and the Google APIs agent `PROJECT_NUMBER@cloudservices.gserviceaccount.com`. Each needs `roles/compute.networkUser` on the subnet (or host project) in the host project, and the GKE agent also needs `roles/container.hostServiceAgentUser` on the host project. For GKE also check the subnet has the secondary ranges for pods and services.

KEY POINTS

- Grant `networkUser` per subnet, not per host project, to keep teams in their own subnets.
- Keep Shared VPC IAM grants for service agents in the same Terraform module that attaches the project.

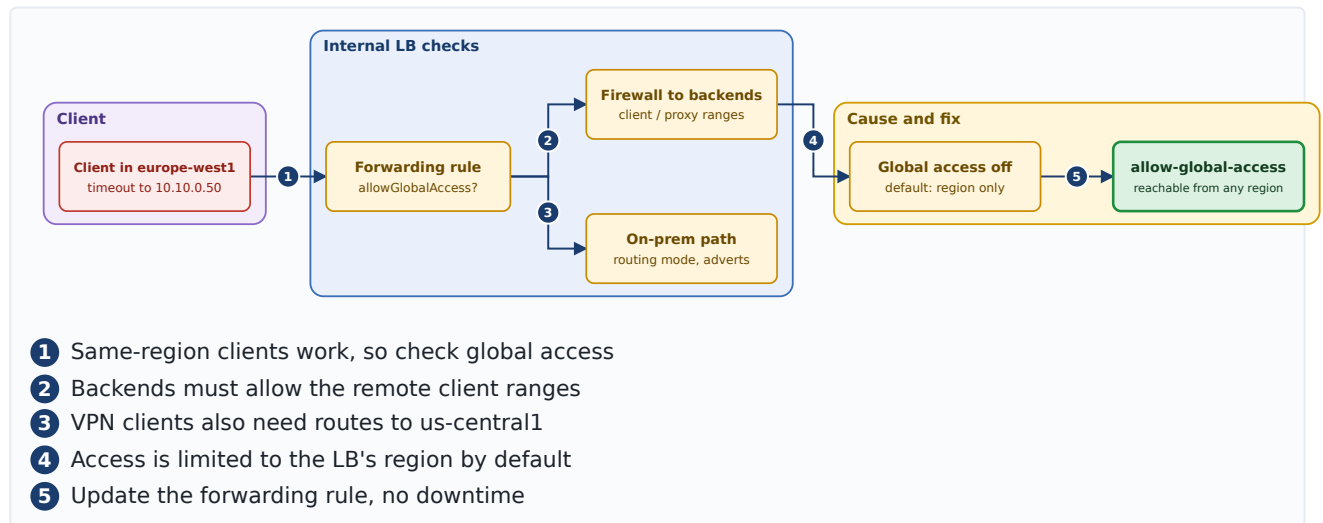
CONFIGURATION EXAMPLE

```

gcloud compute shared-vpc associated-projects list net-host
gcloud compute shared-vpc associated-projects add shop-dev --host-project=net-host
gcloud compute networks subnets add-iam-policy-binding gke-subnet --project=net-host \
  --region=us-central1 --role=roles/compute.networkUser \
  --member=serviceAccount:service-123456789@container-engine-robot.iam.gserviceaccount.com
gcloud projects add-iam-policy-binding net-host --role=roles/container.hostServiceAgentUser \
  --member=serviceAccount:service-123456789@container-engine-robot.iam.gserviceaccount.com
  
```

Q86**Troubleshooting**

An internal load balancer at 10.10.0.50 in us-central1 works for clients in the same region, but VMs in europe-west1 and on-prem users coming through a VPN in europe-west1 time out. How do you fix it?

FLOW DIAGRAM**EXPLANATION**

Internal passthrough and regional internal Application Load Balancers only accept clients from their own region unless global access is enabled on the forwarding rule, so remote clients simply time out. Enable it with `--allow-global-access`; it can be changed on an existing rule without recreating it. Check that the backend firewall allows the remote client ranges (passthrough LBs keep the client IP) or the proxy-only subnet (Application LBs). For on-prem clients entering through a Cloud Router in europe-west1, the VPC also needs global dynamic routing so the router advertises the us-central1 subnet.

KEY POINTS

- Decide on global access when you create each internal LB and set it in Terraform.
- Cross-region traffic is charged as inter-region egress; consider a local backend for heavy users.

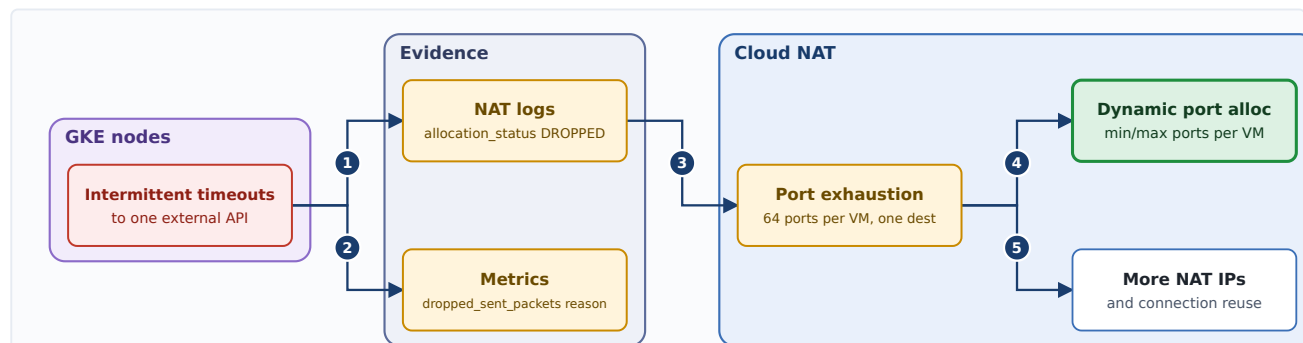
CONFIGURATION EXAMPLE

```

gcloud compute forwarding-rules describe ilb-api-fr --region=us-central1 \
  --format='value(IPAddress,allowGlobalAccess)'
gcloud compute forwarding-rules update ilb-api-fr --region=us-central1 --allow-global-access
gcloud compute networks describe prod-vpc --format='value(routingConfig.routingMode)'
gcloud compute firewall-rules create allow-eu-to-api --network=prod-vpc --direction=INGRESS \
  --rules=tcp:8443 --source-ranges=10.20.0.0/16,172.16.0.0/16 --target-tags=api
  
```

Q87**Troubleshooting**

During peak hours, GKE pods calling a payment provider's API fail with intermittent connection timed out, and NAT logs show allocation_status: DROPPED with reason OUT_OF_RESOURCES. How do you troubleshoot and fix it?

FLOW DIAGRAM

- 1 Enable NAT logging for errors
- 2 Check which nodes drop packets and why
- 3 Many connections to the same IP:port share few ports
- 4 Give busy nodes more ports on demand
- 5 Add IPs and use keep-alive pooling in the app

EXPLANATION

Each NAT IP has about 64,000 source ports, and by default each VM (GKE node) gets 64 of them per destination IP and port, so a node with many pods opening new connections to one API runs out quickly. Confirm with NAT logs (`allocation_status="DROPPED"`) and the `nat/dropped_sent_packets_count` metric with reason `OUT_OF_RESOURCES`. Fix it by enabling dynamic port allocation with a higher `max-ports-per-vm`, raising `min-ports-per-vm`, and adding NAT IPs so there are enough ports for all nodes. In the app, reuse HTTP connections (keep-alive, pooling) instead of opening one per request, and consider a shorter `TCP TIME_WAIT` timeout on the NAT.

KEY POINTS

- Size NAT as $\text{nodes} \times \text{max ports per VM} \leq \text{NAT IPs} \times 64,512$, and alert on dropped packets.
- Tell the payment provider about new NAT IPs before adding them to their allowlist.

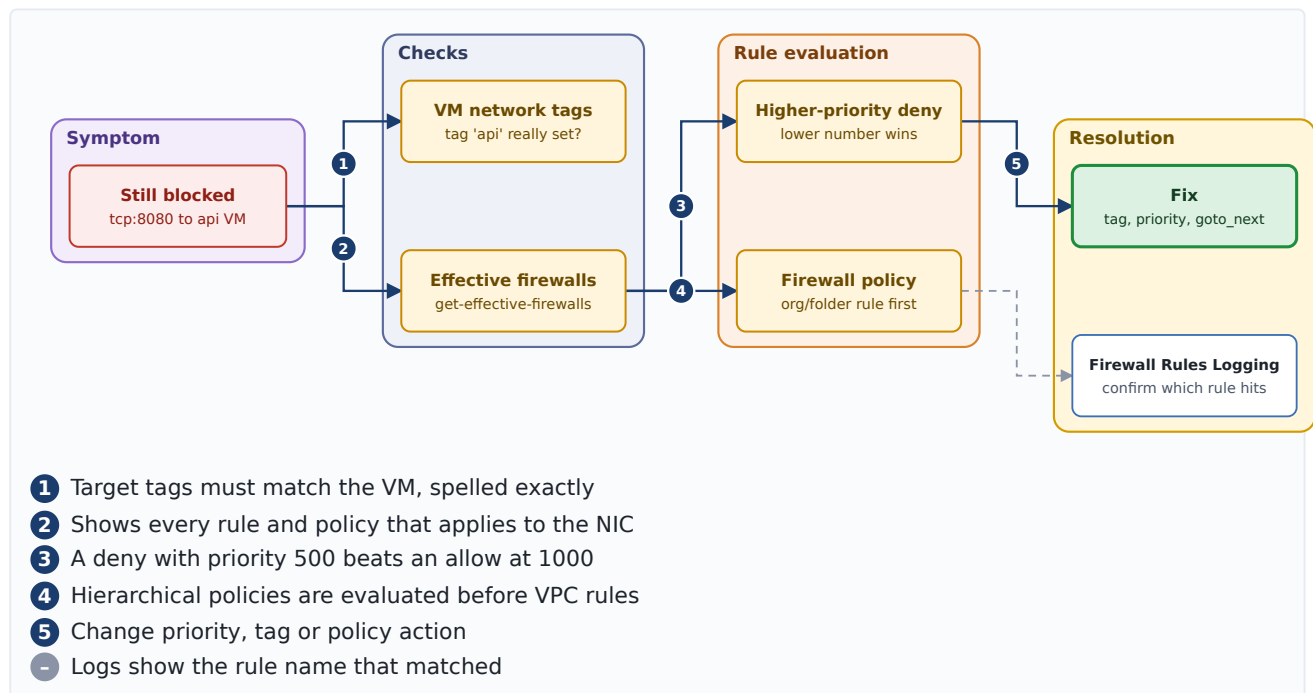
CONFIGURATION EXAMPLE

```

gcloud logging read 'resource.type="nat_gateway" AND jsonPayload.allocation_status="DROPPED"' \
  --limit=20 --freshness=1h
gcloud compute routers nats update nat-cfg --router=nat-router --region=us-central1 \
  --enable-dynamic-port-allocation --no-enable-endpoint-independent-mapping \
  --min-ports-per-vm=256 --max-ports-per-vm=4096 --tcp-time-wait-timeout=30s
gcloud compute routers nats update nat-cfg --router=nat-router --region=us-central1 \
  --nat-external-ip-pool=nat-ip-1,nat-ip-2,nat-ip-3
  
```

Q88**Troubleshooting**

You created `allow-api-8080` (ingress, tcp:8080, target tag `api`, source `10.1.0.0/16`), but connections from the frontend subnet are still dropped. How do you find out why the rule has no effect?

FLOW DIAGRAM**EXPLANATION**

First check the VM really has the `api` tag and that the rule is in the VM's network (a common mistake is a rule in another VPC or a tag typo). Then run `get-effective-firewalls` on the VM's interface: it lists hierarchical firewall policy rules, network firewall policies and VPC rules that apply. VPC rules are evaluated by priority, lowest number first, so a deny at priority 500 wins over your allow at 1000; organization or folder policies are evaluated before VPC rules, and a `deny` there cannot be overridden lower down, only `goto_next` passes the decision on. Enable Firewall Rules Logging on the suspected rules to see which one matches the dropped connection.

KEY POINTS

- Prefer service-account targets over tags; tags can be added by anyone who can edit the VM.
- Keep deny rules in a documented priority band (e.g. 100-499) so teams know what wins.

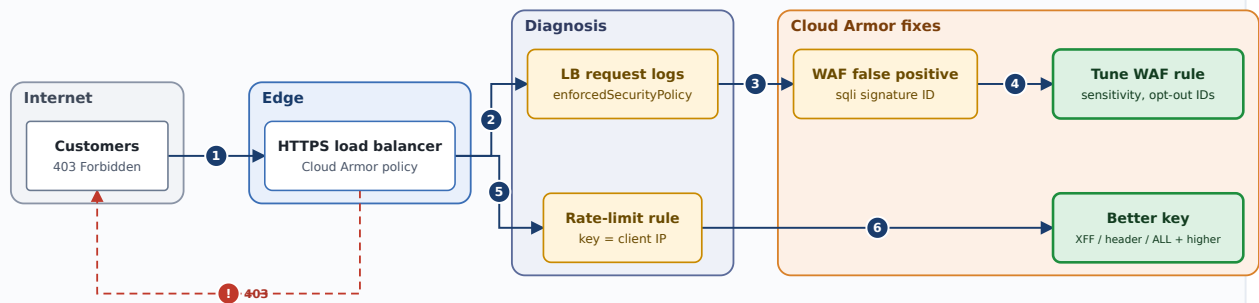
CONFIGURATION EXAMPLE

```

gcloud compute instances describe api-1 --zone=us-central1-a --format='value(tags.items)'
gcloud compute instances network-interfaces get-effective-firewalls api-1 \
  --zone=us-central1-a --network-interface=nic0
gcloud compute instances add-tags api-1 --zone=us-central1-a --tags=api
gcloud compute firewall-rules update allow-api-8080 --priority=400 --enable-logging
  
```

Q89**Troubleshooting**

After enabling Cloud Armor's preconfigured SQL injection rule, some customers get 403 Forbidden when saving addresses with apostrophes, and a whole office behind one NAT IP is being rate-limited. How do you fix it without removing protection?

FLOW DIAGRAM

- 1 Requests are denied at the edge
- ! 403 returned before the backend sees it
- 2 Logs name the rule and preconfigured expression ID
- 3 The apostrophe matches an SQLi signature
- 4 Lower sensitivity or exclude that signature
- 5 Shared NAT IP counts as one client
- 6 Throttle per user key and raise the threshold

EXPLANATION

Look in the load balancer logs for `jsonPayload.enforcedSecurityPolicy`: it shows the rule priority, outcome DENY and `preconfiguredExprIds`, which name the exact WAF signature that matched. Put that rule in preview mode first so users are unblocked while logs still show matches, then lower the sensitivity level or opt out of the specific noisy signature IDs instead of dropping the whole rule. For the office, a rate limit keyed on client IP treats everyone behind one NAT as a single user; use a key such as an HTTP header, cookie or `XFF-IP`, or raise the threshold, and exclude known partner ranges with a higher-priority allow rule. Always roll out new WAF rules in preview and review the logs for a week.

KEY POINTS

- Deploy every new Cloud Armor rule with `--preview` and review matches before enforcing.
- Return a custom error page so support can tell a Cloud Armor block from an app error.

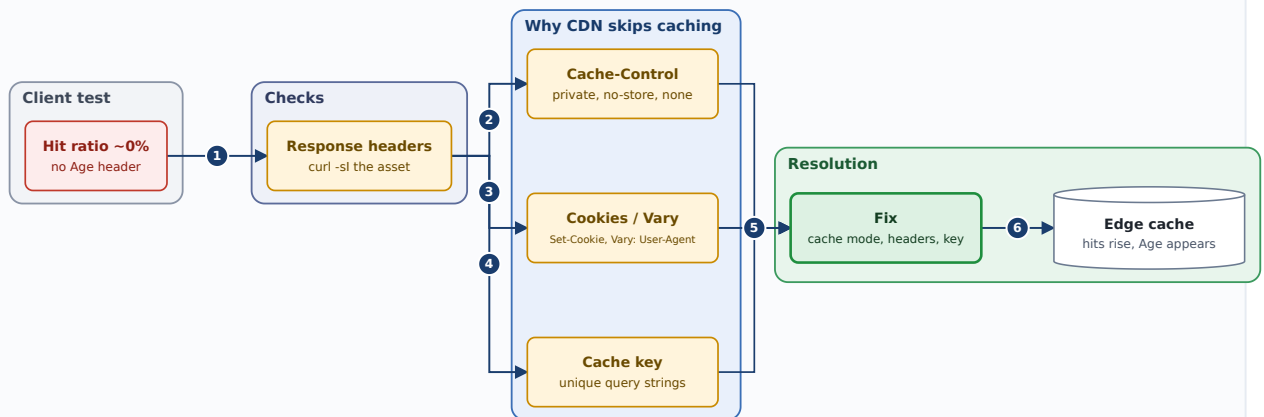
CONFIGURATION EXAMPLE

```

gcloud logging read 'resource.type="http_load_balancer" AND
  jsonPayload.enforcedSecurityPolicy.outcome="DENY" --limit=20 \
  --format='value(jsonPayload.enforcedSecurityPolicy.preconfiguredExprIds)'
gcloud compute security-policies rules update 1000 --security-policy=edge-policy --preview
gcloud compute security-policies rules update 1000 --security-policy=edge-policy \
  --expression="evaluatePreconfiguredWaf('sqli-v33-stable', {'sensitivity': 1})" --no-preview
  
```

Q90**Troubleshooting**

Cloud CDN is enabled on the backend for `static.example.com`, but the cache hit ratio is near 0%, responses have no `Age` header, and origin load has not dropped. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Inspect what the origin actually returns
- 2 USE_ORIGIN_HEADERS obeys the origin
- 3 These headers make responses uncacheable
- 4 ?v=timestamp creates a new cache entry each time
- 5 Set cache mode and headers at origin or backend
- 6 Verify with a second curl: Age > 0

EXPLANATION

Start with `curl -sI` on an asset twice: if the second response has no `Age` header, the edge did not serve it from cache. The usual cause is the origin sending `Cache-Control: private, no-store` or no caching header at all while the backend's cache mode is `USE_ORIGIN_HEADERS`. Responses with `Set-Cookie` or with `Vary` on headers such as `User-Agent` are also not cached, and query strings like `?ts=...` split the cache into many entries. Fix it by sending `Cache-Control: public, max-age=3600` for static files (or switching to `CACHE_ALL_STATIC`), stopping cookies on static paths, and removing unneeded query parameters from the cache key.

KEY POINTS

- Use versioned file names (`app.4f2a.js`) with long TTLs instead of cache-busting query strings.
- Watch the LB log field `jsonPayload.cacheHit` / cache hit ratio after each release.

CONFIGURATION EXAMPLE

```

curl -sI https://static.example.com/js/app.js | grep -iE 'cache-control|age|set-cookie|vary'
gcloud compute backend-services describe static-be --global --format='yaml(cdnPolicy)'
gcloud compute backend-services update static-be --global --enable-cdn \
  --cache-mode=CACHE_ALL_STATIC --default-ttl=3600 --no-cache-key-include-query-string
gcloud storage objects update 'gs://static-site/js/**' \
  --cache-control='public, max-age=86400'

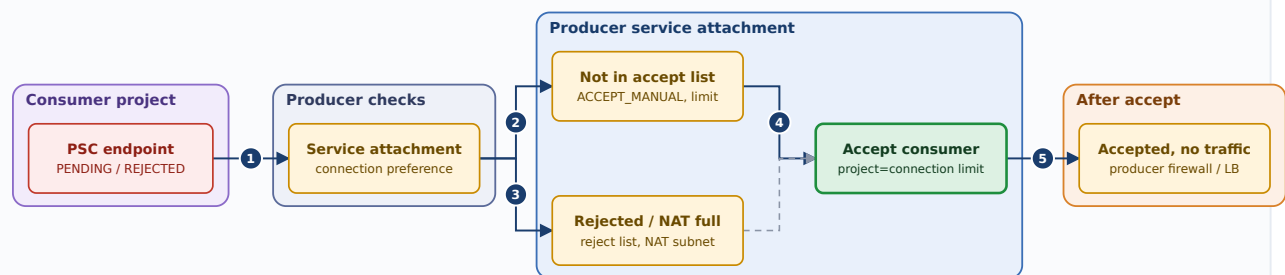
```


Q91

Troubleshooting

A consumer created a Private Service Connect endpoint for a partner's API, but `pscConnectionStatus` stays `PENDING` (and in another project shows `REJECTED`). Once accepted, requests still time out. How do you troubleshoot it?

FLOW DIAGRAM



- 1 Producer reviews the attachment's connected endpoints
- 2 Manual approval needs the consumer project listed
- 3 Reject list or no free PSC NAT addresses
- 4 Add the project with a connection limit
- Or remove from reject list / enlarge the PSC NAT subnet
- 5 Allow the PSC NAT subnet to the backends

EXPLANATION

`PENDING` almost always means the producer's service attachment uses `ACCEPT_MANUAL` and the consumer project (or network) is not in the accept list, or its connection limit is already used; `REJECTED` means it is on the reject list or was explicitly refused. The producer fixes it with `service-attachments update --consumer-accept-list=PROJECT=LIMIT`, and the endpoint moves to `ACCEPTED` without being recreated. Also check the endpoint is in the same region as the service attachment and that the producer's PSC NAT subnet has free addresses. If it is accepted but times out, the producer's firewall must allow the PSC NAT subnet to the load balancer backends, and the backends must be healthy.

KEY POINTS

- Producers should document the accept process and alert on pending connections.
- Size the PSC NAT subnet for the expected number of consumer connections.

CONFIGURATION EXAMPLE

```

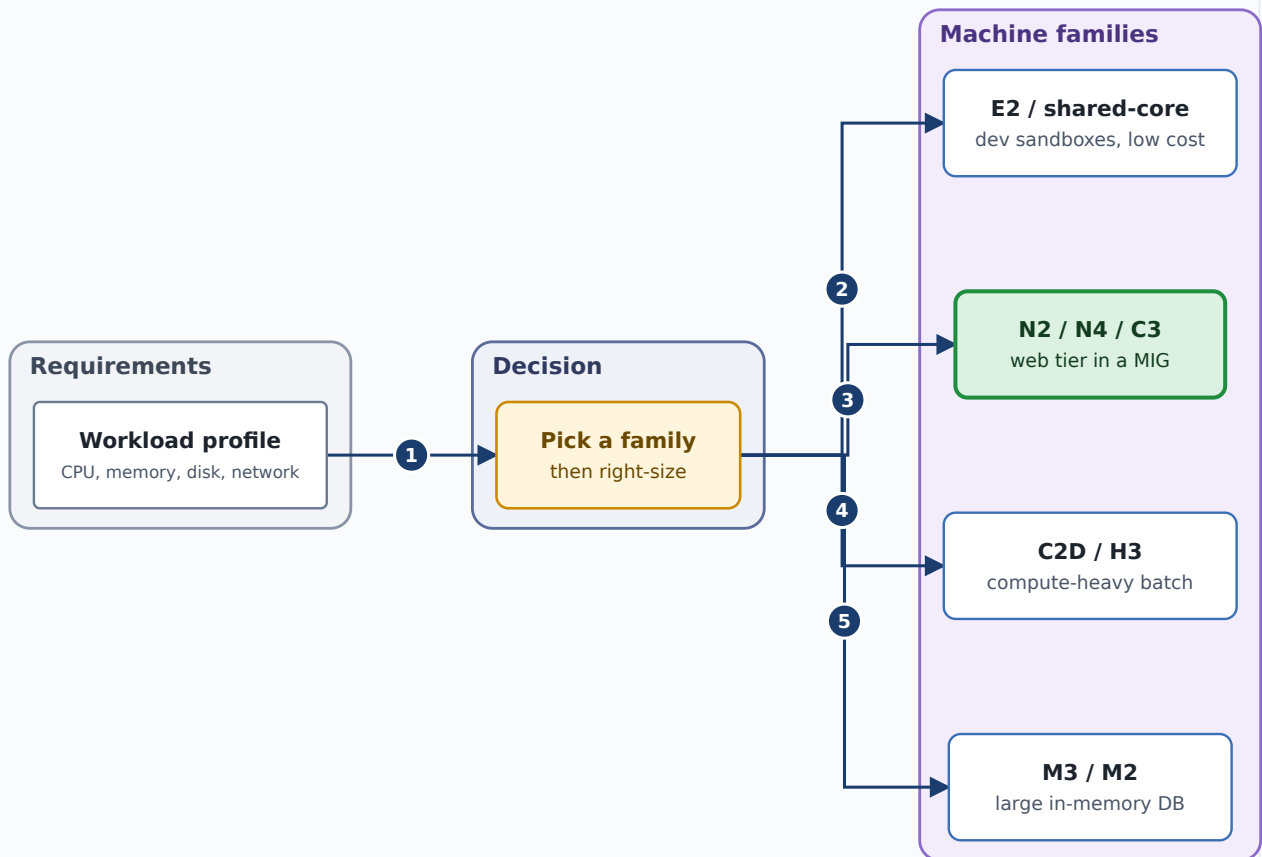
gcloud compute forwarding-rules describe partner-api-ep --region=us-central1 \
  --format='value(pscConnectionStatus,target)'
gcloud compute service-attachments describe api-attach --region=us-central1 \
  --project=partner-prod --format='yaml(connectionPreference,consumerAcceptLists,connectedEndpoints)'
gcloud compute service-attachments update api-attach --region=us-central1 \
  --project=partner-prod --consumer-accept-list=customer-prod=10
gcloud compute firewall-rules create allow-psc-nat --project=partner-prod --network=partner-vpc \
  --direction=INGRESS --rules=tcp:443 --source-ranges=10.99.0.0/24
  
```

4. Compute Engine & Managed Instance Groups

Choosing and building VMs, running them in self-healing groups, keeping them patched and backed up, and moving workloads in from on-prem.

Q92 You are sizing VMs for four workloads: a web tier, a nightly compute-heavy batch, an in-memory SAP HANA database and developer sandboxes. How do you pick machine families and sizes?

FLOW DIAGRAM



- ① Measure or estimate peak vCPU, RAM and IOPS
- ② Burstable, cheapest per vCPU, no SUD
- ③ General purpose, balanced price and performance
- ④ Highest per-core performance for CPU-bound work
- ⑤ Terabytes of memory, certified for SAP
- After a week of real load, apply machine-type recommendations from the Recommender.

EXPLANATION

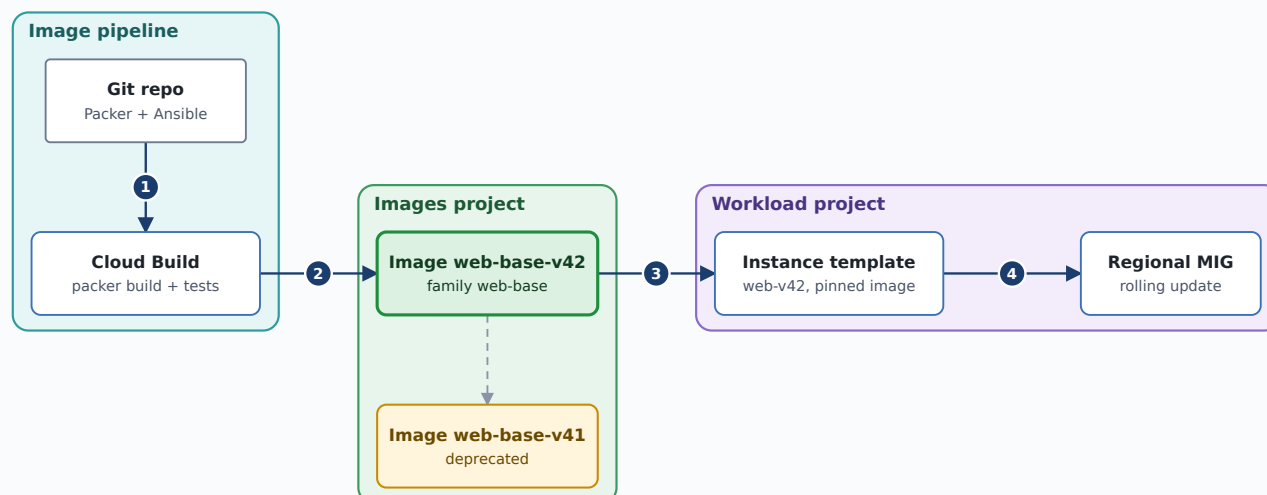
Start from the workload's CPU-to-memory ratio and performance needs. General-purpose families (E2 for cost, N2/N2D/N4 and C3/C4 for balanced or higher performance) fit most web and app tiers; compute-optimized (C2D, H3) suit CPU-bound batch and HPC; memory-optimized (M2, M3) are for very large in-memory databases like SAP HANA; accelerator-optimized (A3, G2) carry GPUs. Use custom machine types on N and E families when predefined shapes waste memory or CPU. Check that the family exists in your target zones, remember that disk and network limits scale with vCPU count, and revisit sizes using the Recommender's machine-type recommendations once real usage data exists.

KEY POINTS

- Right-size before committing to CUDs.
- Per-VM disk IOPS and bandwidth caps depend on machine type, not only on the disk.

Q93 Every new VM runs a 20-minute bootstrap script, so scale-out is slow and servers drift apart. How do you build golden images with Packer and roll them out through instance templates?

FLOW DIAGRAM



- 1 Merge triggers an image build
- 2 Temporary VM configured, captured, then tested
- 3 New template references the exact image
- 4 Rolling update replaces VMs with the new template
- Previous image deprecated, deleted once no template uses it

EXPLANATION

Bake OS hardening, agents and the application runtime into an image with Packer's `googlecompute` builder, run from Cloud Build on every change, and publish it into an image family such as `web-base` in a central images project; share it with `roles/compute.imageUser`. Instance templates are immutable, so create a new template per release that points at the specific image (pin the image rather than the family, so every VM in a release is identical), and roll the MIG to it. Leave only environment-specific settings for a short startup script. Deprecate old images and delete them once no template references them.

KEY POINTS

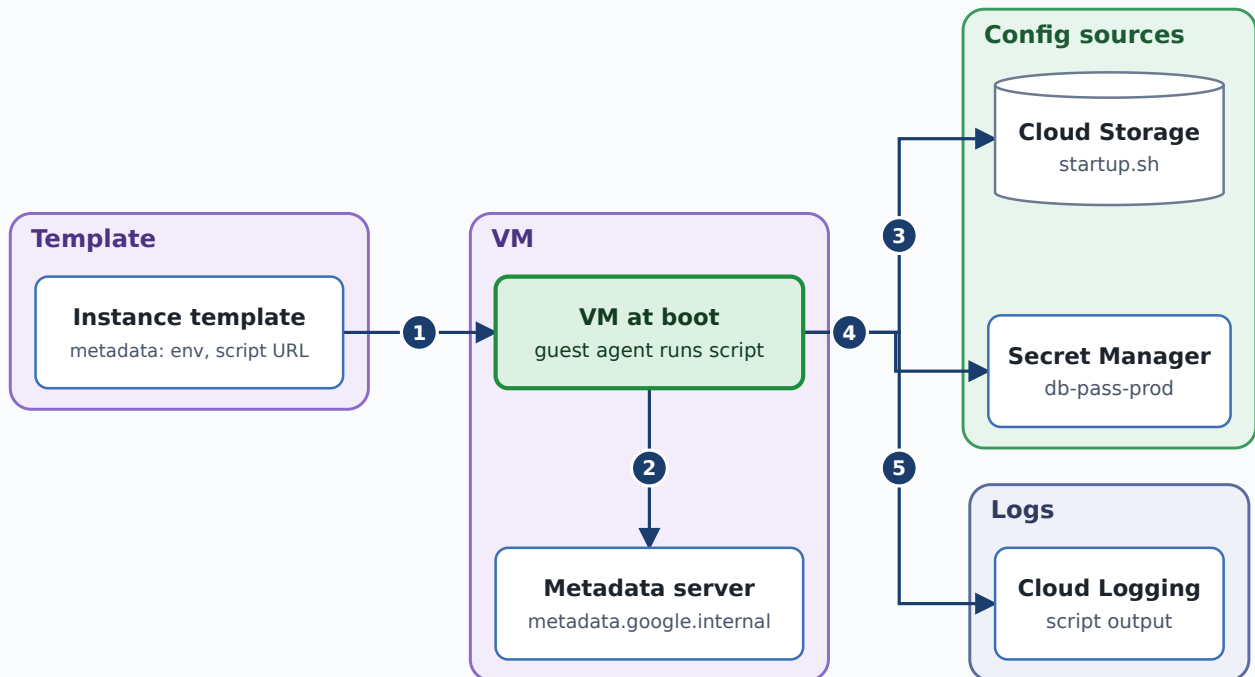
- Faster boot means the autoscaler can react to spikes.
- Images are global resources; use storage locations to meet residency needs.

CONFIGURATION EXAMPLE

```
source "googlecompute" "web" {
  project_id      = "img-factory"
  source_image_family = "debian-12"
  zone           = "europe-west1-b"
  image_name      = "web-base-v42"
  image_family    = "web-base"
  ssh_username    = "packer"
}
# gcloud compute instance-templates create web-v42 --machine-type=n2-standard-4 \
#   --image=web-base-v42 --image-project=img-factory
```

Q94 VMs need environment-specific settings and a database password at boot. How do you use startup scripts and metadata safely?

FLOW DIAGRAM



- 1 VM created with metadata key-values
- 2 Read attributes with header Metadata-Flavor: Google
- 3 Download the script from startup-script-url
- 4 Fetch secrets using the VM's service account
- 5 Output goes to the serial console and logs

EXPLANATION

Put non-secret settings (environment name, feature flags) in instance or project metadata and read them from the metadata server at `http://metadata.google.internal/computeMetadata/v1/` with the `Metadata-Flavor: Google` header. Use `startup-script` or `startup-script-url` (a Cloud Storage object) for boot logic; it runs as root on every boot, so make it idempotent, and check its output in the serial console or the `google_metadata_script_runner` logs. Never put passwords in metadata, since anyone who can view the instance can read it; fetch secrets from Secret Manager with the VM's attached service account instead. Keep the script short and move heavy installs into the image.

KEY POINTS

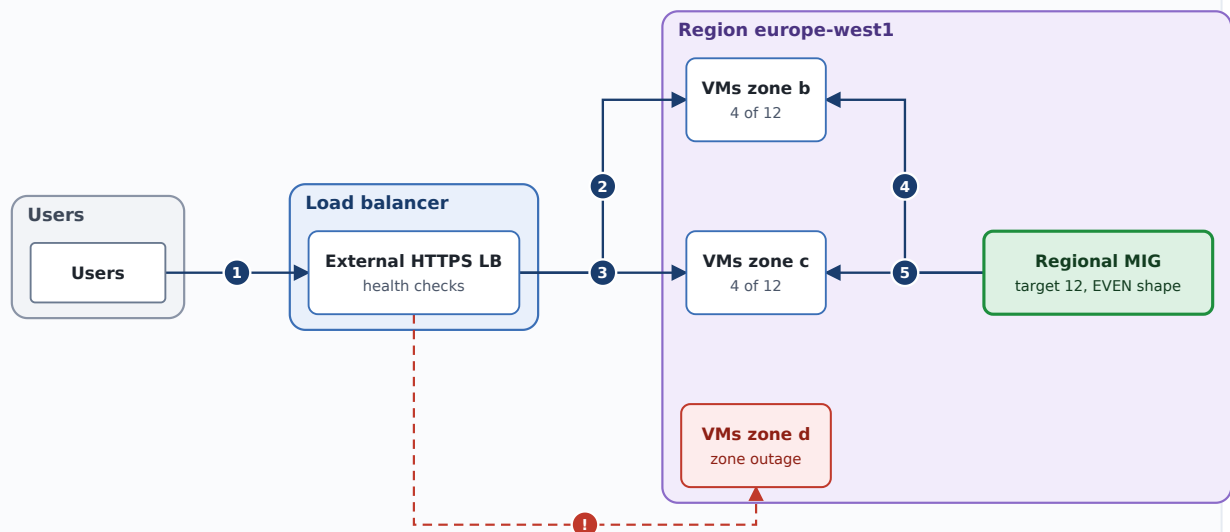
- Instance metadata overrides project metadata for the same key.
- Shutdown scripts are best effort; do not rely on them for critical work.

CONFIGURATION EXAMPLE

```
gcloud compute instances create app-1 --zone=europe-west1-b \
  --service-account=app@app-prod.iam.gserviceaccount.com --scopes=cloud-platform \
  --metadata=env=prod,startup-script-url=gs://app-config/startup.sh
# inside startup.sh
ENV=$(curl -s -H "Metadata-Flavor: Google" \
  http://metadata.google.internal/computeMetadata/v1/instance/attributes/env)
DB_PASS=$(gcloud secrets versions access latest --secret="db-pass-${ENV}")
```

Q95 A stateless web tier must keep serving if a whole zone fails. Should you use a zonal or regional MIG, and how much capacity do you need?

FLOW DIAGRAM



- 2 Traffic spread across healthy zones
- ! Backends fail health checks and are taken out of rotation
- 4 Manager recreates lost VMs in the zones still up

EXPLANATION

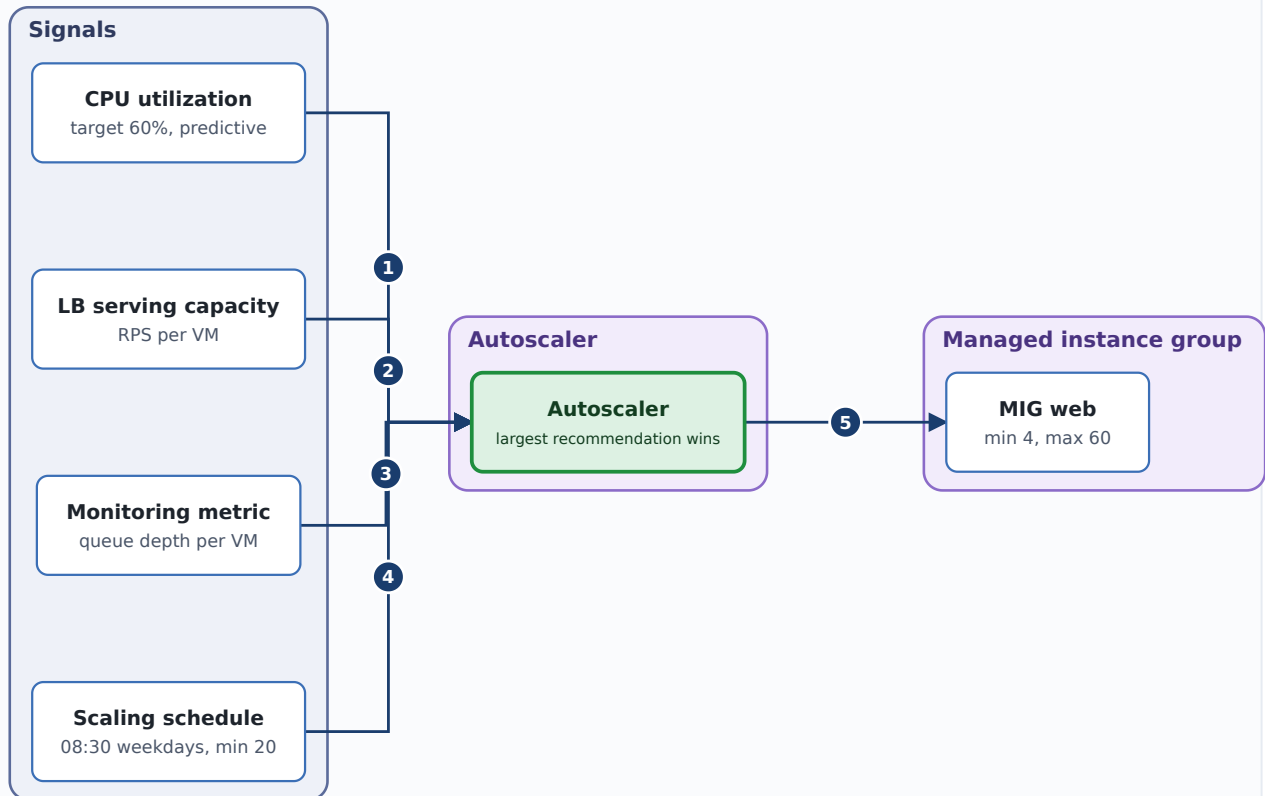
Use a regional MIG, which spreads instances across up to three zones in the region and, when a zone fails, creates replacements in the remaining zones (subject to capacity there). A zonal MIG is only a good fit when instances must sit next to a zonal resource or you run one MIG per zone on purpose. Size for the loss of one zone: with three zones, serving the peak from two means running about 1.5 times the peak need, or letting the autoscaler grow quickly with fast-booting images. Choose the target distribution shape (`EVEN` , `BALANCED` or `ANY`) based on whether you care more about even spread or getting capacity where it is available.

KEY POINTS

- N zones means $N/(N-1)$ capacity to survive one zone loss without scaling.
- Regional MIGs also spread rolling updates across zones.

Q96 Traffic spikes every weekday at 9am and during marketing campaigns; the autoscaler reacts late and sometimes flaps. How do you tune MIG autoscaling?

FLOW DIAGRAM



- ❶ Predictive mode scales ahead of daily patterns
- ❷ Load-based signal from the load balancer
- ❸ Custom metric for worker queues
- ❹ Known peaks: capacity ready before 9am
- ❺ Resize, respecting initialization period and scale-in controls

EXPLANATION

Pick signals that match the load: CPU utilization, load balancer serving capacity, or a Cloud Monitoring metric such as queue depth per instance; with several signals the autoscaler follows whichever recommends the most VMs. For daily patterns enable predictive autoscaling on CPU, and for known events add scaling schedules that raise the minimum ahead of time. Set the initialization (cool-down) period to the real boot-and-warm-up time so new VMs are not counted before they serve, and use scale-in controls to limit how many VMs can be removed in a time window, which stops flapping. Keep a sensible minimum for baseline traffic and a maximum that fits your quota.

KEY POINTS

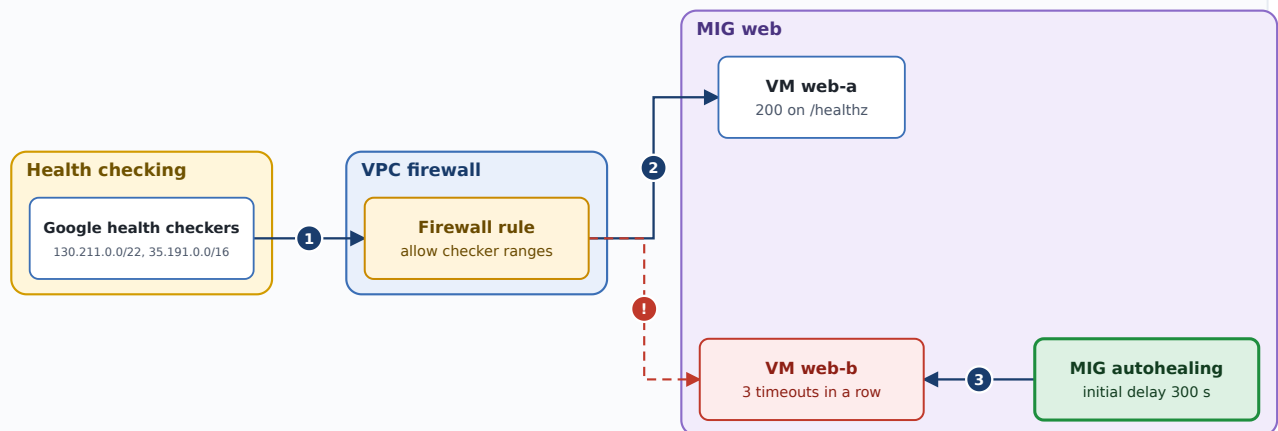
- Slow boots are the most common reason autoscaling feels late; fix the image first.
- Scale-in controls only affect removal, never scale-out.

CONFIGURATION EXAMPLE

```
gcloud compute instance-groups managed set-autoscaling web-mig --region=europe-west1 \
  --min-num-replicas=4 --max-num-replicas=60 --target-cpu-utilization=0.6 \
  --cpu-utilization-predictive-method=optimize-availability --cool-down-period=180 \
  --scale-in-control=max-scaled-in-replicas=5,time-window=600
gcloud compute instance-groups managed update-autoscaling web-mig --region=europe-west1 \
  --set-schedule=morning-peak --schedule-cron="30 8 * * 1-5" \
  --schedule-duration-sec=10800 --schedule-min-required-replicas=20 \
  --schedule-time-zone=Europe/London
```

Q97 Some VMs hang with an application deadlock: the VM is still running, the load balancer avoids it, but it never gets fixed. How do you set up autohealing?

FLOW DIAGRAM



- 1 Probes come from Google's fixed ranges
- 2 Healthy: /healthz returns 200
- ! Unhealthy after the threshold is reached
- 3 VM recreated from the template

EXPLANATION

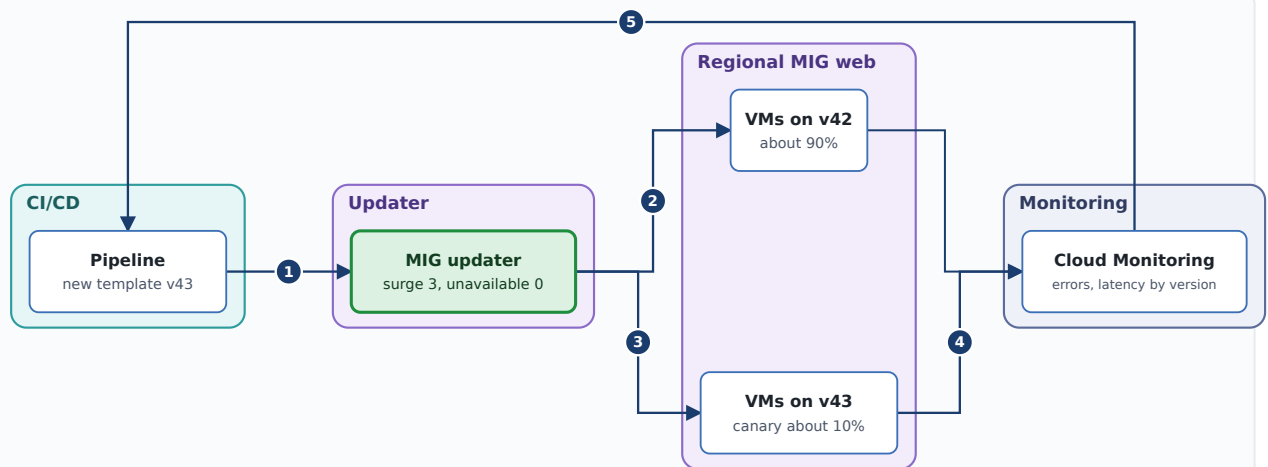
Attach an autohealing health check to the MIG; it is separate from the load balancer's health check, which only removes a VM from traffic. When a VM fails the autohealing check for the unhealthy threshold, the MIG deletes and recreates it from the template. Make the check more lenient than the LB check (longer interval, higher threshold) so a brief overload does not trigger mass recreation, point it at an endpoint that tests the app itself, and set the initial delay to the full boot time so new VMs are not killed while starting. Allow ingress from 130.211.0.0/22 and 35.191.0.0/16 on the health check port.

KEY POINTS

- A health check that depends on a shared database can recreate the whole fleet during a DB outage.
- Autohealing also fixes VMs that were stopped or crashed.

Q98 How do you release a new VM image to a regional MIG as a 10% canary, check it, then promote or roll back without downtime?

FLOW DIAGRAM



- 1 start-update with a canary version
- 2 Most VMs stay on the current template
- 3 New VMs created first, then old ones removed
- 4 Compare canary SLIs with the baseline
- 5 Promote v43 to 100%, or roll back to v42

EXPLANATION

Create a new instance template, then run a rolling update with two versions: the current template as the main version and the new one as `--canary-version` with `target-size=10%`. Set `--max-surge` (at least the number of zones for a regional MIG) and `--max-unavailable=0` so new VMs are created and healthy before old ones are removed. Watch error rate and latency split by version (use a label or metadata to tell them apart), then run `start-update` again with only the new template to promote, or only the old template to roll back. Autohealing and health checks stop the rollout from making broken VMs count as updated.

KEY POINTS

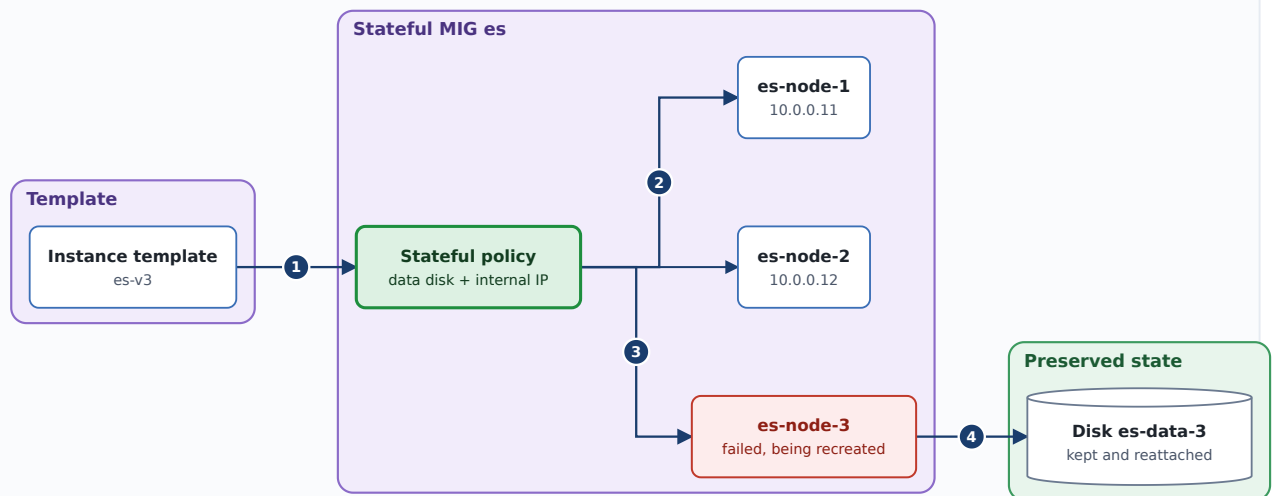
- Rollback is just another rolling update to the previous template.
- Use `--minimal-action=restart` only for metadata-only changes.

CONFIGURATION EXAMPLE

```
gcloud compute instance-groups managed rolling-action start-update web-mig \
  --region=europe-west1 --version=template=web-v42 \
  --canary-version=template=web-v43,target-size=10% \
  --max-surge=3 --max-unavailable=0
# promote
gcloud compute instance-groups managed rolling-action start-update web-mig \
  --region=europe-west1 --version=template=web-v43 --max-surge=3 --max-unavailable=0
```


Q99 A legacy Elasticsearch cluster needs fixed hostnames, fixed internal IPs and data disks that survive VM failure. Can you run it in a MIG?

FLOW DIAGRAM



- ❶ MIG created from template plus stateful config
- ❷ Each instance keeps its name, disk and IP
- ❸ Autohealing recreates the VM with the same name
- ❹ Same data disk and internal IP attached again

EXPLANATION

Yes, with a stateful MIG. Configure a stateful policy that preserves specific disks (by device name), internal or external IPs, and per-instance metadata, and the MIG keeps instance names stable. When autohealing or an update recreates a VM, it gets back the same name, the same data disk and the same IP, so the cluster sees the node return rather than a new one. Stateful MIGs cannot use the autoscaler, so resize manually, and during updates use `--replacement-method=recreate` so names are kept. Also set `auto-delete=never` on stateful disks so deleting a VM does not delete its data.

KEY POINTS

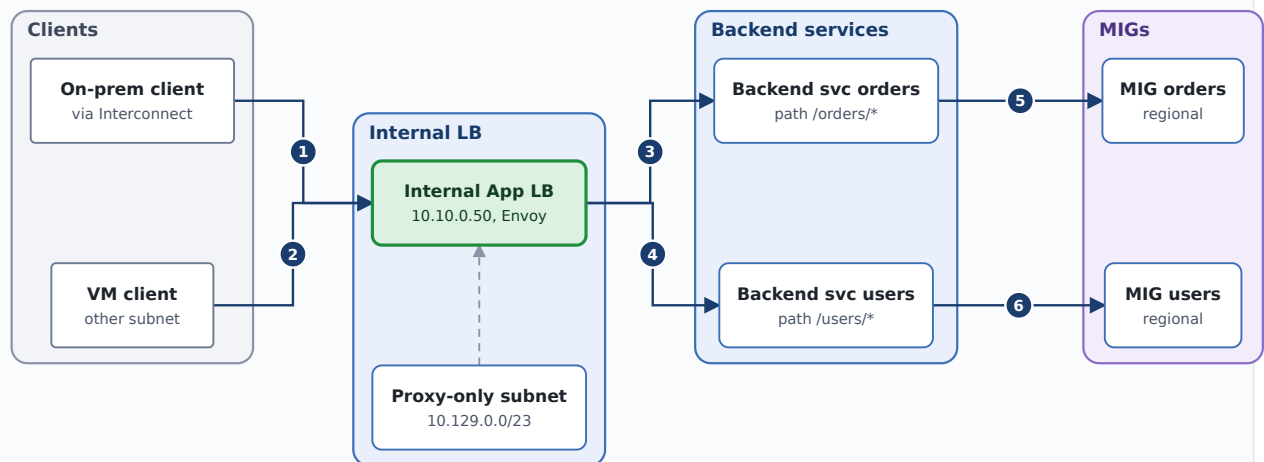
- Stateful MIGs give you autohealing and rolling updates for pets.
- Back up stateful disks with snapshot schedules; the MIG does not.

CONFIGURATION EXAMPLE

```
gcloud compute instance-groups managed create es-mig --zone=europe-west1-b \
  --template=es-v3 --size=3 \
  --stateful-disk=device-name=es-data,auto-delete=never \
  --stateful-internal-ip=enabled,auto-delete=never
gcloud compute instance-groups managed rolling-action start-update es-mig \
  --zone=europe-west1-b --version=template=es-v4 --replacement-method=recreate
```

Q100 Two internal HTTP services on MIGs are called by other VMs and by on-prem systems over Interconnect. How do you design the internal load balancing?

FLOW DIAGRAM



- 1 Private IP reachable over Interconnect
- 3 URL map routes by path
- 5 Only healthy instances get requests
- Proxies take addresses from the proxy-only subnet

EXPLANATION

For HTTP services use a regional internal Application Load Balancer: one private forwarding rule, a URL map that routes paths or hosts to separate backend services, and a regional MIG behind each backend service with its own health check. It is proxy based (managed Envoy), so it needs a proxy-only subnet in the region and backends see connections from that range, with the client IP in `X-Forwarded-For`. For plain TCP/UDP or when backends must see the client IP directly, use an internal passthrough Network Load Balancer instead. Enable global access if clients in other regions must reach it, and advertise the subnet to on-prem through Cloud Router.

KEY POINTS

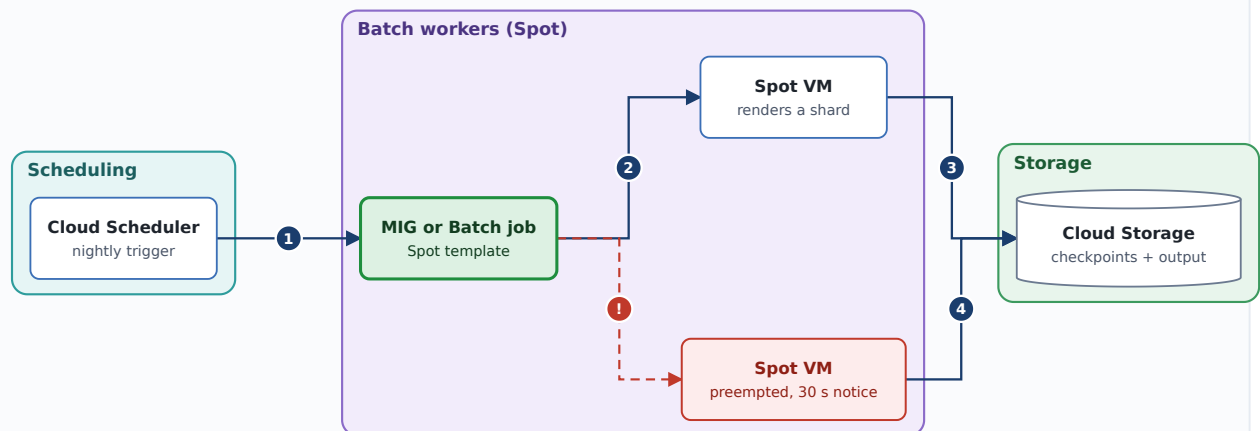
- Firewall: allow the health check ranges and, for L7, the proxy-only subnet to the backends.
- Passthrough ILB = L4 and client IP kept; internal Application LB = L7 routing.

CONFIGURATION EXAMPLE

```
gcloud compute networks subnets create proxy-only --network=prod-vpc \
  --region=europe-west1 --range=10.129.0.0/23 \
  --purpose=REGIONAL_MANAGED_PROXY --role=ACTIVE
gcloud compute backend-services create orders-bs --region=europe-west1 \
  --load-balancing-scheme=INTERNAL_MANAGED --protocol=HTTP \
  --health-checks=orders-hc --health-checks-region=europe-west1
gcloud compute backend-services add-backend orders-bs --region=europe-west1 \
  --instance-group=orders-mig --instance-group-region=europe-west1
```

Q101 A nightly rendering batch runs on 200 on-demand VMs and costs too much. Can you use Spot VMs, and how do you make the job survive preemption?

FLOW DIAGRAM



- 1 Start the job and set group size
- 2 Workers pick up shards from a queue
- 3 Write checkpoints and results often
- ! Google reclaims capacity at any time
- 4 Shutdown script saves progress
- The MIG recreates preempted VMs when capacity returns and they resume from the last checkpoint.

EXPLANATION

Yes: Spot VMs are much cheaper than on-demand (discounts often 60-91%), but Compute Engine can stop them at any time with about 30 seconds' notice, they have no SLA and they do not live migrate. Run them in a MIG or through the Batch service so preempted VMs are recreated, make work idempotent in small shards pulled from a queue, write checkpoints to Cloud Storage, and use a shutdown script to save progress. Set the termination action to `DELETE` or `STOP` as needed, spread across zones or machine types to find capacity, and keep a small on-demand group if the job has a hard deadline.

KEY POINTS

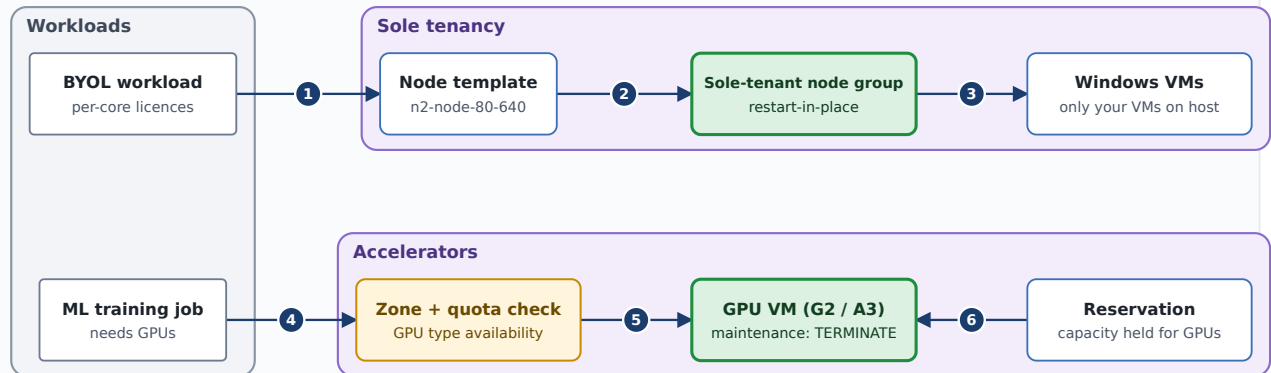
- Spot has no 24-hour limit, unlike the older preemptible VMs.
- Design for preemption as normal, not as an error.

CONFIGURATION EXAMPLE

```
gcloud compute instance-templates create render-spot \
  --machine-type=c2d-standard-16 --provisioning-model=SPOT \
  --instance-termination-action=DELETE \
  --metadata-from-file=shutdown-script=save-progress.sh \
  --image-family=render --image-project=img-factory
```

Q102 A Windows workload has per-core BYOL licences that require dedicated hardware, and the ML team needs GPU VMs. How do you handle sole-tenant nodes and GPUs?

FLOW DIAGRAM



- ❶ Choose node type matching licence cores
- ❷ Group keeps VMs on the same physical host
- ❸ VMs use node affinity labels to land here
- ❹ GPU models vary by zone; quota is per region
- ❺ Create GPU VM in a zone that has the model
- ❻ Reservation makes capacity available when needed

EXPLANATION

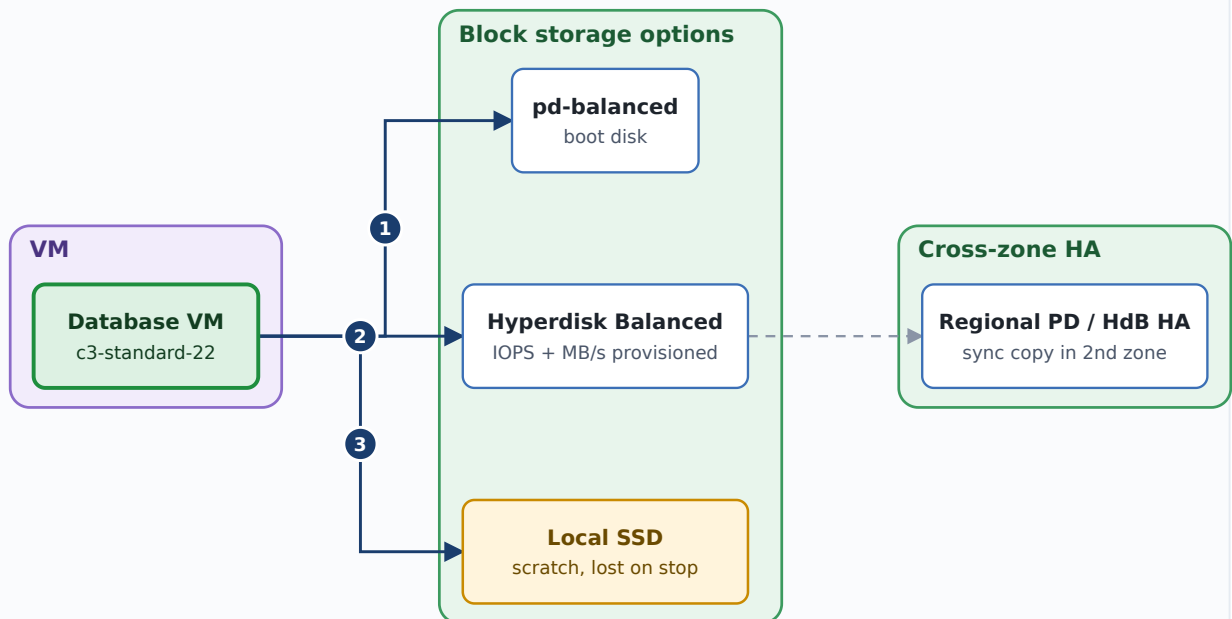
Sole-tenant nodes give you physical servers dedicated to your project: create a node template (node type), a node group, and schedule VMs onto it with node affinity labels. Choose `restart-in-place` maintenance for licences tied to a physical host, and use node group autoscaling to control cost. For GPUs, use accelerator-optimized types (A2, A3, G2) or attach GPUs to N1; GPU VMs generally cannot live migrate, so set the maintenance policy to `TERMINATE` and handle restarts. GPU models are available only in some zones and need regional GPU quota, so check both early and use reservations for training runs that cannot wait for capacity.

KEY POINTS

- You pay for the whole sole-tenant node, used or not.
- Watch the maintenance-event metadata on GPU VMs to checkpoint before stops.

Q103 A PostgreSQL VM needs about 50k IOPS for data, fast scratch space for temp files, and must be recoverable in another zone. Which disk types do you choose?

FLOW DIAGRAM



- 1 General-purpose OS disk
- 2 Data volume sized for performance, not only capacity
- 3 Temp tables and cache at the lowest latency
- Use a replicated disk when zone failover must keep data

EXPLANATION

Use `pd-balanced` for boot disks and most general volumes, and Hyperdisk Balanced (or Hyperdisk Extreme for the highest IOPS) for the database data volume, because Hyperdisk lets you provision IOPS and throughput separately from size, whereas Persistent Disk performance grows with disk size. Local SSD is physically attached and gives the lowest latency, but its data is lost when the VM stops or the host fails, so use it only for scratch or data you can rebuild. For zone-level recovery use regional Persistent Disk or Hyperdisk Balanced High Availability, which replicate synchronously to a second zone, plus snapshots. Check the machine type's per-VM IOPS and throughput limits, since they cap what any disk can deliver.

KEY POINTS

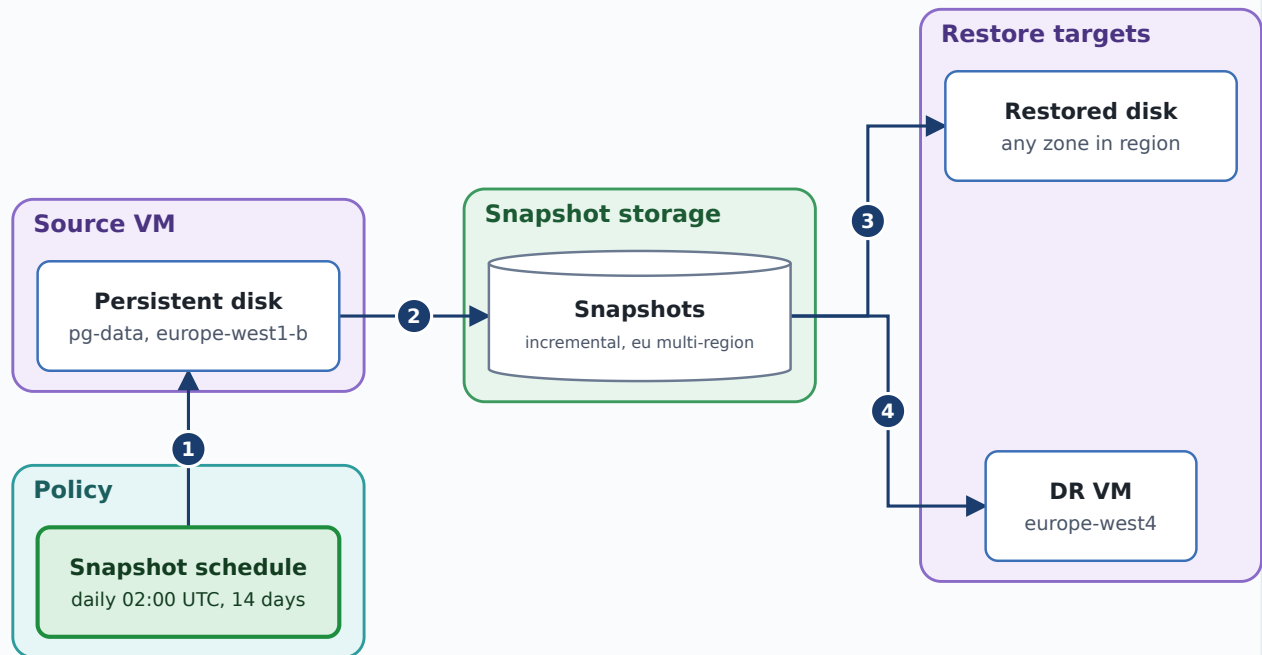
- Hyperdisk support depends on machine series; check the compatibility table.
- Disk performance limits are per disk and per VM; the lower one wins.

CONFIGURATION EXAMPLE

```
gcloud compute disks create pg-data --zone=europe-west1-b \
  --type=hyperdisk-balanced --size=500GB \
  --provisioned-iops=50000 --provisioned-throughput=800
gcloud compute instances attach-disk pg-1 --disk=pg-data --zone=europe-west1-b
```

Q104 You need daily backups of VM data disks with 14-day retention and the ability to restore in another region. How do you set up snapshots?

FLOW DIAGRAM



- 1 Schedule attached to the disk
- 2 Daily snapshot stores only changed blocks
- 3 Restore in place after corruption
- 4 Restore in another region for DR

EXPLANATION

Create a snapshot schedule (a resource policy) with a daily start time, 14-day retention and `keep-auto-snapshots` on source-disk deletion, and attach it to each disk, ideally from Terraform or the instance template. Snapshots are incremental and stored in a multi-regional or regional location you choose, so an `eu` location lets you create disks in any EU region for DR. For databases, use application-consistent snapshots (guest flush on Linux with pre and post scripts, VSS on Windows) or freeze writes briefly. Test restores regularly, and consider machine images for whole-VM copies or Backup and DR for central policies and reporting.

KEY POINTS

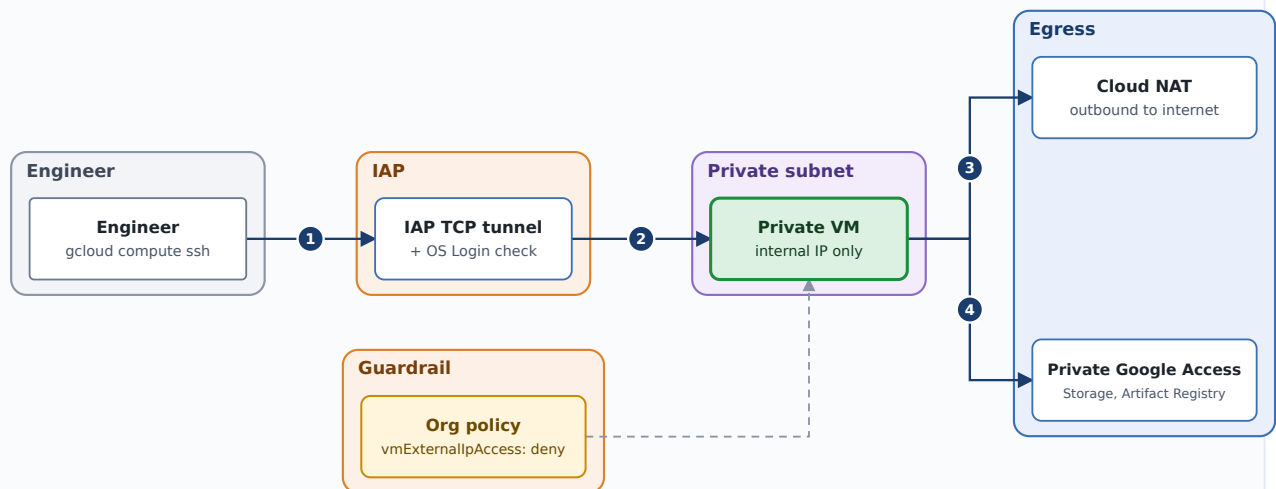
- A snapshot you have never restored is not a backup.
- Schedule times are in UTC.

CONFIGURATION EXAMPLE

```
gcloud compute resource-policies create snapshot-schedule daily-14d \
  --region=europe-west1 --daily-schedule --start-time=02:00 \
  --max-retention-days=14 --on-source-disk-delete=keep-auto-snapshots \
  --storage-location=eu
gcloud compute disks add-resource-policies pg-data \
  --resource-policies=daily-14d --zone=europe-west1-b
```

Q105 Security wants no VM to have an external IP, but engineers still need SSH and VMs must download OS packages. How do you design access?

FLOW DIAGRAM



- ❶ SSH tunnelled over HTTPS with `--tunnel-through-iap`
- ❷ Port 22 allowed only from `35.235.240.0/20`
- ❸ Package mirrors reached through NAT, no inbound path
- ❹ Google APIs reached over internal routing
- Creation of VMs with external IPs is refused

EXPLANATION

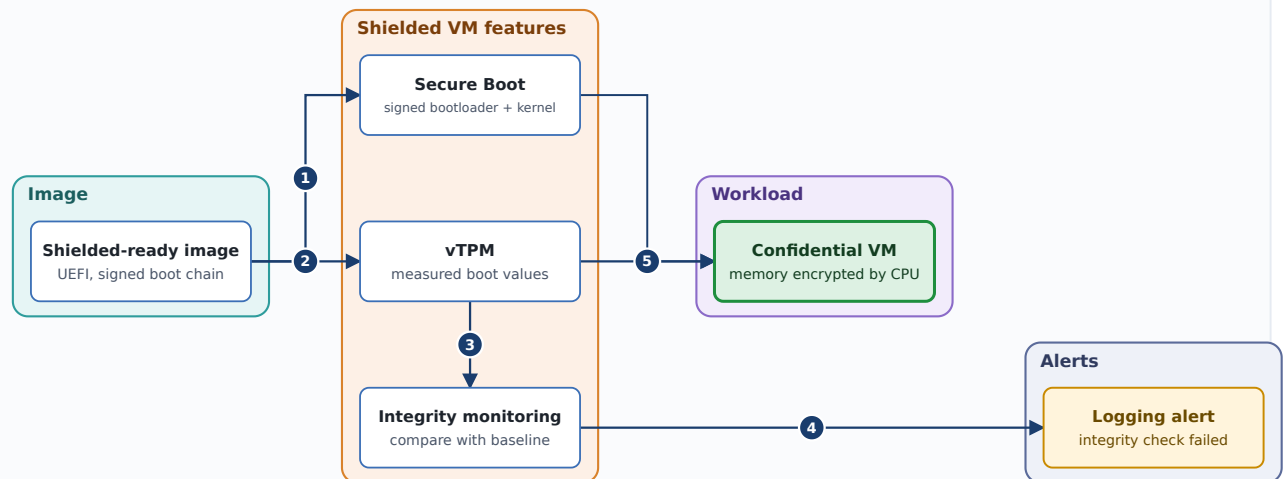
Enforce the org policy `compute.vmExternalIpAccess` to deny external IPs, and give engineers SSH through IAP TCP forwarding (`gcloud compute ssh --tunnel-through-iap`) with a firewall rule allowing port 22 only from `35.235.240.0/20` . Enable OS Login so access is tied to IAM roles and Google identities rather than metadata SSH keys. For outbound traffic, add Cloud NAT on the subnet's region for package repositories and turn on Private Google Access so VMs reach Google APIs without public IPs. There is no bastion to patch, every session is authenticated and logged, and nothing on the VM listens to the internet.

KEY POINTS

- Cloud NAT is outbound only; it never lets connections in.
- Use a private repository mirror (e.g. Artifact Registry remote repos) to reduce internet egress.

Q106 A security review asks for protection against boot-level malware on VMs and encryption of data while it is being processed. What do Shielded VM and Confidential VM give you?

FLOW DIAGRAM



- 1 Only signed boot components are allowed to run
- 2 Each boot stage is measured into the vTPM
- 3 Measurements compared with the learned baseline
- 4 A mismatch produces an event you can alert on
- 5 Workload runs with memory encrypted (SEV, SEV-SNP or TDX)

EXPLANATION

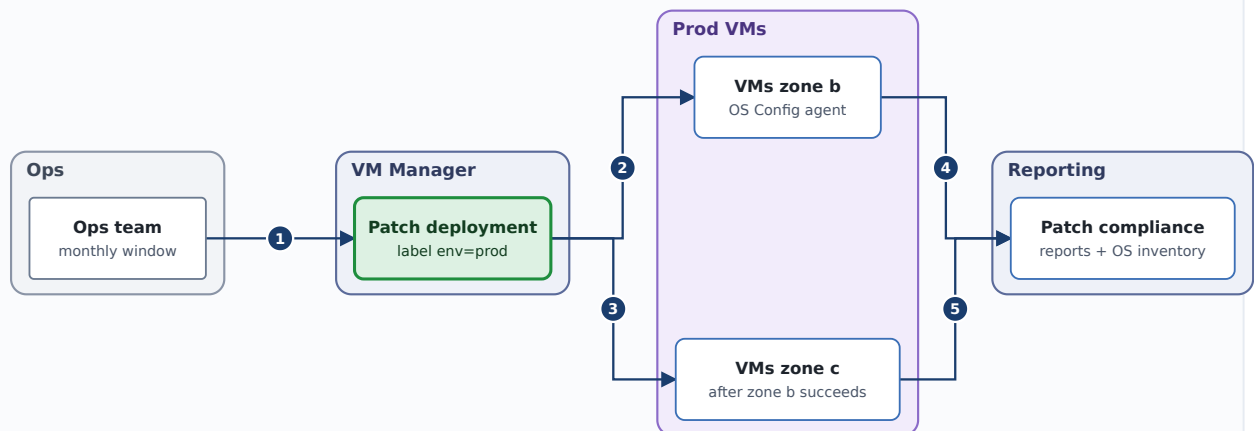
Shielded VM adds Secure Boot (only signed bootloaders and kernels run), a virtual TPM for measured boot, and integrity monitoring that compares each boot with a baseline and logs failures you can alert on; enforce it with the org policy `compute.requireShieldedVm`. Confidential VM encrypts VM memory with keys generated and held by the CPU (AMD SEV or SEV-SNP, Intel TDX), protecting data in use from the host, and supports attestation to prove it. It works only on supported machine series and images, and some features such as live migration or certain GPUs depend on the technology chosen, so check limits before standardising. Together with CMEK for disks, this covers data at rest, in transit and in use.

KEY POINTS

- Shielded VM is on by default for most public images; keep it on.
- Confidential VM usually needs no application changes.

Q107 You run 800 long-lived Linux and Windows VMs and auditors want monthly patching with compliance reports. How do you do it with VM Manager?

FLOW DIAGRAM



- ❶ Schedule, pre/post scripts, reboot setting
- ❷ First zone, disruption budget 10%
- ❸ Next zone only after the first completes
- ❹ Agents report patch results

EXPLANATION

Enable VM Manager (the OS Config agent is included in most Google images, turned on with the metadata `enable-osconfig=TRUE`) and create patch deployments that target VMs by label, zone or name. Schedule them in a maintenance window, roll out zone by zone with a disruption budget, add pre- and post-patch scripts for draining and checks, and choose the reboot behaviour. OS inventory and patch compliance reports show which VMs are missing updates, which is what auditors ask for, and OS policies can enforce required packages or configuration. For MIG-based stateless fleets, prefer rebuilding the image and doing a rolling update instead of patching in place.

KEY POINTS

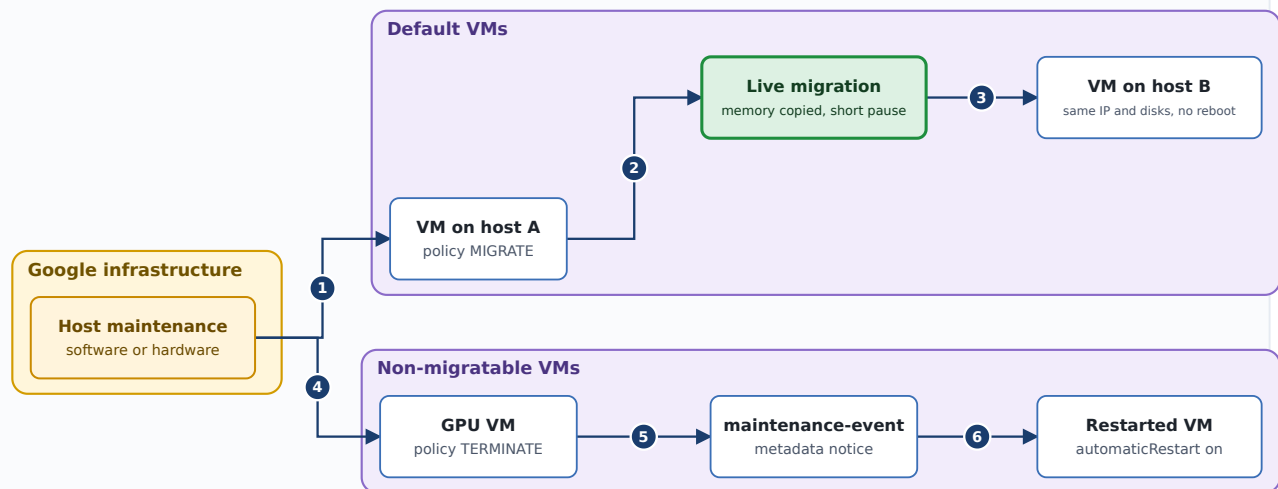
- Patch non-prod first with the same deployment, one week earlier.
- Zone-by-zone rollout limits the blast radius of a bad patch.

CONFIGURATION EXAMPLE

```
# gcloud compute os-config patch-deployments create monthly-prod --file=patch.yaml
instanceFilter:
  groupLabels:
    - labels: {env: prod}
recurringSchedule:
  timeZone: {id: Europe/London}
  timeOfDay: {hours: 2}
  frequency: MONTHLY
  monthly: {weekDayOfMonth: {weekOrdinal: 2, dayOfWeek: SUNDAY}}
  rollout: {mode: ZONE_BY_ZONE, disruptionBudget: {percent: 10}}
```

Q108 What happens to your VMs when Google performs host maintenance, and how do you handle VMs that cannot live migrate, such as GPU VMs?

FLOW DIAGRAM



- ➊ Running VM moved to another host
- ➋ Apps keep running; brief blackout
- ➌ App watches metadata and checkpoints or drains
- ➍ VM stopped, then restarted on a healthy host

EXPLANATION

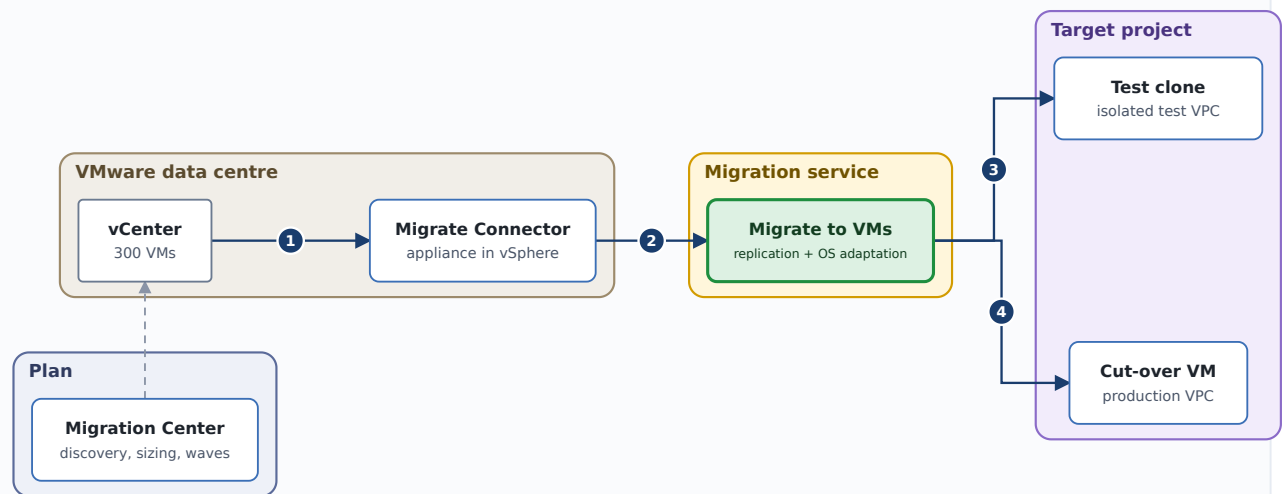
With the default `MIGRATE` policy, Compute Engine live migrates the running VM to another host: memory is copied while the VM runs, followed by a short pause, and the VM keeps its IP, disks and state without a reboot. VMs that cannot live migrate, including most GPU VMs and some specialised types, use `TERMINATE` and are stopped and (with automatic restart) started again after advance notice. Watch the `maintenance-event` metadata key (with `wait_for_change=true`) to drain or checkpoint, check upcoming maintenance notifications, and test your handling with `gcloud compute instances simulate-maintenance-event`. Run such workloads in groups across zones so one restart does not cause an outage.

KEY POINTS

- Live migration is invisible to most apps but can affect very latency-sensitive ones.
- Spot VMs are stopped rather than migrated.

Q109 You must migrate 300 VMware VMs to Compute Engine in waves with minimal downtime. How do you plan and run it with Migrate to Virtual Machines?

FLOW DIAGRAM



- Assessment groups VMs into waves by dependency
- 1 Disks read through vSphere APIs, no guest agent
- 2 Continuous replication over Interconnect or VPN
- 3 Test clones while the source keeps running
- 4 Cut-over: source stopped, final sync, VM started

EXPLANATION

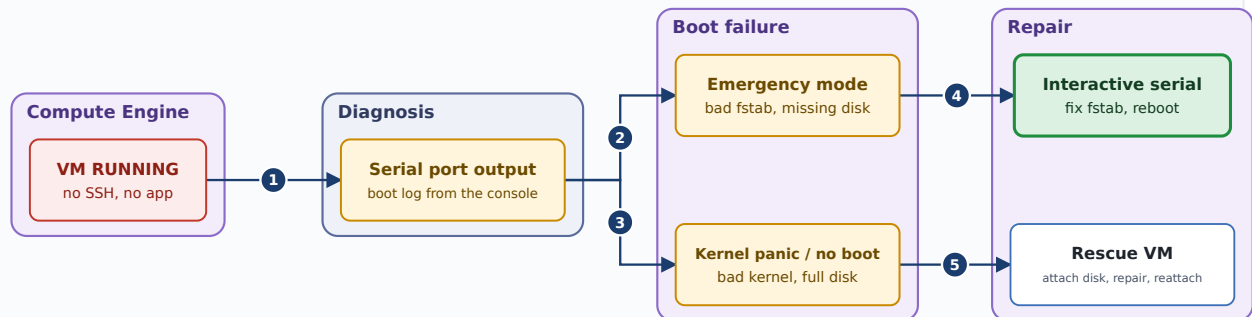
Start with discovery and assessment in Migration Center to map dependencies, right-size target machine types, estimate cost and plan waves. Deploy the Migrate Connector appliance in vSphere, register the source, and start replication for a wave; disks replicate continuously over Interconnect or VPN while the VMs keep running. Migrate to Virtual Machines adapts the OS (drivers, Google guest environment) so it boots on Compute Engine. Create test clones in an isolated VPC to validate each app, then cut over: the source VM is shut down, a final incremental sync runs and the new VM starts, so downtime is roughly the final sync plus boot time. Keep the source powered off but intact until the business signs off, so rollback is possible.

KEY POINTS

- Migrate a low-risk wave first to prove networking, DNS and licensing.
- Plan IP addressing and DNS changes as carefully as the VMs themselves.

Q110**Troubleshooting**

After an engineer edited `/etc/fstab` and rebooted, a VM shows **RUNNING** in the console but SSH and the app are unreachable. How do you troubleshoot a VM that will not boot?

FLOW DIAGRAM

- 1 RUNNING only means the hypervisor started it
- 2 "Dependency failed for /data" and a login prompt
- 3 Panic, grub or read-only errors
- 4 Log in on the serial console as a local user
- 5 Snapshot first, then repair on another VM

EXPLANATION

RUNNING only means the VM was started, so read the serial port output to see how far the boot got. A bad `/etc/fstab` entry or a missing data disk usually drops systemd into emergency mode; you can fix it through the interactive serial console if it is enabled and a local user with a password exists. Otherwise snapshot the boot disk, stop the VM, detach the disk, attach it to a rescue VM in the same zone, mount it and fix the file (add `nofail` to non-root mounts), then reattach it as the boot disk. The same rescue flow fixes a broken kernel update, a full root disk or a bad `sshd` config.

KEY POINTS

- Add `nofail` to every non-root `fstab` entry.
- Take a snapshot before risky OS changes; restore is faster than repair.

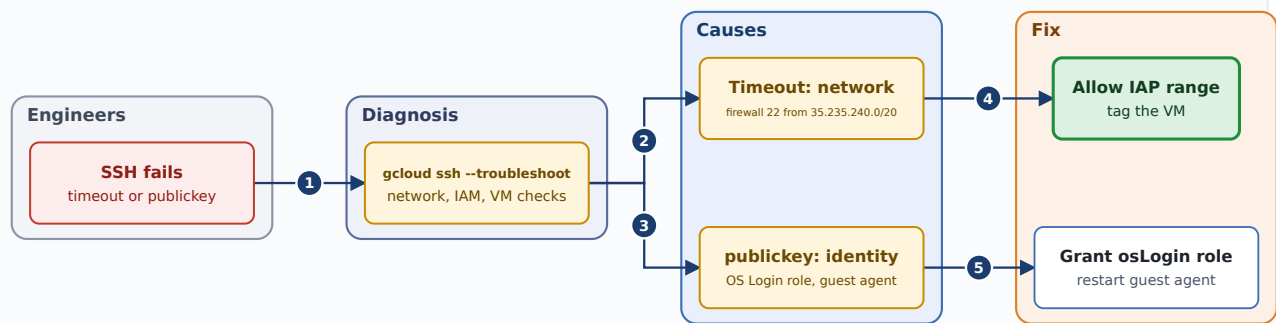
CONFIGURATION EXAMPLE

```

gcloud compute instances get-serial-port-output app-1 --zone=us-central1-a | tail -50
gcloud compute instances add-metadata app-1 --zone=us-central1-a --metadata=serial-port-enable=TRUE
gcloud compute connect-to-serial-port app-1 --zone=us-central1-a
# rescue flow
gcloud compute instances stop app-1 --zone=us-central1-a
gcloud compute instances detach-disk app-1 --disk=app-1 --zone=us-central1-a
gcloud compute instances attach-disk rescue-1 --disk=app-1 --zone=us-central1-a
  
```

Q111**Troubleshooting**

`gcloud compute ssh web-3 --tunnel-through-iap` fails with Connection timed out for one engineer and Permission denied (publickey) for another. How do you troubleshoot SSH to a VM?

FLOW DIAGRAM

- ❶ Built-in checks point at the failing layer
- ❷ Timeout = packets never reach sshd
- ❸ publickey = reached sshd, key rejected
- ❹ Firewall rule for IAP TCP forwarding
- ❺ Also check disk full or sshd broken via serial

EXPLANATION

A timeout means packets do not reach sshd: with IAP the VM needs a firewall rule allowing TCP 22 from 35.235.240.0/20, the user needs `roles/iap.tunnelResourceAccessor`, and the VM must be running and booted. Permission denied (publickey) means sshd answered but rejected the key: with OS Login enabled the user needs `roles/compute.osLogin` (or `osAdminLogin`), and on metadata-key VMs the guest agent must be running to write the key. `gcloud compute ssh --troubleshoot` runs connectivity, IAM and VM checks for you. If those pass, read the serial console for a full disk or a broken sshd configuration.

KEY POINTS

- Use OS Login everywhere so access follows IAM and leavers lose it at once.
- Keep a firewall rule for the IAP range on a network tag applied by the instance template.

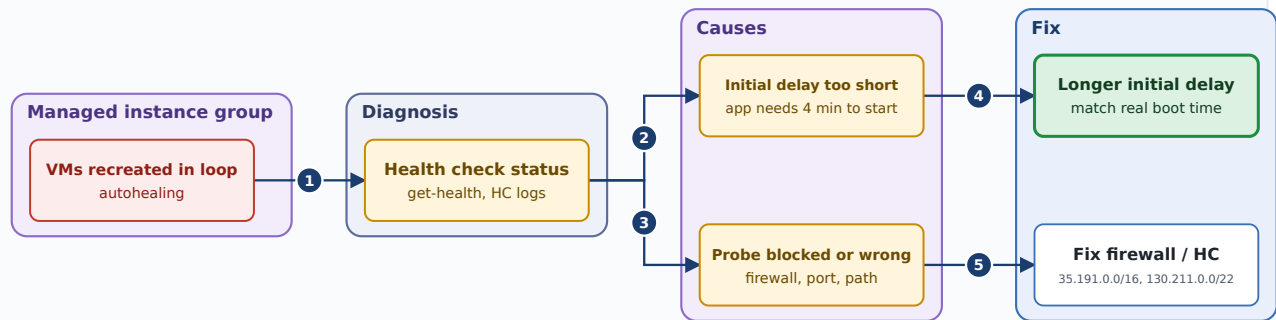
CONFIGURATION EXAMPLE

```

gcloud compute ssh web-3 --zone=europe-west1-b --tunnel-through-iap --troubleshoot
gcloud compute firewall-rules create allow-iap-ssh --network=prod-vpc \
  --direction=INGRESS --allow=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=ssh-iap
gcloud compute instances add-tags web-3 --zone=europe-west1-b --tags=ssh-iap
gcloud projects add-iam-policy-binding prod --member=user:asha@example.com --role=roles/compute.osLogin
  
```

Q112**Troubleshooting**

A regional MIG keeps deleting and recreating its VMs every few minutes; `list-errors` and the console show instances being autohealed shortly after they start. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Which check fails, and when after boot?
- ❷ Healthy later if left alone
- ❸ Never healthy, even when the app runs
- ❹ curl the path and port from inside the VM

EXPLANATION

The autohealing health check marks VMs unhealthy and the MIG recreates them, which never gives them time to become healthy. If the app is fine after a few minutes, the initial delay is shorter than the real boot plus startup time; set it to cover that. If the VM never becomes healthy, the probe is blocked or wrong: allow Google's health check ranges `35.191.0.0/16` and `130.211.0.0/22` to the serving port, and check the port, path and expected response code match what the app serves. While debugging, you can temporarily remove the autohealing policy so the VM stays around for you to inspect.

KEY POINTS

- Use a separate, more tolerant autohealing health check than the load balancer's.
- Bake software into the image so startup time is short and predictable.

CONFIGURATION EXAMPLE

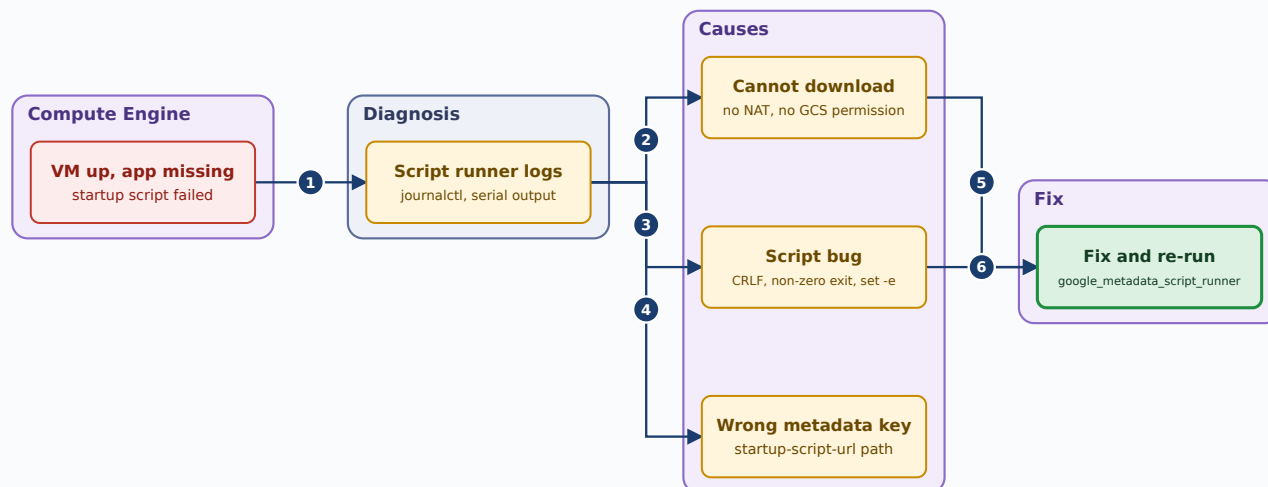
```

gcloud compute instance-groups managed list-errors web-mig --region=us-central1
gcloud compute backend-services get-health web-bs --global
gcloud compute firewall-rules create allow-hc --network=prod-vpc --allow=tcp:8080 \
  --source-ranges=35.191.0.0/16,130.211.0.0/22 --target-tags=web
gcloud compute instance-groups managed update web-mig --region=us-central1 \
  --health-check=web-hc --initial-delay=300
  
```

Q113**Troubleshooting**

New VMs come up but the app is not installed; the startup script seems to fail silently. How do you find out why and fix it?

FLOW DIAGRAM



- ❶ Runner logs each line and the exit status
- ❷ apt or gsutil timeouts, 403 on the script
- ❸ Syntax errors, missing packages
- ❹ "No startup scripts to run"
- ❺ Re-run on the VM to test quickly

EXPLANATION

The startup script runs as root through the guest environment, and its output goes to the `google-startup-scripts` journal unit, the serial console and Cloud Logging. Common causes are no outbound access (no external IP and no Cloud NAT), a service account or scopes that cannot read a `startup-script-url` in Cloud Storage, Windows line endings, or a command that fails and stops the script. No startup scripts to run means the metadata key is wrong, for example `startup_script` instead of `startup-script`. Fix it and re-run the script on the VM with `google_metadata_script_runner startup` before rolling the template out again.

KEY POINTS

- Make scripts idempotent and log clearly; they run on every boot.
- Keep long installs in the image and use the script only for configuration.

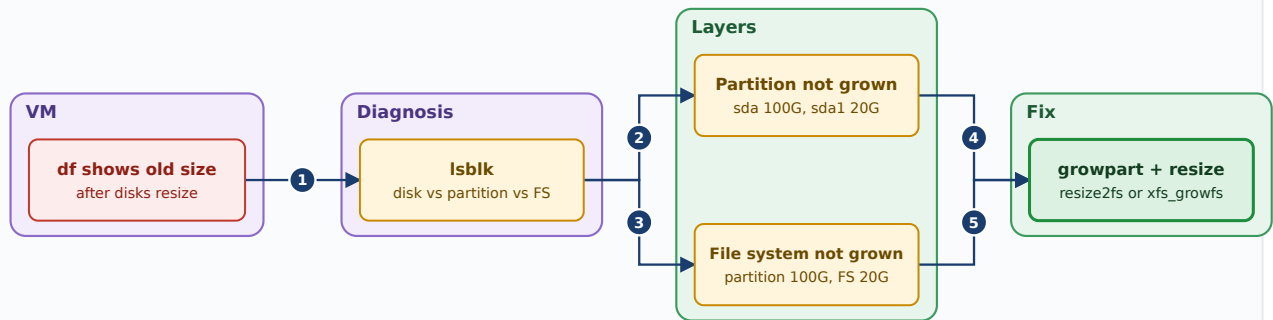
CONFIGURATION EXAMPLE

```

sudo journalctl -u google-startup-scripts.service --no-pager | tail -40
gcloud compute instances get-serial-port-output web-4 --zone=us-central1-b | grep -i startup-script
gcloud compute instances describe web-4 --zone=us-central1-b --
format='yaml(metadata.items[.key],serviceAccounts)'
sudo google_metadata_script_runner startup
  
```

Q114**Troubleshooting**

You resized a VM's boot disk from 20 GB to 100 GB because it was full, but `df -h` inside the VM still shows 20 GB. What is wrong and how do you fix it?

FLOW DIAGRAM

- 1 Compare the three sizes
- 2 Disk is bigger, partition is not
- 3 Partition grown, file system not
- 4 Grow the partition, then the file system

EXPLANATION

Resizing the persistent disk only grows the block device; the partition and the file system on it keep their old size until you grow them. `lsblk` shows the disk, partition and mount sizes side by side. Many public images grow the root partition automatically on the next boot, but on a running VM or custom image run `growpart` on the partition and then `resize2fs` for ext4 or `xfs_growfs` for XFS, all online. Take a snapshot before the resize, and remember a persistent disk can never be shrunk.

KEY POINTS

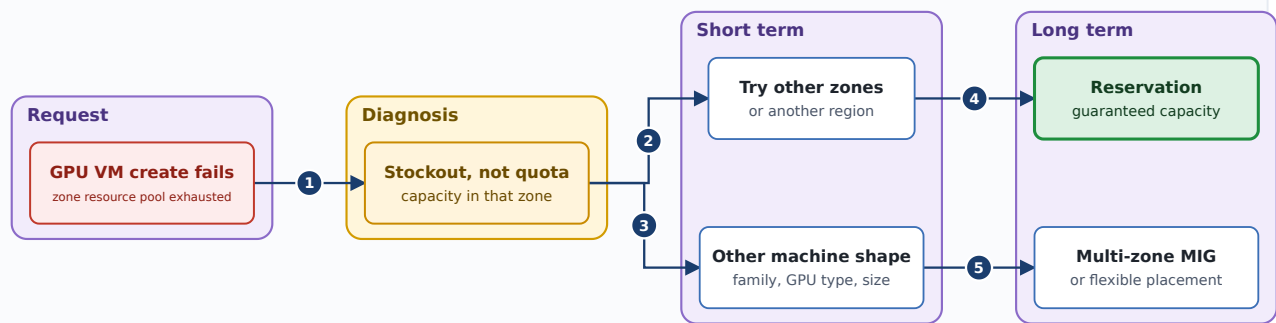
- Check the file system type with `df -T` before choosing the grow command.
- Alert at about 85% disk use so resizes are planned, not emergencies.

CONFIGURATION EXAMPLE

```
lsblk; df -hT /
gcloud compute disks resize web-5 --size=100GB --zone=asia-south1-a
sudo growpart /dev/sda 1
sudo resize2fs /dev/sda1      # ext4
# sudo xfs_growfs /           # xfs
```


Q115**Troubleshooting****Creating GPU VMs in us-central1-a fails with**

ZONE_RESOURCE_POOL_EXHAUSTED: The zone does not have enough resources available to fulfill the request. Quota is fine. What do you do?

FLOW DIAGRAM

- 1 Quota errors say QUOTA_EXCEEDED instead
- 2 Retry in a zone that has capacity
- 3 A different shape may be available
- 4 Reserve capacity before critical events
- 5 Let the MIG spread across zones

EXPLANATION

ZONE_RESOURCE_POOL_EXHAUSTED is a temporary capacity stockout in that zone for that machine type, not a quota problem, so a quota increase does not help. In the short term retry later, try other zones in the region or another region, or pick a different machine family, size or GPU type. For workloads that must start, create a reservation (it holds capacity once created) or buy committed use with a reservation, and let a regional MIG spread VMs across zones so one full zone does not stop scaling. Spot VMs are the most likely to hit stockouts, so give Spot MIGs several zones and shapes.

KEY POINTS

- Reserve GPUs ahead of launches and training runs.
- Design templates and code so the zone is not hard-coded.

CONFIGURATION EXAMPLE

```

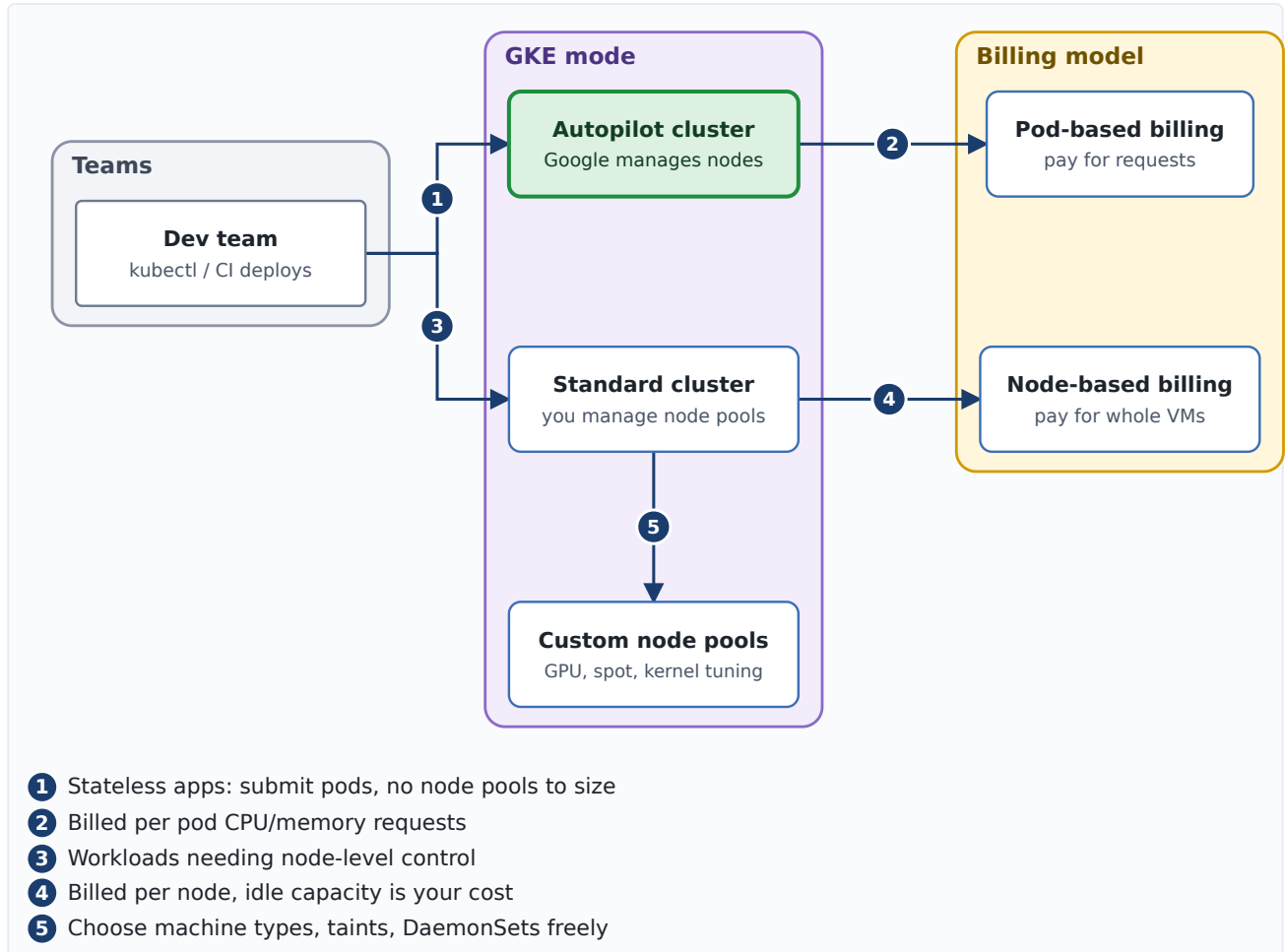
gcloud compute accelerator-types list --filter='zone:us-central1 AND name=nvidia-l4'
gcloud compute instances create train-1 --zone=us-central1-c --machine-type=g2-standard-8 \
  --accelerator=type=nvidia-l4,count=1 --maintenance-policy=TERMINATE --image-family=debian-12 --image-project=debian-cloud
gcloud compute reservations create l4-res --zone=us-central1-c --vm-count=4 \
  --machine-type=g2-standard-8 --accelerator=count=1,type=nvidia-l4
  
```

5. Google Kubernetes Engine (GKE)

Cluster design, networking, scaling, security, upgrades and day-2 operations for GKE Standard and Autopilot.

Q116 A platform team must choose between GKE Standard and Autopilot for 40 stateless microservices and a few GPU training jobs. How do you decide?

FLOW DIAGRAM



EXPLANATION

Autopilot fits most stateless services: Google provisions and upgrades nodes, enforces security defaults, and bills for the resources pods request, so bin-packing waste is not your problem. Standard gives full control of node pools, machine types, privileged DaemonSets and node-level tuning, but you pay for whole nodes and own their sizing and upgrades. Autopilot also supports GPUs and Spot through compute classes and nodeSelectors, so GPU jobs alone do not force Standard. Choose Standard when you need privileged agents, custom kernels, specific node configurations or very high density that Autopilot restrictions block. Many organisations run Autopilot by default and keep a Standard cluster for exceptions.

KEY POINTS

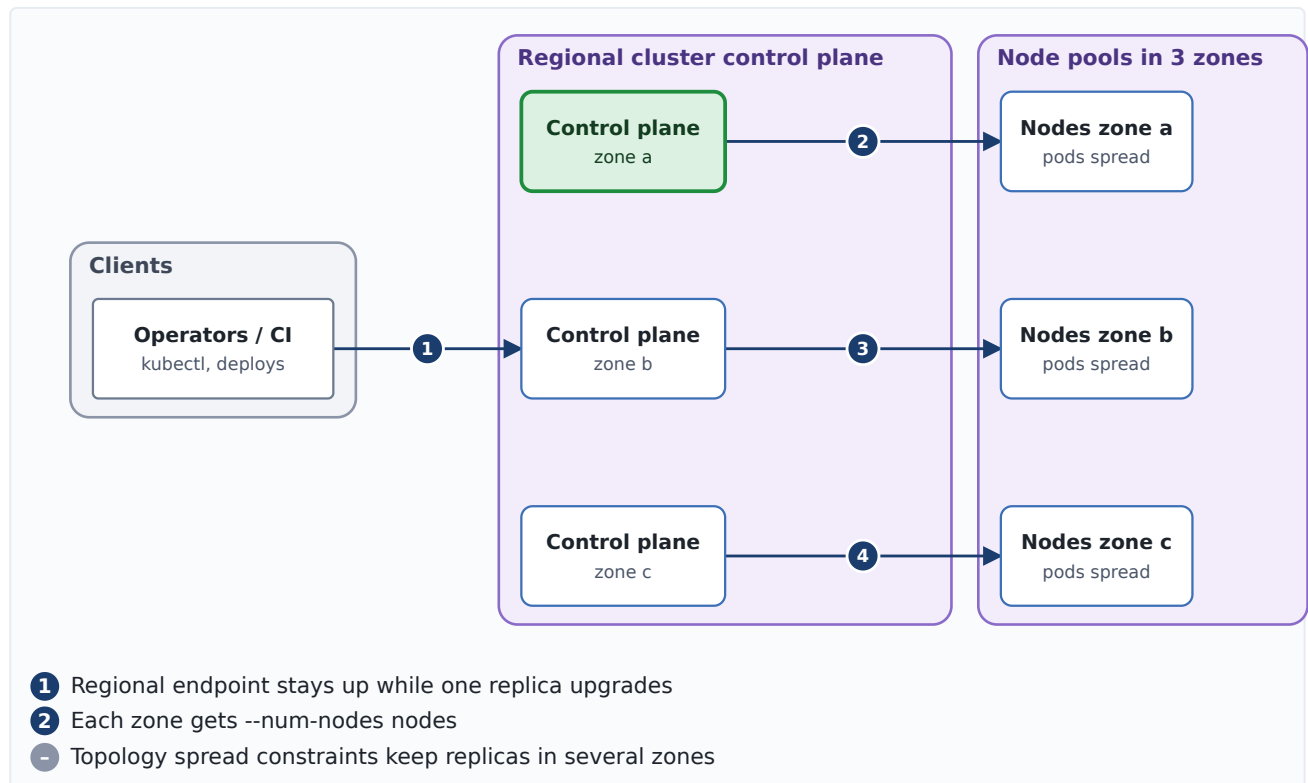
- Autopilot: per-pod billing, hardened defaults, no node SSH or most privileged pods.
- Standard: full node control, you pay for idle node capacity.

CONFIGURATION EXAMPLE

```
gcloud container clusters create-auto apps-ap \
  --region=europe-west1 --release-channel=regular
gcloud container clusters create ml-std \
  --region=europe-west1 --num-nodes=1 --machine-type=e2-standard-4
```

Q117 A zonal GKE cluster's API became unreachable during a control-plane upgrade and a zone outage took all nodes down. How would you redesign it?

FLOW DIAGRAM



EXPLANATION

A zonal cluster has a single control-plane replica, so control-plane upgrades and zone failures make the Kubernetes API unavailable, and single-zone nodes fail together. A regional cluster runs control-plane replicas in three zones and, by default, places nodes in each of those zones, so the API survives upgrades and one zone can fail. Remember `--num-nodes` is per zone, so a regional cluster with `--num-nodes=2` starts with six nodes. Spread workloads with topology spread constraints or pod anti-affinity on `topology.kubernetes.io/zone`, and use regional persistent disks or replicated storage for stateful data.

KEY POINTS

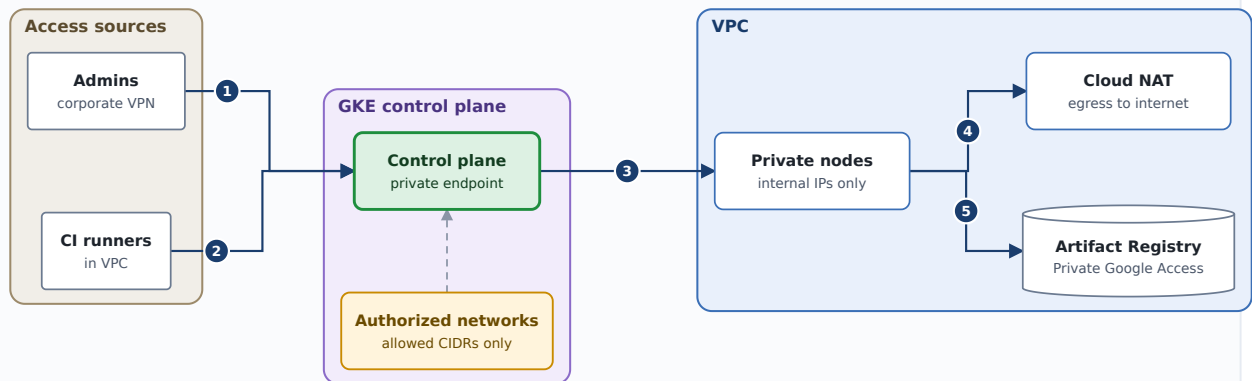
- Regional = 3 control-plane replicas + nodes in multiple zones.
- `--num-nodes` is per zone; budget for 3x nodes.

CONFIGURATION EXAMPLE

```
gcloud container clusters create prod-eu \  
  --region=europe-west1 \  
  --node-locations=europe-west1-b,europe-west1-c,europe-west1-d \  
  --num-nodes=2 --release-channel=stable
```

Q118 Security requires that GKE nodes have no public IPs and the control plane is reachable only from the corporate network and CI runners. How do you build this?

FLOW DIAGRAM



- 1 VPN/Interconnect to private endpoint
- 2 Runner IP in authorized network list
- 3 Control plane to kubelet over peering/PSC
- 4 Outbound only, no inbound from internet
- 5 Image pulls over Private Google Access
- Requests from other IPs are rejected

EXPLANATION

Create the cluster with private nodes so nodes get only internal IPs, and enable control-plane authorized networks so only listed CIDRs such as the VPN range and CI subnet can reach the API. With the private endpoint enabled, the public endpoint is disabled and admins connect over VPN or Interconnect; newer clusters can also use the DNS-based control-plane endpoint protected by IAM. Add Cloud NAT for outbound internet access and enable Private Google Access on the subnet so nodes can pull from Artifact Registry. Check firewall rules if admission webhooks on non-standard ports fail, because the control plane must reach them on the nodes.

KEY POINTS

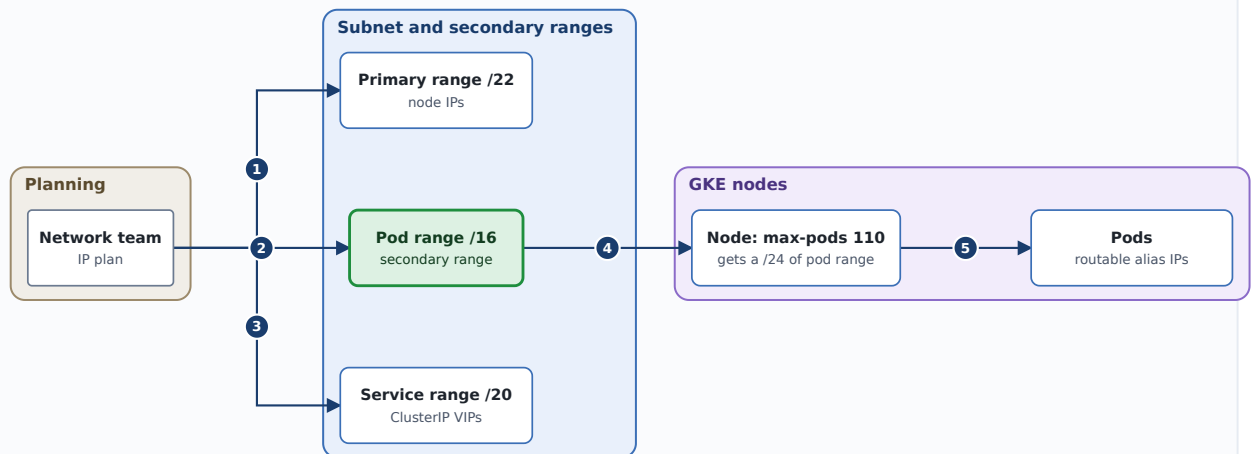
- Private nodes + authorized networks + Cloud NAT is the common baseline.
- Webhooks may need a firewall rule from the control-plane range.

CONFIGURATION EXAMPLE

```
gcloud container clusters create prod-priv --region=europe-west1 \
  --enable-ip-alias --enable-private-nodes \
  --enable-private-endpoint --master-ipv4-cidr=172.16.0.32/28 \
  --enable-master-authorized-networks \
  --master-authorized-networks=10.10.0.0/16,10.20.5.0/24
```

Q119 You are planning subnets for a VPC-native GKE cluster that may grow to 200 nodes. How do you size node, pod and service ranges?

FLOW DIAGRAM



- ❶ One IP per node (+ surge nodes)
- ❷ Pod range sized from nodes x per-node block
- ❸ Services get IPs from their own range
- ❹ Each node takes a fixed alias block
- ❺ Pod IPs are VPC-routable, no overlay

EXPLANATION

In VPC-native mode each node reserves an alias block from the pod secondary range sized from max pods per node: 110 pods reserves a /24, and a lower limit such as 32 reserves a /26. For 200 nodes at 110 pods you need $200 \times /24$, so a /16 pod range (256 blocks) leaves room for surge upgrades. Size the node primary range for maximum nodes plus upgrade surge, and give Services their own range, for example /20. Avoid overlap with on-prem and peered VPCs because pod IPs are routable, and plan non-RFC1918 or additional pod ranges if space is tight.

KEY POINTS

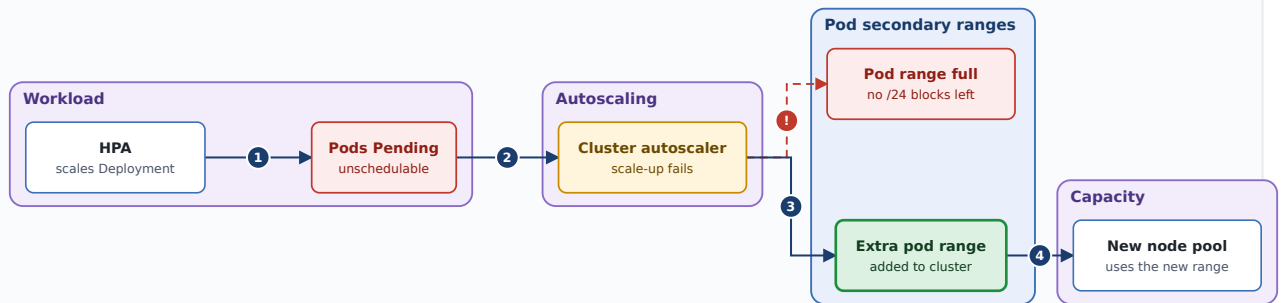
- Pod range need = max nodes x per-node block (/24 at 110 pods).
- Pod IPs are real VPC IPs: no overlaps with peers or on-prem.

CONFIGURATION EXAMPLE

```
gcloud compute networks subnets create gke-eu --network=prod \
  --region=europe-west1 --range=10.30.0.0/22 \
  --secondary-range=pods=10.100.0.0/16,services=10.31.0.0/20
```

Q120 Pods stay Pending during a traffic spike and the cluster autoscaler logs say it cannot add nodes because the pod IP range is exhausted. How do you fix it without rebuilding the cluster?

FLOW DIAGRAM



- 1 More replicas than current nodes fit
- 2 Autoscaler tries to add nodes
- ! IP space exhausted
- 3 Add discontinuous range to the cluster
- 4 New nodes take blocks from new range

EXPLANATION

Confirm the cause in the pod events and the cluster autoscaler visibility logs, which report a `noScaleUp` reason when IP space is exhausted. Add a new secondary range to the subnet and attach it with discontinuous multi-Pod CIDR, then create a node pool that uses it with `--pod-ipv4-range`, or add it cluster-wide with `--additional-pod-ipv4-ranges` so new pools can draw from it. You can also lower max pods per node on new pools so each node uses a smaller block. Recreating the cluster is only needed if the primary node range itself is exhausted and cannot be expanded.

KEY POINTS

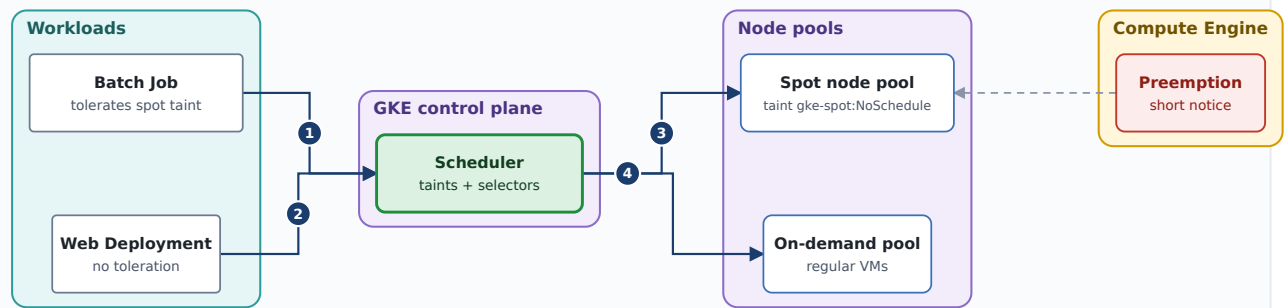
- Discontinuous multi-Pod CIDR adds pod ranges to a live cluster.
- Lower max-pods-per-node shrinks each node's reserved block.

CONFIGURATION EXAMPLE

```
gcloud compute networks subnets update gke-eu --region=europe-west1 \
  --add-secondary-ranges=pods2=10.101.0.0/16
gcloud container node-pools create pool-b --cluster=prod-eu \
  --region=europe-west1 --pod-ipv4-range=pods2 --max-pods-per-node=64
```

Q121 Nightly batch jobs should run on cheap Spot VMs, but web services must never land on them. How do you set up the node pools?

FLOW DIAGRAM



- 1 Toleration + nodeSelector gke-spot=true
- 2 No toleration for the spot taint
- 3 Batch pods placed on Spot nodes
- 4 Web pods only fit on-demand nodes
- Spot VM reclaimed, Job retries pod

EXPLANATION

Create a separate Spot node pool with `--spot` and a taint such as `cloud.google.com/gke-spot=true:NoSchedule`, so pods without a matching toleration are kept off it. Batch pods add the toleration plus a nodeSelector on the `cloud.google.com/gke-spot` label that GKE sets on Spot nodes, so they land only there. Spot VMs can be reclaimed at any time with short notice, so jobs must be retry-safe and checkpoint long work, and Job `backoffLimit` should allow retries. Keep a small on-demand pool for system pods and web services, and enable autoscaling on both pools.

KEY POINTS

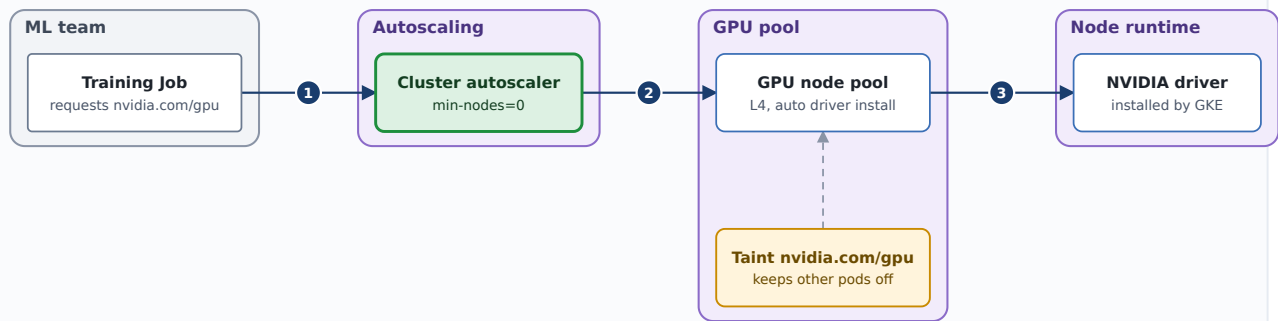
- Taint repels, toleration + nodeSelector attracts.
- Spot workloads must survive sudden node loss.

CONFIGURATION EXAMPLE

```
gcloud container node-pools create spot-batch --cluster=prod-eu \
  --region=europe-west1 --spot --machine-type=e2-standard-8 \
  --enable-autoscaling --min-nodes=0 --max-nodes=20 \
  --node-taints=cloud.google.com/gke-spot=true:NoSchedule
```

Q122 Data scientists need GPU nodes for training, but GPUs are expensive and must scale to zero when idle. How do you configure GKE Standard for this?

FLOW DIAGRAM



- 1 Pending pod asks for a GPU resource
- 2 Scale pool from 0 to 1+ nodes
- 3 Driver installed at boot
- Non-GPU pods are repelled
- When the Job finishes the pool scales back to 0

EXPLANATION

Create a dedicated GPU node pool with `--accelerator` including `gpu-driver-version=latest` so GKE installs the NVIDIA driver, and enable autoscaling with `--min-nodes=0`. GKE taints GPU nodes with `nvidia.com/gpu`, and pods that request `nvidia.com/gpu` get the matching toleration automatically, so ordinary pods do not waste GPU nodes. The cluster autoscaler adds a node when a GPU pod is pending and removes it after it is idle. Check GPU quota in the region and zone availability first, and consider Spot GPUs for fault-tolerant training.

KEY POINTS

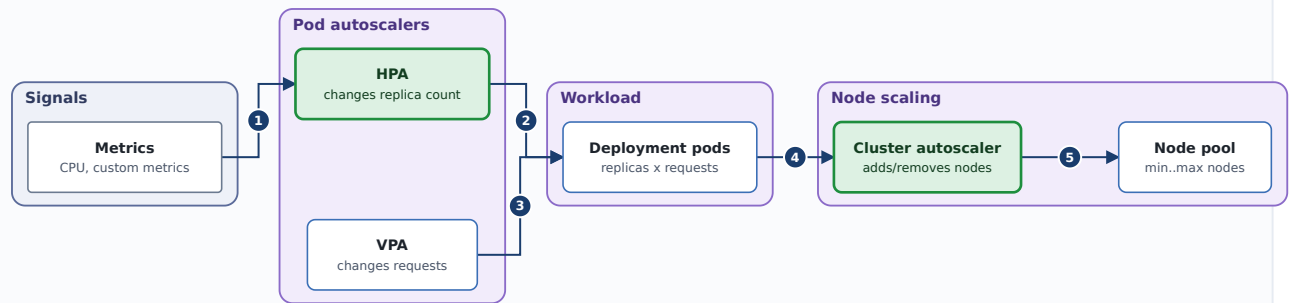
- Request GPUs via `resources.limits nvidia.com/gpu`.
- Scale-to-zero plus GPU quota planning controls cost.

CONFIGURATION EXAMPLE

```
gcloud container node-pools create gpu-l4 --cluster=ml-std \
  --region=europe-west1 --node-locations=europe-west1-b \
  --machine-type=g2-standard-8 \
  --accelerator=type=nvidia-l4,count=1,gpu-driver-version=latest \
  --enable-autoscaling --min-nodes=0 --max-nodes=4
```


Q123 Explain how HPA, VPA and the cluster autoscaler work together on GKE, and what goes wrong if they are combined carelessly.

FLOW DIAGRAM



- 1 Scale out when target utilisation exceeded
- 2 More replicas
- 3 Right-size requests from usage history
- 4 Pending pods trigger scale-up
- 5 Add nodes; remove underused ones later

EXPLANATION

HPA changes the number of replicas based on CPU, memory or custom metrics, VPA changes pod requests based on observed usage, and the cluster autoscaler adds nodes when pods cannot be scheduled and removes underused nodes. The cluster autoscaler reacts to requests, not actual usage, so accurate requests matter. Do not let HPA and VPA act on the same CPU or memory metric, because VPA raising requests lowers the utilisation HPA sees and they fight; use VPA in recommendation mode or HPA on custom metrics instead. Set PodDisruptionBudgets so scale-down evictions stay safe, and set node pool min and max limits to cap cost.

KEY POINTS

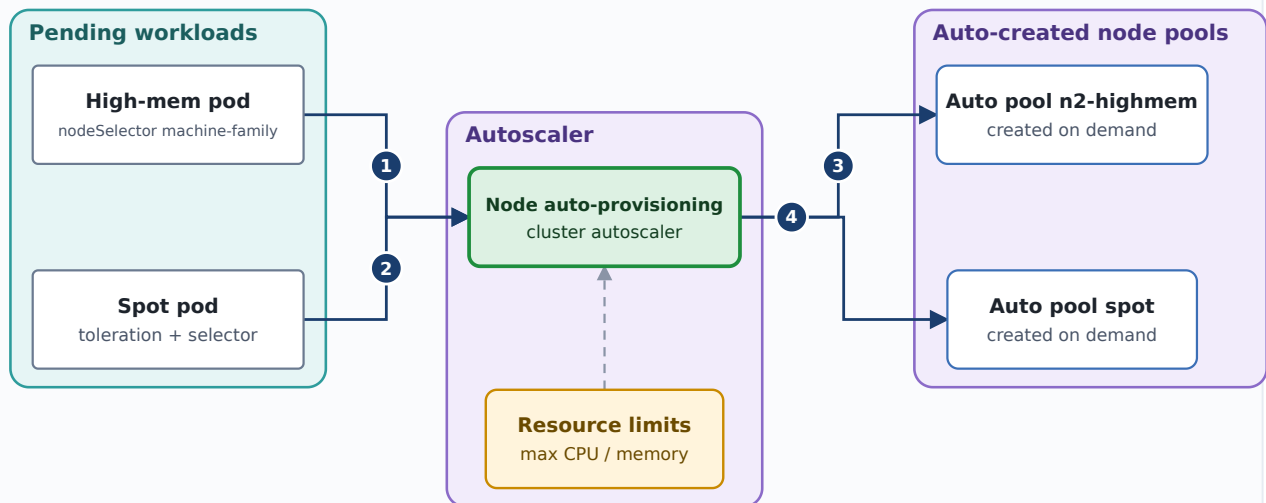
- Cluster autoscaler works on requests, not live usage.
- Avoid HPA and VPA both on CPU/memory for the same workload.

CONFIGURATION EXAMPLE

```
kubectl autoscale deployment web --cpu-percent=60 --min=3 --max=30
gcloud container clusters update prod-eu --region=europe-west1 \
  --enable-vertical-pod-autoscaling
```

Q124 Teams deploy workloads with very different shapes (high-memory, ARM, Spot) and the platform team is tired of creating node pools by hand. What does node auto-provisioning do and how do you limit it?

FLOW DIAGRAM



- 1 No existing pool fits the request
- 3 Creates a pool matching the requirements
- Stops provisioning past cluster-wide limits

EXPLANATION

Node auto-provisioning extends the cluster autoscaler so that, when pending pods fit no existing pool, it creates a new node pool matching their requests, nodeSelectors, tolerations and GPU needs, and deletes it when empty. You must set cluster-wide minimum and maximum CPU and memory, plus GPU limits, which cap total spend. Use auto-provisioning defaults to control the service account, OAuth scopes, boot disk and upgrade settings for created pools. It reduces pool sprawl for teams while Autopilot takes the same idea further by managing all nodes.

KEY POINTS

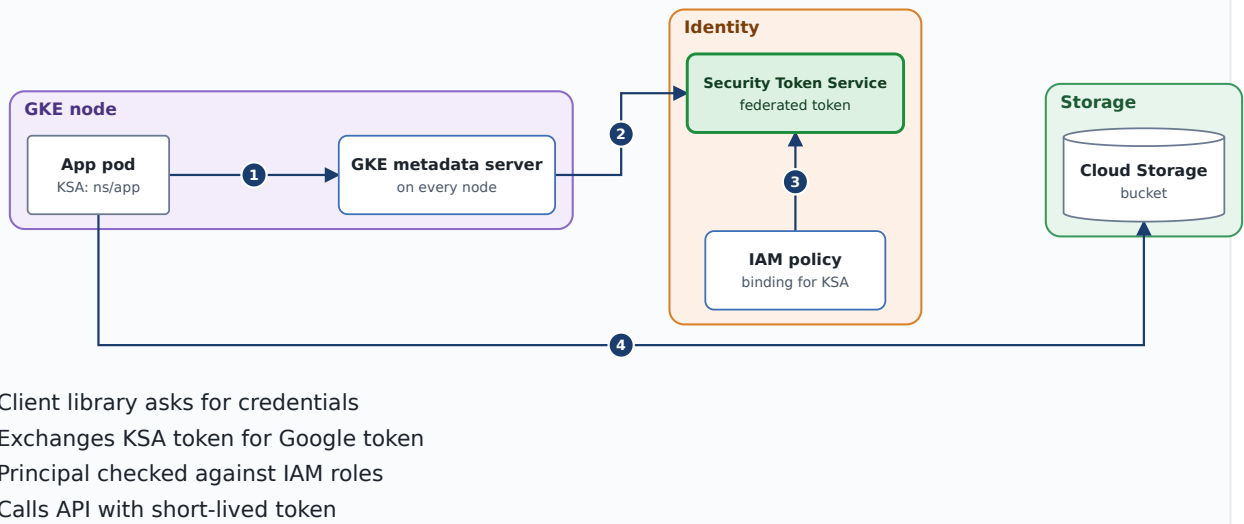
- NAP creates and deletes pools from pending pod requirements.
- Cluster-wide CPU/memory limits are mandatory guardrails.

CONFIGURATION EXAMPLE

```
gcloud container clusters update prod-eu --region=europe-west1 \
  --enable-autoprovisioning \
  --min-cpu=4 --max-cpu=400 --min-memory=16 --max-memory=1600 \
  --autoprovisioning-service-account=gke-nodes@proj.iam.gserviceaccount.com
```

Q125 A pod needs to read from a Cloud Storage bucket, and the current setup mounts a JSON service-account key as a Secret. How do you replace it with Workload Identity?

FLOW DIAGRAM



EXPLANATION

Enable Workload Identity Federation for GKE on the cluster with a workload pool of `PROJECT_ID.svc.id.goog` and run node pools with the GKE metadata server. Grant IAM roles either directly to the Kubernetes service account principal, or bind the KSA to a Google service account with `roles/iam.workloadIdentityUser` and annotate the KSA. Pods using that KSA then get short-lived tokens from the metadata server, with no key files to leak or rotate. Delete the old JSON key and the Secret, and use a separate KSA per workload for least privilege.

KEY POINTS

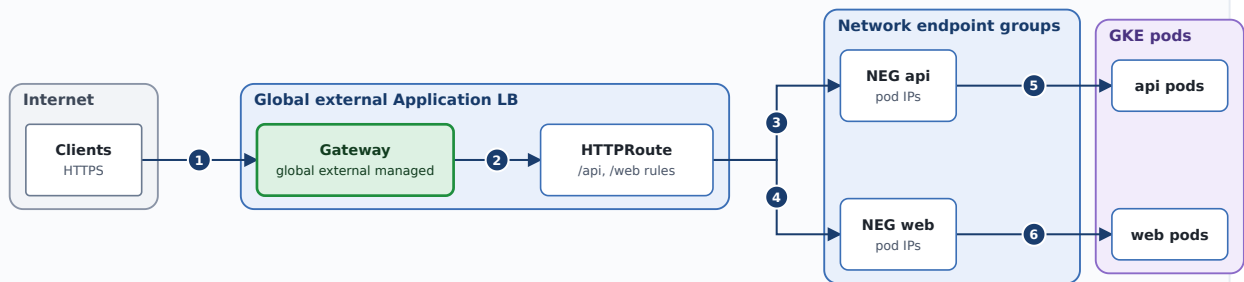
- No long-lived keys; tokens are short-lived and scoped per KSA.
- Node pools need `--workload-metadata=GKE_METADATA`.

CONFIGURATION EXAMPLE

```
gcloud container clusters update prod-eu --region=europe-west1 \
  --workload-pool=my-proj.svc.id.goog
gcloud iam service-accounts add-iam-policy-binding gcs-reader@my-proj.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser \
  --member="serviceAccount:my-proj.svc.id.goog[shop/app]"
kubectl annotate sa app -n shop \
  iam.gke.io/gcp-service-account=gcs-reader@my-proj.iam.gserviceaccount.com
```

Q126 You need HTTPS routing for several services with path-based rules, managed certificates and traffic sent directly to pods. How do you expose them on GKE?

FLOW DIAGRAM



- 1 TLS ends at Google edge, managed cert
- 2 Host/path matching
- 3 Backend = Service api
- 4 Backend = Service web
- 5 LB sends straight to pod IPs

EXPLANATION

Enable the Gateway API on the cluster and create a Gateway with the `gke-l7-global-external-managed` GatewayClass, which provisions a global external Application Load Balancer. HTTPRoutes attach to the Gateway and map hosts and paths to Services, and application teams can own their routes while the platform team owns the Gateway. GKE uses container-native load balancing through NEGs, so the load balancer sends traffic to pod IPs, health-checks pods directly and avoids the extra node hop. Classic GKE Ingress also builds an Application Load Balancer with NEGs, but Gateway API gives cleaner role separation and traffic splitting.

KEY POINTS

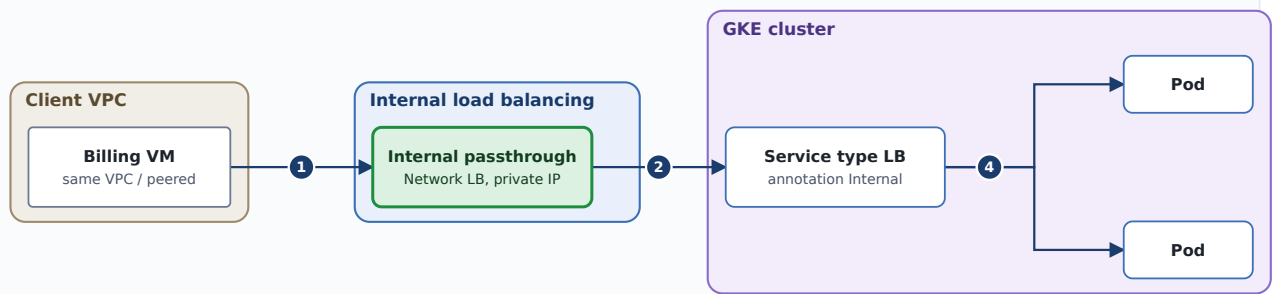
- Gateway = infra owner, HTTPRoute = app owner.
- NEGs mean LB to pod directly, better health checks and latency.

CONFIGURATION EXAMPLE

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: {name: shop, namespace: shop}
spec:
  parentRefs: [{name: external-gw, namespace: infra}]
  rules:
    - matches: [{path: {type: PathPrefix, value: /api}}]
      backendRefs: [{name: api, port: 8080}]
```

Q127 A VM-based billing system in the same VPC must call a GKE service privately, with no public IP. How do you expose it?

FLOW DIAGRAM



- 1 Calls internal IP from the subnet
- 2 Forwarded to backend nodes/endpoints
- For L7 routing use Gateway class gke-l7-ilb (needs a proxy-only subnet)

EXPLANATION

Create a Service of type LoadBalancer with the annotation `networking.gke.io/load-balancer-type: "Internal"`, which provisions an internal passthrough Network Load Balancer with a private IP from the subnet. Clients in the same VPC, peered VPCs or on-prem via VPN/Interconnect can reach it, and you can enable global access if clients are in other regions. For HTTP routing, TLS and header rules, use an internal Gateway with the `gke-l7-ilb` class, which needs a proxy-only subnet in the region. Lock the source ranges with `loadBalancerSourceRanges` or firewall rules.

KEY POINTS

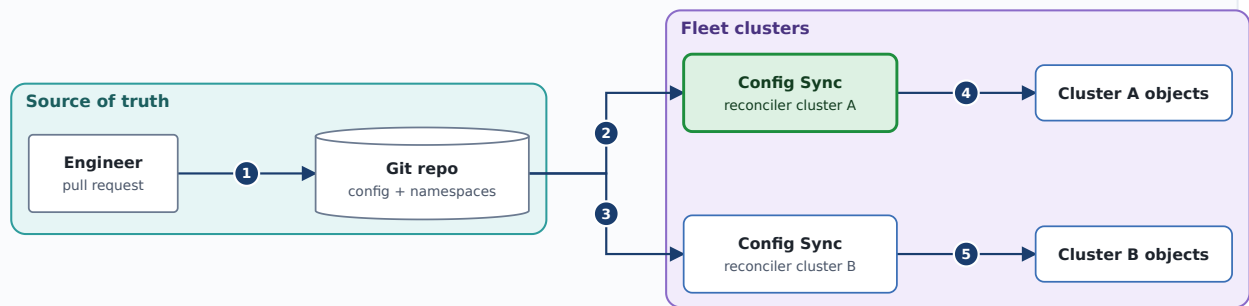
- L4 private: Service annotation Internal.
- L7 private: Gateway `gke-l7-ilb` + proxy-only subnet.

CONFIGURATION EXAMPLE

```
apiVersion: v1
kind: Service
metadata:
  name: billing-api
  annotations:
    networking.gke.io/load-balancer-type: "Internal"
spec:
  type: LoadBalancer
  selector: {app: billing-api}
  ports: [{port: 443, targetPort: 8443}]
```

Q128 Twelve GKE clusters drift because engineers run `kubectl apply` by hand. How do you enforce a GitOps model?

FLOW DIAGRAM



- ❶ Reviewed, merged change
- ❷ Reconciler pulls repo (Git/OCI)
- ❸ Applies and reverts manual drift
- ❹ Applies and reverts manual drift
- ❺ Applies and reverts manual drift

EXPLANATION

Register the clusters in a fleet and enable Config Sync, then define a RootSync that points each cluster at a Git repository or OCI image holding cluster-wide config, with RepoSync objects for team namespaces. The reconciler continuously applies the repo state and reverts manual changes, so drift disappears and Git history becomes the audit log. Remove broad write RBAC from humans so changes go through pull requests with review and CI validation. Argo CD on GKE is a common alternative with a UI and application-level sync, and the same principles apply.

KEY POINTS

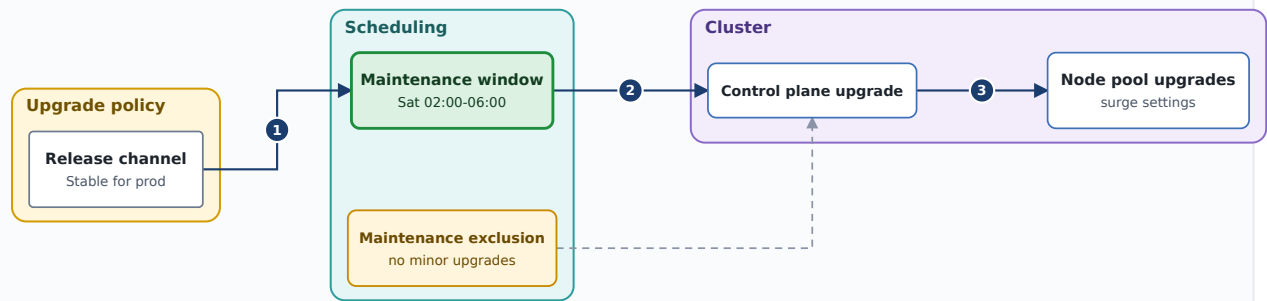
- Git is the only write path; the reconciler corrects drift.
- Use RootSync for platform config, RepoSync per team.

CONFIGURATION EXAMPLE

```
apiVersion: configsync.gke.io/v1beta1
kind: RootSync
metadata: {name: root-sync, namespace: config-management-system}
spec:
  sourceFormat: unstructured
  git:
    repo: https://github.com/acme/gke-config
    branch: main
    dir: clusters/prod
    auth: none
```

Q129 Production GKE clusters got a minor-version upgrade during peak retail season. How do you control when upgrades happen?

FLOW DIAGRAM



- 1 New version becomes available
- 2 Upgrade starts only inside the window
- 3 Nodes follow the control plane
- Blocks minor upgrades during peak season

EXPLANATION

Enrol the cluster in a release channel (Stable for production, Regular for staging) so versions arrive in a predictable order, and test on staging first. Add a recurring maintenance window so automatic upgrades only start in low-traffic hours. For peak periods add a maintenance exclusion: "no upgrades" is limited to 30 days, while "no minor upgrades" and "no minor or node upgrades" can last longer, up to the version's end of support. Security patches still arrive under the minor exclusion, which is usually what you want.

KEY POINTS

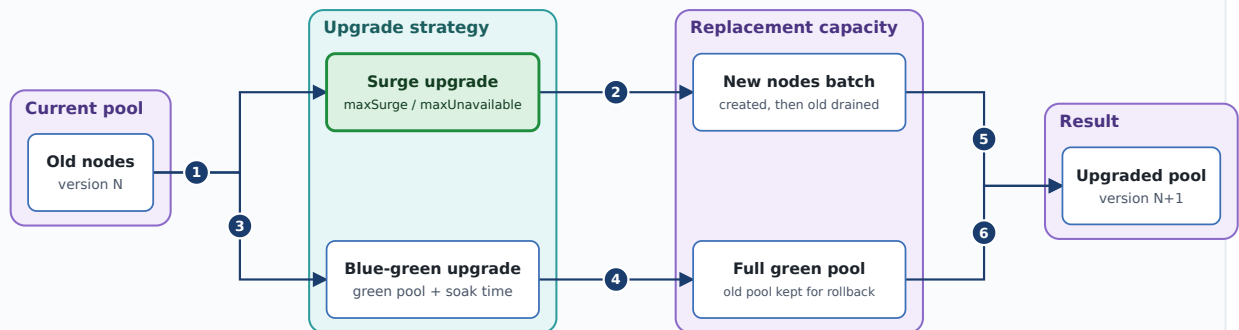
- Release channel picks the version stream; windows pick the time.
- Exclusions freeze upgrades around critical events.

CONFIGURATION EXAMPLE

```
gcloud container clusters update prod-eu --region=europe-west1 \
  --maintenance-window-start=2026-01-03T02:00:00Z \
  --maintenance-window-end=2026-01-03T06:00:00Z \
  --maintenance-window-recurrence="FREQ=WEEKLY;BYDAY=SA"
gcloud container clusters update prod-eu --region=europe-west1 \
  --add-maintenance-exclusion-name=peak \
  --add-maintenance-exclusion-start=2026-11-20T00:00:00Z \
  --add-maintenance-exclusion-end=2027-01-05T00:00:00Z \
  --add-maintenance-exclusion-scope=no_minor_upgrades
```

Q130 Node pool upgrades take hours and occasionally cause capacity dips. Compare surge upgrades and blue-green upgrades and how you would tune them.

FLOW DIAGRAM



- ❶ Default, rolling a few nodes at a time
- ❷ maxSurge=3 speeds up, keeps capacity
- ❸ Critical pools needing fast rollback
- ❹ Cordon blue, drain in batches, soak

EXPLANATION

Surge upgrades replace nodes in rolling batches: `maxSurge` adds extra new nodes and `maxUnavailable` allows nodes to go down without replacement. Raising `maxSurge` speeds up the upgrade and keeps capacity, at the cost of temporary extra VMs and quota, while `maxUnavailable=0` avoids capacity dips. Blue-green upgrades create a full set of new nodes, drain the old pool in batches, and keep it during a soak period so you can roll back quickly, but they need double capacity and quota. Pick blue-green for critical stateful or latency-sensitive pools and tuned surge for the rest.

KEY POINTS

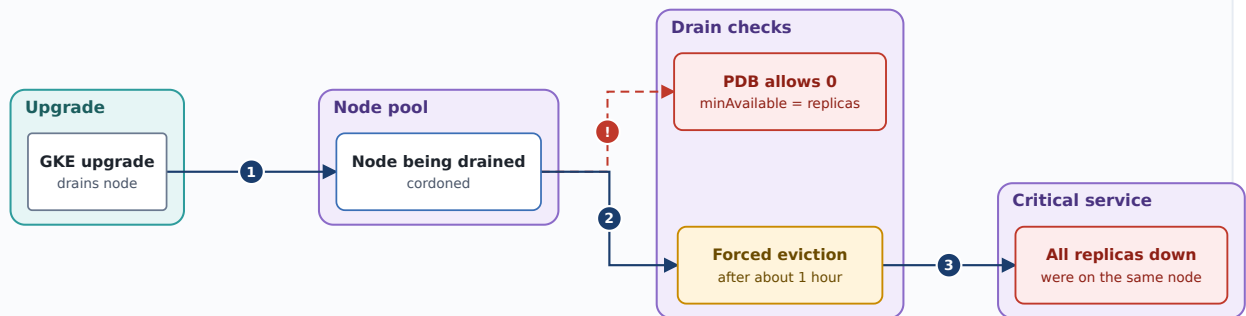
- maxSurge up + maxUnavailable 0 = faster with no capacity loss.
- Blue-green gives fast rollback but needs 2x quota.

CONFIGURATION EXAMPLE

```
gcloud container node-pools update default-pool --cluster=prod-eu \
  --region=europe-west1 --max-surge-upgrade=3 --max-unavailable-upgrade=0
```


Q131 A node pool upgrade stalls for an hour on one node, then pods of a critical service are evicted anyway and it has an outage. What happened and how do you prevent it?

FLOW DIAGRAM



- ❶ Evict pods respecting PDBs
- ❷ ! Eviction refused, 0 disruptions allowed
- ❸ Upgrade waits, then stops honouring PDB
- ❹ All replicas deleted together

EXPLANATION

The PodDisruptionBudget allowed zero disruptions (for example `minAvailable` equal to the replica count), so drain eviction was refused; GKE respects PDBs during node upgrades for up to about one hour and then evicts pods anyway. Because all replicas were on the draining node, they went down together. Fix it by running at least two or three replicas spread across nodes and zones with anti-affinity or topology spread, and setting PDBs that allow at least one disruption, such as `maxUnavailable: 1`. Check `kubectl get pdb -A` before upgrades and alert on budgets with ALLOWED DISRUPTIONS of 0.

KEY POINTS

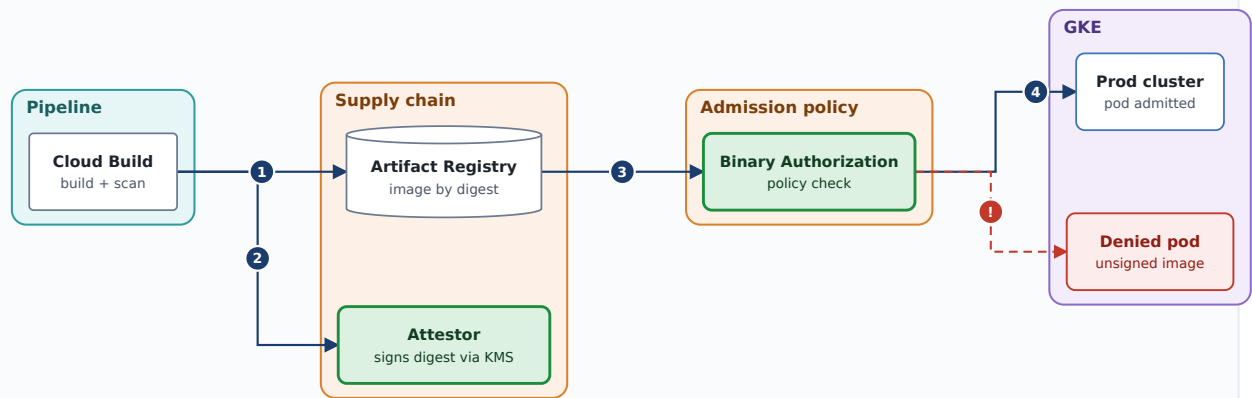
- A PDB that allows 0 disruptions does not protect you; it delays and then fails.
- Replicas > 1 plus spread plus `maxUnavailable: 1`.

CONFIGURATION EXAMPLE

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: checkout-pdb}
spec:
  maxUnavailable: 1
  selector:
    matchLabels: {app: checkout}
```

Q132 Only images built and scanned by the official CI pipeline may run in production GKE. How do you enforce this?

FLOW DIAGRAM



- 1 Push image, record digest
- 2 Create attestation after tests pass
- 3 Deploy references image digest
- 4 Attestation found, allowed
- ! No attestation, blocked

EXPLANATION

Enable Binary Authorization on the cluster and write a policy that requires an attestation from your CI attester for production. After tests and vulnerability scans pass, the pipeline signs the image digest with a Cloud KMS key and creates an attestation. At pod admission GKE checks the policy, so images without a valid attestation, or referenced by mutable tag instead of digest, are rejected. Start in dry-run mode to see audit logs of what would be blocked, and document a break-glass process for emergencies.

KEY POINTS

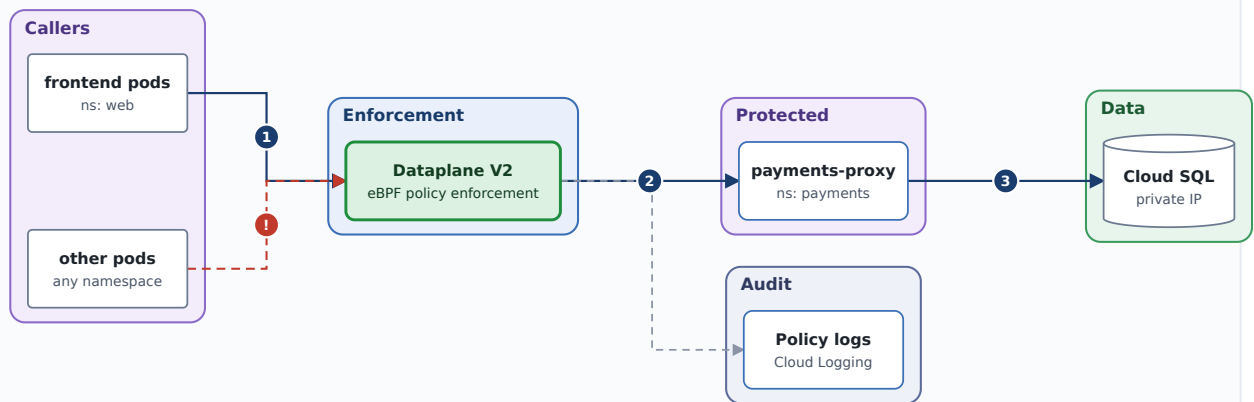
- Attest digests, not tags.
- Roll out in dry-run first, then enforce.

CONFIGURATION EXAMPLE

```
gcloud container clusters update prod-eu --region=europe-west1 \
  --binauthz-evaluation-mode=PROJECT_SINGLETON_POLICY_ENFORCE
```

Q133 Any pod in the cluster can currently call the payments database proxy. How do you restrict pod-to-pod traffic on GKE?

FLOW DIAGRAM



EXPLANATION

Use Dataplane V2, which enforces Kubernetes NetworkPolicy with eBPF and is enabled when the cluster is created (it is the default on Autopilot). Apply a default-deny ingress policy in the payments namespace, then allow only pods labelled as frontend from the web namespace on the needed port. Enable network policy logging to see allowed and denied connections while you roll out. Remember egress to Google APIs and DNS if you also add default-deny egress.

KEY POINTS

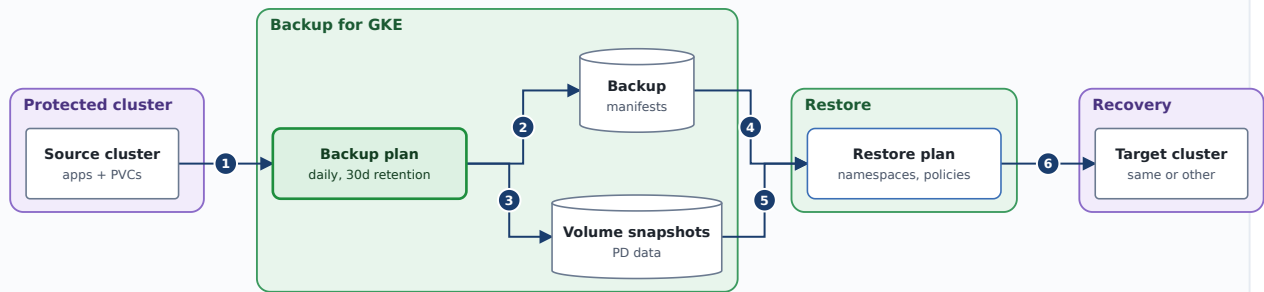
- Default deny, then allow by namespace and pod labels.
- Dataplane V2 is chosen at cluster creation.

CONFIGURATION EXAMPLE

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-frontend, namespace: payments}
spec:
  podSelector: {matchLabels: {app: payments-proxy}}
  ingress:
    - from:
      - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: web}}
        podSelector: {matchLabels: {app: frontend}}
  ports: [{port: 5432}]
```

Q134 A team runs stateful apps with PersistentVolumes on GKE and needs daily backups and the ability to restore into another cluster. What do you use?

FLOW DIAGRAM



- ❶ Agent exports selected resources
- ❷ Kubernetes objects stored
- ❸ Volume data via disk snapshots
- ❹ Restore config and volumes

EXPLANATION

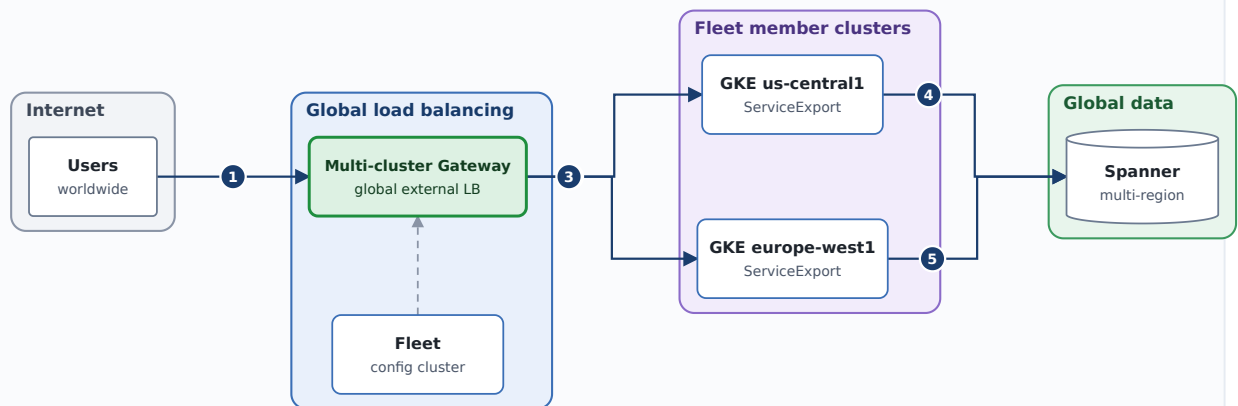
Enable the Backup for GKE add-on and create a backup plan that selects namespaces or applications, runs on a cron schedule, keeps backups for a set number of days and includes volume data and Secrets when required. Backups store Kubernetes resources and snapshot persistent disks. A restore plan defines the target cluster, which namespaces to restore and how to handle conflicts, so you can recover in place or into a new cluster in another region. Test restores regularly, because an untested backup is not a recovery plan, and keep application-consistent hooks or database-native backups for databases.

KEY POINTS

- Captures both manifests and PV data.
- Practise restores into a separate cluster.

Q135 A global app runs in GKE clusters in `us-central1` and `eu-west-1`. How do you give it one global entry point with failover between clusters?

FLOW DIAGRAM



- 1 One anycast IP
- 2 Nearest healthy cluster
- 3 Failover if a region is unhealthy
- Gateway + HTTPRoute live on config cluster

EXPLANATION

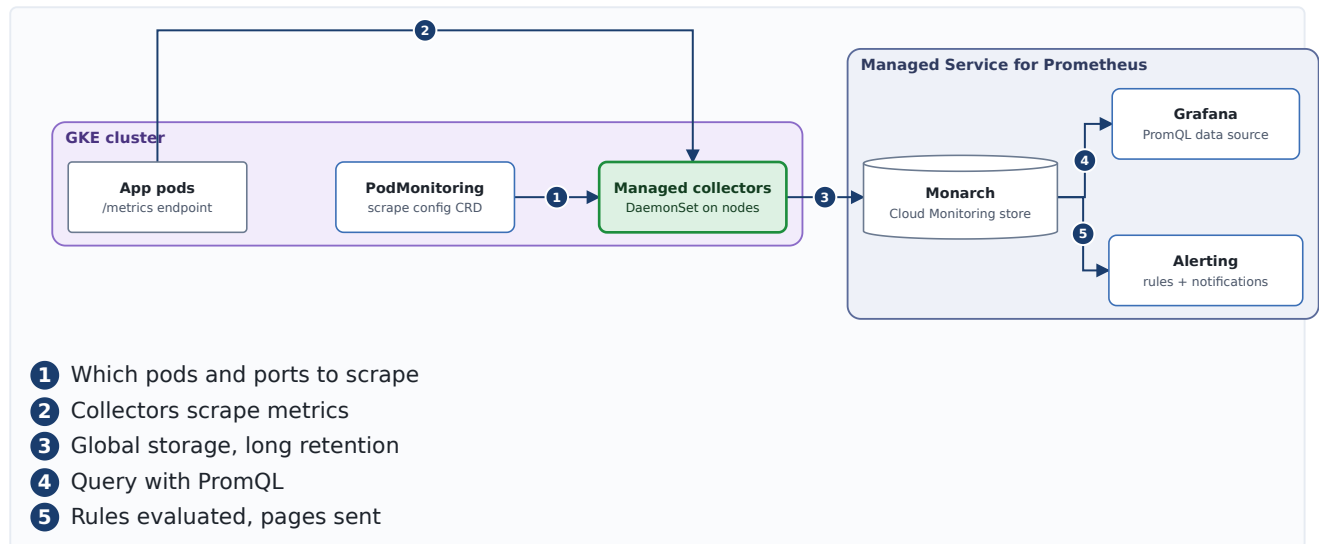
Register both clusters in a fleet, enable multi-cluster Services and multi-cluster Gateway, and pick one cluster as the config cluster. Each cluster exports the Service with a ServiceExport, and on the config cluster you create a Gateway using the `gke-l7-global-external-managed-mc` class with an HTTPRoute pointing at the ServiceImport. The global load balancer sends users to the nearest healthy cluster and fails over automatically when backends in one region are unhealthy. The data layer must also be multi-region, for example Spanner, or failover only moves the problem.

KEY POINTS

- Fleet + ServiceExport + multi-cluster GatewayClass.
- Keep the config cluster itself highly available.

Q136 Teams want Prometheus metrics and Grafana dashboards for GKE apps without running and scaling their own Prometheus servers. What do you propose?

FLOW DIAGRAM



EXPLANATION

Use Google Cloud Managed Service for Prometheus with managed collection, which is enabled by default on recent GKE clusters. Teams declare `PodMonitoring` resources (or `ClusterPodMonitoring`) and managed collectors scrape the targets and write to Monarch, the Cloud Monitoring backend, so there is no Prometheus storage to scale. Query with PromQL from Cloud Monitoring or Grafana, and define alerting rules with the Rules CRDs or Cloud Monitoring alert policies. Watch cardinality, because billing is based on samples ingested.

KEY POINTS

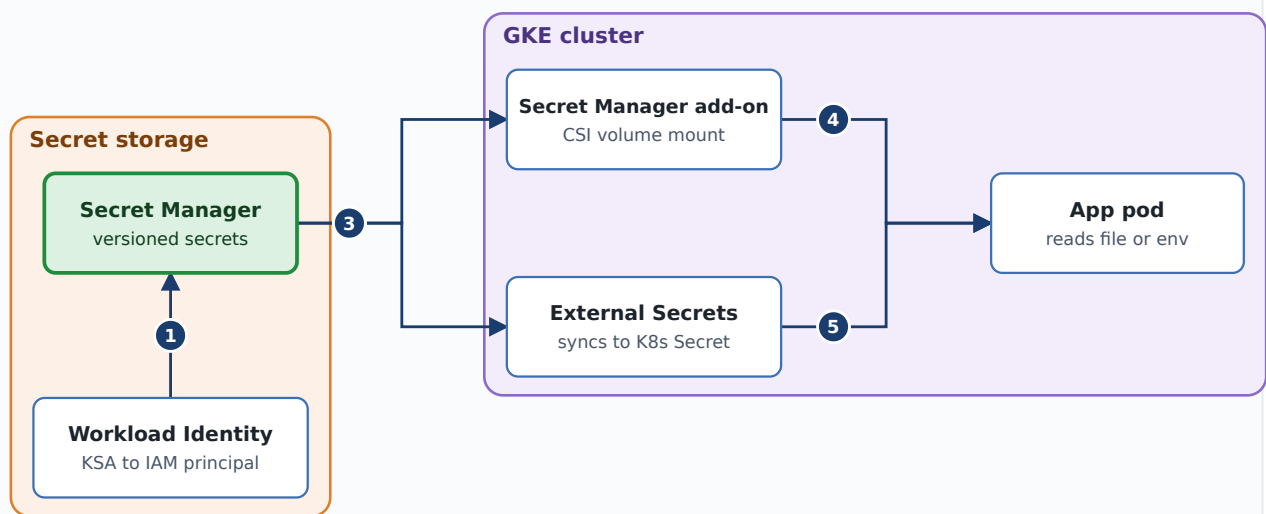
- PodMonitoring replaces hand-written scrape configs.
- Cost follows ingested samples: control labels and scrape intervals.

CONFIGURATION EXAMPLE

```
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata: {name: web, namespace: shop}
spec:
  selector: {matchLabels: {app: web}}
  endpoints:
    - port: metrics
      interval: 30s
```

Q137 Database passwords are stored in Git as base64 Kubernetes Secrets. How do you move to Secret Manager for GKE workloads?

FLOW DIAGRAM



- 1 Pod identity granted secretAccessor
- 2 Mounted as files at pod start
- 3 Alternative: sync to Secret objects

EXPLANATION

Store the values in Secret Manager and delete them from Git, rotating any that were exposed. The GKE Secret Manager add-on mounts secrets into pods as files through the Secrets Store CSI driver using a `SecretProviderClass`, and the pod's Workload Identity principal needs `roles/secretmanager.secretAccessor` on those secrets. If apps need environment variables or native Kubernetes Secrets, External Secrets Operator can sync Secret Manager values into Secrets on a schedule. Base64 is encoding, not encryption, so also enable application-layer secrets encryption with Cloud KMS for any Secrets that remain in etcd.

KEY POINTS

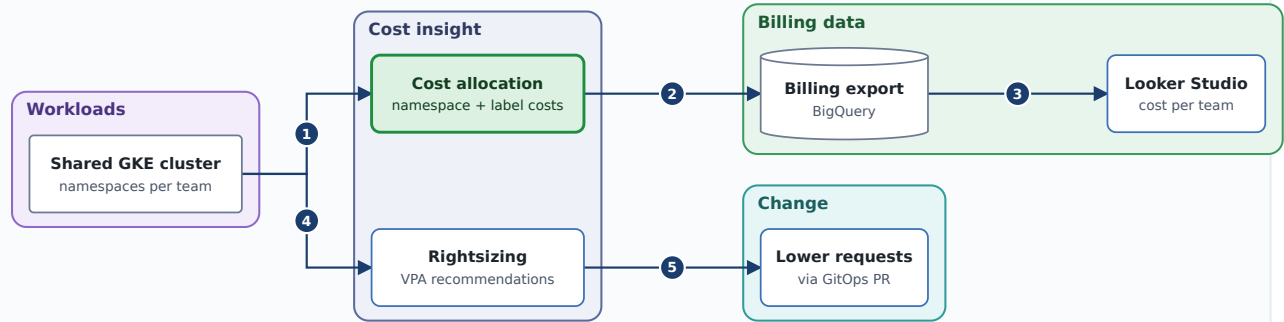
- Git never holds secret values.
- Grant `secretAccessor` per secret to the workload identity.

CONFIGURATION EXAMPLE

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: {name: db-creds, namespace: shop}
spec:
  provider: gke
  parameters:
    secrets: |
      - resourceName: "projects/my-proj/secrets/db-pass/versions/latest"
        path: "db-pass.txt"
```

Q138 Finance wants GKE costs broken down by team, and the platform team suspects pods request far more CPU than they use. How do you tackle both?

FLOW DIAGRAM



- 1 Usage attributed by namespace/labels
- 2 Detailed billing export
- 3 Chargeback reports
- 4 Compare requests vs real usage
- 5 Apply smaller requests safely

EXPLANATION

Enable GKE cost allocation so the detailed Cloud Billing export to BigQuery breaks down cluster cost by namespace and Kubernetes labels, then build chargeback reports per team. For rightsizing, compare requests with actual usage using the GKE workload cost and rightsizing views or VPA in recommendation mode, and lower requests through normal code review. Use the `optimize-utilization` autoscaling profile, Spot pools for tolerant workloads and committed use discounts for the steady baseline. Autopilot bills requests directly, so rightsizing matters there as well.

KEY POINTS

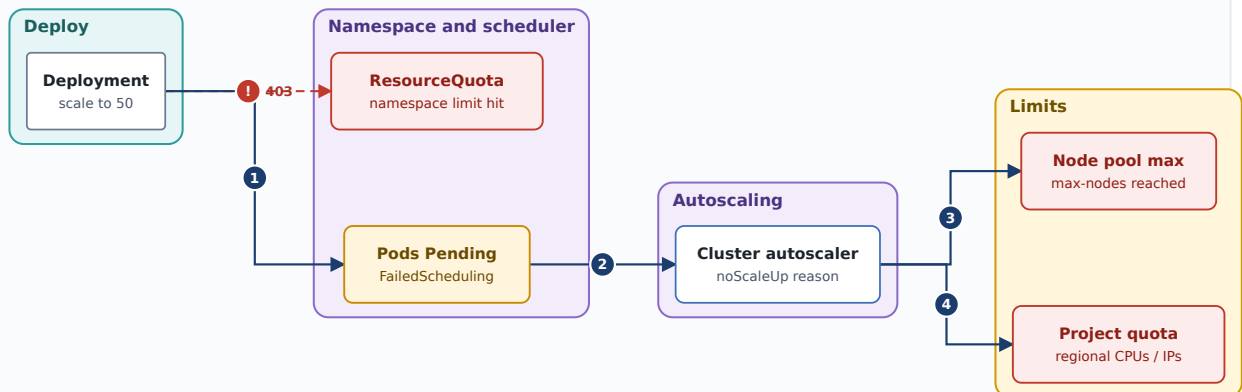
- Cost allocation + BigQuery export = chargeback by namespace/label.
- Requests drive both scheduling and cost.

CONFIGURATION EXAMPLE

```
gcloud container clusters update prod-eu --region=europe-west1 \
  --enable-cost-allocation --autoscaling-profile=optimize-utilization
```


Q139 A new Deployment shows some pods Pending and the ReplicaSet cannot create others, even though node pools have autoscaling enabled. How do you troubleshoot?

FLOW DIAGRAM



- ❗ 403 forbidden: exceeded quota
- ➊ Pods created but not placed
- ➋ Scale-up attempted
- ➌ Pool at its max size
- ➍ Compute Engine quota exceeded

EXPLANATION

First separate two failure modes: if the ReplicaSet events show `exceeded quota`, a namespace ResourceQuota blocked pod creation; if pods exist but are Pending, read their FailedScheduling events. For Pending pods, check the cluster autoscaler visibility logs in Cloud Logging, which give a `noScaleUp` reason such as node pool max size reached, regional CPU or IP address quota exceeded, pod IP range exhausted, or no pool that satisfies the pod's selectors and requests. Fix by raising the ResourceQuota or pool max, requesting more Compute Engine quota, or adjusting requests and node selectors. Oversized requests that no machine type can fit also show up here.

KEY POINTS

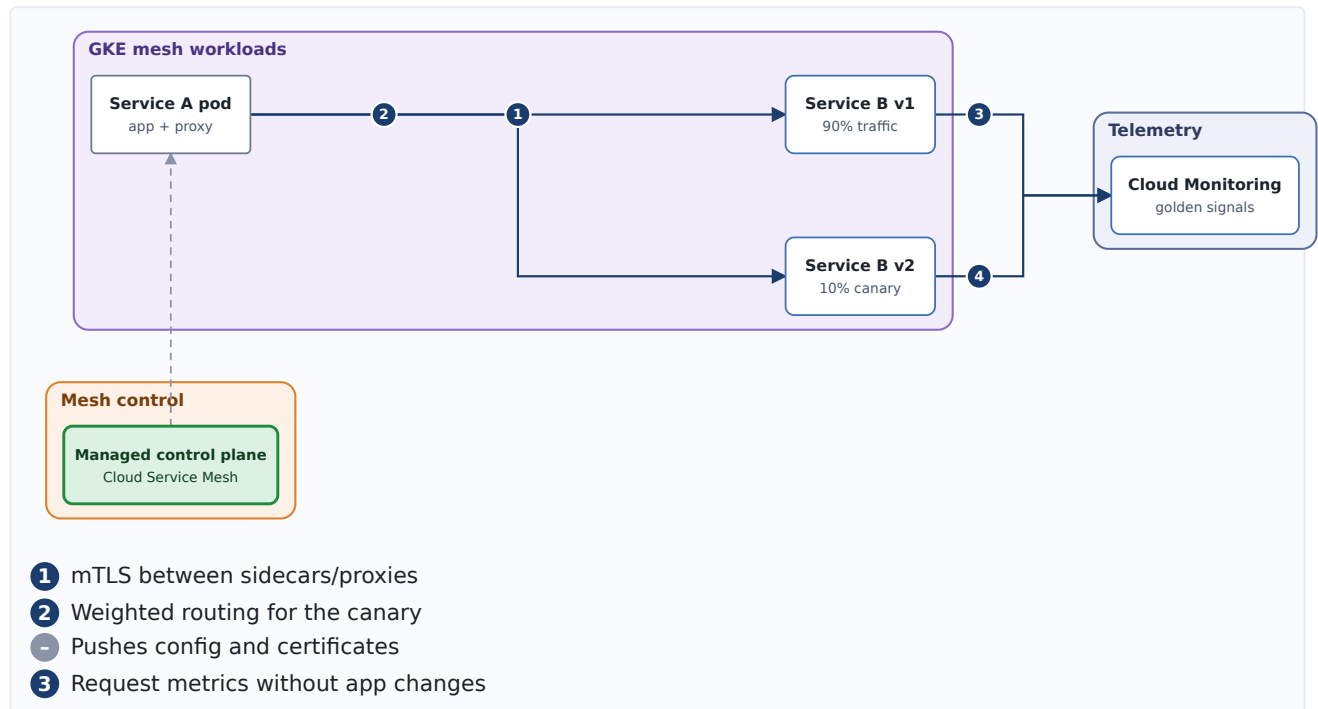
- Quota error at creation vs Pending at scheduling are different problems.
- Autoscaler visibility logs name the exact blocker.

CONFIGURATION EXAMPLE

```
kubectl describe rs -n shop -l app=api | grep -i quota
kubectl get events -n shop --field-selector reason=FailedScheduling
gcloud compute regions describe europe-west1 \
  --format="table(quotas.metric,quotas.usage,quotas.limit)"
```

Q140 Security wants mTLS between all services and the SRE team wants per-service latency metrics and canary traffic splits. When would you add Cloud Service Mesh on GKE?

FLOW DIAGRAM



EXPLANATION

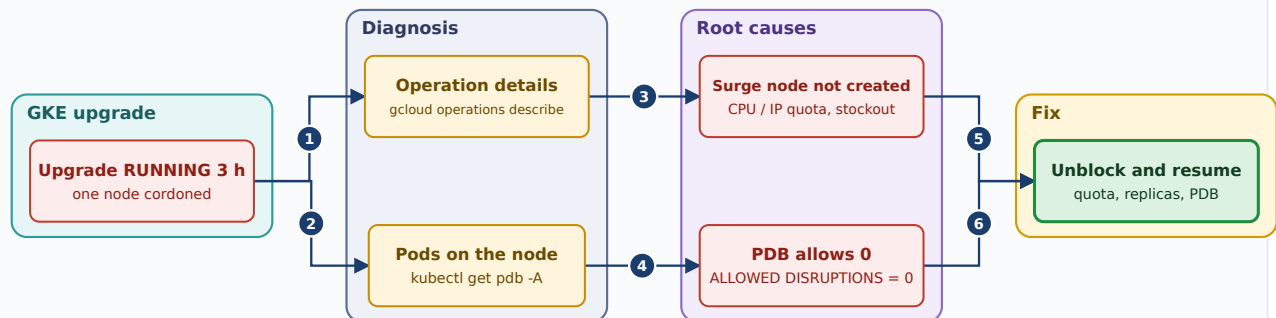
Cloud Service Mesh is Google's managed Istio-based mesh: a managed control plane issues workload certificates and pushes routing config to proxies next to each pod. It gives automatic mTLS with authorization policies, traffic splitting for canaries, retries and timeouts, and consistent request metrics and traces without code changes. Add it when you have many services and need these controls uniformly; for a handful of services, NetworkPolicy plus Gateway API traffic splitting may be enough. Expect extra resource use for proxies and an operational learning curve, so roll it out namespace by namespace.

KEY POINTS

- Mesh = mTLS, traffic policy and telemetry outside app code.
- Adopt per namespace; weigh the proxy overhead.

Q141**Troubleshooting**

A GKE node pool upgrade has shown RUNNING for three hours and one node stays cordoned with pods still on it. How do you find what is blocking the upgrade?

FLOW DIAGRAM

- ❶ Status message of the UPGRADE_NODES operation
- ❷ Which pods still sit on the draining node
- ❸ Errors creating the extra surge VM
- ❹ Eviction refused by a disruption budget

EXPLANATION

A surge upgrade creates a new node, then cordons and drains an old one, so it stalls when either step cannot finish. `gcloud container operations describe` shows the operation's status message; errors such as quota exceeded, a zone resource stockout or an exhausted pod range mean the surge node cannot be created. If the surge node exists, look at the pods left on the cordoned node: a `PodDisruptionBudget` with 0 allowed disruptions (one replica, or `minAvailable` equal to replicas) blocks eviction, and GKE waits up to about an hour per node before forcing it. Fix the quota or use `--max-surge-upgrade=0 --max-unavailable-upgrade=1`, scale the blocked workload to 2+ replicas with `maxUnavailable: 1`, and the upgrade continues.

KEY POINTS

- Before an upgrade, alert on any PDB with ALLOWED DISRUPTIONS 0.
- Surge upgrades need spare CPU quota and pod IPs for the extra nodes.

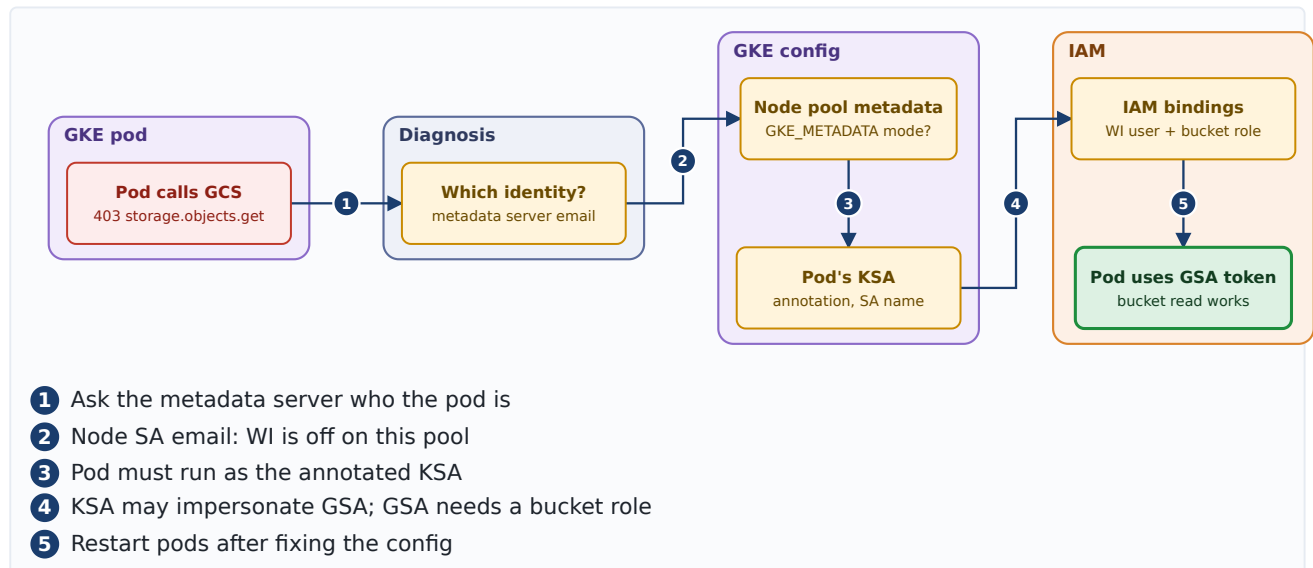
CONFIGURATION EXAMPLE

```

gcloud container operations list --region=europe-west1 \
  --filter="operationType=UPGRADE_NODES AND status=RUNNING"
gcloud container operations describe operation-1727 --region=europe-west1
kubectl get pdb -A          # look for ALLOWED DISRUPTIONS 0
kubectl get pods -A --field-selector spec.nodeName=gke-prod-pool-a-7h2k
kubectl scale deploy/checkout -n shop --replicas=3
gcloud container node-pools update pool-a --cluster=prod --region=europe-west1 \
  --max-surge-upgrade=0 --max-unavailable-upgrade=1
  
```

Q142**Troubleshooting**

A pod that should use Workload Identity gets 403 ... does not have storage.objects.get access from Cloud Storage, and the error names the node's compute service account. What do you check?

FLOW DIAGRAM**EXPLANATION**

First ask the metadata server from inside the pod which identity it gets. If it returns the node's service account, Workload Identity is not active for that pod: the cluster needs a workload pool (`PROJECT_ID.svc.id.goog`) and the node pool must run with `--workload-metadata=GKE_METADATA`, which older or newly added pools may lack. If the identity is right but calls still fail, check that the pod's `serviceAccountName` is the annotated Kubernetes service account, that the Google service account grants `roles/iam.workloadIdentityUser` to `serviceAccount:PROJECT_ID.svc.id.goog[NAMESPACE/KSA]`, and that it has a role such as `roles/storage.objectViewer` on the bucket. You can also skip the Google service account and grant the bucket role directly to the Kubernetes service account's `principal://` identifier.

KEY POINTS

- Pools created before Workload Identity was enabled keep the old metadata mode until updated.
- Remove node-SA bucket access so a misconfigured pod fails loudly instead of using it.

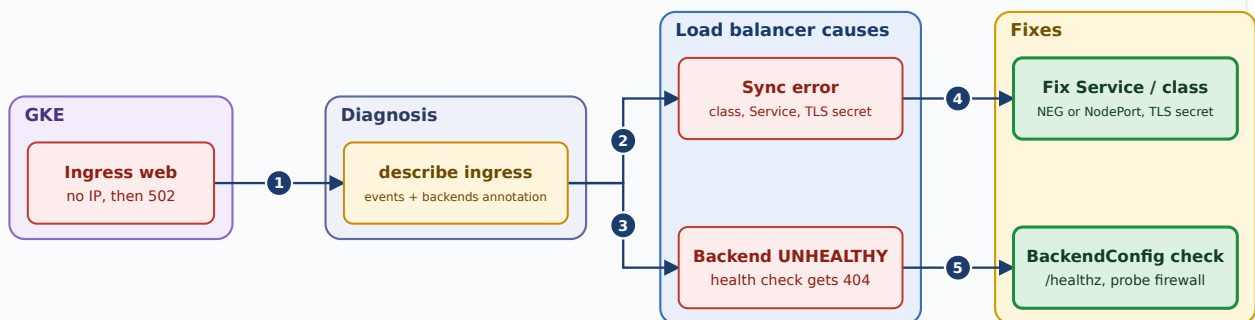
CONFIGURATION EXAMPLE

```

kubectl exec -n shop app-7f9c -- curl -s -H "Metadata-Flavor: Google" \
  http://169.254.169.254/computeMetadata/v1/instance/service-accounts/default/email
gcloud container node-pools update pool-a --cluster=prod --region=europe-west1 \
  --workload-metadata=GKE_METADATA
kubectl annotate sa app -n shop iam.gke.io/gcp-service-account=app-gsa@prod.iam.gserviceaccount.com
gcloud iam service-accounts add-iam-policy-binding app-gsa@prod.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser --member="serviceAccount:prod.svc.id.goog[shop/app]"
gcloud storage buckets add-iam-policy-binding gs://prod-reports \
  --member=serviceAccount:app-gsa@prod.iam.gserviceaccount.com --role=roles/storage.objectViewer
  
```

Q143**Troubleshooting**

A GKE Ingress shows no ADDRESS for 20 minutes; after it finally gets one, the backend is reported UNHEALTHY and users get 502. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Controller events explain what it could not build
- ❷ No forwarding rule created yet
- ❸ LB exists but health checks fail

EXPLANATION

The GKE Ingress controller builds the load balancer, and `kubectl describe ingress` shows its events: errors about a missing Service or port, a TLS secret that does not exist, a wrong `ingressClassName`, a ClusterIP Service without NEG, or a quota limit keep the address from appearing. Once the load balancer exists, the `ingress.kubernetes.io/backends` annotation shows each backend's health. UNHEALTHY usually means the health check hits `/` and the app returns 404 or a redirect; the check copies a readiness probe's path only in some cases, so define it with a BackendConfig. Also make sure the Google health-check ranges 35.191.0.0/16 and 130.211.0.0/22 can reach the pod port, which in Shared VPC may need a firewall rule created by the host project admin.

KEY POINTS

- Give every service a cheap `/healthz` that returns 200 without calling dependencies.
- New load balancers can take several minutes to become healthy; check events before changing things.

CONFIGURATION EXAMPLE

```

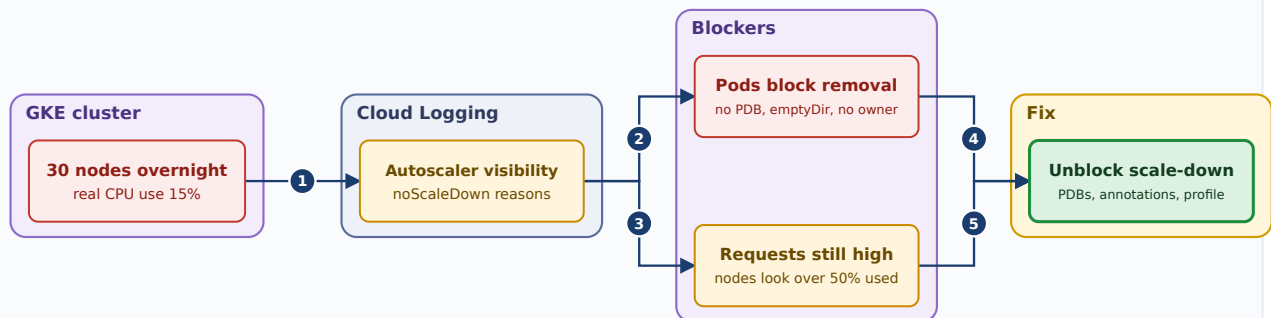
kubectl describe ingress web -n shop | sed -n '/Annotations:/, $p'
kubectl get svc web -n shop -o jsonpath='{.metadata.annotations.cloud.google.com/neg}'
gcloud compute backend-services list --filter="name~k8s1"
gcloud compute backend-services get-health k8s1-abc-shop-web-80-xyz --global
# BackendConfig, referenced from the Service with cloud.google.com/backend-config
apiVersion: cloud.google.com/v1
kind: BackendConfig
metadata: {name: web-hc, namespace: shop}
spec: {healthCheck: {type: HTTP, requestPath: /healthz, port: 8080}}
  
```

Q144

Troubleshooting

At night traffic drops by 80%, but a GKE cluster stays at 30 nodes and the cluster autoscaler never removes any. How do you find what blocks scale-down?

FLOW DIAGRAM



- 1 Visibility logs say why each node stays
- 2 A pod on the node cannot be moved
- 3 Utilization is measured by requests, not usage

EXPLANATION

The cluster autoscaler removes a node only when the pods' requests on it are low and every pod can be moved elsewhere. The cluster-autoscaler visibility logs in Cloud Logging give a `noScaleDown` reason per node. Common blockers are pods not managed by a controller, pods with local storage (`emptyDir`) or the `safe-to-evict: false` annotation, kube-system pods without a `PodDisruptionBudget`, and PDBs that allow 0 disruptions. If nodes simply look busy, the requests are far above real usage, so right-size them (VPA recommendations help); you can also switch the cluster to the `optimize-utilization` autoscaling profile, which scales down more aggressively.

KEY POINTS

- Scale-down decisions use requested resources, so oversized requests keep nodes alive.
- Add PDBs for system workloads such as metrics agents so they do not pin nodes.

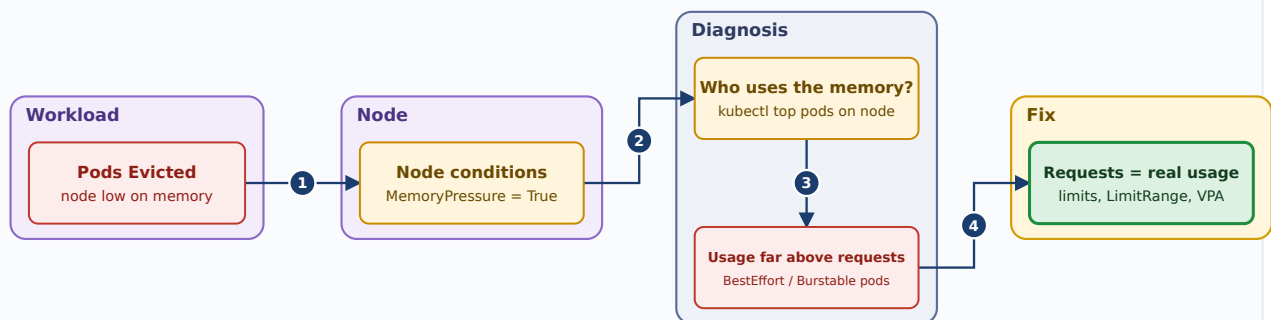
CONFIGURATION EXAMPLE

```

gcloud logging read 'logName="projects/prod/logs/container.googleapis.com%2Fcluster-autoscaler-visibility"'
  AND jsonPayload.noDecisionStatus.noScaleDown:*' --limit=5 --format=json
kubectl describe nodes | grep -A6 "Allocated resources"
kubectl patch deploy report-worker -n data -p \
  '{"spec":{"template":{"metadata":{"annotations":{"cluster-autoscaler.kubernetes.io/safe-to-evict":"true"}}}}}'
gcloud container clusters update prod --region=europe-west1 \
  --autoscaling-profile=optimize-utilization
  
```

Q145**Troubleshooting**

Pods on one node pool keep getting Evicted with The node was low on resource: memory, and nodes show the MemoryPressure condition. How do you troubleshoot and stop it?

FLOW DIAGRAM

- ❶ Pod status names the starved resource
- ❷ Find the pods using the most memory
- ❸ Compare usage with what they requested
- ❹ Scheduler must see the real footprint

EXPLANATION

The scheduler packs pods by their requests, so when pods use much more memory than they requested, the node runs short and the kubelet evicts pods once free memory drops below its eviction threshold. It evicts BestEffort pods first, then Burstable pods using the most memory above their requests, which is why well-behaved pods can be the victims. Use `kubectltop` and the GKE memory-used-versus-requested charts to find the pods that exceed their requests, then set memory requests close to real peak usage, add limits, and use a LimitRange so pods without requests get defaults. Evicted pods stay as Failed objects until deleted, which is only clutter, not the cause.

KEY POINTS

- Use VPA in recommendation mode to learn realistic memory requests.
- Memory-hungry batch jobs belong on their own node pool.

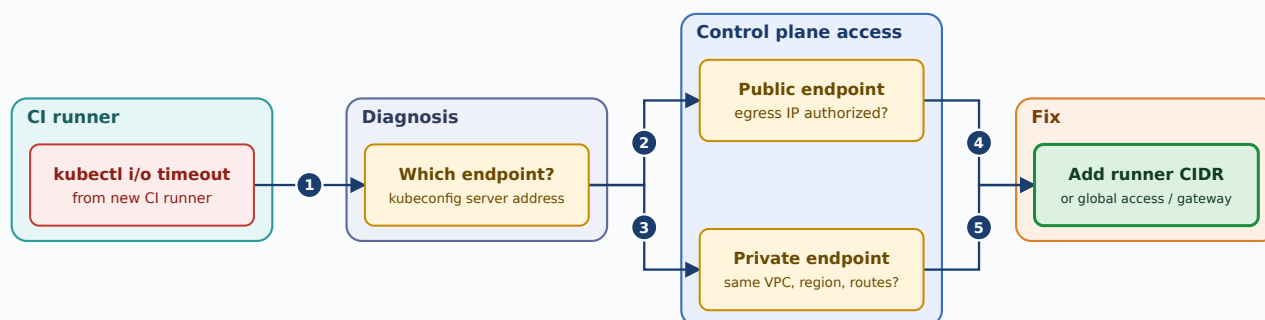
CONFIGURATION EXAMPLE

```

kubectltop get pods -A | grep Evicted
kubectltop describe node gke-prod-pool-a-x1k2 | sed -n '/Conditions:/,/Addresses:/p'
kubectltop top pods -A --sort-by=memory | head -15
kubectltop set resources deploy/report-api -n shop \
  --requests=memory=1Gi --limits=memory=1536Mi
kubectltop delete pods -A --field-selector=status.phase=Failed
  
```

Q146**Troubleshooting**

From a new CI runner, `kubectl get pods` against a private GKE cluster fails with `Unable to connect to the server: dial tcp ...:443: i/o timeout`. It works from the admin VM. How do you fix it?

FLOW DIAGRAM

- ❶ A timeout is a network block, not IAM
- ❷ Source must be in authorized networks
- ❸ Runner needs a private route to the endpoint

EXPLANATION

A timeout, unlike a 403, means packets never reach the control plane, so check the network path, not IAM. Look at the kubeconfig's server address to see whether the runner uses the public or the private endpoint. For the public endpoint, the runner's egress IP (often its Cloud NAT address) must be in master authorized networks. For the private endpoint, the runner must be in the cluster's VPC or a network connected by VPN or Interconnect with the control-plane route advertised, and clients in other regions need private endpoint global access; alternatives are the DNS-based control-plane endpoint on newer GKE versions, Connect Gateway, or a bastion.

KEY POINTS

- `--master-authorized-networks` replaces the whole list: always pass every CIDR.
- Give CI runners static egress IPs so the list stays small and stable.

CONFIGURATION EXAMPLE

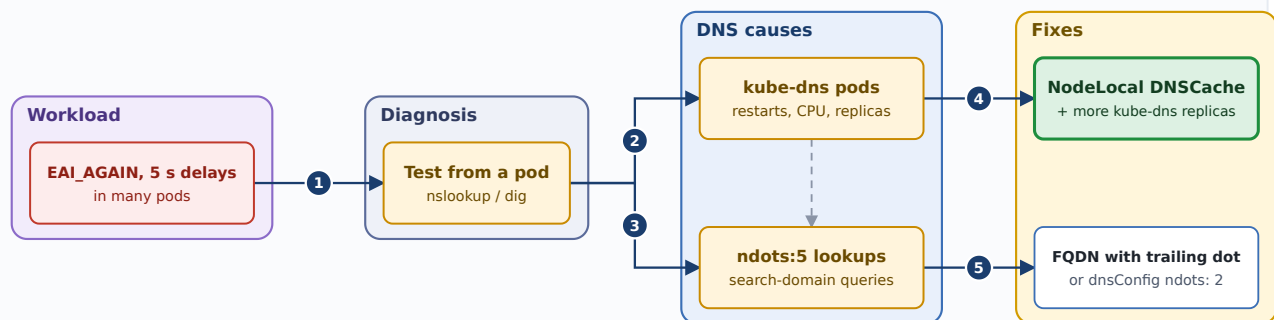
```

kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}'
gcloud container clusters describe prod --region=europe-west1 \
  --format="yaml(masterAuthorizedNetworksConfig,privateClusterConfig)"
gcloud container clusters update prod --region=europe-west1 \
  --enable-master-authorized-networks \
  --master-authorized-networks=10.10.0.0/24,203.0.113.10/32
gcloud container clusters update prod --region=europe-west1 --enable-master-global-access

```


Q147**Troubleshooting**

Pods intermittently fail with `getaddrinfo EAI_AGAIN` or DNS i/o timeout, and some requests take exactly 5 seconds longer than usual. How do you troubleshoot cluster DNS on GKE?

FLOW DIAGRAM

- 1 Reproduce with service and external names
- 2 Overloaded or crashing DNS pods drop queries
- 3 External names first try every search domain
- 4 Cache on each node, fewer conntrack drops
- 5 Fewer queries per lookup
- Also check NetworkPolicy allows port 53 egress

EXPLANATION

Delays of exactly 5 seconds mean a DNS query was dropped and the resolver retried after its default timeout, which points to dropped UDP packets or overloaded kube-dns. Test from a debug pod, then check kube-dns pods for restarts, CPU throttling and replica count (the `kube-dns-autoscaler` ConfigMap controls scaling). The default `ndots:5` makes a name like `api.stripe.com` try every search domain first, multiplying queries, so use FQDNs with a trailing dot or set `ndots: 2` in the pod's `dnsConfig`. Enabling `NodeLocal DNSCache` runs a cache on every node and avoids most conntrack-related drops; also confirm no `NetworkPolicy` blocks UDP and TCP 53 to kube-dns.

KEY POINTS

- Enabling `NodeLocal DNSCache` on an existing cluster recreates nodes: schedule it.
- Watch kube-dns request and error metrics before and after changes.

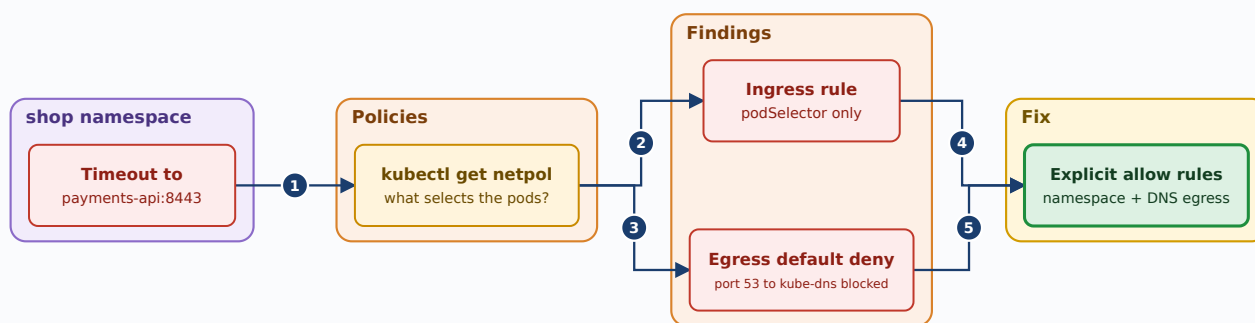
CONFIGURATION EXAMPLE

```

kubectl run dnstest -it --rm --restart=Never --image=busybox:1.36 -- \
  nslookup orders.shop.svc.cluster.local
kubectl get pods -n kube-system -l k8s-app=kube-dns -o wide
kubectl logs -n kube-system -l k8s-app=kube-dns -c kubedns --tail=50
kubectl get configmap kube-dns-autoscaler -n kube-system -o yaml
gcloud container clusters update prod --region=europe-west1 \
  --update-addons=NodeLocalDNS=ENABLED
  
```

Q148**Troubleshooting**

After a team applied a NetworkPolicy in the `payments` namespace, calls from `checkout` in `shop` to `payments-api:8443` time out, and `payments` pods can no longer resolve DNS. What went wrong?

FLOW DIAGRAM

- 1 Once a policy selects a pod, only allowed flows pass
- 2 podSelector alone matches the same namespace only
- 3 Egress policy also blocks DNS lookups

EXPLANATION

As soon as any NetworkPolicy selects a pod for Ingress or Egress, all traffic of that type not explicitly allowed is dropped. A `from` entry with only a `podSelector` matches pods in the policy's own namespace, so `checkout` in `shop` is not allowed; add a `namespaceSelector` in the same entry (same entry means AND, separate entries mean OR). An egress policy must also allow UDP and TCP 53 to kube-dns, or every lookup fails. Remember that policy ports are the pod's container port, not the Service port, and use network policy logging on GKE Dataplane V2 to see which flows are denied.

KEY POINTS

- Roll out default-deny with the allow rules in the same change, and test with a debug pod.
- Label namespaces by name (`kubernetes.io/metadata.name` is set automatically).

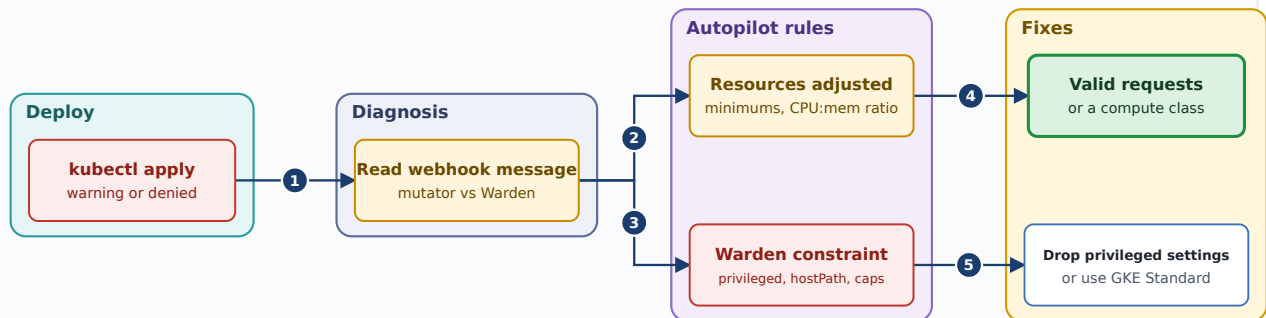
CONFIGURATION EXAMPLE

```

kubectl get netpol -n payments -o yaml | less
ingress:
- from:
  - namespaceSelector:
      matchLabels: {kubernetes.io/metadata.name: shop}
    podSelector:
      matchLabels: {app: checkout}
    ports: [{protocol: TCP, port: 8443}]
egress:
- to: [{namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}}]
  ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
  
```

Q149**Troubleshooting**

On GKE Autopilot, `kubectl apply` prints `autopilot-default-resources-mutator` warnings and pods get more CPU than the manifest asked for, and another Deployment is denied by GKE Warden. How do you troubleshoot?

FLOW DIAGRAM

- 1 The warning names the webhook and the reason
- 2 Requests changed to fit supported ranges
- 3 Pod spec breaks an Autopilot security rule

EXPLANATION

Autopilot bills by pod requests and enforces rules per compute class: minimum and maximum requests, allowed CPU-to-memory ratios and default values when requests are missing, so the mutating webhook raises or fills in values and prints a warning. Check the pod's final resources and the `autopilot.gke.io/resource-adjustment` annotation, then set requests that already fit the ranges, or select a compute class such as `Scale-Out` or `Performance` if the shape does not fit the default class (check current limits in the docs). Denials from GKE Warden mean the spec breaks a security constraint, such as privileged containers, most `hostPath` mounts, host networking or extra Linux capabilities; remove them, use a supported partner workload allowlist, or run that workload on a Standard cluster.

KEY POINTS

- Always set explicit requests on Autopilot; defaults may cost more than you need.
- Dry-run manifests with `kubectl apply --dry-run=server` in CI to catch denials early.

CONFIGURATION EXAMPLE

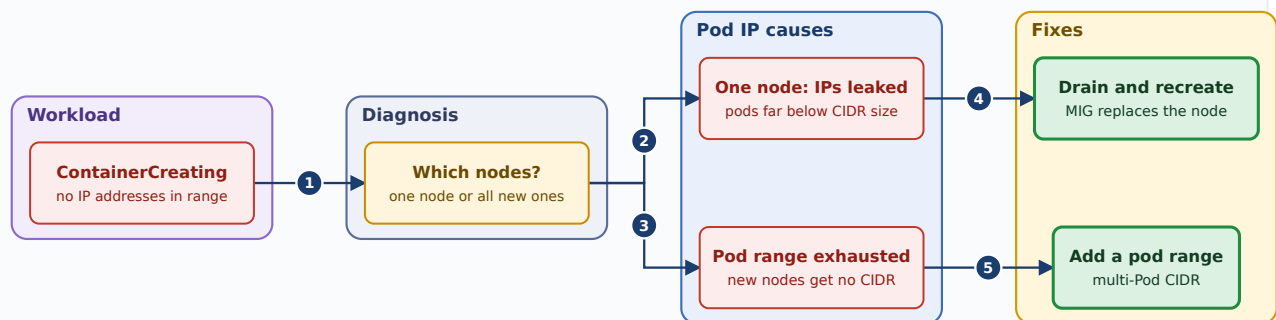
```

kubectl apply -f deploy.yaml --dry-run=server 2>&1 | grep -iE 'autopilot|warden'
kubectl get pod api-6c9f -n shop \
  -o jsonpath='{.metadata.annotations.autopilot.gke.io/resource-adjustment}'
kubectl get pod api-6c9f -n shop -o jsonpath='{.spec.containers[*].resources}'
# in the pod template
nodeSelector: {cloud.google.com/compute-class: Scale-Out}
resources:
  requests: {cpu: 500m, memory: 1Gi, ephemeral-storage: 1Gi}

```

Q150**Troubleshooting****New pods on one node stay in ContainerCreating with**

FailedCreatePodSandBox ... failed to allocate for range 0: no IP addresses available in range set. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Compare node podCIDR with pods running on it
- 2 Allocations were not released
- 3 Cluster cannot hand out node ranges

EXPLANATION

Each node gets a slice of the cluster's pod range (a /24 for the default 110 max pods) and the CNI plugin assigns pod IPs from it. If the error appears on one node while it runs far fewer pods than its slice holds, IP allocations have leaked, for example after container runtime restarts or failed pod teardown; cordon and drain the node and let the managed instance group recreate it, and upgrade to a GKE patch version that fixes any known leak. If many or all new nodes are affected, the cluster's pod secondary range is full, so add another range with discontinuous multi-Pod CIDR or use a smaller max-pods-per-node on new pools. Count running non-hostNetwork pods against the node's podCIDR to tell the two cases apart.

KEY POINTS

- Size pod ranges for the maximum node count, not today's count.
- Alert on nodes stuck in ContainerCreating for more than a few minutes.

CONFIGURATION EXAMPLE

```

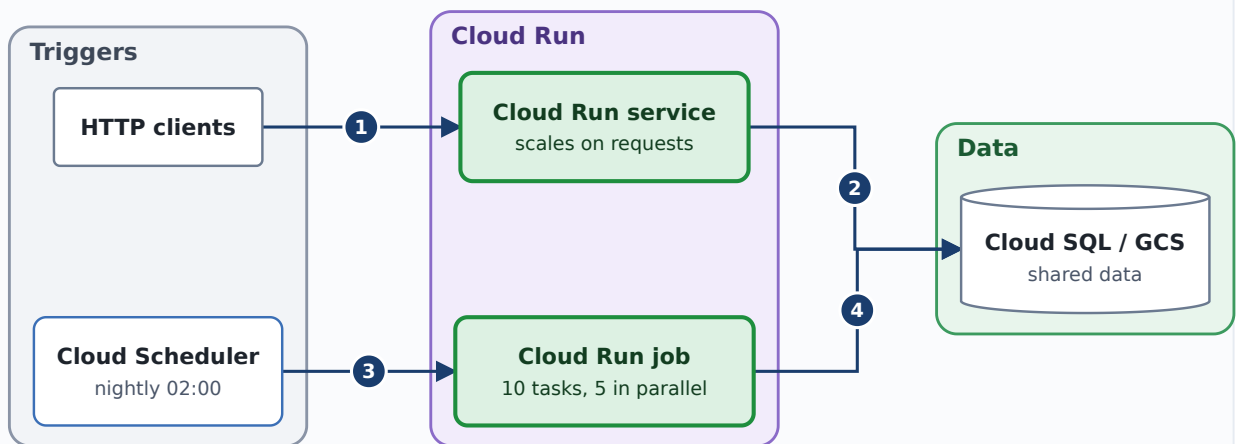
kubectl describe pod web-5f7 -n shop | grep -A3 FailedCreatePodSandBox
kubectl get node gke-prod-pool-a-9x2k -o jsonpath='{.spec.podCIDR}'
kubectl get pods -A --field-selector spec.nodeName=gke-prod-pool-a-9x2k --no-headers | wc -l
kubectl drain gke-prod-pool-a-9x2k --ignore-daemonsets --delete-emptydir-data
gcloud compute instance-groups managed recreate-instances gke-prod-pool-a-1a2b-grp \
  --instances=gke-prod-pool-a-9x2k --zone=europe-west1-b
  
```

6. Serverless & Eventing: Cloud Run, Functions & Pub/Sub

Running containers and functions without managing servers, and wiring them together with events, queues and workflows.

Q151 A team has an HTTP API and a nightly batch export that runs for 40 minutes. Both are containers. How do you run them on Cloud Run?

FLOW DIAGRAM



- 1 Request/response; the container listens on \$PORT
- 3 Starts an execution; tasks run to completion, no port needed
- 4 Each task uses CLOUD_RUN_TASK_INDEX to pick its shard

EXPLANATION

Deploy the API as a Cloud Run service, which serves HTTP requests on the port in \$PORT and scales instances with traffic, down to zero if you allow it. Run the export as a Cloud Run job: an execution runs one or more tasks to completion, has no HTTP endpoint, and can retry failed tasks. Split the work across tasks using `CLOUD_RUN_TASK_INDEX` and `CLOUD_RUN_TASK_COUNT`, and trigger it from Cloud Scheduler or Workflows. Jobs allow long task timeouts (hours, check current limits), which avoids holding an HTTP request open for 40 minutes.

KEY POINTS

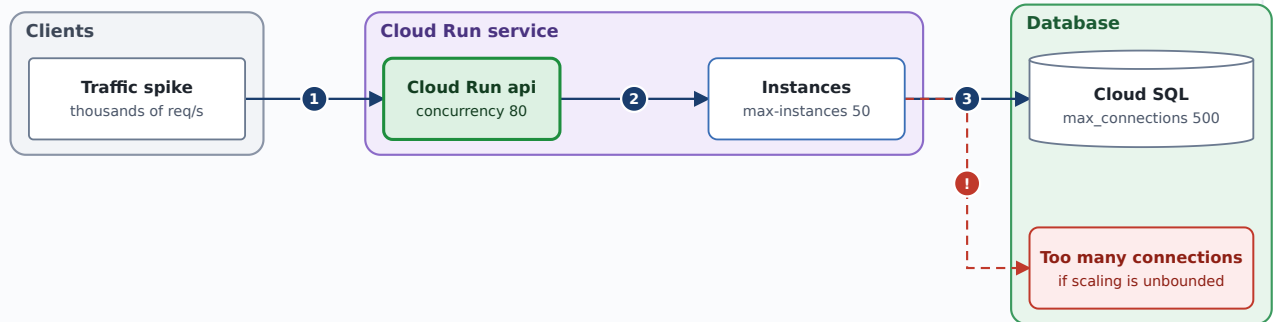
- Service = request-driven; job = run to completion.
- Both use the same container image and IAM model.

CONFIGURATION EXAMPLE

```
gcloud run jobs create nightly-export --image=us-docker.pkg.dev/p/app/export:1.4 \
  --region=us-central1 --tasks=10 --parallelism=5 \
  --max-retries=3 --task-timeout=3600s
gcloud run jobs execute nightly-export --region=us-central1
```

Q152 During a traffic spike a Cloud Run API scaled to 400 instances and Cloud SQL ran out of connections. How do you tune concurrency and scaling?

FLOW DIAGRAM



- 1 Requests arrive faster than one instance can serve
- 2 Each instance takes up to 80 concurrent requests
- 3 50 instances x pool of 5 = 250 connections, under the limit
- ! Unbounded scale-out exhausts database connections
- Requests beyond capacity wait briefly, then get 429

EXPLANATION

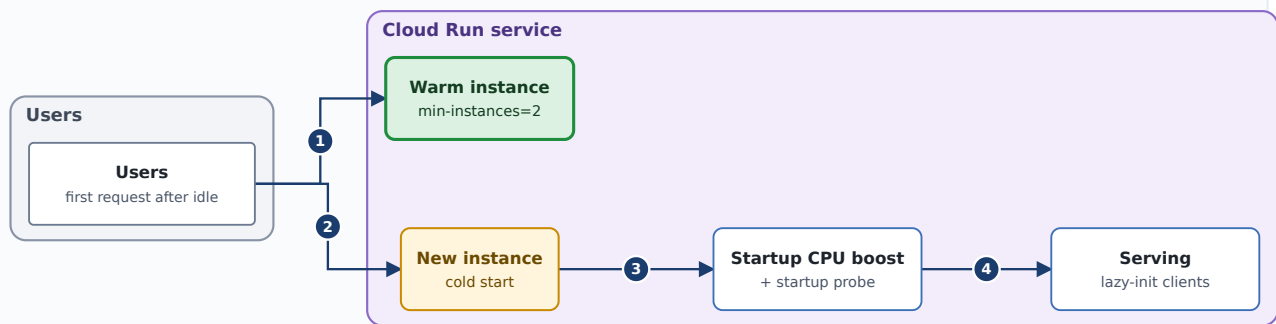
Concurrency sets how many requests one instance handles at once (default 80, up to 1000), and Cloud Run adds instances when that is reached. Cap `--max-instances` so that max instances times the per-instance connection pool stays below the database's connection limit, and use a small pool per instance. Raise concurrency for I/O-bound apps to need fewer instances, and lower it (even to 1) for CPU-heavy or non-thread-safe code. Use the Cloud SQL connector or Auth Proxy, and consider a connection pooler if many services share one database.

KEY POINTS

- Max instances protects downstream systems.
- Load test to find the right concurrency per vCPU.

Q153 Users complain that the first request after a quiet period takes 6 seconds on a Cloud Run service. How do you reduce cold-start latency?

FLOW DIAGRAM



- ❶ Served at once by an idle warm instance
- ❷ Bursts beyond warm capacity still start instances
- ❸ Small image and extra CPU shorten start-up
- ❹ Traffic is sent once the startup probe passes

EXPLANATION

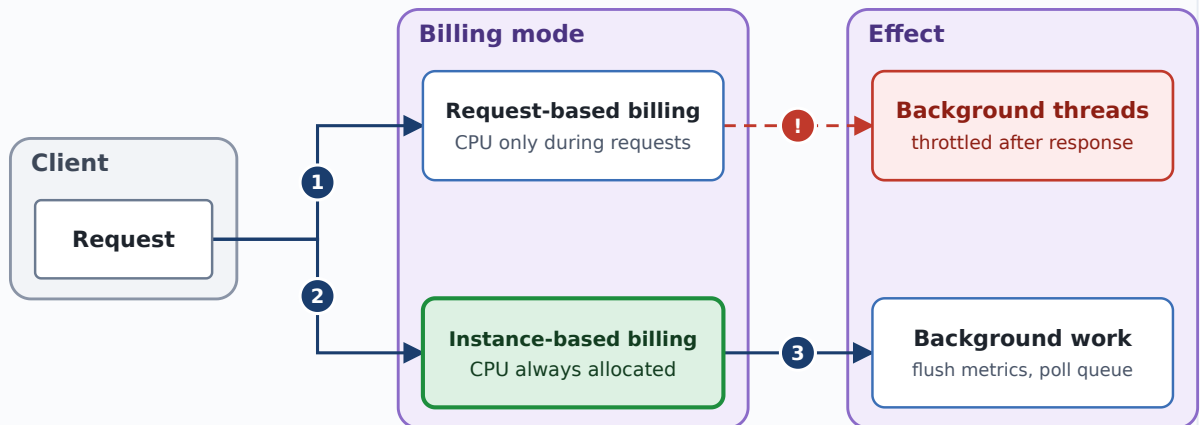
Set `--min-instances` so a few instances stay warm; you pay for them while idle, at a lower rate for idle time. Turn on startup CPU boost, which gives extra CPU while the container starts. Make the container start fast: smaller images, fewer dependencies, lazy initialisation of clients, and for JVM apps options such as class data sharing or native images. Add a startup probe so traffic only arrives when the app is ready, and keep instances warm under steady load by allowing some concurrency.

KEY POINTS

- Min instances trades cost for latency.
- Measure with the container startup latency metric.

Q154 A Cloud Run service returns responses quickly but also flushes metrics and processes a local queue in background threads. That work stalls. Why, and what do you change?

FLOW DIAGRAM



- ❶ Default: CPU is allocated only while requests are in flight
- ❗ Work done after the response gets almost no CPU
- ❸ CPU available outside requests; billed for the instance lifetime

EXPLANATION

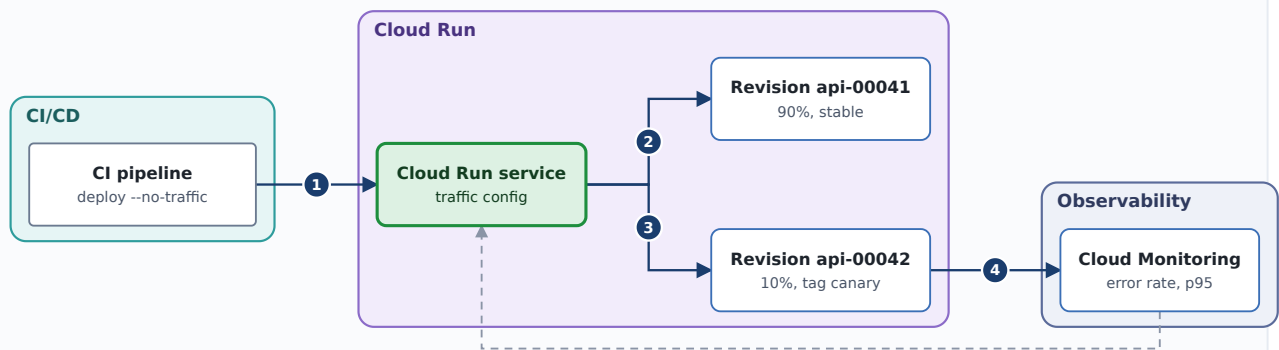
With the default request-based billing, Cloud Run allocates CPU only while an instance is processing requests, so work started after the response is throttled and may never finish. Switch to instance-based billing (CPU always allocated, `--no-cpu-throttling`) when the service must do background work, or move that work into the request path, a Cloud Run job, or a queue consumer. Instance-based billing charges for the whole instance lifetime, so it suits steady traffic; request-based is cheaper for spiky traffic. Instances can still be shut down when idle, so flush state on SIGTERM.

KEY POINTS

- Background work needs CPU always allocated.
- Handle SIGTERM: you get about 10 seconds before shutdown.

Q155 How do you release a new Cloud Run revision to 10% of users, watch it, and roll back in seconds if errors rise?

FLOW DIAGRAM



- 1 Deploy the new revision with no traffic and a tag
- 2 Most users stay on the stable revision
- 3 Shift 10% after testing the tag URL
- 4 Compare errors and latency per revision
- Roll back by sending 100% to the previous revision

EXPLANATION

Every deploy creates an immutable revision, and the service's traffic config decides which revisions get requests. Deploy with `--no-traffic --tag=canary` so you can test the tag URL first, then move 10% of traffic to it and watch revision-level metrics in Cloud Monitoring. Roll back by setting 100% to the previous revision, which takes effect in seconds because the old revision is still deployed. Cloud Deploy can automate canary phases and verification for Cloud Run.

KEY POINTS

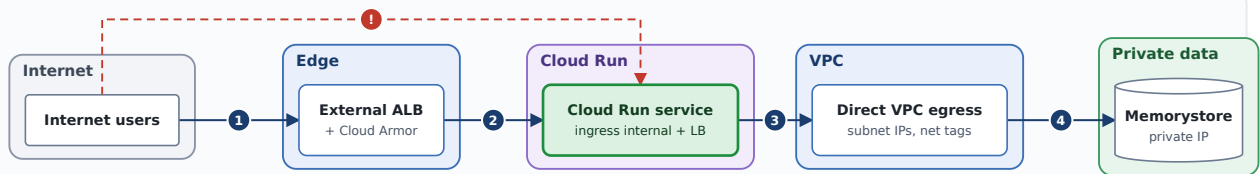
- Revisions are immutable; rollback is a traffic change.
- Tags give each revision its own test URL.

CONFIGURATION EXAMPLE

```
gcloud run deploy api --image=us-docker.pkg.dev/p/app/api:2.3 \
  --region=us-central1 --no-traffic --tag=canary
gcloud run services update-traffic api --region=us-central1 --to-tags=canary=10
# rollback
gcloud run services update-traffic api --region=us-central1 \
  --to-revisions=api-00041-abc=100
```

Q156 A Cloud Run service must only accept traffic through your load balancer and must reach a Memorystore instance on a private IP. How do you configure ingress and egress?

FLOW DIAGRAM



- ② Only LB and internal sources pass the ingress check
- ③ Private-range traffic leaves through the VPC
- ! Direct requests to the run.app URL are rejected

EXPLANATION

Set ingress to `internal-and-cloud-load-balancing` so only the load balancer and internal VPC sources can reach the service, and the default `run.app` URL is blocked. For egress, use Direct VPC egress: instances get IPs from a subnet you choose, can carry network tags for firewall rules, and there are no connector VMs to size or pay for. A Serverless VPC Access connector is the older option; it runs managed VMs with throughput limits and minimum instances. Use `--vpc-egress=private-ranges-only` unless you need all traffic (for example through Cloud NAT for a static IP) to go through the VPC.

KEY POINTS

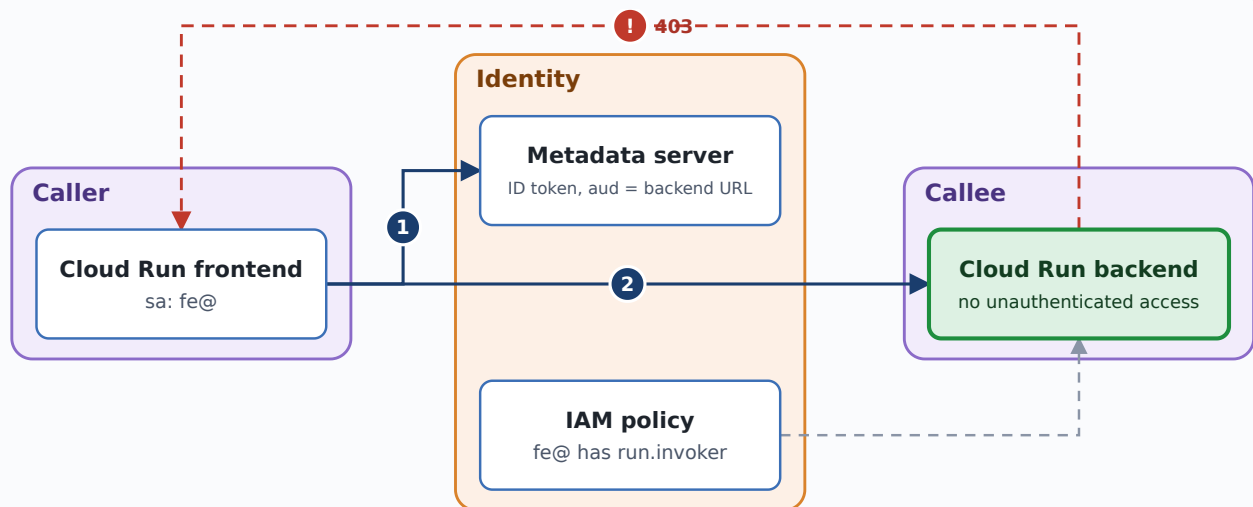
- Size the Direct VPC egress subnet for max instances (check current IP guidance).
- `all-traffic` + Cloud NAT gives a fixed outbound IP.

CONFIGURATION EXAMPLE

```
gcloud run deploy api --image=us-docker.pkg.dev/p/app/api:2.3 \
  --region=us-central1 --ingress=internal-and-cloud-load-balancing \
  --network=prod-net --subnet=run-subnet --network-tags=run-api \
  --vpc-egress=private-ranges-only
```

Q157 A frontend Cloud Run service calls a private backend Cloud Run service. How do you authenticate the call without API keys?

FLOW DIAGRAM



- 1 Fetch a Google-signed ID token for the backend's URL
- 2 Send Authorization: Bearer <ID token>
- Cloud Run checks the token and `run.invoker` before your code runs
- ! 403 if the caller lacks `run.invoker`, 401 if no token

EXPLANATION

Deploy the backend with `--no-allow-unauthenticated` and grant the frontend's service account `roles/run.invoker` on it. The frontend gets an OIDC ID token from the metadata server with the backend URL as audience (client libraries do this for you) and sends it as a Bearer token. Cloud Run validates the token and the IAM binding before the request reaches your container, so the backend has no auth code to maintain. Give each service its own user-managed service account rather than the default compute account.

KEY POINTS

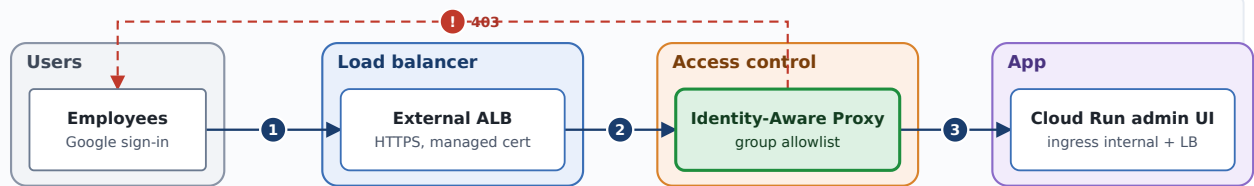
- ID token (audience = service URL), not an access token.
- Least privilege: one service account per service.

CONFIGURATION EXAMPLE

```
gcloud run services add-iam-policy-binding backend --region=us-central1 \
  --member=serviceAccount:fe@prod-p.iam.gserviceaccount.com \
  --role=roles/run.invoker
# manual test from a shell
curl -H "Authorization: Bearer $(gcloud auth print-identity-token)" \
  https://backend-abc123-uc.a.run.app/health
```

Q158 An internal admin UI on Cloud Run must be reachable only by employees in the ops group, without a VPN. How do you protect it?

FLOW DIAGRAM



- ② IAP on the backend service redirects to Google sign-in
- ③ Request forwarded with a signed JWT header the app can verify
- ❗ 403 for users outside the ops group

EXPLANATION

Put the service behind an external Application Load Balancer with a serverless NEG, enable Identity-Aware Proxy on the backend service, and grant the ops group `roles/iap.httpsResourceAccessor`. IAP authenticates users with their Google identity and can add context-aware access levels such as device policy. Set the Cloud Run ingress so the run.app URL cannot bypass the load balancer, and verify the `x-goog-iap-jwt-assertion` header in the app for defence in depth. IAP can also be enabled directly on Cloud Run services without a load balancer; check current feature status.

KEY POINTS

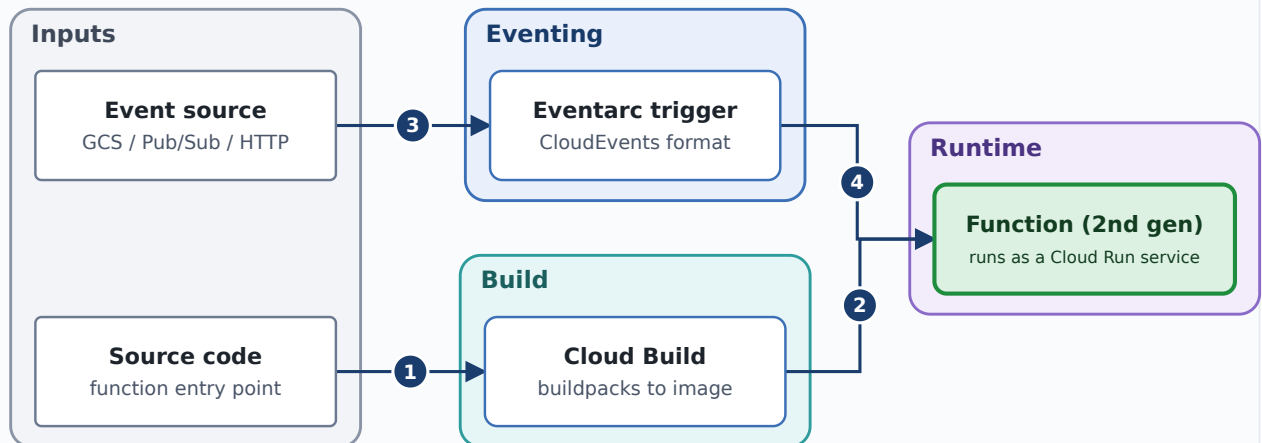
- IAP = zero-trust access by identity, not network location.
- Verify the IAP JWT in the app.

CONFIGURATION EXAMPLE

```
gcloud compute backend-services update admin-bs --global --iap=enabled
gcloud iap web add-iam-policy-binding --resource-type=backend-services \
  --service=admin-bs --member=group:ops@example.com \
  --role=roles/iap.httpsResourceAccessor
```

Q159 Why would you choose Cloud Functions 2nd gen (Cloud Run functions) over 1st gen, and how does it work under the hood?

FLOW DIAGRAM



- ❶ Deploy builds a container with buildpacks
- ❷ Image is deployed as a Cloud Run service
- ❸ Events from many Google sources
- ❹ Delivered as a CloudEvent over HTTP

EXPLANATION

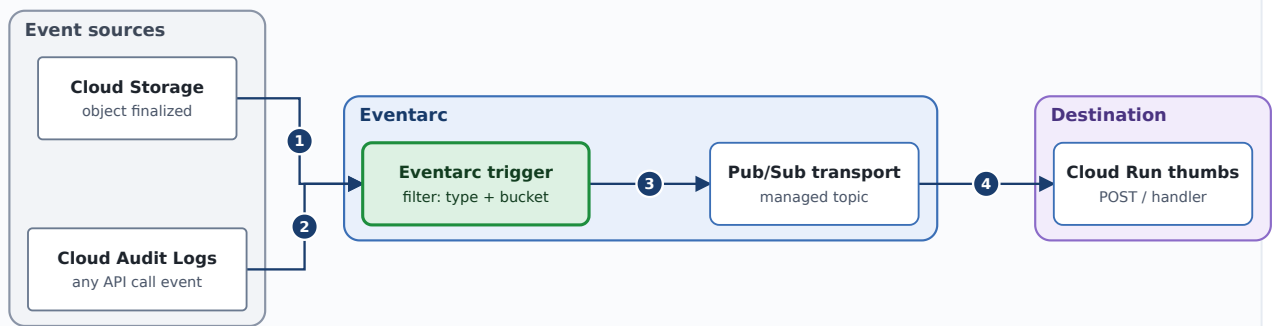
2nd gen functions are built with Cloud Build and buildpacks and run as Cloud Run services, so they get Cloud Run features: many concurrent requests per instance, longer HTTP timeouts (up to 60 minutes), larger instance sizes, traffic splitting and min instances. Event-driven functions are wired through Eventarc, which supports many more sources than 1st gen and delivers events in CloudEvents format. You write only the function code, and Google manages the image and runtime updates. For new work, 2nd gen is the default choice.

KEY POINTS

- Concurrency > 1 reduces cold starts and cost.
- Event-driven timeouts are shorter than HTTP ones; check current limits.

Q160 When a file lands in a Cloud Storage bucket, a Cloud Run service must create a thumbnail. How do you wire this with Eventarc and what IAM is needed?

FLOW DIAGRAM



- 1 Direct event: google.cloud.storage.object.v1.finalized
- 2 Audit-log events cover services without direct events
- 3 Eventarc uses Pub/Sub underneath
- 4 Pushed with an OIDC token for the trigger's service account

EXPLANATION

Create an Eventarc trigger in the bucket's location with filters for the finalized event type and the bucket, pointing at the Cloud Run service. The trigger's service account needs `roles/run.invoker` on the service and `roles/eventarc.eventReceiver`, and the Cloud Storage service agent needs `roles/pubsub.publisher` so it can publish events. Events arrive as CloudEvents over HTTP; return 2xx only after the work succeeds so failures are retried. Write thumbnails to a different bucket or prefix so the trigger does not fire on its own output.

KEY POINTS

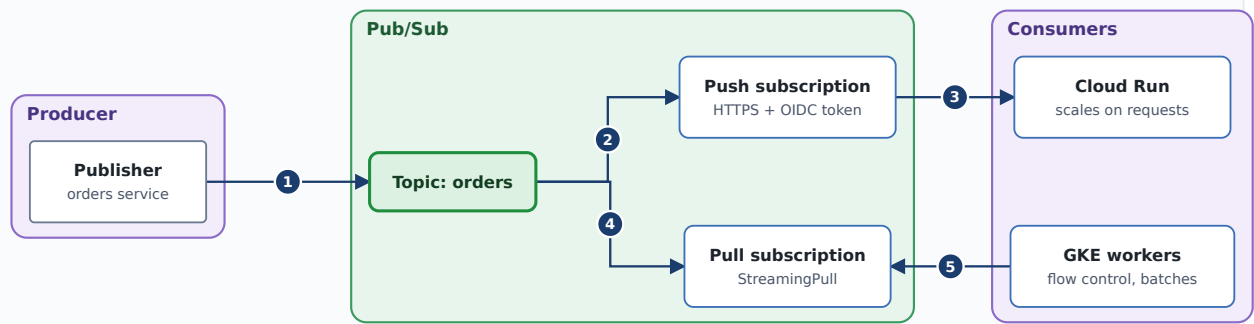
- Avoid infinite loops: never write output that matches the trigger filter.
- Handlers must be idempotent; delivery is at least once.

CONFIGURATION EXAMPLE

```
gcloud eventarc triggers create thumbs-trigger --location=us-central1 \
--destination-run-service=thumbs --destination-run-region=us-central1 \
--event-filters="type=google.cloud.storage.object.v1.finalized" \
--event-filters="bucket=uploads-bkt" \
--service-account=trigger-sa@prod-p.iam.gserviceaccount.com
```

Q161 An orders topic feeds a Cloud Run service and a fleet of GKE batch workers.
When do you use push and when pull subscriptions?

FLOW DIAGRAM



- ❶ Publish once
- ❷ Each subscription gets its own copy
- ❸ Pub/Sub POSTs the message; a 2xx response is the ack
- ❺ Workers pull, process and ack at their own pace

EXPLANATION

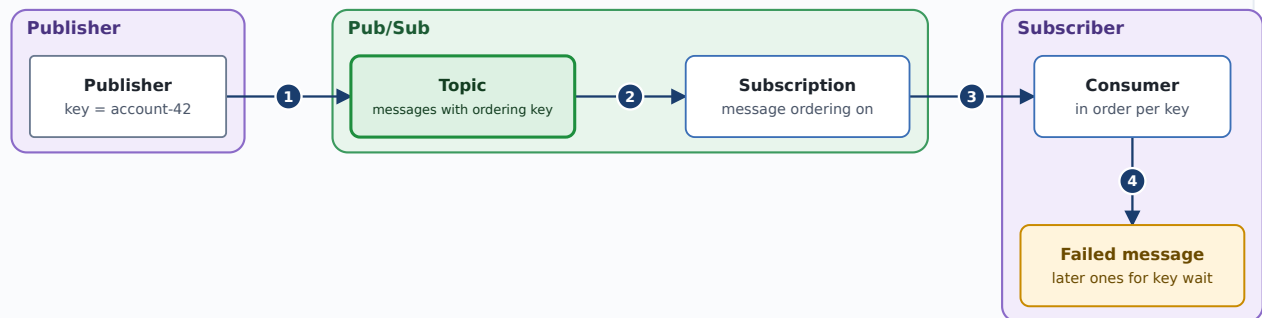
Push subscriptions send each message as an HTTPS POST to an endpoint, which suits serverless consumers like Cloud Run that scale on requests; a 2xx response acks the message and Pub/Sub applies push backoff on errors. Pull subscriptions let workers fetch messages with StreamingPull, control flow and batch sizes, and are better for high throughput, long processing, or consumers without a public endpoint. Exactly-once delivery is only available for pull. Each subscription receives every message, so add a subscription per independent consumer.

KEY POINTS

- Push: endpoint-driven, easy with Cloud Run.
- Pull: consumer-controlled throughput and flow control.

Q162 Account events (open, debit, credit, close) must be processed in order per account, but different accounts can be processed in parallel. How do you do this with Pub/Sub?

FLOW DIAGRAM



- 1 Publish with an ordering key, ordering enabled in the client
- 3 Same key is delivered in publish order; keys run in parallel
- 4 A nack or timeout redelivers it before later messages of that key

EXPLANATION

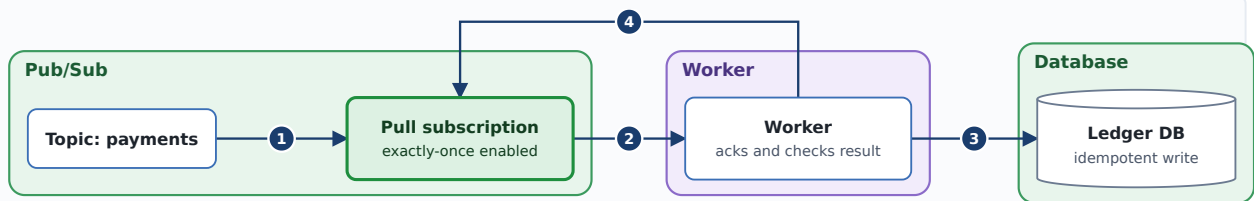
Publish each event with the account ID as the ordering key, enable message ordering in the publisher client, and create the subscription with `--enable-message-ordering`. Pub/Sub then delivers messages with the same key in the order they were published in a region, while different keys are delivered in parallel. If a publish fails, the client pauses that key until you resume it, and if a consumer fails a message it is redelivered before later messages of the same key, so one bad message can block a key. Pair this with a dead-letter topic and keep ordering keys high-cardinality to avoid hot keys.

KEY POINTS

- Ordering is per key and per region.
- Throughput per ordering key is limited; check current limits.

Q163 A payment consumer occasionally processes the same message twice. What does Pub/Sub exactly-once delivery guarantee, and is it enough?

FLOW DIAGRAM



- ② Not redelivered after a successful ack
- ③ Side effect still needs a unique payment ID
- ④ Ack returns success or failure; retry on failure
- Pull subscriptions only; guarantee applies within a region

EXPLANATION

With exactly-once delivery enabled on a pull subscription, Pub/Sub does not redeliver a message once it has been acked successfully within the ack deadline, and the ack call tells you whether it succeeded. It does not cover push subscriptions, and it cannot make your external side effects atomic: if the worker writes to the database and then crashes before acking, the message is redelivered. So still make the write idempotent, for example with a unique payment ID constraint, and extend ack deadlines for long work. Duplicates can also come from the publisher retrying, which exactly-once delivery does not remove.

KEY POINTS

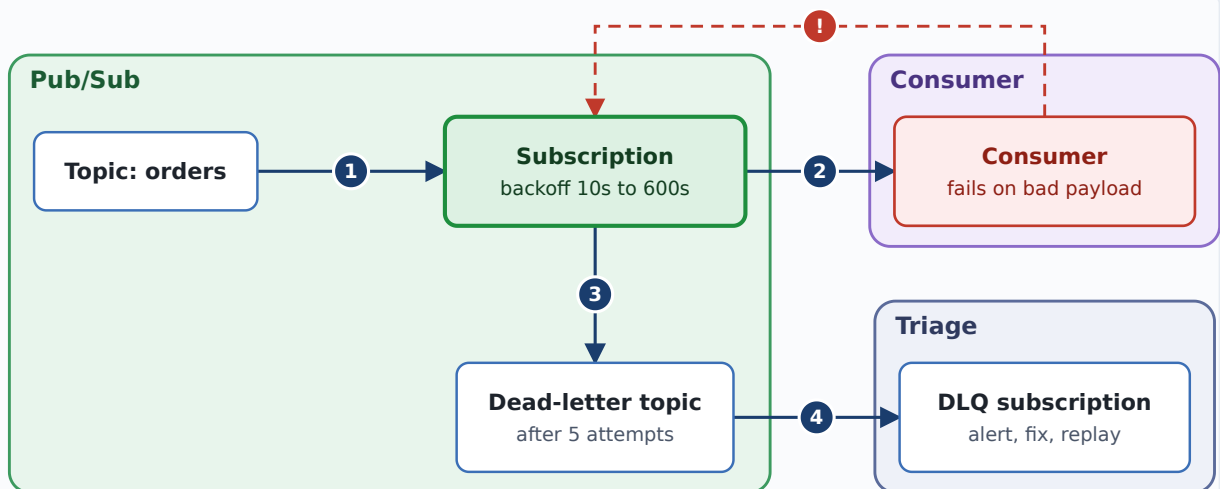
- Exactly-once delivery reduces duplicates; idempotency removes their effect.
- Check ack results in the client library.

CONFIGURATION EXAMPLE

```
gcloud pubsub subscriptions create pay-sub --topic=payments \
  --enable-exactly-once-delivery --ack-deadline=60
```

Q164 A malformed message keeps failing and is redelivered forever, delaying other work. How do you handle poison messages in Pub/Sub?

FLOW DIAGRAM



- ② Delivery attempt
- ! Nack or missed deadline: redelivered with exponential backoff
- ③ After max delivery attempts the message is forwarded
- ④ Inspect, fix and republish; alert on backlog

EXPLANATION

Configure a retry policy with exponential backoff (for example 10 to 600 seconds) so failures do not hammer the consumer, and attach a dead-letter topic with max delivery attempts (5 to 100). After that many attempts the message moves to the dead-letter topic, where a separate subscription is monitored and messages can be fixed and replayed. The Pub/Sub service agent needs publisher rights on the dead-letter topic and subscriber rights on the source subscription. Alert on the dead-letter backlog and on `oldest_unacked_message_age` for the main subscription.

KEY POINTS

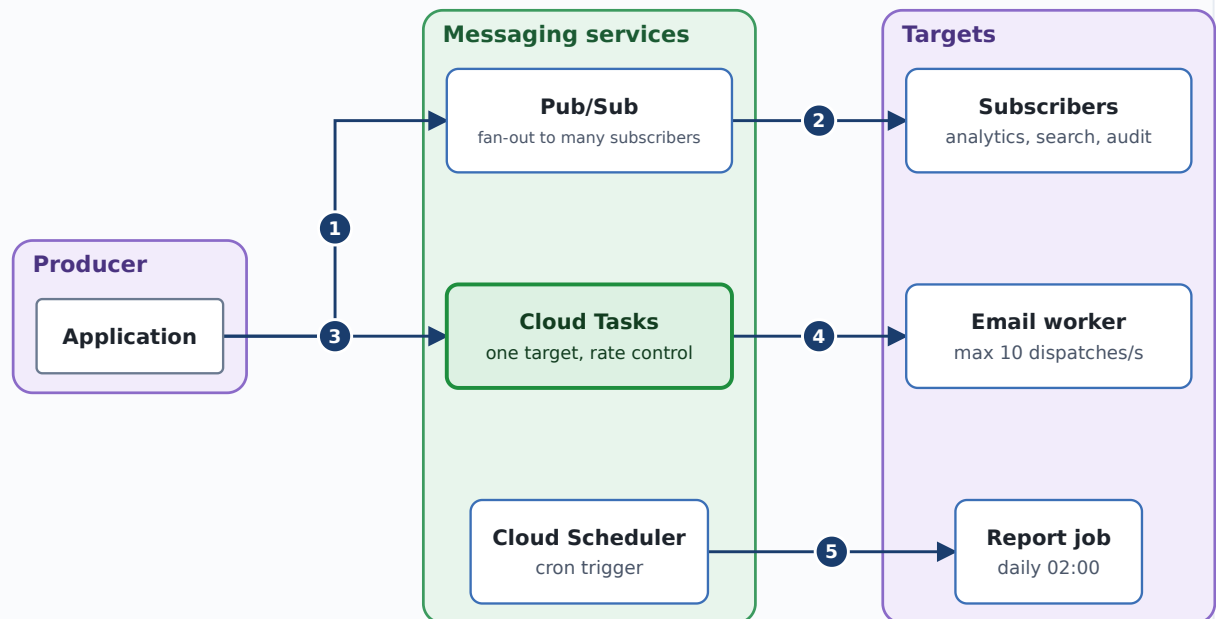
- Delivery attempts are counted approximately; design for that.
- Grant the Pub/Sub service agent roles on both resources.

CONFIGURATION EXAMPLE

```
gcloud pubsub subscriptions update orders-sub \
  --dead-letter-topic=orders-dlt --max-delivery-attempts=5 \
  --min-retry-delay=10s --max-retry-delay=600s
```

Q165 You need to send welcome emails through a provider limited to 10 calls per second, fan out order events to analytics, and run a nightly report. Which of Pub/Sub, Cloud Tasks and Cloud Scheduler fits each?

FLOW DIAGRAM



- 1 Publisher does not know who consumes events
- 3 Enqueue a task with an explicit target and optional ETA
- 4 Queue enforces dispatch rate, concurrency and retries
- 5 Cron schedule calls HTTP, Pub/Sub or a job

EXPLANATION

Use Pub/Sub for event fan-out: publishers do not know the consumers, and each subscription gets every message. Use Cloud Tasks for the emails: each task targets a specific handler, and the queue enforces a maximum dispatch rate and concurrency, retries with backoff, and can delay tasks to a scheduled time. Use Cloud Scheduler for the nightly report; it is a managed cron that calls an HTTP endpoint, publishes to Pub/Sub or starts a workflow, with OIDC auth. Tasks is about controlled delivery of individual work items, Pub/Sub about decoupled streams.

KEY POINTS

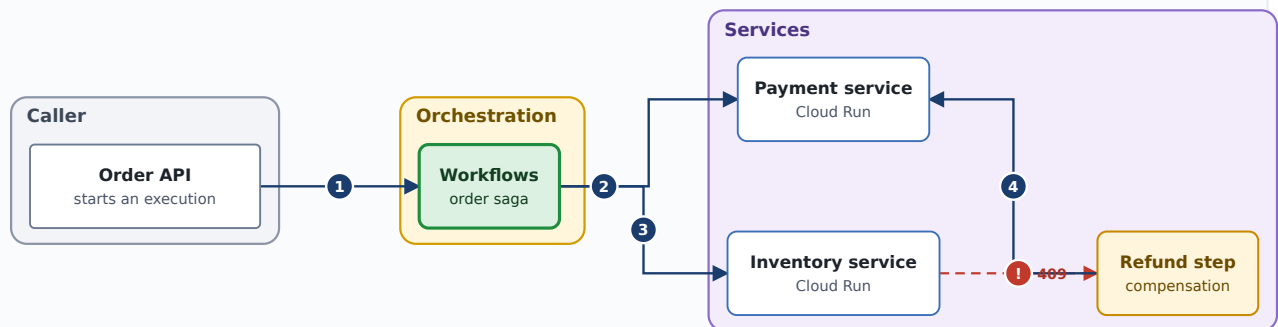
- Rate limiting a downstream API = Cloud Tasks.
- Scheduler has no queue; it only fires on time.

CONFIGURATION EXAMPLE

```
gcloud tasks queues create email-q --location=us-central1 \
  --max-dispatches-per-second=10 --max-concurrent-dispatches=5 --max-attempts=5
gcloud scheduler jobs create http nightly-report --location=us-central1 \
  --schedule="0 2 * * *" --time-zone="Europe/London" \
  --uri=https://report-abc123-uc.a.run.app/run --http-method=POST \
  --oidc-service-account-email=sched@prod-p.iam.gserviceaccount.com
```

Q166 Placing an order calls a payment service, then an inventory service; if inventory fails the payment must be refunded. How do you orchestrate this on Google Cloud?

FLOW DIAGRAM



- 1** Execution starts with the order as input
- 2** Charge card; OIDC auth and retry policy per step
- 3** Reserve stock
- !** 409 out of stock: the except block runs compensation
- 4** Refund the earlier charge, then fail the execution

EXPLANATION

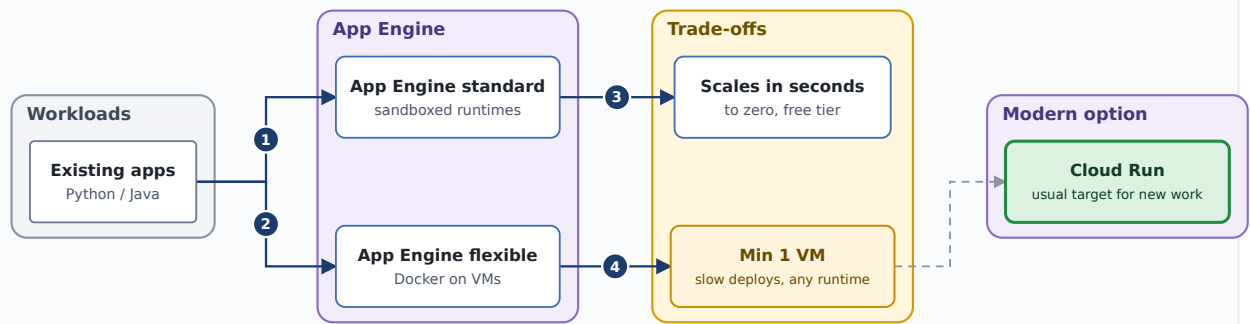
Use Workflows to orchestrate the saga: each step calls a service with built-in OIDC auth, retry policies and timeouts, and `try/except` blocks let you run compensating steps such as a refund when a later step fails. Workflows keeps execution state durably, so you do not need your own state machine, and executions can wait on callbacks or sleep for long periods. Choreography with Pub/Sub events is an alternative when services should stay fully decoupled, but it makes the overall flow harder to see and to compensate. Keep each service call idempotent because steps may be retried.

KEY POINTS

- Orchestration (Workflows) vs choreography (events).
- Every step must be safe to retry.

Q167 You inherit two App Engine apps, one on standard and one on flexible. How do they differ, and would you move them?

FLOW DIAGRAM



- 3 Fast scaling, limited to supported runtimes
- 4 Always-on VMs, minutes to deploy and scale
- Flexible apps are good candidates to move

EXPLANATION

App Engine standard runs your code in language runtimes on a managed sandbox, scales from zero in seconds and has a free tier, but you are limited to supported runtimes and versions. Flexible runs your container on Compute Engine VMs, supports any runtime and background processes, but keeps at least one instance running and deploys and scales in minutes. Both remain supported, but new container workloads usually fit Cloud Run better. Flexible apps are the first candidates to migrate because they are already containers; standard apps that rely on bundled legacy services need more work.

KEY POINTS

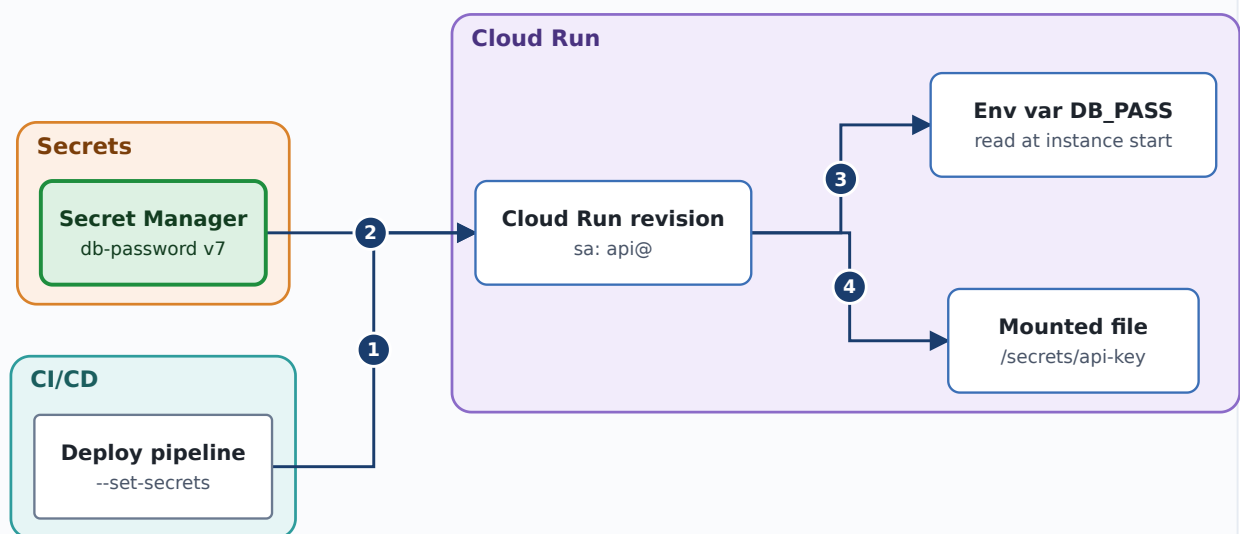
- Standard: fast scale, sandboxed; flexible: VMs, slower.
- Cloud Run is the default for new stateless services.

CONFIGURATION EXAMPLE

```
# app.yaml (standard)
runtime: python312
instance_class: F2
automatic_scaling:
  min_instances: 0
  max_instances: 20
  target_cpu_utilization: 0.65
```

Q168 A Cloud Run service needs a database password and an API key. How do you provide secrets securely and handle rotation?

FLOW DIAGRAM



- 1 Deploy references secret name and version, never the value
- 2 The service account needs secretAccessor on each secret
- 3 Env vars are resolved when the instance starts; pin a version
- 4 A volume mount with latest reads the current version on access

EXPLANATION

Store secrets in Secret Manager and reference them from Cloud Run as environment variables or mounted files, granting the service's own service account `roles/secretmanager.secretAccessor` on just those secrets. Environment variables are resolved at instance start, so pin a specific version and deploy a new revision to rotate. Mounted secrets with `latest` are read on access, so the app can pick up a rotated value without a redeploy if it re-reads the file. Never bake secrets into images or pass plain values in deploy commands; use rotation schedules and Pub/Sub notifications from Secret Manager to trigger updates.

KEY POINTS

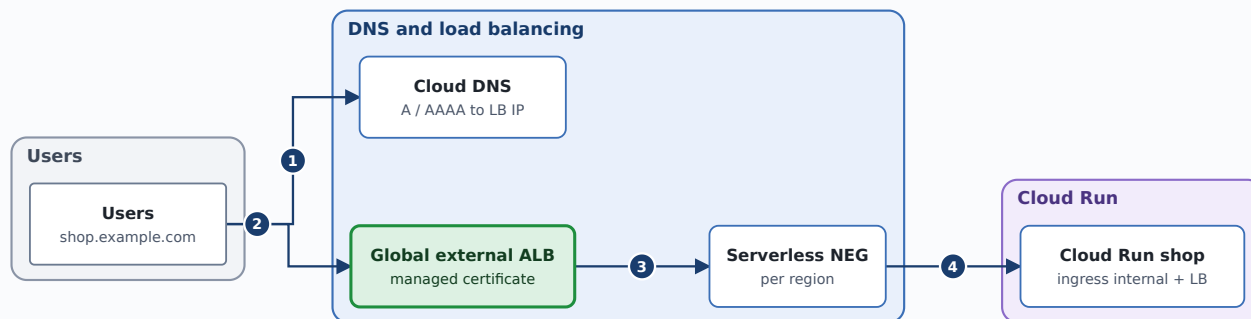
- Pin versions for env vars; `latest` suits mounted files.
- Access per secret, per service account.

CONFIGURATION EXAMPLE

```
gcloud run deploy api --image=us-docker.pkg.dev/p/app/api:2.3 \
  --region=us-central1 \
  --service-account=api@prod-p.iam.gserviceaccount.com \
  --set-secrets=DB_PASS=db-password:7,/secrets/api-key=api-key:latest
```

Q169 Marketing wants `shop.example.com` to serve a Cloud Run service in production. Would you use Cloud Run domain mapping or a load balancer?

FLOW DIAGRAM



- ❶ Resolve the domain to the reserved global IP
- ❷ HTTPS to the anycast IP; TLS ends at the edge
- ❸ URL map host rule picks the backend

EXPLANATION

Cloud Run's built-in domain mapping is limited (preview, selected regions, no Cloud Armor or CDN), so for production use a global external Application Load Balancer with a serverless NEG. Reserve a global IP, point the A/AAAA records at it, attach a Google-managed certificate, and set the service's ingress to internal and load balancing. This also gives you Cloud Armor, Cloud CDN, multi-region backends and several services under one domain through URL maps. Firebase Hosting is another option for simple web front ends.

KEY POINTS

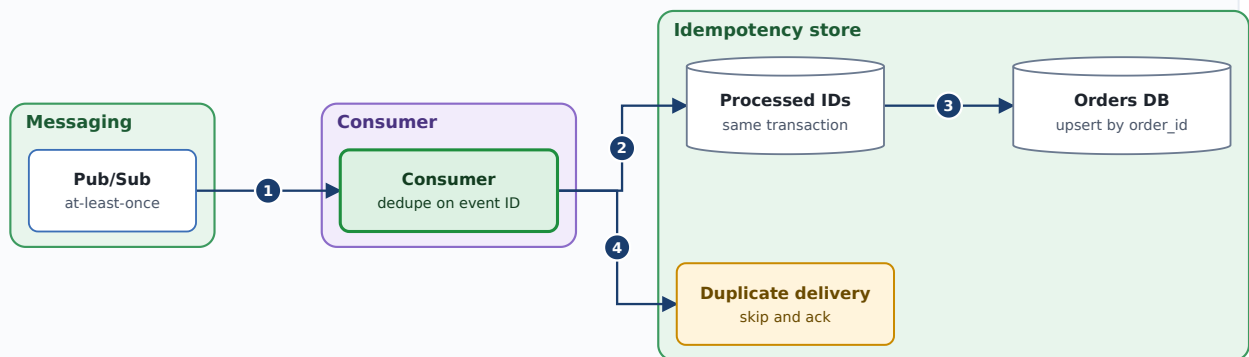
- LB = production path for custom domains.
- One LB can front many services by host and path.

CONFIGURATION EXAMPLE

```
gcloud compute addresses create shop-ip --global
gcloud compute ssl-certificates create shop-cert --domains=shop.example.com --global
gcloud compute url-maps create shop-map --default-service=shop-bs
gcloud compute target-https-proxies create shop-proxy \
  --url-map=shop-map --ssl-certificates=shop-cert
gcloud compute forwarding-rules create shop-fr --global \
  --target-https-proxy=shop-proxy --address=shop-ip --ports=443
```

Q170 Pub/Sub and Eventarc deliver at least once, and customers were charged twice after a redelivery. How do you make event consumers idempotent?

FLOW DIAGRAM



- ❶ The same message may arrive more than once
- ❷ Try to insert the event ID (unique constraint)
- ❸ New ID: record it and apply the change in one transaction
- ❹ ID already seen: ack without repeating the side effect

EXPLANATION

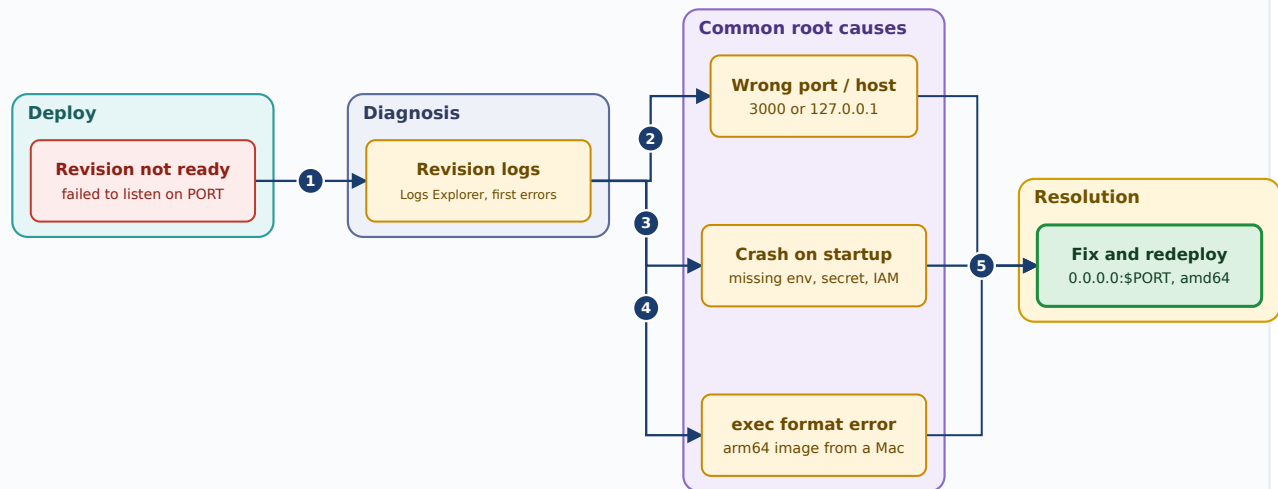
Give every event a stable business ID (for example the order or payment ID, or the CloudEvent `id`) and record processed IDs in the same transaction as the side effect, using a unique constraint so a second attempt fails cleanly. Prefer operations that are naturally idempotent, such as upserts or setting a status, over increments. When calling external APIs, pass an idempotency key if the provider supports one. Ack only after the transaction commits, and expire old IDs with a TTL sized to the maximum redelivery window.

KEY POINTS

- Dedupe on a business key, not the delivery attempt.
- Write the ID and the effect atomically.

Q171**Troubleshooting**

`gcloud run deploy` fails with 'The user-provided container failed to start and listen on the port defined provided by the `PORT=8080` environment variable'. The image runs fine on the developer's laptop. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Read the container's own logs for the revision
- ❷ App must listen on `$PORT` on all interfaces
- ❸ A stack trace means the app exits before listening
- ❹ Image built for the wrong CPU architecture
- ❺ Fix config, rebuild for linux/amd64, redeploy

EXPLANATION

The message only says the health check on the port failed; the real reason is in the revision's logs, so read them first. The most common causes are an app listening on a hard-coded port such as 3000 or on 127.0.0.1 instead of `0.0.0.0:$PORT`, and an app that crashes at startup because an environment variable, secret or database is missing or its service account lacks a permission. An `exec format error` means the image was built for arm64 (for example on an Apple silicon laptop) and must be rebuilt for `linux/amd64`. Slow startups can also exceed the startup probe; increase the probe timeout or do heavy initialisation lazily.

KEY POINTS

- Reproduce locally with `docker run -e PORT=8080 -p 8080:8080` using the exact image from Artifact Registry.
- Build images in Cloud Build, not on laptops, so the architecture is always correct.

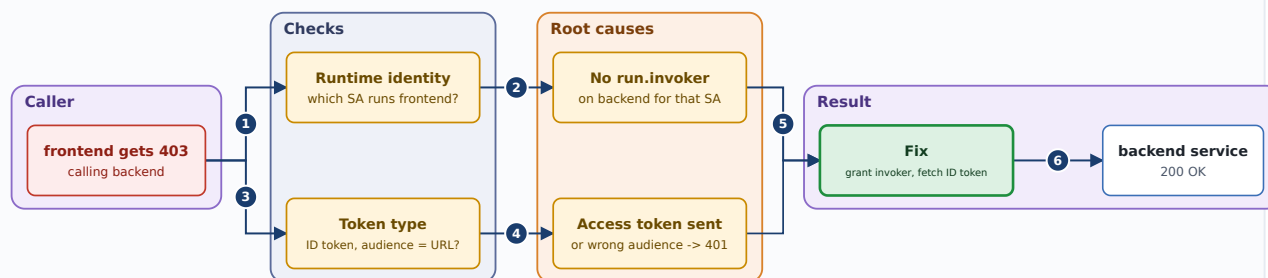
CONFIGURATION EXAMPLE

```

gcloud logging read 'resource.type="cloud_run_revision" AND
  resource.labels.service_name="orders-api" AND severity>=ERROR' --limit=20 --freshness=1h
docker inspect --format '{{.Os}}/{{.Architecture}}' \
  us-docker.pkg.dev/shop/app/orders-api:1.4.2
docker buildx build --platform linux/amd64 -t us-docker.pkg.dev/shop/app/orders-api:1.4.3 --push .
gcloud run deploy orders-api --image=us-docker.pkg.dev/shop/app/orders-api:1.4.3 \
  --region=us-central1 --port=8080
  
```

Q172**Troubleshooting**

The frontend Cloud Run service gets 403 Forbidden (and sometimes 401) when calling the private backend Cloud Run service. Calls with your own user token from Cloud Shell work. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Your user works, so the SA is the difference
- 2 403 means authenticated but not allowed
- 3 Check what the code puts in Authorization
- 4 Cloud Run needs an OIDC ID token, not OAuth
- 5 Grant roles/run.invoker to the frontend SA
- 6 Call succeeds with Bearer ID token

EXPLANATION

A 403 means Cloud Run accepted the token but the identity has no `roles/run.invoker` on the backend; check which service account `frontend` runs as (often the default compute SA) and grant the role to that account on the backend service. A 401 usually means the wrong token: the code must send a Google-signed ID token whose audience is the backend URL (or a configured custom audience), not an OAuth access token. Fetch it from the metadata server or with a Google auth library, and send it as `Authorization: Bearer`. If the backend's ingress is set to internal, calls must also leave the frontend through a VPC (Direct VPC egress or a connector) to be accepted.

KEY POINTS

- Give each service its own service account so invoker grants are explicit and auditable.
- Use Google auth libraries, which cache and refresh ID tokens automatically.

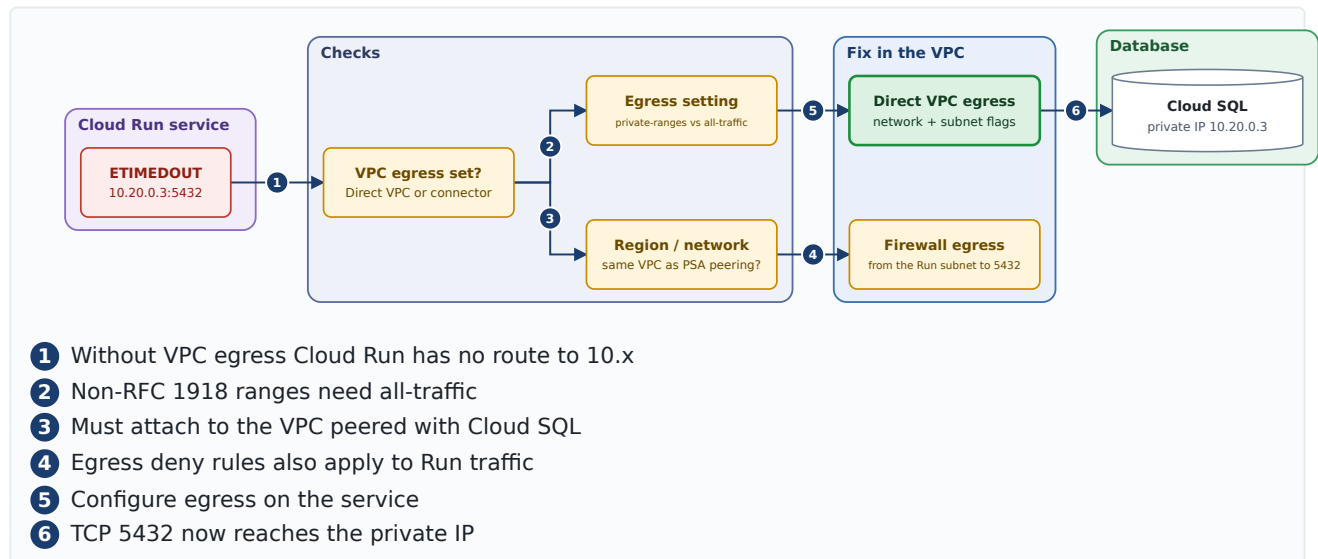
CONFIGURATION EXAMPLE

```

gcloud run services describe frontend --region=us-central1 \
  --format='value(spec.template.spec.serviceAccountName)'
gcloud run services get-iam-policy backend --region=us-central1
gcloud run services add-iam-policy-binding backend --region=us-central1 \
  --member=serviceAccount:frontend-sa@shop.iam.gserviceaccount.com --role=roles/run.invoker
# inside frontend: get an ID token for the backend URL
curl -s -H 'Metadata-Flavor: Google' \
  'http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/identity?
  audience=https://backend-abc123-uc.a.run.app'
  
```

Q173**Troubleshooting**

A Cloud Run service fails with `connect ETIMEDOUT 10.20.0.3:5432` when connecting to a Cloud SQL instance that has only a private IP. The same connection works from a GKE pod in the VPC. How do you troubleshoot it?

FLOW DIAGRAM**EXPLANATION**

Cloud Run runs outside your VPC, so it can only reach a private IP after you configure Direct VPC egress (`--network` and `--subnet`) or a Serverless VPC Access connector in the same region. With `private-ranges-only` , only RFC 1918 and a few private ranges go through the VPC; if Private Service Access uses another range, set `--vpc-egress=all-traffic` . Check the service attaches to the same VPC that has the Private Service Access peering with Cloud SQL, and that no egress firewall rule blocks the Cloud Run subnet or connector range to port 5432. If you use the Cloud SQL connector libraries or `--add-cloudsql-instances` , they must be told to use the private IP, otherwise they try the public IP.

KEY POINTS

- Give Direct VPC egress its own subnet sized for max instances (it uses IPs per instance).
- Test the path with a Connectivity Test from the Run subnet range to the database IP.

CONFIGURATION EXAMPLE

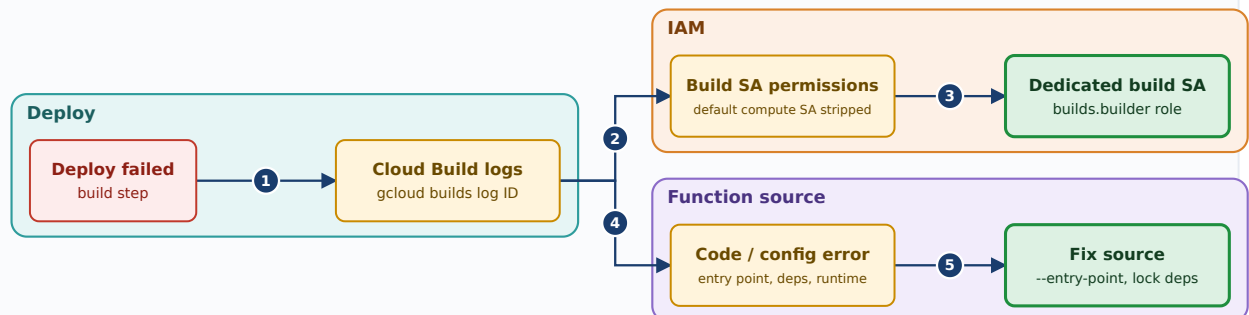
```

gcloud run services describe orders-api --region=us-central1 \
  --format='yaml(spec.template.metadata.annotations)'
gcloud sql instances describe orders-db --
format='value(ipAddresses,settings.ipConfiguration.privateNetwork)'
gcloud run services update orders-api --region=us-central1 \
  --network=prod-vpc --subnet=run-egress --vpc-egress=private-ranges-only
gcloud compute firewall-rules list --filter="network~prod-vpc AND direction=EGRESS" \
  --format='table(name,priority,destinationRanges.list(),denied[].map().firewall_rule().list())'

```

Q174**Troubleshooting**

`gcloud functions deploy thumbnailer --gen2` fails with 'Build failed ... Could not build the function due to a missing permission on the build service account', and a second function fails with 'Function thumbnail is not defined in the provided module'. How do you troubleshoot both?

FLOW DIAGRAM

- ❶ Deploys are Cloud Build jobs: read their logs
- ❷ Permission error on the build service account
- ❸ Grant the role or pass `--build-service-account`
- ❹ Build or load error in the function code
- ❺ Match the entry point name and pin dependencies

EXPLANATION

A Cloud Functions deploy runs a Cloud Build job, so the details are in `gcloud builds log`, not in the deploy output. The permission error means the service account used for builds (in many newer projects the default Compute Engine service account, often stripped of roles by the org policy that disables automatic grants) cannot write logs, read the source or push to Artifact Registry; grant it `roles/cloudbuild.builds.builder` or, better, create a dedicated build service account and pass `--build-service-account`. The deployer also needs `iam.serviceAccountUser` on the runtime and build service accounts. The second error means `--entry-point` does not match a function exported by the code; other code failures are dependency install errors from `requirements.txt` or `package.json`, or an unsupported runtime version.

KEY POINTS

- Use a dedicated build and runtime service account per function, created in Terraform.
- Run the function locally with the Functions Framework before deploying.

CONFIGURATION EXAMPLE

```

gcloud builds list --region=us-central1 --limit=5 --format='table(id,status,createTime)'
gcloud builds log 1a2b3c4d-5e6f --region=us-central1
gcloud projects add-iam-policy-binding media-prod --role=roles/cloudbuild.builds.builder \
  --member=serviceAccount:fn-build@media-prod.iam.gserviceaccount.com
gcloud functions deploy thumbnailer --gen2 --region=us-central1 --runtime=python312 \
  --source=. --entry-point=make_thumbnail --trigger-http \
  --build-service-account=projects/media-prod/serviceAccounts/fn-build@media-prod.iam.gserviceaccount.com

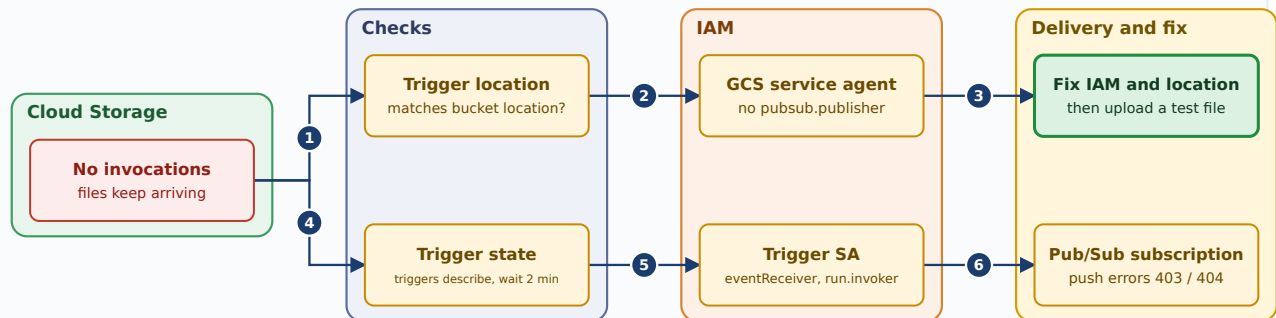
```

Q175

Troubleshooting

An Eventarc trigger for `google.cloud.storage.object.v1.finalized` on bucket `uploads-prod` was created without errors, but the Cloud Run service is never invoked when files are uploaded. How do you troubleshoot it?

FLOW DIAGRAM



- 1 A us multi-region bucket needs trigger location us
- 2 Storage events are published by the GCS agent
- 3 Grant roles/pubsub.publisher to that agent
- 4 New triggers take up to two minutes to activate
- 5 The trigger identity must be allowed to invoke
- 6 The transport subscription shows delivery errors

EXPLANATION

For Cloud Storage direct events the trigger location must match the bucket location, so a bucket in the `us` multi-region needs a trigger in `us`, not `us-central1`. The Cloud Storage service agent publishes the events and needs `roles/pubsub.publisher`; this is the most common missing grant in projects where automatic role grants are disabled. The trigger's service account needs `roles/eventarc.eventReceiver` and `roles/run.invoker` on the target service. Eventarc delivers through a Pub/Sub subscription it creates, so check that subscription's push metrics and the Cloud Run request logs for 403 or 404 responses, and remember a new trigger can take up to two minutes to start delivering.

KEY POINTS

- Keep trigger, bucket and service in Terraform together, including the service agent grants.
- Add a synthetic upload test after deployment that checks the service was invoked.

CONFIGURATION EXAMPLE

```

gcloud storage buckets describe gs://uploads-prod --format='value(location)'
gcloud eventarc triggers describe uploads-trigger --location=us \
  --format='yaml(eventFilters,serviceAccount,transport)'
GCS_SA=$(gcloud storage service-agent --project=media-prod)
gcloud projects add-iam-policy-binding media-prod \
  --member=serviceAccount:$GCS_SA --role=roles/pubsub.publisher
gcloud run services add-iam-policy-binding thumbnailer --region=us-central1 \
  --member=serviceAccount:eventarc-sa@media-prod.iam.gserviceaccount.com --role=roles/run.invoker

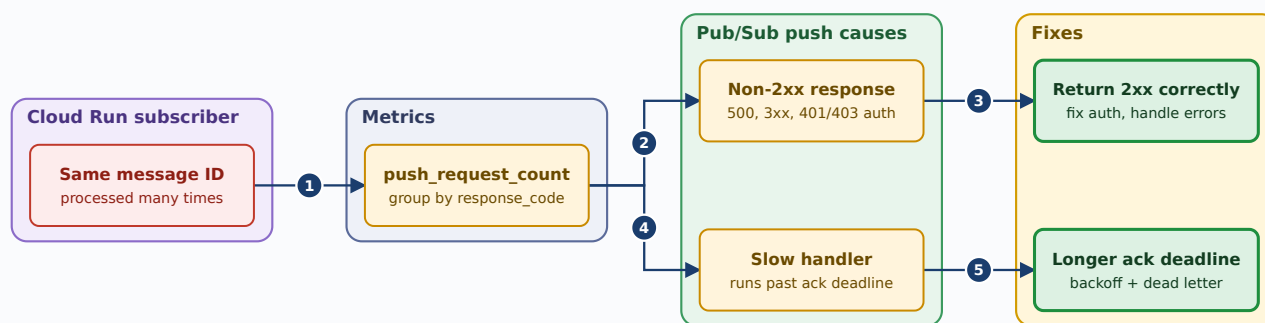
```

Q176

Troubleshooting

A Pub/Sub push subscription to a Cloud Run endpoint redelivers the same messages again and again; logs show the same message ID processed 5-10 times and `push_request_count` has many non-success responses. How do you troubleshoot it?

FLOW DIAGRAM



- 1 See which response codes Pub/Sub receives
- 2 Only 2xx acks a push message
- 3 Grant invoker to the push SA, fix handler errors
- 4 Deadline exceeded: Pub/Sub retries while you work
- 5 Raise ack deadline, add retry backoff and DLQ

EXPLANATION

For push subscriptions, a message is acknowledged only when the endpoint returns a 2xx status within the ack deadline; any other code, a redirect, or a slow response makes Pub/Sub send it again. Group the subscription/`push_request_count` metric by response code: 401 or 403 means the push service account (OIDC token) lacks `roles/run.invoker` or the audience is wrong, and 5xx means the handler is failing. If most responses are success but slow, the handler runs longer than the ack deadline (10 seconds by default), so raise it up to 600 seconds and set the Cloud Run request timeout above it. Add an exponential retry policy and a dead-letter topic with a max delivery count, and keep the handler idempotent because at-least-once delivery still allows duplicates.

KEY POINTS

- Return 2xx only after the work is committed, and store processed message IDs for idempotency.
- Alert on dead-letter topic messages and on push latency close to the ack deadline.

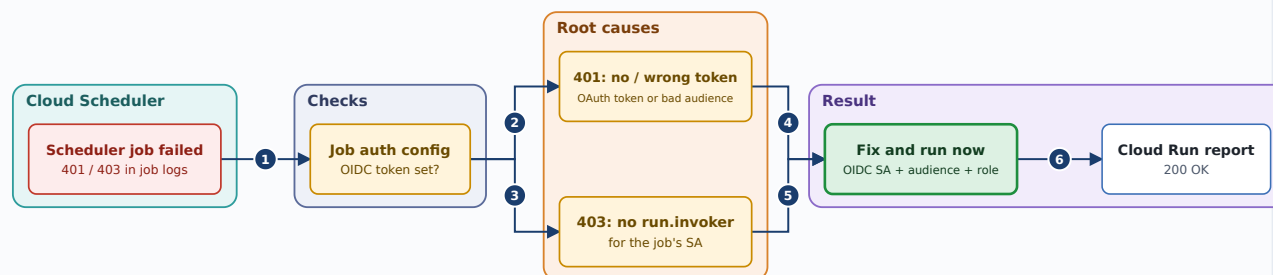
CONFIGURATION EXAMPLE

```

gcloud pubsub subscriptions describe orders-push \
  --format='yaml(ackDeadlineSeconds,pushConfig,retryPolicy,deadLetterPolicy)'
gcloud pubsub subscriptions update orders-push --ack-deadline=120 \
  --min-retry-delay=10s --max-retry-delay=600s \
  --dead-letter-topic=orders-dlq --max-delivery-attempts=5
gcloud run services add-iam-policy-binding orders-worker --region=us-central1 \
  --member=serviceAccount:push-sa@shop.iam.gserviceaccount.com --role=roles/run.invoker
  
```

A Cloud Scheduler job that calls a private Cloud Run endpoint every night fails; its logs show `status: UNAUTHENTICATED (HTTP 401)` or `PERMISSION_DENIED (403)`. How do you troubleshoot and fix it?

FLOW DIAGRAM



- 1 Describe the job's HTTP target settings
- 2 Cloud Run needs an OIDC ID token, not OAuth
- 3 Token is valid but the SA may not invoke
- 4 Set `--oidc-service-account-email` and audience
- 5 Grant `roles/run.invoker` to that SA
- 6 Trigger a manual run to confirm

EXPLANATION

Describe the job and check its HTTP target: to call Cloud Run it needs an `oidcToken` with a service account, and the audience must be the service URL (a trailing path or a different URL gives 401). OAuth tokens are only for Google APIs on `*.googleapis.com`, so using `--oauth-service-account-email` against Cloud Run also fails. A 403 means the token is fine but that service account lacks `roles/run.invoker` on the target service. The person or pipeline that creates the job also needs `iam.serviceAccountUser` on the service account, and the Cloud Scheduler service agent must be able to create tokens for it; run the job manually after the fix to confirm.

KEY POINTS

- Use one dedicated scheduler service account per target with only `run.invoker`.
- Alert on failed Cloud Scheduler runs, not only on the absence of the nightly report.

CONFIGURATION EXAMPLE

```

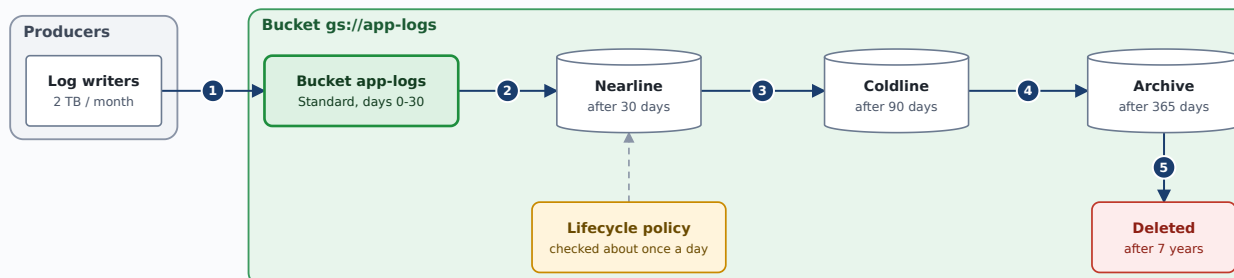
gcloud scheduler jobs describe nightly-report --location=us-central1 \
  --format='yaml(httpTarget.uri,httpTarget.oidcToken,status)'
gcloud scheduler jobs update http nightly-report --location=us-central1 \
  --oidc-service-account-email=sched-report@shop.iam.gserviceaccount.com \
  --oidc-token-audience=https://report-abc123-uc.a.run.app
gcloud run services add-iam-policy-binding report --region=us-central1 \
  --member=serviceAccount:sched-report@shop.iam.gserviceaccount.com --role=roles/run.invoker
gcloud scheduler jobs run nightly-report --location=us-central1
  
```

7. Storage & Databases

Cloud Storage, Cloud SQL, AlloyDB, Spanner, Firestore, Bigtable, Memorystore, BigQuery and Filestore: how to pick them, run them and keep them safe.

Q178 An application writes 2 TB of logs a month to Cloud Storage. They are read often for 30 days, rarely after that, and must be kept 7 years. How do you keep storage cost low?

FLOW DIAGRAM



- 1 New objects land in Standard for frequent reads
- 2 Rule SetStorageClass at age 30 days
- 3 Rule at age 90 days
- 4 Rule at age 365 days
- 5 Delete rule at about 2555 days (7 years)
- The lifecycle config drives every transition; no code or cron needed

EXPLANATION

Keep new logs in Standard and add Object Lifecycle Management rules that change the storage class as the objects age: Nearline at 30 days, Coldline at 90, Archive at 365, and a Delete rule at about 2555 days. Cloud Storage evaluates the rules asynchronously, so a transition can happen a day or so after the threshold. Colder classes have lower storage prices but higher retrieval costs and minimum storage durations (30, 90 and 365 days), so only move data that is really cold. Lifecycle transitions themselves are not charged as early deletion, but deleting or rewriting an object before its minimum duration is.

KEY POINTS

- Minimum durations: Nearline 30, Coldline 90, Archive 365 days.
- Reads from Archive are still millisecond latency, just more expensive per GB.

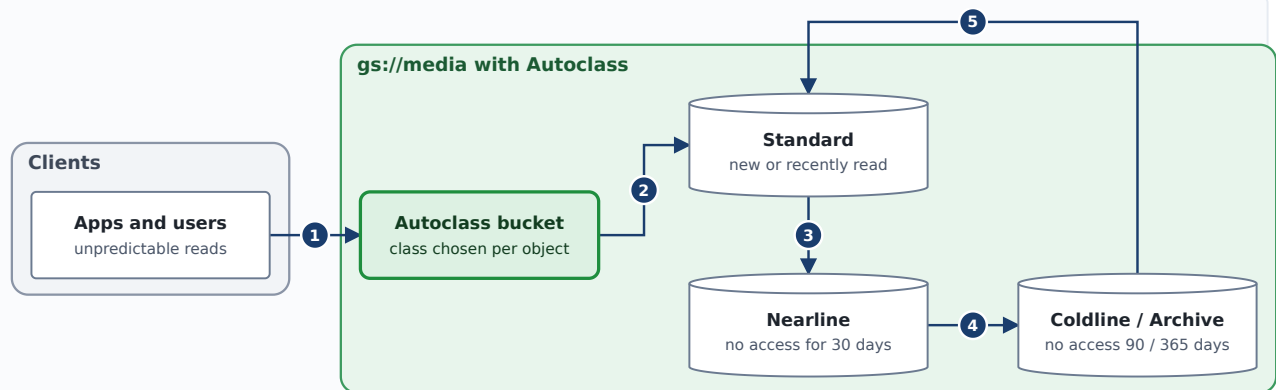
CONFIGURATION EXAMPLE

```
{ "rule": [
  { "action": { "type": "SetStorageClass", "storageClass": "NEARLINE", "condition": { "age": 30 } },
  { "action": { "type": "SetStorageClass", "storageClass": "COLDLINE", "condition": { "age": 90 } },
  { "action": { "type": "SetStorageClass", "storageClass": "ARCHIVE", "condition": { "age": 365 } },
  { "action": { "type": "Delete", "condition": { "age": 2555 } } } ] }
```

```
gcloud storage buckets update gs://app-logs --lifecycle-file=lifecycle.json
```


Q179 A media bucket holds millions of objects and nobody knows which ones are still used. Writing lifecycle rules by age would move popular old files to cold storage. What do you do?

FLOW DIAGRAM



- ❶ Read and write objects as usual
- ❷ Every new object starts in Standard
- ❸ Not read for 30 days, moved to Nearline
- ❹ Colder classes if the terminal class is Archive
- ❺ Any read moves the object back to Standard

EXPLANATION

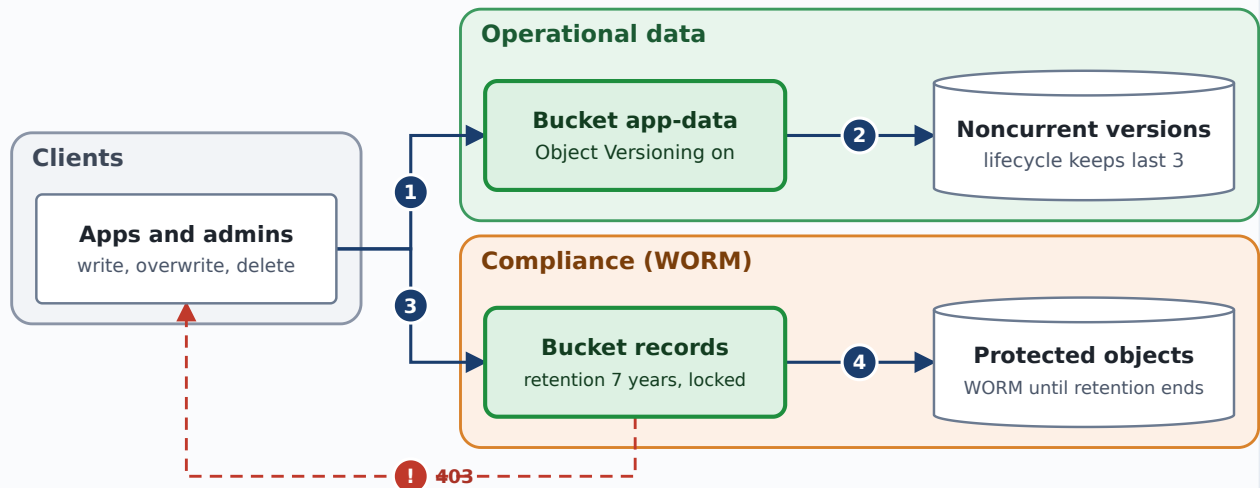
Enable Autoclass on the bucket. Cloud Storage then tracks access per object: objects start in Standard, move to Nearline after 30 days without access, and (if you set the terminal class to Archive) on to Coldline and Archive; a read moves the object back to Standard. Autoclass transitions do not incur early-deletion or retrieval charges, but there is a small monthly management fee per object, and objects smaller than 128 KiB are not moved. Use it when access is unpredictable; keep explicit lifecycle rules when the pattern is known, because they are cheaper and easier to reason about.

KEY POINTS

- Default terminal class is Nearline; set Archive to go colder.
- Autoclass replaces class-changing lifecycle rules; Delete rules still work.

Q180 You must protect operational files from accidental overwrite or deletion, and separately keep financial records in write-once form for 7 years for auditors. How do you configure Cloud Storage?

FLOW DIAGRAM



- ❶ Overwrite or delete keeps the old generation as noncurrent
- ❷ Lifecycle trims old generations (numNewerVersions, days noncurrent)
- ❸ Records are written once
- ❹ Each object is protected until it is 7 years old
- ❗ 403 on delete or replace before the retention period ends
- ⊖ Restore: copy a noncurrent generation (object#generation) over the live object

EXPLANATION

Use two buckets, because Object Versioning and a retention policy cannot be enabled on the same bucket. The operational bucket gets Object Versioning (plus the default soft delete window), so an overwrite or delete leaves a noncurrent generation that you can copy back; lifecycle rules limit how many old generations you keep. The records bucket gets a 7-year retention policy, and once you lock it the policy cannot be removed or shortened and the bucket cannot be deleted while it holds protected objects. Event-based or temporary holds cover cases like legal hold on individual objects.

KEY POINTS

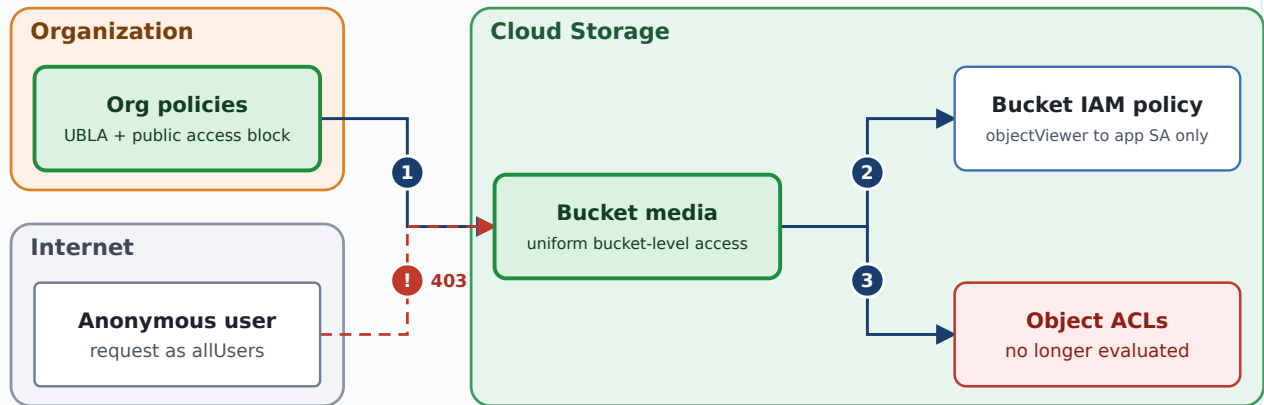
- Locking a retention policy is irreversible: test it on a sandbox bucket first.
- Soft delete gives a short recovery window even without versioning.

CONFIGURATION EXAMPLE

```
gcloud storage buckets update gs://app-data --versioning
gcloud storage buckets update gs://records --retention-period=7y
# irreversible: the policy can no longer be removed or reduced
gcloud storage buckets update gs://records --lock-retention-period
# restore an old generation
gcloud storage cp gs://app-data/report.csv#16958000000000000000 gs://app-data/report.csv
```

Q181 A security review finds some objects in production buckets are public through legacy object ACLs. How do you close this and stop it from happening again?

FLOW DIAGRAM



- ❶ Constraints force UBLA and public access prevention on all buckets
- ❷ Access is decided only by IAM (bucket, project, org)
- ❸ Existing object ACLs stop granting access
- ❹ 403 public access prevention blocks allUsers and allAuthenticatedUsers

EXPLANATION

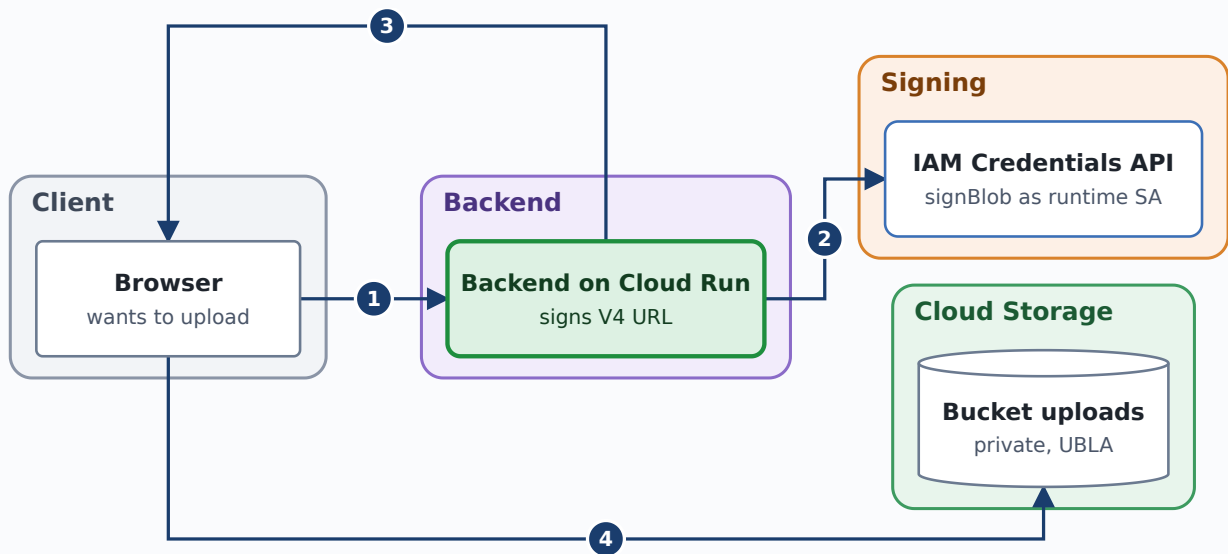
Turn on uniform bucket-level access so object ACLs are ignored and only IAM decides access, and enable public access prevention so no one can grant allUsers or allAuthenticatedUsers. Enforce both across the organization with the org policy constraints `storage.uniformBucketLevelAccess` and `storage.publicAccessPrevention`, so new buckets inherit them. Before switching, check whether any app depends on per-object ACLs and replace that with IAM or signed URLs. UBLA can be turned off again only within 90 days; after that it is permanent for the bucket.

KEY POINTS

- If content must be public, serve it through a load balancer with Cloud CDN from a dedicated bucket.
- Use IAM Conditions on object name prefixes for finer access.

Q182 Users upload videos of up to 5 GB from a browser. You do not want the files to pass through your Cloud Run backend and the bucket must stay private. How do you design it?

FLOW DIAGRAM



- 1 Request an upload slot for file.mp4
- 2 Sign with the service account, no key file
- 3 Return a signed PUT URL valid for 15 minutes
- 4 Browser uploads straight to Cloud Storage

EXPLANATION

The backend authenticates the user, then creates a V4 signed URL for a single object name with method PUT, the expected content type and a short expiry (V4 allows up to 7 days; use minutes). On Cloud Run there is no private key, so the client library signs through the IAM Credentials `signBlob` API; the runtime service account needs `roles/iam.serviceAccountTokenCreator` on itself. The browser uploads directly to the private bucket, so the backend never handles the bytes. For very large or unreliable uploads use a signed URL that starts a resumable upload, and configure CORS on the bucket for your web origin.

KEY POINTS

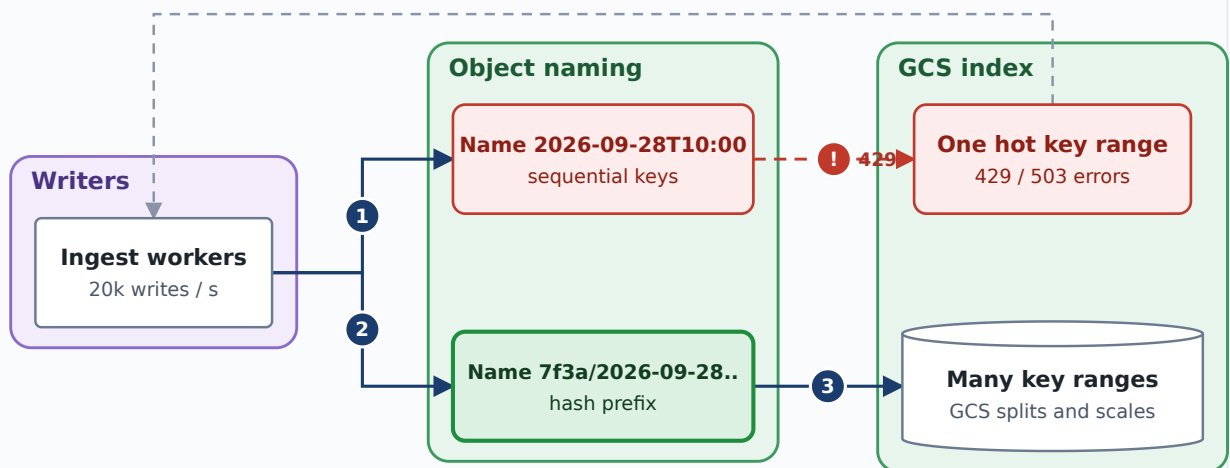
- Anyone holding the URL can use it until it expires: keep expiry short and one object per URL.
- React to the upload with a Pub/Sub notification or Eventarc trigger.

CONFIGURATION EXAMPLE

```
import google.auth, google.auth.transport.requests
from datetime import timedelta
from google.cloud import storage
creds, _ = google.auth.default()
creds.refresh(google.auth.transport.requests.Request())
blob = storage.Client().bucket("uploads").blob("u/123/file.mp4")
url = blob.generate_signed_url(version="v4", method="PUT",
    expiration=timedelta(minutes=15), content_type="video/mp4",
    service_account_email=creds.service_account_email, access_token=creds.token)
```

Q183 An ingest job writes 20,000 small objects per second named by timestamp, and during bursts it gets 429 and 503 errors from Cloud Storage. What is going on and how do you fix it?

FLOW DIAGRAM



- 1 Timestamp-first names all sort next to each other
- ! 429 every write lands on the same index range
- 2 Add a short hash in front of the name
- 3 Load spreads, so Cloud Storage can split ranges
- Retry with exponential backoff and jitter

EXPLANATION

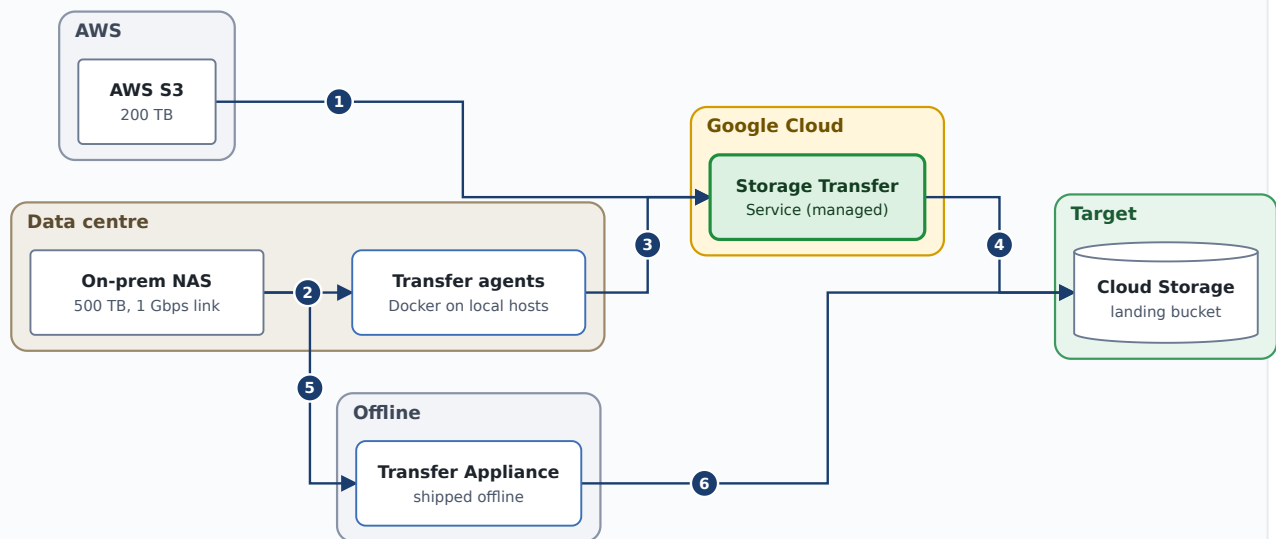
Cloud Storage stores object names in a sorted index split into key ranges. A bucket starts at roughly 1,000 object writes and 5,000 reads per second and scales up by splitting ranges, but sequential names (timestamps, counters) keep sending all writes to the newest range, so it cannot spread load. Put a short hash or random prefix before the timestamp, ramp traffic up gradually (the guideline is doubling about every 20 minutes), and retry 429/503 with exponential backoff. For many tiny files, batching them into larger objects (Avro, Parquet, tar) also cuts request cost; for large files use parallel composite uploads.

KEY POINTS

- The limit is per key range, not per bucket: spreading names matters more than more buckets.
- Check current request-rate guidance in the Cloud Storage docs.

Q184 You must move 200 TB from AWS S3 and 500 TB from an on-prem NAS (1 Gbps link) into Cloud Storage. Which transfer options do you use?

FLOW DIAGRAM



- ❶ Cloud-to-cloud job, no agents to run
- ❷ Agents read the file system (NFS / POSIX)
- ❸ Agents stream over VPN or Interconnect
- ❹ Checksummed, incremental, schedulable copies
- ❺ When the link is too slow, copy to an appliance
- ❻ Google uploads the data from its facility

EXPLANATION

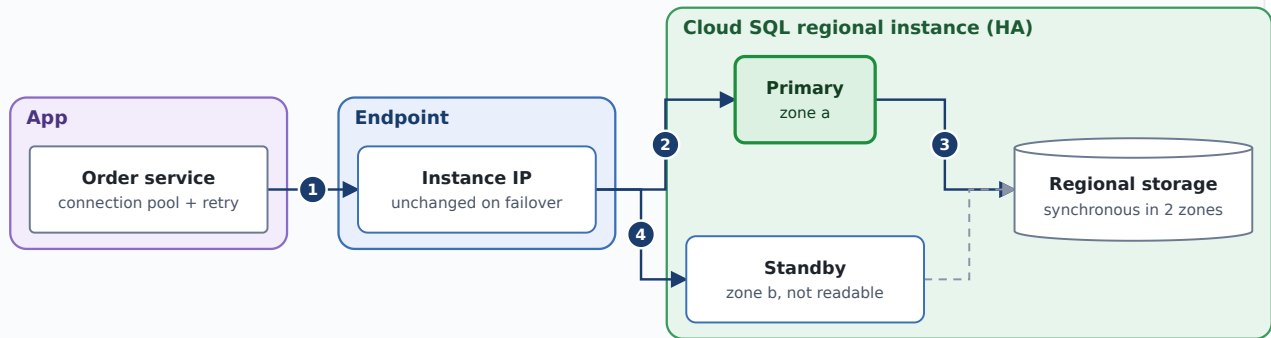
Use Storage Transfer Service for S3: it is managed, runs in parallel, verifies checksums and can repeat incrementally until cutover. For the NAS, first do the math: 500 TB over 1 Gbps is about 46 days at full line rate, and more in practice, so Transfer Appliance is usually the better choice; you copy data to the device and ship it back. If the timeline allows the network path, Storage Transfer Service with on-prem agents in an agent pool handles file-system sources and can throttle bandwidth. In both cases run a final incremental sync before switching applications.

KEY POINTS

- Rule of thumb: 1 Gbps moves about 10 TB per day at full utilisation.
- Use agents' bandwidth limits so the transfer does not starve production traffic.

Q185 An order service on Cloud SQL for PostgreSQL must survive a zone outage with minimal downtime. How do you configure it and what behaviour should the app expect?

FLOW DIAGRAM



- 1 App connects to one IP or connection name
- 2 All reads and writes go to the primary
- 3 Every write is replicated synchronously to both zones
- 4 Zone failure: standby becomes primary, IP moves
- Standby attaches the same data, so no committed writes are lost

EXPLANATION

Create the instance with `--availability-type=REGIONAL`. Cloud SQL keeps a standby in another zone of the same region and replicates writes synchronously, so a failover loses no committed transactions; the IP address and connection name stay the same. Failover typically completes in about a minute, during which existing connections drop, so the application needs connection retry with backoff and short connect timeouts. The standby cannot serve reads; add read replicas for read scaling. Test with a manual failover before going live.

KEY POINTS

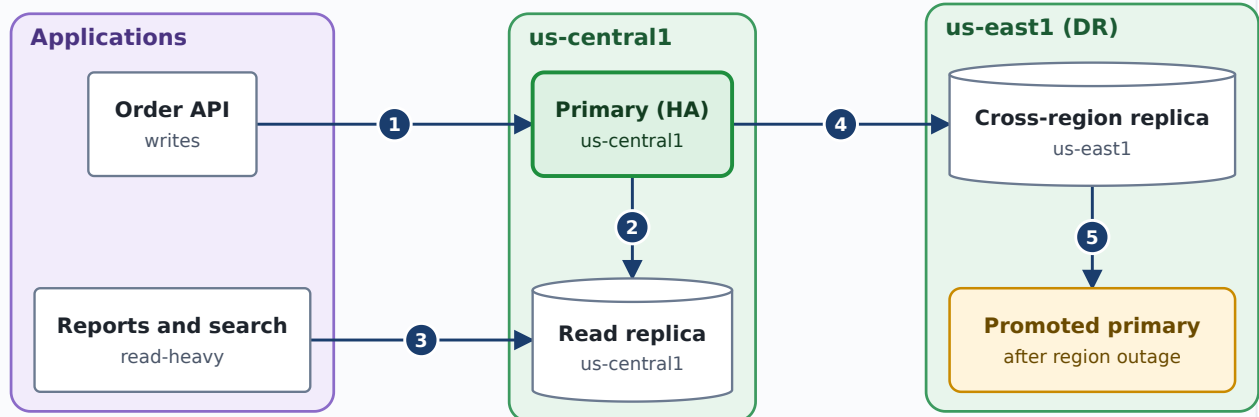
- HA protects against zone failure, not region failure: add a cross-region replica for DR.
- HA doubles the instance cost because the standby is a full-size instance.

CONFIGURATION EXAMPLE

```
gcloud sql instances create orders-db \
  --database-version=POSTGRES_16 --region=us-central1 \
  --availability-type=REGIONAL --tier=db-custom-4-16384
# rehearse a failover
gcloud sql instances failover orders-db
```

Q186 Reporting queries are slowing down the primary Cloud SQL instance, and the business also wants a plan for a full region outage. How do you use replicas?

FLOW DIAGRAM



- 1 Writes only go to the primary
- 2 Asynchronous replication, small lag possible
- 3 Read traffic uses the replica's own IP
- 4 Asynchronous replication to another region
- 5 Region down: promote the replica and repoint apps

EXPLANATION

Create an in-region read replica and point reporting and search at its own IP (or a pooler that routes reads), which takes load off the primary. Replication is asynchronous, so reads can be slightly stale; code that must read its own writes should use the primary. For region failure, add a cross-region replica and document the promotion runbook: promotion makes it a standalone primary (it does not fail back automatically), and apps must be repointed through DNS or config. Cloud SQL Enterprise Plus also offers a DR replica with switchover that keeps the topology.

KEY POINTS

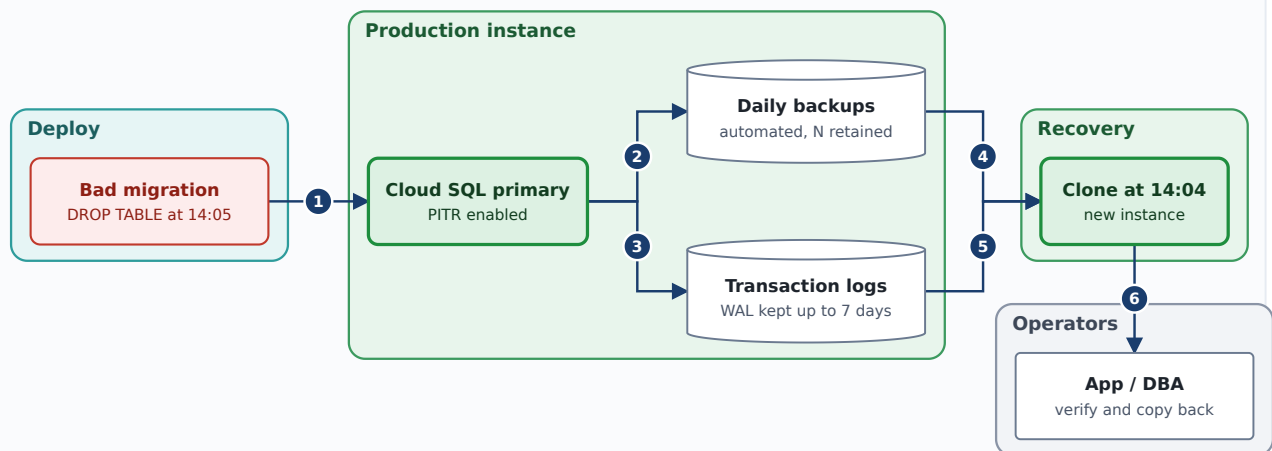
- Watch `replica_lag` / `replication_lag` metrics and alert on them.
- A promoted replica cannot rejoin as a replica; you rebuild replication afterwards.

CONFIGURATION EXAMPLE

```
gcloud sql instances create orders-read --master-instance-name=orders-db \
  --region=us-central1
gcloud sql instances create orders-dr --master-instance-name=orders-db \
  --region=us-east1
# during a regional outage
gcloud sql instances promote-replica orders-dr
```


Q187 A bad migration dropped a table in production at 14:05. Cloud SQL has automated backups and point-in-time recovery enabled. How do you recover?

FLOW DIAGRAM



- 1 The migration runs and drops the table
- 2 Backups taken daily in the backup window
- 3 Logs retained for the PITR window
- 4 Clone starts from the last backup before 14:04
- 5 Logs are replayed up to the chosen timestamp
- 6 Check the data, then export the table back or cut over

EXPLANATION

Do not restore over the production instance. Clone it to a new instance at a timestamp just before the drop, for example 14:04 UTC; Cloud SQL restores the nearest earlier backup and replays the transaction logs up to that point. Verify the table in the clone, then either export and reload just that table into production (keeping writes made after 14:05) or cut the application over to the clone. Retained transaction log days are limited (up to 7 on Enterprise, longer on Enterprise Plus), so check the window first.

KEY POINTS

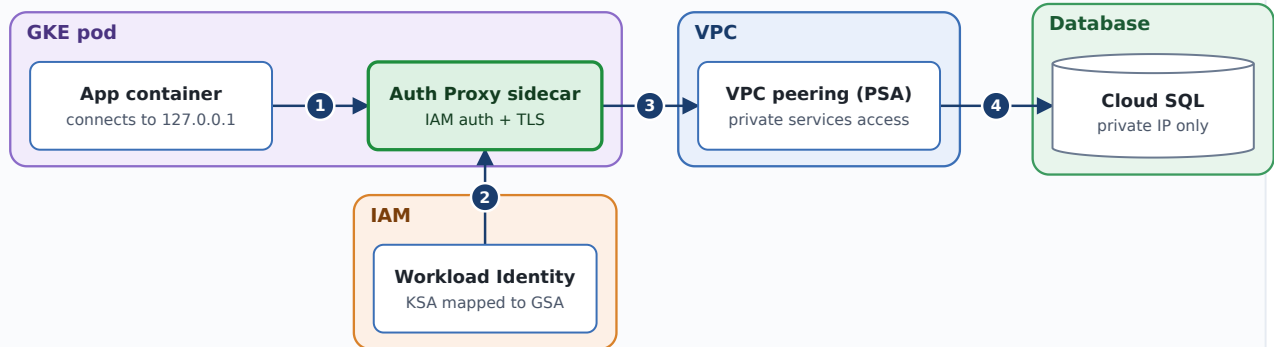
- Restoring a backup onto an existing instance overwrites all its data.
- Run migrations with a pre-deploy on-demand backup to shorten recovery.

CONFIGURATION EXAMPLE

```
gcloud sql instances patch orders-db --backup-start-time=02:00 \
  --retained-backups-count=14 --enable-point-in-time-recovery \
  --retained-transaction-log-days=7
gcloud sql instances clone orders-db orders-restore \
  --point-in-time='2026-09-28T14:04:00Z'
```

Q188 A GKE workload must reach Cloud SQL without a public IP and without storing database certificates or passwords in the pod. How do you connect it?

FLOW DIAGRAM



- 1 App uses localhost:5432, no certs in the image
- 2 Proxy gets a token for a GSA with `cloudsql.client`
- 3 Encrypted tunnel to the instance's private IP
- 4 Instance has no public IP at all

EXPLANATION

Give the instance a private IP through private services access (or Private Service Connect) and disable the public IP. Run the Cloud SQL Auth Proxy as a sidecar with `--private-ip`; it authorises with IAM (the Google service account mapped through Workload Identity needs `roles/cloudsql.client`) and wraps the connection in TLS, so the app just talks to localhost. Add `--auto-iam-authn` and create an IAM database user to remove the password as well. The language connectors (Java, Python, Go, Node) do the same job in-process if you prefer no sidecar.

KEY POINTS

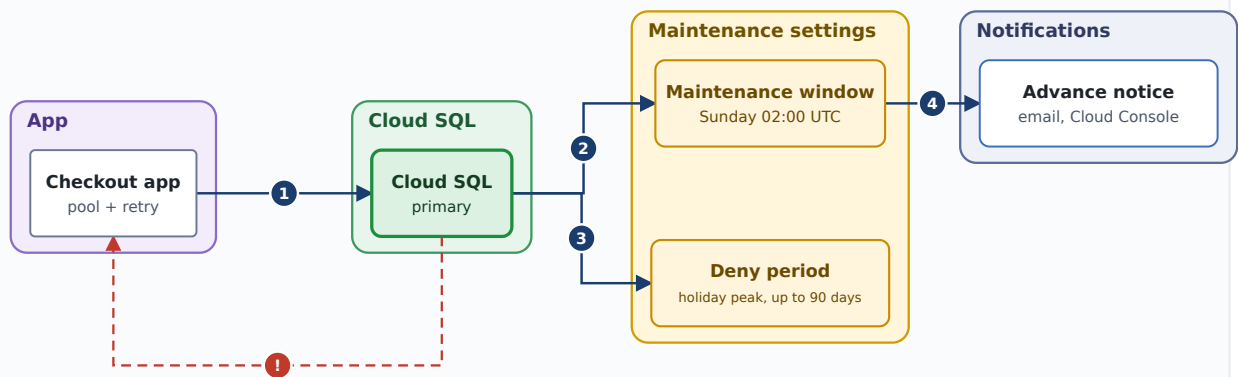
- Private services access needs an allocated IP range sized for future instances.
- Run the proxy as a native sidecar (init container with `restartPolicy Always`) so it starts first.

CONFIGURATION EXAMPLE

```
- name: cloud-sql-proxy
  image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.11.0
  args: ["--private-ip", "--auto-iam-authn", "--port=5432",
        "prod-app:us-central1:orders-db"]
  securityContext:
    runAsNonRoot: true
```

Q189 A Cloud SQL instance restarted for maintenance on a Tuesday afternoon and the checkout app threw errors. How do you prevent this?

FLOW DIAGRAM



- 1 Normal traffic
- 2 Updates only run in the chosen hour
- 3 No maintenance during the deny period
- 4 Opt in to maintenance notifications
- ! Short disconnect during maintenance: app must reconnect

EXPLANATION

Set a maintenance window on a quiet day and hour, and add deny maintenance periods (up to 90 days) around peaks such as sales events. Choose the maintenance timing (earlier or later in the rollout) so test instances receive updates before production. Maintenance still causes a short disconnect, so the app must retry connections and transactions; the Auth Proxy and connectors handle reconnects well. Subscribe to maintenance notifications, and reschedule an upcoming event from the console if it clashes with a release.

KEY POINTS

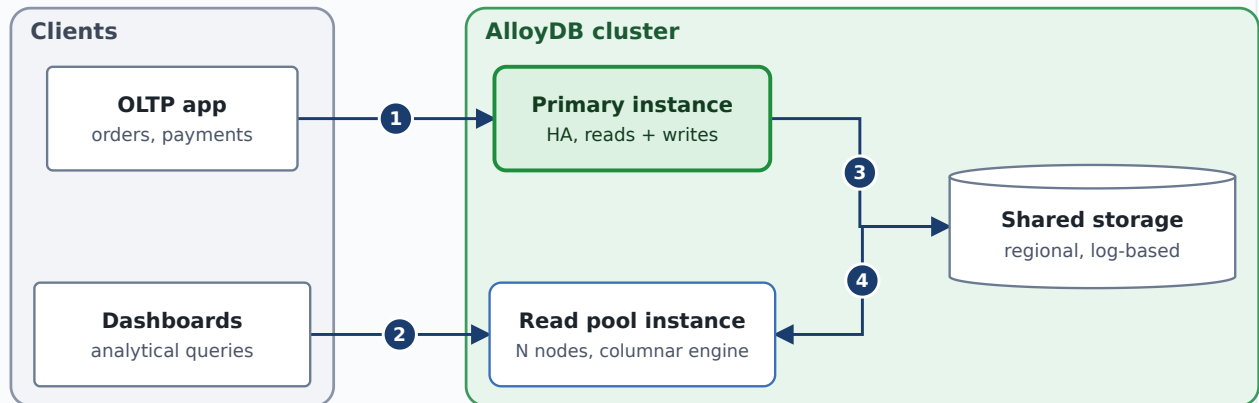
- Without a window, maintenance can happen at any time.
- Enterprise Plus has near-zero downtime maintenance; check current behaviour.

CONFIGURATION EXAMPLE

```
gcloud sql instances patch orders-db \
  --maintenance-window-day=SUN --maintenance-window-hour=2
gcloud sql instances patch orders-db \
  --deny-maintenance-period-start-date=2026-11-20 \
  --deny-maintenance-period-end-date=2027-01-05 \
  --deny-maintenance-period-time=00:00:00
```

Q190 A PostgreSQL OLTP system on Cloud SQL now also serves heavy analytical dashboards and is hitting CPU limits. When would you move it to AlloyDB?

FLOW DIAGRAM



- ❶ Transactions go to the primary
- ❷ Analytics use the read pool endpoint
- ❸ Writes go to the distributed storage layer
- ❹ Read pool reads the same storage, no full copy

EXPLANATION

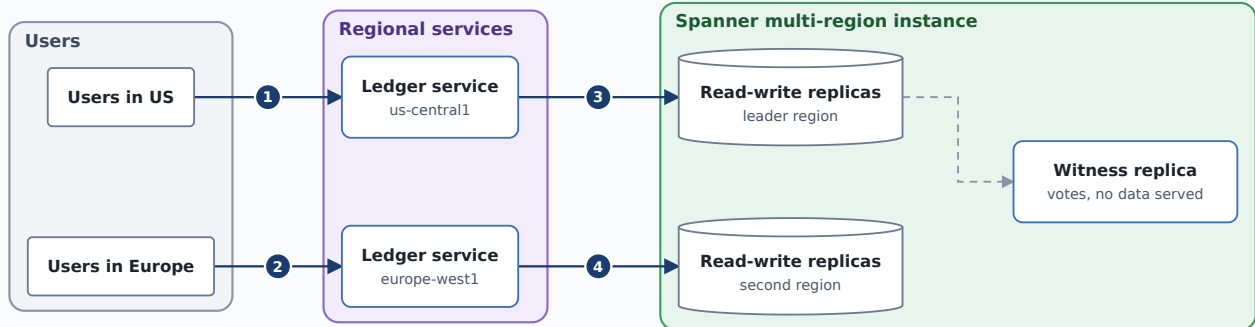
AlloyDB is PostgreSQL-compatible with a disaggregated storage layer, so read pool nodes share storage with the primary instead of each holding a copy, and replica lag is low. Its columnar engine keeps hot columns in memory in a columnar format, which speeds up analytical queries on the same data without an ETL job. Choose it when you need higher transactional throughput than Cloud SQL, many read replicas behind one endpoint, or mixed OLTP and analytics. Stay on Cloud SQL for smaller or cost-sensitive databases, or when you need MySQL or SQL Server.

KEY POINTS

- Migrate with Database Migration Service from Cloud SQL or self-managed PostgreSQL.
- Very heavy analytics still belong in BigQuery.

Q191 A payments ledger must accept writes from users in the US and Europe, stay strongly consistent, and target 99.999% availability. Why would you choose Spanner and how is it set up?

FLOW DIAGRAM



- ③ Writes commit when a Paxos quorum across regions agrees
- ④ Strong reads served locally after a freshness check
- Witness helps form a quorum if a region fails

EXPLANATION

Spanner is a relational database that scales writes horizontally and gives external consistency across regions, using synchronous Paxos replication and TrueTime timestamps. A multi-region configuration has a 99.999% availability SLA, so a region can fail without losing committed writes or needing a manual failover. Choose it when data outgrows a single Cloud SQL or AlloyDB primary, or when you need strong consistency across regions; the trade-off is higher write latency (cross-region quorum) and cost. Size it with processing units or nodes and enable autoscaling.

KEY POINTS

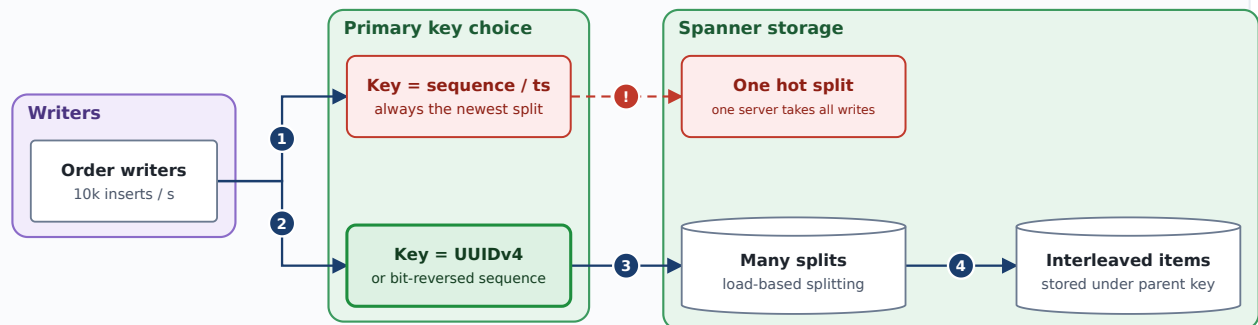
- Stale reads (e.g. 15 seconds old) are served by the nearest replica with lower latency.
- Spanner supports GoogleSQL and a PostgreSQL dialect.

CONFIGURATION EXAMPLE

```
gcloud spanner instances create ledger \
  --config=nam-eur-asia1 --processing-units=1000 \
  --description="Payments ledger"
gcloud spanner databases create ledgerdb --instance=ledger
```

Q192 A new Spanner table uses an auto-incrementing order ID as primary key. Write latency rises and one server shows most of the load. What is wrong and how do you design the schema?

FLOW DIAGRAM



- 1 Monotonic keys sort to the end of the table
- ! high latency and lock contention on one split
- 2 Random-looking keys spread over the key space
- 3 Spanner splits and balances the load
- 4 Child rows sit next to their parent row

EXPLANATION

Spanner orders rows by primary key and splits the key space across servers, so monotonically increasing keys (sequences, timestamps) send every insert to the last split and create a hotspot. Use a UUIDv4 or a bit-reversed sequence as the key, or put a well-distributed column such as customer ID first. Interleave child tables (order items under orders) so related rows are stored together and joins are local. Also avoid indexes whose leading column is a timestamp, because the index has the same hotspot problem.

KEY POINTS

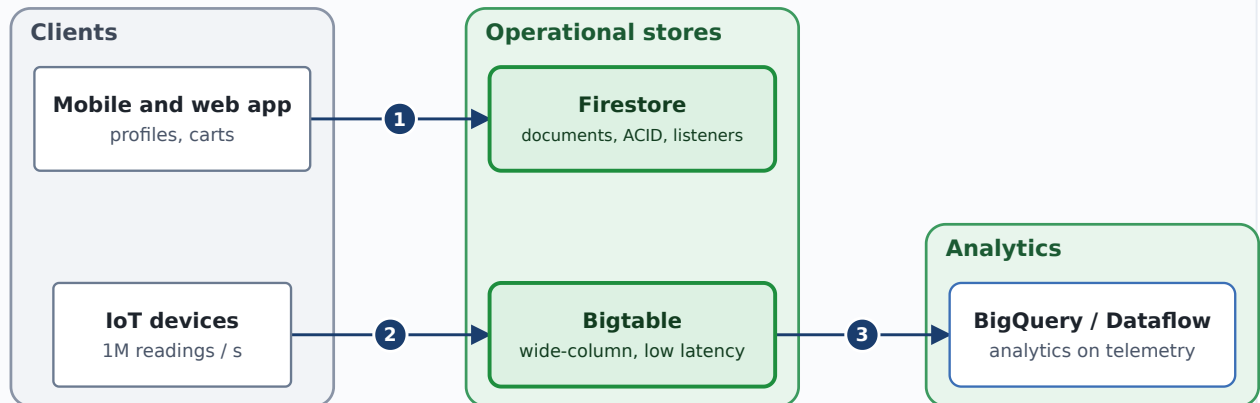
- Key Visualizer shows hot key ranges over time.
- Commit timestamps are fine as a column, not as the first key part.

CONFIGURATION EXAMPLE

```
CREATE TABLE Orders (  
  OrderId STRING(36) NOT NULL,    -- UUIDv4, not a sequence  
  CustomerId STRING(36) NOT NULL,  
  CreatedAt TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),  
) PRIMARY KEY (OrderId);  
CREATE TABLE OrderItems (  
  OrderId STRING(36) NOT NULL, ItemId INT64 NOT NULL, Sku STRING(64),  
) PRIMARY KEY (OrderId, ItemId), INTERLEAVE IN PARENT Orders ON DELETE CASCADE;
```

Q193 A product team needs a store for user profiles and carts in a mobile app, and another for 1 million sensor readings per second. Would you pick Firestore or Bigtable for each?

FLOW DIAGRAM



- ❶ SDKs with offline sync, security rules, real-time updates
- ❷ Huge write throughput, single-row atomic writes
- ❸ Batch or streaming jobs aggregate the time series

EXPLANATION

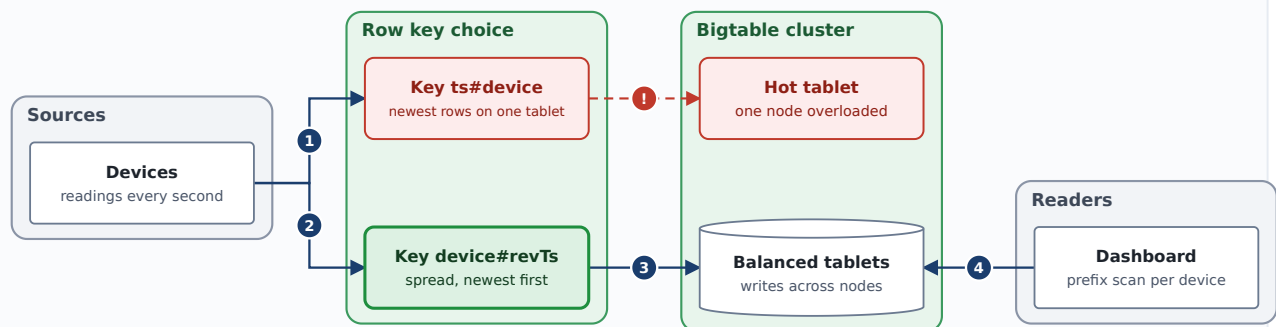
Firestore fits the app data: a serverless document database with multi-document transactions, secondary indexes, real-time listeners and client SDKs with offline support, billed per operation. Bigtable fits the telemetry: a wide-column store that scales linearly with nodes to millions of operations per second at single-digit millisecond latency, but with atomicity only per row and access mainly by row key or key range. Bigtable needs row key design and a minimum node cost, so it is overkill for small datasets. Neither is a warehouse, so send analytical workloads to BigQuery.

KEY POINTS

- Firestore: flexible queries on documents. Bigtable: key-range scans at scale.
- Bigtable is HBase API compatible, which eases migration from HBase.

Q194 Sensor data in Bigtable uses the row key `timestamp#deviceId`. Writes are slow and one node is overloaded, and dashboards need the latest readings per device. How do you redesign the key?

FLOW DIAGRAM



- 1 Timestamp first: all current writes are adjacent
- ! slow writes on a single tablet
- 2 Device ID first, then reversed timestamp
- 3 Writes spread over many tablets
- 4 Scan prefix device# returns the latest rows first

EXPLANATION

Bigtable sorts rows by key and assigns key ranges (tablets) to nodes, so a timestamp-first key sends all current writes to one tablet. Lead with a high-cardinality field such as device ID and append a reversed timestamp (max value minus the timestamp), so writes spread across tablets and a prefix scan on the device returns the newest readings first. Keep keys short, avoid hashing if you need range scans, and use garbage-collection policies (max versions or max age) on column families. Check the result with Key Visualizer.

KEY POINTS

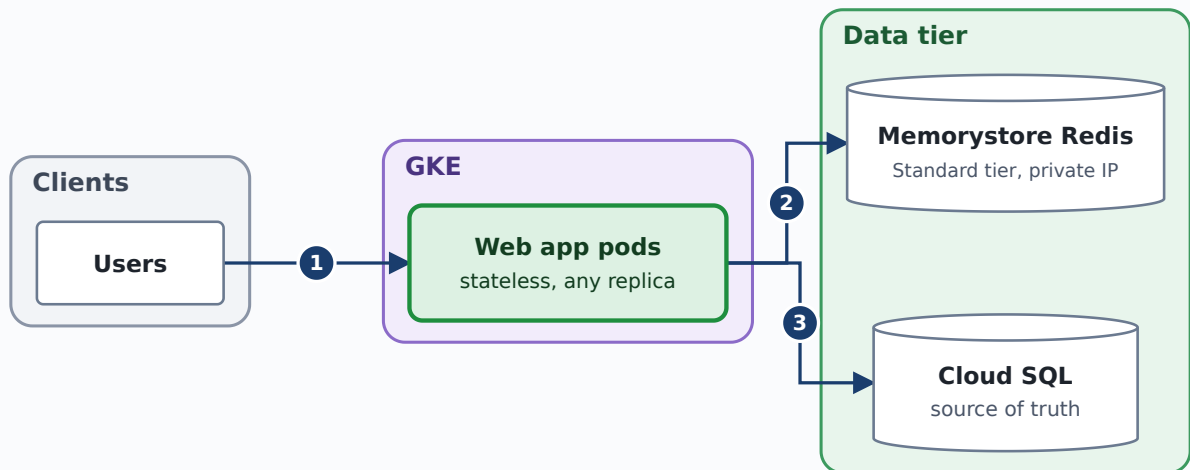
- Field promotion: move query fields into the key instead of relying on filters.
- Very busy single devices may still need a salt or bucket in the key.

CONFIGURATION EXAMPLE

```
cbt -instance=iot createtable telemetry
cbt -instance=iot createfamily telemetry m
cbt -instance=iot setgcpolicy telemetry m maxage=30d
# row key: <deviceId>#<Long.MAX_VALUE - epochMillis>
cbt -instance=iot read telemetry prefix=dev-4711# count=10
```


Q195 A stateless web app on GKE needs shared user sessions and a cache in front of Cloud SQL for product pages. How do you use Memorystore?

FLOW DIAGRAM



- ❶ Any pod can serve any request
- ❷ Read session and cached product by key
- ❸ On a cache miss, read the database
- Cache-aside: after a miss, SET the key with a TTL (e.g. 300 s); sessions use a TTL equal to session lifetime

EXPLANATION

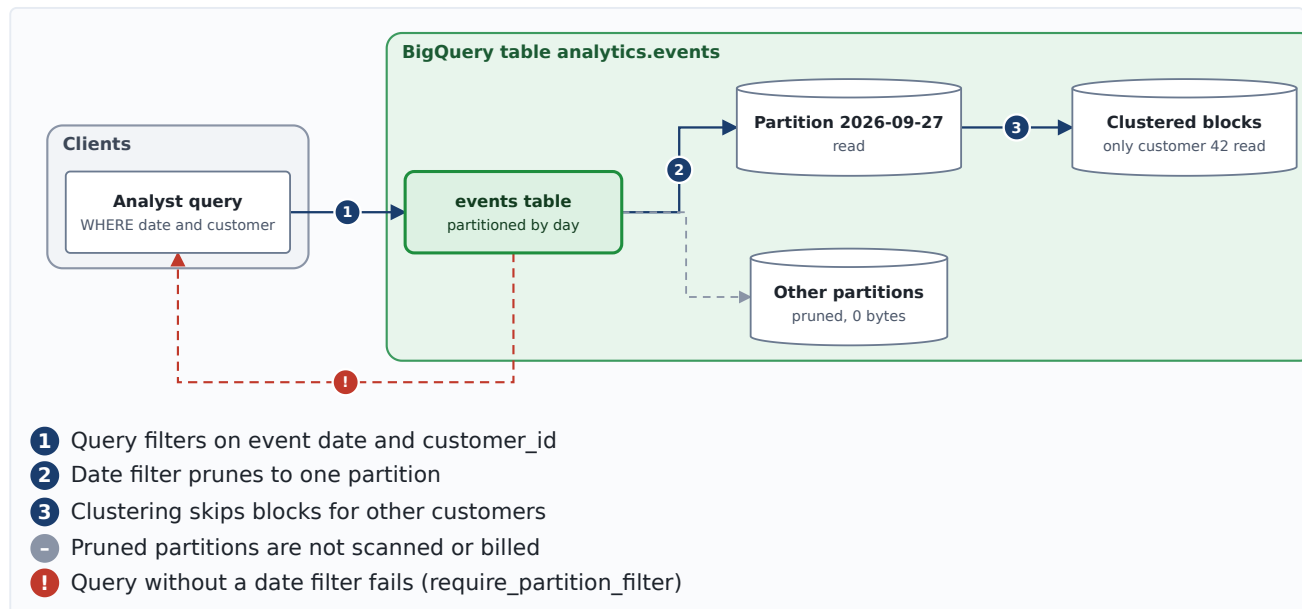
Use Memorystore for Redis Standard tier, which runs a replica in another zone with automatic failover, and connect over its private IP from the VPC (Cloud Run needs Direct VPC egress or a connector). Store sessions under a session ID with a TTL, so any pod can serve the user. For product pages use cache-aside: read Redis, on a miss read Cloud SQL and write the value with a TTL, and delete the key when the product changes. Set `maxmemory-policy` (for example `allkeys-lru` for a pure cache) and watch memory usage and evictions.

KEY POINTS

- Basic tier has no replica: a failure loses all data.
- For very large or write-heavy caches consider Memorystore for Redis Cluster or Valkey.

Q196 A 20 TB BigQuery events table is queried by date and customer, and every query scans the whole table. How do you redesign it?

FLOW DIAGRAM



EXPLANATION

Partition the table by the event date (`PARTITION BY DATE(event_ts)`) so a date filter prunes whole partitions, and cluster by the columns most used in filters, such as `customer_id` and `event_type`, so BigQuery skips storage blocks within each partition. Set `require_partition_filter` so a query without a date predicate fails instead of scanning 20 TB. A partition expiration drops old data automatically. The dry run estimate shows partition pruning, but clustering savings only show in the bytes actually billed.

KEY POINTS

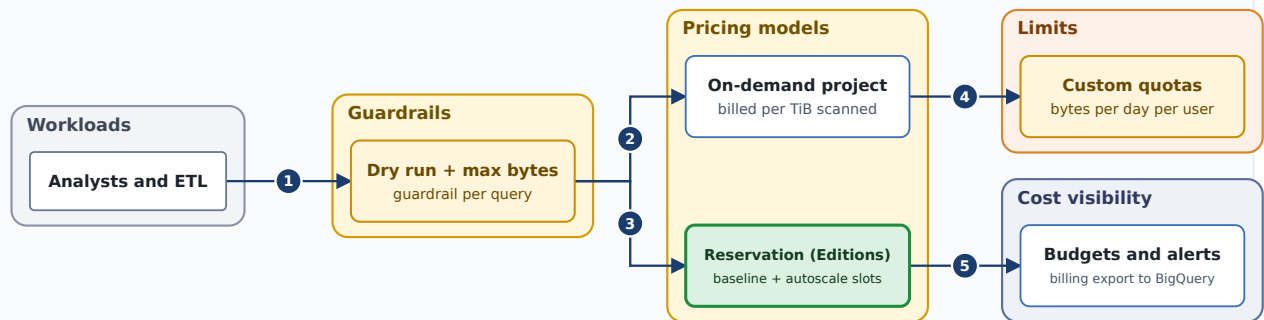
- Partition limit is 10,000 per table: use monthly partitions for very long histories.
- Filters must be on the partition column directly, not on a function of it, to prune.

CONFIGURATION EXAMPLE

```
CREATE TABLE analytics.events (  
  event_ts TIMESTAMP, customer_id STRING, event_type STRING, payload JSON)  
PARTITION BY DATE(event_ts)  
CLUSTER BY customer_id, event_type  
OPTIONS (require_partition_filter = TRUE, partition_expiration_days = 400);  
  
INSERT INTO analytics.events SELECT * FROM analytics.events_old;
```

Q197 BigQuery spend is unpredictable: nightly ETL is steady, but analysts run large ad-hoc queries. How do you choose between on-demand and capacity pricing and control cost?

FLOW DIAGRAM



- 1 Estimate bytes before running
- 2 Ad-hoc analyst project stays on-demand
- 3 ETL project is assigned to a reservation
- 4 Daily quota caps runaway scans
- 5 Track slot usage and spend against budget

EXPLANATION

On-demand bills per TiB scanned and suits spiky, smaller usage; capacity pricing (Standard, Enterprise, Enterprise Plus editions) bills for slots, with a baseline plus autoscaling up to a maximum, and suits steady heavy workloads such as ETL. Split workloads into projects and assign only the ETL project to a reservation, leaving ad-hoc analysis on-demand with custom quotas on bytes scanned per user and per project per day. Add `maximum_bytes_billed` on scheduled and BI queries, and alert on spend using billing export and `INFORMATION_SCHEMA.JOBS`. Revisit the split every month using actual slot usage.

KEY POINTS

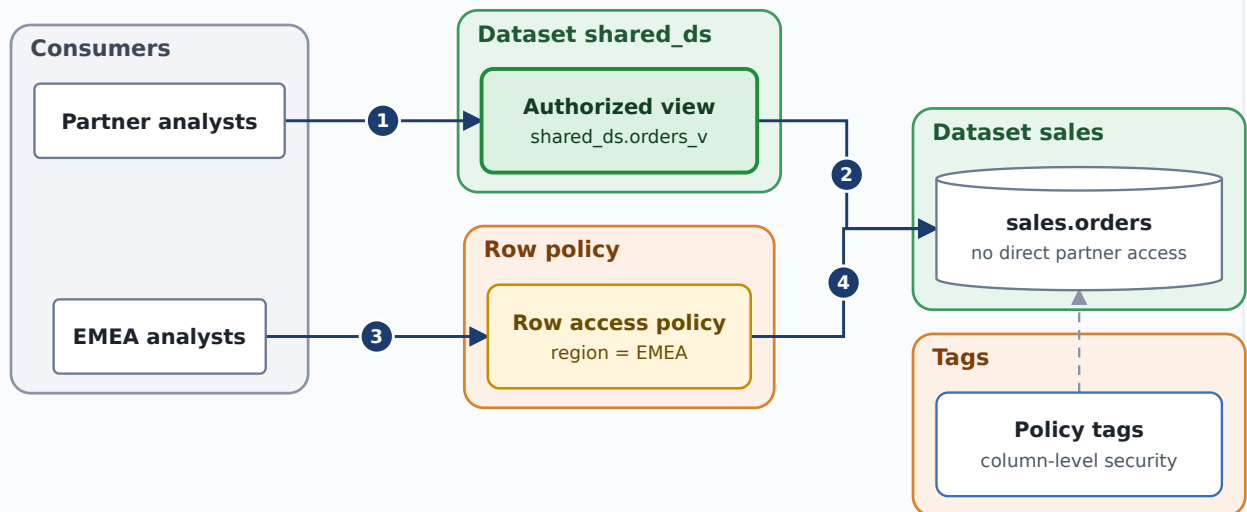
- Commitments (1 or 3 years) lower the slot price for the baseline.
- Query cache hits and table previews are free on-demand.

CONFIGURATION EXAMPLE

```
bq query --use_legacy_sql=false --dry_run 'SELECT ...'
bq query --use_legacy_sql=false --maximum_bytes_billed=100000000000 'SELECT ...'
bq mk --project_id=bq-admin --location=US --reservation \
  --edition=ENTERPRISE --slots=100 --autoscale_max_slots=400 etl
bq mk --reservation_assignment --reservation_id=bq-admin:US.etl \
  --job_type=QUERY --assignee_type=PROJECT --assignee_id=etl-prod
```

Q198 A partner must see only three columns of the sales table, and EMEA analysts must see only EMEA rows, without copying data. How do you do this in BigQuery?

FLOW DIAGRAM



- 1 Partner has dataViewer on shared_ds only
- 2 View is authorized on the sales dataset
- 3 Analysts query the table directly
- 4 Only rows with region = EMEA are returned
- PII columns need Fine-Grained Reader on the tag

EXPLANATION

Create a view that selects only the allowed columns in a separate dataset, grant the partner access to that dataset only, and add the view as an authorized view on the source dataset, so it can read the table on the partner's behalf. For the analysts, create a row access policy on the table that grants the EMEA group access filtered on `region = 'EMEA'`; users not covered by any policy see no rows. Use policy tags for column-level security on PII, and dynamic data masking if some users should see masked values. No data is copied, so there is one source of truth.

KEY POINTS

- Once any row access policy exists, users need a matching policy to see rows.
- Authorized datasets let every view in a dataset be authorized at once.

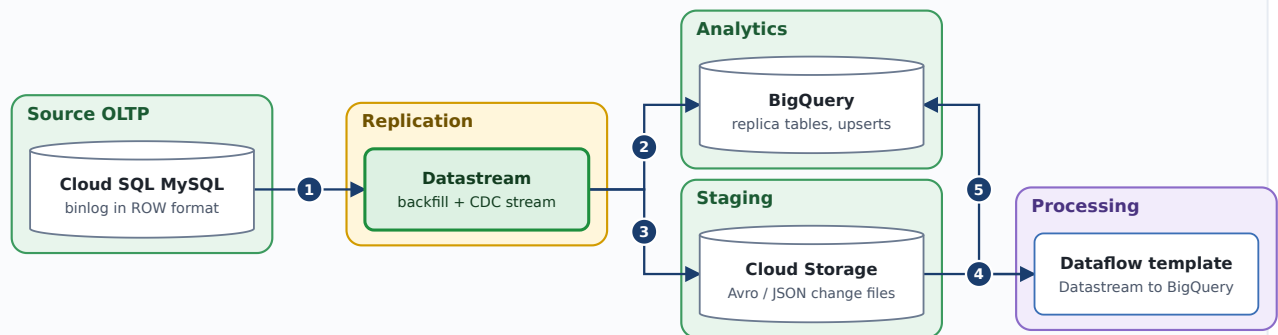
CONFIGURATION EXAMPLE

```
CREATE VIEW shared_ds.orders_v AS
SELECT order_id, order_date, amount FROM sales.orders;
-- then add shared_ds.orders_v as an authorized view on dataset sales

CREATE ROW ACCESS POLICY emea_only ON sales.orders
GRANT TO ("group:emea-analysts@example.com")
FILTER USING (region = "EMEA");
```

Q199 Analysts want orders from a Cloud SQL for MySQL database in BigQuery within minutes, without nightly exports that load the primary. How do you build it?

FLOW DIAGRAM



- ❶ Reads the binlog, no triggers on the source
- ❷ Native BigQuery destination applies changes
- ❸ Option: land change files for custom processing
- ❹ Transform or enrich the events
- ❺ Merge into BigQuery tables

EXPLANATION

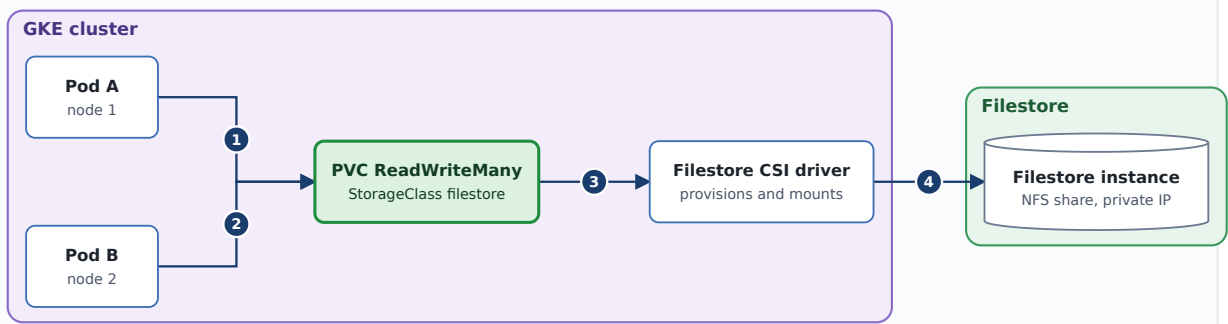
Use Datastream: it does an initial backfill and then reads the MySQL binlog (or PostgreSQL logical replication, Oracle LogMiner) and streams changes with low impact on the source. The simplest path writes directly into BigQuery, which applies inserts, updates and deletes to replica tables with a configurable staleness. If you need transformation, write to Cloud Storage and use the Datastream-to-BigQuery Dataflow template. Connect privately with a private connectivity configuration (VPC peering) and give Datastream a dedicated replication user.

KEY POINTS

- Source needs binlog in ROW format and enough binlog retention.
- Set the BigQuery max staleness to trade freshness against cost.

Q200 Several GKE pods on different nodes must read and write the same files (a legacy CMS). Persistent disks only allow one writer node. What do you use?

FLOW DIAGRAM



- ❶ Both pods mount the same claim
- ❸ Claim triggers dynamic provisioning
- ❹ Creates the instance and mounts NFS on each node

EXPLANATION

Use Filestore, a managed NFS file server, through the GKE Filestore CSI driver, which supports the ReadWriteMany access mode. Enable the driver on the cluster and use a built-in class such as `standard-rwx` or `enterprise-rwx`, or define your own StorageClass with the tier and VPC. Pick the tier by durability and performance: Basic tiers are zonal, while Regional and Enterprise tiers survive a zone failure. Basic tiers have a large minimum size (1 TiB or more), so check the minimum capacity and pricing per tier.

KEY POINTS

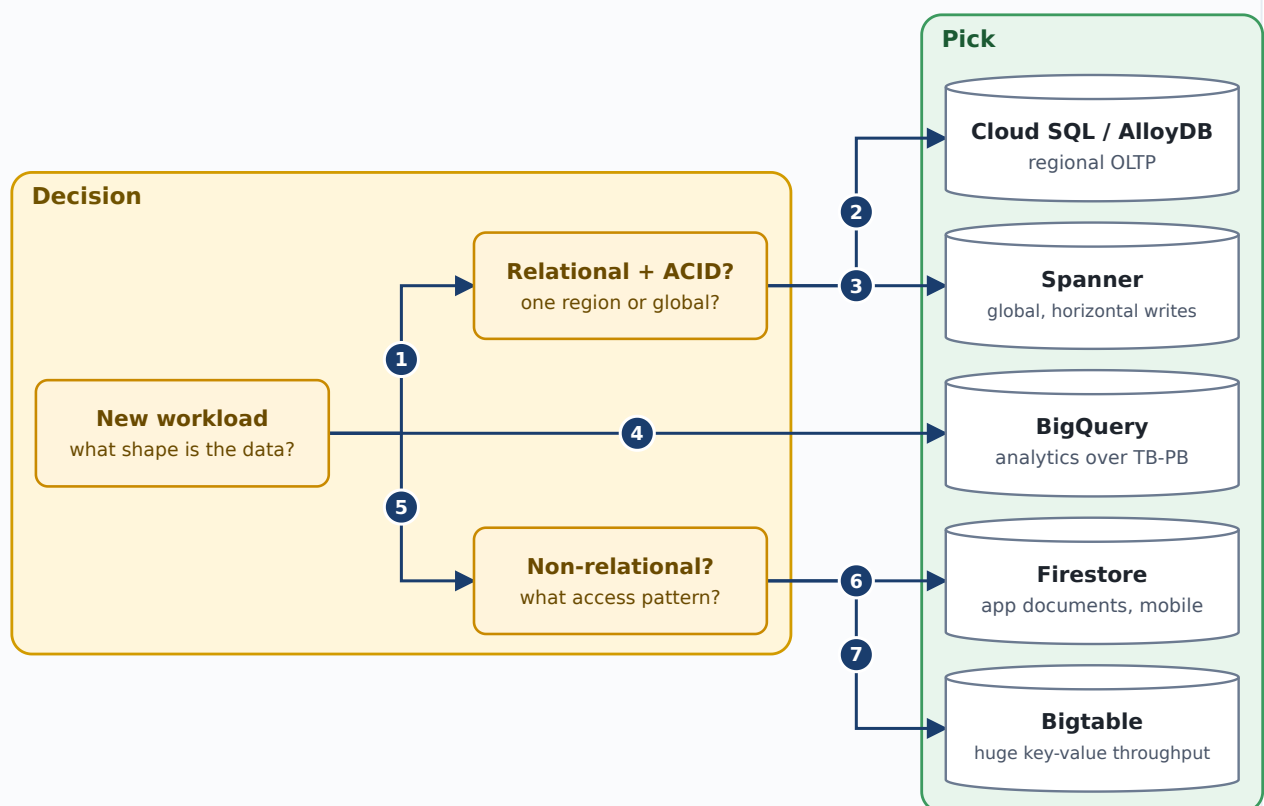
- Cloud Storage FUSE is an option for read-heavy object access, not for POSIX locking.
- Filestore uses a private IP in your VPC, so plan the reserved range.

CONFIGURATION EXAMPLE

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: filestore-rwx
provisioner: filestore.csi.storage.gke.io
parameters:
  tier: standard
  network: prod-vpc
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Q201 In a design review you are asked how you choose the database for a new workload on Google Cloud. Walk through your decision process.

FLOW DIAGRAM



- 1 Needs joins, transactions, SQL
- 2 Fits one primary in one region
- 3 Needs global scale or 99.999%
- 4 Scans and aggregations for analysis
- 5 Key-value or document access
- 6 Flexible documents, real-time sync
- 7 Time series, IoT, very high QPS
- Memorystore sits in front of any of these as a cache; Firestore and Cloud Storage cover files and objects

EXPLANATION

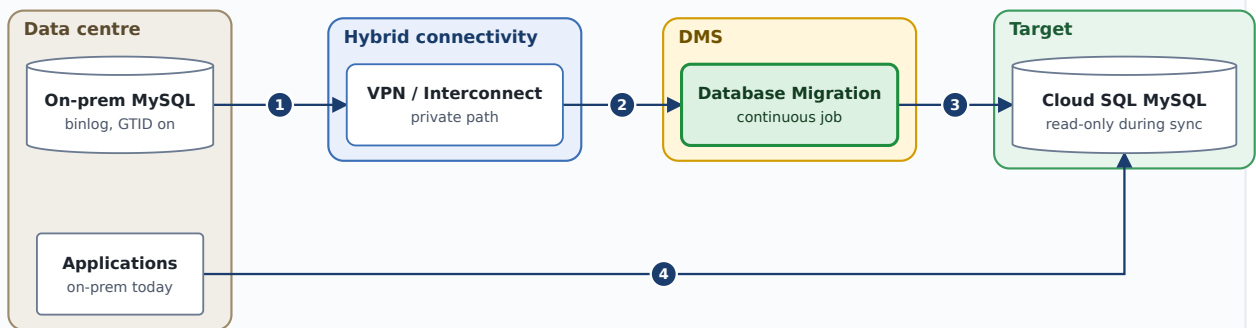
Start from the access pattern and consistency needs, not from the product list. Relational data with transactions goes to Cloud SQL (MySQL, PostgreSQL, SQL Server) or AlloyDB for higher PostgreSQL performance; if it must scale writes beyond one primary or run active in several regions with strong consistency, use Spanner. Analytical scans over large data belong in BigQuery. Document data for apps fits Firestore, and very high-throughput key-range access such as time series fits Bigtable; Memorystore adds a cache in front of any of them. Then check non-functional needs: RPO/RTO, regional versus multi-regional, team skills, and cost at the expected size.

KEY POINTS

- Most systems use more than one store: OLTP plus BigQuery for analytics is common.
- Pick the simplest option that meets the scale and availability target.

Q202 You must migrate a 2 TB on-prem MySQL database to Cloud SQL with less than 10 minutes of downtime. How do you plan it?

FLOW DIAGRAM



- ❶ Private connectivity (VPC peering over VPN)
- ❷ Job reads the source with a replication user
- ❸ Full dump, then continuous binlog replication
- ❹ Stop writes, lag reaches 0, promote, repoint apps

EXPLANATION

Use Database Migration Service with a continuous migration job: it takes an initial dump and then replicates changes from the binlog, so the target stays close to the source while you test. Prepare the source (binlog in ROW format, GTID, a replication user, enough binlog retention) and connect privately over VPN or Interconnect. When the replication delay is near zero, stop writes on the source, wait for lag to reach zero, promote the Cloud SQL instance and point applications at it; that window is the only downtime. Rehearse the whole cutover once and keep the source intact as the rollback path.

KEY POINTS

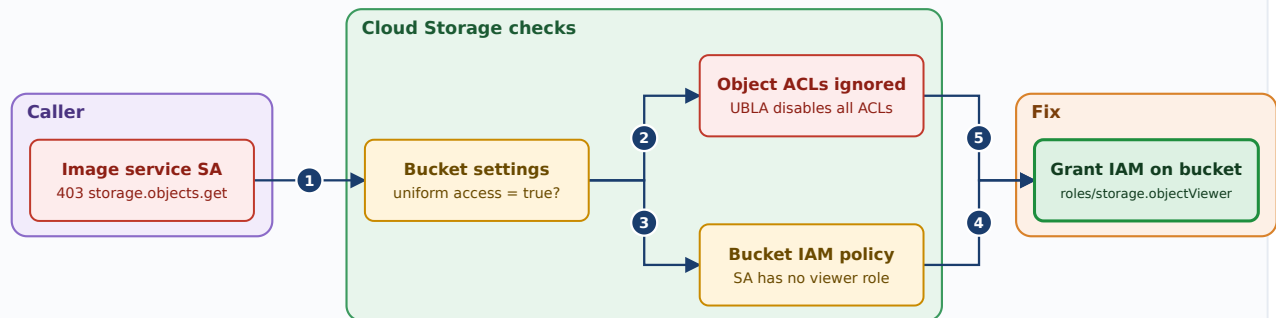
- Run the DMS test step before starting the job; it checks connectivity and source settings.
- Users, grants and some objects may need to be recreated after promotion.

CONFIGURATION EXAMPLE

```
gcloud database-migration migration-jobs create mysql-to-csdl \
  --region=us-central1 --type=CONTINUOUS \
  --source=onprem-mysql-cp --destination=csql-orders-cp \
  --peer-vpc=projects/prod-app/global/networks/prod-vpc
gcloud database-migration migration-jobs start mysql-to-csdl --region=us-central1
gcloud database-migration migration-jobs promote mysql-to-csdl --region=us-central1
```


Q203**Troubleshooting**

After the security team enabled uniform bucket-level access on `gs://media-prod`, the image service gets `403 AccessDenied: ... does not have storage.objects.get access`, although every object still has a **READER ACL** for its service account. How do you fix it?

FLOW DIAGRAM

- ❶ Error names the permission and the bucket
- ❷ With UBLA on, only IAM decides access
- ❸ `get-iam-policy`: no binding for this SA
- ❹ Grant the role at bucket level (or a managed folder)

EXPLANATION

Uniform bucket-level access turns off object and bucket ACLs, so the old per-object **READER** grants no longer count; only **IAM** on the bucket (or on managed folders) decides access. Check the bucket with `gcloud storage buckets describe` and read its **IAM** policy: the service account usually has no binding because it relied on ACLs. Grant `roles/storage.objectViewer` (or `objectAdmin` for writers) to that service account on the bucket. If the **IAM** binding exists and access is still denied, look for public access prevention (for `allUsers` grants), an **IAM** deny policy or a **VPC Service Controls** perimeter.

KEY POINTS

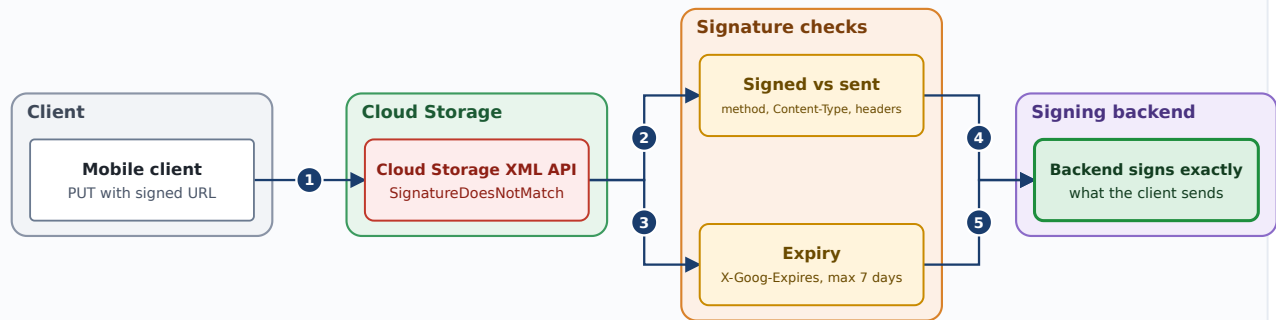
- Before enabling **UBLA**, list which principals rely on **ACLs** and convert them to **IAM** bindings; **UBLA** can be reverted only within 90 days.
- Use managed folders when different prefixes need different readers.

CONFIGURATION EXAMPLE

```

gcloud storage buckets describe gs://media-prod \
  --format='yaml(uniform_bucket_level_access,public_access_prevention)'
gcloud storage buckets get-iam-policy gs://media-prod
gcloud storage buckets add-iam-policy-binding gs://media-prod \
  --member=serviceAccount:img-svc@prod.iam.gserviceaccount.com \
  --role=roles/storage.objectViewer
  
```

FLOW DIAGRAM



- 1 Request is checked against the signature
- 2 Error body shows the StringToSign GCS expected
- 3 Link older than its expiry or clock skew
- 4 Sign PUT with the same Content-Type header
- 5 Issue fresh links on demand, not in emails

EXPLANATION

`SignatureDoesNotMatch` means the request differs from what was signed: a different HTTP method (URL signed for GET, used for PUT), a `Content-Type` or other signed header that the client sends differently, or a changed object name or encoding. The error body includes the canonical request and string-to-sign that Cloud Storage computed, so compare it with what the backend signed. `ExpiredToken` means the URL is older than its `X-Goog-Expires` value; V4 URLs last at most 7 days, and a wrong clock on the signing server can make them expire early. Keep links short-lived and have the app generate a new URL when the user clicks, instead of mailing long-lived links.

KEY POINTS

- When signing with an impersonated SA (signBlob), the signer needs `roles/iam.serviceAccountTokenCreator`.
- For browser uploads, also check the bucket CORS config allows the method and headers.

CONFIGURATION EXAMPLE

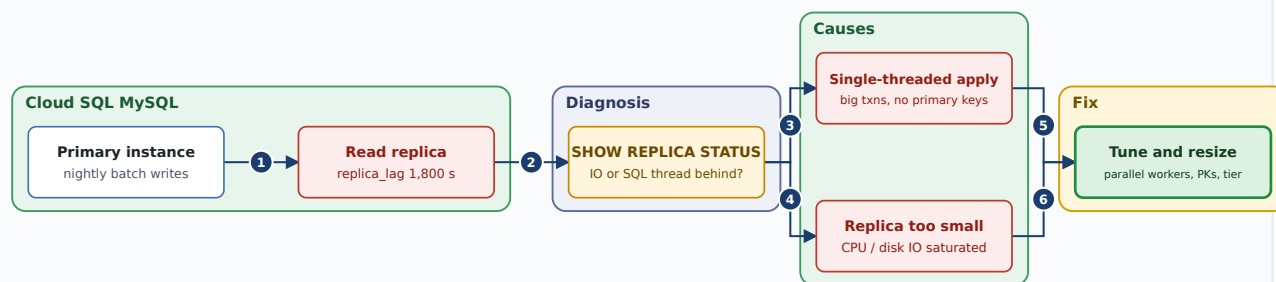
```
# sign an upload URL for PUT with a fixed Content-Type, valid 15 minutes
gcloud storage sign-url gs://uploads-prod/u/123/photo.jpg --http-verb=PUT \
  --duration=15m --headers=content-type=image/jpeg \
  --impersonate-service-account=signer@prod.iam.gserviceaccount.com
# the client must send the same header
curl -X PUT -H 'Content-Type: image/jpeg' --upload-file photo.jpg "$SIGNED_URL"
```

Q205

Troubleshooting

Reports read from a Cloud SQL for MySQL read replica show data that is 30 minutes old, and the `replica_lag` metric keeps rising during the nightly batch. How do you diagnose and fix it?

FLOW DIAGRAM



- 1 Binlog events copied and replayed
- 2 Seconds_Behind_Source and thread state
- 3 SQL thread busy replaying one huge transaction
- 4 Replica metrics at 100% CPU or IOPS
- 6 Match the replica tier and disk to the primary

EXPLANATION

Compare `replica_lag` with the replica's CPU and disk metrics, then run `SHOW REPLICA STATUS` to see whether the IO thread (fetching) or the SQL thread (applying) is behind. Most lag comes from the apply side: one large transaction or bulk `UPDATE` must be replayed in one piece, tables without primary keys make row-based replay scan whole tables, and the replica may be a smaller tier than the primary. Break the batch into smaller transactions, add primary keys, enable parallel replication flags (`replica_parallel_workers` on supported versions), and give the replica the same tier and disk size as the primary. Long reporting queries on the replica also compete for resources, so move heavy reports to a separate replica or BigQuery.

KEY POINTS

- Alert on `replica_lag` and route stale-sensitive reads to the primary.
- Disk performance grows with disk size: a small replica disk can be the bottleneck.

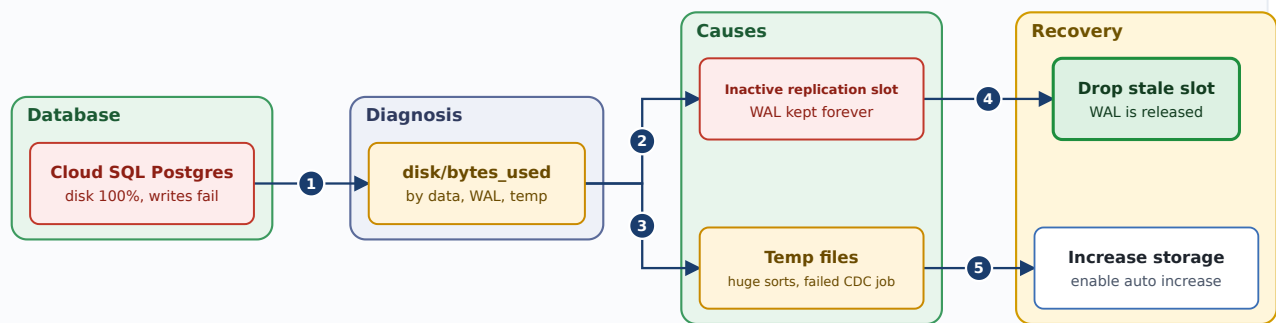
CONFIGURATION EXAMPLE

```

SHOW REPLICA STATUS\G  -- Seconds_Behind_Source, Replica_SQL_Running_State
gcloud sql instances patch reports-replica --tier=db-custom-8-32768
gcloud sql instances patch reports-replica \
  --database-flags=replica_parallel_workers=8,replica_preserve_commit_order=on
# note: --database-flags replaces the full flag list
  
```

Q206**Troubleshooting**

A Cloud SQL for PostgreSQL instance's disk went from 60% to 100% in two days; writes fail and the instance shows a storage warning, but table sizes barely changed. What is using the space and how do you recover?

FLOW DIAGRAM

- 1 Metrics Explorer: storage split by type
- 2 pg_replication_slots shows active = false
- 3 Temp bytes grow during big queries
- 4 Only after confirming no consumer needs it
- 5 Add disk now; it cannot be shrunk later

EXPLANATION

If tables did not grow, the space is usually write-ahead log (WAL) kept by a replication slot whose consumer stopped, such as a Datastream or Debezium CDC job that was paused or deleted, or temporary files from large sorts and hash joins. Check `pg_replication_slots` for inactive slots and how much WAL each holds, and look at the storage metrics in Cloud Monitoring. Drop a slot only after confirming nothing still needs it; the WAL is then released. To recover writes quickly, increase the storage size (it applies online but can never be decreased) and turn on automatic storage increase so it does not fill again.

KEY POINTS

- Alert on disk utilisation above about 80% and on inactive replication slots.
- Auto increase only grows the disk: set a limit so a runaway job cannot grow it without bound.

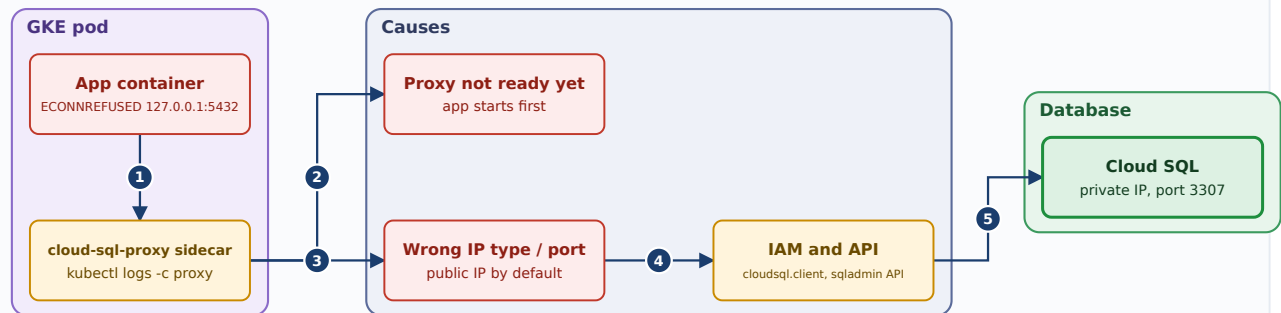
CONFIGURATION EXAMPLE

```

SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS wal_held
FROM pg_replication_slots;
SELECT pg_drop_replication_slot('old_cdc_slot');
gcloud sql instances patch orders-db --storage-size=500GB
gcloud sql instances patch orders-db --storage-auto-increase \
  --storage-auto-increase-limit=1000
  
```

Q207**Troubleshooting**

A GKE app using the Cloud SQL Auth Proxy sidecar logs `connect ECONNREFUSED 127.0.0.1:5432` at startup, and sometimes the proxy logs failed to connect to instance. The instance has only a private IP. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Nothing listens yet, or on another port
- ❷ Start proxy first: native sidecar
- ❸ Add `--private-ip`, check `--port` matches app
- ❹ Proxy logs `NOT_AUTHORIZED` or API disabled?
- ❺ Then the proxy reaches the instance

EXPLANATION

Connection refused on localhost means nothing is listening on that port yet: the app started before the proxy, the proxy crashed, or it listens on a different port than the app uses. Read the proxy container logs: failed to connect to instance with a dial timeout usually means the proxy tries the public IP (the default) for an instance that has only a private IP, so add `--private-ip` and make sure the pod runs in the same VPC. A 403 or `NOT_AUTHORIZED` means the pod's Workload Identity service account lacks `roles/cloudsql.client` or the Cloud SQL Admin API is disabled. Run the proxy as a native sidecar (an init container with `restartPolicy: Always`) so it starts before the app, and firewall rules must allow egress to the instance on port 3307.

KEY POINTS

- Enable the proxy health check and a startup probe so pods wait until it is ready.
- With private IP and no need for IAM auth, the app can also connect directly to the IP with TLS.

CONFIGURATION EXAMPLE

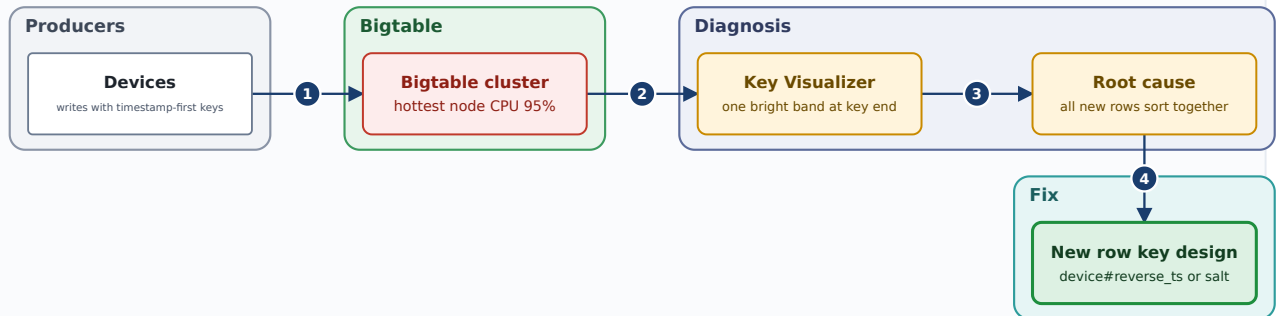
```

kubectl logs deploy/orders -c cloud-sql-proxy --tail=50
initContainers:
- name: cloud-sql-proxy
  image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.14.1
  restartPolicy: Always           # native sidecar, starts before the app
  args: ["--private-ip", "--port=5432", "--health-check",
        "--http-address=0.0.0.0", "prod:us-central1:orders-db"]

```

Q208**Troubleshooting****A Bigtable table for IoT events uses row keys like**

20260930T101500#device42 . Writes are slow, p99 latency is high, and the hottest node's CPU is at 95% while the cluster average is 30%. What is the cause and the fix?

FLOW DIAGRAM

- 1 Every write lands on the same tablet
- 2 Heat map shows a hot, moving range
- 3 Timestamp prefix makes keys increase
- 4 Spread writes: field promotion or hash prefix

EXPLANATION

Keys that start with a timestamp (or a sequential ID) always increase, so all new writes go to the last tablet on one node while other nodes sit idle; adding nodes does not help. Confirm it with the `cpu_load_hottest_node` metric and Key Visualizer, which shows a bright band at the end of the key space. Redesign the key so writes spread out: put a well-distributed field first, such as `device42#<reversed timestamp>`, or add a small hash prefix (salting) if one device is also hot. The same problem appears in Spanner with monotonically increasing primary keys; there use UUIDv4 or a bit-reversed sequence instead.

KEY POINTS

- Design keys around the main read pattern, since rows are sorted by key.
- Changing a key means writing a new table and backfilling; test with Key Visualizer before launch.

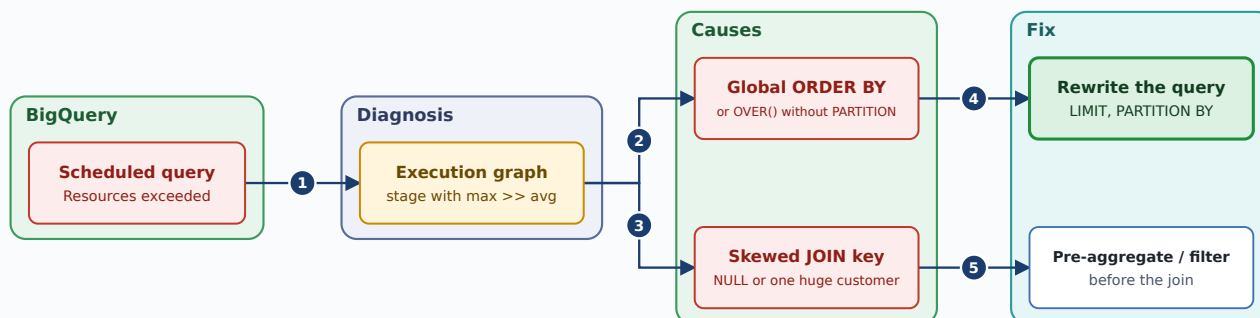
CONFIGURATION EXAMPLE

```

# Bigtable: inspect current keys
cbt -project=prod -instance=iot read events count=5
# new key: <device_id>#<Long.MAX_VALUE - epoch_ms> e.g. device42#9223370259854775807
-- Spanner equivalent: avoid increasing primary keys
CREATE SEQUENCE event_seq OPTIONS (sequence_kind = 'bit_reversed_positive');
CREATE TABLE events (
  id INT64 DEFAULT (GET_NEXT_SEQUENCE_VALUE(SEQUENCE event_seq)),
  device STRING(64), ts TIMESTAMP) PRIMARY KEY (id);
  
```

Q209**Troubleshooting**

A daily BigQuery job fails with Resources exceeded during query execution: The query could not be executed in the allotted memory. It used to run in 10 minutes. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Open the job's execution graph
- 2 One final stage runs on a single worker
- 3 One worker processes most of the rows
- 4 Avoid sorting everything in one place
- 5 Filter NULL keys, aggregate first

EXPLANATION

This error means one step needed more memory than a single worker has, which usually comes from operations that cannot be split: an `ORDER BY` over the whole result without `LIMIT`, a window function with `OVER()` and no `PARTITION BY`, `ARRAY_AGG` or `STRING_AGG` collecting huge groups, or a join where one key (often `NULL` or a single big customer) has most rows. Open the execution graph and look for a stage where the maximum compute time is far above the average, which shows skew. Fix the query: add `LIMIT` to large sorts, partition window functions, filter `NULL` keys and aggregate before joining, or break the job into steps with intermediate tables. Growth in data is the usual reason a query that worked before now fails.

KEY POINTS

- Check `INFORMATION_SCHEMA.JOBS` over time: rising `total_slot_ms` warns before failure.
- More slots do not fix a single-worker memory limit; the query shape must change.

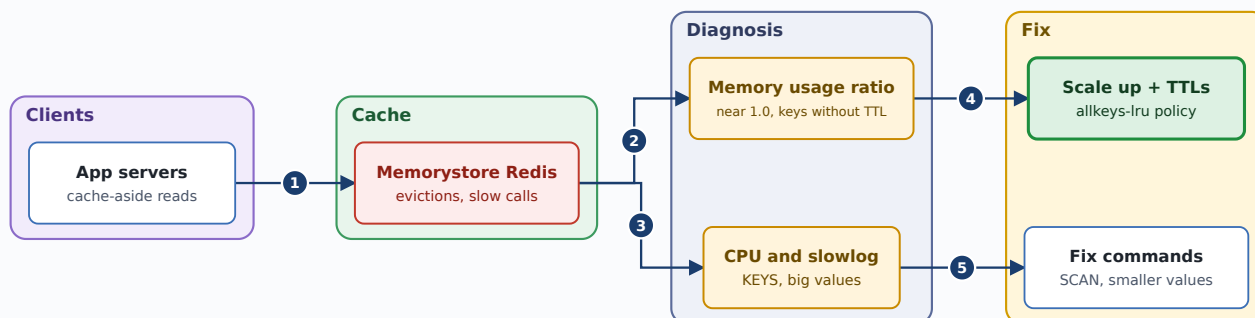
CONFIGURATION EXAMPLE

```

SELECT job_id, total_slot_ms, error_result.message
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 14 DAY)
  AND query LIKE '%daily_customer_rank%' ORDER BY creation_time;
-- before: RANK() OVER (ORDER BY revenue DESC)
-- after:  RANK() OVER (PARTITION BY region ORDER BY revenue DESC)
  
```

Q210**Troubleshooting**

After a marketing launch, a Memorystore for Redis cache shows `evicted_keys` rising, the hit ratio drops from 95% to 60%, and some calls take 200 ms. How do you diagnose and fix it?

FLOW DIAGRAM

- 1 More keys than memory allows
- 2 `usage_ratio` and `evicted_keys` metrics
- 3 `SLOWLOG GET` shows blocking commands
- 4 Right-size and expire keys
- 5 Remove `O(N)` commands from hot paths

EXPLANATION

Evictions mean the instance reached its memory limit, so Redis drops keys and the app falls back to the database, which explains the lower hit ratio and slow calls. Check `stats/memory/usage_ratio`, `evicted_keys` and system memory usage in Cloud Monitoring, and use `INFO memory` and `--bigkeys` to find what fills memory; often new keys were added without a TTL. The default policy `volatile-lru` only evicts keys that have a TTL, so keys without one fill memory; set TTLs, consider `allkeys-lru`, and scale the instance size. For latency, look at CPU and `SLOWLOG`: Redis runs commands on one thread, so `KEYS *`, large `HGETALL` or multi-megabyte values block every other client.

KEY POINTS

- Alert when memory usage ratio passes about 0.8 so you scale before evictions start.
- Use a Standard tier instance for production so a failover does not empty the cache.

CONFIGURATION EXAMPLE

```

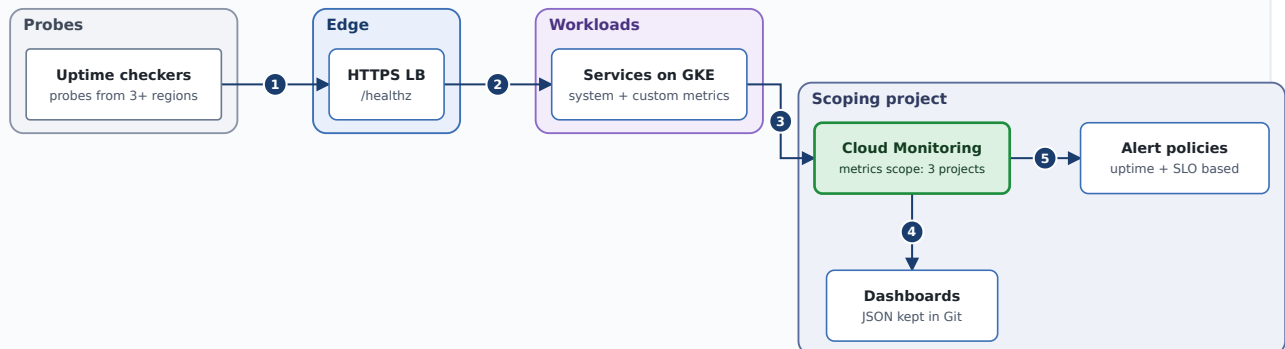
gcloud redis instances describe cache-prod --region=us-central1 \
  --format='yaml(memorySizeGb,redisConfigs,tier)'
redis-cli -h 10.0.0.3 INFO memory | grep -E 'used_memory_human|maxmemory_policy'
redis-cli -h 10.0.0.3 --bigkeys
redis-cli -h 10.0.0.3 SLOWLOG GET 10
gcloud redis instances update cache-prod --region=us-central1 --size=10
gcloud redis instances update cache-prod --region=us-central1 \
  --update-redis-config maxmemory-policy=allkeys-lru
  
```


8. Observability & SRE

Metrics, logs, traces and alerts on Google Cloud, plus the SRE practices (SLOs, error budgets, incidents, on-call, toil) that make them useful.

Q211 You are asked to set up monitoring for a new public web service running on GKE in three projects. What do you configure first in Cloud Monitoring?

FLOW DIAGRAM



- 1 HTTPS check every minute from several regions
 - 2 Real traffic and probes reach the pods
 - 3 GKE system metrics, Managed Prometheus, custom metrics
 - 4 Golden-signal dashboards per service
 - 5 Alert when checks fail from 2+ regions
- Uptime check results are stored as metrics in the scoping project, next to the service metrics

EXPLANATION

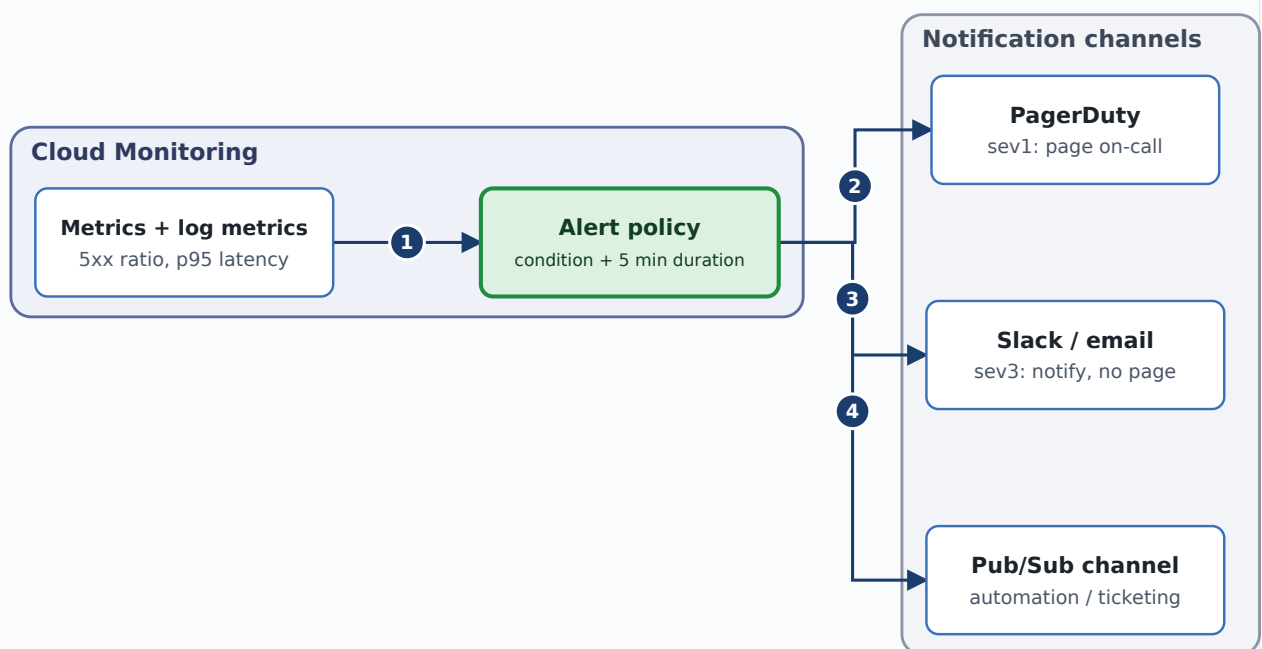
Create a scoping project and add the three projects to its metrics scope, so one place shows all metrics and dashboards. Add uptime checks against the public endpoint from several regions; they test the full path (DNS, TLS, load balancer, app) and feed an uptime alert that fires only when more than one region fails. Build dashboards per service for latency, traffic, errors and saturation, and keep their JSON in Git so they are reviewed and reproducible (Terraform `google_monitoring_dashboard`). GKE system metrics come by default; add Managed Service for Prometheus or custom metrics for app-level signals.

KEY POINTS

- Uptime checks need the checker IP ranges allowed if a firewall or Cloud Armor is in front.
- Metric scopes give visibility only; IAM is still per project.

Q212 Every alert currently goes to one shared email list and gets ignored. How do you design alert policies and notification channels so the right people act?

FLOW DIAGRAM



- ❶ Condition evaluated over a rolling window
- ❷ User-facing breach pages the on-call
- ❸ Non-urgent issues notify the owning team
- ❹ Machines react (open a ticket, run a fix)

EXPLANATION

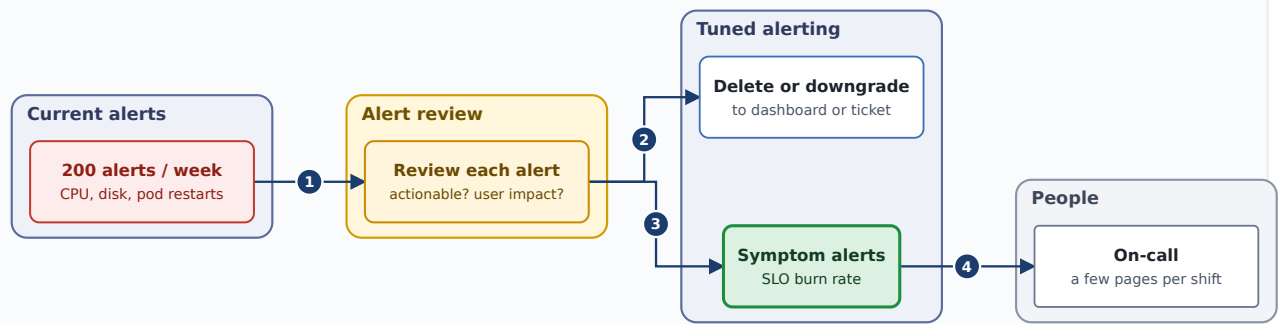
Write each policy around a clear condition with a duration window (for example 5xx ratio above 2% for 5 minutes) so short spikes do not fire, and add documentation with a runbook link that shows up in the notification. Route by severity: pages go to PagerDuty or Opsgenie for the owning team's on-call, lower severities go to a team Slack channel or ticket queue, and Pub/Sub channels drive automation. Use user labels such as `team` and `severity` on policies so routing and reporting are consistent. Manage policies and channels in Terraform so every service gets the same baseline.

KEY POINTS

- An alert with no owner or no action is noise.
- Set auto-close so resolved incidents do not stay open forever.

Q213 Your on-call engineers receive about 200 alerts a week, mostly CPU and pod restarts, and they have started to ignore pages. How do you fix alert fatigue?

FLOW DIAGRAM



- ❶ Look at the last 4 weeks of pages
- ❷ No action needed: remove or send to a ticket queue
- ❸ User impact: alert on the symptom, not the cause
- ❹ Pages are rare, urgent and actionable

EXPLANATION

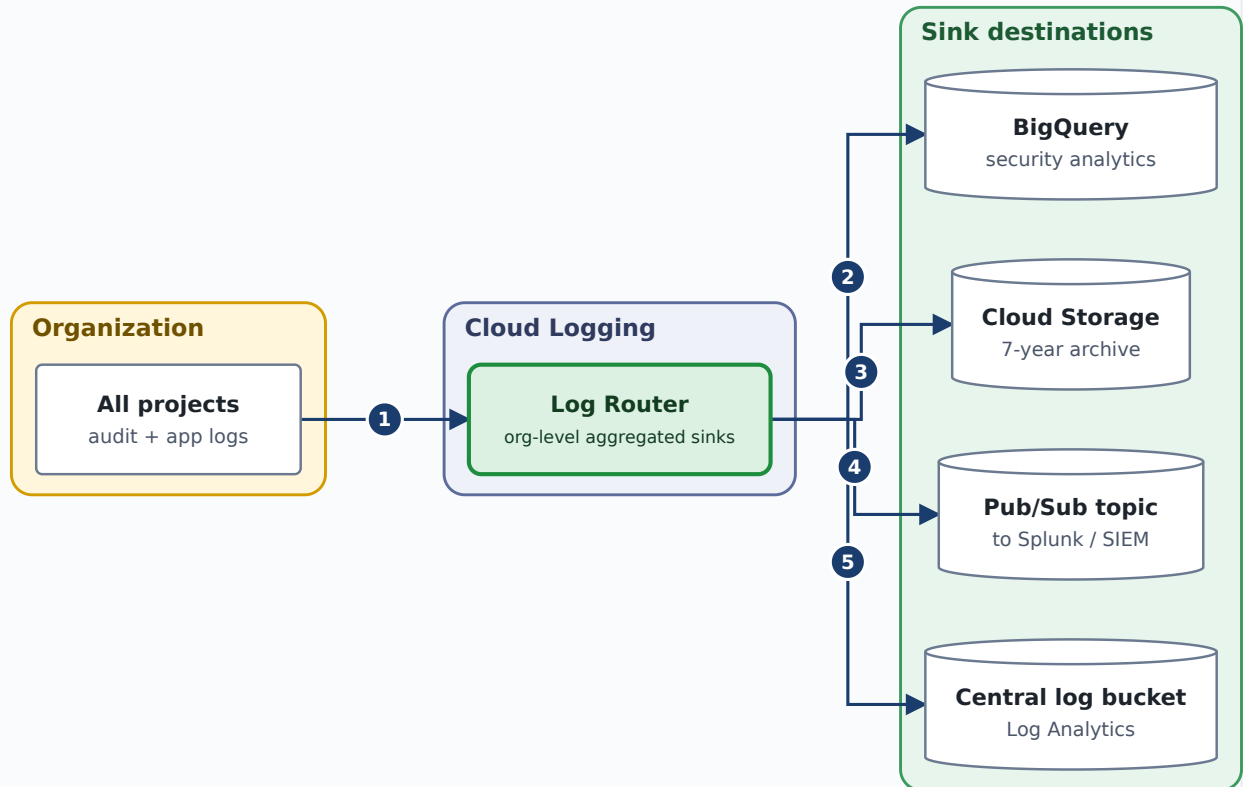
Review every alert from the last few weeks and ask two questions: did a human need to act, and were users affected. Delete or downgrade cause-based alerts (high CPU, single pod restart) to dashboards or tickets, and page only on symptoms such as SLO burn rate, availability and latency. Group related conditions into one policy, add sensible durations, and make sure each page has a runbook. Track pages per shift as a team metric and review it in the weekly on-call handoff.

KEY POINTS

- A good target is a small number of pages per shift (SRE guidance is about two incidents per 12-hour shift at most).
- Every page should be urgent, actionable and caused by real or imminent user impact.

Q214 Security wants all audit logs from every project in the organization in BigQuery for analysis, kept 7 years in cheap storage, and streamed to Splunk. How do you route them?

FLOW DIAGRAM



- 1 Every log entry passes through the router
- 2 Sink with a filter for audit logs
- 3 All audit logs, Archive-class bucket with lifecycle
- 4 Streaming export to the SIEM via Dataflow or add-on
- 5 Central bucket for Log Explorer queries

EXPLANATION

Create aggregated sinks at the organization (or folder) level with `--include-children`, each with a filter such as `logName:"cloudaudit.googleapis.com"`. Point one sink at a BigQuery dataset (use partitioned tables), one at a Cloud Storage bucket with a retention policy and lifecycle to Archive for the 7-year copy, and one at a Pub/Sub topic that Splunk reads through the Dataflow template or its add-on. Each sink has its own writer identity service account, which you must grant write access on the destination. Keep the destinations in a dedicated security project with tight IAM.

KEY POINTS

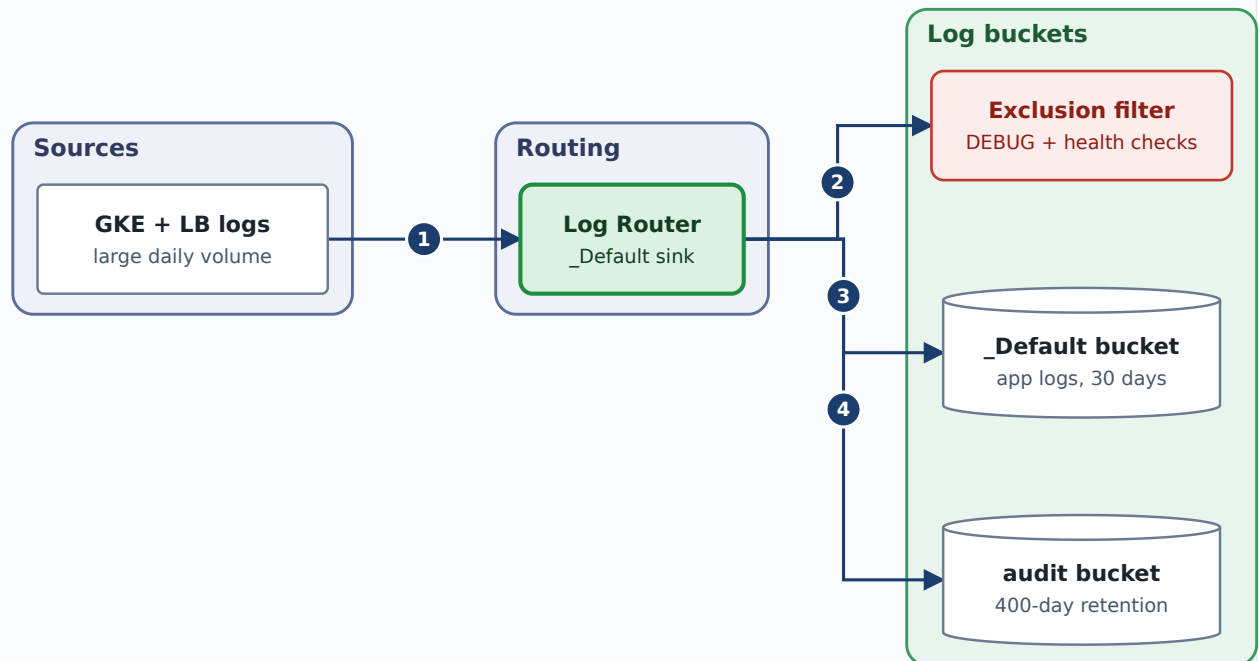
- Sinks only route new entries; they do not backfill old logs.
- Data Access audit logs are off by default for most services; enable them where needed.

CONFIGURATION EXAMPLE

```
gcloud logging sinks create audit-to-bq \
  bigquery.googleapis.com/projects/sec-logs/datasets/audit \
  --organization=123456789 --include-children --use-partitioned-tables \
  --log-filter='logName:"cloudaudit.googleapis.com"'
# grant the sink's writer identity access to the dataset
gcloud logging sinks describe audit-to-bq --organization=123456789 \
  --format='value(writerIdentity)'
```

Q215 The Cloud Logging bill has tripled because of debug logs from GKE and load balancer request logs. How do you cut cost without losing what you need?

FLOW DIAGRAM



- ❶ All entries arrive at the router
- ❷ Excluded entries are not stored or billed
- ❸ Remaining app logs, default retention
- ❹ Security logs go to a bucket with longer retention
- _Required bucket keeps Admin Activity audit logs for 400 days at no charge and cannot be changed

EXPLANATION

Find the biggest producers with the log volume metrics or Log Analytics, then add exclusion filters on the `_Default` sink for DEBUG entries, health-check requests and other noise; excluded entries are not ingested into that bucket, so they are not billed for storage. Lower the load balancer's logging sample rate on the backend service instead of logging every request. Keep default retention short on the `_Default` bucket and route logs you must keep longer into a separate log bucket with its own retention. Fix the source too: set log levels per environment in the application.

KEY POINTS

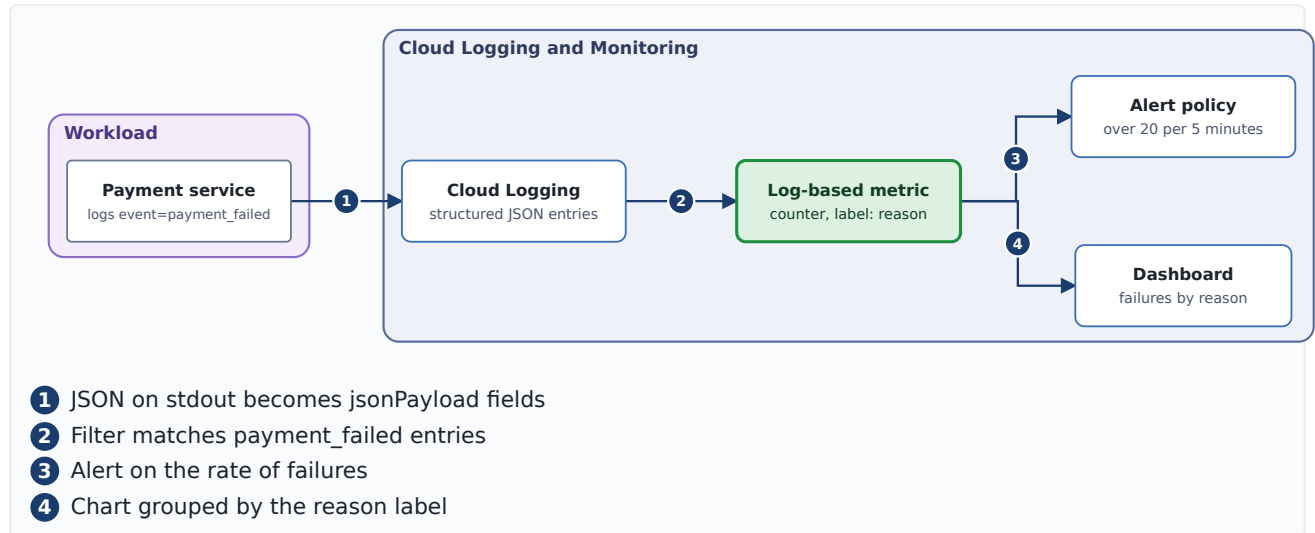
- Exclusions drop data for good; route a sample to cheap storage if you might need it.
- Retention beyond the default 30 days is billed separately.

CONFIGURATION EXAMPLE

```
gcloud logging sinks update _Default \
  --add-exclusion=name=drop-debug,filter='severity<INFO'
gcloud logging buckets create audit-bucket --location=global --retention-days=400
gcloud logging buckets update _Default --location=global --retention-days=14
gcloud compute backend-services update web-bs --global \
  --enable-logging --logging-sample-rate=0.1
```

Q216 The payment service writes a structured log line for each failed payment but has no metric for it. The team wants an alert and a chart by failure reason. What do you build?

FLOW DIAGRAM



EXPLANATION

Create a user-defined counter log-based metric with a filter on the structured field, for example `jsonPayload.event="payment_failed"`, and a label extracted from `jsonPayload.reason`. Cloud Monitoring then treats it like any other metric, so you can alert on its rate and chart it by reason. Keep label cardinality low (reasons, not user IDs), because each label value creates a time series. Log-based metrics only count entries from the time they are created and are billed like custom metrics, so check pricing for high volumes.

KEY POINTS

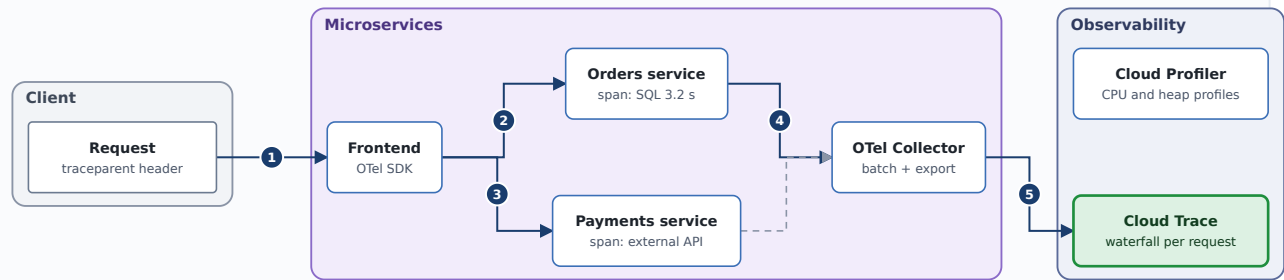
- Distribution metrics can extract numbers (like latency) from log fields.
- If the code can emit a metric directly, that is cheaper and faster than log parsing.

CONFIGURATION EXAMPLE

```
gcloud logging metrics create payment_failures \
  --description="Failed payments" \
  --log-filter='resource.type="k8s_container" AND jsonPayload.event="payment_failed"'
# labels such as reason need a config file:
gcloud logging metrics create payment_failures --config-from-file=metric.yaml
```

Q217 A checkout request passes through five microservices and sometimes takes 4 seconds, but no single service looks slow on its dashboards. How do you find the cause?

FLOW DIAGRAM



- 1 Trace context starts or is continued
- 2 W3C traceparent propagated on each call
- 4 Spans exported over OTLP
- 5 Trace shows which span took the time
- Cloud Profiler agents in each service sample CPU and heap continuously for slow code paths

EXPLANATION

Instrument the services with OpenTelemetry so every call propagates trace context and records spans, and export them through an OpenTelemetry Collector to Cloud Trace. The trace waterfall for a slow request shows where the time goes, for example a 3.2-second SQL span or a retry loop, which per-service averages hide. Sample a small fraction of normal traffic and keep all error or slow traces if your sampler supports it. When a span is slow inside the service's own code, Cloud Profiler shows which functions use CPU or allocate memory in production with low overhead.

KEY POINTS

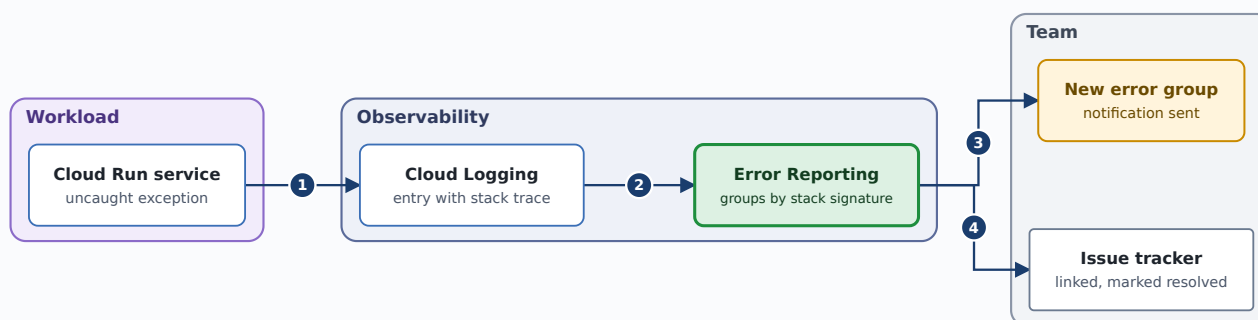
- Put the trace ID in every log line so logs and traces link in the console.
- OpenTelemetry keeps you vendor-neutral; only the exporter is Google-specific.

CONFIGURATION EXAMPLE

```
receivers:
  otlp: {protocols: {grpc: {}}}
processors:
  batch: {}
exporters:
  googlecloud: {project: prod-app}
service:
  pipelines:
    traces: {receivers: [otlp], processors: [batch], exporters: [googlecloud]}
```

Q218 A Cloud Run service throws a new exception after a release, but nobody notices until users complain. How do you use Error Reporting to catch this earlier?

FLOW DIAGRAM



- ❶ Stack trace written to stderr or as structured JSON
- ❷ Parsed and grouped automatically
- ❸ First occurrence of a group notifies the team
- ❹ Link the group to a ticket; a regression reopens it

EXPLANATION

Error Reporting reads stack traces from Cloud Logging for supported languages and groups identical errors, showing counts, first and last seen, and the service version. Make sure errors are logged with the full stack trace (or as a `ReportedErrorEvent` JSON entry with `serviceContext`) so they are picked up and tied to a version. Turn on notifications for new error groups to the team's channel, so a new exception after a release is visible within minutes. Link groups to issues and mark them resolved; if the same error comes back, Error Reporting treats it as a new occurrence and notifies again.

KEY POINTS

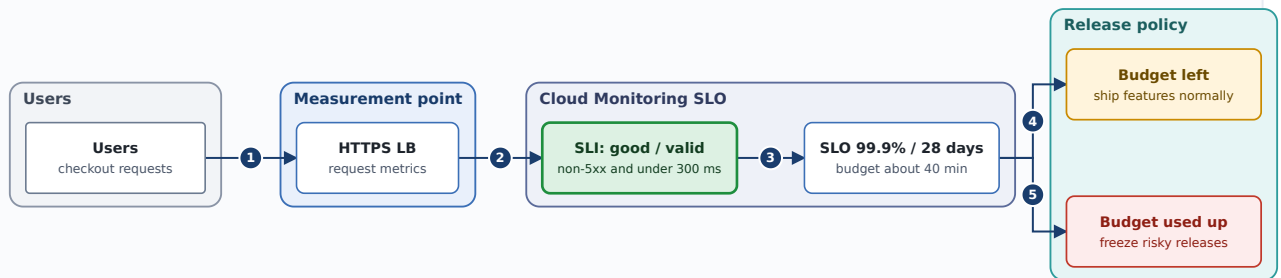
- Include service and version in `serviceContext` to see which release introduced an error.
- Pair it with a canary rollout so new errors are seen on a small share of traffic.

CONFIGURATION EXAMPLE

```
{ "severity": "ERROR",
  "@type": "type.googleapis.com/google.devtools.clouderrorreporting.v1beta1.ReportedErrorEvent",
  "message": "Traceback (most recent call last):\n File \"app.py\", line 42 ...",
  "serviceContext": { "service": "checkout", "version": "v43" }}
```


Q219 Product wants the checkout service to be "always up". How do you turn that into an SLI, an SLO and an error budget the teams can use?

FLOW DIAGRAM



- 1 Measure close to the user
- 2 Count good requests versus all valid requests
- 3 Compare to the target over a rolling window
- 4 Budget remaining: normal release pace
- 5 Budget spent: reliability work first

EXPLANATION

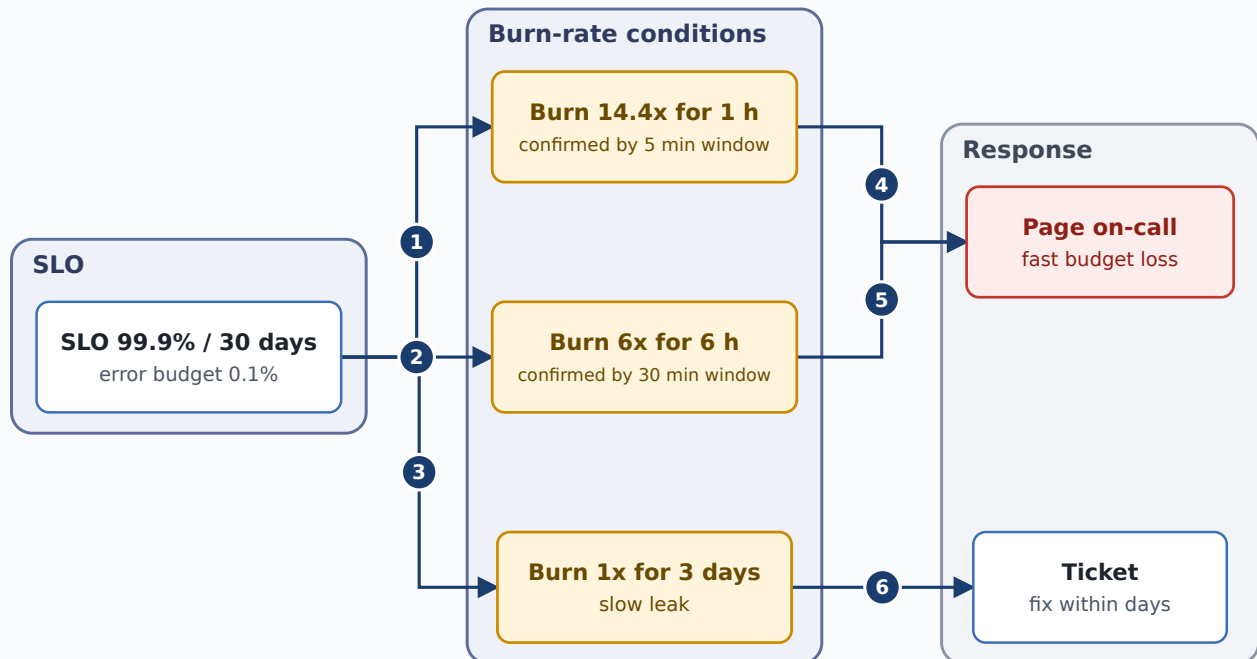
Pick SLIs that reflect what users feel, measured at the load balancer: availability (share of requests that are not 5xx) and latency (share faster than 300 ms). Agree an SLO with product, for example 99.9% over 28 days, which leaves an error budget of 0.1%, roughly 40 minutes of full outage or the equivalent in failed requests. Define it as a service and SLO in Cloud Monitoring so compliance and budget are tracked automatically. Write an error budget policy: while budget remains, teams release normally; when it is spent, they slow releases and prioritise reliability work.

KEY POINTS

- 100% is the wrong target: it blocks all change and users cannot tell the difference.
- Start with a few SLIs per user journey, not one per microservice.

Q220 A plain "error rate above 1%" alert either pages too late or pages for short blips. How do you alert on an SLO instead?

FLOW DIAGRAM



- ❶ 2% of the monthly budget gone in 1 hour
- ❷ 5% of the budget gone in 6 hours
- ❸ 10% of the budget gone in 3 days
- ❹ Slow burns become tickets, not pages

EXPLANATION

Alert on burn rate, the speed at which the error budget is used: a burn rate of 1 uses exactly the budget over the SLO window. Use multiwindow, multi-burn-rate alerts as in the Google SRE workbook: page when the burn rate exceeds 14.4 over 1 hour (2% of a 30-day budget) or 6 over 6 hours (5%), each confirmed by a short window so the alert clears quickly after recovery, and open a ticket for a burn rate of 1 over 3 days. Cloud Monitoring supports this directly with the `select_slo_burn_rate` filter on an SLO. Short blips that barely touch the budget no longer page anyone.

KEY POINTS

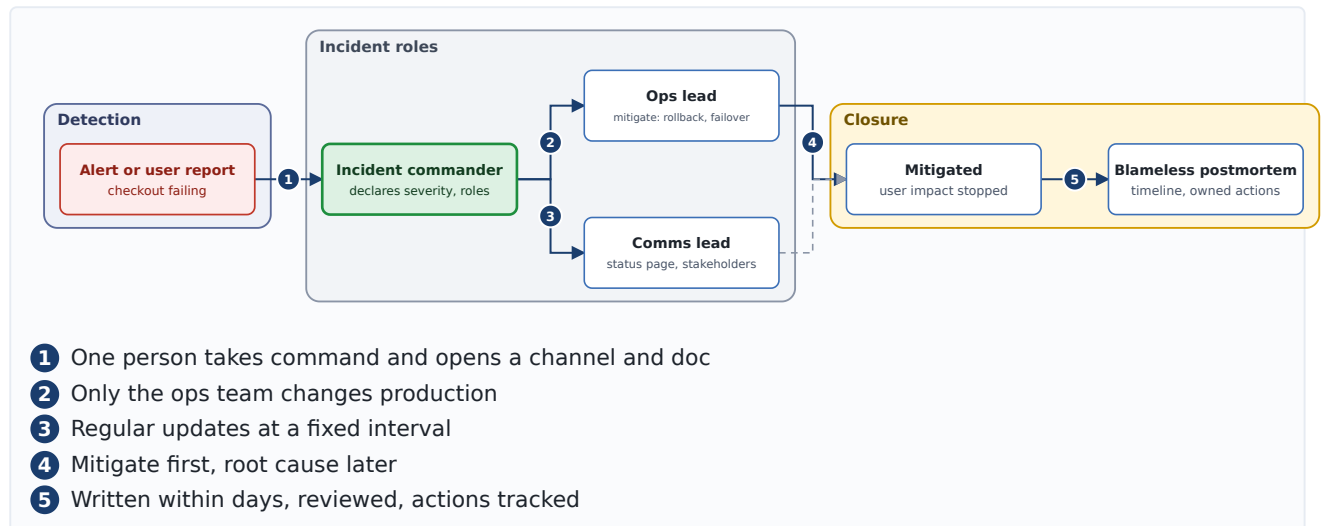
- The short confirmation window (about 1/12 of the long one) stops alerts from firing long after recovery.
- Tune thresholds to your SLO window; the numbers above assume 30 days.

CONFIGURATION EXAMPLE

```
resource "google_monitoring_alert_policy" "checkout_fast_burn" {
  display_name = "Checkout SLO fast burn"
  combiner     = "AND"
  conditions {
    display_name = "burn rate > 14.4 over 1h"
    condition_threshold {
      filter      = "select_slo_burn_rate(\"${google_monitoring_slo.checkout.name}\", \"3600s\")"
      comparison  = "COMPARISON_GT"
      threshold_value = 14.4
      duration    = "0s"
    }
  }
}
# second conditions block: same filter with "300s" (5-minute window)
```

Q221 A major outage hits checkout and five engineers are debugging in the same chat while leadership keeps asking for updates. How should the incident be run and closed?

FLOW DIAGRAM



EXPLANATION

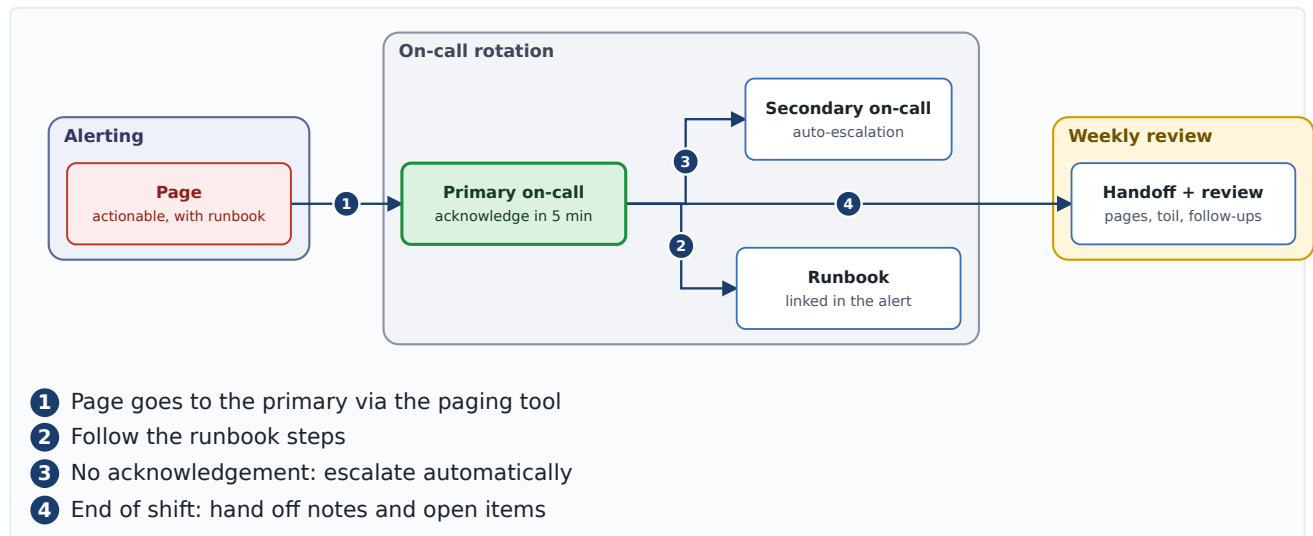
Use a clear incident structure: an incident commander who coordinates and decides, an operations lead (and helpers) who are the only ones changing production, and a communications lead who posts updates to stakeholders and the status page at a fixed interval. Keep a live incident document with a timeline of actions and observations. Focus on mitigation first (roll back, fail over, shed load) and dig into root cause after users are safe. Afterwards, write a blameless postmortem covering impact, timeline, contributing factors and action items with owners and due dates, and track those items to completion.

KEY POINTS

- Declare incidents early; it is cheap to downgrade later.
- Blameless means focusing on systems and processes, not on who made the change.

Q222 You are setting up on-call for a platform team of six. What practices keep on-call sustainable and effective?

FLOW DIAGRAM



EXPLANATION

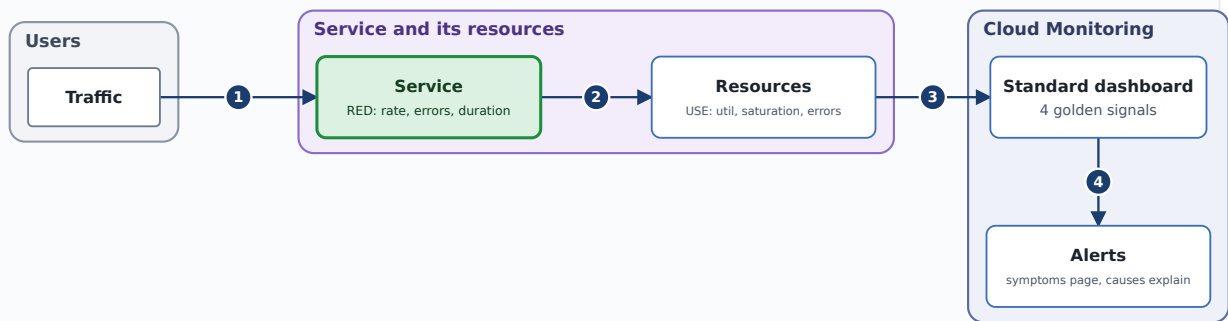
Run a rotation with a primary and a secondary, typically one week each, and with six people each engineer is on call about one week in six. Configure escalation in the paging tool so an unacknowledged page reaches the secondary and then a manager. Every alert links to a runbook, and the on-call engineer has the access and permissions needed to act (break-glass if required). Hold a short handoff each week to review pages, false alarms and toil, and feed fixes into the backlog; compensate on-call time and protect engineers from project work while they carry a heavy pager.

KEY POINTS

- Follow-the-sun rotations across time zones avoid night pages.
- Track pages per shift and time to acknowledge as team health metrics.

Q223 Each team builds dashboards differently and incidents take long to triage. What standard set of signals do you ask every service to expose?

FLOW DIAGRAM



- ❶ Requests per second arrive
- ❷ Requests use CPU, memory, connections
- ❸ RED per endpoint plus USE (CPU, queue depth, pool usage)
- ❹ Page on symptoms, use causes for diagnosis

EXPLANATION

Ask every service to expose the four golden signals: latency, traffic, errors and saturation. For request-driven services the RED method (rate, errors, duration per endpoint) covers the first three; for resources such as nodes, disks, connection pools and queues the USE method (utilisation, saturation, errors) explains why the service is slow. Put them on a standard dashboard template so any engineer can read any service. Page on user-facing symptoms (errors, latency) and use resource signals to diagnose the cause.

KEY POINTS

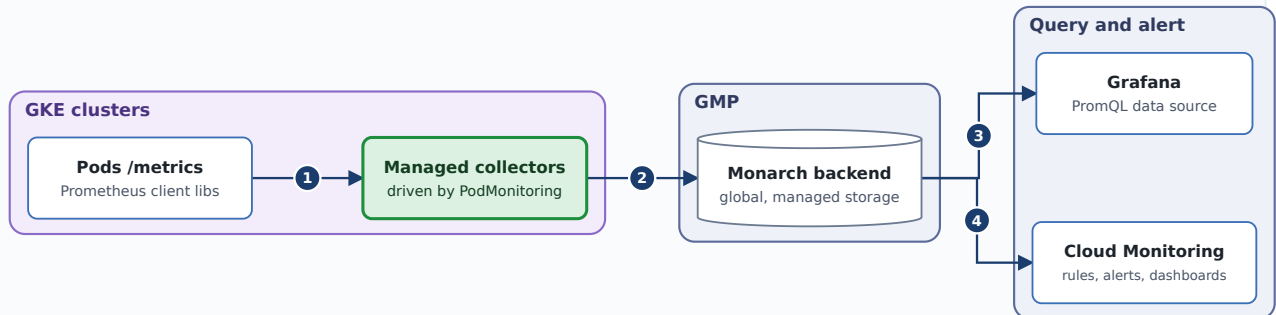
- Track latency of errors separately; fast failures can hide in averages.
- Use percentiles (p50, p95, p99), not averages, for latency.

CONFIGURATION EXAMPLE

```
# Rate
sum(rate(http_requests_total{job="checkout"}[5m]))
# Errors (ratio)
sum(rate(http_requests_total{job="checkout",code=~"5.."}[5m]))
/ sum(rate(http_requests_total{job="checkout"}[5m]))
# Duration p95
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="checkout"}[5m])))
```

Q224 Your team runs self-managed Prometheus on each GKE cluster and struggles with storage, HA and global queries. How would Google Cloud Managed Service for Prometheus help?

FLOW DIAGRAM



- 1 Collectors on each node scrape the pods
- 2 Samples written to managed global storage
- 3 Grafana queries all clusters with PromQL
- 4 Rules and alerts evaluated centrally

EXPLANATION

Managed Service for Prometheus keeps the Prometheus data model and PromQL but replaces the storage and HA work with Google's Monarch backend. On GKE you enable managed collection and describe what to scrape with `PodMonitoring` or `ClusterPodMonitoring` resources; collectors run on each node, and data from all clusters and projects can be queried together. Existing Grafana dashboards keep working by pointing the Prometheus data source at the GMP query frontend or the Cloud Monitoring PromQL API, and Cloud Monitoring can run the alerting and recording rules. Pricing is per sample ingested, so control cardinality and scrape intervals.

KEY POINTS

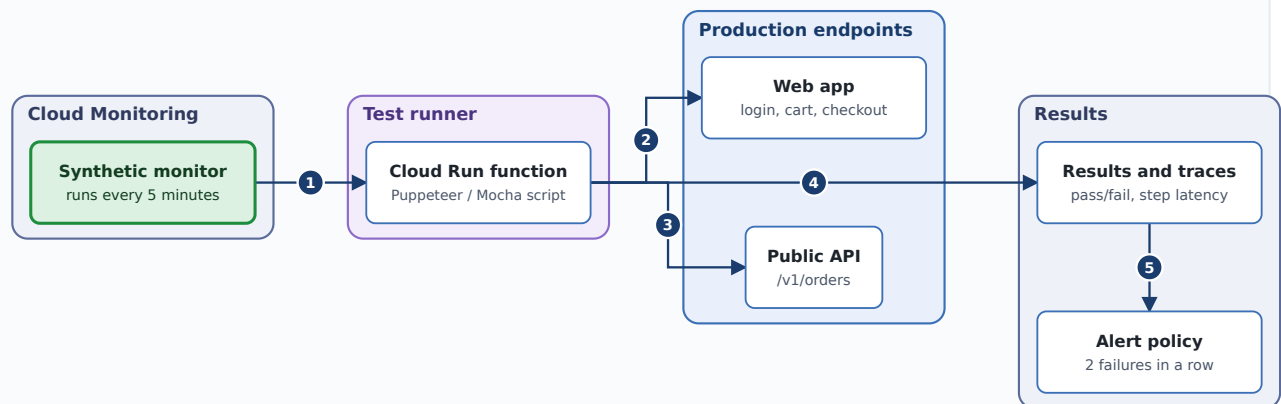
- Self-deployed collection is an option if you want to keep the Prometheus binary and configs.
- Drop high-cardinality labels at the source to keep cost down.

CONFIGURATION EXAMPLE

```
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: checkout
spec:
  selector:
    matchLabels: {app: checkout}
  endpoints:
    - port: metrics
      interval: 30s
```

Q225 Uptime checks are green, yet users sometimes cannot finish checkout because a login step fails. How do you monitor the whole user journey?

FLOW DIAGRAM



- ❶ Monitor triggers the test function on schedule
- ❷ Scripted browser journey with a test account
- ❸ API checks with assertions on the response
- ❹ Each run records success, duration and logs
- ❺ Alert when the journey fails repeatedly

EXPLANATION

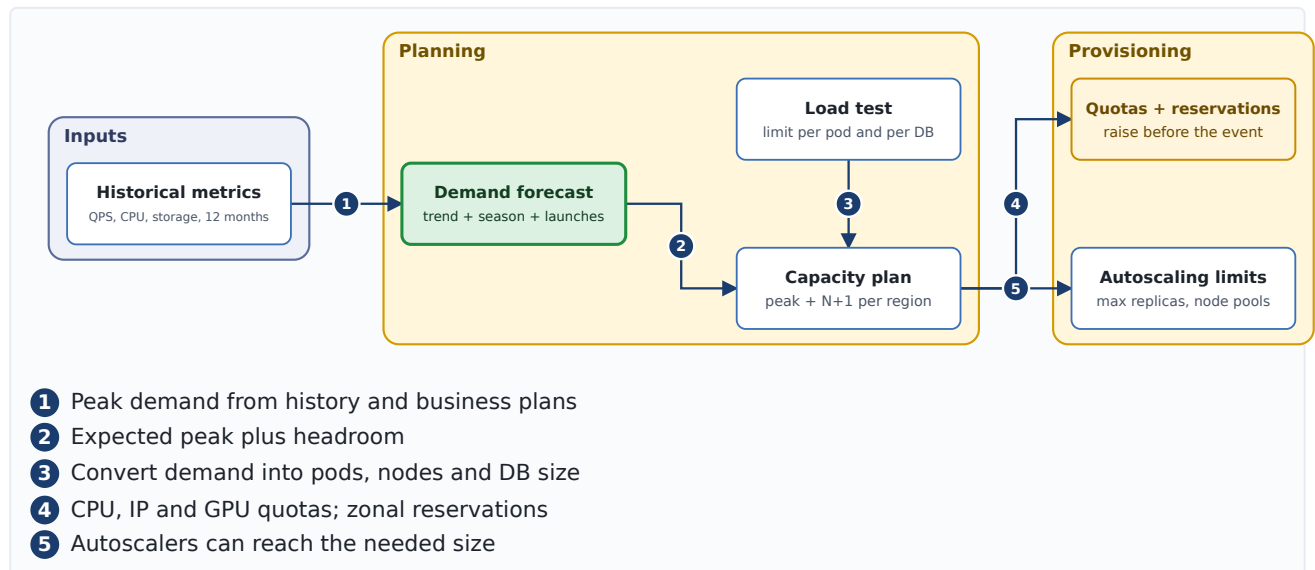
Use Cloud Monitoring synthetic monitors, which run a script you write (Node.js with Mocha or Puppeteer) in a Cloud Run function on a schedule. The script logs in with a dedicated test account, adds an item to the cart and runs checkout up to a safe step, asserting on each response. Results are stored as metrics with logs and traces, so you can alert when the journey fails twice in a row and see which step broke. Keep test data isolated (flagged test accounts, no real payments) and run from more than one region if users are global.

KEY POINTS

- Synthetic checks catch problems when real traffic is low, such as at night.
- Broken link checkers are also available as a synthetic monitor template.

Q226 Black Friday traffic is expected to be 5 times normal. How do you plan capacity on Google Cloud so the platform holds up?

FLOW DIAGRAM



EXPLANATION

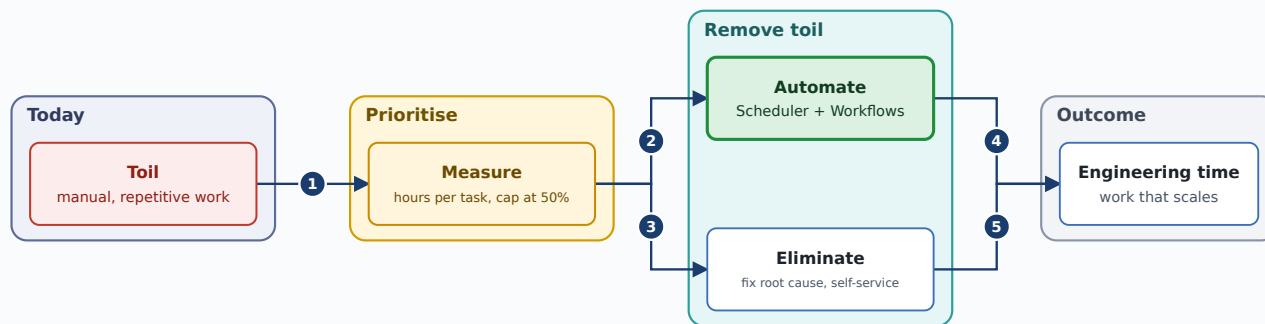
Start from data: last year's peak, current growth and marketing plans give a forecast of peak requests per second. Load test each tier to find how much one pod, node or database instance can handle, and turn the forecast into capacity with headroom for losing a zone or region (N+1). Check and raise quotas (regional CPUs, IP addresses, persistent disk, GPUs) weeks ahead, and use reservations for VMs you must be sure to get. Raise autoscaler maximums, pre-warm where scale-up is slow (minimum instances, node pool sizes), and check that backends such as Cloud SQL and third-party APIs can take the load.

KEY POINTS

- Quota is not capacity: reservations guarantee zonal resources, quotas only allow them.
- Rehearse the peak with a load test and a game day.

Q227 Your SRE team spends most of its week on manual tasks: rotating certificates, cleaning old disks, restarting stuck jobs. How do you reduce toil?

FLOW DIAGRAM



- 1 Track toil in tickets and on-call reviews
- 2 Automate tasks that must still happen
- 3 Remove the need for the task entirely
- 4 Freed time goes to projects

EXPLANATION

Define toil as work that is manual, repetitive, automatable, reactive and adds no lasting value, and measure it: log time per task type for a few weeks. Keep toil under a cap (Google SRE uses 50% of time) and pick the tasks with the highest cost first. Eliminate where possible (Certificate Manager for certificate renewal, lifecycle rules for old data, fixes for jobs that get stuck), and automate the rest with Cloud Scheduler triggering Workflows, Cloud Run jobs or functions. Review toil in each on-call handoff so new sources are caught early.

KEY POINTS

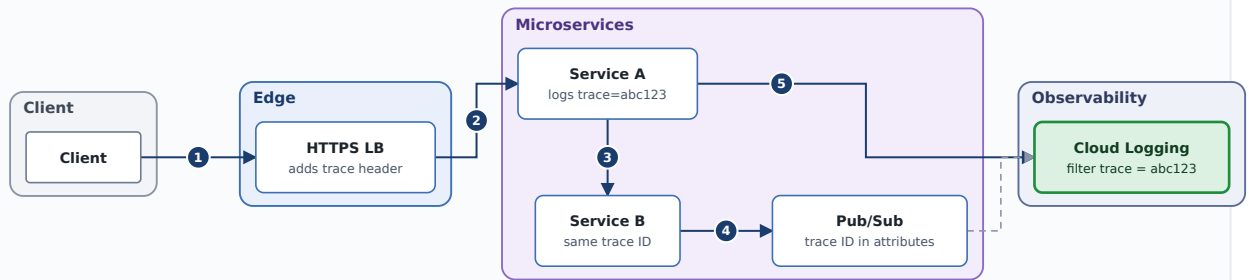
- Automation needs owners, alerts and tests like any other service.
- Self-service tools remove requests that land on the team.

CONFIGURATION EXAMPLE

```
gcloud scheduler jobs create http cleanup-old-disks \
  --location=us-central1 --schedule="0 3 * * *" \
  --uri=https://workflowexecutions.googleapis.com/v1/projects/ops/locations/us-
central1/workflows/cleanup/executions \
  --http-method=POST \
  --oauth-service-account-email=ops-automation@ops.iam.gserviceaccount.com
```

Q228 Debugging a failed order means searching logs in six services and a Pub/Sub consumer by timestamp. How do you make one request traceable across all of them?

FLOW DIAGRAM



- 1 Request gets a trace context if it has none
- 2 Header passed to the first service
- 3 Services forward traceparent on every call
- 4 Publisher copies the trace ID into message attributes
- 5 Each log entry carries the trace field
- Consumer logs with the same trace ID

EXPLANATION

Use one correlation ID per request and propagate it everywhere: the W3C `traceparent` header (the Google load balancer adds a trace header if none is present) for HTTP and gRPC, and a message attribute for Pub/Sub so async consumers keep it. Write structured JSON logs that set the `logging.googleapis.com/trace` field to `projects/PROJECT_ID/traces/TRACE_ID`, plus `spanId`; Cloud Logging then groups the entries and links them to Cloud Trace. OpenTelemetry libraries handle the propagation and can inject the IDs into log records automatically. One filter on the trace ID then shows the whole request across services.

KEY POINTS

- Also return the ID to the client (e.g. in an error page) so support can quote it.
- Add business IDs like `order_id` as log fields, not as the correlation ID.

CONFIGURATION EXAMPLE

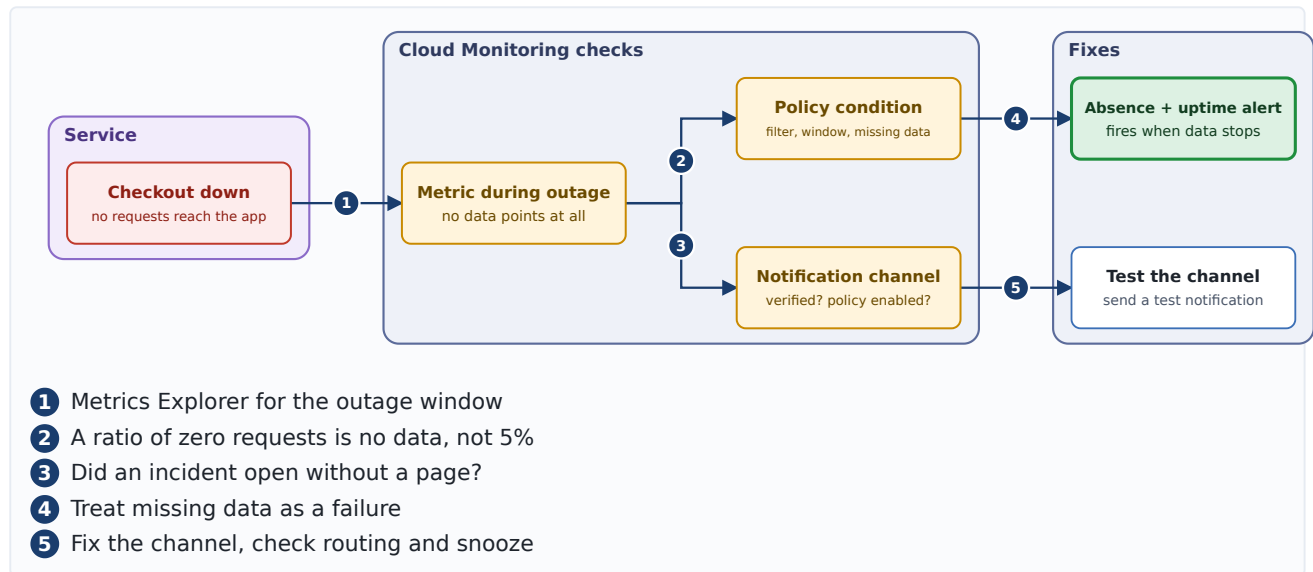
```
{
  "severity": "INFO",
  "message": "order created",
  "logging.googleapis.com/trace": "projects/prod-app/traces/4bf92f3577b34da6a3ce929d0e0e4736",
  "logging.googleapis.com/spanId": "00f067aa0ba902b7",
  "order_id": "A-1009"
}
```

Q229

Troubleshooting

The checkout service was down for 25 minutes, but the alert "error rate > 5%" never fired and nobody was paged. How do you find out why and fix the alerting?

FLOW DIAGRAM



EXPLANATION

Open the metric in Metrics Explorer for the outage window: when a service is fully down it often sends no requests at all, so an error-rate condition has no data and never crosses the threshold. Next check the policy itself: a filter on old resource labels (for example after moving from VMs to GKE), a long duration or alignment window, or a disabled policy or active snooze. If an incident was opened but nobody was paged, the notification channel is unverified, deleted or points to an old address. Fix it by adding an uptime check or a metric-absence condition, setting `evaluationMissingData` to treat missing data as failing, and testing channels regularly.

KEY POINTS

- Alert on symptoms users see (uptime, SLO burn rate), not only on internal metrics.
- Run a game day: stop a test service and confirm the page arrives.

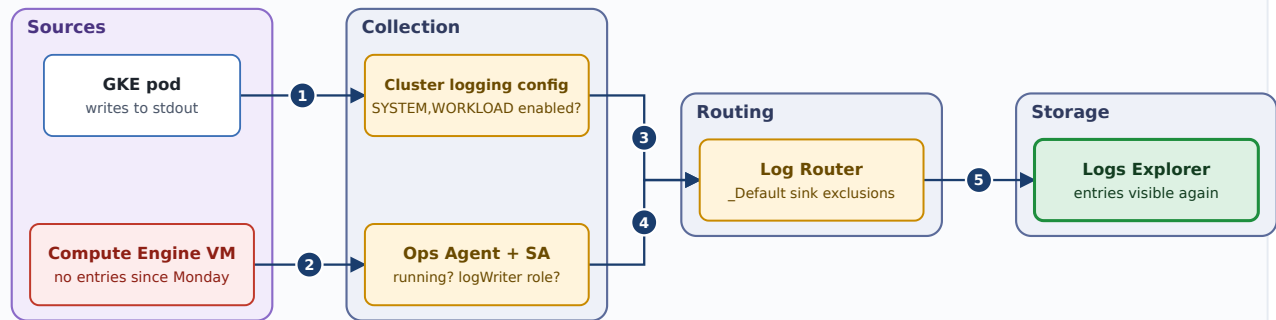
CONFIGURATION EXAMPLE

```

gcloud alpha monitoring policies list \
  --format='table(displayName,enabled,conditions[0].displayName,notificationChannels)'
gcloud alpha monitoring channels list --format='table(displayName,type,verificationStatus,enabled)'
# policy condition fragment (YAML)
conditionThreshold:
  evaluationMissingData: EVALUATION_MISSING_DATA_ACTIVE
conditionAbsent:
  duration: 300s
  
```

Q230**Troubleshooting**

Application logs from a GKE workload (and from one Compute Engine VM) no longer appear in Logs Explorer, although `kubectl logs` shows them. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Workload logging may be turned off
- 2 Agent stopped, or SA cannot write logs
- 3 Check exclusion filters on `_Default`
- 5 Remove the exclusion, logs flow again

EXPLANATION

First check whether the logs are collected: on GKE, the cluster's logging config must include `WORKLOAD` (someone may have set it to system logs only), and the node service account needs `roles/logging.logWriter`. On a VM, check that the Ops Agent is installed and running and that the VM's service account has the log writer role and a scope that allows logging; a VM without external IP also needs Private Google Access. If collection is fine, look at the Log Router: an exclusion filter on the `_Default` sink added to cut costs may be dropping these logs, and excluded logs are not stored. Also check that the application writes to `stdout` or `stderr`, since files inside the container are not collected.

KEY POINTS

- Review exclusion filters in code review; a broad filter silently drops logs.
- Alert on a log-based metric that should never be zero, such as request logs per minute.

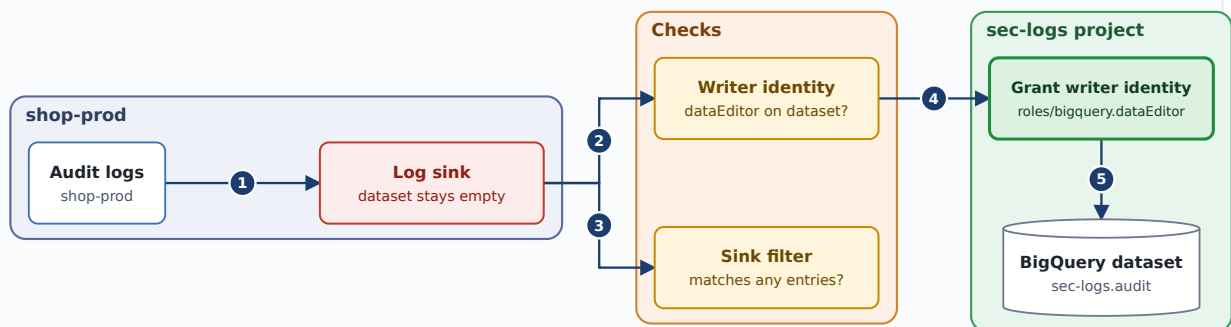
CONFIGURATION EXAMPLE

```

gcloud container clusters describe shop-gke --region=us-central1 \
  --format='value(loggingConfig.componentConfig.enableComponents)'
gcloud container clusters update shop-gke --region=us-central1 --logging=SYSTEM,WORKLOAD
gcloud logging sinks describe _Default --format='yaml(filter,exclusions)'
sudo systemctl status 'google-cloud-ops-agent*'
sudo journalctl -u google-cloud-ops-agent-fluent-bit --since '1 hour ago' | tail -20
  
```

Q231**Troubleshooting**

A new log sink should export audit logs from project `shop-prod` to a BigQuery dataset in project `sec-logs`, but the dataset stays empty after an hour. How do you fix it?

FLOW DIAGRAM

- 1 Sink routes only new matching entries
- 2 Writes use the sink's own service account
- 3 Test the filter in Logs Explorer
- 4 Grant the role in the destination project
- 5 Tables appear within minutes

EXPLANATION

A sink writes with its own writer identity, a Google-managed service account shown in `gcloud logging sinks describe`; creating a sink to another project does not grant it anything, so it needs `roles/bigquery.dataEditor` on the dataset or project `sec-logs`. Cloud Logging reports failed writes as sink errors, visible in the Log Router and through notifications to project contacts. Also test the sink's filter in Logs Explorer: a typo in `logName` or a filter for Data Access logs that are not enabled matches nothing. Remember that a sink only exports entries that arrive after it is created, and VPC Service Controls or org policies around the destination project can also block the write.

KEY POINTS

- Use partitioned tables in the sink (`--use-partitioned-tables`) to keep query costs low.
- For many projects, create one aggregated sink at the folder or organization level.

CONFIGURATION EXAMPLE

```

gcloud logging sinks describe audit-to-bq --project=shop-prod --format='value(writerIdentity)'
# e.g. serviceAccount:service-123456789@gcp-sa-logging.iam.gserviceaccount.com
bq query --project_id=sec-logs --use_legacy_sql=false \
  'GRANT `roles/bigquery.dataEditor` ON SCHEMA audit
  TO "serviceAccount:service-123456789@gcp-sa-logging.iam.gserviceaccount.com"'
gcloud logging read 'logName:"cloudaudit.googleapis.com%2Factivity"' \
  --project=shop-prod --limit=3 --freshness=1h

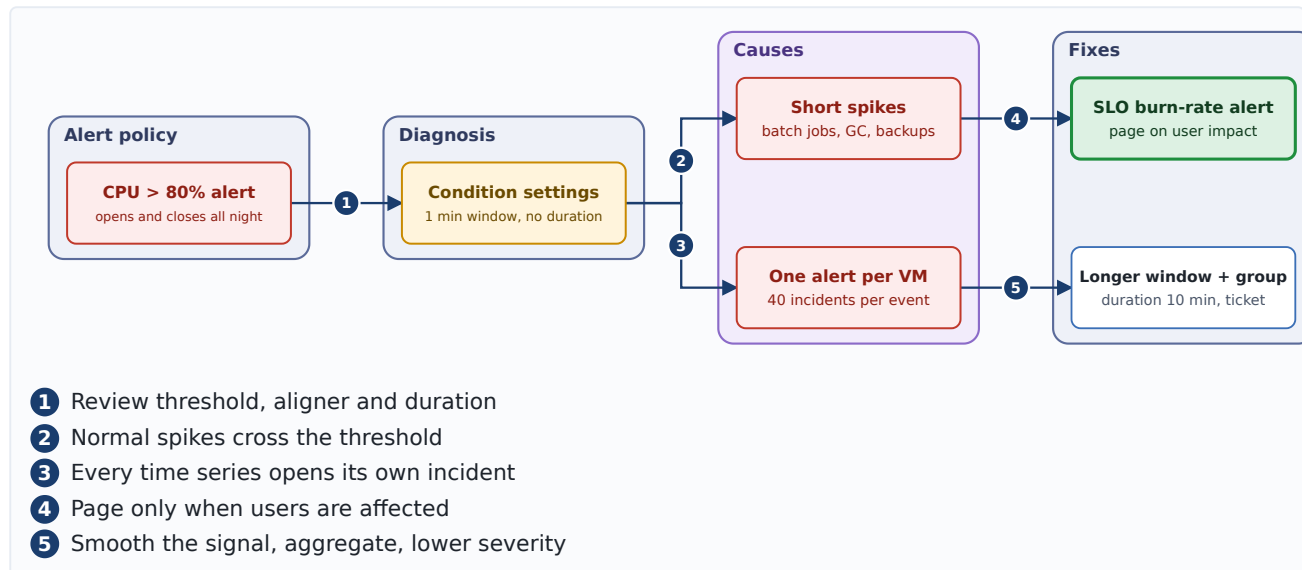
```

Q232

Troubleshooting

On-call gets 40 pages a night from a "CPU > 80%" alert that opens and closes every few minutes, and the team has started ignoring pages. How do you fix the noisy alerting?

FLOW DIAGRAM



EXPLANATION

Flapping comes from a condition that reacts to single data points: a one-minute alignment window with no duration means any short spike from a batch job or garbage collection opens an incident, and it closes a minute later. Use a longer alignment period and a duration (for example mean over 5 minutes, true for 10 minutes), and aggregate across the instance group so one event gives one incident, not one per VM. More importantly, CPU is a cause, not a symptom: page on SLO burn-rate alerts for latency and errors, and send resource alerts to a ticket or chat channel instead of the pager. Snooze alerts during planned maintenance and review the noisiest policies each week.

KEY POINTS

- Every page should need a human action; if not, delete it or downgrade it.
- Use multi-window burn-rate alerts (fast and slow) for SLOs.

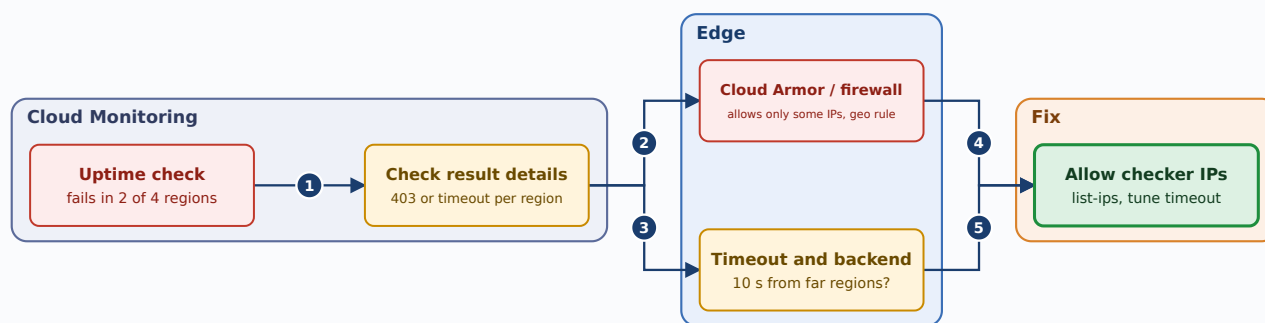
CONFIGURATION EXAMPLE

```

conditionThreshold:
  filter: metric.type="compute.googleapis.com/instance/cpu/utilization" AND resource.type="gce_instance"
  aggregations:
    - alignmentPeriod: 300s
      perSeriesAligner: ALIGN_MEAN
      crossSeriesReducer: REDUCE_MEAN
      groupByFields: [metadata.user_labels."app"]
  comparison: COMPARISON_GT
  thresholdValue: 0.8
  duration: 600s
  
```

Q233**Troubleshooting**

An uptime check for `https://api.example.com/healthz` passes from USA and Europe but fails from Asia Pacific and South America with timeouts or 403. Real users say the site works. How do you troubleshoot it?

FLOW DIAGRAM

- ❶ Failures grouped by checker region
- ❷ 403 = request blocked at the edge
- ❸ Timeout = slow or unreachable path
- ❹ Allow the uptime check source ranges
- ❺ Raise the timeout, check regional routing

EXPLANATION

Uptime checks run from fixed checker locations, so look at the failures by region: a 403 from only some regions usually means a Cloud Armor policy or firewall allows a list of IPs that misses those checkers, or a geo-blocking rule denies traffic from those countries. Get the current source ranges with `gcloud monitoring uptime list-ips` and allow them, or add a rule that allows the checker traffic to the health path. Timeouts from far regions point to a timeout set too low, DNS or geo-routing that sends those regions to an unhealthy backend, or a regional service reached across the world. Alert only when at least two regions fail, so one checker location's network issue does not page.

KEY POINTS

- Uptime check IPs change over time: re-read list-ips instead of hard-coding them once.
- For private services, use private uptime checks through Service Directory.

CONFIGURATION EXAMPLE

```

gcloud monitoring uptime list-ips | grep -E 'ASIA_PACIFIC|SOUTH_AMERICA' | head
gcloud compute security-policies describe edge-policy --format='yaml(rules)'
# RANGES = comma-separated CIDRs taken from list-ips (max 10 per rule)
gcloud compute security-policies rules create 900 --security-policy=edge-policy \
  --src-ip-ranges="$RANGES" --action=allow --description='uptime checkers'

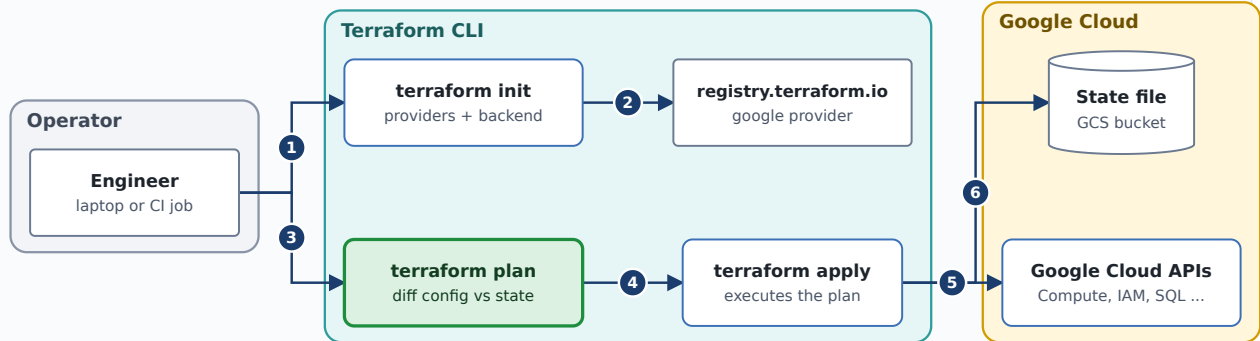
```

9. Terraform & Infrastructure as Code

How senior engineers run Terraform on Google Cloud: state, modules, CI pipelines, policy, refactoring and safe recovery.

Q234 A new engineer asks what actually happens when they run `terraform init`, `plan` and `apply` against a GCP project. Walk them through it.

FLOW DIAGRAM



- 1 `init` downloads providers and configures the backend
- 2 Provider versions resolved and written to `.terraform.lock.hcl`
- 3 `plan` refreshes real resources and compares with config
- 4 Saved plan file is applied without re-diffing
- 5 Provider calls the GCP APIs to create/update/delete
- 6 New state written with resource IDs and attributes

EXPLANATION

`terraform init` downloads the providers pinned in `.terraform.lock.hcl`, initialises modules and configures the backend where state lives. `terraform plan` reads the state, refreshes what exists in GCP, and prints the create/update/destroy actions needed to make reality match the code. `terraform apply` executes those actions through the provider, which calls the Google APIs, and then writes the new state. In CI you save the plan with `-out` so that apply runs exactly what was reviewed.

KEY POINTS

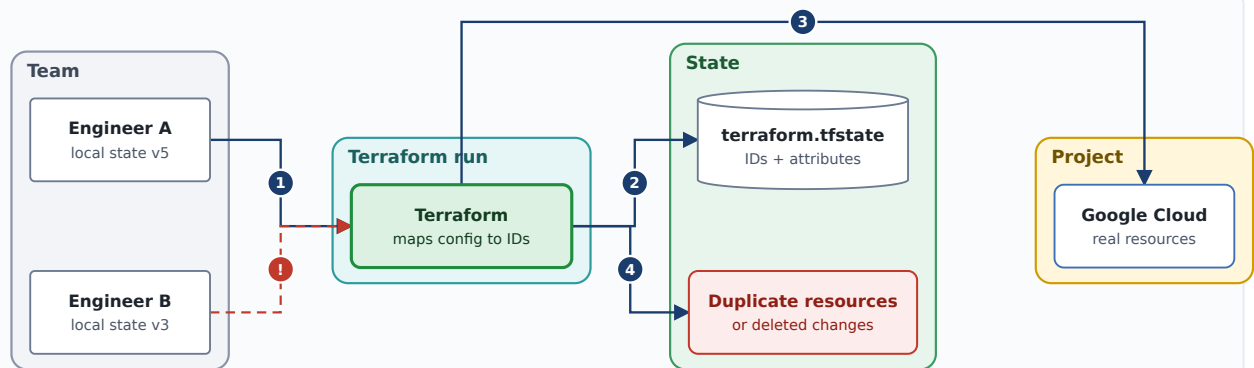
- Plan is a proposal; apply of a saved plan runs exactly that proposal.
- Commit `.terraform.lock.hcl` so every run uses the same provider build.

CONFIGURATION EXAMPLE

```
terraform init -input=false
terraform plan -input=false -out=tfplan
terraform show -no-color tfplan > plan.txt
terraform apply -input=false tfplan
```


Q235 Why does Terraform need a state file at all, and what goes wrong if two engineers keep their own local copies?

FLOW DIAGRAM



- 1 Apply with state v5
- 2 State records `google_compute_instance.web = id 123`
- 3 Only the diff is sent to the API
- ! Stale v3 state: resources look missing
- 4 B's apply tries to recreate or revert changes

EXPLANATION

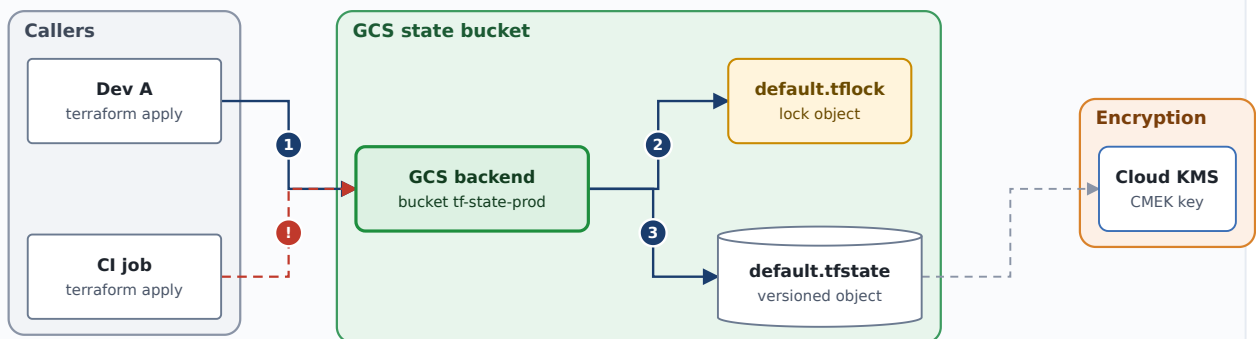
State is Terraform's map from each resource address in your code to the real object ID in GCP, plus cached attributes and dependency data. Without it Terraform cannot tell whether `google_compute_instance.web` already exists, so it would try to create it again. With separate local copies each engineer has a different view, and applies either fail with 409 already exists or silently revert the other person's changes. The fix is one shared remote backend with locking, and never editing state by hand.

KEY POINTS

- State can contain secrets in plain text: restrict access to it.
- One shared remote state per root module, never local copies.

Q236 Set up remote state for a team on GCP. How do you store it in GCS and stop two applies from running at the same time?

FLOW DIAGRAM



- 1 First apply takes the lock
- 2 Lock object created atomically
- ! Error acquiring the state lock
- 3 State read and written; object versioning keeps history
- Bucket encrypted with a CMEK key

EXPLANATION

Create a dedicated GCS bucket in a separate admin project with uniform bucket-level access, object versioning and public access prevention, and configure it as the `gcs` backend. The `gcs` backend supports locking natively by creating a lock file next to the state, so a second apply fails with a lock error instead of corrupting state. Versioning lets you restore an earlier state object if something goes wrong. Only the Terraform service accounts get `roles/storage.objectAdmin` on the bucket.

KEY POINTS

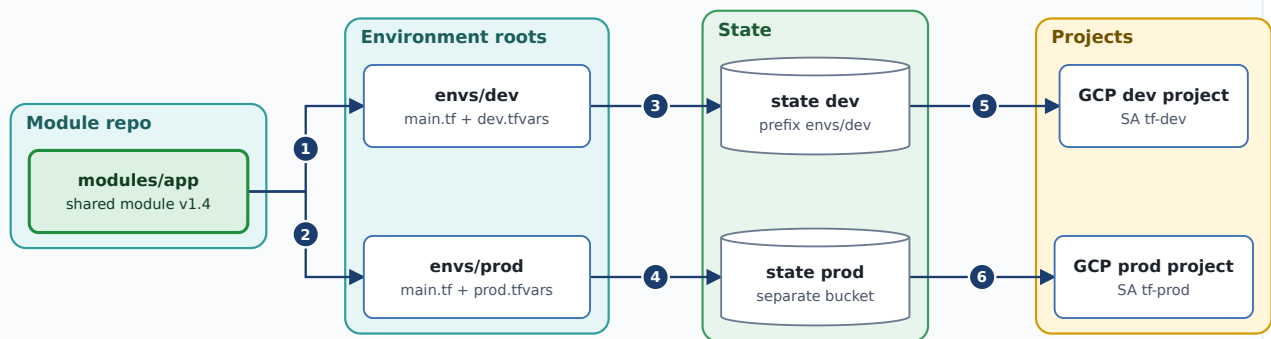
- No extra lock table is needed on GCS, unlike S3 with DynamoDB in older setups.
- Use `terraform force-unlock` only after confirming no run is active.

CONFIGURATION EXAMPLE

```
terraform {
  backend "gcs" {
    bucket = "acme-tf-state-prod"
    prefix = "network/prod"
  }
}
# gcloud storage buckets update gs://acme-tf-state-prod --versioning
```

Q237 You manage dev, stage and prod with Terraform. Would you use workspaces or separate directories, and why?

FLOW DIAGRAM



- 1 Dev pins module ref v1.5 to test first
- 2 Prod pins v1.4 until promoted
- 3 Each root has its own backend
- 5 Different SA and permissions per env

EXPLANATION

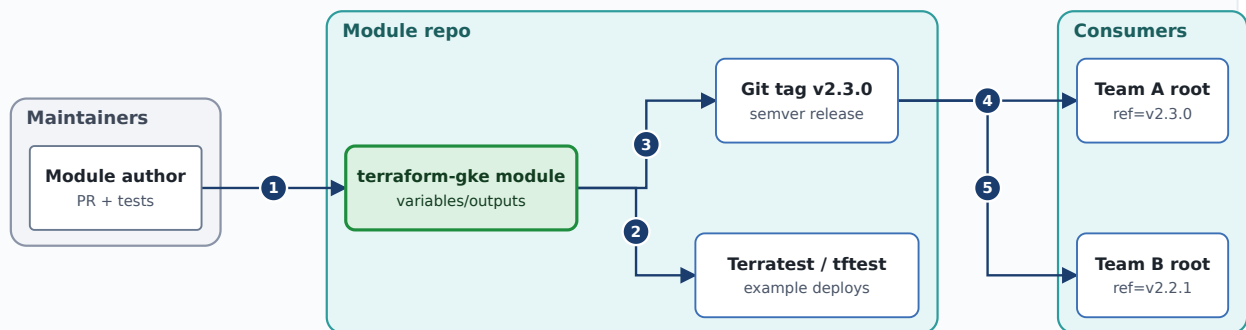
For environments with different projects, credentials and approval rules I prefer separate root directories (or separate stacks) that call the same versioned module. Each directory has its own backend, its own service account and can pin a different module version, so a change reaches prod only after it was promoted. Workspaces share one configuration and one backend configuration and differ only by `terraform.workspace`, which makes it easy to apply to the wrong environment and hard to give prod stricter access. Workspaces are fine for short-lived copies of the same stack, such as per-branch test environments.

KEY POINTS

- Isolation of state and credentials is the main goal, not less code.
- Workspaces suit identical, disposable copies.

Q238 How do you design a reusable Terraform module for a GKE cluster that several teams will consume, and how do you version it?

FLOW DIAGRAM



- 1 Small interface: required inputs, safe defaults
- 2 Tests run the examples in a sandbox project
- 3 Release tag; breaking change = major bump
- 4 Consumers pin an exact tag
- 5 Teams upgrade on their own schedule

EXPLANATION

Keep the module focused on one job with a small set of typed, validated variables, secure defaults (private nodes, Workload Identity, shielded nodes) and useful outputs such as the cluster endpoint and node service account. Do not configure providers inside the module; let the caller pass them. Publish releases as semantic-version Git tags or in a private registry, and consumers pin `?ref=v2.3.0` or a `version` constraint. Run `terraform test` or Terratest against example configurations in CI before tagging, and bump the major version on any change that forces resource replacement or renames inputs.

KEY POINTS

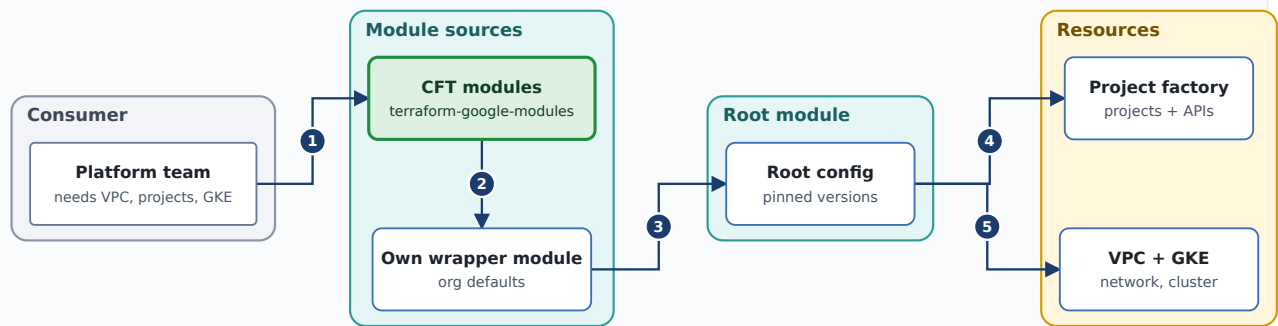
- Pin exact module versions in prod roots.
- Consider Google's Cloud Foundation Toolkit modules instead of writing from scratch.

CONFIGURATION EXAMPLE

```
module "gke" {
  source = "terraform-google-modules/kubernetes-engine/google//modules/private-cluster"
  version = "~> 33.0"
  project_id = var.project_id
  name       = "prod-gke"
  region     = "europe-west1"
  network    = module.vpc.network_name
}
```

Q239 What are the Cloud Foundation Toolkit (CFT) modules, and when would you use them instead of writing your own?

FLOW DIAGRAM



- ❶ Pick maintained, tested Google modules
- ❷ Thin wrapper sets naming, labels, policies
- ❸ Teams consume the wrapper
- ❹ project-factory creates projects
- ❺ network and kubernetes-engine modules

EXPLANATION

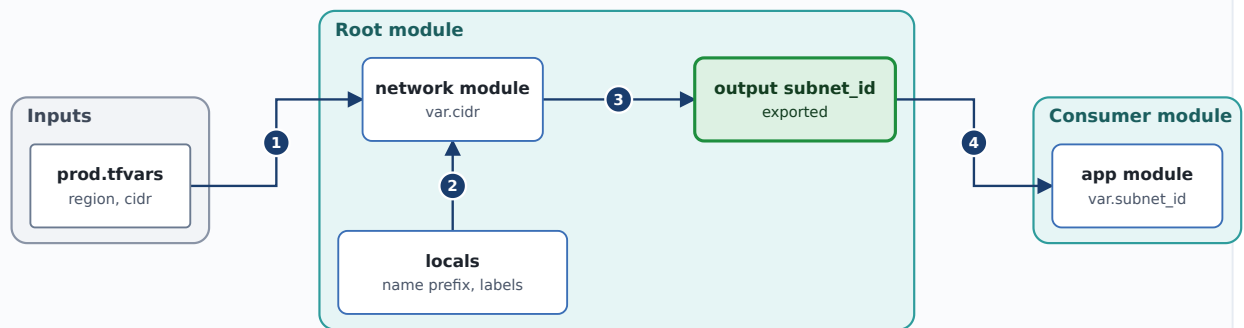
CFT is a set of open-source Terraform modules maintained under `terraform-google-modules` for common building blocks such as project-factory, network, kubernetes-engine, cloud-sql and IAM bindings, plus blueprints for a full foundation. They follow Google best practices and are tested against the provider, so they save time and reduce mistakes. A common pattern is a thin internal wrapper that calls the CFT module and fixes your org's naming, labels and security defaults. Write your own module when the CFT one is too generic or pulls in far more resources than you need.

KEY POINTS

- Pin CFT module versions; majors can change inputs.
- Wrap, don't fork, so you still get upstream fixes.

Q240 Explain variables, locals and outputs in Terraform with an example of how a network module passes its subnet to an app module.

FLOW DIAGRAM



- 1 Input variables with type and validation
- 2 locals compute repeated values once
- 3 Module output exposes the subnet ID
- 4 Root wires module.network.subnet_id into app

EXPLANATION

Input variables are the module's parameters, declared with a type, description, optional default and validation rules, and set from tfvars files, `-var` flags or `TF_VAR_` environment variables. Locals are named expressions inside a module used to avoid repeating things such as a naming prefix or common labels. Outputs expose values from a module, so the root can pass `module.network.subnet_id` into another module, and they also show after apply or can be read by other stacks. Mark variables and outputs holding secrets as `sensitive = true` so they are hidden in plan output.

KEY POINTS

- Validation blocks catch bad input before any API call.
- Sensitive hides values in output but they are still in state.

CONFIGURATION EXAMPLE

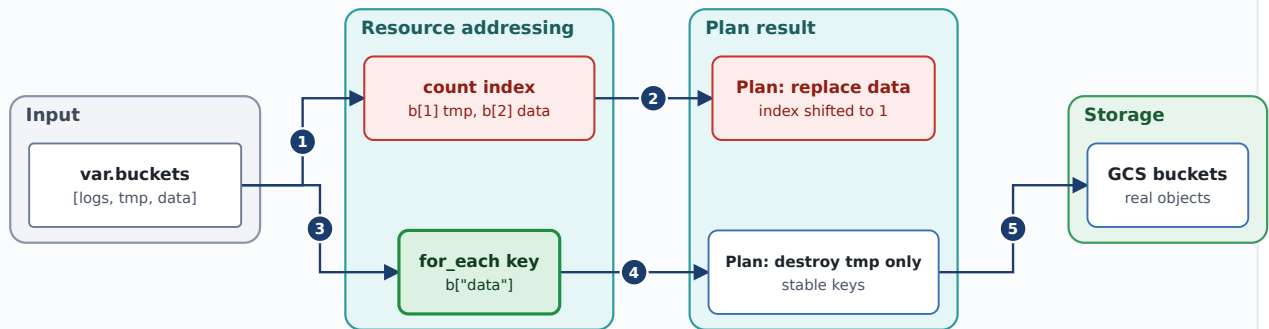
```
variable "env" {
  type = string
  validation {
    condition     = contains(["dev", "stage", "prod"], var.env)
    error_message = "env must be dev, stage or prod."
  }
}

locals { labels = { env = var.env, owner = "platform" } }

output "subnet_id" { value = google_compute_subnetwork.app.id }
```

Q241 A module creates buckets with `count` from a list. Someone removes the second name and Terraform wants to destroy and recreate the third bucket. Why, and how do you fix it?

FLOW DIAGRAM



- ❶ Removing tmp shifts data from [2] to [1]
- ❷ Terraform sees a changed name at index 1
- ❸ `for_each` keys by bucket name
- ❹ Only the removed key is destroyed

EXPLANATION

With `count` resources are addressed by list position, so deleting an element shifts every later element to a new index and Terraform plans to change or replace them. `for_each` over a set or map addresses each instance by a stable key such as the bucket name, so removing one key only destroys that one resource. Use `count` only for identical copies or a simple on/off switch (`count = var.enabled ? 1 : 0`). To migrate existing `count` resources to `for_each` without recreation, add `moved` blocks from `google_storage_bucket.b[2]` to `google_storage_bucket.b["data"]` .

KEY POINTS

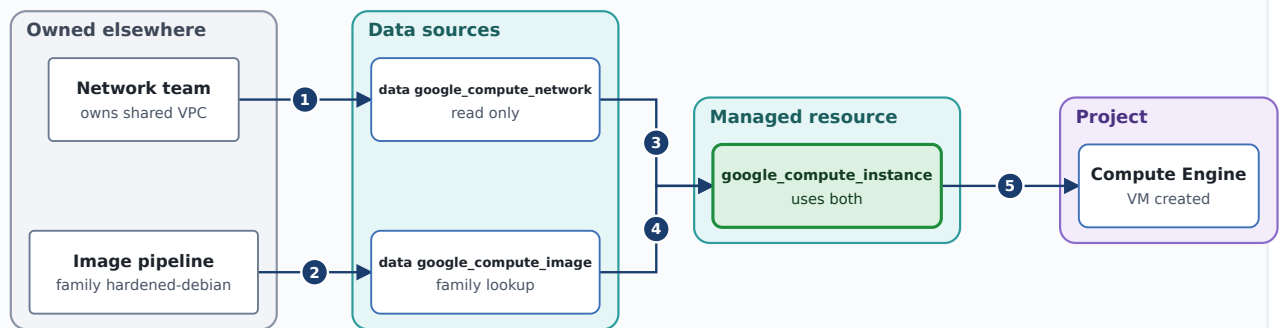
- `for_each` keys must be known at plan time.
- Use `moved` blocks to change addressing without downtime.

CONFIGURATION EXAMPLE

```
resource "google_storage_bucket" "b" {
  for_each = toset(var.buckets)
  name     = "${var.project_id}-${each.key}"
  location = "EU"
  uniform_bucket_level_access = true
}
```

Q242 Your Terraform needs the ID of a VPC and the latest hardened image that another team manages. How do you reference them without managing them?

FLOW DIAGRAM



- 1 Data source reads the existing network
- 2 Resolves the newest image in the family
- 3 self_link passed to network_interface
- 4 image self_link for boot disk
- 5 Only the VM is in this state file

EXPLANATION

Use data sources, which read existing objects through the provider during plan and never create or delete them. `data "google_compute_network"` looks up the shared VPC by name and project, and `data "google_compute_image"` with `family` returns the newest image in that family. Because the lookup runs on every plan, a new image can trigger a VM replacement, so pin the image or add `ignore_changes` on the boot disk if that is not wanted. For values another Terraform stack produces you can also read its outputs with `terraform_remote_state`, although passing values via data sources or config is looser coupling.

KEY POINTS

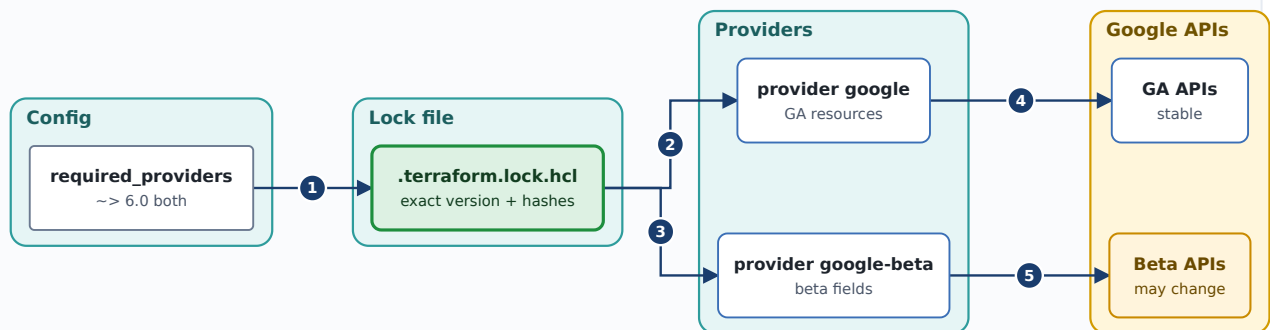
- Data sources are read-only and live outside your state's ownership.
- Floating lookups can cause surprise replacements.

CONFIGURATION EXAMPLE

```
data "google_compute_image" "base" {
  family = "hardened-debian-12"
  project = "acme-images"
}
data "google_compute_network" "shared" {
  name     = "shared-vpc"
  project = "acme-net-host"
}
```


Q243 Some resources you need are only in the `google-beta` provider. How do you pin provider versions and use both providers safely?

FLOW DIAGRAM



- 1 init selects a version inside the constraint
- 2 Same build on every machine and CI run
- 4 Most resources use google
- 5 Only resources with provider = google-beta

EXPLANATION

Declare both `google` and `google-beta` in `required_providers` with a pessimistic constraint such as `~> 6.0`, and keep them on the same version so their behaviour matches. Commit `.terraform.lock.hcl`, which records the exact version and checksums, and upgrade deliberately with `terraform init -upgrade` in a PR. Only resources that need beta features set `provider = google-beta`; everything else stays on GA. Read the provider upgrade guide before crossing a major version because defaults and field names can change.

KEY POINTS

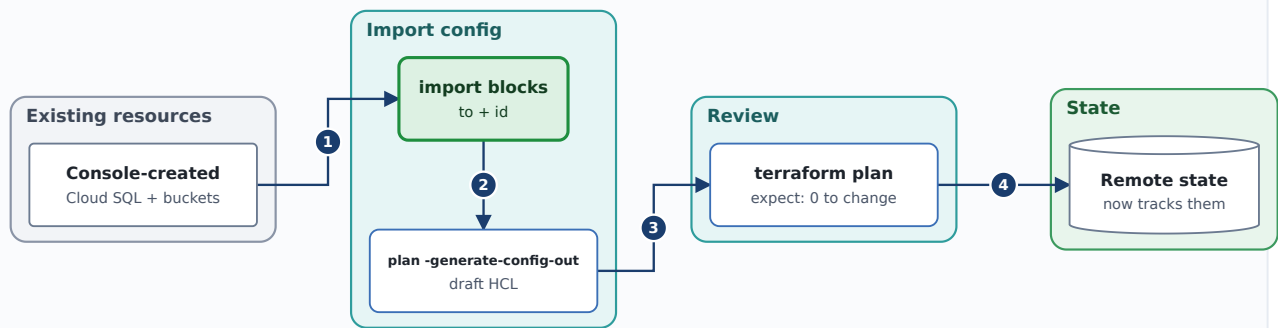
- Constraint in code, exact version in the lock file.
- Beta features have no stability guarantee.

CONFIGURATION EXAMPLE

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    google      = { source = "hashicorp/google",      version = "~> 6.0" }
    google-beta = { source = "hashicorp/google-beta", version = "~> 6.0" }
  }
}
```

Q244 A Cloud SQL instance and several buckets were created by hand in the console. How do you bring them under Terraform without recreating them?

FLOW DIAGRAM



- 1 Write import blocks with the resource IDs
- 2 Terraform can generate starter HCL
- 3 Edit config until plan shows no changes
- 4 apply records them in state; nothing is recreated

EXPLANATION

Since Terraform 1.5 you declare `import` blocks with the target address and the provider's import ID, then run `terraform plan -generate-config-out=generated.tf` to get draft HCL. Clean up the generated config, move it into proper modules, and repeat plan until it shows the resources as imports with no changes. Apply then writes them into state without touching the real resources, and the import is reviewable in a PR like any other change. The older `terraform import` CLI command does the same but is not part of the plan review. Tools such as `gcloud beta resource-config bulk-export` can also produce HCL for many resources.

KEY POINTS

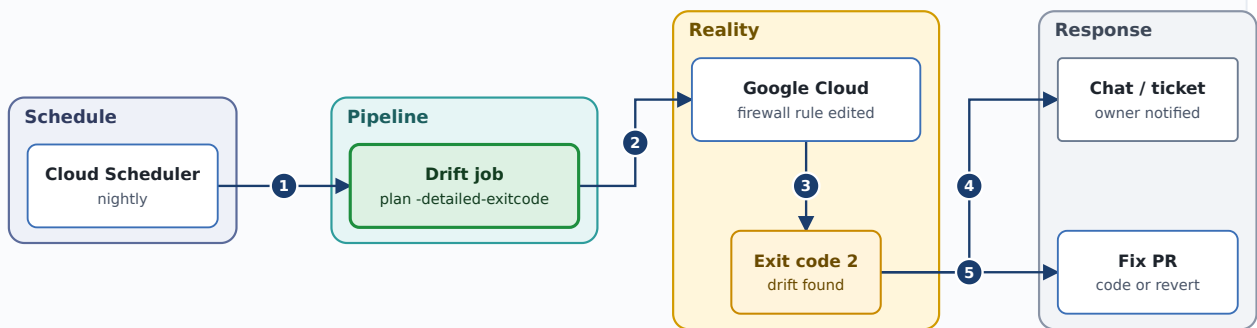
- Aim for a zero-change plan before the first apply.
- Import blocks can stay in code as a record, or be removed after.

CONFIGURATION EXAMPLE

```
import {
  to = google_sql_database_instance.main
  id = "projects/acme-prod/instances/orders-db"
}
import {
  to = google_storage_bucket.logs
  id = "acme-prod-logs"
}
# terraform plan -generate-config-out=generated.tf
```

Q245 Engineers sometimes change firewall rules in the console during incidents. How do you detect and handle this drift?

FLOW DIAGRAM



- 1 Run plan against each prod stack
- 2 Refresh reads live resources
- 3 Diff between code and live state
- 4 Alert with the plan output
- 5 Either codify the change or re-apply

EXPLANATION

Schedule a read-only pipeline that runs `terraform plan -detailed-exitcode` (or `plan -refresh-only`) on every production stack; exit code 2 means there are changes and the job posts the plan to the owning team. The team then either updates the code to adopt the emergency change or re-applies to revert it. Prevent drift by removing broad console write access, using Cloud Audit Logs to see who changed what, and giving an audited break-glass path for incidents. Recurring drift on one attribute that another system owns can be handled with `ignore_changes`.

KEY POINTS

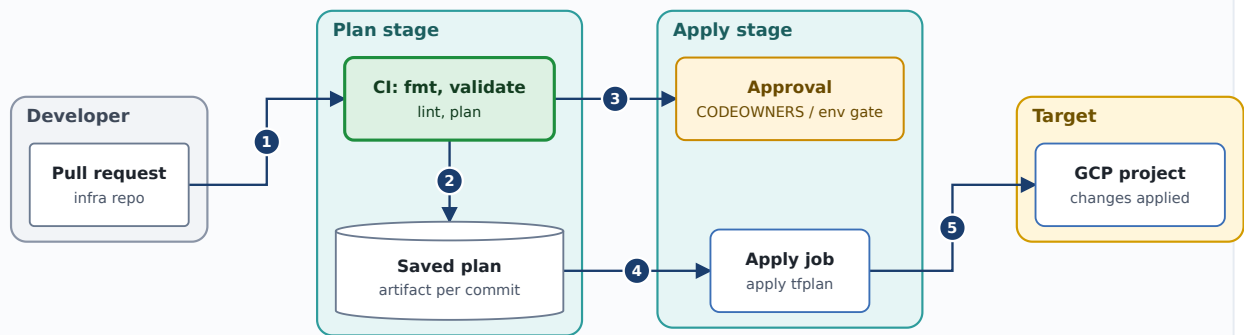
- Exit codes: 0 no changes, 1 error, 2 changes present.
- Drift is often an access problem, not a Terraform problem.

CONFIGURATION EXAMPLE

```
terraform plan -detailed-exitcode -lock=false -input=false -out=drift.plan
rc=$?
if [ $rc -eq 2 ]; then
  terraform show -no-color drift.plan | ./notify-chat.sh "#net-alerts"
fi
```

Q246 Design a CI pipeline for Terraform where every PR shows a plan, a human approves, and apply runs the exact reviewed plan.

FLOW DIAGRAM



- ❶ Checks run on every push
- ❷ Plan saved as artifact, summary posted to PR
- ❸ Reviewer reads the plan, approves
- ❹ Apply uses the saved plan after merge/approval
- ❺ Short-lived credentials via WIF

EXPLANATION

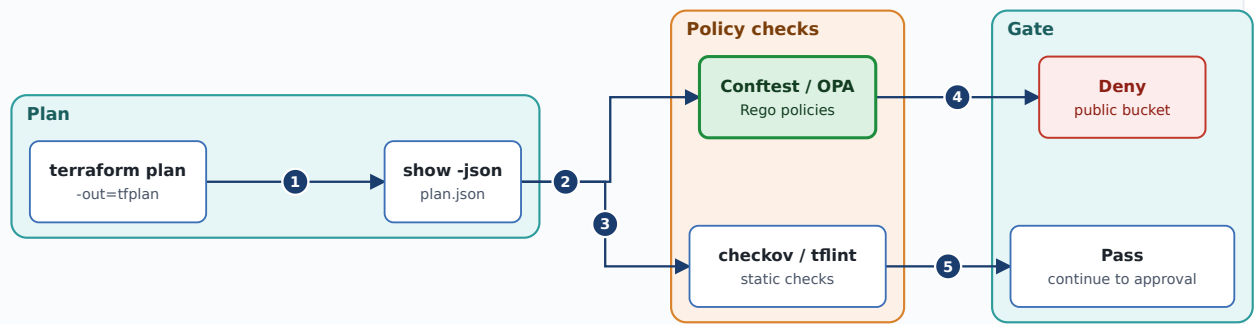
On each PR run `terraform fmt -check`, `validate`, linters and `plan -out=tfplan`, post the human-readable plan as a PR comment and store the binary plan as an artifact tied to the commit SHA. Require an approval from code owners, plus an environment protection rule for prod. The apply job downloads that exact plan and runs `terraform apply tfplan`; if state changed since the plan, Terraform refuses the stale plan and you re-plan. Only the pipeline's service account can write to prod, so there are no applies from laptops.

KEY POINTS

- Apply a saved plan, never a fresh unreviewed one.
- Serialise applies per stack; the state lock enforces it.

Q247 Security wants to block Terraform changes that create public buckets or VMs with external IPs before they are applied. How do you add policy-as-code?

FLOW DIAGRAM



- ❶ Convert the plan to JSON
- ❷ Rego evaluates resource_changes
- ❸ Scan HCL and plan for known misconfigs
- ❹ Fail the job with a clear message
- ❺ All policies pass

EXPLANATION

Export the plan with `terraform show -json tfplan` and evaluate it with Conftest/OPA Rego policies, for example denying `google_storage_bucket_iam_member` with `allUsers` or instances with `access_config`. Add static scanners such as tfint (with the google ruleset) and checkov for common misconfigurations. Terraform Cloud/Enterprise uses Sentinel or OPA for the same job, and `gcloud beta terraform vet` checks plans against constraint templates. Back this up with Organization Policy constraints, such as `storage.publicAccessPrevention`, so the API rejects bad resources even if someone bypasses the pipeline.

KEY POINTS

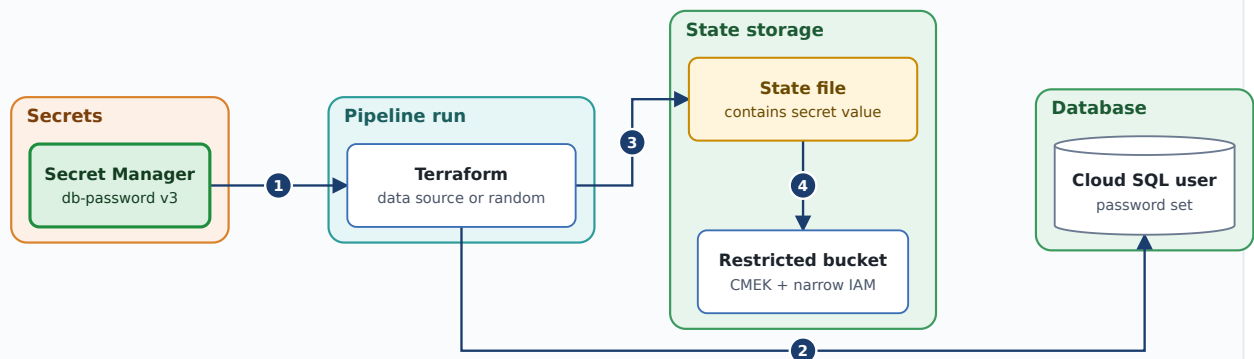
- Pipeline policy gives fast feedback; org policy is the hard guarantee.
- Policies are code: review, test and version them.

CONFIGURATION EXAMPLE

```
package terraform
deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "google_storage_bucket_iam_member"
  rc.change.after.member == "allUsers"
  msg := sprintf("%s makes a bucket public", [rc.address])
}
```

Q248 A Terraform config needs a database password. How do you handle secrets so they are not in Git and are protected in state?

FLOW DIAGRAM



- 1 Read secret at plan/apply time
- 2 Passed to google_sql_user
- 3 Value ends up in state
- 4 Protect state like a secret store

EXPLANATION

Never put secrets in `.tf` or `tfvars` files in Git. Either generate them with `random_password` and write them to Secret Manager, or read an existing secret with the `google_secret_manager_secret_version` data source, and mark the variables `sensitive`. Remember the value is still stored in state in plain text, so the state bucket needs CMEK encryption, versioning and IAM limited to the Terraform service accounts. Where possible avoid passwords entirely, for example IAM database authentication for Cloud SQL or Workload Identity for apps. Newer Terraform versions also support ephemeral values and write-only arguments that keep secrets out of state; check provider support.

KEY POINTS

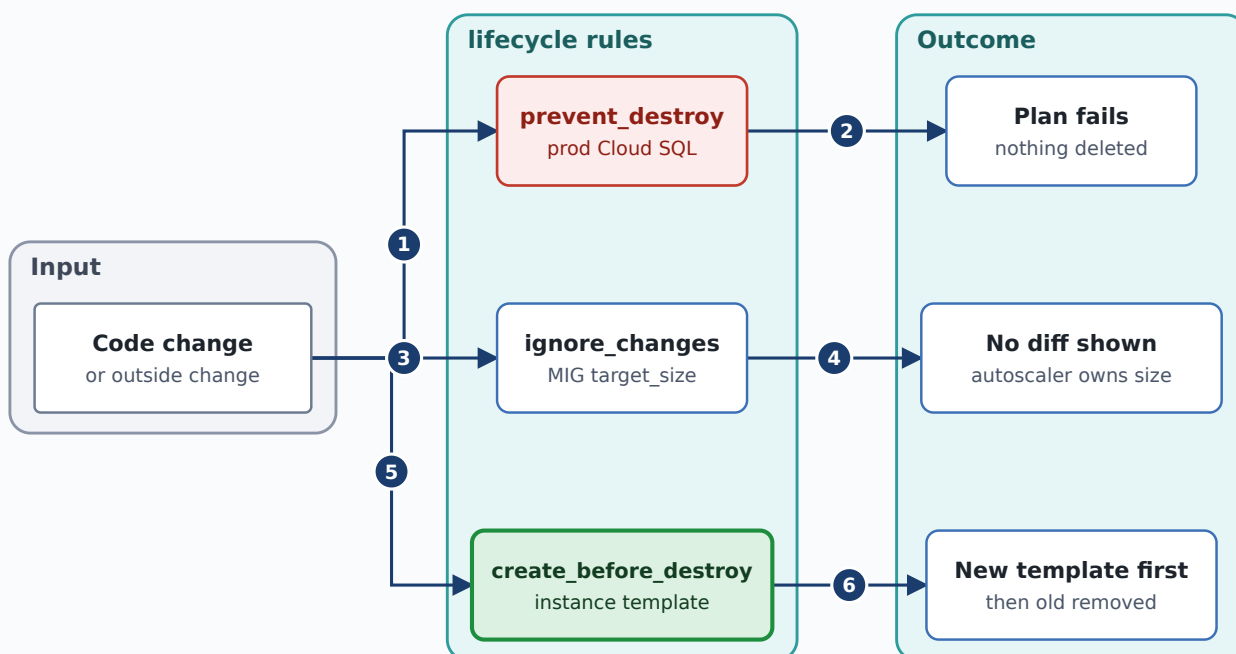
- `sensitive` hides output, it does not encrypt state.
- Best secret is no secret: prefer IAM-based auth.

CONFIGURATION EXAMPLE

```
resource "random_password" "db" { length = 24 }
resource "google_secret_manager_secret_version" "db" {
  secret      = google_secret_manager_secret.db.id
  secret_data = random_password.db.result
}
resource "google_sql_user" "app" {
  name       = "app"
  instance   = google_sql_database_instance.main.name
  password   = random_password.db.result
}
```

Q249 Explain `prevent_destroy`, `create_before_destroy` and `ignore_changes`, with a real case for each.

FLOW DIAGRAM



- ❶ Plan would destroy the database
- ❷ Error; nothing is deleted
- ❸ Autoscaler changed `target_size`
- ❹ Attribute ignored in the diff
- ❺ Template needs replacement
- ❻ MIG never points at a deleted template

EXPLANATION

`prevent_destroy = true` makes any plan that would destroy the resource fail, which protects things like a production Cloud SQL instance or state bucket from an accidental rename. `create_before_destroy = true` builds the replacement before deleting the old object, which is needed for instance templates referenced by a managed instance group or certificates attached to a load balancer; the new object usually needs a unique name via `name_prefix` or a random suffix. `ignore_changes` tells Terraform to ignore attributes changed by something else, such as a MIG size set by the autoscaler or labels added by another tool. Also consider the resource's own `deletion_protection` field, which blocks deletion at the API level too.

KEY POINTS

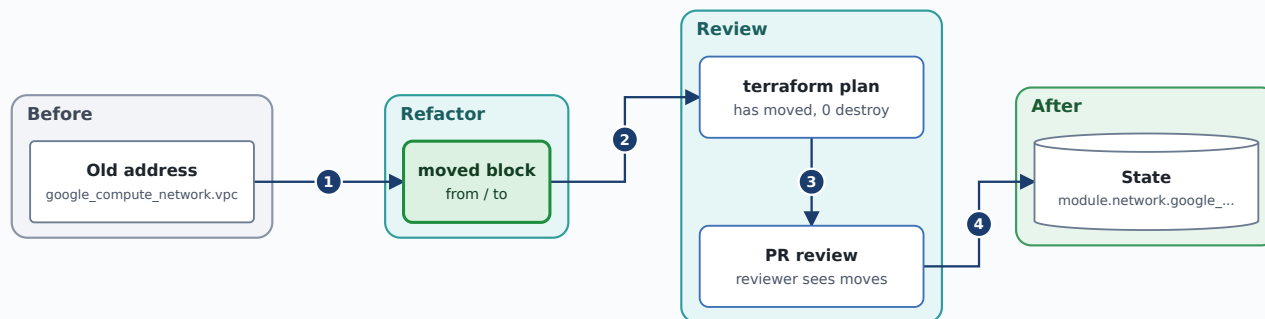
- `prevent_destroy` is lost if the resource block is removed from code.
- Use API-level `deletion_protection` for critical data.

CONFIGURATION EXAMPLE

```
resource "google_compute_instance_template" "web" {
  name_prefix = "web-"
  machine_type = "e2-standard-2"
  lifecycle { create_before_destroy = true }
}
resource "google_sql_database_instance" "main" {
  lifecycle { prevent_destroy = true }
}
```

Q250 You want to move resources from the root module into a new `network` module. How do you refactor without Terraform destroying and recreating the VPC?

FLOW DIAGRAM



- 1 Declare old and new addresses
- 2 Plan shows the move, no replace
- 3 Reviewed like any change
- 4 Apply updates addresses in state only

EXPLANATION

Add `moved` blocks that map each old address to its new address, for example `from = google_compute_network.vpc` and `to = module.network.google_compute_network.vpc`. Plan then reports the resource as moved with no changes, and apply just updates the address in state. Unlike running `terraform state mv` by hand, moved blocks are code-reviewed and work for every workspace or environment that uses the configuration. Keep the blocks for a while, especially in shared modules, so consumers upgrading from older versions are also migrated.

KEY POINTS

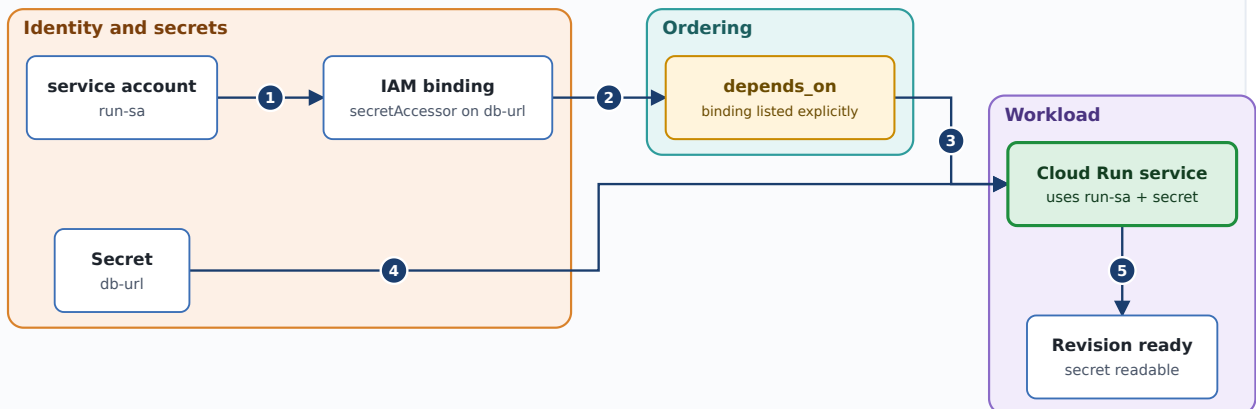
- Moved blocks also work for renames and count to `for_each`.
- Check the plan says 0 to destroy before applying.

CONFIGURATION EXAMPLE

```
moved {
  from = google_compute_network.vpc
  to   = module.network.google_compute_network.vpc
}
moved {
  from = google_compute_subnetwork.app
  to   = module.network.google_compute_subnetwork.app
}
```


Q251 A Cloud Run service fails on first apply because the IAM binding for its service account's Secret Manager access is created after it. How does Terraform order things and when do you use `depends_on`?

FLOW DIAGRAM



- 1 Implicit: binding references the SA email
- 2 Service does not reference the binding
- 3 `depends_on` adds the missing graph edge
- 4 Implicit: service references the secret
- 5 Created only after access is granted

EXPLANATION

Terraform builds a dependency graph from references: if resource B uses an attribute of A, A is created first and resources without references run in parallel. The Cloud Run service references the secret and the service account but not the IAM binding, so Terraform may create the service before access exists and the revision fails. Adding `depends_on = [google_secret_manager_secret_iam_member.run]` to the service adds that missing edge. Use `depends_on` only for hidden dependencies like this, because it makes plans more conservative, and on modules it can make data sources defer to apply time.

KEY POINTS

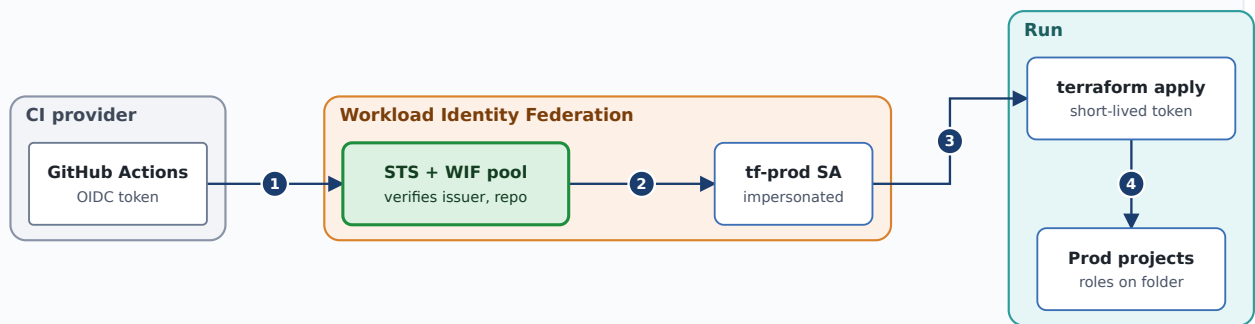
- Prefer real references over `depends_on`.
- IAM changes can take a short time to propagate even when ordered.

CONFIGURATION EXAMPLE

```
resource "google_cloud_run_v2_service" "api" {
  name      = "api"
  location = "europe-west1"
  template { service_account = google_service_account.run.email }
  depends_on = [google_secret_manager_secret_iam_member.run]
}
```

Q252 How should the CI pipeline authenticate to GCP to run Terraform, without storing a service account key?

FLOW DIAGRAM



- 1 Exchange the OIDC token for a federated token
- 2 Attribute condition: repo and branch = main
- 3 Access token valid about 1 hour
- 4 Only the roles this stack needs

EXPLANATION

Use Workload Identity Federation: create a workload identity pool and OIDC provider for GitHub (or GitLab, Azure DevOps), with an attribute condition limiting it to your org, repo and protected branch or environment. Grant the provider's principal `roles/iam.workloadIdentityUser` on a dedicated Terraform service account, and the job gets short-lived tokens with no key to leak. Use separate service accounts for plan (read-only) and apply, and per environment, each granted only the roles its stacks need at folder or project level. Also disable service account key creation with the `iam.disableServiceAccountKeyCreation` org policy.

KEY POINTS

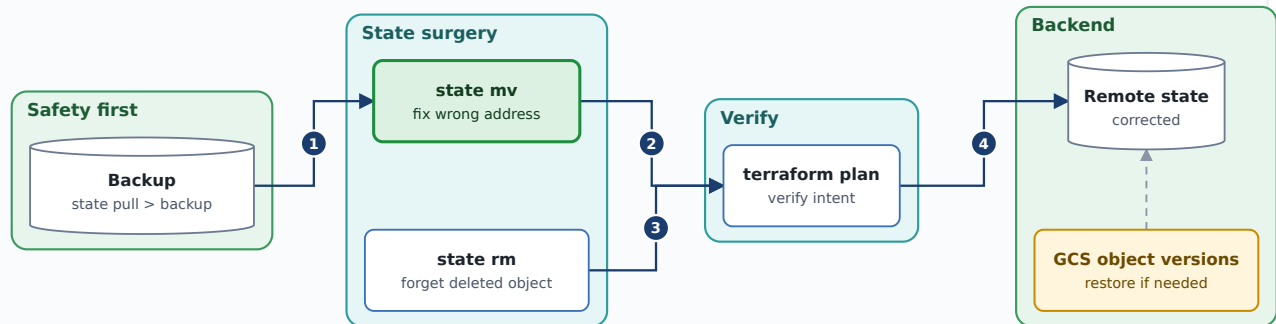
- Restrict by repository and ref, not only by pool.
- Split plan and apply identities.

CONFIGURATION EXAMPLE

```
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider:
      projects/123456/locations/global/workloadIdentityPools/github/providers/gh
    service_account: tf-prod@acme-admin.iam.gserviceaccount.com
- run: terraform init && terraform apply -input=false tfplan
```

Q253 State is out of sync after a bad refactor: one resource is under the wrong address and another was deleted outside Terraform. How do you recover safely?

FLOW DIAGRAM



- 1 Take the lock, back up current state
- 2 Address fixed without API calls
- 3 Object no longer tracked
- 4 Plan shows only expected changes
- Roll back to an older object version

EXPLANATION

First stop other runs and save a copy with `terraform state pull > backup.tfstate`; with GCS versioning you can also restore an earlier object. Use `terraform state mv` to move a resource to its correct address and `terraform state rm` to forget an object that no longer exists or should be managed elsewhere; neither command touches the real resource. For a deleted resource you can also just run `plan`, which will propose recreating it. Afterwards run `terraform plan` and confirm it shows only the changes you expect, and prefer `moved` and `removed` blocks in code for planned refactors.

KEY POINTS

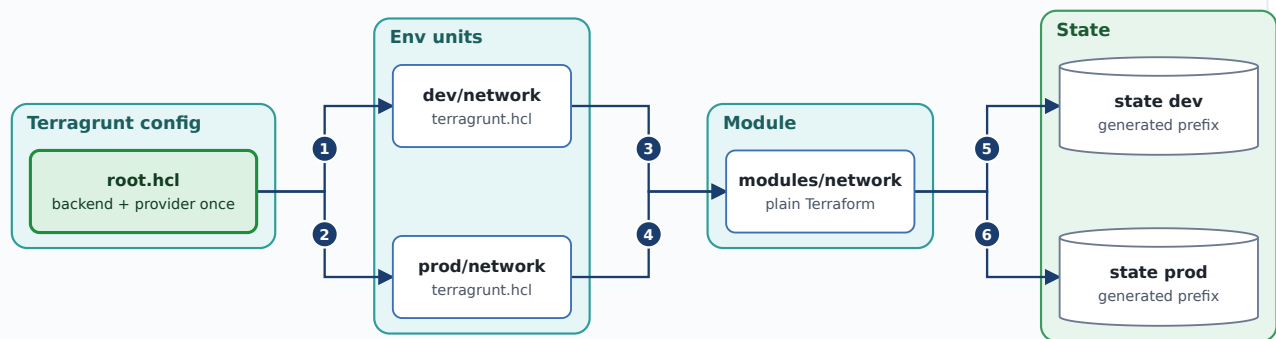
- state commands change only state, never the resources.
- Always back up and plan after surgery.

CONFIGURATION EXAMPLE

```
terraform state pull > backup-$(date +%F).tfstate
terraform state list | grep sql
terraform state mv 'google_sql_database_instance.db' 'module.db.google_sql_database_instance.main'
terraform state rm 'google_storage_bucket.tmp_old'
terraform plan
```

Q254 Your team has 40 environment folders with copy-pasted backend and provider blocks. Where does Terragrunt help, and what are the alternatives?

FLOW DIAGRAM



- ❶ include inherits remote_state
- ❸ Inputs passed to one module
- ❺ Backend prefix from folder path

EXPLANATION

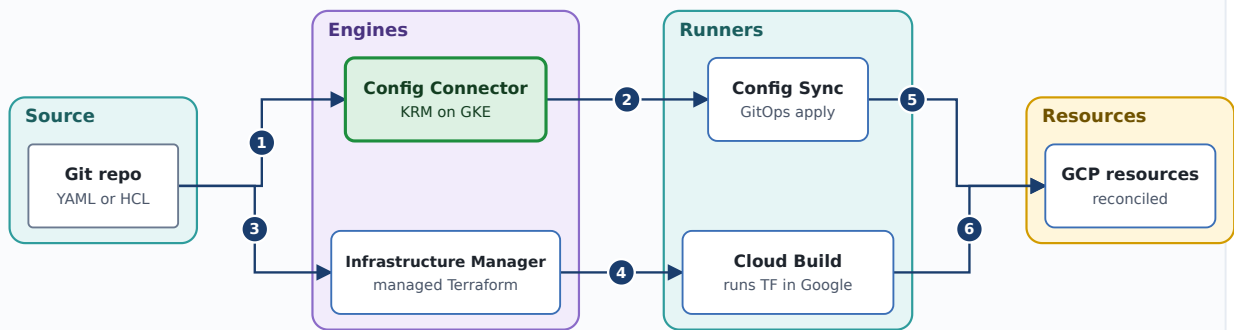
Terragrunt is a thin wrapper around Terraform that generates backend and provider blocks from a parent file, derives the state prefix from the folder path, passes inputs to a module, and can run many units with dependencies between them (`run-all`). It removes a lot of boilerplate in large multi-environment layouts. The cost is another tool and more indirection for new engineers to learn. Alternatives include Terraform Cloud/HCP workspaces, Terraform stacks, or simple scripts plus partial backend configuration (`-backend-config`).

KEY POINTS

- Keep modules plain Terraform so they work without Terragrunt.
- Use it when boilerplate, not logic, is the pain.

Q255 When would you choose Config Connector or Infrastructure Manager over running Terraform yourself?

FLOW DIAGRAM



- ❶ Resources as Kubernetes objects
- ❷ Controllers reconcile continuously
- ❸ Terraform configs deployed via API
- ❹ State stored and runs managed by Google

EXPLANATION

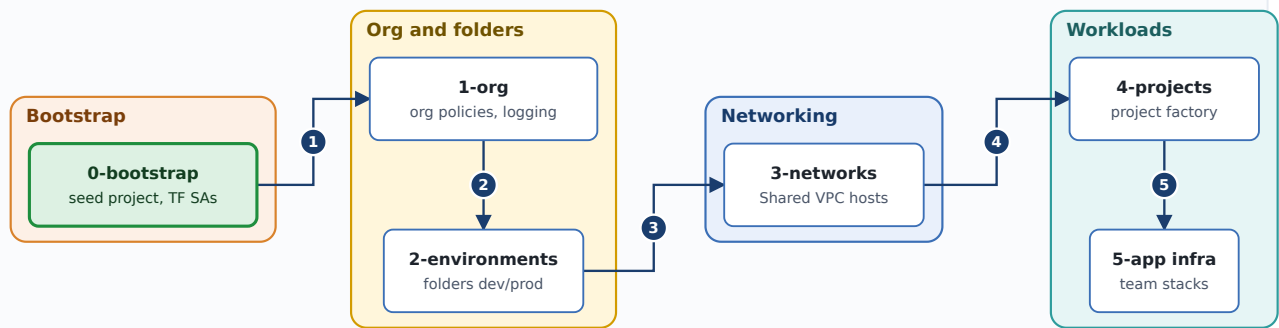
Config Connector exposes GCP resources as Kubernetes custom resources, and its controllers continuously reconcile them, so teams already doing GitOps on GKE can manage buckets, Pub/Sub and IAM in the same YAML and tooling as their apps; drift is corrected automatically. Infrastructure Manager is a Google-managed service that runs your Terraform configurations with Cloud Build and stores the state for you, which suits teams that want Terraform without running their own runners and backends. Self-managed Terraform with CI gives the most control, the widest provider ecosystem and portability. Many organisations use Terraform for the foundation and Config Connector for app-level resources.

KEY POINTS

- Config Connector: continuous reconciliation, Kubernetes-native.
- Infrastructure Manager: Terraform with managed state and execution.

Q256 You are building a GCP landing zone for 50 teams with Terraform. How do you structure the stacks for the org, folders, networking and projects?

FLOW DIAGRAM



- ❶ State bucket and CI identities first
- ❷ Folders under the org
- ❸ Host projects per environment
- ❹ Service projects attached to Shared VPC
- ❺ Teams own their app stacks

EXPLANATION

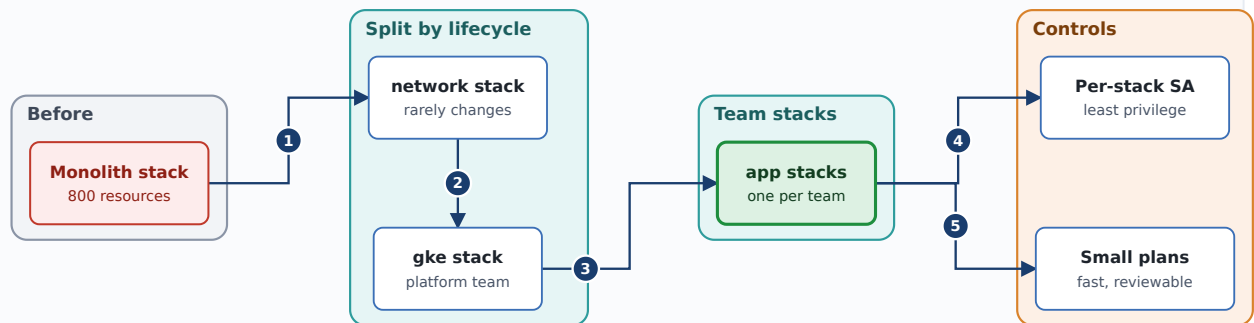
Split the foundation into layered stacks, as in Google's terraform-example-foundation: bootstrap (seed project, state buckets, CI service accounts), org (org policies, log sinks, security settings), environments (folders), networks (Shared VPC host projects, hub-and-spoke or VPC per environment), projects (project factory) and then app stacks owned by teams. Each layer has its own state, pipeline and service account with the least privilege for that layer, and later layers read earlier outputs through remote state or known names. New projects come from a project factory request (YAML or a PR), which applies labels, budgets, APIs and Shared VPC attachment consistently.

KEY POINTS

- Layered stacks change at different speeds and need different owners.
- Start from terraform-example-foundation rather than a blank repo.

Q257 One root module manages the network, GKE and 30 apps for prod, and plans now take 20 minutes and every change feels risky. How do you reduce the blast radius?

FLOW DIAGRAM



- 1 Move resources with moved/removed + import
- 2 Outputs read by name or remote state
- 4 A bad app change cannot touch the VPC
- 5 Minutes, not tens of minutes

EXPLANATION

Split the monolith into stacks by lifecycle and ownership: network, shared platform (GKE, DNS), and one stack per app or team, each with its own state, pipeline and service account. A mistake in an app stack then cannot delete the VPC because its identity has no permission and its state does not contain it. Migrate with `removed` blocks (or `state rm`) in the old stack and `import` blocks in the new one, verifying zero-change plans on both sides. Add `prevent_destroy` or `deletion_protection` on critical resources, and avoid `-target` as a routine workaround because it hides the real dependency picture.

KEY POINTS

- Smaller state = faster plans and smaller mistakes.
- Least-privilege SAs per stack enforce the boundary.

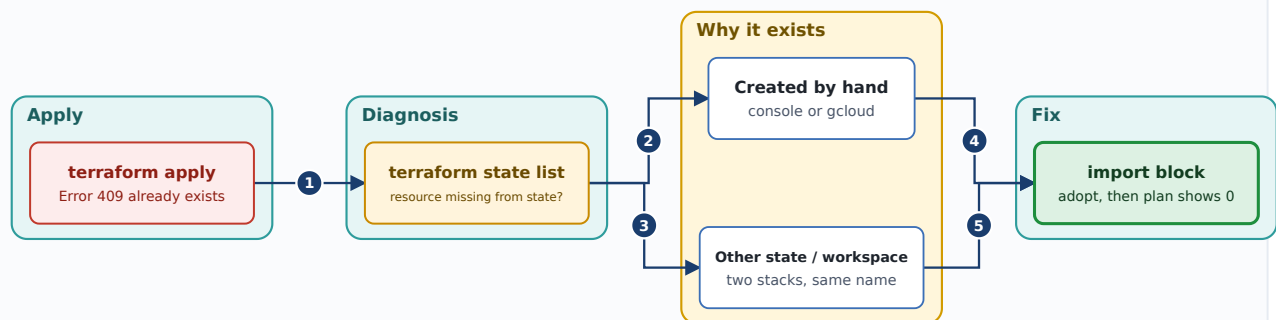
CONFIGURATION EXAMPLE

```
# old stack: stop managing, keep the resource
removed {
  from = google_container_cluster.prod
  lifecycle { destroy = false }
}
# new stack
import {
  to = google_container_cluster.prod
  id = "projects/acme-prod/locations/europe-west1/clusters/prod"
}
```

Q258**Troubleshooting**

terraform apply fails with Error 409: The resource 'projects/prod/global/networks/prod-vpc' already exists. Nobody changed the code. How do you fix it?

FLOW DIAGRAM



- ❶ Terraform tries to create what it does not track
- ❷ Audit logs show who created it
- ❸ Another root module may already own it
- ❹ Import it into this state
- ❺ Or remove it from one stack and import in the other

EXPLANATION

A 409 means Google Cloud already has a resource with that name, but it is not in this state file, so Terraform tries to create it again. Check `terraform state list` and the Admin Activity audit logs to see who created it: often someone made it in the console, an earlier apply timed out after creating it, or a second stack or workspace manages the same name. If this stack should own it, add an `import` block (Terraform 1.5+) or run `terraform import`, then run `plan` until it shows no changes. If another stack owns it, reference it with a data source instead of creating it twice.

KEY POINTS

- One resource must be owned by exactly one state; use data sources elsewhere.
- Add a random suffix to names of resources that do not need fixed names.

CONFIGURATION EXAMPLE

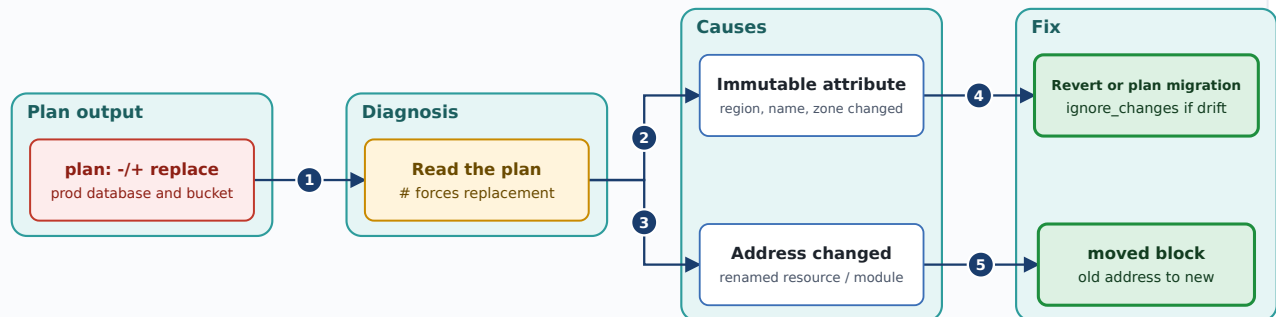
```
terraform state list | grep google_compute_network
gcloud logging read 'protoPayload.methodName="v1.compute.networks.insert"' --limit=5
# adopt it (Terraform 1.5+)
import {
  to = google_compute_network.prod
  id = "projects/prod/global/networks/prod-vpc"
}
terraform plan # expect: 0 to add, 0 to destroy
```


Q259

Troubleshooting

A small PR makes `terraform plan show` `-/+ destroy` and then create replacement for a production Cloud SQL instance and a bucket. How do you find out why and avoid the replacement?

FLOW DIAGRAM



- 1 Find the line marked forces replacement
- 2 ForceNew attribute differs from real resource
- 3 Old address destroyed, new address created
- 4 Change it back or migrate data deliberately
- 5 Tell Terraform it is the same object

EXPLANATION

Look for `# forces replacement` in the plan: it marks the attribute that cannot be updated in place (a ForceNew attribute such as `name`, `region`, `database_version` on some changes, or `location` on a bucket). If the PR changed that value, revert it or plan a real migration; if the value differs only because the provider normalises it or someone edited the resource, align the code or use `ignore_changes`. If the plan shows a destroy of one address and a create of another, the resource or module was renamed: add a `moved` block from the old address to the new one. Add `prevent_destroy` to stateful resources so a surprise replacement fails the plan instead of deleting data.

KEY POINTS

- Fail CI when a plan contains deletes of protected resources (check `terraform show -json`).
- Cloud SQL `deletion_protection` adds a second safety net.

CONFIGURATION EXAMPLE

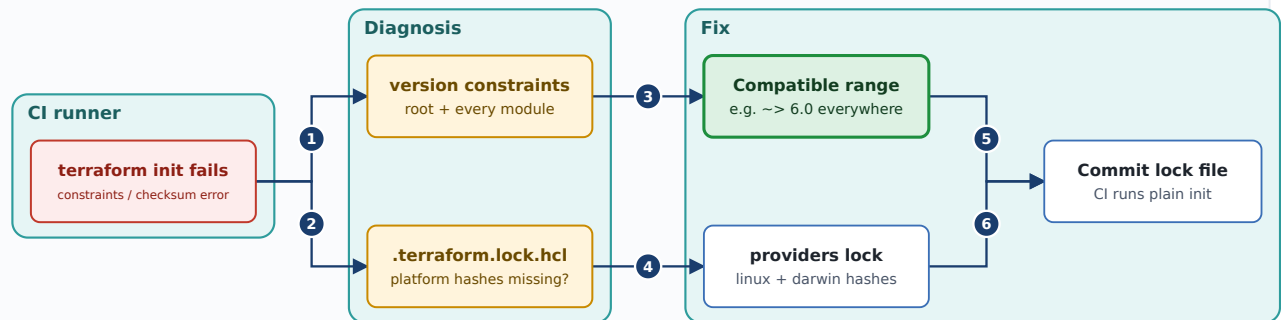
```

terraform plan -out=tf.plan | grep -B2 -A2 'forces replacement'
terraform show -json tf.plan | jq -r '.resource_changes[]
  | select(.change.actions|index("delete")) | .address'
moved {
  from = google_sql_database_instance.db
  to   = module.database.google_sql_database_instance.main
}

```

Q260**Troubleshooting**

After `terraform init -upgrade`, CI fails with `Error: Failed to query available provider packages ... no available releases match the given constraints`, **and other engineers get** the cached package does not match any of the checksums in `.terraform.lock.hcl`. **How do you fix it?**

FLOW DIAGRAM

- ❶ terraform providers shows who asks for what
- ❷ Lock was upgraded on one OS only
- ❸ Remove conflicting pins, e.g. = 5.x vs >= 6
- ❹ Record hashes for all platforms used
- ❺ Review the upgrade like code

EXPLANATION

`terraform providers` prints the version constraints from the root and each module; the error means two of them cannot be satisfied together, for example a module pinned to `= 5.40.0` while the root needs `>= 6.0`. Relax or update the module constraint (or upgrade the module version) so one release fits all. The checksum error means the lock file was regenerated on one platform (say macOS) and lacks hashes for the Linux CI runners; run `terraform providers lock` for every platform and commit the file. After that, CI should run plain `terraform init`, never `-upgrade`, so provider upgrades only happen in a reviewed PR.

KEY POINTS

- Modules should use minimum versions (`>=`); only root modules pin tightly.
- Read the provider changelog before a major version upgrade: defaults and resources change.

CONFIGURATION EXAMPLE

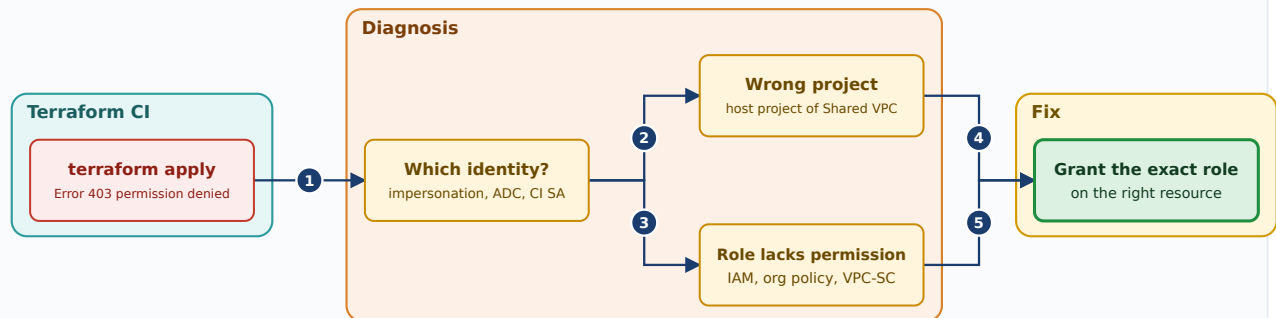
```

terraform providers
terraform init -upgrade
terraform providers lock \
  -platform=linux_amd64 -platform=darwin_arm64 -platform=darwin_amd64
git add .terraform.lock.hcl && git commit -m "Upgrade google provider to 6.x"
# CI:
terraform init -input=false -lockfile=readonly

```

Q261**Troubleshooting****The Terraform pipeline fails with Error 403: Permission**

'compute.subnetworks.create' denied on resource ... (or it may not exist) although the SA has Editor on the project. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Check the principal in the audit log
- 2 Resource may live in another project
- 3 Editor cannot set IAM policies, for example
- 4 e.g. compute.networkAdmin on the host project
- 5 Policy Troubleshooter shows the missing binding

EXPLANATION

First confirm which identity Terraform really uses: the Data Access or Admin Activity audit log entry for the denied call shows `principalEmail`, and it may be a user's ADC or the default compute SA instead of the Terraform SA if impersonation is not set. Then check the project in the error: with Shared VPC, subnets live in the host project, so the SA needs roles there, not in the service project. Editor also lacks some permissions, such as setting IAM policies (`setIamPolicy`) or managing some org-level settings. Grant the specific predefined role on the right project or folder, and if the SA already has it, check IAM deny policies, org policies and VPC Service Controls; (or it may not exist) can also mean the API or resource really is missing.

KEY POINTS

- Give the Terraform SA narrow predefined roles per stack instead of Editor or Owner.
- Set `impersonate_service_account` in the provider so every run uses the same identity.

CONFIGURATION EXAMPLE

```

gcloud logging read 'protoPayload.status.code=7 AND
  protoPayload.methodName:"subnetworks.insert" --project=net-host \
  --format='value(protoPayload.authenticationInfo.principalEmail)' --limit=3
gcloud projects add-iam-policy-binding net-host \
  --member=serviceAccount:tf-net@iac.iam.gserviceaccount.com \
  --role=roles/compute.networkAdmin

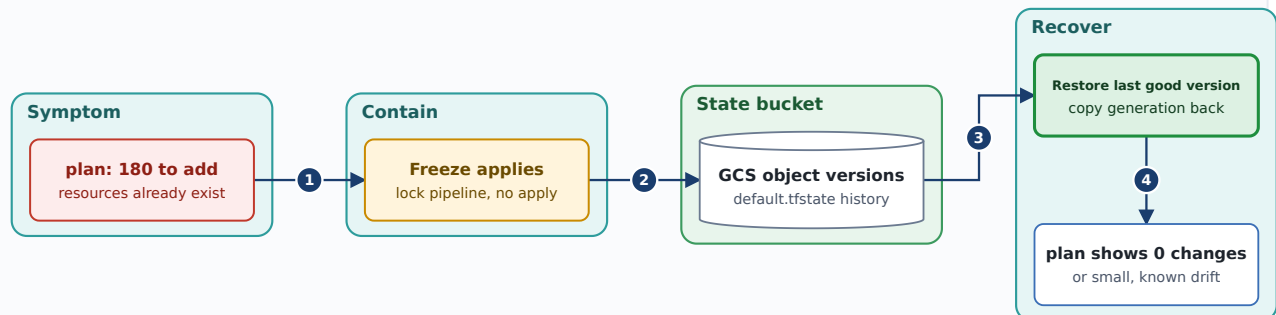
```

Q262

Troubleshooting

Someone overwrote the Terraform state in GCS with an almost empty file, and now `terraform plan` wants to create 180 resources that already exist. How do you recover?

FLOW DIAGRAM



- 1 Do not apply: it would hit 409 errors or duplicates
- 2 List noncurrent versions of the state object
- 3 Pick the version before the bad write (check serial)
- 4 Verify with plan before re-enabling CI

EXPLANATION

Stop all applies first, because an apply now would try to create duplicates or fail halfway. If object versioning is on the state bucket, list the versions of `default.tfstate`, download the one before the bad write, check its `serial` and resource count, and copy that generation back as the live object. Run `terraform plan`: it should show zero or only known drift. Without versioning, you must rebuild state by importing resources (import blocks can be generated with `terraform plan -generate-config-out`), which is slow, so versioning plus restricted write access on the bucket is the real fix.

KEY POINTS

- Enable versioning and a retention rule on the state bucket from day one.
- Only the CI service account should have write access to the state bucket.

CONFIGURATION EXAMPLE

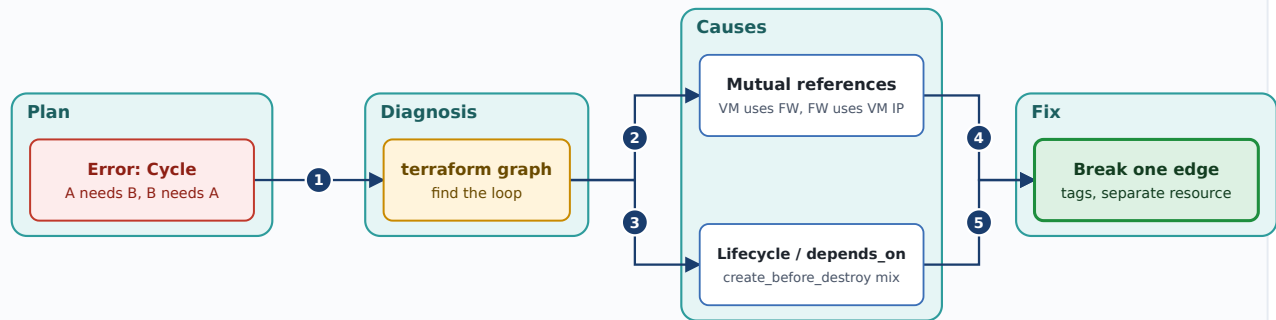
```

gsutil ls -la gs://acme-tfstate/prod/network/default.tfstate
gsutil cp gs://acme-tfstate/prod/network/default.tfstate#1727400000123456 good.tfstate
jq '.serial, (.resources | length)' good.tfstate
gsutil cp good.tfstate gs://acme-tfstate/prod/network/default.tfstate
terraform plan

```

Q263**Troubleshooting**

`terraform plan` fails with `Error: Cycle: google_compute_instance.app, google_compute_firewall.app, ...`. What causes a cycle and how do you break it?

FLOW DIAGRAM

- ❶ Error lists the resources in the loop
- ❷ Two resources read each other's attributes
- ❸ Needless `depends_on` or mixed lifecycle rules
- ❹ Use a static value both can read
- ❺ Remove the extra dependency

EXPLANATION

Terraform builds a dependency graph from references and `depends_on`, and a cycle means resource A needs a value from B while B needs a value from A, so neither can be created first. Read the resources listed in the error (or render `terraform graph`) and find the reference that closes the loop, for example a firewall rule using the VM's IP while the VM references the firewall. Break it with a value both can use without depending on each other (network tags, a reserved `google_compute_address`, a local), or split the association into its own resource, such as a separate IAM member or attachment resource. Also remove unnecessary `depends_on`, and note that a `create_before_destroy` resource depending on one without it can cause cycles during replacement.

KEY POINTS

- Prefer separate association resources (IAM member, attachments) over inline blocks.
- Use `depends_on` only for hidden dependencies Terraform cannot see.

CONFIGURATION EXAMPLE

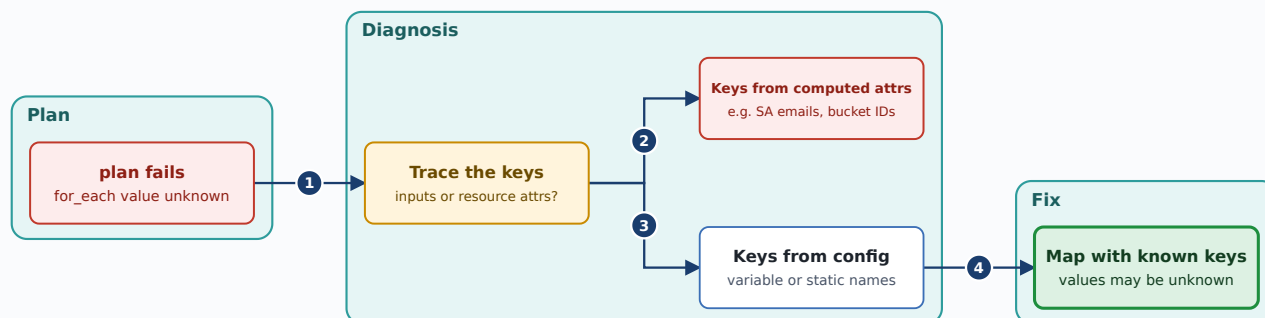
```

terraform graph -type=plan | dot -Tsvg > graph.svg
# break the loop: firewall targets a tag, not the VM's IP
resource "google_compute_firewall" "app" {
  network    = google_compute_network.vpc.id
  target_tags = ["app"]
  source_ranges = ["10.10.0.0/16"]
  allow {
    protocol = "tcp"
    ports    = ["8080"]
  }
}

```

Q264**Troubleshooting**

terraform plan fails with The "for_each" set includes values derived from resource attributes that cannot be determined until apply. **How do you fix it?**

FLOW DIAGRAM

- 1 Trace the for_each expression back
- 2 Keys only exist after apply
- 3 Rewrite keys from input names
- 4 Keys known at plan, values filled at apply
- A first apply with -target is only a one-off workaround

EXPLANATION

for_each keys must be known during plan, because Terraform uses them as resource addresses; here they come from attributes that only exist after apply, such as a service account email or a resource ID. Rewrite the expression so the keys come from configuration (the names in a variable or local) and put the computed values in the map's values, for example `{ for name in var.sa_names : name => google_service_account.sa[name].email }`. The same applies to count with a computed length. Applying once with -target works as an emergency workaround, but it is not a fix, because the next fresh environment will hit the same error.

KEY POINTS

- Rule of thumb: keys from inputs, values from resources.
- Module inputs used in for_each inside the module must be known too.

CONFIGURATION EXAMPLE

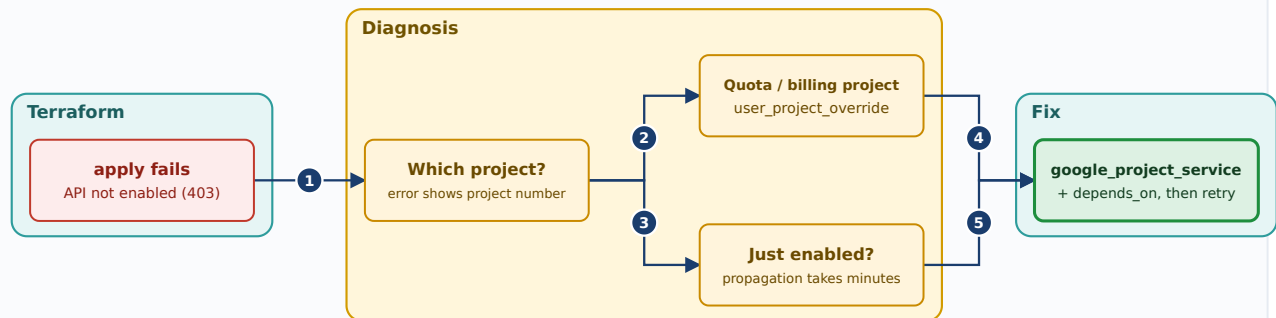
```

variable "sa_names" { type = set(string) }
resource "google_service_account" "sa" {
  for_each = var.sa_names
  account_id = each.key
}
resource "google_project_iam_member" "logs" {
  for_each = var.sa_names # known at plan
  project = var.project
  role    = "roles/logging.logWriter"
  member  = "serviceAccount:${google_service_account.sa[each.key].email}"
}
  
```

Q265**Troubleshooting**

Apply on a new project fails with Error 403: Cloud Run Admin API has not been used in project 123456 before or it is disabled. The next run fails with the same error even though you enabled it. How do you fix it properly?

FLOW DIAGRAM



- ❶ Number may be the SA's project, not the target
- ❷ Some APIs bill the caller's project
- ❸ Fresh enables need a short wait
- ❹ Enable in the right project
- ❺ Manage APIs in code with explicit dependencies

EXPLANATION

Check the project number in the message: it is often the project that owns the Terraform service account or ADC quota project, not the target project, because some APIs check the caller's quota project. Enable the API there, or set `user_project_override = true` and `billing_project` in the provider so calls bill the target project. If you enable the API in the same apply, Terraform may call it before enablement has propagated, so manage APIs with `google_project_service`, make resources `depends_on` it, and retry after a minute or add a short `time_sleep`. Project factories usually enable all required APIs in an earlier stack.

KEY POINTS

- Set `disable_on_destroy = false` so removing the code does not turn off APIs in use.
- Enable APIs in the project factory stage, before workload stacks run.

CONFIGURATION EXAMPLE

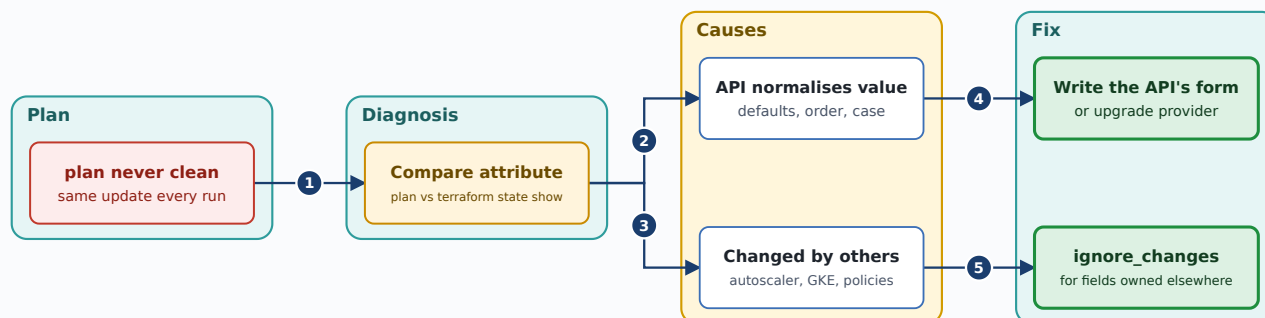
```
gcloud services list --enabled --project=prod-app | grep run
gcloud services enable run.googleapis.com --project=prod-app
resource "google_project_service" "run" {
  project      = "prod-app"
  service      = "run.googleapis.com"
  disable_on_destroy = false
}
```

Q266

Troubleshooting

Every terraform plan shows the same ~ update in-place on a GKE node pool and a bucket even right after apply. How do you stop the perpetual diff?

FLOW DIAGRAM



- 1 Which attribute flips back and forth?
- 2 API returns a different but equal value
- 3 Something outside Terraform updates it
- 4 Match the stored value exactly
- 5 Let the other owner manage that field

EXPLANATION

Find the exact attribute in the plan and compare it with `terraform state show`. If Google Cloud returns the value in another form (a default filled in, a list in a different order, lower case, a full URL instead of a short name), write the value the way the API returns it, or upgrade the provider since many such diffs are fixed as bugs. If another system owns the field, for example the cluster autoscaler changing `node_count`, GKE adding labels, or an org policy adding settings, add that attribute to `lifecycle { ignore_changes = [...] }`. Do not ignore whole resources: keep the list narrow so real drift is still visible.

KEY POINTS

- For autoscaled node pools set `initial_node_count` and ignore `node_count`.
- A daily plan in CI should end with no changes; treat noise as a bug.

CONFIGURATION EXAMPLE

```

terraform plan -no-color | grep -A3 '~ '
terraform state show google_container_node_pool.apps | grep node_count
resource "google_container_node_pool" "apps" {
  # ...
  initial_node_count = 3
  autoscaling {
    min_node_count = 3
    max_node_count = 20
  }
  lifecycle { ignore_changes = [node_count, node_config[0].resource_labels] }
}

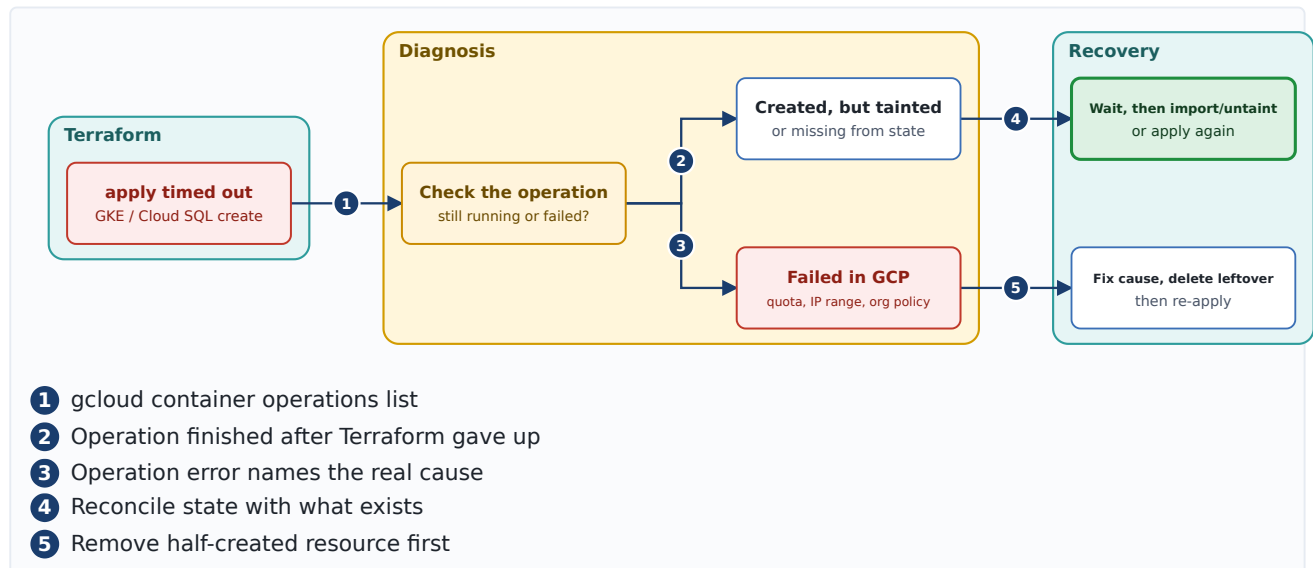
```


Q267

Troubleshooting

`terraform apply` creating a GKE cluster fails after 40 minutes with Error waiting for creating GKE cluster: timeout while waiting for state to become 'DONE' . The next plan wants to create it again. What do you do?

FLOW DIAGRAM



EXPLANATION

A Terraform timeout does not cancel the Google Cloud operation, so check it with `gcloud container operations list` (or `gcloud sql operations list`). If it is still running, wait; if it finished successfully, the resource may be in state as tainted (plan shows replace) or missing (plan shows create, and apply would get 409). Run `terraform untaint` if the resource is healthy, or import it if it is missing. If the operation failed, its error gives the real cause, such as exhausted quota, an overlapping secondary range or an org policy, so fix that, delete the half-created resource and apply again; raise the `timeouts` block only if the operation is simply slow.

KEY POINTS

- Read the Google operation error, not only the Terraform timeout message.
- Set realistic `timeouts { create = "60m" }` for slow resources.

CONFIGURATION EXAMPLE

```

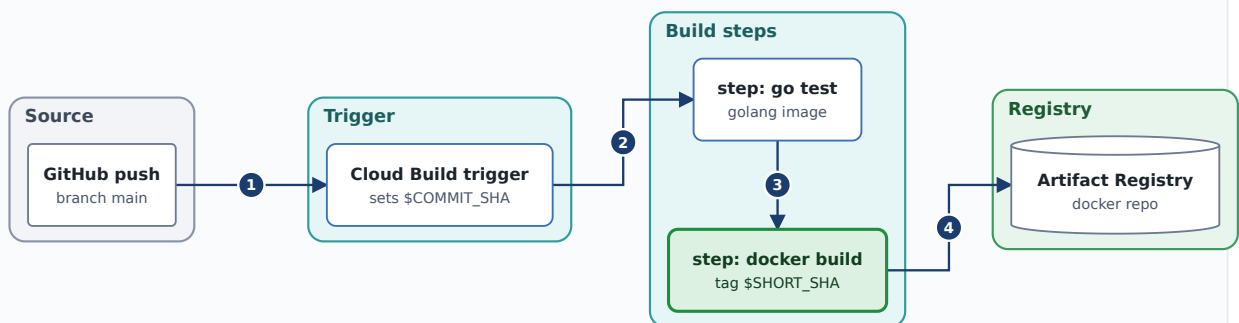
gcloud container operations list --region=us-central1 --filter='status!=DONE'
gcloud container operations describe operation-1727-abc --region=us-central1
terraform state list | grep google_container_cluster
terraform untaint google_container_cluster.prod      # if created and healthy
terraform import google_container_cluster.prod projects/prod/locations/us-central1/clusters/prod-gke
  
```

10. CI/CD: Cloud Build, Artifact Registry, Cloud Deploy & Pipelines

Building, storing, securing and releasing software on Google Cloud, from the first commit to a safe production rollout.

Q268 Walk through a `cloudbuild.yaml` that tests, builds and pushes a container image. How do steps, substitutions and images work?

FLOW DIAGRAM



- 1 Webhook event starts a build
- 2 Each step runs in its own container, `/workspace` shared
- 3 Steps run in order unless `waitFor` says otherwise
- 4 images: pushes and records the digest

EXPLANATION

A build config is a list of `steps`; each step runs a container image (`name`) with `args` , and all steps share the `/workspace` volume so outputs from one step are visible to the next. Built-in substitutions such as `$PROJECT_ID` , `$COMMIT_SHA` and `$SHORT_SHA` are filled in by the trigger, and user substitutions start with an underscore, like `$_REGION` . Listing the image under `images` makes Cloud Build push it at the end and record its digest in the build results, which later steps and provenance use. Steps run sequentially by default; `waitFor: ['-']` lets steps start in parallel.

KEY POINTS

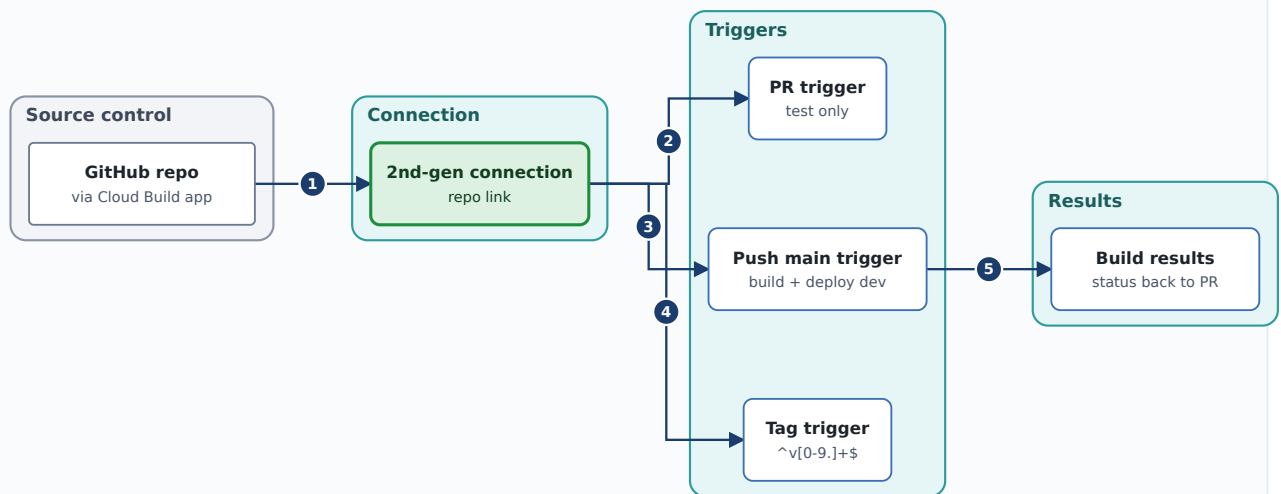
- Tag images with the commit SHA, never only `latest` .
- Set `options.logging` and a `timeout` explicitly.

CONFIGURATION EXAMPLE

```
steps:
- name: golang:1.22
  entrypoint: go
  args: ['test', './...']
- name: gcr.io/cloud-builders/docker
  args: ['build', '-t', '$_REGION-docker.pkg.dev/$PROJECT_ID/apps/api:$SHORT_SHA', '.']
images: ['$_REGION-docker.pkg.dev/$PROJECT_ID/apps/api:$SHORT_SHA']
substitutions: { _REGION: europe-west1 }
options: { logging: CLOUD_LOGGING_ONLY }
```

Q269 You need separate triggers for pull requests, merges to main and release tags on a GitHub repo. How do you set them up in Cloud Build?

FLOW DIAGRAM



- ❶ GitHub App installed; token kept in Secret Manager
- ❷ Pull request event, cloudbuild-pr.yaml
- ❸ Push to branch ^main\$
- ❹ Push new tag, release build
- ❺ Check runs reported to GitHub

EXPLANATION

Connect the repository with a Cloud Build repository connection (2nd gen), which uses the GitHub App or a GitLab token stored in Secret Manager. Create one trigger per event: a pull-request trigger that runs tests and can require a `/gcbrun` comment from a collaborator for forks, a push trigger on `^main$` that builds and deploys to dev, and a tag trigger on `^v[0-9]+\.[0-9]+\.[0-9]+$` for releases. Each trigger can point at a different config file and run as its own user-managed service account with only the permissions it needs. Build status is reported back to the PR as a check.

KEY POINTS

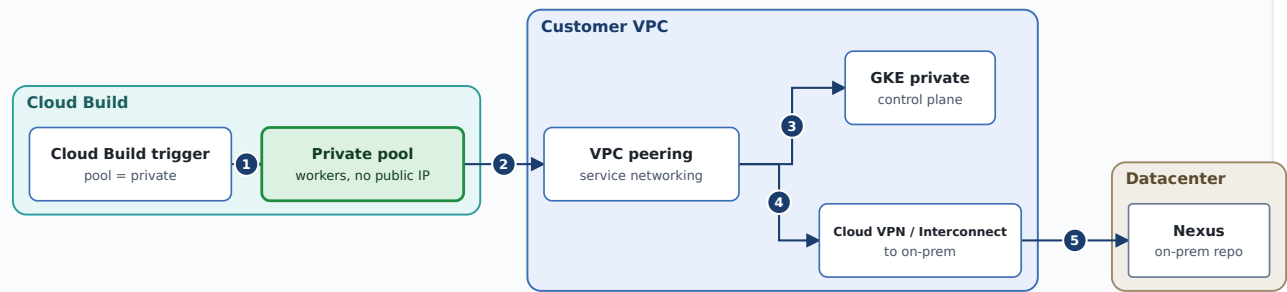
- Use a separate service account per trigger.
- Protect PR triggers from untrusted forks with comment control.

CONFIGURATION EXAMPLE

```
gcloud builds triggers create github --name=api-release \
  --repository=projects/acme-ci/locations/europe-west1/connections/gh/repositories/api \
  --tag-pattern='^v[0-9]+\.[0-9]+\.[0-9]+$' \
  --build-config=cloudbuild-release.yaml --region=europe-west1 \
  --service-account=projects/acme-ci/serviceAccounts/cb-release@acme-ci.iam.gserviceaccount.com
```

Q270 Builds must reach a private GKE control plane and an on-prem Nexus with no public IPs. How do you run Cloud Build in that network?

FLOW DIAGRAM



- 1 Build runs on the private worker pool
- 2 Pool peered to your VPC network
- 3 kubectl reaches the private endpoint
- 4 Routes exported to on-prem
- 5 Pull dependencies over private link

EXPLANATION

Create a Cloud Build private pool peered to your VPC through Private Services Access, set `noExternalIp` if builds must not reach the internet, and reference the pool in `options.pool.name`. Builds then run on dedicated workers with private IPs inside the peered network, so they can reach private GKE endpoints, internal load balancers and on-prem systems over VPN or Interconnect (with custom route export on the peering). Private pools also let you choose larger machine types and use VPC Service Controls. For a private GKE control plane, make sure the pool range is in the cluster's authorized networks.

KEY POINTS

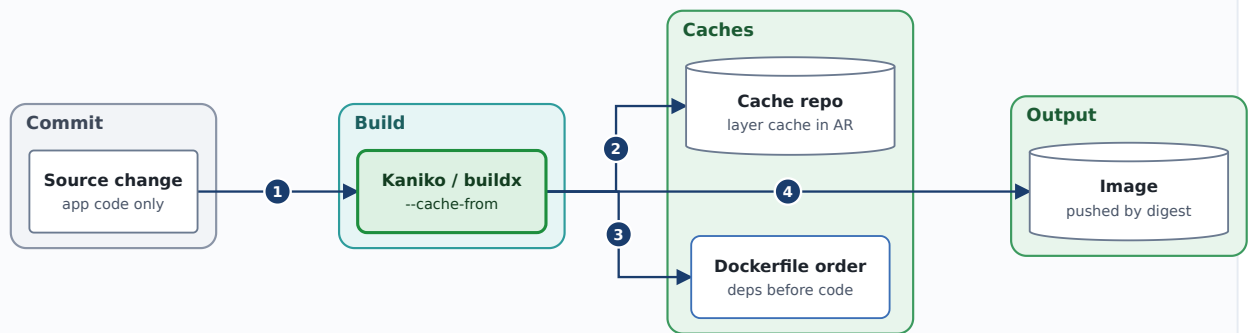
- Default pools run on Google-managed shared workers with internet access.
- Peering is not transitive: export custom routes where needed.

CONFIGURATION EXAMPLE

```
options:
  pool:
    name: projects/acme-ci/locations/europe-west1/workerPools/private-pool
steps:
- name: gcr.io/cloud-builders/kubectl
  args: ['rollout', 'status', 'deploy/api']
  env: ['CLOUDSDK_CONTAINER_CLUSTER=prod', 'CLOUDSDK_COMPUTE_REGION=europe-west1']
```

Q271 Docker builds in Cloud Build take 15 minutes because every layer is rebuilt. How do you speed them up?

FLOW DIAGRAM



- ❶ Build starts on a fresh worker
- ❷ Reuse unchanged layers from the registry
- ❸ Only layers after COPY are rebuilt
- ❹ Build in minutes instead of 15

EXPLANATION

Cloud Build workers start empty, so cache must come from outside: use Kaniko with `--cache=true` and a cache repo in Artifact Registry, or `docker buildx` with `--cache-from/--cache-to type=registry`. Order the Dockerfile so dependency manifests are copied and installed before the application source, so a code change only invalidates the last layers. Use multi-stage builds, a `.dockerignore`, and for language caches (Maven, npm, Go) restore and save them to a GCS bucket in extra steps. Larger machine types (`options.machineType: E2_HIGHCPU_8`) also help CPU-heavy builds.

KEY POINTS

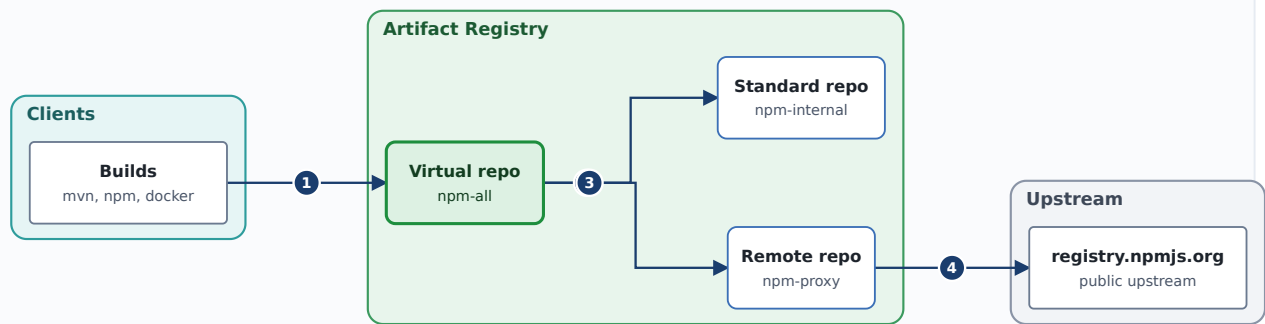
- Cache lives in the registry or GCS, not on the worker.
- Layer order matters more than any tool.

CONFIGURATION EXAMPLE

```
- name: gcr.io/kaniko-project/executor:latest
  args:
    - --destination=europe-west1-docker.pkg.dev/$PROJECT_ID/apps/api:$SHORT_SHA
    - --cache=true
    - --cache-repo=europe-west1-docker.pkg.dev/$PROJECT_ID/cache/api
    - --cache-ttl=168h
```

Q272 How do you organise Artifact Registry for Docker images and Maven/npm packages, including caching public dependencies?

FLOW DIAGRAM



- 1 One URL for all packages
- 2 Internal packages first (priority)
- 3 Then the proxy for public packages
- 4 Fetched once, then cached

EXPLANATION

Create standard repositories per format and purpose (for example `docker-apps`, `maven-internal`, `npm-internal`), in the regions where you build and run. Add remote repositories that proxy Docker Hub, Maven Central or npmjs so builds cache upstream packages, which protects you from rate limits and upstream outages. A virtual repository combines standard and remote repos behind one URL with an upstream priority order, which also reduces dependency-confusion risk because internal packages are resolved first. Grant `roles/artifactregistry.reader` and `writer` per repo, not per project.

KEY POINTS

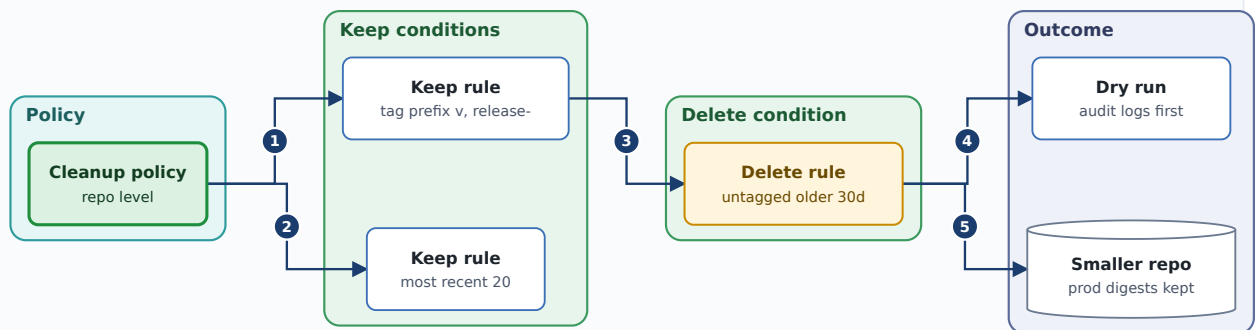
- Remote repos cache public dependencies and cut rate-limit failures.
- Virtual repos give one endpoint with controlled priority.

CONFIGURATION EXAMPLE

```
gcloud artifacts repositories create npm-proxy --repository-format=npm \
  --location=europe-west1 --mode=remote-repository \
  --remote-npm-repo=NPMJS
gcloud artifacts repositories create npm-all --repository-format=npm \
  --location=europe-west1 --mode=virtual-repository \
  --upstream-policy-file=upstreams.json
```

Q273 Your Docker repo has 40,000 images and storage costs keep rising. How do you clean it up without deleting images still running in prod?

FLOW DIAGRAM



- 1 Keep policies always win over delete
- 2 Keep recent versions per package
- 4 Review what would be deleted
- 5 Enforce after checking

EXPLANATION

Use Artifact Registry cleanup policies on the repository: delete rules (for example untagged images older than 30 days) plus keep rules (tags with prefix `v` or `release-`, and the most recent N versions per package). Keep rules take precedence over delete rules, so tagged releases survive. Start with dry-run mode and check the logs of what would be deleted, then enforce. Make sure what prod runs is protected: deploy by immutable release tags that match a keep rule, or keep the N most recent versions, and turn on tag immutability for release repos.

KEY POINTS

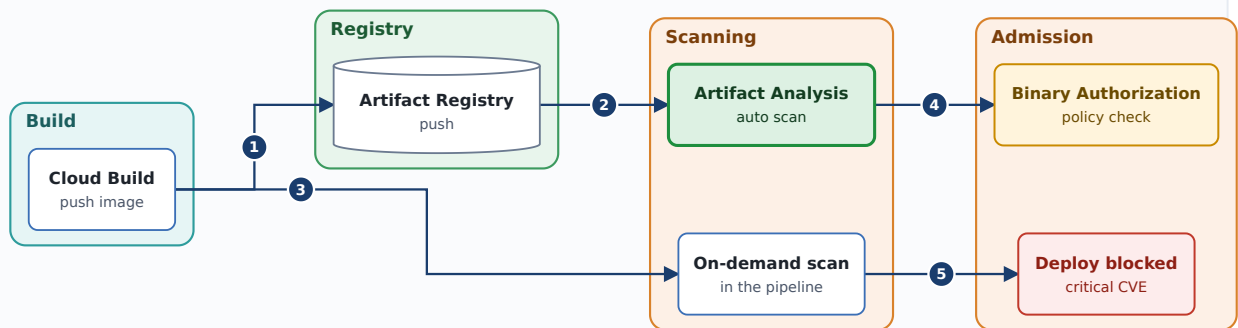
- Dry-run first, then enforce.
- Deploy by digest but keep a release tag so keep rules can match.

CONFIGURATION EXAMPLE

```
[{"name": "keep-releases", "action": {"type": "Keep"}, "condition": {"tagState": "tagged", "tagPrefixes": ["v", "release-"]}}, {"name": "keep-recent", "action": {"type": "Keep"}, "mostRecentVersions": {"keepCount": 20}}, {"name": "delete-old-untagged", "action": {"type": "Delete"}, "condition": {"tagState": "untagged", "olderThan": "30d"}}]
```

Q274 How do you find and block container images with critical vulnerabilities before and after they are deployed?

FLOW DIAGRAM



- 1 Image pushed
- 2 Scanned on push; rescanned as new CVEs appear
- 3 gcloud artifacts docker images scan before push
- 4 Findings feed attestation / policy
- 5 Fail the build on CRITICAL

EXPLANATION

Enable the Container Scanning API so Artifact Analysis scans images on push and keeps updating findings as new vulnerabilities are published. For a gate, run an on-demand scan in the pipeline (`gcloud artifacts docker images scan then list-vulnerabilities`) and fail the build above a severity threshold, or only create an attestation when the image passes. Binary Authorization on GKE or Cloud Run then admits only images with that attestation. For images already running, use GKE security posture or findings in Security Command Center to see affected workloads and rebuild on a patched base image.

KEY POINTS

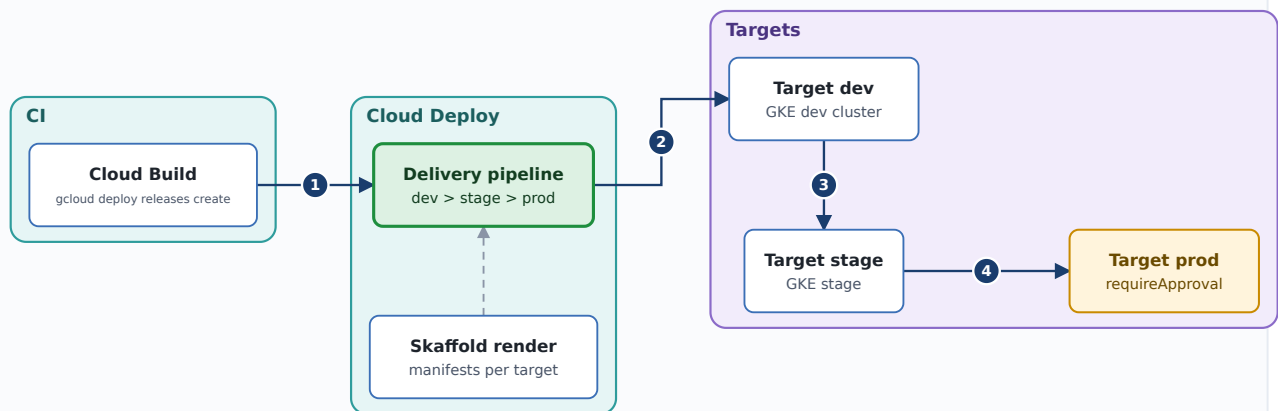
- Gate in the pipeline, enforce at admission.
- Rebuild regularly: base image CVEs appear after release.

CONFIGURATION EXAMPLE

```
gcloud artifacts docker images scan \
  europe-west1-docker.pkg.dev/acme/apps/api@sha256:3b1f... \
  --remote --format='value(response.scan)' > scan_id
gcloud artifacts docker images list-vulnerabilities $(cat scan_id) \
  --format='value(vulnerability.effectiveSeverity)' | grep -q CRITICAL && exit 1
```


Q275 Explain how Cloud Deploy models a release from dev to prod on GKE: delivery pipeline, targets, releases and rollouts.

FLOW DIAGRAM



- 1 Release = image digests + rendered manifests
- 2 First rollout goes to dev automatically
- 3 Promote the same release
- 4 Approval required before rollout
- Scaffold profiles render each target

EXPLANATION

A delivery pipeline defines the ordered stages, and each stage points to a target, which is a GKE cluster, Cloud Run location, GKE Enterprise fleet or custom target. When CI creates a release, Cloud Deploy uses Scaffold to render the manifests for every target once, pinning image digests, so the same artifact moves through all stages. A rollout is the deployment of that release to one target; you promote the release to the next stage manually or with automation, and targets can require approval. Rollouts run as the execution service account configured per target.

KEY POINTS

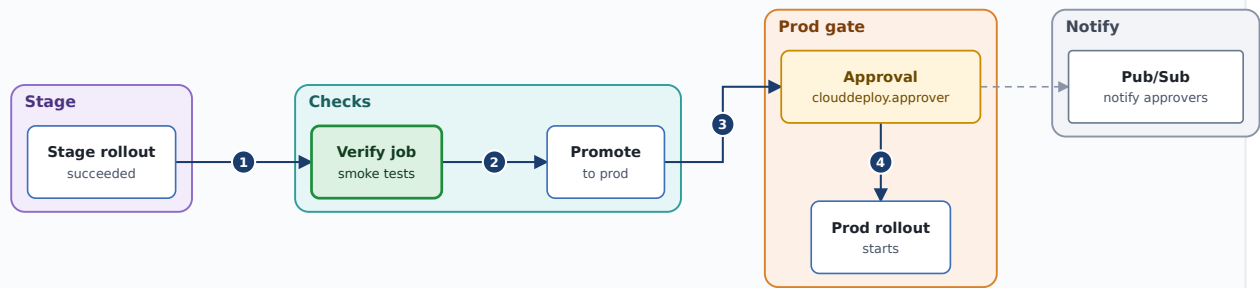
- Rendered manifests are frozen at release time.
- Separate execution SAs per target limit blast radius.

CONFIGURATION EXAMPLE

```
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata: { name: api }
serialPipeline:
  stages:
    - { targetId: dev }
    - { targetId: stage, profiles: [stage] }
    - { targetId: prod, profiles: [prod], strategy: { standard: { verify: true } } }
```

Q276 Production releases need a manager's approval and must be promoted only after stage passes checks. How do you implement this in Cloud Deploy?

FLOW DIAGRAM



- 1 Scaffold verify container runs tests
- 2 Automation or manual promote
- 3 Target has requireApproval: true
- 4 Approved by a separate person
- Approval request published to Pub/Sub

EXPLANATION

Set `requireApproval: true` on the prod target and give only release managers `roles/clouddeploy.approver`, so a rollout to prod waits in a pending approval state. Add a verify phase to the stage (and prod) strategy, which runs a test container defined in `scaffold.yaml` after deploy, and the rollout fails if tests fail. Promotion can be manual (`gcloud deploy releases promote`) or automated with a Cloud Deploy automation rule after a successful rollout. Cloud Deploy publishes approval and rollout notifications to Pub/Sub, which you can route to chat or email.

KEY POINTS

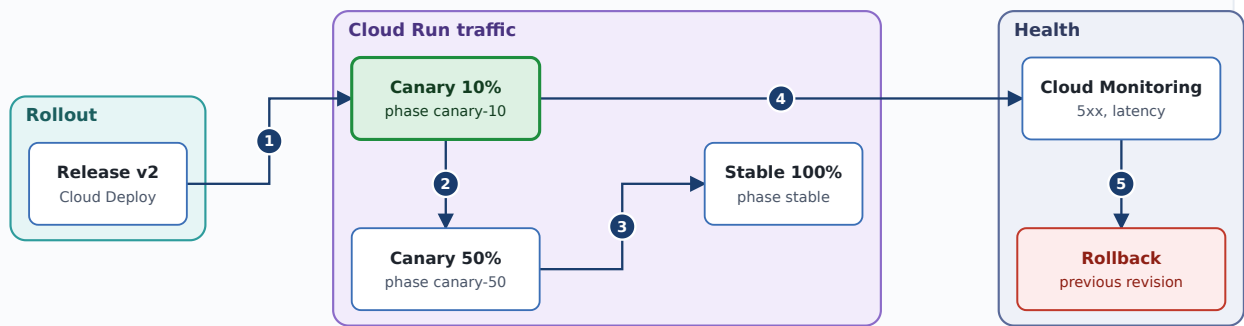
- Approver role separate from the release creator.
- Verify turns a deploy into a tested deploy.

CONFIGURATION EXAMPLE

```
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata: { name: prod }
requireApproval: true
gke:
  cluster: projects/acme-prod/locations/europe-west1/clusters/prod
executionConfigs:
- usages: [RENDER, DEPLOY, VERIFY]
  serviceAccount: deploy-prod@acme-prod.iam.gserviceaccount.com
```

Q277 Roll a new version of a Cloud Run service out to 10% then 50% of traffic with Cloud Deploy, and roll back if errors rise. How?

FLOW DIAGRAM



- ❶ Traffic split to the new revision
- ❷ Advance after checks
- ❸ All traffic to v2
- ❹ Watch error rate per revision
- ❺ gcloud deploy targets rollback

EXPLANATION

Use the canary strategy in the delivery pipeline stage with percentages: [10, 50] ; Cloud Deploy creates phases that shift Cloud Run traffic to the new revision, and a final stable phase moves to 100%. Between phases you advance the rollout manually or by automation, and you can add verify jobs or watch Cloud Monitoring on per-revision request metrics. If errors rise, `gcloud deploy targets rollback` creates a rollout of the previous successful release, and on Cloud Run the old revision is still there so traffic moves back fast. The same strategy works on GKE using Gateway API or service-based traffic splitting.

KEY POINTS

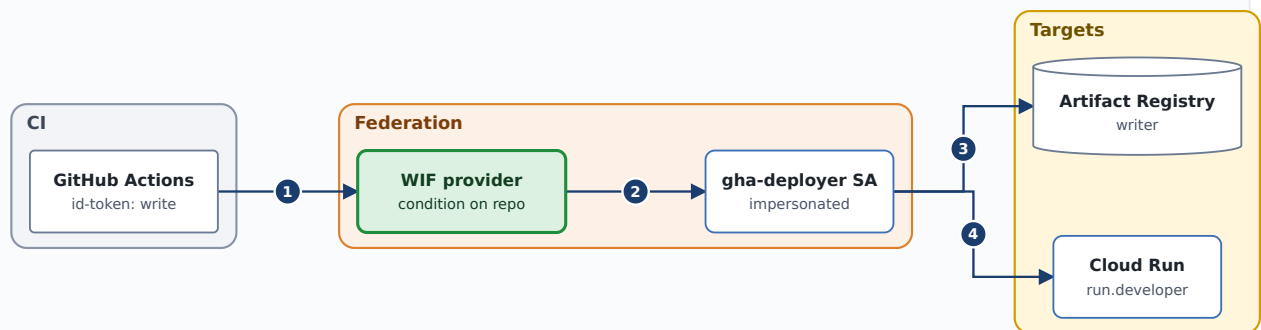
- Canary phases are driven by the percentages list.
- Rollback is a new rollout of the last good release.

CONFIGURATION EXAMPLE

```
serialPipeline:
  stages:
    - targetId: prod-run
      profiles: [prod]
      strategy:
        canary:
          runtimeConfig: { cloudRun: { automaticTrafficControl: true } }
          canaryDeployment: { percentages: [10, 50], verify: true }
```

Q278 GitHub Actions needs to build an image, push to Artifact Registry and deploy to Cloud Run with no JSON keys. How do you set it up?

FLOW DIAGRAM



- 1 OIDC token exchanged at STS
- 2 attribute.repository == acme/api
- 3 docker push with short-lived token
- 4 gcloud run deploy --image digest

EXPLANATION

Create a workload identity pool with a GitHub OIDC provider and an attribute condition such as `assertion.repository_owner == 'acme'`, then allow the specific repository principal to impersonate a deployer service account with `roles/iam.workloadIdentityUser`. In the workflow grant `permissions: id-token: write`, use `google-github-actions/auth` with the provider and service account, then `gcloud auth configure-docker` and push. The service account has only `artifactregistry.writer` on the repo, `run.developer` on the service and `iam.serviceAccountUser` on the Cloud Run runtime SA. Use GitHub environments with required reviewers for prod jobs.

KEY POINTS

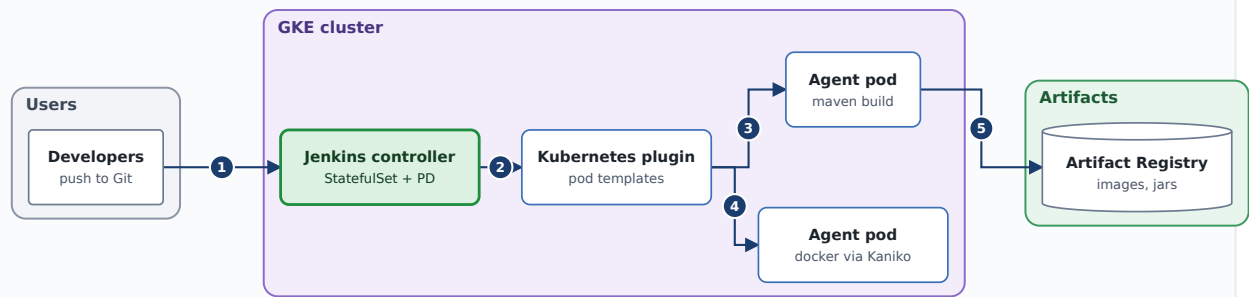
- No keys to rotate or leak.
- Scope by repository and, for prod, by environment claim.

CONFIGURATION EXAMPLE

```
permissions: { contents: read, id-token: write }
steps:
- uses: actions/checkout@v4
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: ${vars.WIF_PROVIDER}
    service_account: gha-deployer@acme-prod.iam.gserviceaccount.com
- run: gcloud auth configure-docker europe-west1-docker.pkg.dev --quiet
- run: docker build -t $IMG:${github.sha} . && docker push $IMG:${github.sha}
```

Q279 Your company still runs Jenkins. How would you host it on GKE so build agents scale on demand?

FLOW DIAGRAM



- ❶ Webhook starts a job
- ❷ Plugin asks the API server for a pod
- ❸ Ephemeral agent per build
- ❹ Deleted when the build ends
- ❺ Publish artifacts with Workload Identity

EXPLANATION

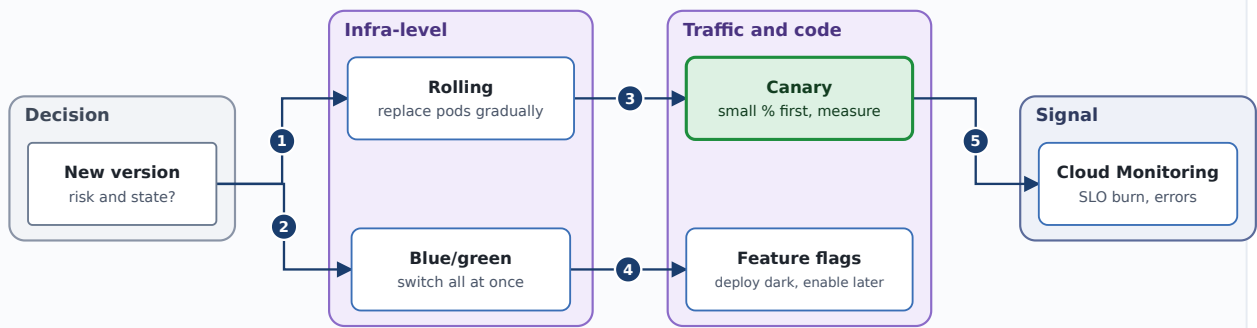
Run the Jenkins controller as a StatefulSet (the official Helm chart) with a persistent disk for `JENKINS_HOME`, behind an internal load balancer or IAP. The Kubernetes plugin starts an ephemeral agent pod per build from pod templates, and the cluster autoscaler or Autopilot adds nodes when many builds queue, so you pay for agents only while they run. Agents use Workload Identity to push to Artifact Registry without keys, and image builds use Kaniko or BuildKit instead of mounting the Docker socket. Back up `JENKINS_HOME`, keep plugins pinned and manage configuration with Configuration as Code (JCasC).

KEY POINTS

- Controller is stateful; agents are disposable.
- Avoid Docker-in-Docker with privileged pods.

Q280 How do you choose between rolling, blue/green, canary and feature-flag releases for a service?

FLOW DIAGRAM



- 1 Default for stateless, low risk
- 2 Instant switch and instant rollback
- 3 Risky change: limit exposure
- 4 Separate deploy from release
- 5 Compare canary vs stable metrics

EXPLANATION

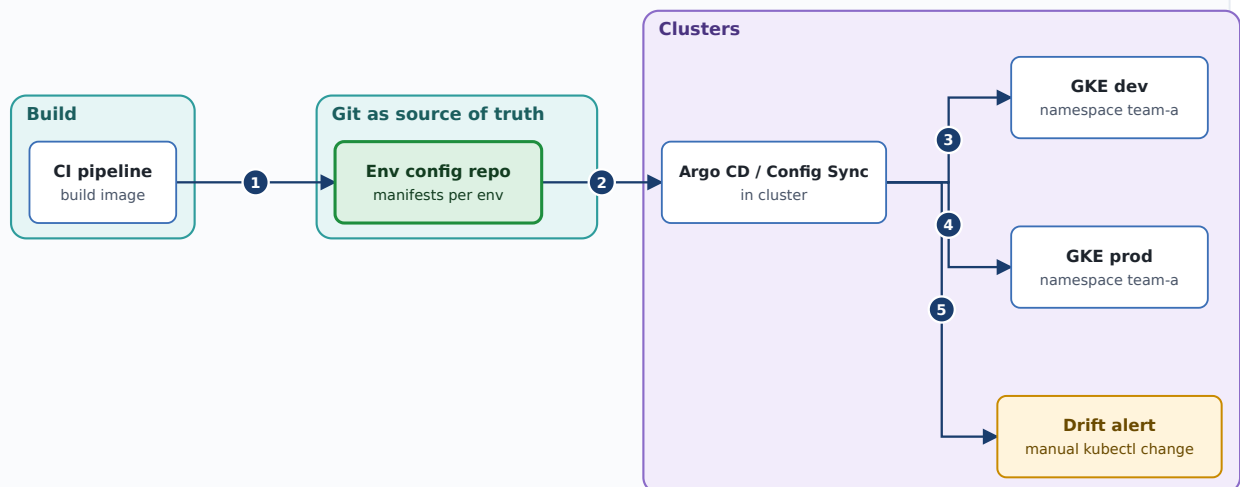
Rolling updates are the default for stateless services on GKE or MIGs: cheap and simple, but old and new versions run side by side and rollback is another rollout. Blue/green runs a full second environment and switches traffic at once, giving instant rollback at double capacity cost, useful for big changes or when mixed versions are unsafe. Canary sends a small percentage of traffic to the new version and compares error rate and latency before continuing, which is best for high-traffic, high-risk services with good metrics. Feature flags separate deploy from release, letting you turn features on per user or percentage and switch off without redeploying.

KEY POINTS

- Every strategy needs backward-compatible data changes.
- Canary needs enough traffic and good metrics to mean anything.

Q281 Several teams deploy to shared GKE clusters. How would you implement GitOps with Argo CD or Config Sync?

FLOW DIAGRAM



- ❶ PR updates the image tag/digest
- ❷ Controller pulls the desired state
- ❸ Sync to dev automatically
- ❹ Prod syncs after PR merge to prod folder
- ❺ Out-of-band changes reverted or flagged

EXPLANATION

Keep the desired state for every environment in a Git config repo (Helm values or Kustomize overlays), separate from app source. CI builds the image and opens a PR that bumps the digest in the dev overlay; merging it is the deployment, and promotion to prod is another reviewed PR. Argo CD or Config Sync runs in the cluster, pulls the repo and applies it, reports sync and health status, and reverts manual `kubectl` changes. For many teams use Argo CD Projects or Config Sync RepoSync per namespace so each team can only manage its own namespaces, and keep cluster-wide objects in a platform repo.

KEY POINTS

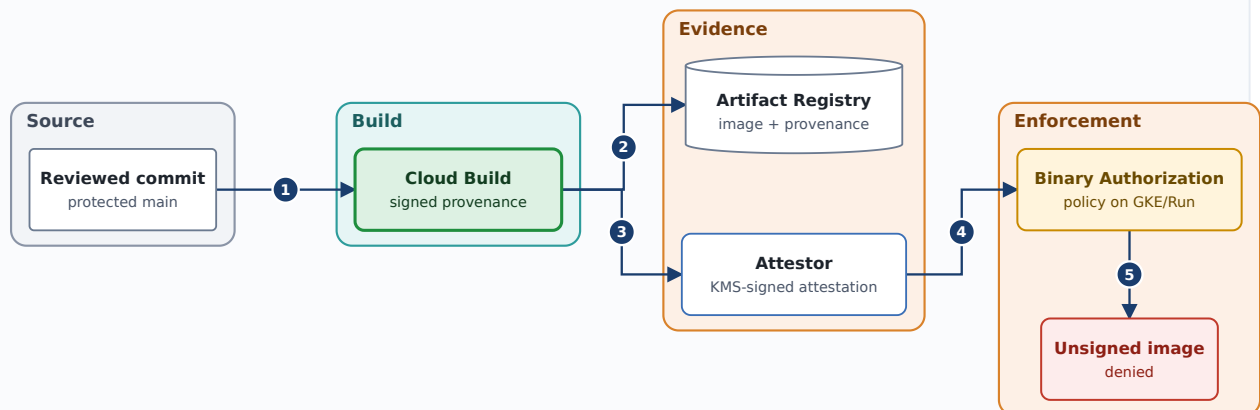
- Pull-based: the cluster needs no inbound CI credentials.
- Git history is the audit trail and rollback is git revert.

CONFIGURATION EXAMPLE

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: team-a-api-prod, namespace: argocd }
spec:
  project: team-a
  source: { repoURL: https://github.com/acme/env-config, path: team-a/api/overlays/prod, targetRevision:
main }
  destination: { server: https://kubernetes.default.svc, namespace: team-a }
  syncPolicy: { automated: { prune: true, selfHeal: true } }
```

Q282 An auditor asks how you prove that the image running in prod was built by your pipeline from reviewed code. How do you use SLSA provenance and Binary Authorization?

FLOW DIAGRAM



- ❶ Trigger builds the exact commit
- ❷ Provenance: source, builder, digest
- ❸ Attestation created after checks pass
- ❹ Admission checks attestation for the digest
- ❺ Other images are blocked and logged

EXPLANATION

Cloud Build generates signed build provenance for images listed in `images`, recording the source repo and commit, the build config and the output digest, which can meet SLSA build level 3 requirements for trusted builders. Store it with the image in Artifact Registry and view it with `gcloud artifacts docker images describe --show-provenance`. After tests and vulnerability checks, the pipeline creates a Binary Authorization attestation signed with a Cloud KMS key, and the GKE or Cloud Run policy requires that attestation, so only images built and checked by your pipeline can run. Use dry-run mode first, and keep a break-glass procedure that is logged.

KEY POINTS

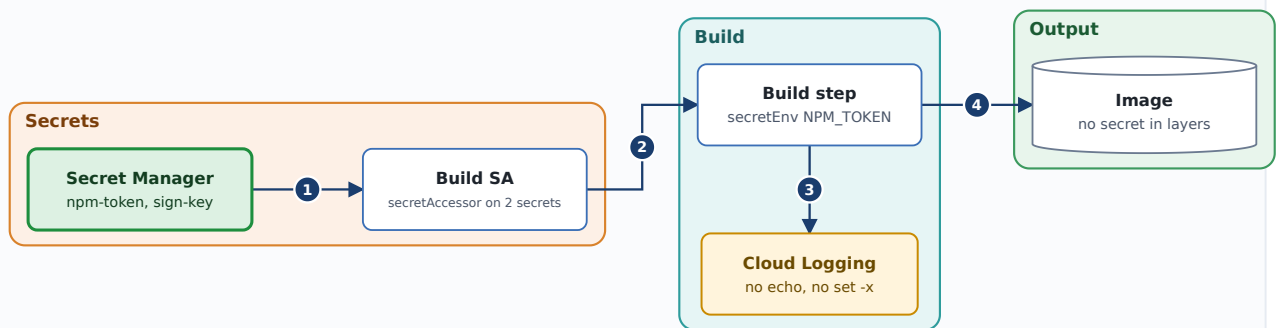
- Provenance says how it was built; attestation says it passed your checks.
- Deploy by digest so the attestation matches.

CONFIGURATION EXAMPLE

```
gcloud container binauthz attestations sign-and-create \
  --artifact-url=europe-west1-docker.pkg.dev/acme/apps/api@sha256:3b1f... \
  --attestor=built-by-ci --attestor-project=acme-sec \
  --keyversion=projects/acme-sec/locations/global/keyRings/binauthz/cryptoKeys/ci/cryptoKeyVersions/1
```


Q283 Build steps need an npm token and a signing key. How do you pass secrets into Cloud Build without leaking them in logs or the repo?

FLOW DIAGRAM



- 1 IAM on each secret, not project-wide
- 2 availableSecrets maps versions to env vars
- 3 Script must not print the value
- 4 Use build secrets, not ARG, in Dockerfile

EXPLANATION

Store the values in Secret Manager and grant the build's service account

`roles/secretmanager.secretAccessor` only on those secrets. In `cloudbuild.yaml` declare them under `availableSecrets.secretManager` and reference them in a step with `secretEnv`, using `$$VAR` so Cloud Build does not treat them as substitutions. Do not pass secrets as Docker `ARG`s because they remain in image history; use BuildKit `--secret` mounts instead. Avoid `set -x` and echoing, and prefer keyless options where possible, such as Artifact Registry for npm with the build's identity or KMS signing without exporting the key.

KEY POINTS

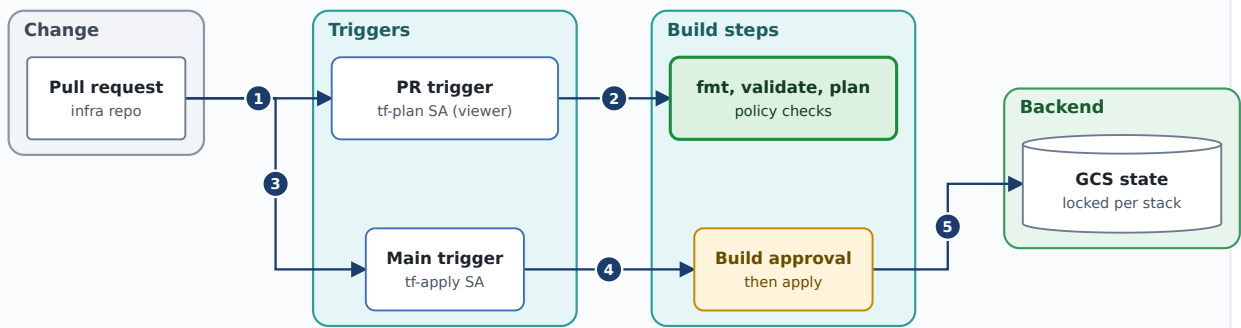
- Secrets pulled at run time, never in the repo or image.
- `$$` escapes the variable for the shell.

CONFIGURATION EXAMPLE

```
steps:
- name: node:20
  entrypoint: bash
  args: ['-c', 'echo "///registry.npmjs.org/:_authToken=$$NPM_TOKEN" > .npmrc && npm ci']
  secretEnv: ['NPM_TOKEN']
availableSecrets:
  secretManager:
  - versionName: projects/$PROJECT_ID/secrets/npm-token/versions/latest
    env: NPM_TOKEN
```

Q284 Design a Cloud Build pipeline for Terraform that plans on PRs and applies on merge to main.

FLOW DIAGRAM



- ❶ Runs on every PR update
- ❷ Plan posted back to the PR
- ❸ Merge to main
- ❹ Trigger requires approval
- ❺ apply with state lock

EXPLANATION

Create two triggers: a PR trigger that runs `terraform fmt -check`, `validate`, `tflint/policy checks` and `plan`, using a service account with read-only roles, and a push-to-main trigger that runs `apply` with a separate service account that can change resources. Mark the `apply` trigger as requiring approval so a person approves the build in the console before it runs. Use the `hashicorp/terraform` image pinned to a version, the GCS backend with locking, and one directory per stack so the build only plans and applies what changed. Store the plan output as a build artifact for audit.

KEY POINTS

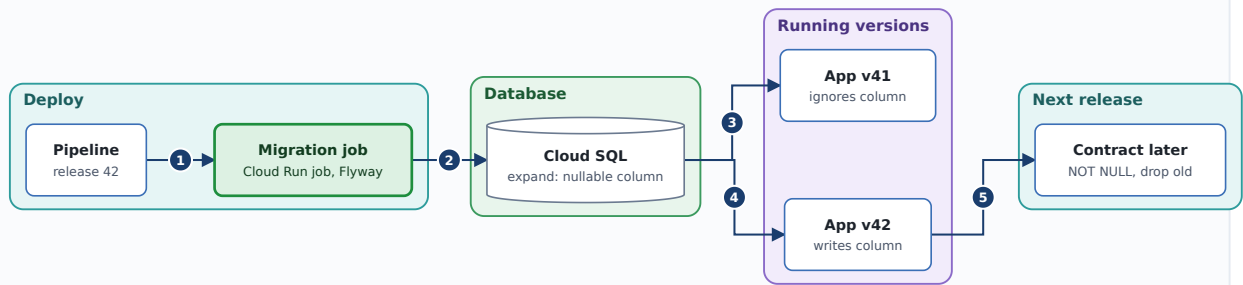
- Read-only identity for plan, write identity for apply.
- Pin the Terraform image version.

CONFIGURATION EXAMPLE

```
steps:
- name: hashicorp/terraform:1.9
  dir: envs/prod/network
  entrypoint: sh
  args: ['-c', 'terraform init -input=false && terraform plan -input=false -out=tfplan && terraform apply -input=false tfplan']
serviceAccount: projects/acme-ci/serviceAccounts/tf-apply@acme-ci.iam.gserviceaccount.com
options: { logging: CLOUD_LOGGING_ONLY }
```

Q285 A release adds a new required column. How do you run database migrations in the pipeline without breaking the old version during rollout?

FLOW DIAGRAM



- 1 Run migrations before the app rollout
- 2 Additive, backward-compatible change
- 3 Old version keeps working
- 4 New version rolls out gradually
- 5 Tighten schema once v41 is gone

EXPLANATION

Use the expand/contract pattern: the first release only adds things (a nullable column, a new table), so both the old and new app versions work against the same schema during a rolling or canary deploy. Run migrations as a separate pipeline step before the rollout, for example a Cloud Run job or Kubernetes Job running Flyway or Liquibase with a versioned, reviewed script, and make it fail the release on error. A later release backfills data and adds the NOT NULL constraint or drops old columns once no old version runs. Avoid long table locks on large tables and take a backup or rely on point-in-time recovery before risky migrations.

KEY POINTS

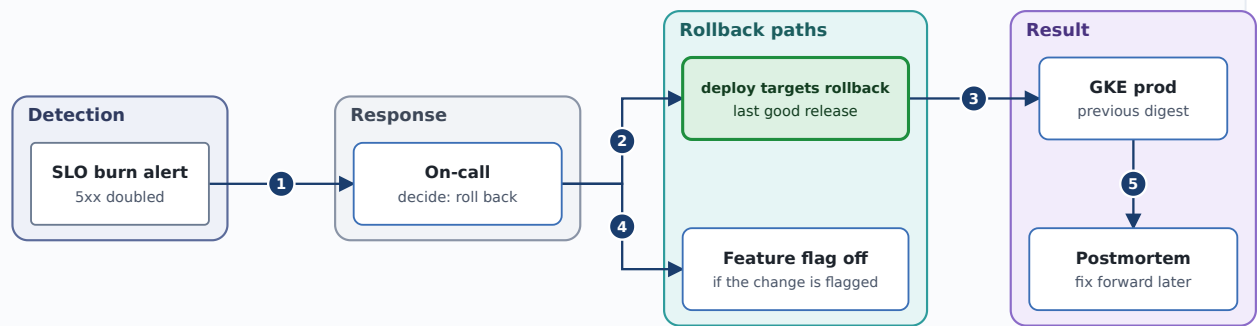
- Every migration must be safe for the previous app version.
- Rollback of code is easy; rollback of data is not.

CONFIGURATION EXAMPLE

```
gcloud run jobs deploy db-migrate --region=europe-west1 \
  --image=europe-west1-docker.pkg.dev/acme/apps/migrations:42 \
  --set-cloudsql-instances=acme-prod:europe-west1:orders \
  --service-account=migrate@acme-prod.iam.gserviceaccount.com
gcloud run jobs execute db-migrate --region=europe-west1 --wait
```

Q286 A release reached prod on GKE and error rates doubled 10 minutes later. What is your rollback procedure, and what makes it fast?

FLOW DIAGRAM



- ❶ Alert links to the latest rollout
- ❷ Roll back first, debug after
- ❸ Re-deploys the previous rendered release
- ❹ Faster than a redeploy when available
- ❺ Root cause and a fix in a new release

EXPLANATION

Roll back first and investigate afterwards: with Cloud Deploy run `gcloud deploy targets rollback prod`, which creates a new rollout of the last successful release with its already rendered manifests and pinned digests. Without Cloud Deploy, `kubectl rollout undo` or a git revert in the GitOps repo does the same. Rollback is fast and safe only if the previous artifact is still in the registry, database changes are backward compatible, and config lives with the release. If the change sits behind a feature flag, switching it off is quicker still. Afterwards fix forward with a new release and record the incident against the change failure rate.

KEY POINTS

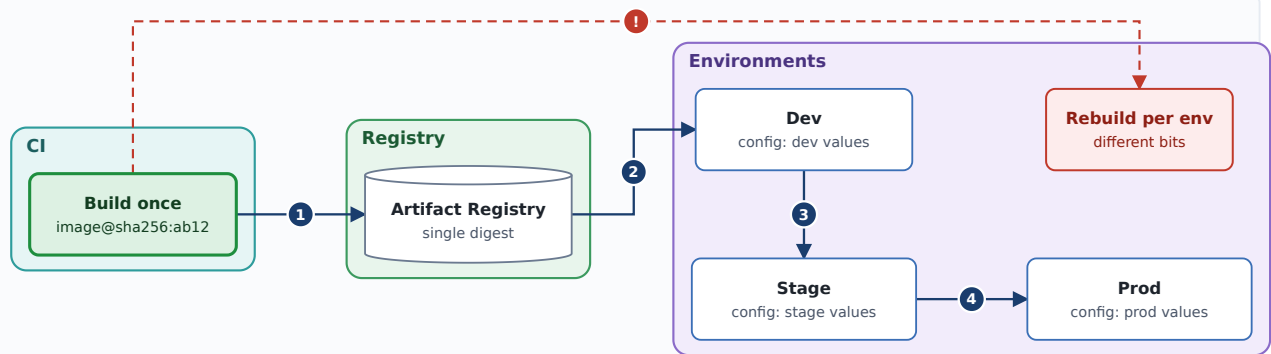
- Decide rollback criteria before the release, not during it.
- Non-reversible migrations make rollback impossible: avoid them.

CONFIGURATION EXAMPLE

```
gcloud deploy targets rollback prod \
  --delivery-pipeline=api --region=europe-west1
# GitOps alternative
git revert <bad-commit> && git push origin main
```

Q287 Teams rebuild the image for each environment, and prod once got a different dependency than stage. How do you promote one artifact from dev to stage to prod?

FLOW DIAGRAM



- 1** Push once, record the digest
- 2** Deploy digest with dev config
- 3** Promote the same digest
- 4** Tested bits are the prod bits
- !** Anti-pattern: rebuilding per env ships untested bits

EXPLANATION

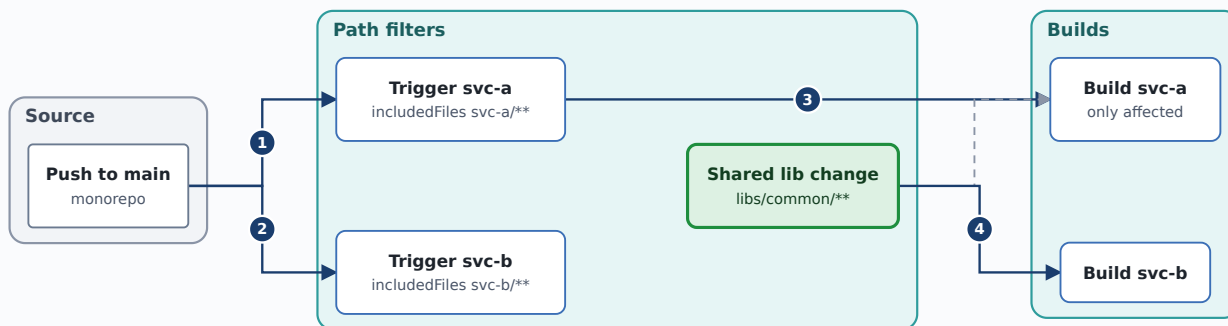
Build the artifact once, push it to Artifact Registry and refer to it by digest from then on; every environment deploys that same digest and only configuration differs. Environment-specific values come from environment variables, ConfigMaps, Kustomize overlays or Scaffold profiles, and secrets come from Secret Manager at run time. Cloud Deploy enforces this by rendering a release with pinned digests and promoting it through targets. If you use separate registries per environment, copy by digest (for example `gcrane cp` or `gcloud artifacts docker tags add`), never rebuild.

KEY POINTS

- Promote digests, not tags or rebuilds.
- Config differs per env; the binary does not.

Q288 A monorepo has 15 services and every commit rebuilds all of them. How do you build and deploy only what changed?

FLOW DIAGRAM



- 1 Only files under svc-a changed
- 2 Skipped when nothing under svc-b changed
- 3 Build and deploy svc-a
- Lib change triggers all dependents
- 4 Dependency graph (Bazel/Nx) selects services

EXPLANATION

Create one trigger per service with `includedFiles` (and `ignoredFiles` for docs) so a commit only starts builds for paths that changed, and include shared library paths in the filters of the services that use them. For larger repos use a build tool that knows the dependency graph, such as Bazel, Nx or Turborepo, to compute affected targets and cache results remotely. Keep each service's build config next to its code and version services independently. Protect against a missed dependency with a nightly full build.

KEY POINTS

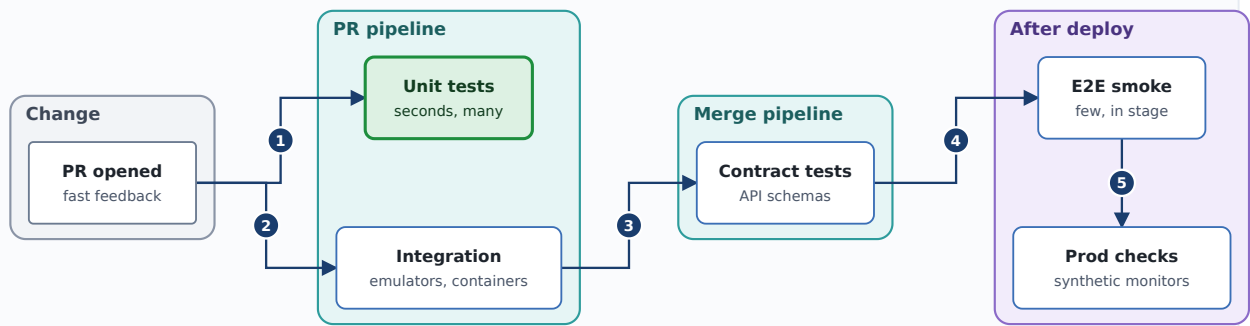
- Path filters are simple; a dependency graph is precise.
- Remote caching makes unchanged targets free.

CONFIGURATION EXAMPLE

```
gcloud builds triggers create github --name=svc-a \
  --repository=projects/acme-ci/locations/europe-west1/connections/gh/repositories/mono \
  --branch-pattern='^main$' --build-config=svc-a/cloudbuild.yaml \
  --included-files='svc-a/**,libs/common/**' --region=europe-west1
```

Q289 Your CI takes 45 minutes because most tests are end-to-end. How do you structure the testing pyramid across the pipeline?

FLOW DIAGRAM



- ❶ Run in parallel, fail fast
- ❷ Pub/Sub, Firestore emulators or Testcontainers
- ❸ Consumers and providers verified
- ❹ Small set of critical user journeys
- ❺ Uptime checks and SLO alerts

EXPLANATION

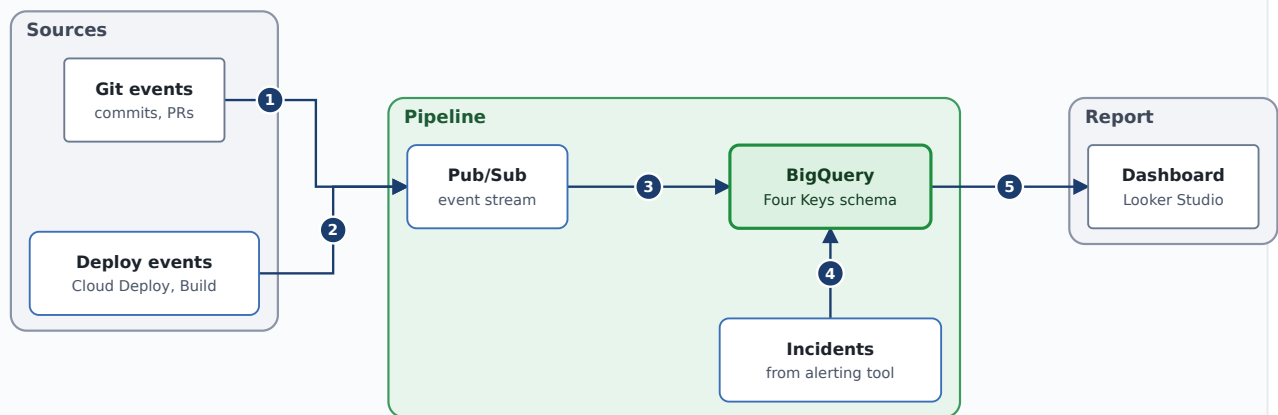
Make most tests fast unit tests that run on every PR in parallel, then a smaller set of integration tests that use emulators (Pub/Sub, Firestore, Spanner) or Testcontainers instead of shared environments. Replace many end-to-end tests with contract tests between services, and keep only a handful of E2E smoke tests for critical journeys run after deploying to stage. In production rely on canary analysis, synthetic checks and SLO alerts. Track flaky tests and quarantine them, because a flaky suite teaches people to ignore red builds.

KEY POINTS

- Aim for PR feedback in under 10 minutes.
- Push checks left, but keep a few real end-to-end ones.

Q290 Leadership asks how good your delivery process is. Which DORA metrics do you track and how do you collect them on GCP?

FLOW DIAGRAM



- ❶ Webhooks with commit timestamps
- ❷ Deployment success or failure
- ❸ Parse and store events
- ❹ Link incidents to deployments
- ❺ Four metrics per service, weekly trend

EXPLANATION

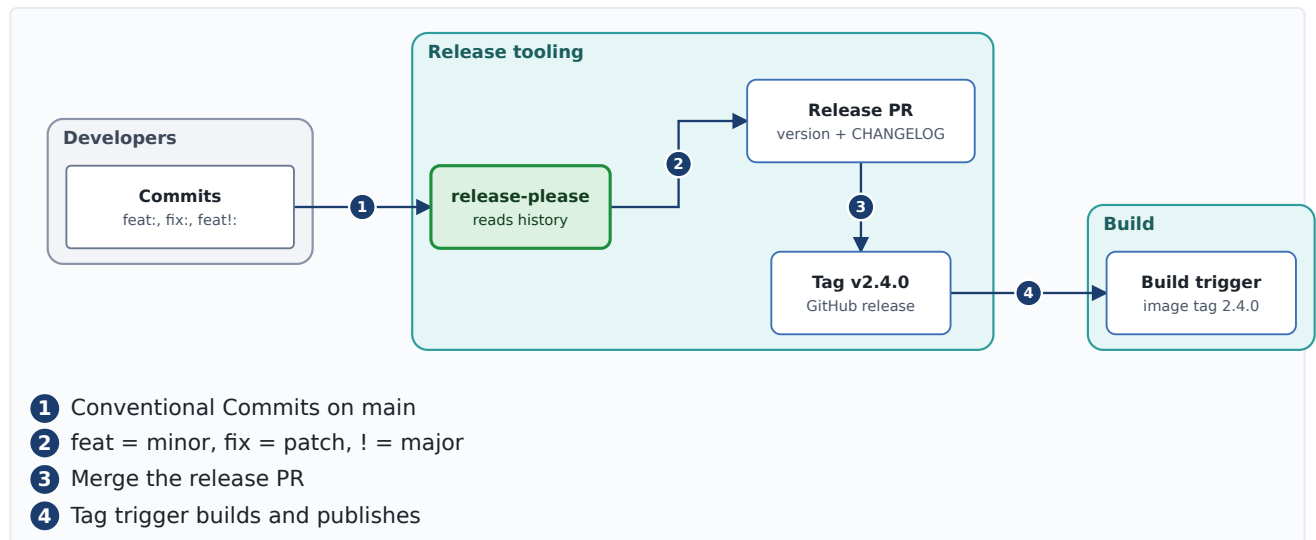
Track the four DORA metrics: deployment frequency, lead time for changes (commit to running in prod), change failure rate (deployments causing incidents or rollbacks) and time to restore service; newer DORA reports also discuss deployment rework rate. Collect events from Git webhooks, Cloud Build and Cloud Deploy notifications and your incident tool into BigQuery, for example with Google's open-source Four Keys approach, and show trends per service. Use them to find bottlenecks such as slow reviews or long approvals, not to rank individuals. Speed and stability metrics should improve together.

KEY POINTS

- Throughput: frequency and lead time. Stability: failure rate and restore time.
- Measure teams and systems, never individuals.

Q291 How do you version releases and generate changelogs automatically for a service consumed by other teams?

FLOW DIAGRAM



EXPLANATION

Use semantic versioning: MAJOR for breaking API changes, MINOR for backward-compatible features, PATCH for fixes, so consumers know what an upgrade means. Enforce Conventional Commit messages (feat: , fix: , feat!:) with a PR title check, and let a tool such as release-please or semantic-release compute the next version, update `CHANGELOG.md` and create the tag and release. A tag trigger then builds the artifact and pushes it with the version tag plus the commit SHA. For deployed services a build number or SHA is enough internally, but semver matters for libraries, APIs and Helm charts.

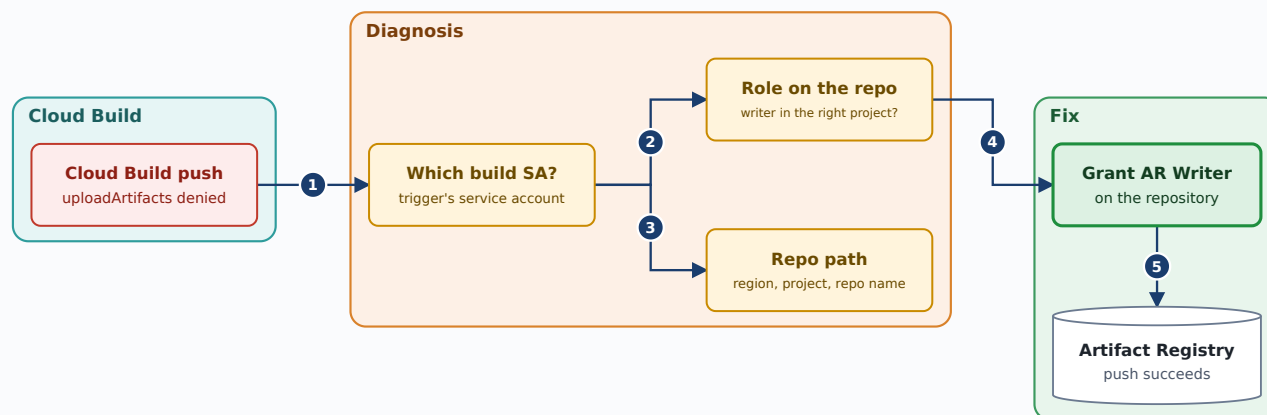
KEY POINTS

- Version from commit history, not from memory.
- Tags should be immutable once published.

Q292**Troubleshooting****A Cloud Build step fails with denied: Permission**

"artifactregistry.repositories.uploadArtifacts" denied on resource ...

when pushing the image. The build worked last week. What do you check?

FLOW DIAGRAM

- 1 Build logs and trigger config show the SA
- 2 Newer projects no longer use the legacy Cloud Build SA
- 3 A typo or wrong region also gives denied
- 4 roles/artifactregistry.writer

EXPLANATION

Find which service account the build runs as: triggers can use a user-specified SA, and newer projects default to the Compute Engine default SA or require one instead of the legacy

PROJECT_NUMBER@cloudbuild.gserviceaccount.com, so a changed trigger or new project often explains "it worked last week". Grant that SA roles/artifactregistry.writer on the repository, in the project that owns the repository. Also check the image path (REGION-docker.pkg.dev/PROJECT/REPO/IMAGE): a wrong region or repo name, or a repository that does not exist, can surface as a permission error. If IAM looks right, check VPC Service Controls perimeters around the registry project.

KEY POINTS

- Use one dedicated, least-privilege build SA per pipeline and set it on the trigger.
- Grant registry roles on the repository, not the whole project.

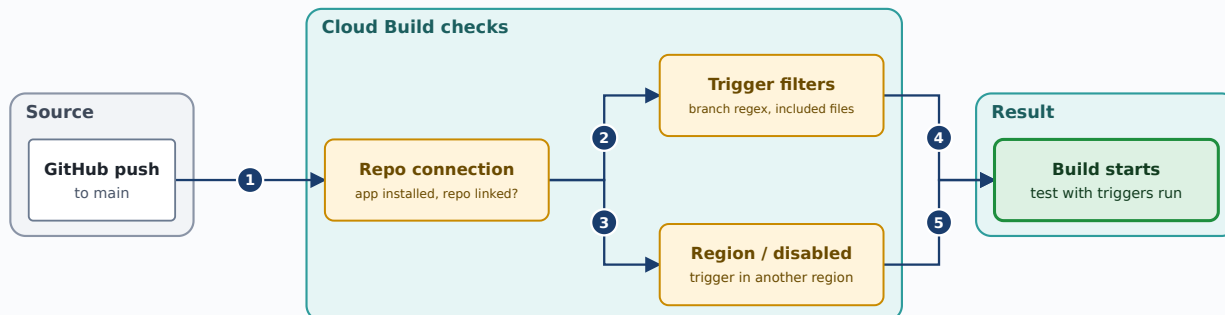
CONFIGURATION EXAMPLE

```

gcloud builds triggers describe app-main --region=us-central1 --format='value(serviceAccount)'
gcloud artifacts repositories add-iam-policy-binding app-images \
  --project=shared-images --location=us-central1 \
  --member=serviceAccount:ci-build@prod-ci.iam.gserviceaccount.com \
  --role=roles/artifactregistry.writer
gcloud artifacts repositories list --project=shared-images --location=us-central1
  
```

Q293**Troubleshooting**

Developers push to `main` but the Cloud Build trigger never starts; there is no failed build, just nothing. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Webhook delivered? Check the app's recent deliveries
- 2 `^main$` vs `main`, `includedFiles`, `ignoredFiles`
- 3 Trigger disabled or in a region you are not viewing
- 4 Fix the filter, then run the trigger manually

EXPLANATION

Start at the source: in GitHub, check the Cloud Build app is still installed on the repository and look at recent webhook deliveries for errors; a revoked installation or a moved repo breaks the connection silently. Then describe the trigger: the branch pattern is a regular expression (`^main$`), the event type must match (push vs pull request), and `includedFiles` or `ignoredFiles` can skip commits that touch other paths. Also confirm the trigger is enabled and look in the right region, because triggers can be regional and the console shows one region at a time. Run the trigger manually with `gcloud builds triggers run` to separate trigger problems from build problems.

KEY POINTS

- Keep trigger definitions in Terraform so filter changes are reviewed.
- Alert when no build has run on `main` for a working day.

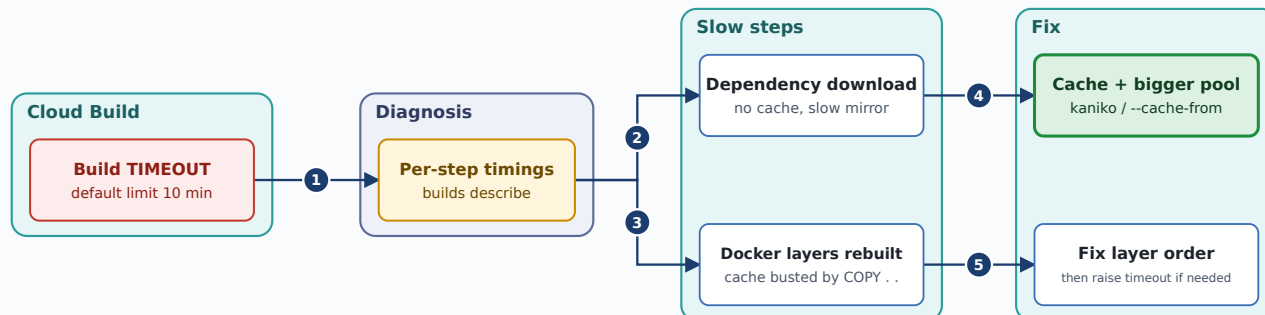
CONFIGURATION EXAMPLE

```

gcloud builds triggers list --region=us-central1 \
  --format='table(name,disabled,github.push.branch,includedFiles)'
gcloud builds triggers describe app-main --region=us-central1
gcloud builds triggers run app-main --region=us-central1 --branch=main
gcloud builds list --region=us-central1 --limit=5
  
```

Q294**Troubleshooting**

Cloud Build jobs that used to take 8 minutes now fail with `TIMEOUT` after 10 minutes. How do you find what got slow and fix it?

FLOW DIAGRAM

- 1 Which step's duration grew?
- 2 npm / Maven fetch dominates
- 3 Every layer rebuilds after a small change
- 4 Remote repo cache in Artifact Registry
- 5 Copy lockfile, install, then copy source

EXPLANATION

The default build timeout is 10 minutes, so a build that slowly grew past it now fails; look at the per-step timing in `gcloud builds describe` to see which step grew. Common causes are dependencies downloaded from the internet on every build, a Dockerfile that copies the whole source before installing dependencies (so any change busts the cache), larger test suites, or builds queuing on a busy private pool. Fix the cause: order Dockerfile layers so dependency install is cached, use `--cache-from` or Kaniko cache, cache packages through an Artifact Registry remote repository, and use a larger machine type for CPU-heavy steps. Raise `timeout` in `cloudbuild.yaml` only after that, so a hung step still fails in reasonable time.

KEY POINTS

- Set per-step `timeout` on steps that can hang, like integration tests.
- Track build duration as a metric and alert on a trend, not only on failure.

CONFIGURATION EXAMPLE

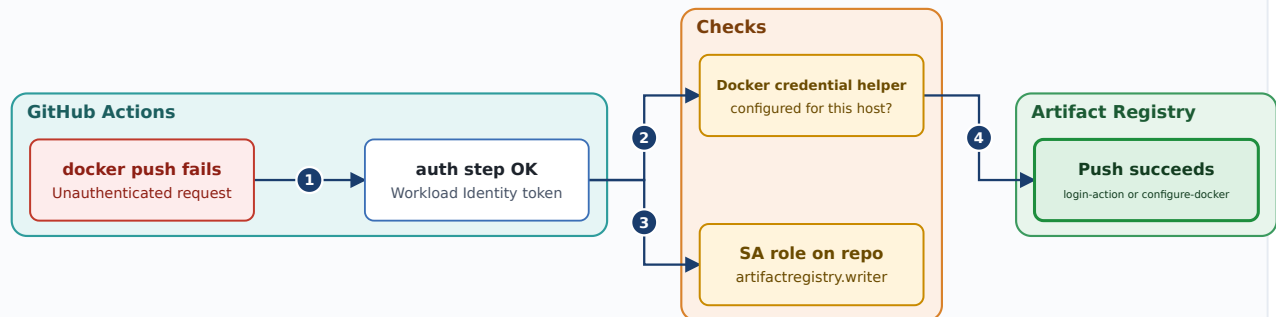
```

gcloud builds describe BUILD_ID --region=us-central1 \
  --format='table(steps.id,steps.timing.startTime,steps.timing.endTime)'
# cloudbuild.yaml
options:
  machineType: E2_HIGHCPU_8
  timeout: 1200s

```

Q295**Troubleshooting****A GitHub Actions job authenticates to Google Cloud**

successfully, but `docker push us-central1-docker.pkg.dev/...` fails with denied: Unauthenticated request . What is missing?

FLOW DIAGRAM

- 1 gcloud is logged in, Docker is not
- 2 Docker needs credentials for REGION-docker.pkg.dev
- 3 After login, missing roles give a 403 instead
- 4 Configure the helper for the exact regional host

EXPLANATION

The auth action gives gcloud credentials, but Docker keeps its own credentials, so Docker sends no token to the registry and gets `Unauthenticated`. Either run `gcloud auth configure-docker us-central1-docker.pkg.dev` (the host must match the region in the image path) or use `docker/login-action` with the access token from the auth step (`token_format: access_token`, username `oauth2accesstoken`). Once Docker is authenticated, a missing role shows up as a permission-denied error instead, so also grant the deploy SA `roles/artifactregistry.writer` on the repository. A common trap is configuring `gcr.io` or another region than the one you push to.

KEY POINTS

- Use Workload Identity Federation, not JSON keys, for the auth step.
- Keep the registry host in one workflow variable so login and push always match.

CONFIGURATION EXAMPLE

```

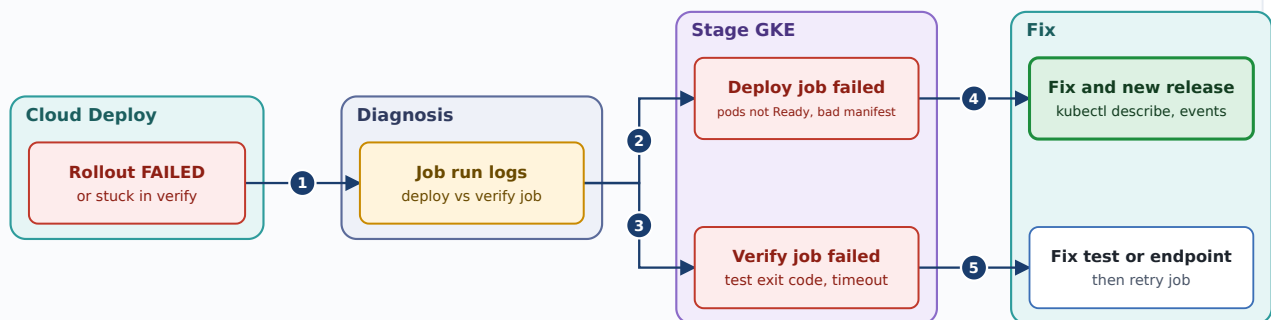
- uses: google-github-actions/auth@v2
  id: auth
  with:
    workload_identity_provider: ${vars.WIF_PROVIDER}
    service_account: ${vars.DEPLOY_SA}
    token_format: access_token
- uses: docker/login-action@v3
  with:
    registry: us-central1-docker.pkg.dev
    username: oauth2accesstoken
    password: ${steps.auth.outputs.access_token}
- run: docker push us-central1-docker.pkg.dev/prod-app/app/web:${github.sha}
  
```

Q296

Troubleshooting

A Cloud Deploy rollout to the stage GKE target shows FAILED in the verify phase (or stays IN_PROGRESS for 30 minutes). How do you troubleshoot it?

FLOW DIAGRAM



- 1 Rollout shows which phase and job failed
- 2 Deployment never became healthy
- 3 Verify container returned non-zero
- 4 Image pull, probes, config, quota
- 5 Test hits wrong URL, missing SA role

EXPLANATION

Describe the rollout to see which phase and job failed, then open the job run's Cloud Build logs, which Cloud Deploy uses to run deploy and verify jobs. If the deploy job failed or timed out, the workload never became ready: check pod events for ImagePullBackOff, failing readiness probes or missing ConfigMaps. If the verify job failed, the verification container exited non-zero: check it tests the right in-cluster URL, that the execution service account has the roles it needs, and that the test itself is not flaky. After fixing configuration you can retry the job; after fixing code, create a new release.

KEY POINTS

- Keep verification tests short and deterministic, with their own timeout.
- Give each target a dedicated execution service account with only the roles it needs.

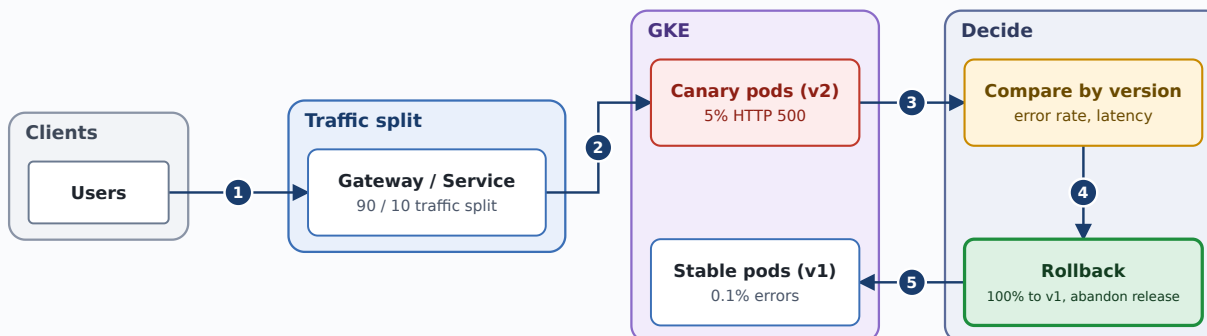
CONFIGURATION EXAMPLE

```

gcloud deploy rollouts describe rel-042-to-stage-0001 \
  --release=rel-042 --delivery-pipeline=web --region=us-central1
gcloud deploy job-runs list --rollout=rel-042-to-stage-0001 \
  --release=rel-042 --delivery-pipeline=web --region=us-central1
kubectl get events -n web --sort-by=.lastTimestamp | tail -20
gcloud deploy rollouts retry-job rel-042-to-stage-0001 --job-id=verify \
  --phase-id=stable --release=rel-042 --delivery-pipeline=web --region=us-central1
  
```

Q297**Troubleshooting**

During a canary on GKE, the 10% canary pods return 5% HTTP 500s while stable pods are clean. How do you confirm it, roll back safely and stop the release from going further?

FLOW DIAGRAM

- ② Small share reaches the new version
- ③ Metrics split by version label
- ④ Canary clearly worse: stop now
- ⑤ Send all traffic back to stable

EXPLANATION

Confirm the difference with metrics split by version (a `version` label on pods, or per-deployment metrics in Cloud Monitoring) and look at canary logs for the exception. If the canary is clearly worse, roll back first: in Cloud Deploy, roll the target back to the previous release and abandon the bad release so it cannot be promoted; with plain Kubernetes, scale the canary to zero or run `kubectl rollout undo`. Traffic returns to the stable version, which never stopped running, so recovery is quick. Then block promotion by fixing the release rather than retrying it, and add an automated analysis step so the next bad canary stops itself.

KEY POINTS

- Define the canary pass criteria (error rate, p95) before the release, not during it.
- Keep the database schema backward compatible so rollback is always possible.

CONFIGURATION EXAMPLE

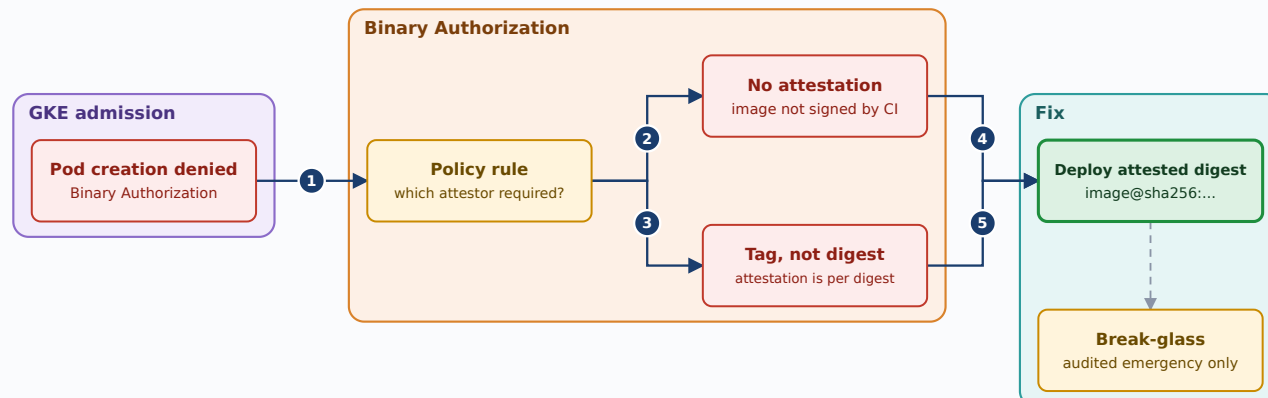
```

kubectl get pods -n web -L version
gcloud deploy targets rollback prod --delivery-pipeline=web --region=us-central1
gcloud deploy releases abandon rel-043 --delivery-pipeline=web --region=us-central1
# plain Kubernetes alternative
kubectl scale deployment web-canary -n web --replicas=0
  
```

Q298**Troubleshooting****A GKE deployment is rejected with admission webhook**

"imagepolicywebhook.image-policy.k8s.io" denied the request: Image ...

denied by Binary Authorization . **How do you find out why and fix it?**

FLOW DIAGRAM

- 1 Event text names the policy and image
- 2 List attestations for the digest
- 3 Tag resolved to an unattested digest
- 4 Rebuild through the pipeline that signs
- 5 Reference the digest in manifests
- Only for incidents, alerts security

EXPLANATION

Read the full denial message and the policy: it says which rule applied (project default or a cluster-specific rule) and which attestors are required. Then check whether an attestation exists for that exact image digest; attestations are tied to digests, so an image built by hand, pushed outside CI, or referenced by a tag that now points to a new digest will be rejected. Fix it by deploying the digest produced and attested by the pipeline, and make manifests use `image@sha256:...`. If the signing step failed, check the CI service account still has `roles/containeranalysis.notes.attacher`, `roles/binaryauthorization.attestorsViewer` and the KMS signer role on the signing key; use the break-glass annotation only for real emergencies, since it is logged and should trigger an alert.

KEY POINTS

- Try new policies in dry-run mode first and watch the audit logs.
- Exempt system images with allowlist patterns instead of disabling enforcement.

CONFIGURATION EXAMPLE

```

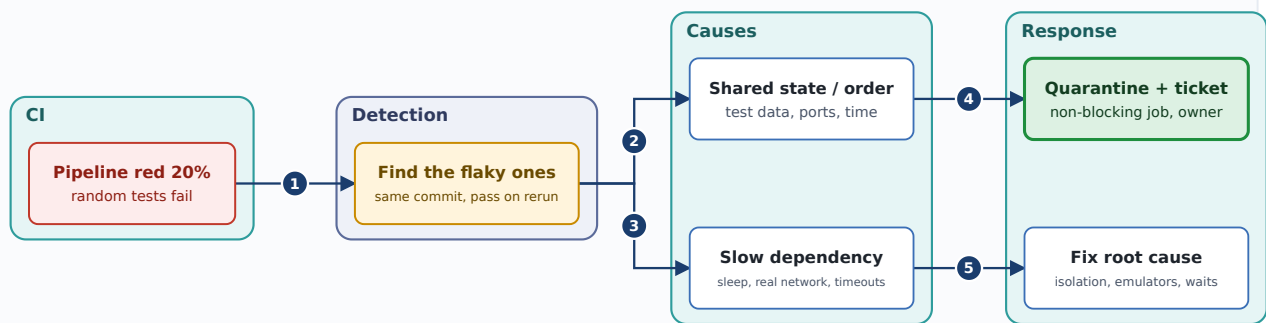
kubectl get events -n web --field-selector reason=FailedCreate | tail -5
gcloud container binauthz policy export
gcloud container binauthz attestations list --attestor=built-by-ci \
  --attestor-project=prod-sec \
  --artifact-url=us-central1-docker.pkg.dev/prod-app/app/web@sha256:3f1c...
gcloud artifacts docker images describe us-central1-docker.pkg.dev/prod-app/app/web:v43 \
  --format='value(image_summary.digest)'
```


Q299

Troubleshooting

The main pipeline fails about one run in five on different integration tests that pass on rerun, and engineers have started clicking "retry" by habit. How do you deal with the flaky tests?

FLOW DIAGRAM



- 1 Collect JUnit results across runs
- 2 Tests depend on each other or shared data
- 3 Fixed sleeps and external services
- 4 Unblock main right away
- 5 Then fix and move back

EXPLANATION

Collect test results (JUnit XML) from every run and find tests that fail and pass on the same commit; those are flaky, not broken code. Move them to a quarantined, non-blocking job with an owner and a ticket, so main is trustworthy again while they are fixed. Typical causes are shared test data or execution order, fixed `sleep` calls instead of waiting for a condition, real network calls to external services, time zones and clocks, and resource limits on the runner. Fix them by isolating data per test, using emulators (Pub/Sub, Firestore, Spanner) or test containers, and polling with timeouts; limit automatic retries to one and report them, so retries do not hide new flakiness.

KEY POINTS

- Track a flaky-test rate per week and make fixing flakes part of normal work.
- Run a new test many times in CI before it becomes blocking.

CONFIGURATION EXAMPLE

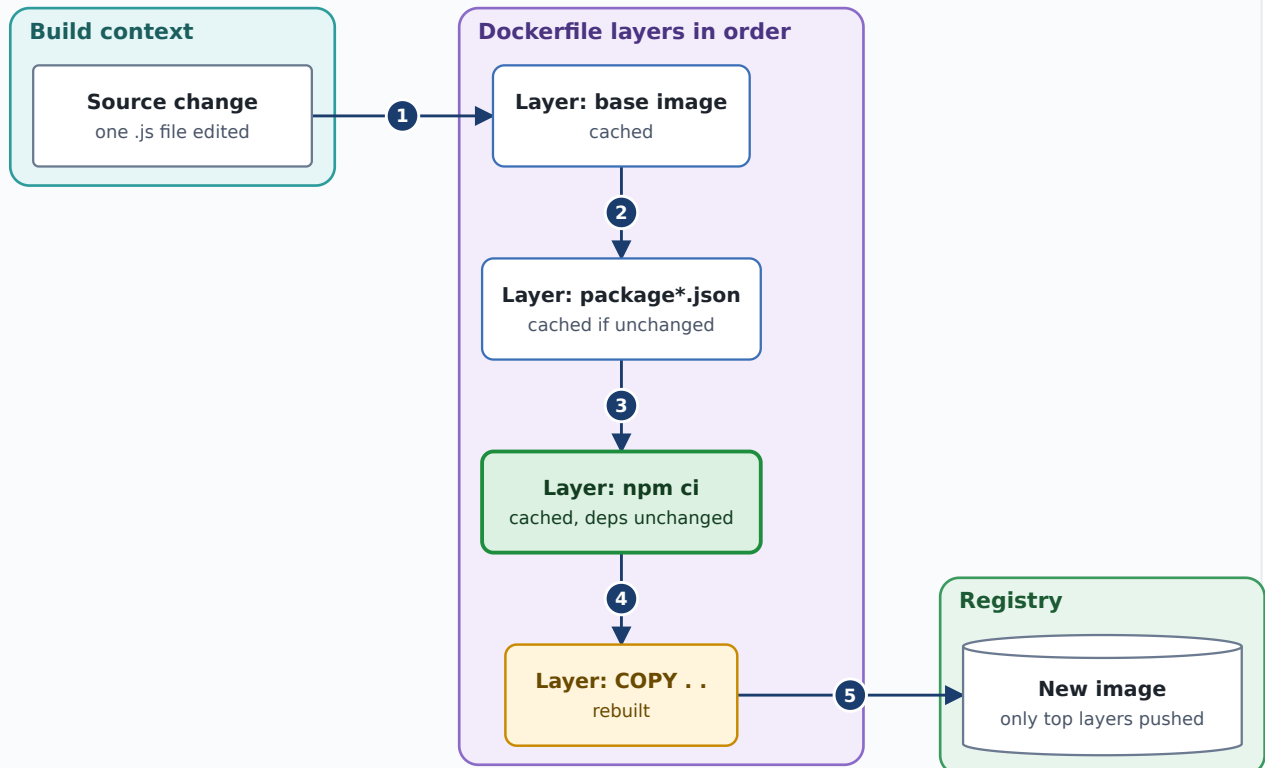
```
# run a suspect test 50 times to measure flakiness
for i in $(seq 1 50); do
  go test ./orders -run TestCheckout -count=1 >/dev/null 2>&1 || echo "fail $i"
done
# pytest: quarantine by marker, keep it out of the blocking job
pytest -m "not quarantine" --junitxml=reports/junit.xml
```

11. Docker & Containers

Building small, secure, fast images and running containers correctly, from local development to the registry.

Q300 Every Docker build of a Node.js service takes 6 minutes even when only one source file changed. Why, and how do you fix the Dockerfile?

FLOW DIAGRAM



- 1 Builder checks each instruction's cache key
- 2 Lock files copied first
- 3 Dependency install reused from cache
- 4 Cache busts only from here down
- 5 Push changed layers only

EXPLANATION

Each Dockerfile instruction creates a layer, and once one layer's cache key changes every later layer is rebuilt. If the Dockerfile runs `COPY . .` before `npm ci`, any source edit invalidates the dependency install. Copy only `package.json` and `package-lock.json` first, install dependencies, then copy the rest of the source, so edits rebuild only the final layers. Order instructions from least to most frequently changing, and in CI reuse cache from the registry with BuildKit `--cache-from` or Kaniko/Cloud Build cache options.

KEY POINTS

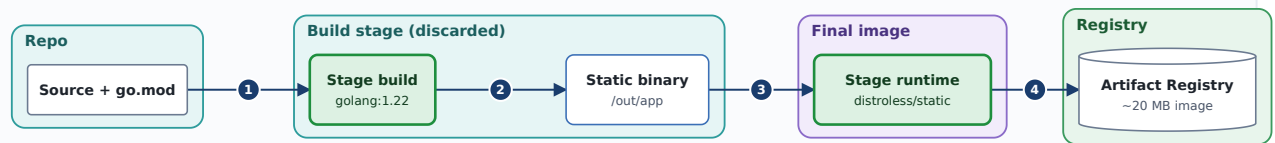
- Cache invalidation cascades downward.
- Copy dependency manifests before source code.

CONFIGURATION EXAMPLE

```
FROM node:20-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "server.js"]
```

Q301 A Go service image is 1.1 GB because it contains the full toolchain. How do you use a multi-stage build to ship only the binary?

FLOW DIAGRAM



- ❶ Compile with full toolchain
- ❷ `CGO_ENABLED=0 go build`
- ❸ `COPY --from=build` only the binary
- ❹ Push small final stage

EXPLANATION

Use one stage with the full SDK to download modules and compile, and a second minimal stage that copies only the built artefact with `COPY --from=build`. Everything in the build stage, including compilers, caches and source, is left out of the final image. For Go, build a static binary with `CGO_ENABLED=0` and use a distroless or scratch base; for Java, build with Maven in the first stage and copy the JAR onto a JRE-only image. The result is a much smaller image with fewer packages to scan and patch.

KEY POINTS

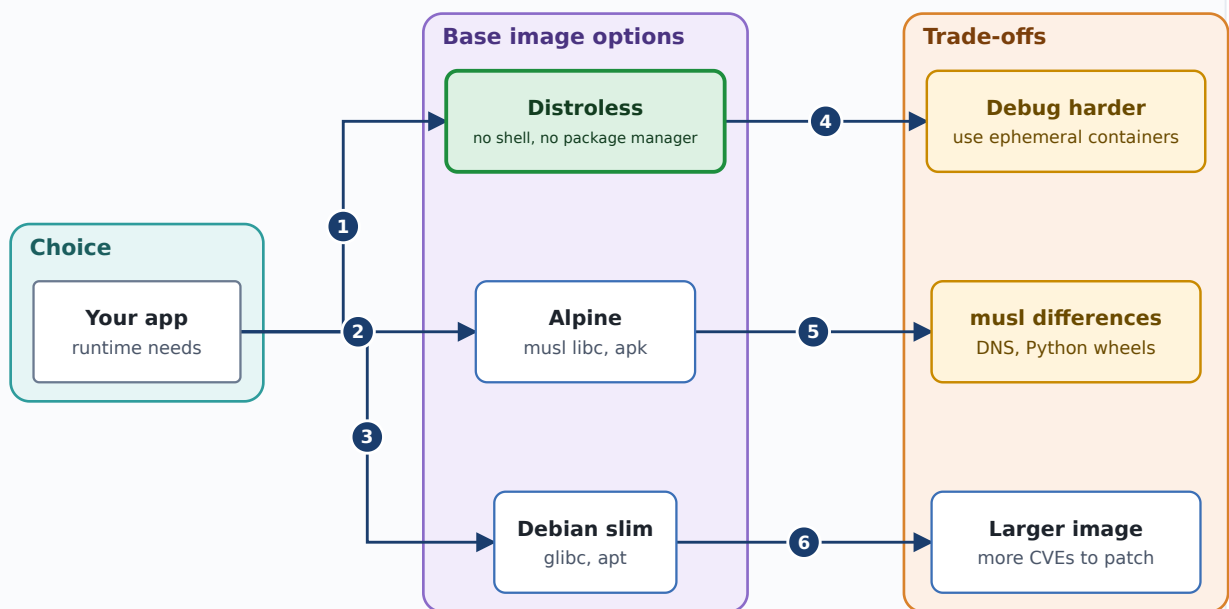
- Only the last stage ships.
- Smaller image = faster pulls, smaller attack surface.

CONFIGURATION EXAMPLE

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/app
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/app /app
ENTRYPOINT ["/app"]
```

Q302 The security team wants smaller base images. Compare distroless, Alpine and Debian slim, and when would each cause problems?

FLOW DIAGRAM



- ❶ Static binaries, JVM, Node, Python variants
- ❷ Tiny image with a shell
- ❸ Maximum compatibility

EXPLANATION

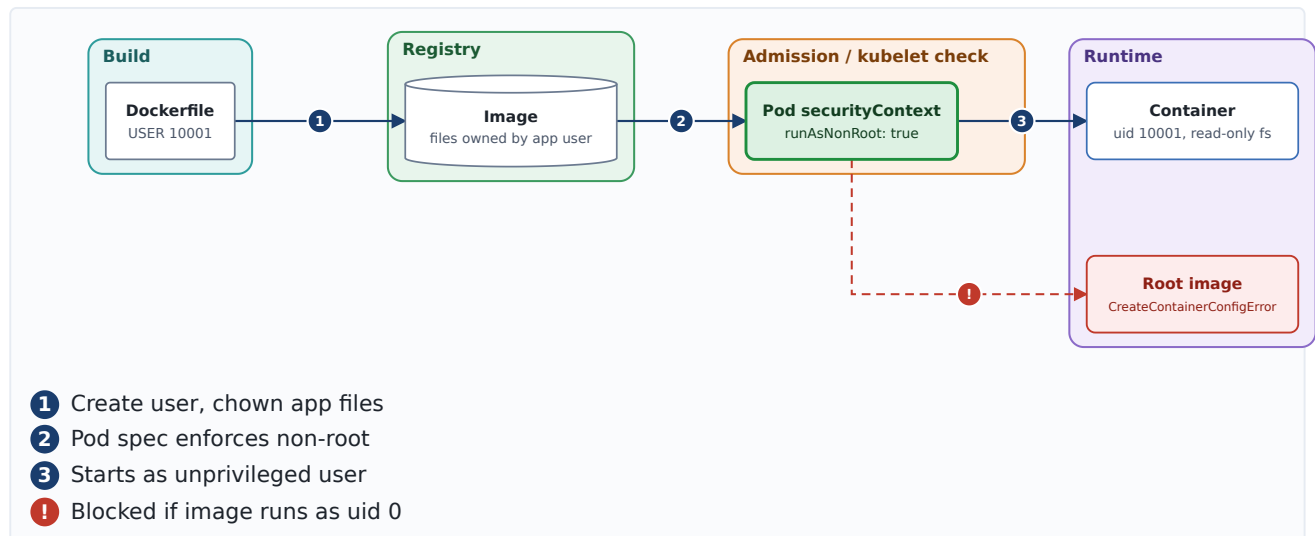
Distroless images contain only the runtime and its libraries, with no shell or package manager, which cuts CVE count and blocks many post-exploit tools; the cost is harder debugging, which you handle with `kubectl debug` ephemeral containers or a `:debug` tag. Alpine is very small and has a shell, but it uses musl libc, so some prebuilt binaries and Python wheels do not work and DNS resolution behaves differently from glibc. Debian slim is larger but most compatible. Pick distroless for production services where possible, slim when you need glibc tooling, and pin versions by digest for repeatable builds.

KEY POINTS

- Distroless: smallest attack surface, no shell.
- Alpine: musl can break native deps.

Q303 A penetration test found containers running as root. How do you make images and pods run as a non-root user?

FLOW DIAGRAM



EXPLANATION

In the Dockerfile create an unprivileged user with a fixed numeric UID, give it ownership of only the files it must write, and switch with `USER 10001`; many distroless images offer a `:nonroot` variant. In Kubernetes set `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, drop all capabilities and use a read-only root filesystem where the app allows it. The kubelet refuses to start a container that would run as UID 0 when `runAsNonRoot` is set. Enforce the policy cluster-wide with Pod Security Admission `restricted` or policy controllers.

KEY POINTS

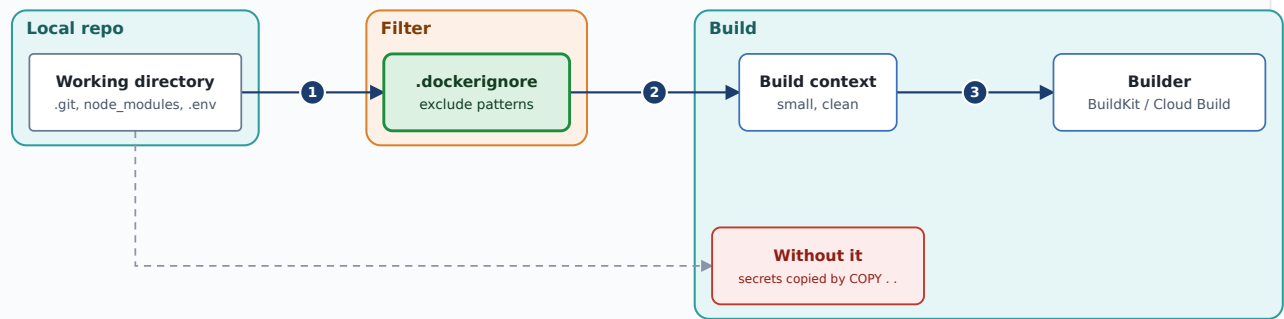
- Use a numeric UID so `runAsNonRoot` can verify it.
- Bind to ports above 1024 as non-root.

CONFIGURATION EXAMPLE

```
FROM python:3.12-slim
RUN useradd --uid 10001 --create-home app
WORKDIR /home/app
COPY --chown=app:app . .
USER 10001
CMD ["python", "main.py"]
```

Q304 Builds upload a 2 GB context and a leaked `.env` file was found inside a published image. What does `.dockerignore` do and what should it contain?

FLOW DIAGRAM



- ❶ Files matched are never sent
- ❷ Only needed sources remain
- ❸ Faster upload, stable cache keys
- Everything reaches the builder

EXPLANATION

The `.dockerignore` file removes paths from the build context before it is sent to the builder, so they can never be copied into the image. Exclude `.git`, local dependency folders like `node_modules`, build output, test data, IDE files and every secret file such as `.env` or key files. This shrinks upload time, avoids cache busts from irrelevant file changes, and prevents accidental leaks via `COPY . .`. Also prefer explicit `COPY` of needed directories over copying the whole context.

KEY POINTS

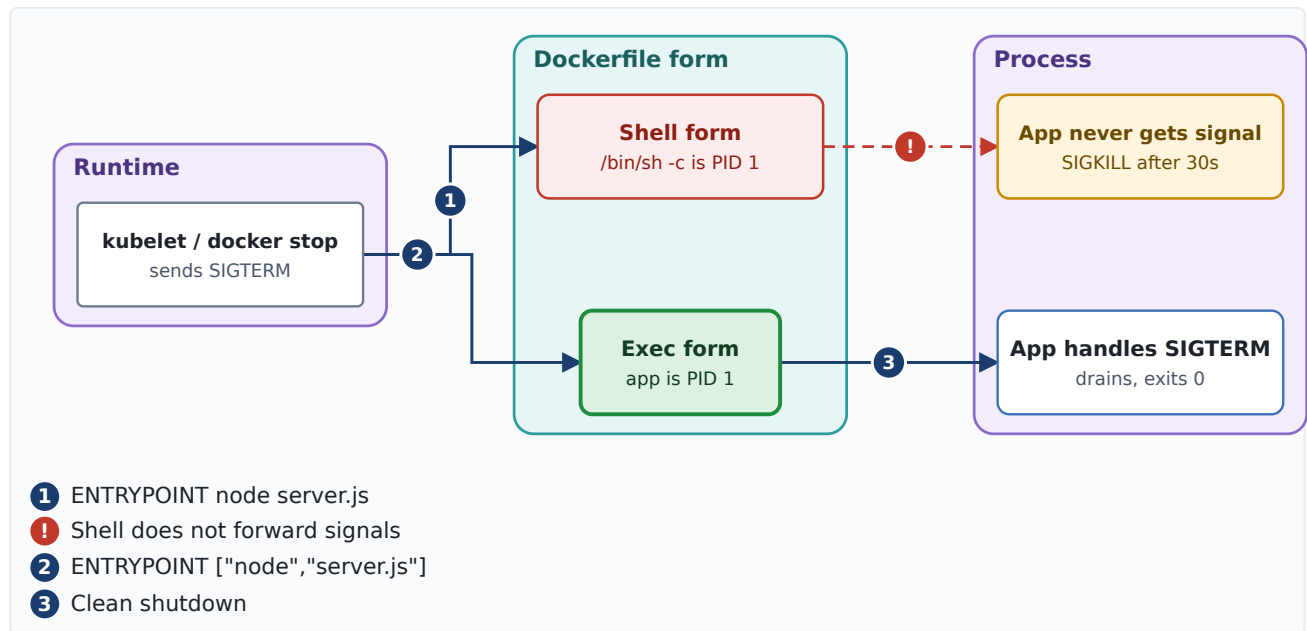
- Smaller context = faster builds and stable caching.
- Never rely on `.dockerignore` alone for secrets; keep them out of the repo.

CONFIGURATION EXAMPLE

```
.git
node_modules
dist
*.log
.env
*.pem
Dockerfile*
.dockerignore
```

Q305 A container ignores SIGTERM and is killed after the grace period on every deploy. How do ENTRYPOINT and CMD work, and what is wrong?

FLOW DIAGRAM



EXPLANATION

ENTRYPOINT defines the executable and CMD supplies default arguments; arguments passed at `docker run` or in a pod's `args` replace CMD, while `command` in Kubernetes replaces ENTRYPOINT. In shell form (`ENTRYPOINT node server.js`) Docker runs `/bin/sh -c`, the shell becomes PID 1 and usually does not forward SIGTERM, so the app is killed hard at the end of the grace period. Use exec form with a JSON array so the app is PID 1 and receives the signal, and make the app handle SIGTERM. If you need a wrapper script, end it with `exec "$@"`, or use a small init such as tini for zombie reaping.

KEY POINTS

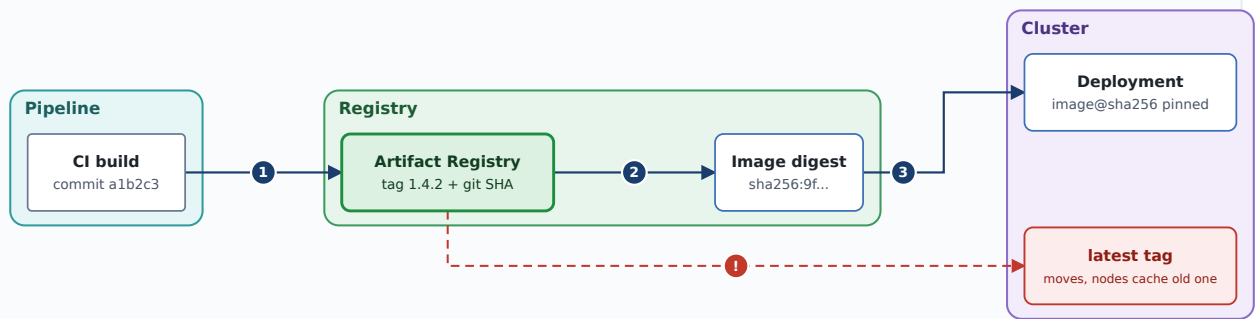
- Exec form (JSON array) for correct signal handling.
- ENTRYPOINT = program, CMD = default args.

CONFIGURATION EXAMPLE

```
ENTRYPOINT ["/usr/bin/tini", "--", "python", "worker.py"]
CMD ["--queue=default"]
# docker run img --queue=high -> overrides CMD only
```

Q306 Production pods are running different code even though they all reference `api:latest` . How should you tag and reference images?

FLOW DIAGRAM



- 1 Push unique tag, never reuse it
- 2 Content-addressed ID
- 3 Deploy by digest for exact bits
- ! Mutable tag resolves differently per node

EXPLANATION

A mutable tag like `latest` points at different images over time, and nodes pull it at different moments or keep cached copies, so replicas can run different builds. Tag every build with an immutable, unique tag such as the Git SHA or a semantic version, and enable immutable tags on the Artifact Registry repository so a tag cannot be overwritten. Deploy by digest (`image@sha256:...`) when you need the exact bits, which also works well with Binary Authorization. Keep human-friendly tags for discovery, but let manifests reference the unique tag or digest.

KEY POINTS

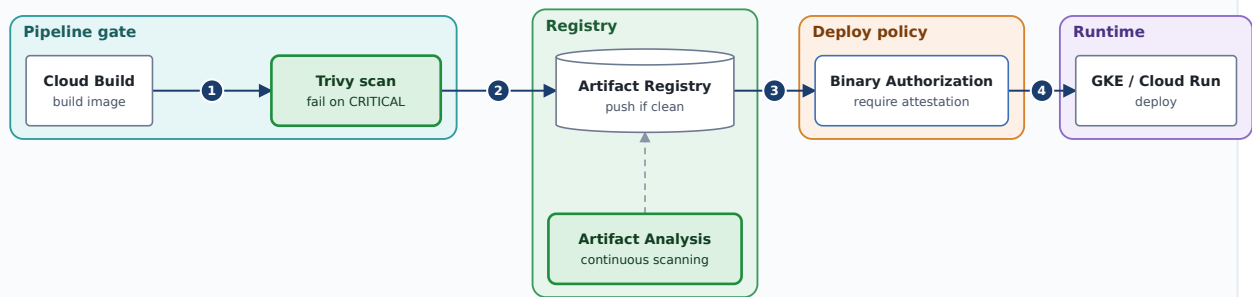
- Unique tag per build; never overwrite.
- Digests are the only truly immutable reference.

CONFIGURATION EXAMPLE

```
gcloud artifacts repositories create apps --location=europe-west1 \
  --repository-format=docker --immutable-tags
docker build -t europe-west1-docker.pkg.dev/proj/apps/api:$GIT_SHA .
docker push europe-west1-docker.pkg.dev/proj/apps/api:$GIT_SHA
```


Q307 How do you make sure container images with critical vulnerabilities do not reach production on Google Cloud?

FLOW DIAGRAM



- 1 Scan before pushing
- 2 Only passing images pushed
- 3 Attested digest checked
- 4 Admitted
- New CVEs found on stored images

EXPLANATION

Scan in the pipeline before pushing, for example with Trivy using `--exit-code 1 --severity CRITICAL`, or with the On-Demand Scanning API, so the build fails fast. Enable Artifact Analysis on Artifact Registry so pushed images are scanned automatically and re-evaluated as new CVEs are published. Gate deployment with Binary Authorization attestations created only after scans pass. Rebuild images regularly to pick up patched base layers, and track exceptions with an expiry date rather than ignoring findings forever.

KEY POINTS

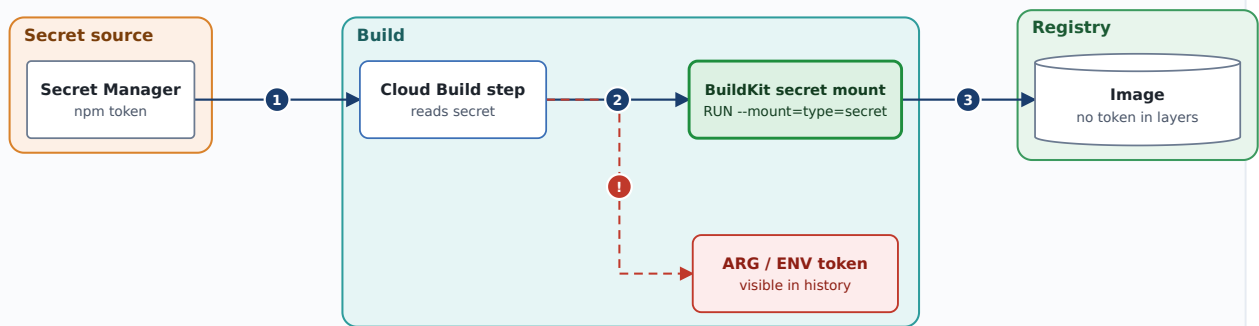
- Scan in CI and continuously in the registry.
- Rebuild often; base-image CVEs keep appearing.

CONFIGURATION EXAMPLE

```
trivy image --exit-code 1 --severity CRITICAL,HIGH \  
--ignore-unfixed europe-west1-docker.pkg.dev/proj/apps/api:$GIT_SHA
```

Q308 A Dockerfile uses `ARG NPM_TOKEN` to install private packages. Why is this a problem and how should secrets be used at build time?

FLOW DIAGRAM



- ❶ availableSecrets in cloudbuild.yaml
- ❷ Passed as a temporary file
- ❸ Mounted only during that RUN
- ❗ docker history shows build args

EXPLANATION

Build args and ENV values are recorded in image metadata and layers, so anyone who can pull the image can read them with `docker history` or by inspecting layers. Use BuildKit secret mounts, `RUN --mount=type=secret,id=np`, which expose the secret as a file only during that single instruction and never write it to a layer. In Cloud Build, fetch the value from Secret Manager with `availableSecrets` and pass it to the build. Runtime secrets should come from Secret Manager or the platform at start-up, never baked into the image.

KEY POINTS

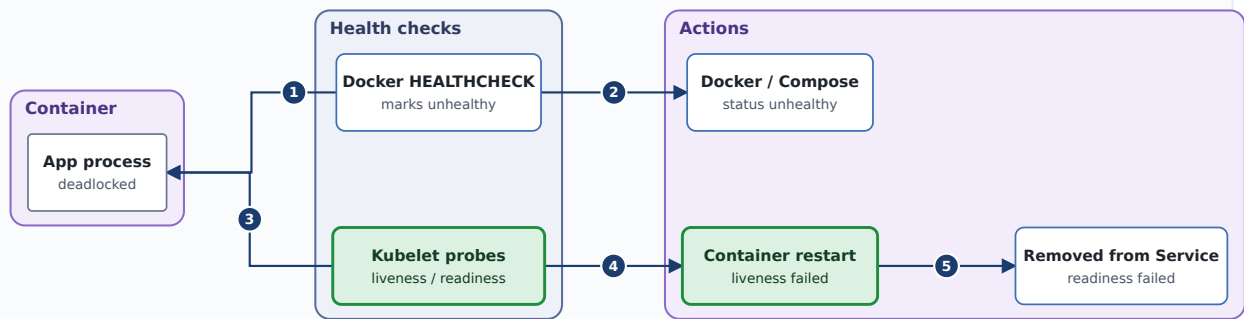
- ARG/ENV secrets leak through image history.
- Use `--mount=type=secret` for build-time credentials.

CONFIGURATION EXAMPLE

```
# syntax=docker/dockerfile:1
FROM node:20-slim
WORKDIR /app
COPY package*.json .npmrc ./
RUN --mount=type=secret,id=np,env=NPM_TOKEN npm ci
# build: docker build --secret id=np,env=NPM_TOKEN .
```

Q309 A container shows as running but the app inside is deadlocked. How do Docker HEALTHCHECK and Kubernetes probes handle this?

FLOW DIAGRAM



- ❶ Runs command every interval
- ❷ Compose depends_on can wait for healthy
- ❸ HTTP/TCP/exec checks
- ❹ Liveness failures restart it
- ❺ Readiness failures stop traffic

EXPLANATION

A Dockerfile `HEALTHCHECK` runs a command periodically and marks the container healthy or unhealthy, which Docker Compose and Swarm can act on, but it does not restart the container on plain Docker. Kubernetes ignores the Docker HEALTHCHECK and uses its own probes: liveness restarts a hung container, readiness removes it from Service endpoints, and startup protects slow boots. For a deadlock, expose a cheap `/healthz` endpoint that checks the app's own loop, not downstream dependencies, and wire it to the liveness probe. Keep readiness separate so dependency outages stop traffic without causing restart storms.

KEY POINTS

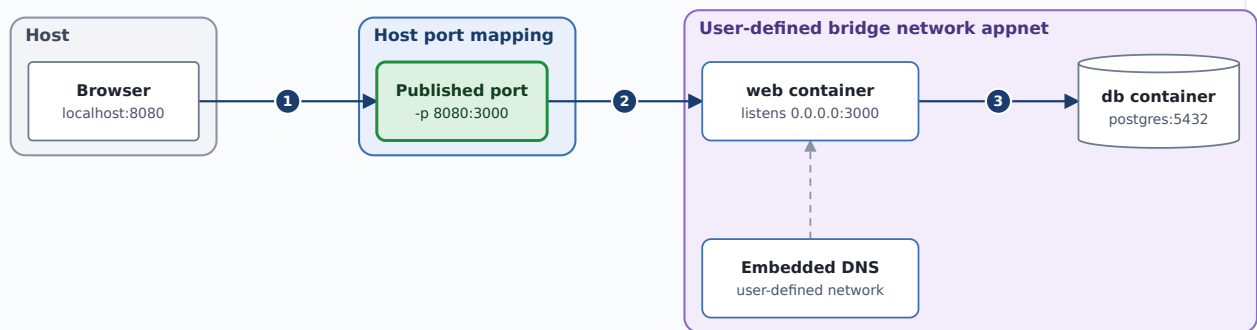
- Kubernetes uses probes, not Dockerfile HEALTHCHECK.
- Liveness checks self; readiness may check dependencies.

CONFIGURATION EXAMPLE

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD ["/app/healthcheck"]
```

Q310 Two containers on a developer laptop cannot reach each other by name, and the app is not reachable from the browser. Explain Docker networking basics.

FLOW DIAGRAM



- 1 Host port forwarded into container
- 2 App must bind 0.0.0.0, not 127.0.0.1
- 3 Connects to db:5432 by name
- Names resolve only on user-defined networks

EXPLANATION

Containers on the default bridge network cannot resolve each other by name; create a user-defined bridge network so Docker's embedded DNS resolves container names. Traffic from the host reaches a container only through a published port such as `-p 8080:3000`, and the app must listen on `0.0.0.0` inside the container, because `127.0.0.1` there is the container's own loopback. Containers on the same network talk on the container port directly, not the published one. Compose creates a project network automatically, so service names work there.

KEY POINTS

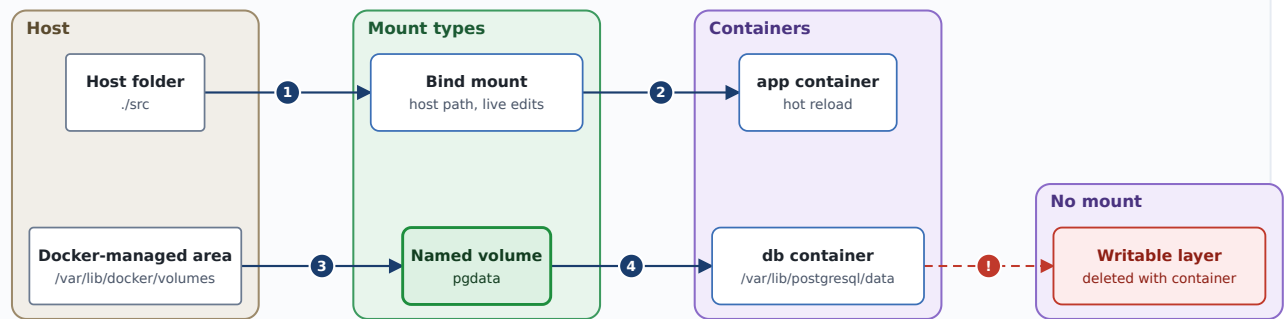
- User-defined networks give name resolution.
- Bind to 0.0.0.0 inside the container.

CONFIGURATION EXAMPLE

```
docker network create appnet
docker run -d --name db --network appnet -e POSTGRES_PASSWORD=dev postgres:16
docker run -d --name web --network appnet -p 8080:3000 \
  -e DB_HOST=db myapp:dev
```

Q311 When would you use a named volume versus a bind mount, and why did database data disappear after `docker rm`?

FLOW DIAGRAM



- 2 Code changes visible instantly
- 4 Data survives container removal
- ! Data written without a volume is lost

EXPLANATION

A bind mount maps a specific host folder into the container. It suits local development, such as source code and config, but depends on host paths and permissions. A named volume is storage that Docker manages; it is the right place for database data and it survives `docker rm` until you run `docker volume rm`. Without a mount, data lives in the container's writable layer and is deleted with the container. Images like `postgres` declare a `VOLUME`, so Docker creates an anonymous volume with a random ID instead; a new container gets a fresh one, so the data seems lost. Use `tmpfs` mounts for scratch data that should never touch disk.

KEY POINTS

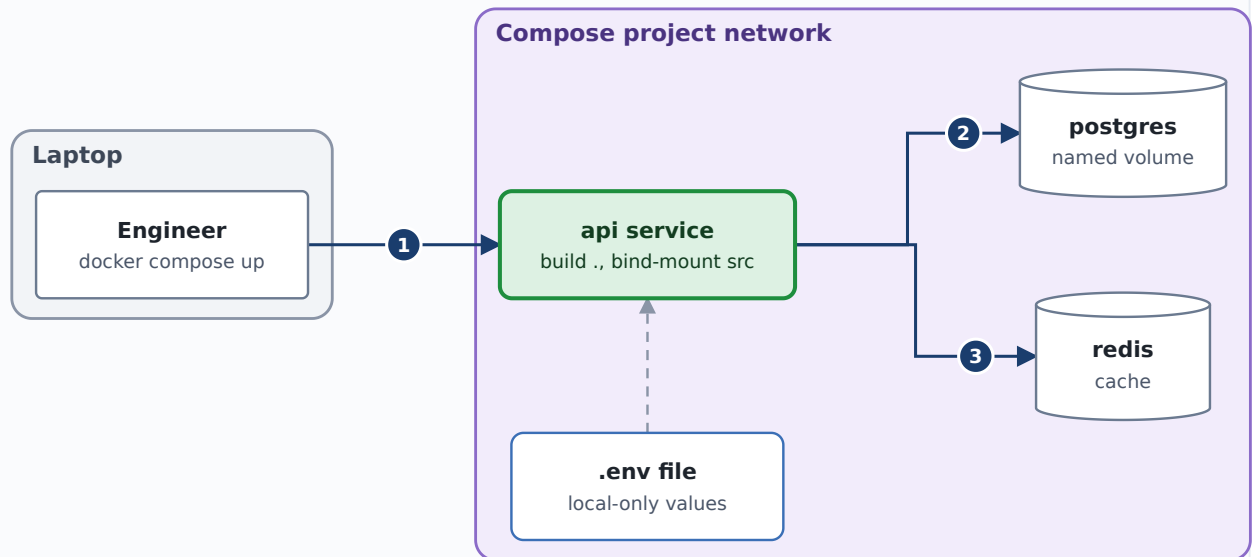
- Volumes for data, bind mounts for dev source.
- Writable layer is ephemeral.

CONFIGURATION EXAMPLE

```
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name app -v "$PWD/src:/app/src" myapp:dev
```

Q312 New engineers need a full local stack (API, Postgres, Redis) running in one command. How do you set this up with Docker Compose?

FLOW DIAGRAM



- 1 Builds and starts services
- 2 Waits for healthy database
- 3 Service names as hostnames
- Variables interpolated, not committed

EXPLANATION

Define each dependency as a Compose service on the project network, so the API reaches `postgres` and `redis` by service name. Build the API from the local Dockerfile, bind-mount source for hot reload, keep database data in a named volume, and use `depends_on` with `condition: service_healthy` plus a healthcheck so the API waits for Postgres. Put local-only values in an uncommitted `.env` file and commit a `.env.example`. Compose is for local development and tests, not a production orchestrator.

KEY POINTS

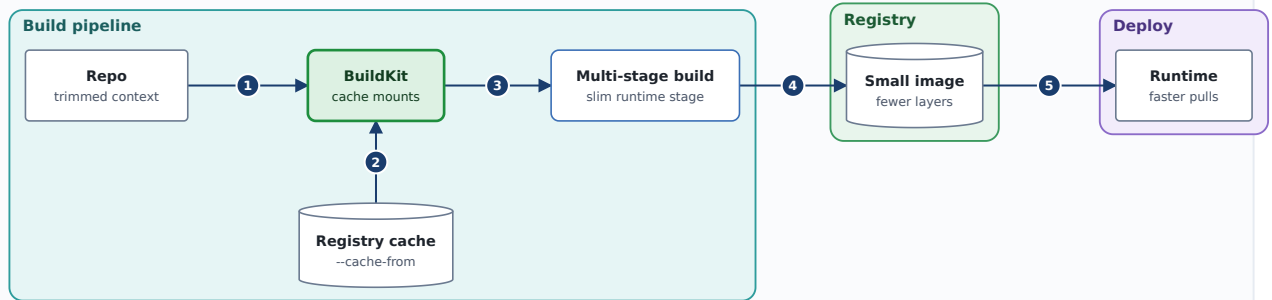
- Service names are DNS names on the Compose network.
- `depends_on` + healthcheck avoids startup races.

CONFIGURATION EXAMPLE

```
services:
  api:
    build: .
    ports: ["8080:8080"]
    volumes: [".:/src:/app/src"]
    depends_on: {postgres: {condition: service_healthy}}
  postgres:
    image: postgres:16
    healthcheck: {test: ["CMD", "pg_isready"], interval: 5s}
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes: {pgdata: {}}
```

Q313 Image builds in Cloud Build take 12 minutes and images are 900 MB. List the changes you would make to cut both.

FLOW DIAGRAM



- ❶ .dockerignore, stable layer order
- ❷ Reuse layers from last build
- ❸ Deps cached across builds
- ❹ Copy only runtime artefacts
- ❺ Smaller pulls, faster scale-out

EXPLANATION

For build time, trim the context with `.dockerignore`, order layers so dependencies install before source copies, use BuildKit cache mounts such as `--mount=type=cache,target=/root/.npm`, and import cache from the registry with `--cache-from` (or Kaniko cache) because Cloud Build workers start empty. Use larger Cloud Build machine types for CPU-heavy compiles. For size, use multi-stage builds, a slim or distroless runtime base, install only production dependencies and clean package manager caches in the same RUN step that created them. Check results with `docker history` or a tool like `dive` to find the largest layers.

KEY POINTS

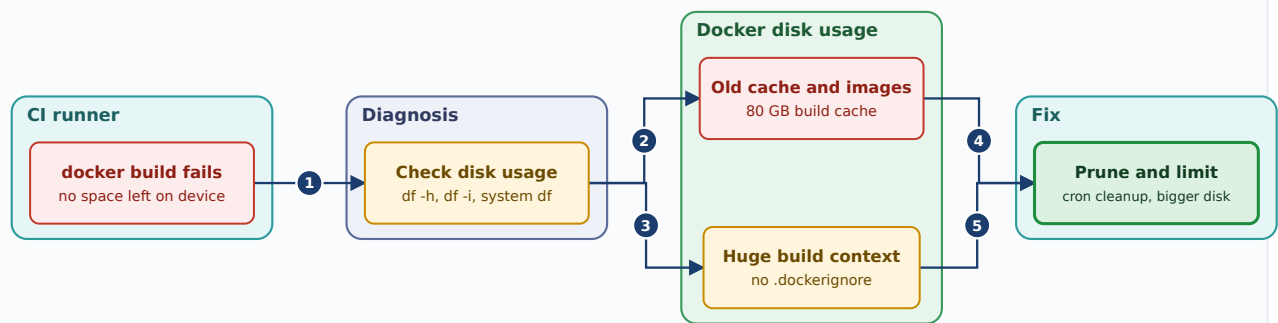
- Cloud Build starts cold: import cache from the registry.
- Deleting files in a later layer does not shrink earlier layers.

CONFIGURATION EXAMPLE

```
docker buildx build \
  --cache-from=type=registry,ref=europe-west1-docker.pkg.dev/proj/apps/api:cache \
  --cache-to=type=registry,ref=europe-west1-docker.pkg.dev/proj/apps/api:cache,mode=max \
  -t europe-west1-docker.pkg.dev/proj/apps/api:$GIT_SHA --push .
```

Q314**Troubleshooting**

docker build on a CI runner fails with `write /var/lib/docker/...: no space left on device`, **although the final image is only 400 MB.**
How do you fix it?

FLOW DIAGRAM

- 1 Check blocks and inodes on the Docker disk
- 2 Months of cache and dangling layers
- 3 Context copied in full to the builder

EXPLANATION

The error is about the builder's disk, not the image size: every build leaves layers, build cache and stopped containers under `/var/lib/docker` on a long-lived runner. Check free space and inodes with `df -h` and `df -i`, then `docker system df` shows how much is images, containers, volumes and build cache. Prune old cache and unused images, and add a `.dockerignore` so the build does not copy `node_modules`, `.git` or test data into the context. Make cleanup a scheduled job on the runner, or use ephemeral runners or Cloud Build, which start clean for every build.

KEY POINTS

- Run `docker builder prune --filter until=72h` on a timer on shared runners.
- Put Docker's data directory on its own, larger disk.

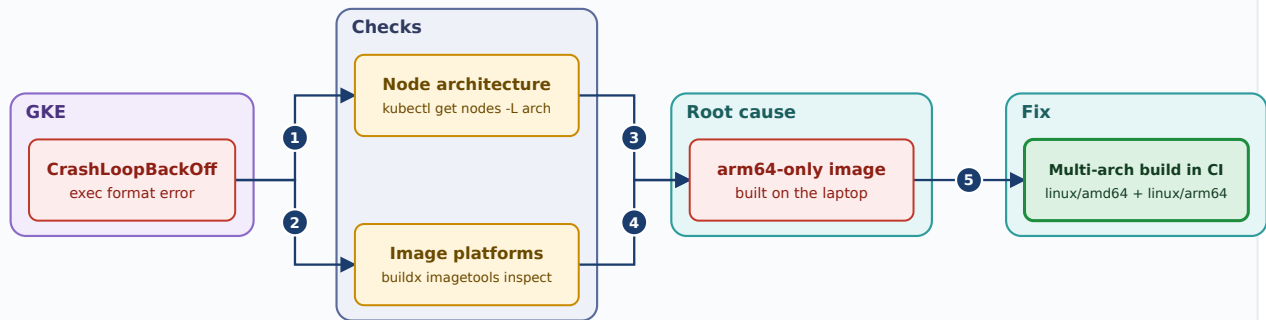
CONFIGURATION EXAMPLE

```

df -h /var/lib/docker && df -i /var/lib/docker
docker system df -v | head -25
docker builder prune -af --filter until=72h
docker image prune -af --filter until=168h
docker container prune -f
du -sh --exclude=.git . && cat .dockerignore
  
```


Q315**Troubleshooting**

An image built on a developer's Apple Silicon Mac runs fine locally, but on GKE the pod crash-loops with `exec /usr/local/bin/api: exec format error`. Why, and how do you fix it?

FLOW DIAGRAM

- ❶ Nodes are amd64
- ❷ Image manifest lists only linux/arm64
- ❸ Build and push a manifest list from CI

EXPLANATION

`exec format error` means the kernel cannot run the binary, almost always because the CPU architecture differs: an image built on an Apple Silicon Mac is `linux/arm64` by default, while most GKE nodes are `amd64`. Compare the node's `kubernetes.io/arch` label with the platforms listed by `docker buildx imagetools inspect`. Fix it by building in CI with `docker buildx build --platform=linux/amd64,linux/arm64`, which pushes a multi-arch manifest list so each node pulls the right variant. The same error can also come from a shell script entrypoint with no `#!/` line or Windows line endings, so check that if the architectures match.

KEY POINTS

- Build release images only in CI, never push from laptops.
- If you run Arm node pools, publish multi-arch images for every service.

CONFIGURATION EXAMPLE

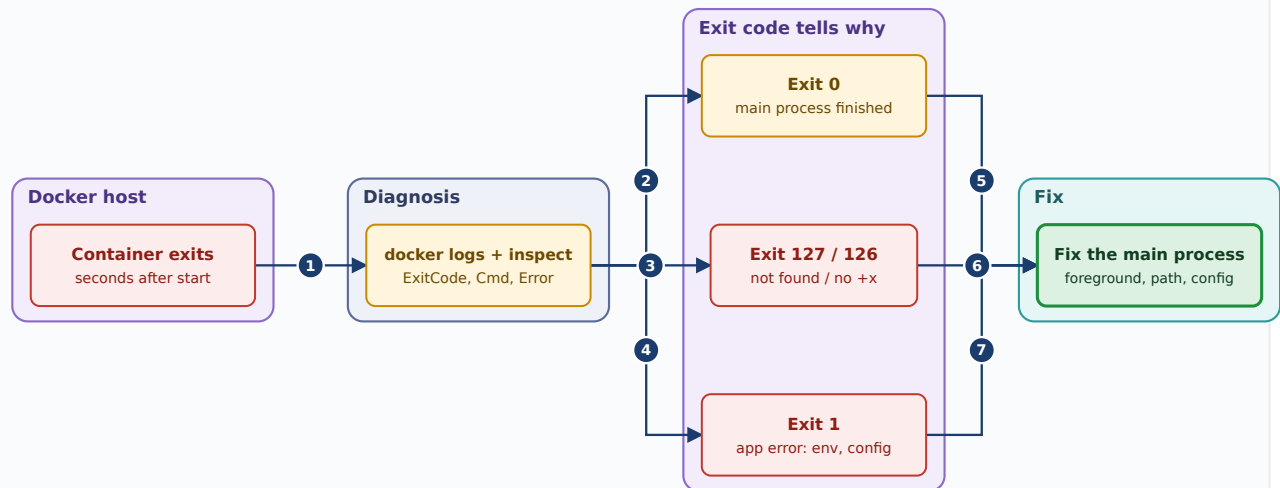
```

kubectl get nodes -L kubernetes.io/arch
docker buildx imagetools inspect europe-west1-docker.pkg.dev/proj/apps/api:1.4
docker buildx build --platform=linux/amd64,linux/arm64 \
  -t europe-west1-docker.pkg.dev/proj/apps/api:1.5 --push .
head -1 entrypoint.sh | od -c | head -2 # check shebang and \r

```

Q316**Troubleshooting**

`docker run -d myapp` prints a container ID, but `docker ps` shows nothing and the container is `Exited` seconds later. How do you debug it?

FLOW DIAGRAM

- ❶ Read the exit code and last output
- ❷ Service started as a daemon, or CMD is a one-off
- ❸ Wrong path, missing binary or no +x
- ❹ App crashed on missing settings

EXPLANATION

A container lives only as long as its main process (PID 1), so start with `docker ps -a`, `docker logs` and `docker inspect` to get the exit code and the exact command. Exit 0 means the process finished normally: a script that ends, or a server started in daemon mode such as `nginx` without `daemon off;`. Exit 127 or 126 means the command was not found or is not executable, often a wrong path, a missing shell in a distroless image, or a script without execute permission. Exit 1 usually means the app itself failed, for example a missing environment variable or config file, which the logs show; to look around, start a shell with `--entrypoint sh`.

KEY POINTS

- Run the service in the foreground as PID 1; do not use `tail -f /dev/null` hacks.
- Use `exec-form` CMD (`["node", "server.js"]`) so signals and errors reach the app.

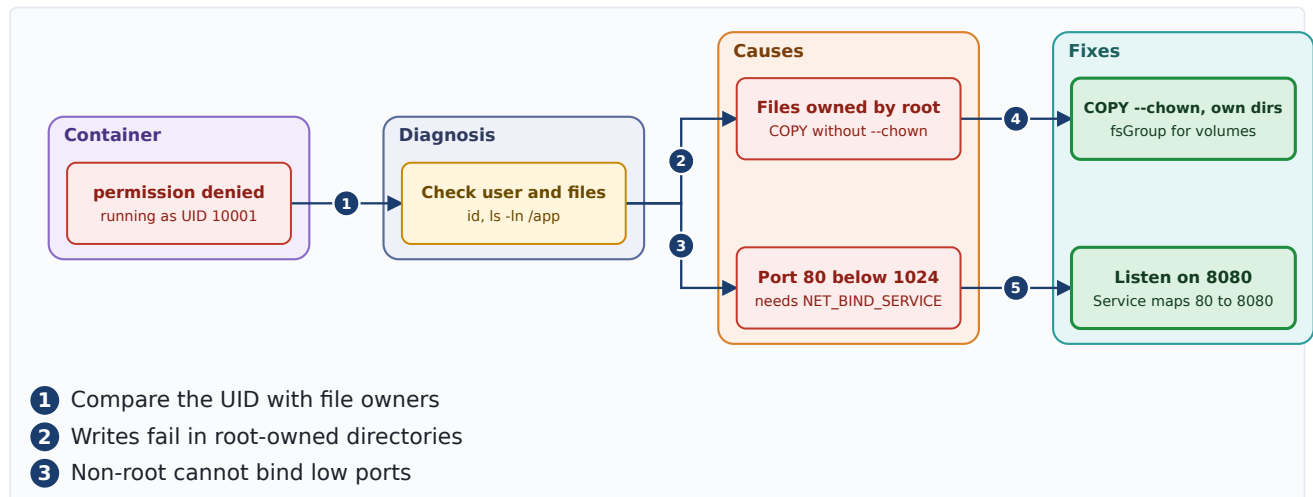
CONFIGURATION EXAMPLE

```

docker ps -a --filter name=myapp
docker logs myapp
docker inspect myapp --format '{{.State.ExitCode}} {{.State.Error}} {{.Config.Entrypoint}}
{{.Config.Cmd}}'
docker run -it --rm --entrypoint sh myapp:1.0
# nginx must stay in the foreground:
CMD ["nginx", "-g", "daemon off;"]
  
```

Q317**Troubleshooting**

After adding `USER 10001` to a Dockerfile, the container fails with `EACCES: permission denied, open '/app/logs/app.log'` and `listen tcp :80: bind: permission denied`. How do you fix it without going back to root?

FLOW DIAGRAM**EXPLANATION**

Files added with `COPY` or created by `RUN` belong to root unless you say otherwise, so a non-root user can read them but not write to them. Create the directories the app writes to and give them to the app user with `chown`, or use `COPY --chown=10001:10001`, and on Kubernetes set `fsGroup` so mounted volumes are writable. Ports below 1024 need the `NET_BIND_SERVICE` capability, so make the app listen on 8080 and let the Service or load balancer expose port 80. Check with `id` and `ls -ln` inside the image; with `readOnlyRootFilesystem`, write logs to stdout and temp files to an emptyDir.

KEY POINTS

- Use a numeric UID so `runAsNonRoot` can verify it.
- Log to stdout instead of files inside the container.

CONFIGURATION EXAMPLE

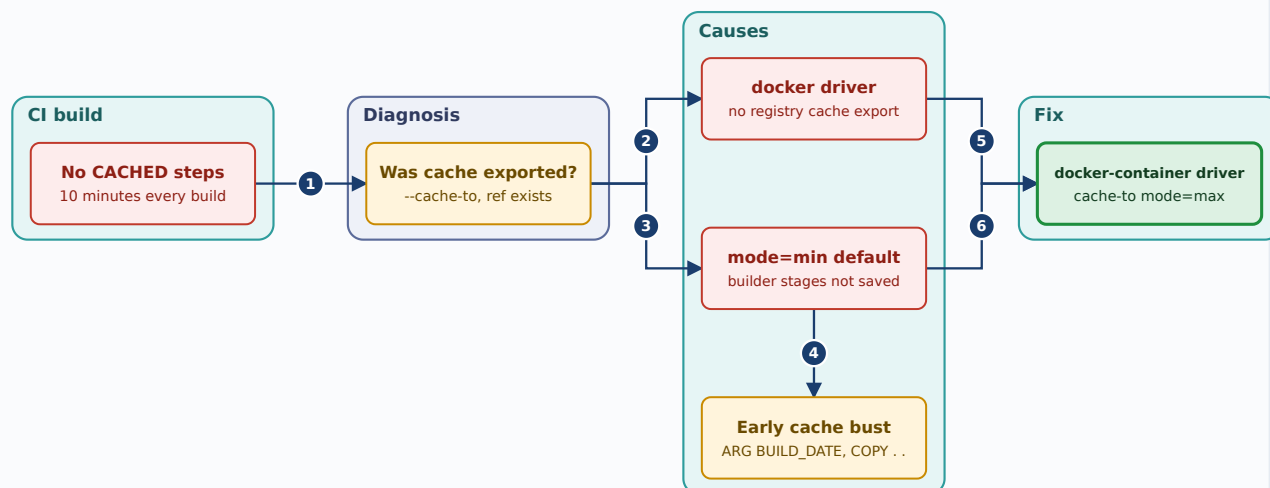
```

docker run --rm --entrypoint sh api:1.2 -c 'id; ls -ln /app'
RUN groupadd -g 10001 app && useradd -u 10001 -g app -s /sbin/nologin app \
  && mkdir -p /app/logs && chown -R 10001:10001 /app/logs
COPY --chown=10001:10001 . /app
USER 10001
EXPOSE 8080
# Kubernetes: securityContext: {runAsUser: 10001, runAsNonRoot: true, fsGroup: 10001}

```

Q318**Troubleshooting**

CI runs `docker buildx build --cache-from type=registry,...`, but the build log never shows `CACHED` steps and every build takes 10 minutes. How do you find out why the cache is ignored?

FLOW DIAGRAM

- ❶ `--cache-from` only reads what a build wrote
- ❷ `buildx ls` shows the driver
- ❸ Only final-stage layers were cached
- ❹ A changing value early busts all later steps

EXPLANATION

`--cache-from` only imports cache that an earlier build exported, so first check that builds also pass `--cache-to` and that the cache reference exists in the registry. The default `docker` `buildx` driver cannot export registry cache, so CI needs a builder created with `--driver docker-container`. The default `mode=min` saves only layers of the final image, so in a multi-stage build the expensive dependency stage is never cached; use `mode=max`. If the cache is imported but still missed, something changes early in the Dockerfile, such as an `ARG BUILD_DATE` or `COPY . .` before installing dependencies, and every later step rebuilds.

KEY POINTS

- Keep a separate `:buildcache` tag so cache and release images do not mix.
- Put values that change every build (version, date) in the last instructions.

CONFIGURATION EXAMPLE

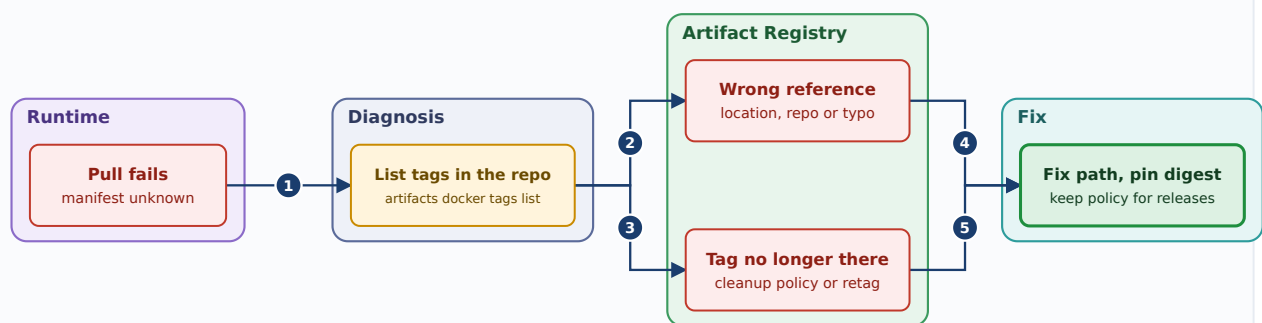
```

docker buildx ls
docker buildx create --use --name ci --driver docker-container
REPO=europe-west1-docker.pkg.dev/proj/apps
docker buildx build \
  --cache-from=type=registry,ref=$REPO/api:buildcache \
  --cache-to=type=registry,ref=$REPO/api:buildcache,mode=max \
  -t $REPO/api:$GIT_SHA --push .

```

Q319**Troubleshooting**

Deploys fail with `manifest unknown` (or `failed to resolve reference ... not found`) when pulling `api:1.8.2` from Artifact Registry, though the CI log shows the push succeeded last week. What do you check?

FLOW DIAGRAM

- ❶ Does this exact tag exist here?
- ❷ Pushed to a different repo or location
- ❸ Deleted by a cleanup policy

EXPLANATION

`manifest unknown` means the registry has no image for that name and tag, so compare the exact reference in the deployment with what `gcloud artifacts docker tags list` shows. Common causes are a different location (`us-docker.pkg.dev` vs `europe-west1-docker.pkg.dev`), project or repository, a typo or `v` prefix in the tag, or a CI job that pushed to another repository. If the tag existed before, check the repository's cleanup policies, which may have deleted it as an old version, or whether someone moved the tag to another digest. A different message, `no matching manifest for linux/amd64`, means the image exists but not for the node's architecture.

KEY POINTS

- Deploy by digest (`@sha256:...`) so a moved or deleted tag cannot change what runs.
- Add a keep rule for release tags in cleanup policies and dry-run policies first.

CONFIGURATION EXAMPLE

```

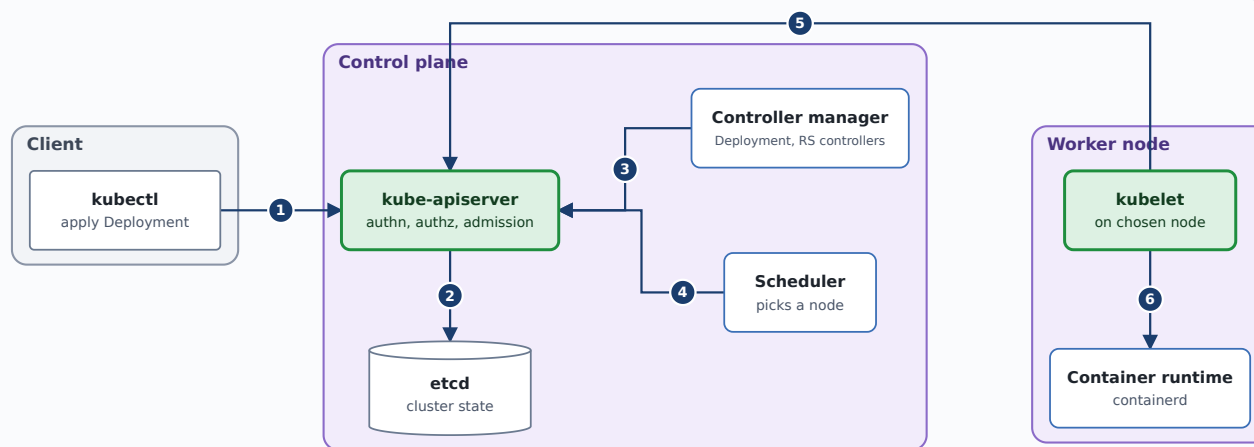
gcloud artifacts docker tags list europe-west1-docker.pkg.dev/proj/apps/api
gcloud artifacts docker images list europe-west1-docker.pkg.dev/proj/apps/api \
  --include-tags --sort-by=~UPDATE_TIME --limit=10
gcloud artifacts repositories describe apps --location=europe-west1 \
  --format="yaml(cleanupPolicies,cleanupPolicyDryRun)"
kubectl get deploy api -n shop -o jsonpath='{.spec.template.spec.containers[*].image}'
  
```

12. Kubernetes Core Concepts

The Kubernetes building blocks every GKE engineer is expected to explain and debug: control plane, workloads, networking, config, scheduling, storage and packaging.

Q320 Walk through what happens inside the cluster between `kubectl apply -f deployment.yaml` and a container running on a node.

FLOW DIAGRAM



- 1 Request validated and admitted
- 2 Deployment object persisted
- 3 Creates ReplicaSet, then Pods
- 4 Binds each Pod to a node
- 5 Watches Pods bound to its node
- 6 Pulls image, starts containers
- kube-proxy on each node programs Service rules

EXPLANATION

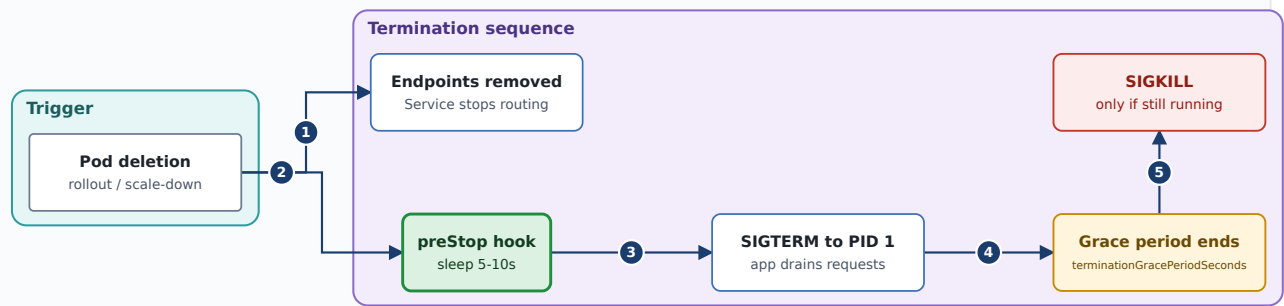
The API server authenticates and authorises the request, runs admission controllers, and stores the Deployment in etcd, which is the only component that talks to etcd. The Deployment controller in kube-controller-manager sees it through a watch and creates a ReplicaSet, whose controller creates Pod objects. The scheduler assigns each unscheduled Pod to a node based on resources, affinity and taints, and writes the binding back. The kubelet on that node watches for Pods bound to it, asks the container runtime (containerd) to pull images and start containers, and reports status, while kube-proxy or the dataplane programs Service routing.

KEY POINTS

- Everything goes through the API server; controllers reconcile via watches.
- Scheduler only decides placement; kubelet runs the pod.

Q321 Users see failed requests every time pods are replaced during a deploy.
Explain the pod termination sequence and how to make shutdown graceful.

FLOW DIAGRAM



- 1 Endpoint removal propagates to proxies
- 2 Runs in parallel with step 1
- 3 Then the container gets SIGTERM
- 4 App should exit before this
- 5 Forced kill

EXPLANATION

When a pod is deleted it is marked Terminating, and two things happen in parallel: it is removed from Service endpoints, and the kubelet runs any `preStop` hook and then sends SIGTERM to the container's PID 1. Because endpoint removal takes time to reach every kube-proxy and load balancer, a pod that exits immediately can still receive requests and fail them. Add a short `preStop` sleep, make the app stop accepting new work and finish in-flight requests on SIGTERM, and set `terminationGracePeriodSeconds` longer than the drain time. The container is killed with SIGKILL if it is still running when the grace period ends.

KEY POINTS

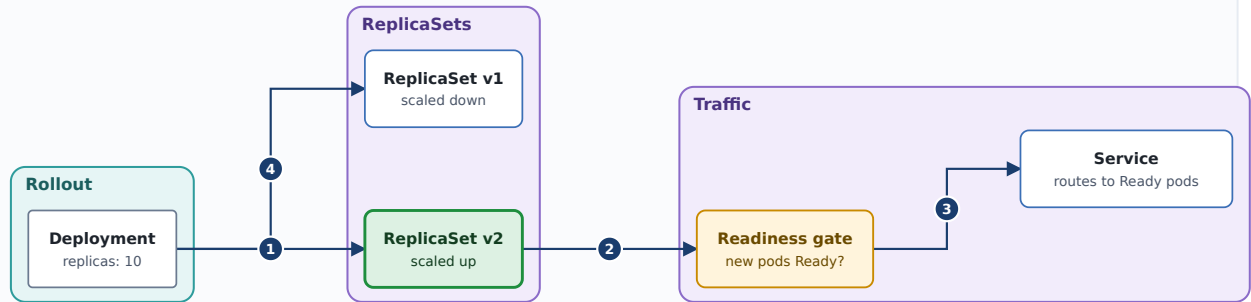
- Endpoint removal and SIGTERM race; `preStop` sleep covers the gap.
- Grace period includes the `preStop` hook time.

CONFIGURATION EXAMPLE

```
spec:
  terminationGracePeriodSeconds: 45
  containers:
  - name: web
    lifecycle:
      preStop:
        exec: {command: ["sleep", "10"]}
```

Q322 A Deployment of 10 replicas must update with no loss of capacity, and the cluster has little spare room. How do maxSurge and maxUnavailable control the rollout?

FLOW DIAGRAM



- ❶ maxSurge=2: up to 12 pods total
- ❷ Waits for readiness + minReadySeconds
- ❸ Ready pods join endpoints
- ❹ maxUnavailable=0: remove old only after new Ready

EXPLANATION

A rolling update creates a new ReplicaSet and shifts replicas from old to new. `maxSurge` is how many pods may exist above the desired count, and `maxUnavailable` is how many may be missing below it; both accept numbers or percentages and default to 25%. For no capacity loss set `maxUnavailable: 0` and `maxSurge: 1` or more, which needs spare room for the extra pods; if the cluster is full, allow `maxUnavailable: 1` instead and accept a small dip. Readiness probes and `minReadySeconds` decide when a new pod counts as available, so a bad readiness check stalls the rollout rather than breaking traffic.

KEY POINTS

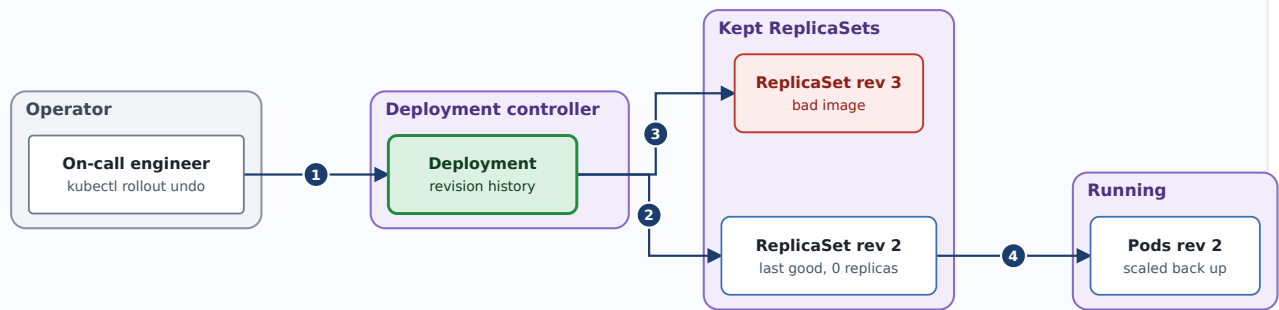
- `maxUnavailable 0` needs surge capacity.
- Rollout progress depends on readiness.

CONFIGURATION EXAMPLE

```
spec:
  replicas: 10
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 2, maxUnavailable: 0}
```


Q323 A new release is returning errors ten minutes after rollout. How do you roll back quickly, and how does Kubernetes keep previous versions?

FLOW DIAGRAM



- ❶ `undo --to-revision=2`
- ❷ Old ReplicaSet scaled up
- ❸ Bad one scaled to 0
- ❹ Normal rolling update rules apply

EXPLANATION

Each change to the pod template creates a new ReplicaSet, and old ReplicaSets are kept scaled to zero up to `revisionHistoryLimit` (default 10). `kubectl rollout undo deployment/api` rolls back to the previous revision, or `--to-revision=N` to a specific one, using the same rolling update strategy. Check history with `kubectl rollout history` and progress with `kubectl rollout status`. In a GitOps setup, revert the commit as well, or the reconciler will reapply the bad version.

KEY POINTS

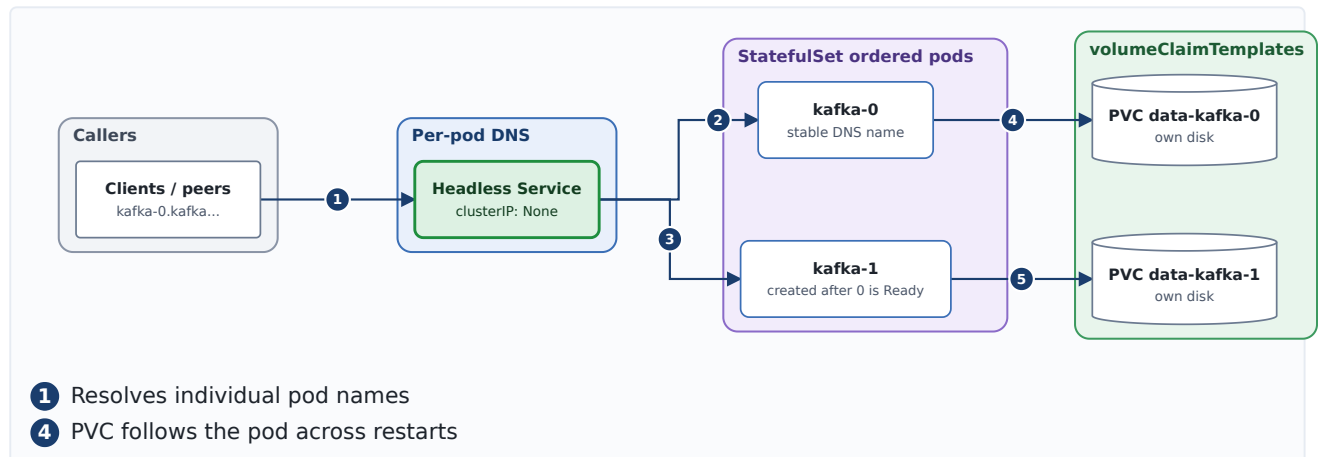
- Rollback = scale an old ReplicaSet back up.
- With GitOps, roll back in Git too.

CONFIGURATION EXAMPLE

```
kubectl rollout history deployment/api -n shop
kubectl rollout undo deployment/api -n shop --to-revision=2
kubectl rollout status deployment/api -n shop --timeout=5m
```

Q324 You must run a 3-node Kafka or ZooKeeper-style cluster on Kubernetes where each member needs a stable name and its own disk. Why a StatefulSet and how does it behave?

FLOW DIAGRAM



EXPLANATION

A StatefulSet gives each pod a stable ordinal name (`kafka-0` , `kafka-1`), a stable DNS entry through a headless Service, and its own PersistentVolumeClaim from `volumeClaimTemplates` that reattaches when the pod is rescheduled. Pods are created, scaled and by default updated in order, which suits systems that need stable identity and quorum. Deleting or scaling down a StatefulSet does not delete its PVCs by default, which protects data but needs cleanup. Use a PodDisruptionBudget and anti-affinity across zones, and consider a managed service or an operator when one exists.

KEY POINTS

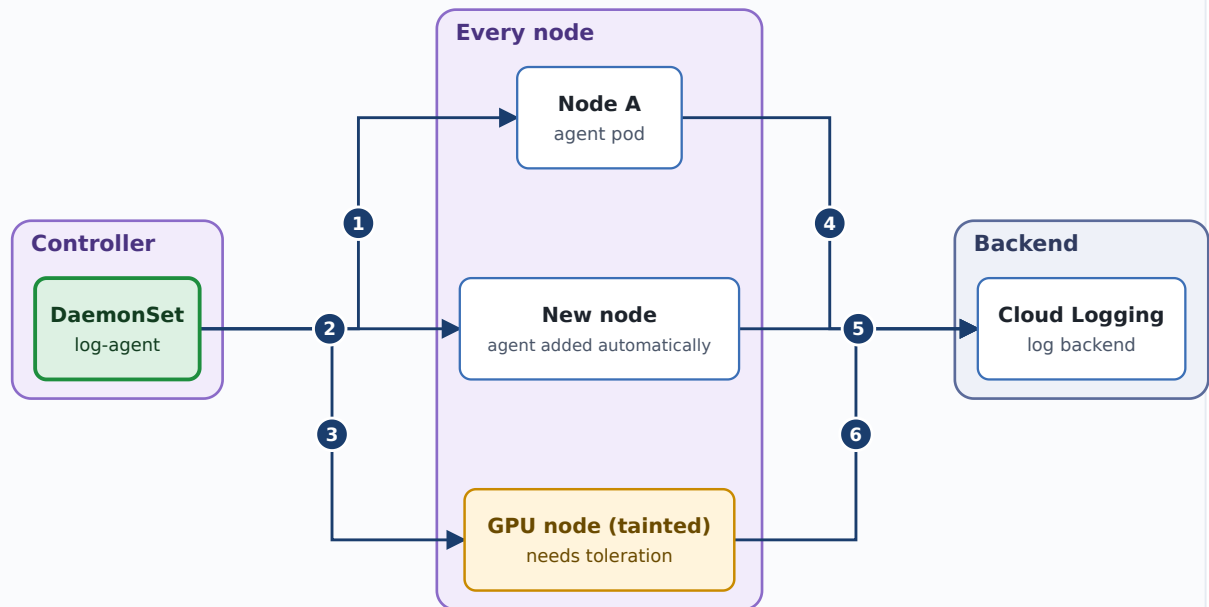
- Stable name + stable storage + ordered operations.
- PVCs outlive the pods by default.

CONFIGURATION EXAMPLE

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: kafka}
spec:
  serviceName: kafka
  replicas: 3
  selector: {matchLabels: {app: kafka}}
  volumeClaimTemplates:
  - metadata: {name: data}
    spec: {accessModes: [ReadWriteOnce], resources: {requests: {storage: 100Gi}}}
```

Q325 A log shipper must run exactly one pod on every node, including new nodes added by the autoscaler and tainted GPU nodes. How do you deploy it?

FLOW DIAGRAM



- 1 One pod per matching node
- 2 Scheduled when node joins
- 3 Tolerations let it run on tainted nodes

EXPLANATION

A DaemonSet runs one copy of a pod on every node that matches its selector, adds a pod when a node joins and removes it when the node leaves, which suits log agents, node exporters and CNI or CSI plugins. To reach tainted nodes such as GPU or Spot pools, add matching tolerations, or a broad operator: Exists toleration for agents that must run everywhere. Use nodeSelector or node affinity to limit it to a subset of nodes. Give it small, fixed requests and a high priority class so it is not evicted first under pressure.

KEY POINTS

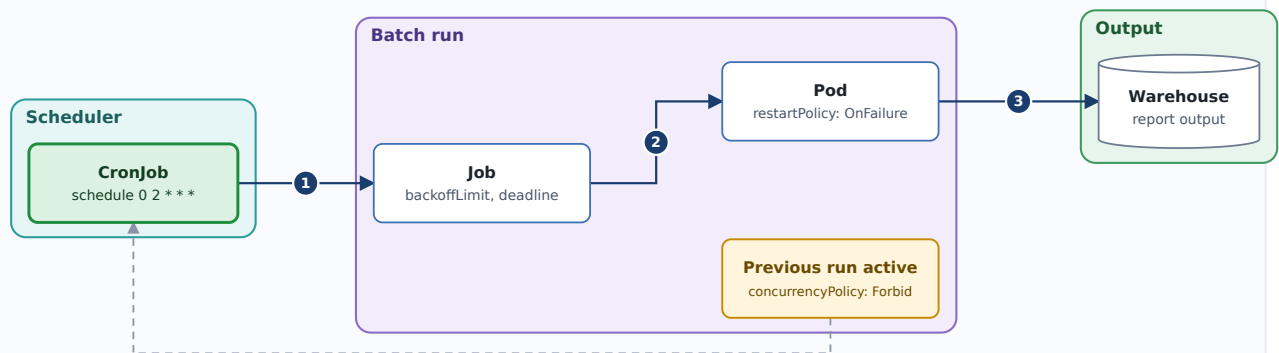
- One pod per node, follows node lifecycle.
- Tolerations decide whether it reaches tainted nodes.

CONFIGURATION EXAMPLE

```
spec:
  template:
    spec:
      priorityClassName: system-node-critical
      tolerations:
        - operator: Exists
      containers:
        - name: agent
          image: fluent/fluent-bit:3.0
```

Q326 A nightly report CronJob sometimes runs twice at once and sometimes leaves failed pods piling up. How do you configure Jobs and CronJobs correctly?

FLOW DIAGRAM



- 1 Creates a Job per schedule tick
- 2 Retries up to `backoffLimit`
- 3 Idempotent write
- New run skipped while old one runs

EXPLANATION

A Job runs pods until a set number complete successfully, retrying failures up to `backoffLimit`, and `activeDeadlineSeconds` caps total runtime. A CronJob creates Jobs on a schedule; set `concurrencyPolicy: Forbid` to skip a run while the previous one is still active, or `Replace` to cancel the old one. Limit leftovers with `successfulJobsHistoryLimit`, `failedJobsHistoryLimit` and `ttlSecondsAfterFinished`. Set `timeZone` if the schedule must follow local time, and make the job idempotent because CronJobs can occasionally create zero or two Jobs for one tick.

KEY POINTS

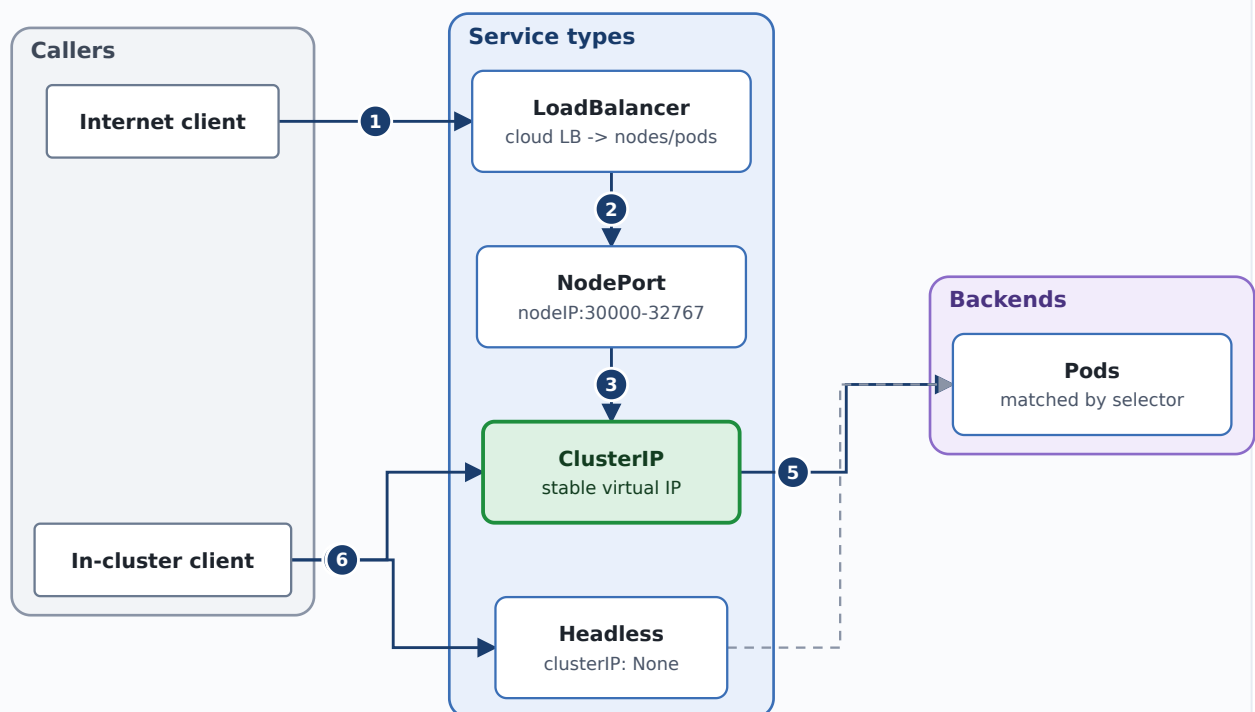
- `concurrencyPolicy` prevents overlapping runs.
- Jobs must be idempotent.

CONFIGURATION EXAMPLE

```
spec: # CronJob nightly-report
  schedule: "0 2 * * *"
  timeZone: "Europe/London"
  concurrencyPolicy: Forbid
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      activeDeadlineSeconds: 3600
      template: {spec: {restartPolicy: OnFailure, containers: [{name: report, image: report:1.3}]}}
```

Q327 Explain ClusterIP, NodePort, LoadBalancer and headless Services, and which you would use for an internal API, a public endpoint and a database cluster.

FLOW DIAGRAM



- 1 Public or internal LB IP
- 2 LoadBalancer builds on NodePort (or NEG)
- 3 NodePort builds on ClusterIP
- 4 Internal API calls
- 5 kube-proxy / eBPF load-balances
- 6 DNS lookup returns pod IPs
- Clients connect to pods directly

EXPLANATION

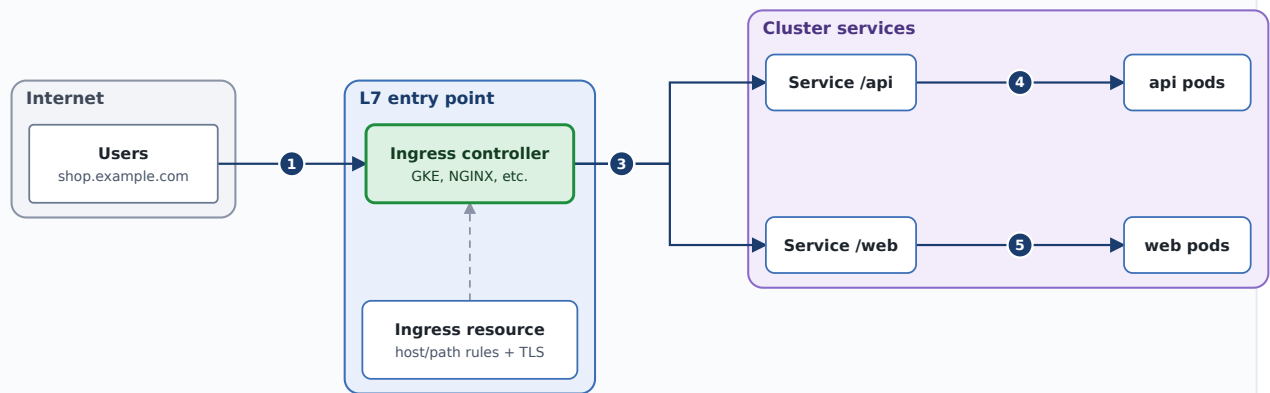
ClusterIP gives a stable virtual IP and DNS name reachable only inside the cluster, which is right for an internal API. NodePort also opens a port in the 30000-32767 range on every node, and LoadBalancer additionally provisions a cloud load balancer, so a public endpoint uses LoadBalancer, or better an Ingress or Gateway for HTTP. A headless Service (`clusterIP: None`) has no virtual IP; DNS returns the individual pod IPs, which StatefulSet members and client-side load balancing such as database drivers need. All types select pods by label and route only to Ready endpoints.

KEY POINTS

- Types build on each other: ClusterIP < NodePort < LoadBalancer.
- Headless = DNS to pod IPs, no load balancing.

Q328 You have five HTTP services and do not want five external load balancers. How does an Ingress solve this and what does it need to work?

FLOW DIAGRAM



- 1 One IP, TLS terminated
- 2 Path /api
- 3 Path /
- Controller watches Ingress objects

EXPLANATION

An Ingress is a set of HTTP routing rules (hosts, paths, TLS secrets) that an Ingress controller implements, so one load balancer can front many Services. The resource does nothing without a controller; on GKE the built-in controller creates a Google Application Load Balancer, while NGINX or others run as pods behind a single LoadBalancer Service. Choose the controller with `ingressClassName`, and store certificates as TLS Secrets or use managed certificates. Gateway API is the newer, more expressive successor and is worth using for new designs.

KEY POINTS

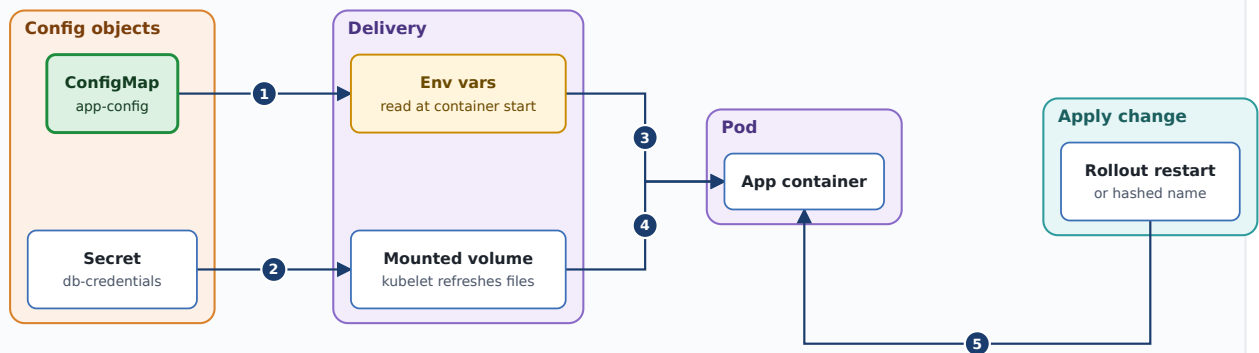
- No controller = no effect.
- Gateway API supersedes Ingress for new work.

CONFIGURATION EXAMPLE

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: shop}
spec:
  ingressClassName: nginx
  tls: [{hosts: [shop.example.com], secretName: shop-tls}]
  rules:
    - host: shop.example.com
      http:
        paths:
          - {path: /api, pathType: Prefix, backend: {service: {name: api, port: {number: 8080}}}}
```

Q329 A team updated a ConfigMap but the app kept using old values. How do ConfigMaps and Secrets reach pods and when do changes apply?

FLOW DIAGRAM



- 1 envFrom / valueFrom
- 2 Files under /etc/secret
- 3 Fixed until restart
- 4 Updated after a delay, not with subPath
- 5 Restart picks up new env values

EXPLANATION

ConfigMaps hold non-sensitive configuration and Secrets hold sensitive values (base64-encoded, encrypted at rest only if configured); both can be exposed as environment variables or mounted files. Environment variables are read when the container starts and never change, so the pod must be restarted. Mounted volumes are refreshed by the kubelet after a short delay, except when mounted with `subPath`, but the app must reload the file itself. The usual pattern is to trigger a rollout on change, using a config hash annotation (Helm) or a hashed name from Kustomize's `configMapGenerator`.

KEY POINTS

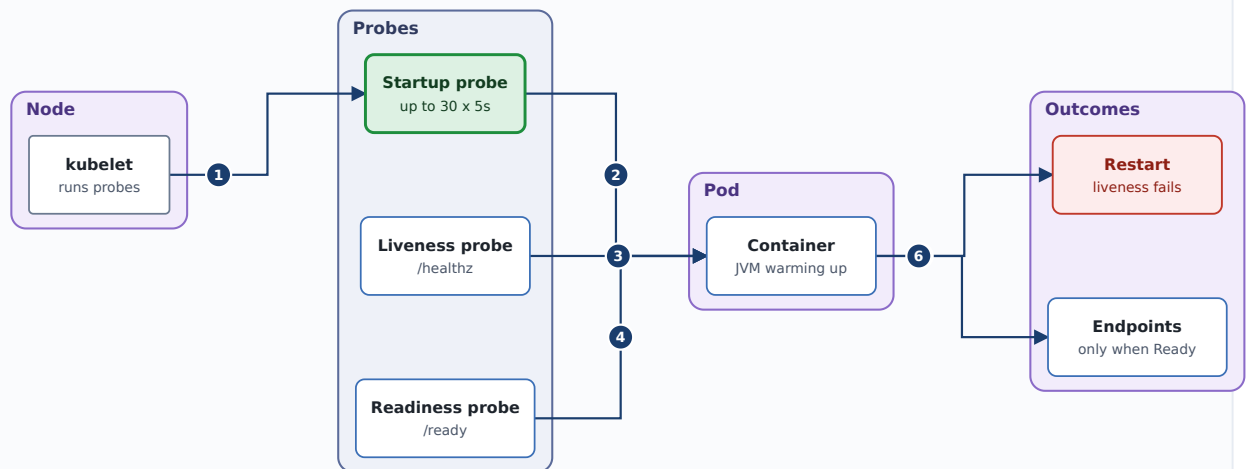
- Env vars need a restart; volumes update but app must reload.
- Hash-named ConfigMaps give automatic rollouts.

CONFIGURATION EXAMPLE

```
kubectl create configmap app-config --from-file=config.yaml \
-o yaml --dry-run=client | kubectl apply -f -
kubectl rollout restart deployment/api
```

Q330 A Java service takes 90 seconds to start and gets killed in a restart loop by its liveness probe. How should liveness, readiness and startup probes be set?

FLOW DIAGRAM



- 1 Runs first; others wait until it succeeds
- 3 Checks only after startup passes
- 5 Hung process restarted
- 6 Traffic only when ready

EXPLANATION

Add a startup probe with `failureThreshold` x `periodSeconds` longer than the worst-case boot time (for example $30 \times 5s = 150s$); liveness and readiness probes do not run until it succeeds, so slow starts no longer trigger restarts. Liveness should check only that the process itself is responsive, because failing it restarts the container, while readiness controls whether the pod receives Service traffic and can include critical dependencies. Keep probe endpoints cheap and set sensible timeouts. Never point liveness at a database or downstream service, or an outage there restarts every pod.

KEY POINTS

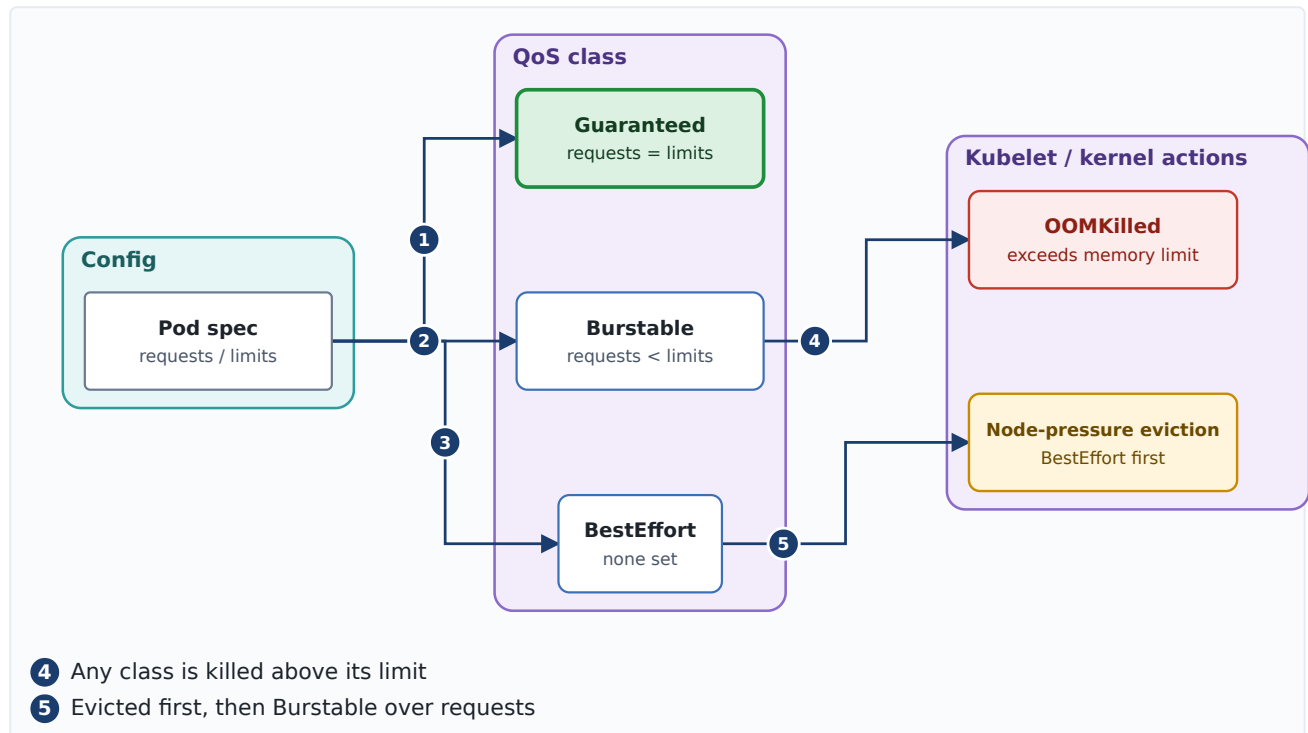
- Startup probe protects slow boots.
- Liveness = restart, readiness = traffic.

CONFIGURATION EXAMPLE

```
startupProbe:
  httpGet: {path: /healthz, port: 8080}
  failureThreshold: 30
  periodSeconds: 5
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 10
readinessProbe:
  httpGet: {path: /ready, port: 8080}
```


Q331 Pods of a memory-heavy service are repeatedly OOMKilled and some others get evicted when a node is under pressure. Explain requests, limits and QoS classes.

FLOW DIAGRAM



EXPLANATION

Requests are what the scheduler reserves on a node, and limits are the ceiling enforced at runtime: CPU above the limit is throttled, while memory above the limit makes the kernel OOM-kill the container (exit code 137, reason OOMKilled). QoS class follows from the settings: Guaranteed when every container has equal requests and limits, Burstable when some requests are set, BestEffort when none are. Under node memory pressure the kubelet evicts BestEffort pods first, then Burstable pods using more than their requests. Set memory requests close to real usage, give the limit headroom (or equal it for Guaranteed), and check JVM or runtime heap settings against the limit.

KEY POINTS

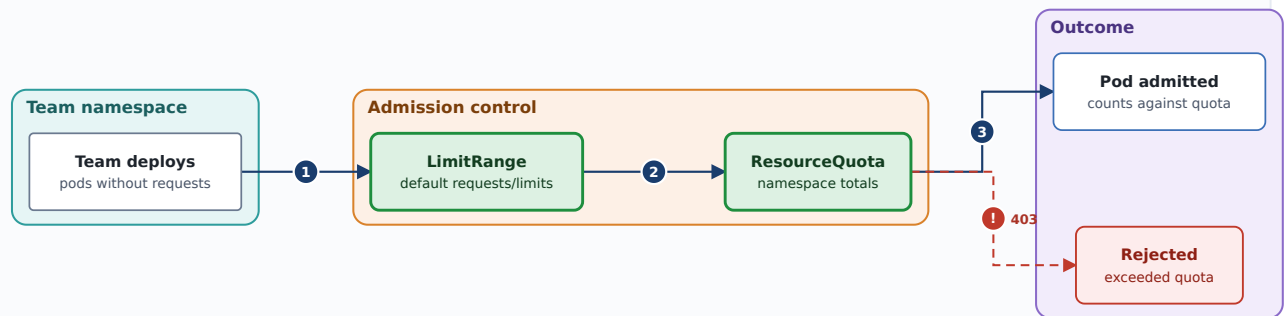
- Memory over limit = OOMKilled; CPU over limit = throttled.
- QoS class decides eviction order.

CONFIGURATION EXAMPLE

```
resources:
  requests: {cpu: 500m, memory: 1Gi}
  limits: {memory: 1Gi}
# kubectl describe pod api-xyz | grep -A3 "Last State"
```

Q332 In a shared cluster one team's namespace consumed most of the capacity, and some of their pods had no requests at all. How do you enforce fair limits per namespace?

FLOW DIAGRAM



- ❶ Missing requests filled with defaults
- ❷ Checks sum of requests/limits
- ❸ Within budget
- ❗ 403 forbidden: exceeded quota

EXPLANATION

A ResourceQuota caps the total requests, limits, object counts and storage a namespace may use, and once it covers CPU or memory, every pod must declare those values or it is rejected. A LimitRange sets default requests and limits for containers that omit them, plus minimum and maximum values per container, so teams cannot deploy unbounded pods. Together they turn a shared cluster into fair shares per team. Pods rejected by quota fail at creation, so watch ReplicaSet events rather than looking for Pending pods.

KEY POINTS

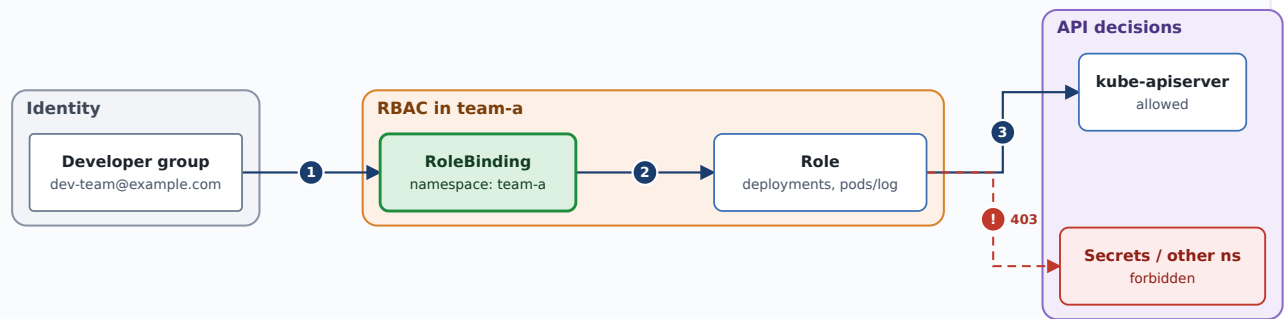
- LimitRange = per-container defaults and bounds.
- ResourceQuota = namespace totals.

CONFIGURATION EXAMPLE

```
apiVersion: v1
kind: ResourceQuota
metadata: {name: team-a, namespace: team-a}
spec:
  hard:
    requests.cpu: "40"
    requests.memory: 160Gi
    limits.memory: 200Gi
    pods: "200"
```

Q333 Developers should deploy and read logs in their own namespace but not read Secrets or touch other namespaces. How do you set this up with RBAC?

FLOW DIAGRAM



- ❶ Group subject bound in namespace
- ❷ Grants only listed verbs/resources
- ❸ get/list/create/update allowed
- ❗ 403 no rule matches

EXPLANATION

RBAC is additive: a Role lists allowed verbs on resources within one namespace, and a RoleBinding grants that Role to users, groups or service accounts in that namespace; ClusterRoles and ClusterRoleBindings do the same cluster-wide. Create a Role in `team-a` that allows managing Deployments, Services and ConfigMaps and reading pods and `pods/log`, but omit Secrets, and bind it to the team's group. Because there are no deny rules, access to other namespaces and Secrets is simply never granted. On GKE, IAM also applies, so give developers only minimal project roles such as `roles/container.clusterViewer` and let RBAC control namespace access.

KEY POINTS

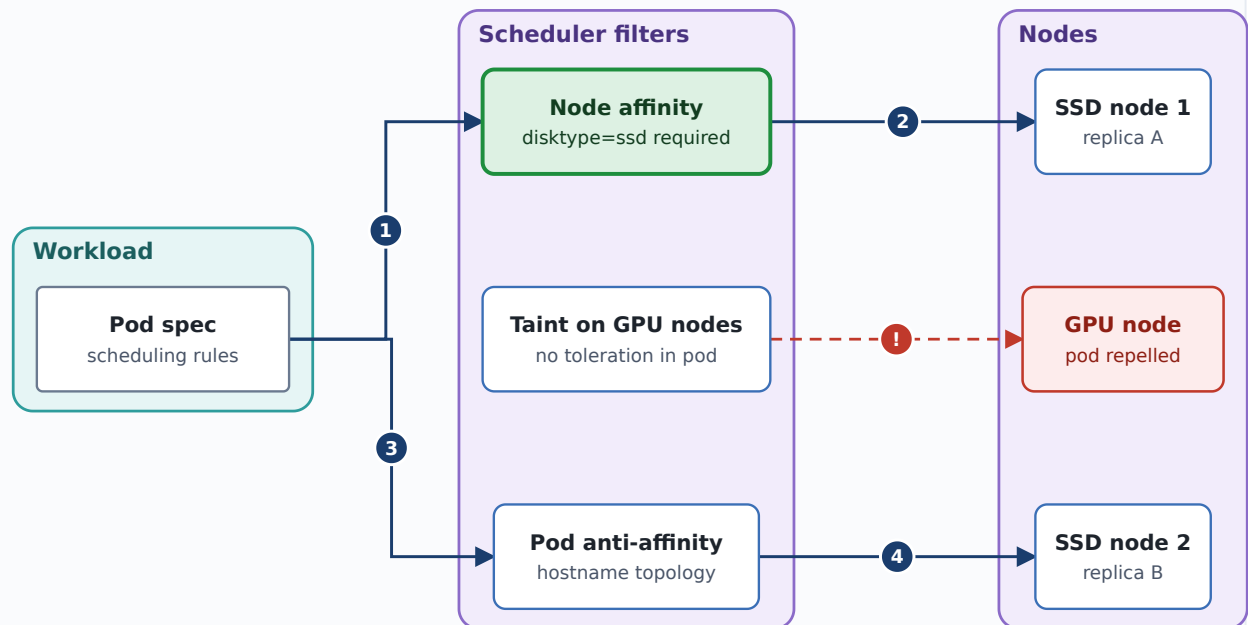
- Role + RoleBinding = namespace scope; no deny rules exist.
- Test with `kubectl auth can-i --as.`

CONFIGURATION EXAMPLE

```
kubectl create role deployer -n team-a \
  --verb=get,list,watch,create,update,patch \
  --resource=deployments,services,configmaps,pods,pods/log
kubectl create rolebinding deployer -n team-a \
  --role=deployer --group=dev-team@example.com
kubectl auth can-i get secrets -n team-a \
  --as=alice@example.com --as-group=dev-team@example.com
```

Q334 A service must run on SSD-backed nodes, never on the GPU pool, and its replicas must be spread so one node failure does not take out more than one. How do you express this?

FLOW DIAGRAM



- ② Only nodes labelled `disktype=ssd`
- ④ Second replica on another node
- ! NoSchedule taint, no toleration

EXPLANATION

Taints are set on nodes to repel pods, and only pods with a matching toleration can land there, so tainting the GPU pool keeps this service off it. Node affinity attracts pods to nodes by label, and `requiredDuringSchedulingIgnoredDuringExecution` makes `disktype=ssd` a hard rule. Pod anti-affinity with `topologyKey: kubernetes.io/hostname` keeps replicas on different nodes; use the zone key or topology spread constraints to spread across zones. Hard rules can leave pods Pending when capacity is short, so use preferred rules where a soft preference is enough.

KEY POINTS

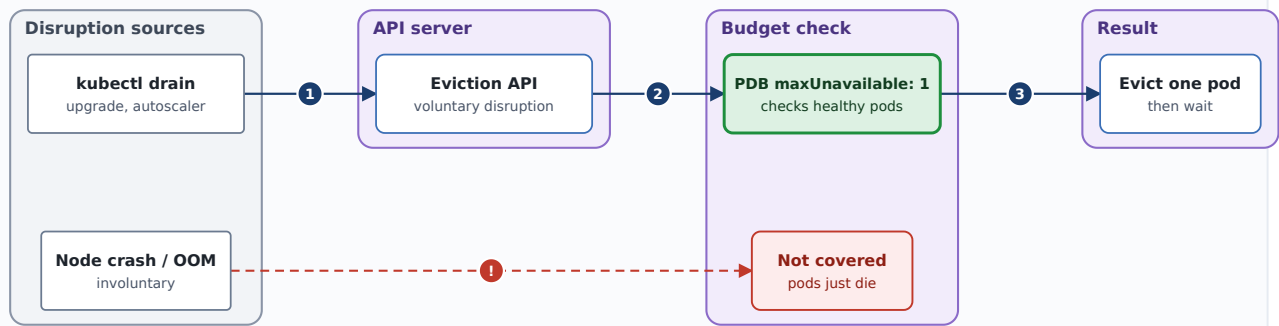
- Taint/toleration repels; affinity attracts.
- Anti-affinity or topology spread for failure isolation.

CONFIGURATION EXAMPLE

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions: [{key: disktype, operator: In, values: [ssd]}]
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector: {matchLabels: {app: api}}
        topologyKey: kubernetes.io/hostname
```

Q335 What does a PodDisruptionBudget protect against, what does it not protect against, and how do you size it?

FLOW DIAGRAM



- ❶ Drain uses evictions
- ❷ Allowed only if budget remains
- ❸ Next eviction after pod is Ready again
- ❗ PDBs ignore involuntary failures

EXPLANATION

A PDB limits how many pods of an application can be down at once during voluntary disruptions that use the Eviction API, such as node drains for upgrades, cluster autoscaler scale-down and maintenance. It does not prevent involuntary disruptions like node crashes or OOM kills, and a direct `kubectl delete pod` bypasses it. Size it with `maxUnavailable: 1` or a percentage for most apps, or `minAvailable` for quorum systems, and always leave at least one allowed disruption. A PDB that allows zero disruptions blocks drains and upgrades until they time out or force past it.

KEY POINTS

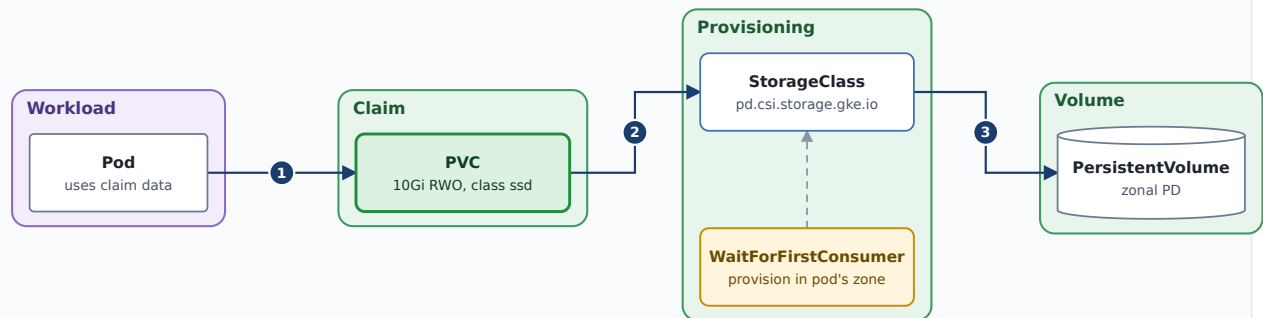
- Only voluntary, eviction-based disruptions are covered.
- Always leave allowed disruptions > 0.

CONFIGURATION EXAMPLE

```
kubectl get pdb -A
# NAME      MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS
# api-pdb   N/A            1                1
```

Q336 A developer creates a PVC for a database and it stays Pending, and later a pod cannot start in another zone. Explain PV, PVC and StorageClass and how dynamic provisioning works.

FLOW DIAGRAM



- 1 Pod references the claim
- 2 Class chooses provisioner and params
- 3 CSI driver creates the disk
- Waits until the pod is scheduled

EXPLANATION

A PersistentVolumeClaim is a request for storage with a size, access mode and StorageClass; the StorageClass names a CSI provisioner and parameters, and dynamic provisioning creates a matching PersistentVolume and binds it. A claim stays Pending if the class does not exist, the provisioner fails, or the class uses `WaitForFirstConsumer` and no pod uses it yet, which is expected. Zonal disks can attach only in their zone, which is why a pod rescheduled elsewhere cannot start; `WaitForFirstConsumer` provisions in the pod's zone, and regional PDs survive a zone loss. `reclaimPolicy: Retain` keeps the disk when the claim is deleted.

KEY POINTS

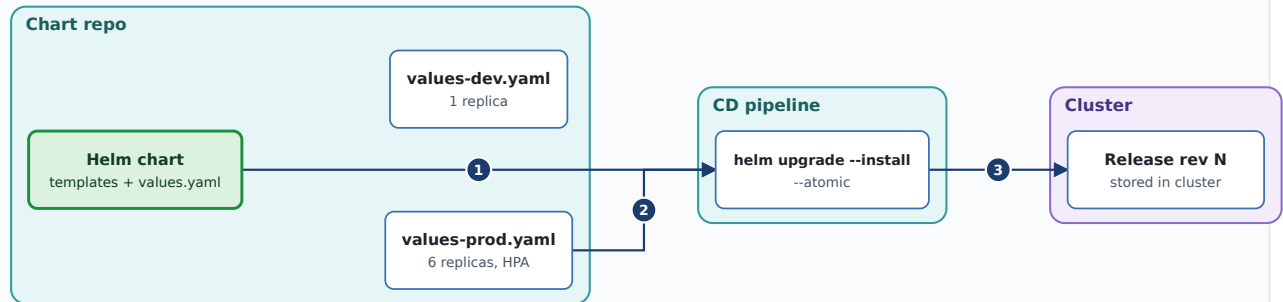
- `WaitForFirstConsumer` avoids zone mismatches.
- `ReadWriteOnce` disks attach to one node at a time.

CONFIGURATION EXAMPLE

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: ssd}
provisioner: pd.csi.storage.gke.io
parameters: {type: pd-balanced}
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain
allowVolumeExpansion: true
```

Q337 The same application is deployed to dev, staging and prod with copy-pasted YAML. How would a Helm chart with values files improve this, and how do you deploy it safely?

FLOW DIAGRAM



- ❶ Templates rendered with values
- ❷ -f overrides defaults
- ❸ Upgrade, roll back on failure

EXPLANATION

A Helm chart turns the manifests into templates with defaults in `values.yaml`, and each environment supplies a small values file with only what differs, such as replica count, resources and hostnames. `helm upgrade --install` creates or upgrades a release, stores its revision history in the cluster, and `--atomic` rolls back automatically if the upgrade fails. Lint and render charts in CI with `helm lint` and `helm template`, and pin chart versions. Keep secrets out of values files and pull them from Secret Manager instead.

KEY POINTS

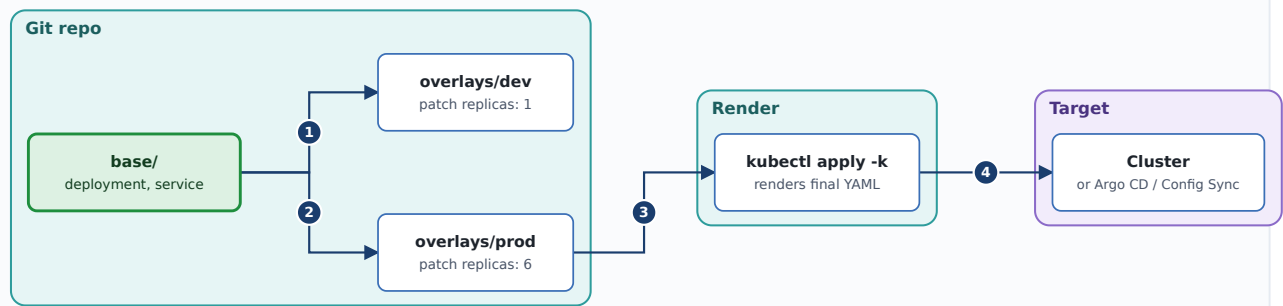
- One chart, per-environment values files.
- `--atomic --wait` for safe upgrades.

CONFIGURATION EXAMPLE

```
helm lint ./charts/api
helm upgrade --install api ./charts/api -n shop \
  -f charts/api/values-prod.yaml --set image.tag=$GIT_SHA \
  --atomic --wait --timeout 10m
```

Q338 A team prefers plain YAML over templates. How do Kustomize bases and overlays handle environment differences?

FLOW DIAGRAM



- ❶ Overlay references base
- ❸ Patches, images, labels applied
- ❹ Plain manifests applied

EXPLANATION

Kustomize keeps a base of ordinary manifests and applies overlays per environment that patch fields, set namespaces, labels and image tags, and generate ConfigMaps or Secrets, without any templating language. `kubectl apply -k overlays/prod` or `kustomize build` renders the final YAML, and GitOps tools such as Argo CD and Config Sync understand Kustomize directly. Generated ConfigMaps get a content hash in their name, so changes trigger rollouts automatically. Kustomize suits in-house apps, while Helm is better for distributing reusable, heavily parameterised packages.

KEY POINTS

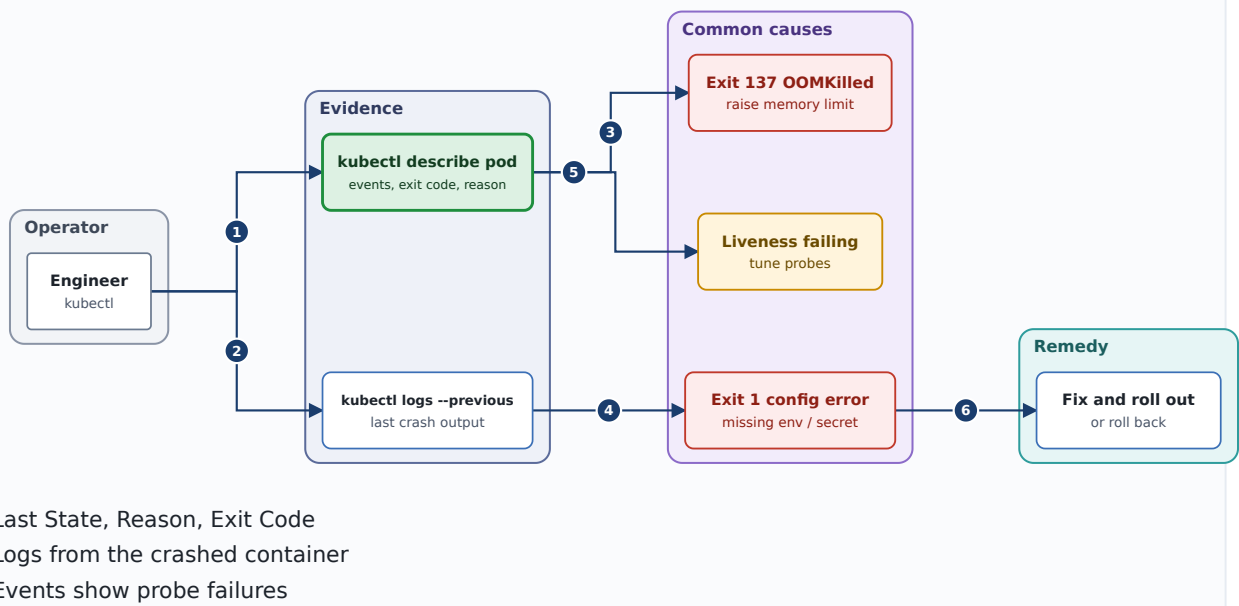
- No templates: patches over a shared base.
- `configMapGenerator` adds hash suffixes for rollouts.

CONFIGURATION EXAMPLE

```
# overlays/prod/kustomization.yaml
resources: [../../base]
namespace: shop-prod
images:
- {name: api, newName: europe-west1-docker.pkg.dev/proj/apps/api, newTag: "1.4.2"}
patches:
- target: {kind: Deployment, name: api}
  patch: '[{"op": "replace", "path": "/spec/replicas", "value": 6}]'
```


Q339 A pod is in CrashLoopBackOff after a deploy. Walk through how you debug it.

FLOW DIAGRAM



EXPLANATION

CrashLoopBackOff means the container starts and exits repeatedly, and the kubelet waits longer between restarts. Start with `kubectl describe pod` for events, Last State, reason and exit code: 137 with OOMKilled points to the memory limit, 1 or other app codes point to application errors, and probe failure events point to liveness settings. Then run `kubectl logs --previous` to see the output of the crashed instance, which often shows missing config, bad credentials or unreachable dependencies. If the image has no shell, use `kubectl debug` with an ephemeral container, and roll back if the fix is not quick.

KEY POINTS

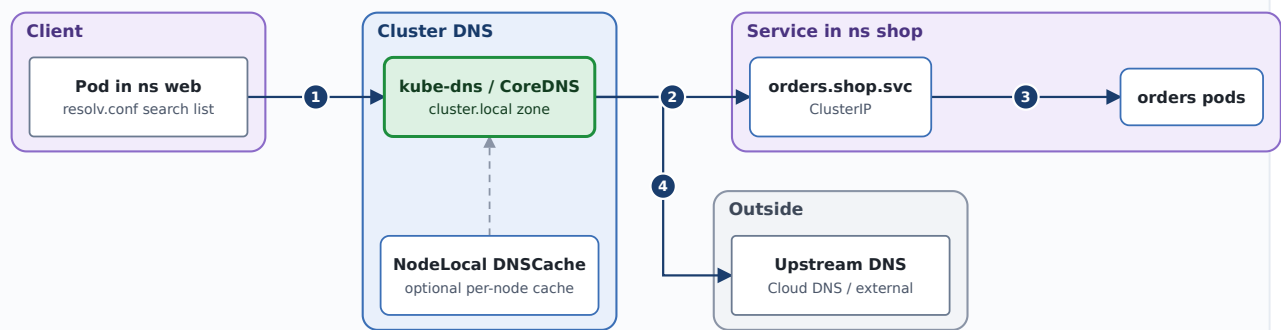
- describe for exit code/events, logs --previous for app output.
- 137 = killed (often OOM), 1 = app error.

CONFIGURATION EXAMPLE

```
kubectl describe pod api-7d9c -n shop
kubectl logs api-7d9c -n shop --previous
kubectl get events -n shop --sort-by=.lastTimestamp | tail -20
kubectl debug -it api-7d9c -n shop --image=busybox:1.36 --target=api
```

Q340 An app in namespace `web` cannot reach `orders` in namespace `shop` using the name `orders`, and DNS lookups feel slow. How does cluster DNS work?

FLOW DIAGRAM



- 1 orders -> orders.web.svc... fails
- 2 orders.shop resolves
- Caches on node, fewer timeouts
- 4 External names forwarded

EXPLANATION

Each Service gets a DNS name `<service>.<namespace>.svc.cluster.local`, and a pod's `resolv.conf` search list includes its own namespace first, so a bare `orders` only resolves within `web`. Use `orders.shop` or the full name to reach another namespace. The default `ndots:5` means names with fewer dots are tried against every search domain first, which multiplies lookups for external names; use fully qualified names with a trailing dot or lower `ndots` in `dnsConfig` for chatty clients. On GKE, NodeLocal DNSCache or Cloud DNS for GKE reduces latency and load on kube-dns.

KEY POINTS

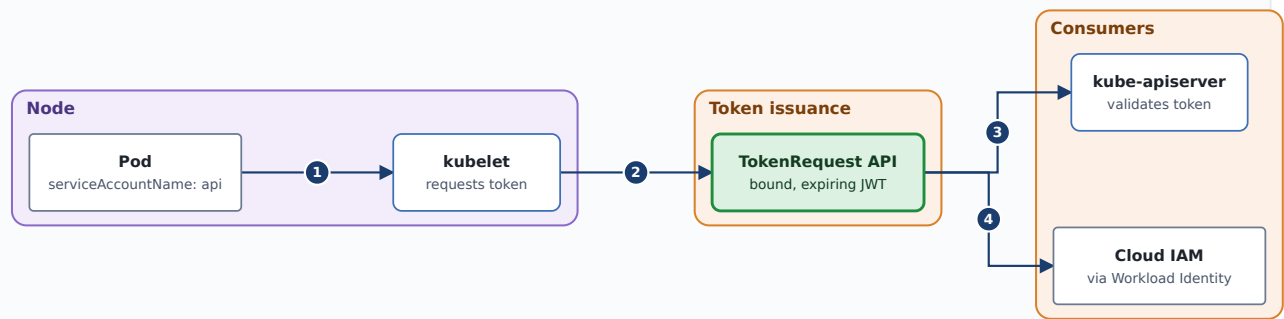
- Cross-namespace: `service.namespace`.
- `ndots:5` causes extra lookups for external names.

CONFIGURATION EXAMPLE

```
kubectrl run dnstest -n web --rm -it --image=busybox:1.36 -- \
nslookup orders.shop.svc.cluster.local
```

Q341 Pods mount a service-account token they never use, and an audit asks how Kubernetes service account tokens work today. Explain and harden it.

FLOW DIAGRAM



- 1 Projected token volume
- 2 Audience + expiry set
- 3 Auth as `system:serviceaccount:ns:api`
- 4 Federated for Google APIs
- `automountServiceAccountToken: false` when a pod never calls the API

EXPLANATION

Every pod runs as a Kubernetes service account, `default` unless set, and the kubelet mounts a projected token obtained from the TokenRequest API that is bound to the pod, has an audience and expires, and the kubelet rotates it automatically. Long-lived token Secrets are no longer created automatically for service accounts in current Kubernetes versions. Harden by creating a dedicated service account per workload with minimal RBAC, setting `automountServiceAccountToken: false` for pods that never call the API, and never granting permissions to the `default` account. On GKE, the same identity maps to Google IAM through Workload Identity, so cloud access follows the same per-workload design.

KEY POINTS

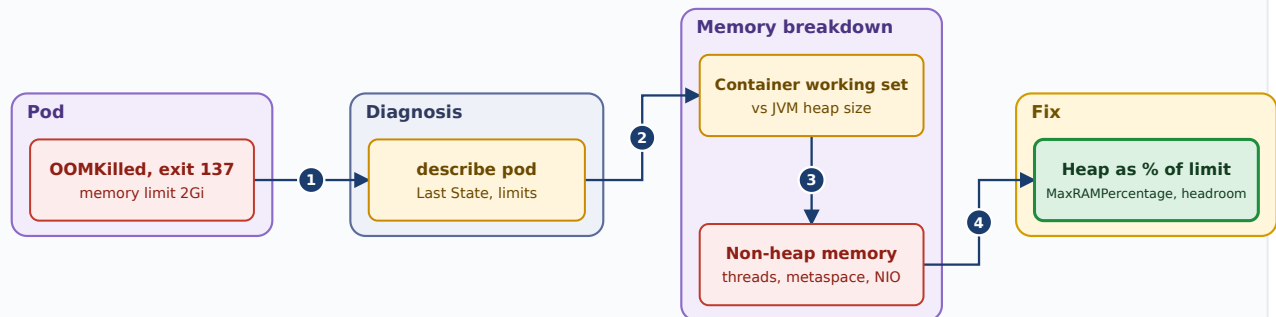
- Tokens are short-lived, audience-bound, auto-rotated.
- One SA per workload; disable automount when unused.

CONFIGURATION EXAMPLE

```
apiVersion: v1
kind: ServiceAccount
metadata: {name: api, namespace: shop}
automountServiceAccountToken: false
# kubectl create token api -n shop --duration=10m
```

Q342**Troubleshooting**

A Java pod restarts every few hours with Last State: Terminated, Reason: OOMKilled, Exit Code: 137, but the dashboard shows heap usage well below the 2 GiB limit. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Confirm OOMKilled and the limit that was hit
- 2 The kernel counts the whole container
- 3 Heap is only part of the process memory
- 4 Size heap and limit together

EXPLANATION

OOMKilled with exit code 137 means the kernel killed the container because its total memory (the cgroup working set) reached the limit; the JVM heap is only part of that. Thread stacks, metaspace, direct buffers, native libraries and sidecar processes in the same container all count, and a 60-second metric can miss a short spike. Compare `container/memory/used_bytes` with the heap, and turn on Native Memory Tracking to see where non-heap memory goes. Set the heap as a share of the limit with `-XX:MaxRAMPercentage=75` instead of a fixed `-Xmx` near the limit, raise the limit if real usage needs it, and set the memory request close to real usage.

KEY POINTS

- Leave 25% or more headroom between heap and container limit for JVMs.
- Alert on restarts with reason OOMKilled, not only on heap usage.

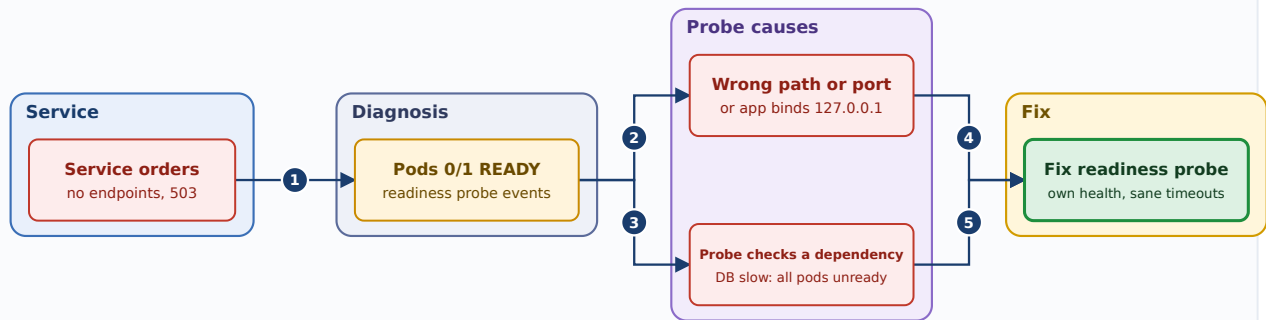
CONFIGURATION EXAMPLE

```

kubectl describe pod api-7c9d -n shop | grep -A5 'Last State'
kubectl top pod api-7c9d -n shop --containers
kubectl exec api-7c9d -n shop -- cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/memory.current
kubectl exec api-7c9d -n shop -- jcmd 1 VM.native_memory summary
kubectl set env deploy/api -n shop \
  JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:NativeMemoryTracking=summary"
kubectl set resources deploy/api -n shop --requests=memory=2Gi --limits=memory=3Gi
  
```

Q343**Troubleshooting**

Pods of `orders` are Running but show `0/1 READY`, requests through the Ingress return 503, and `kubectl get endpoints orders` is empty. What do you check?

FLOW DIAGRAM

- 1 Only Ready pods become endpoints
- 2 Unhealthy: HTTP 404 or connection refused
- 3 Unhealthy: timeout while a dependency is down

EXPLANATION

A Service only sends traffic to pods whose readiness probe passes, so Running but not Ready pods leave it with no endpoints and the load balancer returns 503. `kubectl describe pod` shows Readiness probe failed with the reason: a 404 means the wrong path, connection refused means the wrong port or an app listening only on 127.0.0.1, and a timeout often means `timeoutSeconds: 1` is too short. A readiness check that calls the database takes every pod out at once when the database is slow, so keep it about the pod's own ability to serve. Test the endpoint from inside the pod, fix the probe, and endpoints appear within one probe period.

KEY POINTS

- Readiness decides traffic, liveness decides restarts: never point liveness at dependencies.
- Alert when a Service has zero ready endpoints.

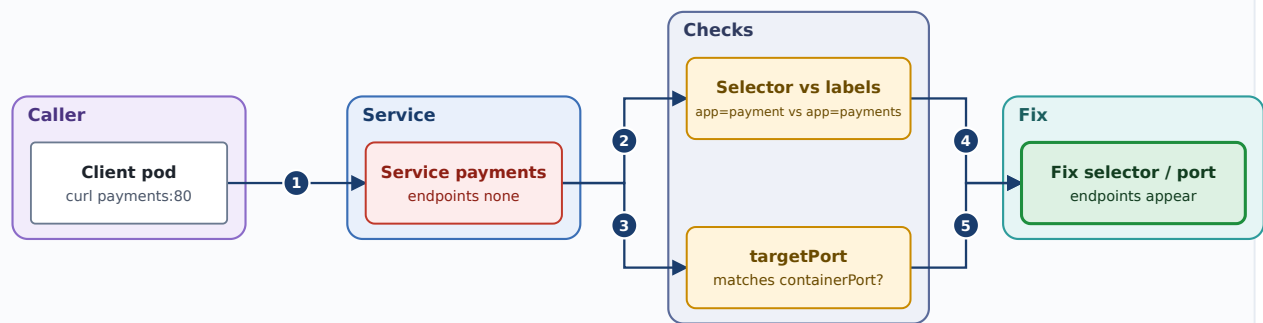
CONFIGURATION EXAMPLE

```

kubectl get endpointslices -n shop -l kubernetes.io/service-name=orders
kubectl describe pod orders-6d4f -n shop | grep -i -A2 readiness
kubectl exec orders-6d4f -n shop -- wget -qO- http://127.0.0.1:8080/ready
readinessProbe:
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  timeoutSeconds: 3
  failureThreshold: 3
  
```

Q344**Troubleshooting**

A new Service `payments` times out, yet its pods are Running and Ready and `curl` to a pod IP works. `kubectl get endpoints payments` shows `<none>`. How do you find the problem?

FLOW DIAGRAM

- 1 Name resolves but nothing behind it
- 2 No pod carries every selector label
- 3 Endpoints exist but the port is wrong

EXPLANATION

A Service selects pods by labels, and every key-value pair in its selector must match, so one typo (`app: payment` vs `app: payments`) or an extra label such as `version: v2` leaves it with no endpoints. Compare `kubectl get svc -o jsonpath='{.spec.selector}'` with `kubectl get pods --show-labels`, and run `kubectl get pods -l` with the Service's selector to see what it really matches. If endpoints exist but connections are refused, the `targetPort` does not match the port the container listens on (a named port must match the container port's name). Fix the Service, since a Deployment's selector is immutable and cannot simply be edited.

KEY POINTS

- Generate labels and selectors from one place (Helm helpers or Kustomize labels).
- Use named ports so a port change in the pod does not break the Service.

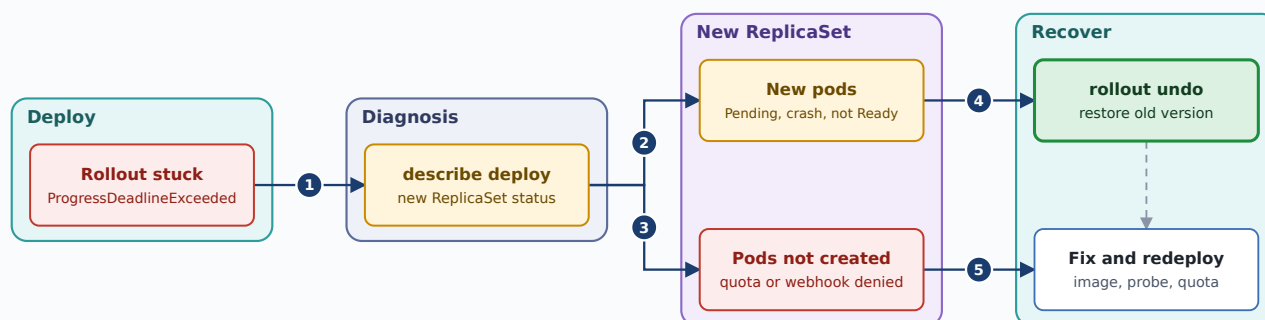
CONFIGURATION EXAMPLE

```

kubectl get svc payments -n shop -o jsonpath='{.spec.selector}{"\n"}{.spec.ports}{"\n"}'
kubectl get pods -n shop --show-labels | grep payments
kubectl get pods -n shop -l app=payment      # what the Service selects
kubectl patch svc payments -n shop -p '{"spec":{"selector":{"app":"payments"}}}'
kubectl get endpointslices -n shop -l kubernetes.io/service-name=payments
  
```

Q345**Troubleshooting**

`kubectl rollout status deploy/web` hangs and then fails with error: deployment "web" exceeded its progress deadline, with half the pods still on the old version. How do you troubleshoot it?

FLOW DIAGRAM

- 1 Progressing=False, reason ProgressDeadlineExceeded
- 2 Why new pods never become Ready
- 3 ReplicaSet events show FailedCreate
- 4 Roll back first if users are affected
- Then fix the cause and roll forward

EXPLANATION

When new pods do not become available within `progressDeadlineSeconds` (600 by default), the Deployment sets `Progressing=False` with reason `ProgressDeadlineExceeded`, but it does not roll back on its own. Find the new ReplicaSet with `kubectl get rs` and look at its pods: Pending means no room for the surge pods, `ImagePullBackOff` or `CrashLoopBackOff` means a bad image or config, and Running but not Ready means a failing readiness probe. If the ReplicaSet has too few pods and its events show `FailedCreate`, a ResourceQuota or admission webhook refused them. Run `kubectl rollout undo` to restore service, fix the cause, and deploy again.

KEY POINTS

- Make CI fail the deploy on `rollout status` timeout and roll back automatically.
- Keep `revisionHistoryLimit` above 0 so undo has something to go back to.

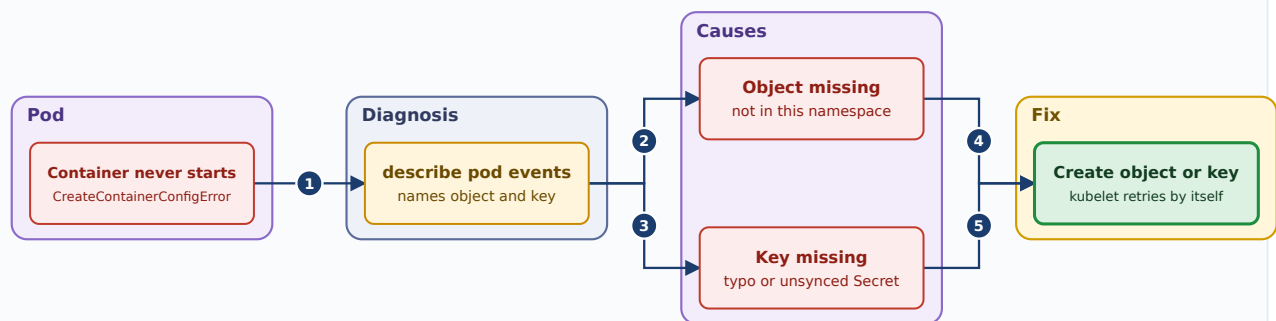
CONFIGURATION EXAMPLE

```

kubectl describe deploy web -n shop | grep -A4 Conditions
kubectl get rs -n shop -l app=web
kubectl describe rs web-5d8f9c -n shop | tail -5
kubectl get pods -n shop -l app=web,pod-template-hash=5d8f9c
kubectl rollout undo deploy/web -n shop
kubectl rollout status deploy/web -n shop --timeout=5m
  
```

Q346**Troubleshooting****After deploying to a new namespace, pods show**

CreateContainerConfigError with configmap "app-config" not found or couldn't find key DB_PASSWORD in Secret staging/db-creds. **How do you fix it?**

FLOW DIAGRAM

- 1 The event text is precise
- 2 ConfigMaps and Secrets are namespaced
- 3 Object exists but lacks the referenced key

EXPLANATION

CreateContainerConfigError means the kubelet could not build the container's configuration, and the pod events say exactly what is missing. ConfigMaps and Secrets are namespaced, so objects created in default or dev do not exist for pods in staging; a missing key usually comes from a typo or from a secret-sync tool (External Secrets, Secret Manager CSI) that has not synced yet. Create the object or key in the pod's namespace and the kubelet retries automatically, no redeploy needed. The same error also appears when runAsNonRoot: true is set but the image runs as root, which the event also states.

KEY POINTS

- Deploy ConfigMaps and Secrets in the same release as the workload.
- Mark truly optional references with optional: true.

CONFIGURATION EXAMPLE

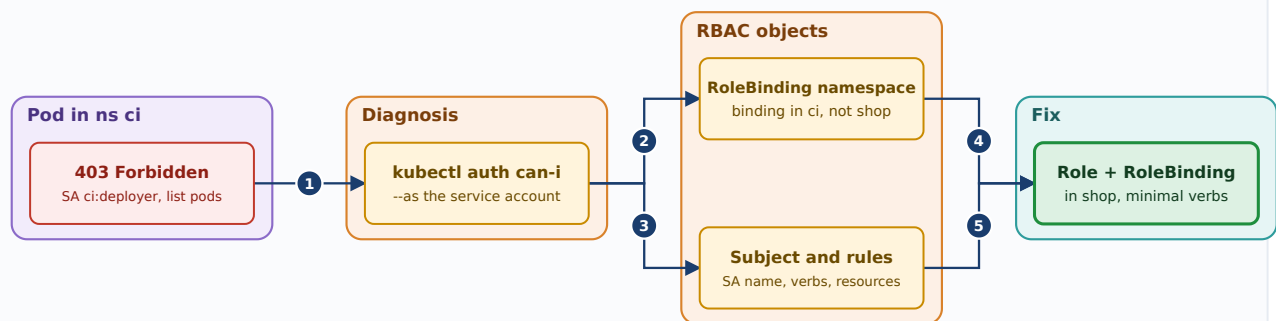
```

kubectl describe pod api-7f9c -n staging | grep -i -A2 error
kubectl get configmap,secret -n staging
kubectl get secret db-creds -n staging -o jsonpath='{.data}' | jq 'keys'
kubectl create configmap app-config -n staging --from-file=config/staging.yaml
kubectl create secret generic db-creds -n staging --from-file=DB_PASSWORD=./db_password.txt
kubectl get pods -n staging -w

```


Q347**Troubleshooting**

A deploy tool running in a pod fails with `pods is forbidden: User "system:serviceaccount:ci:deployer" cannot list resource "pods" in API group "" in the namespace "shop". How do you fix it with least privilege?`

FLOW DIAGRAM

- 1 Reproduce as that identity
- 2 RoleBindings grant only in their own namespace
- 3 Wrong SA name or missing verb or resource

EXPLANATION

The message names the identity, verb, resource, API group and namespace, so reproduce it with `kubectl auth can-i --as=system:serviceaccount:ci:deployer`. A RoleBinding only grants permissions in its own namespace, so a binding in `ci` does nothing for `shop`; also check that the subject names the right service account and namespace, and that the Role lists the resource in the right API group (`""` for `pods`, `apps` for `deployments`). If the error shows `default` as the service account, the pod spec is missing `serviceAccountName`. Create a Role in `shop` with only the verbs needed and bind it to the service account; avoid a ClusterRoleBinding to `edit` or `cluster-admin`.

KEY POINTS

- Review with `kubectl auth can-i --list --as=...` after every RBAC change.
- One service account per tool, never the namespace's `default`.

CONFIGURATION EXAMPLE

```

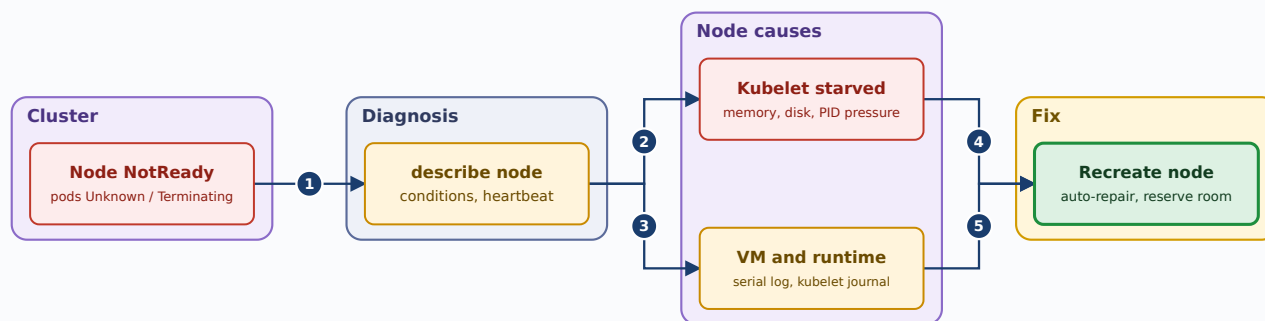
kubectl auth can-i list pods -n shop --as=system:serviceaccount:ci:deployer
kubectl get rolebindings,clusterrolebindings -A -o wide | grep deployer
kubectl create role pod-reader -n shop --verb=get,list,watch --resource=pods
kubectl create rolebinding deployer-pods -n shop --role=pod-reader \
  --serviceaccount=ci:deployer
kubectl auth can-i --list -n shop --as=system:serviceaccount:ci:deployer

```

Q348**Troubleshooting**

A GKE node flips to `NotReady`, its pods show `Unknown` or `Terminating`, and new pods avoid it. How do you troubleshoot it?

FLOW DIAGRAM



- 1 Kubelet stopped posting node status
- 2 Pressure conditions before it went silent
- 3 VM stopped, preempted or runtime down

EXPLANATION

`NotReady` means the control plane stopped receiving the kubelet's heartbeats, so the node is tainted as unreachable and its pods are evicted after the default 300-second toleration. `kubectl describe node` shows the conditions: `MemoryPressure`, `DiskPressure` or `PIDPressure` before the failure point to workloads starving the kubelet, while `Kubelet stopped posting node status` points to the VM itself. Check the VM's serial port output and kubelet and containerd logs, and whether it was a Spot preemption or host maintenance. GKE node auto-repair recreates nodes that stay `NotReady` for about 10 minutes; fix the cause by setting requests and limits, using a larger boot disk, or limiting PIDs.

KEY POINTS

- Keep node auto-repair on and alert on nodes `NotReady` for more than 5 minutes.
- Spread replicas across nodes so one bad node never takes a whole service down.

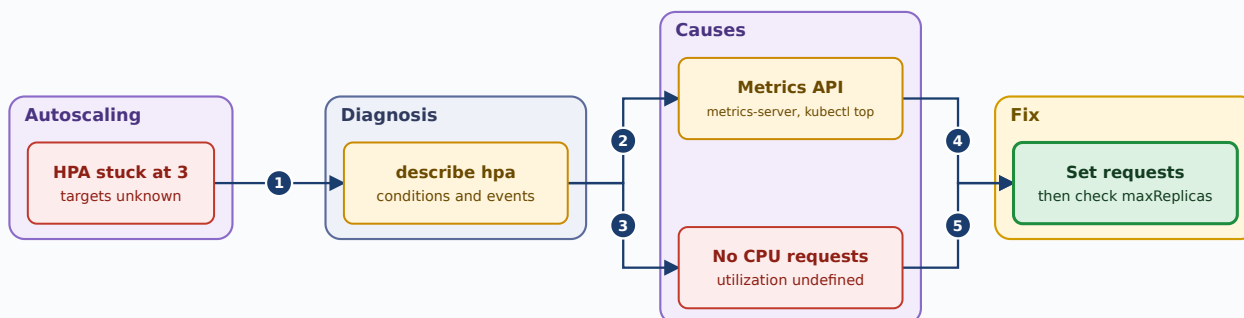
CONFIGURATION EXAMPLE

```

kubectl describe node gke-prod-pool-a-7h2k | sed -n '/Conditions:/,/Addresses:/p'
kubectl get events -A --field-selector involvedObject.name=gke-prod-pool-a-7h2k
gcloud compute instances get-serial-port-output gke-prod-pool-a-7h2k \
  --zone=europe-west1-b | tail -50
gcloud compute ssh gke-prod-pool-a-7h2k --zone=europe-west1-b -- \
  'sudo journalctl -u kubelet --since "30 min ago" | tail -50'
gcloud container node-pools describe pool-a --cluster=prod --region=europe-west1 \
  --format="value(management.autoRepair)"
  
```

Q349**Troubleshooting**

During a load spike the HPA stays at 3 replicas and `kubectl get hpa` shows `TARGETS <unknown>/70%` with `FailedGetResourceMetric` events. Why, and how do you fix it?

FLOW DIAGRAM

- 1 ScalingActive=False shows the reason
- 2 Metrics API unavailable or pods not reporting
- 3 Utilization = usage / request

EXPLANATION

Target utilization is a percentage of the container's request, so if any container in the pod (sidecars too) has no CPU request, the HPA cannot compute it and shows `<unknown>`. Check `kubectl describe hpa`: `ScalingActive=False` with `FailedGetResourceMetric` means the resource metrics API returned nothing, so confirm `kubectl top pods` works and the `vlbeta1.metrics.k8s.io` `APIService` is available. For custom or external metrics, the adapter (for example the Custom Metrics Stackdriver Adapter) must be installed and the metric name must match. Once metrics flow, `ScalingLimited=True` means you hit `maxReplicas`.

KEY POINTS

- Set CPU requests on every container, including sidecars.
- Newly started pods are ignored briefly, so scaling lags a few seconds behind load.

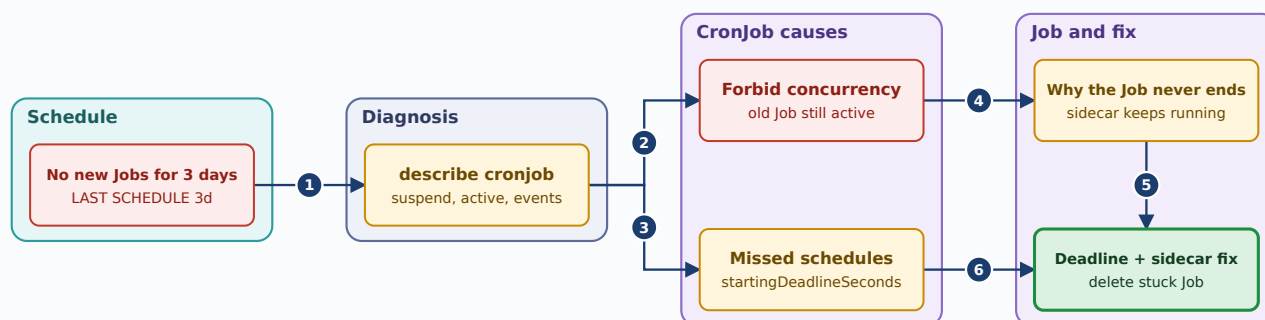
CONFIGURATION EXAMPLE

```

kubectl describe hpa web -n shop | sed -n '/Conditions:$/, $p'
kubectl top pods -n shop -l app=web
kubectl get apiservice vlbeta1.metrics.k8s.io
kubectl get deploy web -n shop \
  -o jsonpath='{range .spec.template.spec.containers[*]}{.name}{ " "}{.resources.requests}{ "\n"}{end}'
kubectl set resources deploy/web -n shop -c=web --requests=cpu=250m,memory=256Mi
  
```

Q350**Troubleshooting**

A CronJob `nightly-export` has not created a Job for three days, and the last Job it created still shows 0/1 completions with its pod Running. How do you troubleshoot both?

FLOW DIAGRAM

- 1 Check suspend and the active Job list
- 2 A hung Job blocks every later run
- 3 Too many missed runs stop scheduling
- 4 Pod stays Running after the work is done

EXPLANATION

First check `.spec.suspend` and `.status.active`: with `concurrencyPolicy: Forbid`, one Job that never finishes blocks every later schedule. A Job completes only when its pod exits successfully, and a common trap is a sidecar such as the Cloud SQL Auth Proxy that keeps running after the main container ends; run it as a native sidecar (an init container with `restartPolicy: Always`) or make the app stop it. Set `activeDeadlineSeconds` on the Job so a hang is killed, and `backoffLimit` so failures stop retrying. If `startingDeadlineSeconds` is unset and the controller missed more than 100 schedules, it stops creating Jobs, so set a deadline, delete the stuck Job and check that `timeZone` gives the time you expect.

KEY POINTS

- Alert when a CronJob's last successful time is older than its interval.
- Every Job should have `activeDeadlineSeconds`.

CONFIGURATION EXAMPLE

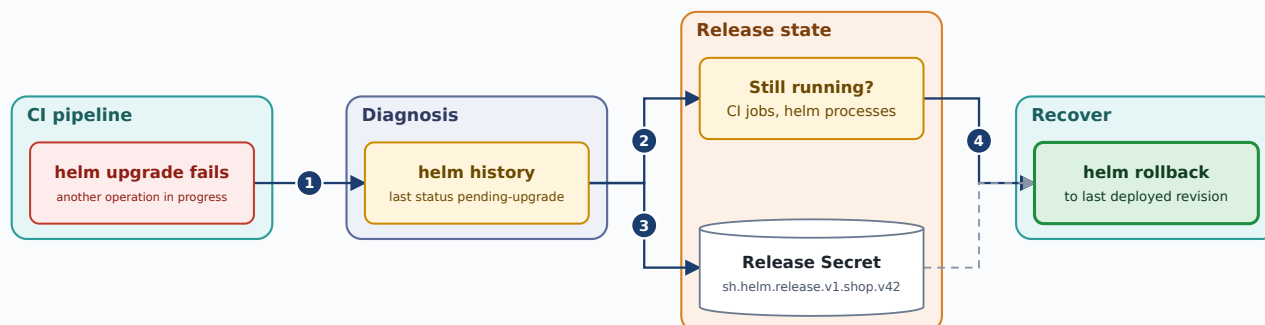
```

kubectl get cronjob nightly-export -n data \
  -o jsonpath='{.spec.suspend} {.spec.concurrencyPolicy} {.status.active}{"\n"}'
kubectl get jobs -n data --sort-by=.metadata.creationTimestamp
kubectl get pods -n data -l job-name=nightly-export-28790160
kubectl logs -n data job/nightly-export-28790160 --all-containers --tail=20
kubectl delete job nightly-export-28790160 -n data
kubectl patch cronjob nightly-export -n data -p \
  '{"spec":{"startingDeadlineSeconds":3600,"jobTemplate":{"spec":{"activeDeadlineSeconds":7200}}}}'

```

Q351**Troubleshooting**

`helm upgrade shop ./chart` **fails with** `Error: UPGRADE FAILED: another operation (install/upgrade/rollback) is in progress`. **The previous CI job was cancelled mid-deploy. How do you recover safely?**

FLOW DIAGRAM

- ❶ Killed helm left a pending revision
- ❷ Make sure nothing is still deploying
- ❸ Status is stored as a label on the Secret
- ❹ Rollback clears the pending state
- Last resort: delete only the pending Secret

EXPLANATION

Helm 3 stores every release revision as a Secret in the release namespace, labelled with its status, and a helm process killed mid-upgrade leaves the latest revision in `pending-upgrade`, so Helm refuses any new operation. Confirm with `helm history` and make sure no other pipeline or person is still running helm for that release. Then `helm rollback` to the last `deployed` revision, which creates a new revision and clears the pending state; for a first install stuck in `pending-install`, `uninstall` and `install` again. Deleting the pending revision's Secret also works but is a last resort, since it hides what happened.

KEY POINTS

- Use `--atomic --timeout` so a failed upgrade rolls itself back.
- Give CI deploy jobs a concurrency lock per release and do not cancel them mid-deploy.

CONFIGURATION EXAMPLE

```

helm history shop -n shop --max 5
helm list -n shop --pending
kubectl get secret -n shop -l owner=helm,name=shop --show-labels | tail -3
helm rollback shop 41 -n shop --wait
# last resort, only when nothing is running:
kubectl delete secret sh.helm.release.v1.shop.v42 -n shop
helm upgrade shop ./chart -n shop --atomic --timeout 10m

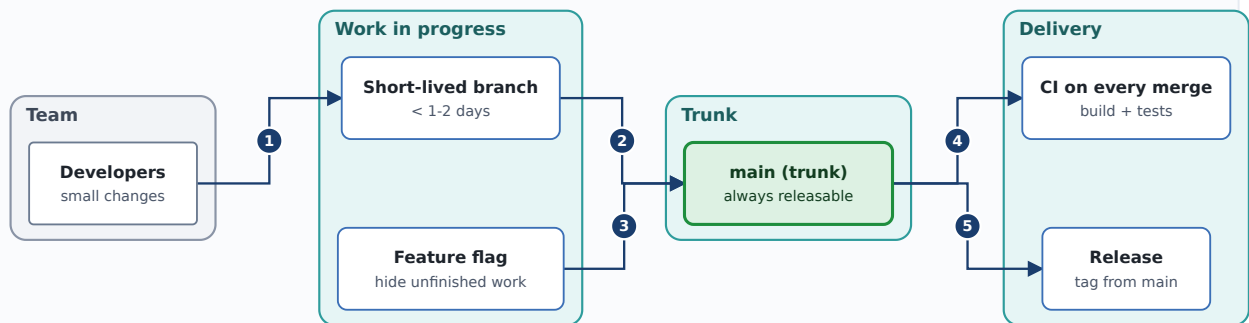
```

13. Git, DevOps Practices & DevSecOps

Day-to-day Git skills, team practices and security built into the delivery process, as asked in senior DevOps interviews.

Q352 Your team releases once a month from long-lived GitFlow branches and merges are painful. Would you move to trunk-based development, and how?

FLOW DIAGRAM



- 1 Branch from main, open PR early
- 2 Merge after review and green CI
- 3 Incomplete features merged but switched off
- 4 Every commit on main is tested
- 5 Release any green commit

EXPLANATION

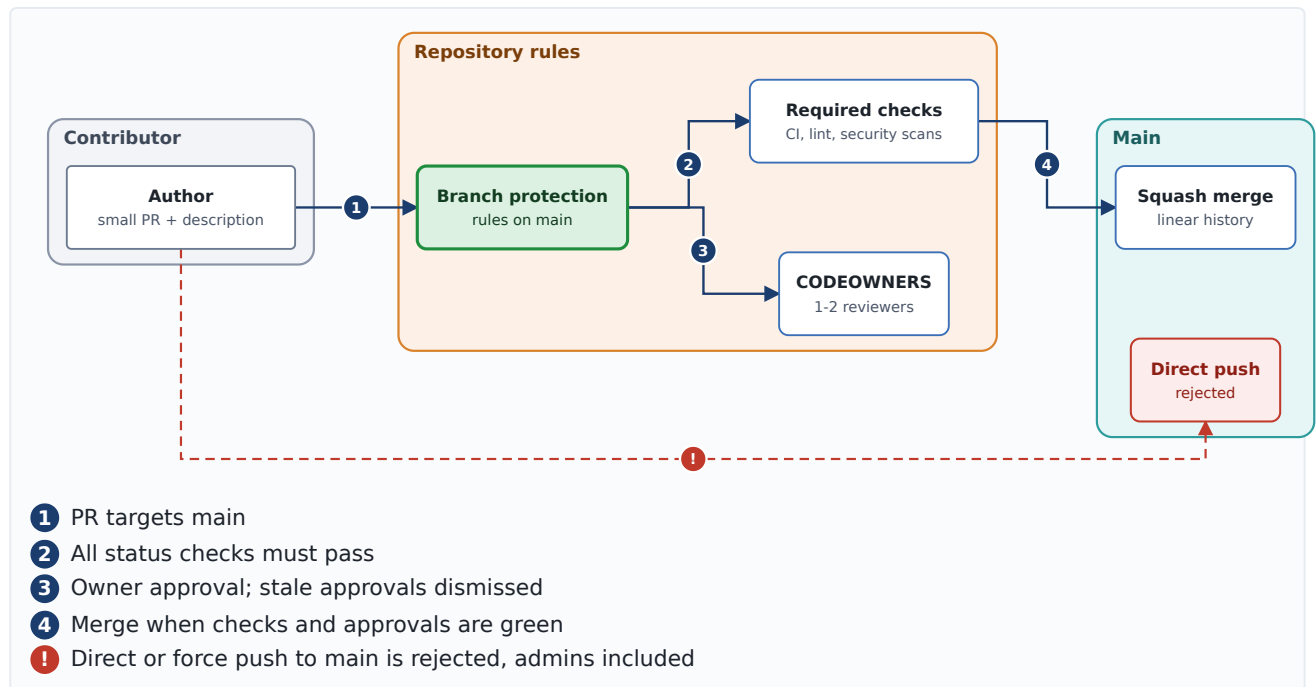
Trunk-based development means everyone merges small changes into `main` at least daily through short-lived branches and PRs, keeping `main` always releasable, which removes big merge conflicts and supports continuous delivery. GitFlow, with long-lived `develop`, `release` and `hotfix` branches, fits products with scheduled versioned releases or several supported versions, but it delays integration. To move, invest first in fast reliable CI and automated tests, then use feature flags to merge unfinished work safely, and cut releases by tagging `main` (or short-lived release branches if you must support old versions). DORA research links trunk-based development with higher delivery performance.

KEY POINTS

- Small batches plus fast CI are the prerequisite.
- Feature flags separate merging from releasing.

Q353 How do you protect the `main` branch and make pull request reviews useful rather than a rubber stamp?

FLOW DIAGRAM



EXPLANATION

Turn on branch protection or rulesets for `main`: require a PR, one or two approvals including a CODEOWNERS approval for sensitive paths, passing status checks, up-to-date branches or a merge queue, dismissal of stale approvals on new commits, and block force pushes and deletions, with admins included. Keep PRs small (a few hundred lines) with a clear description and linked ticket, so reviewers can check design, correctness, tests and security instead of style, which linters and formatters handle. Agree on review turnaround times, because slow reviews hurt lead time more than most other things. Signed commits and required conversation resolution add further assurance.

KEY POINTS

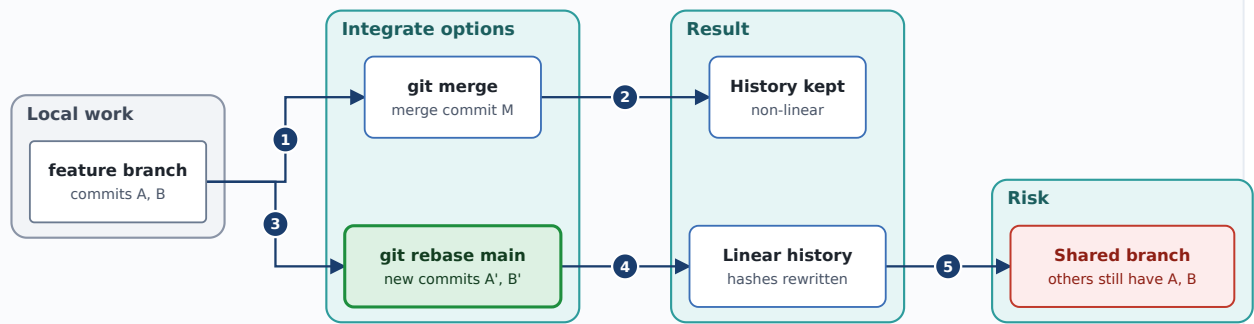
- Automate style; humans review logic and risk.
- Protection rules must apply to admins too.

CONFIGURATION EXAMPLE

```
# .github/CODEOWNERS
/terraform/prod/ @acme/platform-leads
/.github/workflows/ @acme/devops
*.sql @acme/dba
```

Q354 When do you merge and when do you rebase? A colleague rebased a shared branch and now others have broken histories. What happened?

FLOW DIAGRAM



- ❶ Combine histories, nothing rewritten
- ❸ Replay commits on top of main
- ❹ Clean, easy to bisect
- ❺ Force push breaks collaborators

EXPLANATION

Merge joins two histories with a merge commit and never rewrites existing commits, so it is safe on shared branches but produces a non-linear history. Rebasing replays your commits on top of another branch, creating new commits with new hashes, giving a clean linear history that is easy to read and bisect. Because it rewrites history, rebasing a branch others have pulled means they hold the old commits, and after a force push their next pull duplicates or conflicts. Rule of thumb: rebase your own local or unshared branch to update it, merge (or squash merge) into shared branches, and use `git push --force-with-lease` when you must rewrite your own remote branch.

KEY POINTS

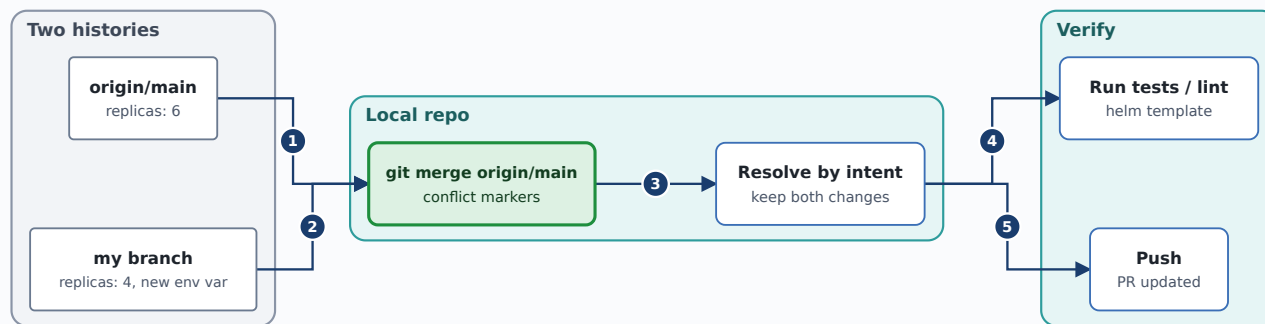
- Never rewrite history others have pulled.
- `--force-with-lease` refuses to overwrite commits you have not seen.

CONFIGURATION EXAMPLE

```
git fetch origin
git rebase origin/main          # update my own feature branch
git push --force-with-lease     # safe force push of my branch
# on a shared branch instead:
git merge origin/main
```


Q355 Your PR has a merge conflict in `values-prod.yaml` with a change a colleague just merged. How do you resolve it safely?

FLOW DIAGRAM



- 1 Fetch the latest main
- 2 Git cannot pick automatically
- 3 Read both sides, talk to the author if unsure
- 4 Render and validate before committing
- 5 git add, commit, push

EXPLANATION

Fetch and merge (or rebase onto) the latest `origin/main` locally, then open the conflicted file and read both sides between the `<<<<<<`, `=====` and `>>>>>>` markers. Decide based on intent, not on which side is newer: here keep the colleague's `replicas: 6` and add your new environment variable, and ask them if the intent is unclear. Remove the markers, run the relevant checks (for example `helm template` and the unit tests), then `git add` and complete the merge or `git rebase --continue`. `git mergetool` or an IDE three-way view helps, and `git rerere` can remember resolutions for repeated conflicts.

KEY POINTS

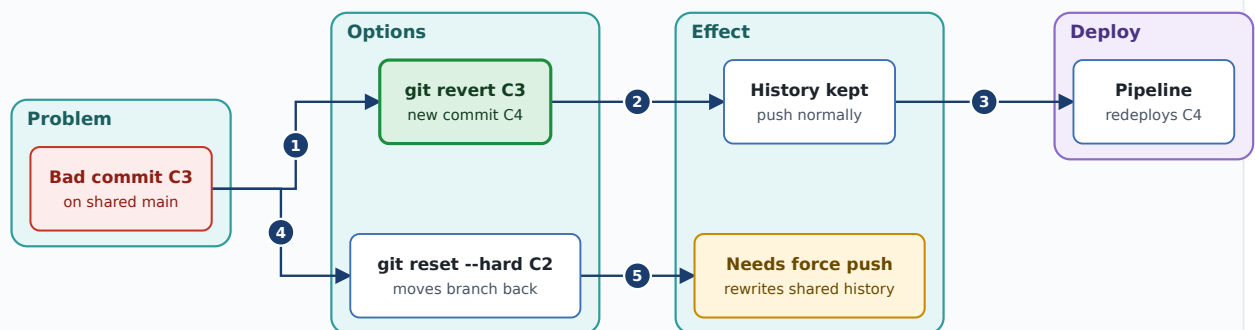
- Resolve by understanding both changes.
- Always re-run tests after resolving.

CONFIGURATION EXAMPLE

```
git fetch origin
git merge origin/main          # CONFLICT in values-prod.yaml
git diff --name-only --diff-filter=U
# edit file, remove markers
git add deploy/values-prod.yaml
git commit                    # or: git rebase --continue
```

Q356 A bad commit was merged to `main` and deployed. Do you use `git revert` or `git reset` to undo it?

FLOW DIAGRAM



- 1 Adds an inverse commit
- 2 Everyone's clones stay valid
- 3 Normal CI/CD rolls out the fix
- 4 Only for local or unpushed work
- 5 Blocked by branch protection on main

EXPLANATION

On a shared branch use `git revert <sha>`, which creates a new commit that undoes the change, keeps history intact, goes through the normal PR and pipeline, and leaves everyone's clones valid. `git reset` moves the branch pointer back and discards commits (`--hard` also discards working changes), so on `main` it would need a force push that rewrites shared history and is normally blocked by branch protection. Use reset only for local commits that have not been pushed. Reverting a merge commit needs `-m 1` to pick the mainline parent, and if you later want the change back you revert the revert.

KEY POINTS

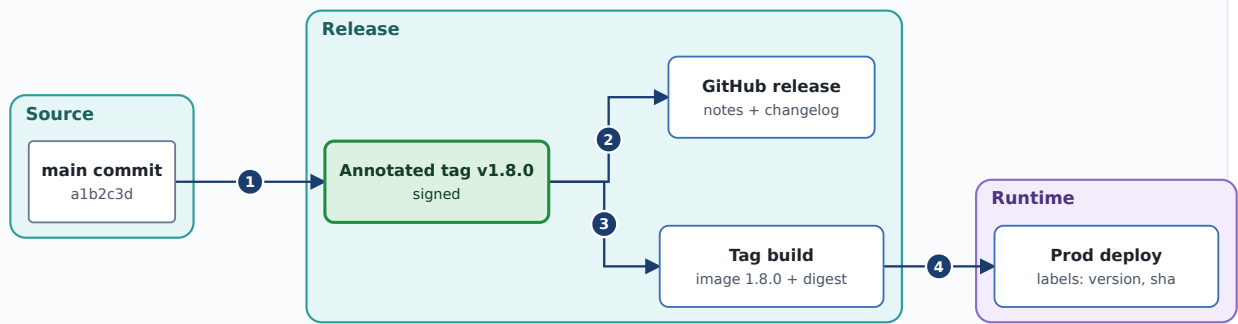
- Shared history: revert. Local history: reset is fine.
- Rollback the deploy first if needed, then revert in Git.

CONFIGURATION EXAMPLE

```
git switch -c revert-bad-change origin/main
git revert 3f2a91c          # normal commit
git revert -m 1 8be41d0    # reverting a merge commit
git push -u origin revert-bad-change  # open PR
```

Q357 How do you use Git tags and releases so that a production deployment can always be traced back to exact source code?

FLOW DIAGRAM



- 1 git tag -a -s v1.8.0
- 2 Release notes from commits
- 3 Trigger on tag pattern
- 4 Deployed image carries the tag and SHA

EXPLANATION

Create annotated (and ideally signed) tags such as `v1.8.0` on the exact commit being released; annotated tags store author, date and message, unlike lightweight tags. Protect tag patterns so release tags cannot be moved or deleted, and let a tag-triggered pipeline build the artifact once and label it with the version and commit SHA. A GitHub or GitLab release attached to the tag holds release notes and artifacts. At runtime, expose the version and SHA as labels, environment variables or a `/version` endpoint so on-call can go from a running pod to the commit in seconds.

KEY POINTS

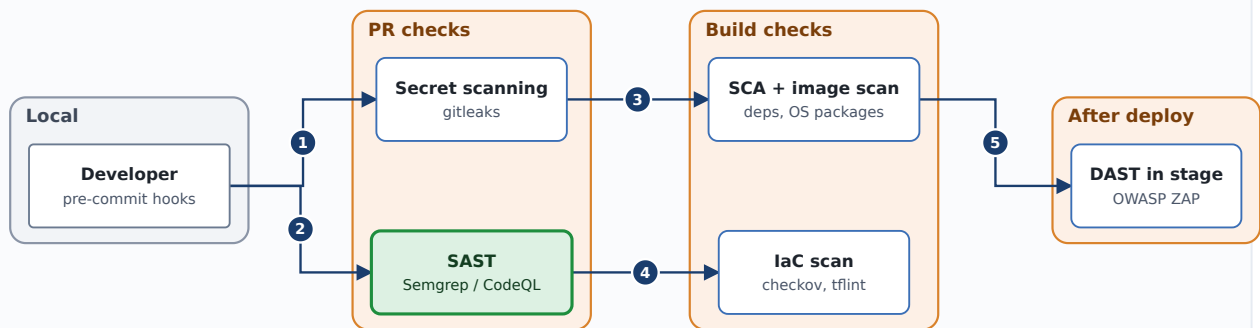
- Tags must be immutable once pushed.
- Every artifact should record its commit SHA.

CONFIGURATION EXAMPLE

```
git tag -a v1.8.0 -m "Release 1.8.0" a1b2c3d
git push origin v1.8.0
git describe --tags          # e.g. v1.8.0-3-g9f1e2aa
```

Q358 How would you add shift-left security to a CI pipeline for a containerised service on GCP?

FLOW DIAGRAM



- 1 Block secrets before they reach Git
- 2 Code patterns: injection, unsafe crypto
- 3 Known CVEs in dependencies and base image
- 4 Misconfigured Terraform and manifests
- 5 Scan the running app in staging

EXPLANATION

Shift-left means finding security issues as early and cheaply as possible, with feedback in the developer's PR. Add secret scanning (pre-commit gitleaks plus GitHub secret scanning with push protection), SAST such as CodeQL or Semgrep, SCA for dependencies (Dependabot, OSV-Scanner) and container image scanning with Artifact Analysis, plus IaC scanning with checkov or tfint for Terraform and Kubernetes manifests. Run DAST such as OWASP ZAP against the staging deployment, since it tests the running app. Fail builds only on high-confidence, high-severity findings and route the rest to backlog, otherwise teams learn to ignore the tools.

KEY POINTS

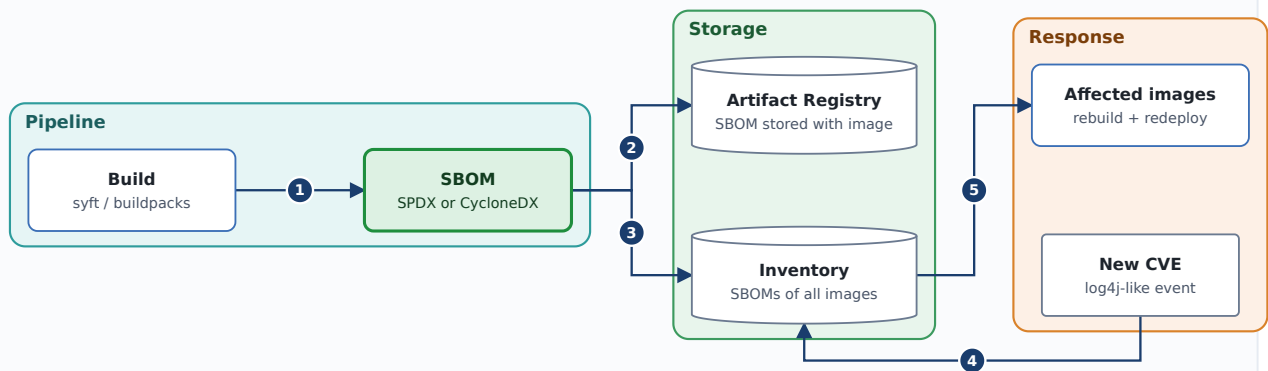
- Fast, low-noise checks in PRs; slower scans nightly.
- Tune rules; noisy tools get bypassed.

CONFIGURATION EXAMPLE

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks: [{ id: gitleaks }]
- repo: https://github.com/antonbabenko/pre-commit-terraform
  rev: v1.92.0
  hooks: [{ id: terraform_fmt }, { id: terraform_tflint }]
```

Q359 A customer asks for an SBOM for your product and a new critical CVE is announced in a popular library. How do SBOMs help you respond?

FLOW DIAGRAM



- 1 Generated for every image
- 2 gcloud artifacts sbom export
- 3 Queryable across all services
- 4 Which images contain the package?
- 5 List in minutes, not days

EXPLANATION

An SBOM (software bill of materials) is a machine-readable list of every component and version in an artifact, in formats such as SPDX or CycloneDX. Generate one per image in the pipeline (for example Syft, or `gcloud artifacts sbom export` for images in Artifact Registry) and store it with the artifact or in a central inventory. When a new CVE appears, you query the SBOMs to find exactly which images and services include the vulnerable version, instead of grepping repos. SBOMs also satisfy customer and regulatory requests and pair with provenance to describe both what is in the artifact and how it was built.

KEY POINTS

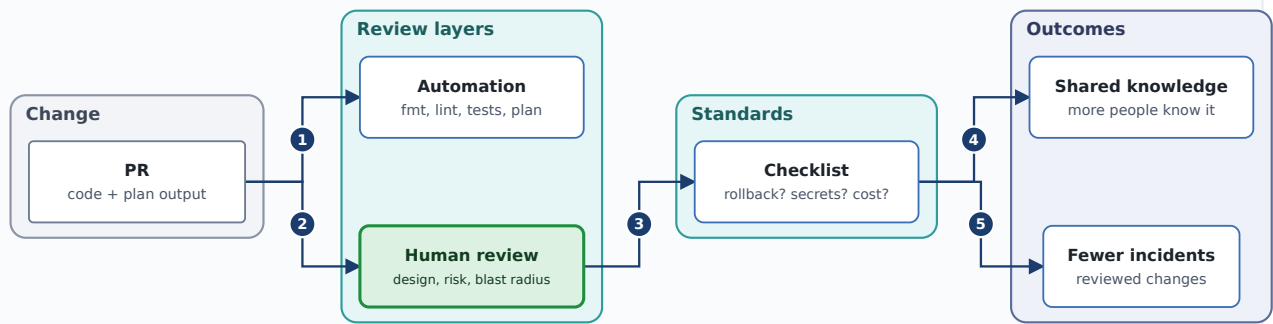
- SBOM = ingredients list; provenance = how it was made.
- Keep SBOMs queryable, not buried in build logs.

CONFIGURATION EXAMPLE

```
gcloud artifacts sbom export \
  --uri=europe-west1-docker.pkg.dev/acme/apps/api@sha256:3b1f...
syft europe-west1-docker.pkg.dev/acme/apps/api:1.8.0 -o cyclonedx-json > sbom.json
grep -c '"name": "log4j-core"' sbom.json
```

Q360 How do you build a good code review culture that covers both infrastructure (Terraform, Kubernetes YAML) and application code?

FLOW DIAGRAM



- ❶ Machines check style and policy
- ❷ Reviewer reads the Terraform plan, not only HCL
- ❸ Same questions every time
- ❹ Reviews spread context

EXPLANATION

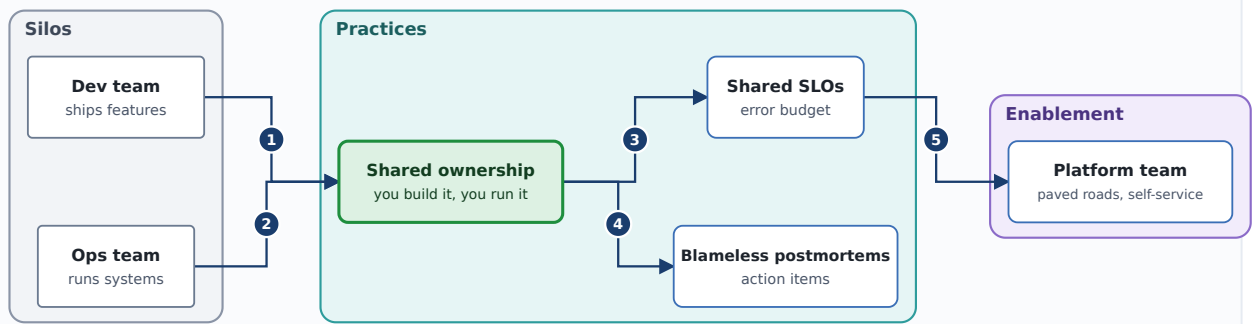
Treat infrastructure changes exactly like application code: every change goes through a PR, automation handles formatting, linting, tests and policy checks, and the Terraform plan or rendered manifests are posted to the PR so reviewers see the real effect. Human reviewers focus on design, correctness, security, blast radius, rollback and cost, using a short shared checklist. Keep the tone about the code, not the person, respond within agreed times, and rotate reviewers so knowledge spreads. For infra, a plan that destroys or replaces a stateful resource should always get an extra look from someone who owns it.

KEY POINTS

- Review the plan, not only the diff.
- Fast, kind, specific feedback keeps people using PRs.

Q361 Developers and operations blame each other after each outage. How would you build a DevOps culture that fixes this?

FLOW DIAGRAM



- 1 Devs join on-call for their services
- 2 Ops become platform and SRE partners
- 3 One reliability target for both
- 4 Learn from failure, fix systems
- 5 Tooling makes the right way the easy way

EXPLANATION

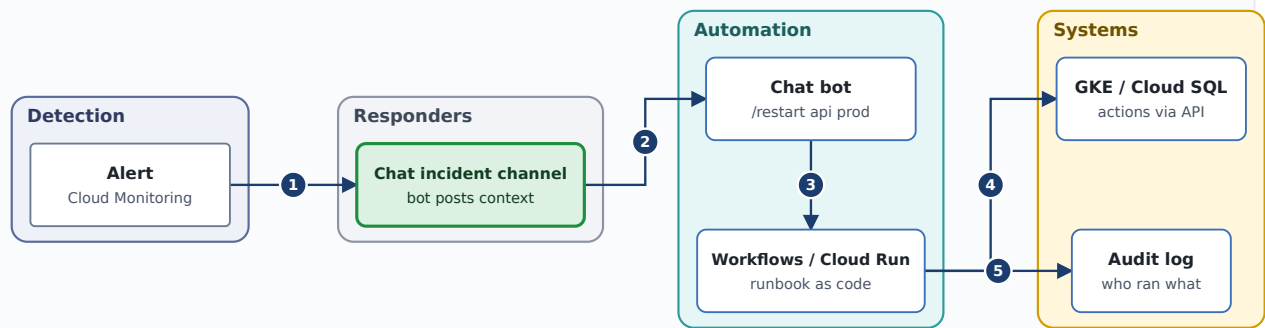
Replace separate goals (devs ship features, ops keep things stable) with shared ones: service-level objectives and an error budget both sides agree on, and teams that own their services in production, including on-call. Run blameless postmortems focused on how the system and process allowed the failure, with tracked action items. A platform team provides paved roads, such as templates, pipelines and observability, so product teams can self-serve safely. Make work visible with shared dashboards and chat channels, and measure with DORA metrics and SLOs rather than counting tickets.

KEY POINTS

- Shared goals remove the incentive to blame.
- Culture changes through practices, not slogans.

Q362 On-call engineers run the same manual steps from a wiki page during incidents. How would you use ChatOps and runbook automation?

FLOW DIAGRAM



- 1 Alert opens a channel with graphs and runbook link
- 2 Engineer runs an approved command
- 3 Bot calls a tested, parameterised runbook
- 4 Scoped service account performs the action
- 5 Result and actor posted back and logged

EXPLANATION

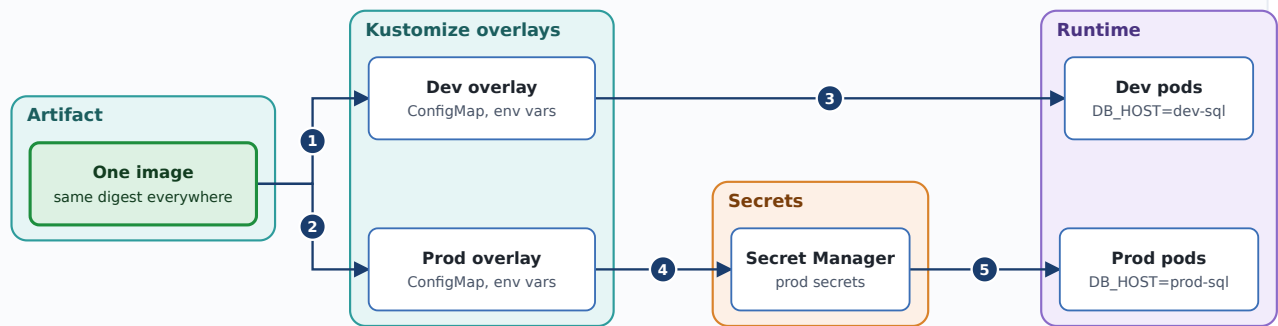
Turn the repeated wiki steps into runbooks as code, for example Cloud Workflows, Cloud Run jobs or scripts in a repo, that are tested, versioned and run with a narrowly scoped service account. Expose them through a chat bot in the incident channel, so commands like `/scale api prod 10` run the same steps every time, and the bot posts results so everyone sees what happened. Alerts should open the channel with dashboards, recent deploys and the runbook link. Require approvals for destructive actions, log every run to Cloud Audit Logs, and automate fully only steps that have proven safe, such as clearing a stuck queue.

KEY POINTS

- Automation removes typos under stress.
- Chat gives a shared, time-stamped record.

Q363 The same container runs in dev, stage and prod, but settings differ. How do you manage configuration per environment following 12-factor principles?

FLOW DIAGRAM



- 1 Base manifests + dev patch
- 2 Base manifests + prod patch
- 3 Config injected at runtime
- 4 Secrets referenced, not stored in Git
- 5 Mounted via CSI driver or env

EXPLANATION

Twelve-factor apps store config in the environment, not in the code or the image, so one artifact can be promoted unchanged. Keep a base set of manifests and small per-environment overlays (Kustomize, Helm values or Scaffold profiles) that set environment variables, ConfigMaps, replica counts and resource limits. Secrets come from Secret Manager at runtime through the Secret Manager CSI driver, External Secrets or Cloud Run secret references, never from Git. Validate config at startup and fail fast when a required value is missing.

KEY POINTS

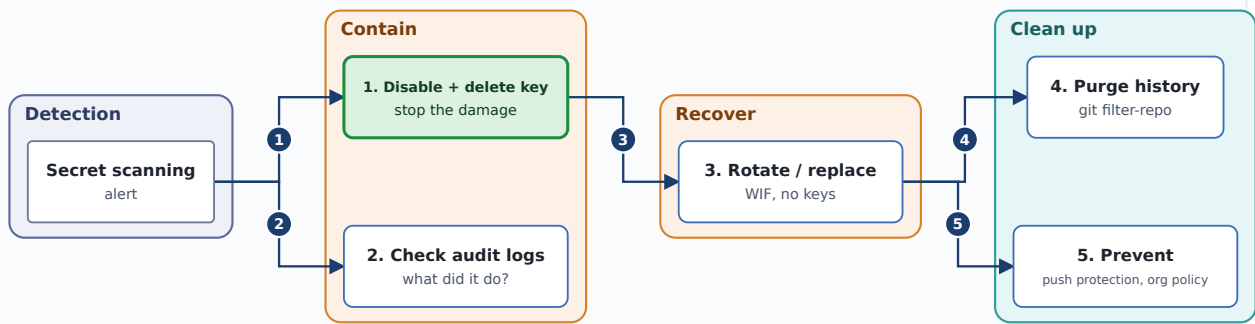
- Build once; configure per environment.
- Config diff between envs should be small and reviewable.

CONFIGURATION EXAMPLE

```
# overlays/prod/kustomization.yaml
resources: [../../base]
images:
- name: api
  newName: europe-west1-docker.pkg.dev/acme/apps/api
  digest: sha256:3b1f...
configMapGenerator:
- { name: api-config, behavior: merge, literals: [LOG_LEVEL=info, DB_HOST=prod-sql.internal] }
```

Q364 A developer pushed a service account key to a public GitHub repo an hour ago. What do you do, in order?

FLOW DIAGRAM



- ❶ Assume it is already compromised
- ❷ Cloud Audit Logs for that key ID
- ❸ New credential or keyless auth
- ❹ Remove from all history, force push
- ❺ Stop it happening again

EXPLANATION

Treat the key as compromised: disable and delete it at once (`gcloud iam service-accounts keys disable KEY_ID --iam-account=ci@acme.iam.gserviceaccount.com` then `delete`), because bots scan public repos within minutes and removing the file from Git does not revoke it. Check Cloud Audit Logs for activity by that key ID and look for new resources, IAM changes or data access, and follow your incident process if anything is suspicious. Replace it with keyless auth such as Workload Identity Federation, or a new secret stored in Secret Manager. Then purge the file from history with `git filter-repo`, force push, ask collaborators to re-clone and ask GitHub to clear cached views, and add push protection plus the `iam.disableServiceAccountKeyCreation` org policy.

KEY POINTS

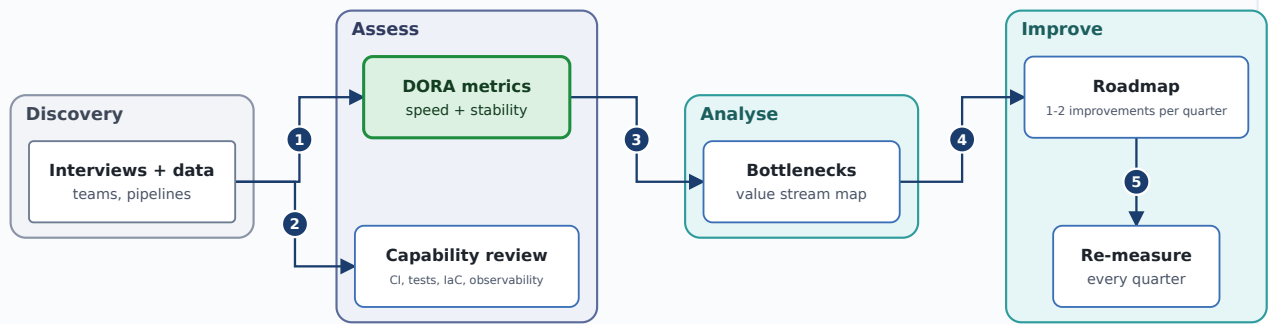
- Revoke first; cleaning Git history comes later.
- Rewriting history does not remove forks or clones.

CONFIGURATION EXAMPLE

```
gcloud iam service-accounts keys disable KEY_ID --iam-account=ci@acme.iam.gserviceaccount.com
gcloud iam service-accounts keys delete KEY_ID --iam-account=ci@acme.iam.gserviceaccount.com
git filter-repo --path keys/ci.json --invert-paths
git push --force --all && git push --force --tags
```

Q365 A CTO asks you to assess how mature the company's DevOps practices are and where to invest first. How do you approach it?

FLOW DIAGRAM



- ❶ Baseline four key metrics per team
- ❷ Which practices exist and how well
- ❸ Where does work wait?
- ❹ Fix the biggest constraint first
- ❺ Check the metrics moved

EXPLANATION

Start with a baseline: measure the DORA metrics per team from pipeline and incident data, and review capabilities such as version control of everything, CI and test automation, trunk-based development, infrastructure as code, deployment automation, observability and security in the pipeline. Map the value stream from commit to production to see where work waits, for example manual approvals, environment requests or flaky tests. Pick the one or two biggest constraints and fix them first, then re-measure each quarter. Present maturity as improvement trends per team, not a single score, and include team feedback such as on-call load and developer experience surveys.

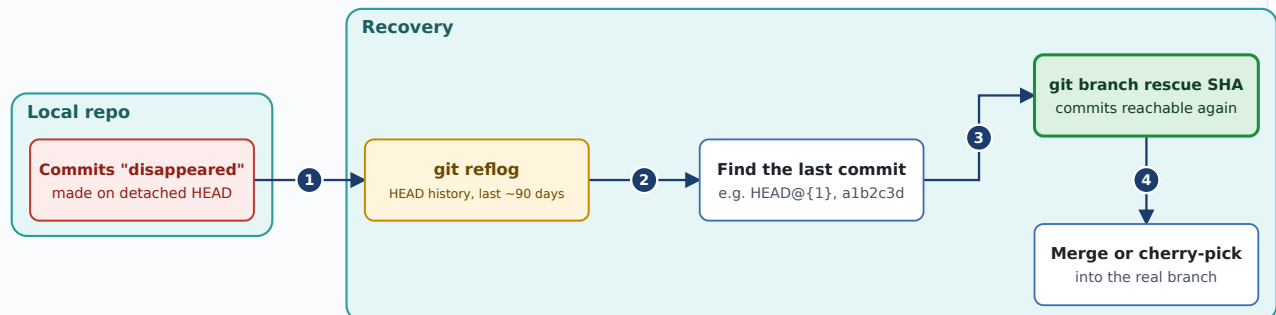
KEY POINTS

- Measure outcomes, not tool adoption.
- Fix the biggest bottleneck, then re-measure.

Q366**Troubleshooting**

A developer made three commits while in detached HEAD state, then ran `git checkout main`, and now `git log` does not show the commits. Are they lost, and how do you get them back?

FLOW DIAGRAM



- 1 Commits still exist, just no branch points to them
- 2 Look for the entry before 'checkout: moving to main'
- 3 A branch keeps them safe from garbage collection
- 4 Bring the work into the feature branch

EXPLANATION

The commits are not lost: they still exist in the object database, but no branch points to them, so `git log` on main does not show them. `git reflog` records every move of HEAD, so find the entry just before `checkout: moving from a1b2c3d to main` and create a branch at that commit with `git branch rescue a1b2c3d`. Then merge or cherry-pick it into the branch it belongs to. Unreachable commits are only removed by garbage collection after the reflog entries expire (by default 30 days for unreachable entries, 90 for reachable ones), so recover them soon.

KEY POINTS

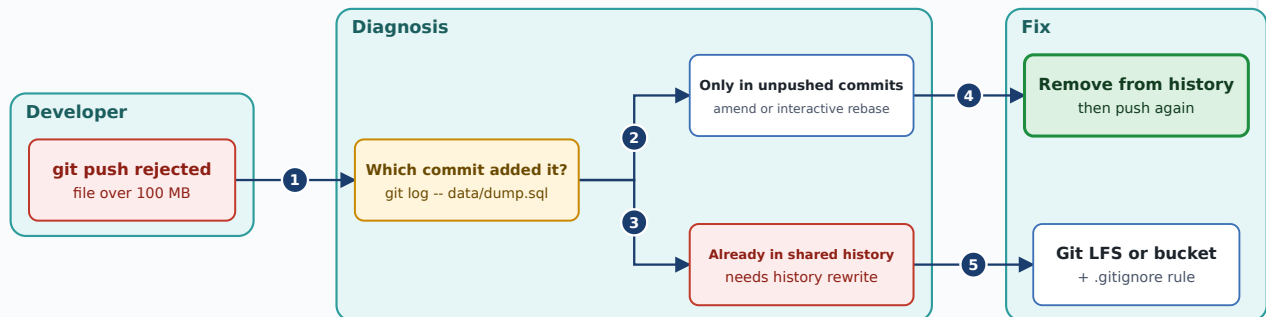
- When Git warns about detached HEAD, run `git switch -c new-branch` before committing.
- Reflog is local only: commits that were never pushed exist only on that laptop.

CONFIGURATION EXAMPLE

```
git reflog -n 15
# a1b2c3d HEAD@{1}: commit: add retry to payment client
git branch rescue/payment-retry a1b2c3d
git switch feature/payments
git merge rescue/payment-retry    # or: git cherry-pick <sha1>..a1b2c3d
```

Q367**Troubleshooting**

`git push` is rejected with `remote: error: File data/dump.sql is 512.00 MB; this exceeds GitHub's file size limit of 100.00 MB`. Deleting the file in a new commit does not help. How do you fix it?

FLOW DIAGRAM

- 1 A later delete does not remove the blob from history
- 2 Commits are only local: rewrite is safe
- 3 Coordinate with the team first
- 4 Drop the file from the commit that added it
- 5 `git filter-repo`, `force-push`, `re-clone`

EXPLANATION

The push sends every commit, and the commit that added the file still contains it even if a later commit deletes it, so the server still sees the 512 MB blob. If the commits are not pushed yet, remove the file from the commit that added it: `git rm --cached` and `git commit --amend` for the last commit, or an interactive rebase for an older one, or simply reset back and recommit without it. If it is already in history others have, use `git filter-repo --path data/dump.sql --invert-paths`, `force-push`, and ask everyone to `re-clone`. Store large files in Git LFS or a Cloud Storage bucket and add the path to `.gitignore`.

KEY POINTS

- Add a pre-commit hook that blocks files over a few MB.
- Keep data dumps in Cloud Storage and reference them by path, not in Git.

CONFIGURATION EXAMPLE

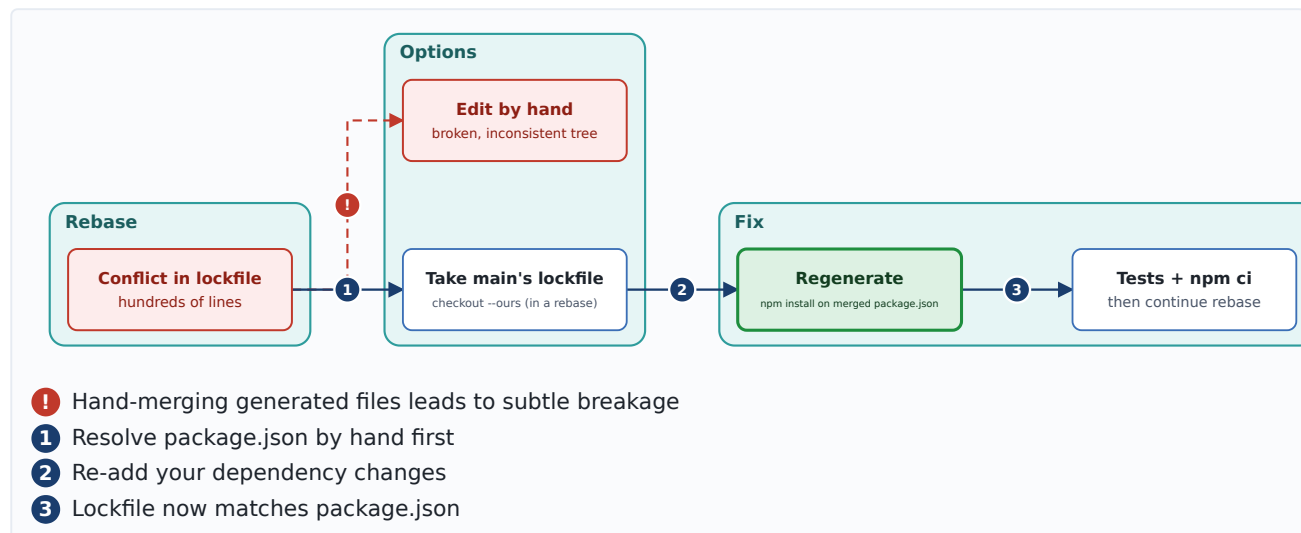
```

git log --oneline --stat -- data/dump.sql
# not pushed yet, file added in the last commit
git rm --cached data/dump.sql && echo "data/*.sql" >> .gitignore
git commit --amend --no-edit
# already shared: rewrite history (coordinate with the team)
git filter-repo --path data/dump.sql --invert-paths
git push --force-with-lease origin main
  
```

Q368**Troubleshooting**

Rebasing a feature branch onto main gives a merge conflict in `package-lock.json` with hundreds of conflicting lines. How do you resolve it safely?

FLOW DIAGRAM



EXPLANATION

A lockfile is generated, so do not merge it by hand. Resolve `package.json` first (keep both sides' dependency changes), then take main's version of the lockfile and let the package manager regenerate it: `npm install` updates the lockfile to match the merged `package.json`. Commit the regenerated file, run `npm ci` and the tests to confirm it installs cleanly, and continue the rebase. Note that during a rebase `--ours` is the branch you are rebasing onto (main) and `--theirs` is your commit, the opposite of a merge.

KEY POINTS

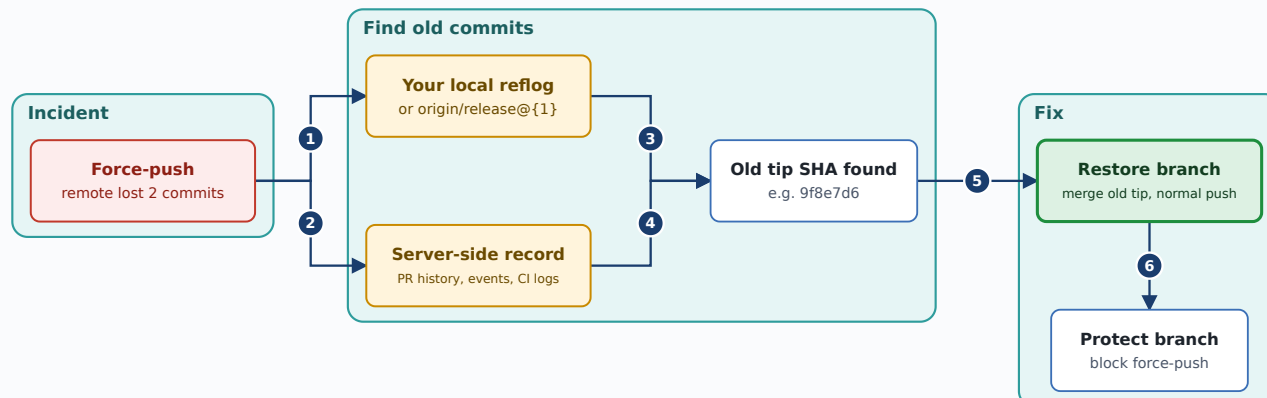
- Keep dependency upgrades in small, separate PRs to reduce lockfile conflicts.
- CI should run `npm ci`, which fails if the lockfile and `package.json` disagree.

CONFIGURATION EXAMPLE

```
git rebase origin/main
# resolve package.json by hand, then:
git checkout --ours package-lock.json # during rebase: 'ours' = main
npm install
git add package.json package-lock.json
npm ci && npm test
git rebase --continue
```

Q369**Troubleshooting**

A teammate ran `git push --force` on the shared `release/2.4` branch and two of your commits pushed yesterday are gone from GitHub. How do you recover them?

FLOW DIAGRAM

- 1 Your clone still has the commits
- 2 If no local copy, the SHA is often in PR or CI
- 4 Fetch the SHA if the server still has it
- 5 Combine both histories, no second force-push
- 6 Stop it happening again

EXPLANATION

Anyone who fetched before the force-push still has the old commits: `git relog show origin/release/2.4` shows where the remote-tracking branch pointed before, and local branch relogs show your own commits. If nobody has a copy, the old SHA often appears in the PR timeline, CI build logs or the repository's activity view, and GitHub can usually serve that commit by SHA for a while. Create a branch at the old tip, merge it with the current branch (or cherry-pick the missing commits) and push normally, so the teammate's new commits are kept too. Then turn on branch protection to block force-pushes, and use `--force-with-lease` for personal branches.

KEY POINTS

- Branch protection rules should forbid force-pushes and deletions on shared branches.
- `--force-with-lease` refuses to overwrite commits you have not fetched.

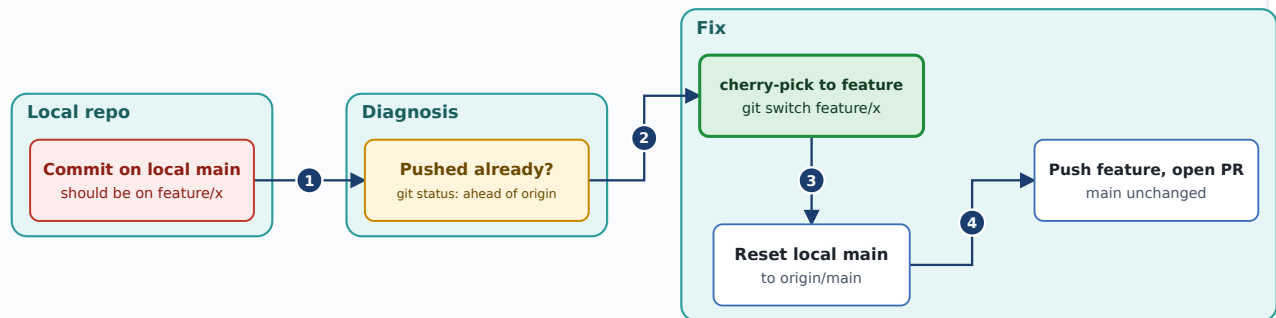
CONFIGURATION EXAMPLE

```

git relog show origin/release/2.4 -n 5
# 9f8e7d6 refs/remotes/origin/release/2.4@{1}: fetch: fast-forward
git switch -c recover/release-2.4 9f8e7d6
git fetch origin && git switch release/2.4 && git reset --hard origin/release/2.4
git merge recover/release-2.4          # keep both sides
git push origin release/2.4           # normal push, no force
  
```

Q370**Troubleshooting**

You committed a fix to `main` locally instead of your feature branch (not pushed yet), and the push to `main` is blocked by branch protection anyway. How do you move the commit to the right branch?

FLOW DIAGRAM

- ❶ Not pushed: safe to move it
- ❷ Copy the commit onto the feature branch
- ❸ Then remove it from local `main`
- ❹ Normal PR flow

EXPLANATION

Since the commit is only local, you can move it safely. Note its SHA, switch to the feature branch and `git cherry-pick` it (or, if the feature branch does not exist yet, create it from the current position with `git switch -c feature/x`, which carries the commit along). Then reset local `main` back to `origin/main` so it matches the remote again. Push the feature branch and open a PR. If the commit had already been pushed to a shared branch, use `git revert` there instead of resetting, so you do not rewrite shared history.

KEY POINTS

- Show the current branch in your shell prompt.
- Protect `main` so direct pushes fail early instead of reaching production.

CONFIGURATION EXAMPLE

```

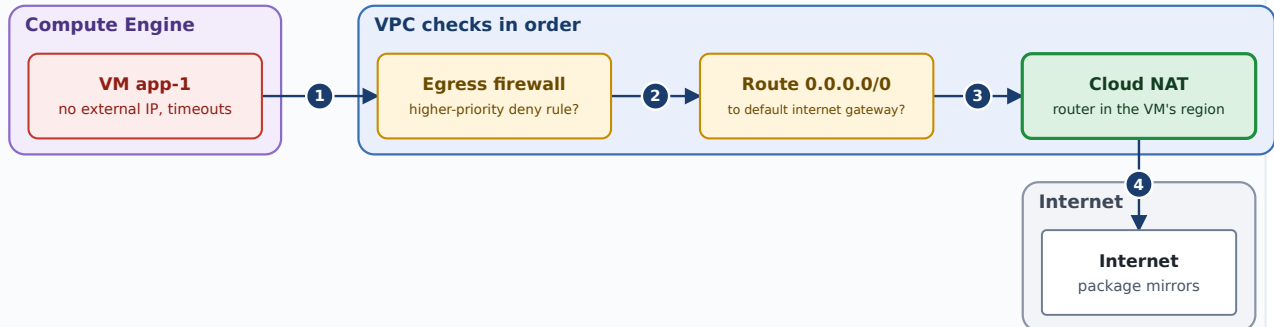
git log --oneline -1          # e.g. 4d5e6f7 fix: null check in parser
git switch feature/x
git cherry-pick 4d5e6f7
git switch main
git reset --hard origin/main  # only because the commit was never pushed
git push -u origin feature/x
  
```


14. Troubleshooting Scenarios

Realistic production incidents on Google Cloud, each with a step-by-step diagnosis path and the fix.

Q371 A new VM without an external IP cannot run `apt-get update`; the connection times out. Other VMs in the project work. How do you troubleshoot it?

FLOW DIAGRAM



- 1 Check effective egress rules and target tags
- 2 Default route present and not overridden
- 3 Without an external IP the VM needs Cloud NAT
- 4 NAT must cover this subnet (and secondary ranges)

EXPLANATION

Run a Connectivity Test from the VM to a public IP on port 443; it evaluates firewall rules, routes and NAT and says which one drops the packet. Common causes are an egress deny rule with higher priority than the allow (often tag-based, so check the VM's network tags), a deleted or replaced default route, and, most often for VMs without external IPs, no Cloud NAT gateway for that region or subnet. Other VMs working usually means they have external IPs or sit in a subnet the NAT covers. If the VM only needs Google APIs, Private Google Access on the subnet is enough.

KEY POINTS

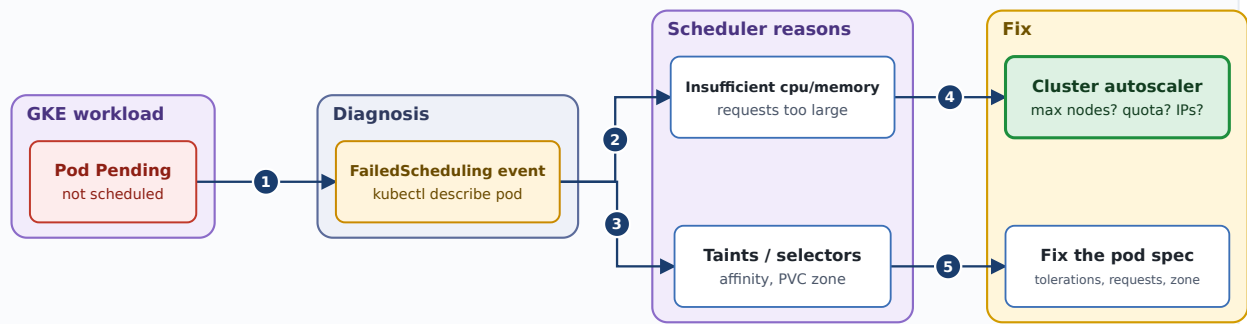
- Cloud NAT is regional: a NAT in `us-central1` does not help a VM in `eu-west1`.
- Check NAT logs for dropped connections due to port exhaustion.

CONFIGURATION EXAMPLE

```
gcloud network-management connectivity-tests create vm-egress \
  --source-instance=projects/prod/zones/us-central1-a/instances/app-1 \
  --destination-ip-address=8.8.8.8 --destination-port=443 --protocol=TCP
gcloud compute routers create nat-router --network=prod-vpc --region=us-central1
gcloud compute routers nats create nat-cfg --router=nat-router --region=us-central1 \
  --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges
```

Q372 After a deployment, several GKE pods stay in Pending for 20 minutes. How do you find the cause and fix it?

FLOW DIAGRAM



- ❶ The event message says why no node fits
- ❷ No node has enough allocatable resources
- ❸ Nodes exist but the pod is not allowed on them
- ❹ Autoscaler should add nodes: check why it did not
- ❺ Correct the constraints in the manifest

EXPLANATION

Start with `kubectl describe pod` and read the FailedScheduling event: it lists how many nodes failed each check (insufficient CPU or memory, untolerated taints, node affinity, volume zone conflicts). If resources are short, check whether the cluster autoscaler tried to scale up (its events and logs show reasons such as max node count reached, quota exceeded or no IP addresses left in the pod range). If the pod is not allowed on nodes, fix tolerations, node selectors or affinity, or make sure a PersistentVolume is in a zone where nodes exist. Also check for requests far higher than real usage, which waste capacity.

KEY POINTS

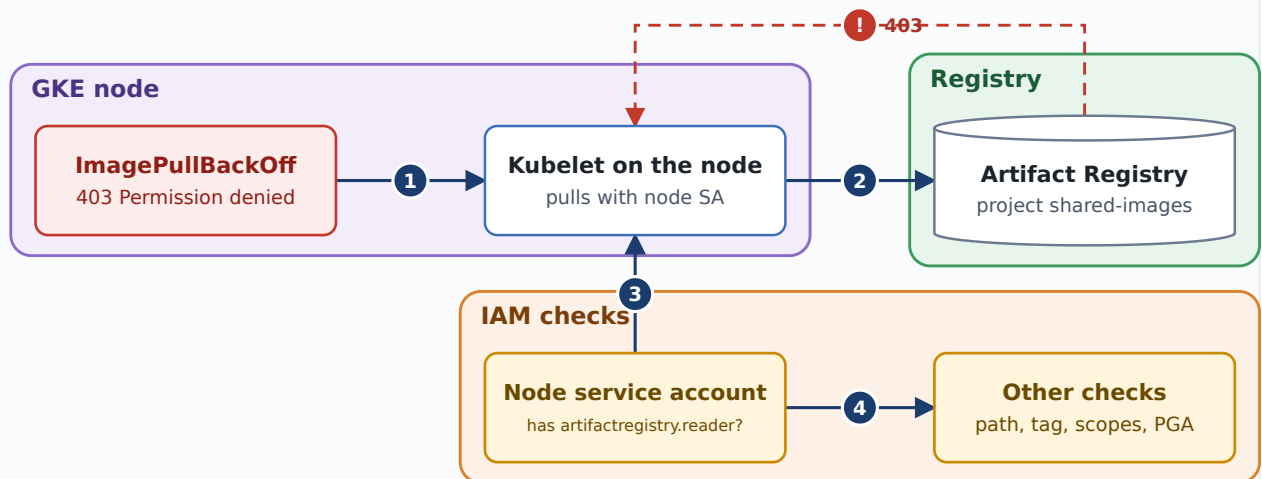
- Pod IP range exhaustion looks like capacity trouble but needs a larger range or new subnet.
- On Autopilot, check the requested resources against Autopilot's supported ranges.

CONFIGURATION EXAMPLE

```
kubectl describe pod web-7d9f -n shop | sed -n '/Events:/,$p'
kubectl get events -n shop --field-selector reason=FailedScheduling
kubectl describe nodes | grep -A6 'Allocated resources'
kubectl get events -n kube-system --field-selector source=cluster-autoscaler
```

Q373 Pods in a new GKE cluster fail with ImagePullBackOff for an image in Artifact Registry in another project. The team already set up Workload Identity for the app. What is wrong?

FLOW DIAGRAM



- 1 describe pod shows the exact pull error
- 2 Image pull uses the node's service account
- ! 403 the node SA has no reader role in shared-images
- 3 Grant Artifact Registry Reader to the node SA
- 4 Then check image path, access scopes, egress

EXPLANATION

Image pulls are done by the kubelet with the node's service account, not by the pod's Kubernetes service account, so Workload Identity does not help here. Read the event text: 403 means permission, 404 or not found usually means a wrong path, region or tag. Grant `roles/artifactregistry.reader` to the node service account on the repository in the other project, and check the node's OAuth scopes allow storage read (or cloud-platform). For private clusters make sure the subnet has Private Google Access or Cloud NAT so the node can reach `*.pkg.dev`.

KEY POINTS

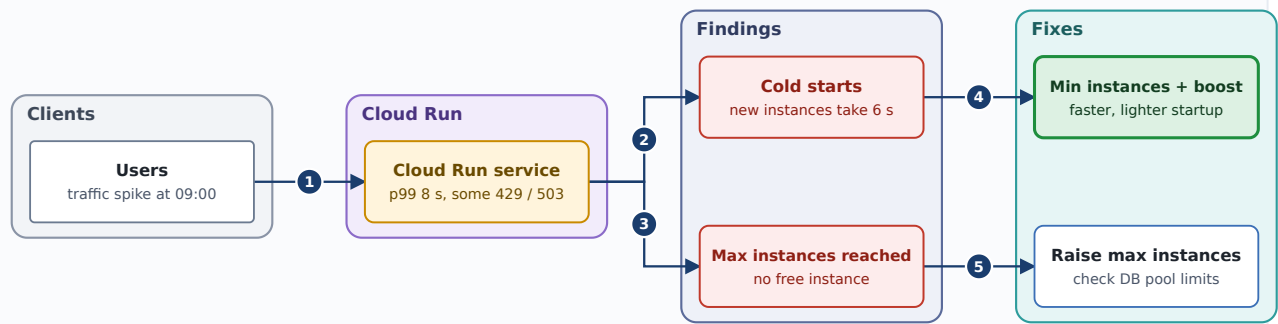
- Use a dedicated, least-privilege node service account instead of the default compute SA.
- An org policy restricting cross-project sharing can also block the grant.

CONFIGURATION EXAMPLE

```
kubectl describe pod web-abc -n shop | grep -A4 -i 'failed'
gcloud artifacts repositories add-iam-policy-binding app-images \
  --project=shared-images --location=us \
  --member=serviceAccount:gke-nodes@prod.iam.gserviceaccount.com \
  --role=roles/artifactregistry.reader
```

Q374 A Cloud Run API shows p99 latency of 8 seconds and some 429/503 errors every morning at 09:00 when traffic jumps. How do you diagnose and fix it?

FLOW DIAGRAM



- 1 Requests queue while instances start
- 2 Startup latency metric and logs show slow init
- 3 Instance count flat at the maximum
- 4 Keep warm instances and speed up startup
- 5 Allow more instances or requests per instance

EXPLANATION

Look at the container startup latency and instance count metrics next to request latency. If latency spikes line up with new instances starting, cold starts are the cause: set `--min-instances` so warm instances are ready, enable startup CPU boost, and make startup lighter (lazy loading, smaller image, no heavy work before listening on the port). If the instance count sits at the maximum while 429s appear, raise `--max-instances` or concurrency, but check downstream limits first, such as Cloud SQL connections. A failing startup probe or a container that crashes on start produces 503s, which show up in the service logs.

KEY POINTS

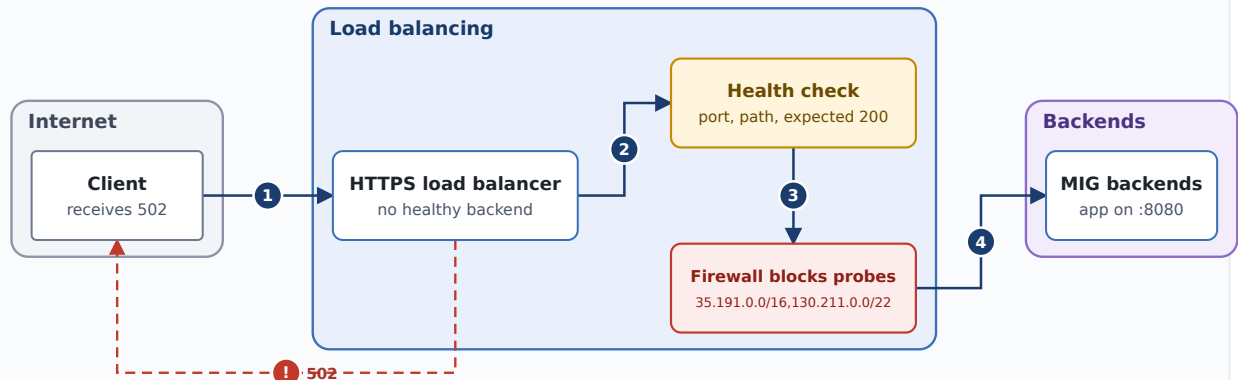
- Min instances cost money even when idle; size them for the morning ramp only if needed.
- Cloud Scheduler can raise min instances before a known peak.

CONFIGURATION EXAMPLE

```
gcloud run services update orders-api --region=us-central1 \
  --min-instances=3 --max-instances=100 --concurrency=80 --cpu-boost
```

Q375 Users get 502 errors from a global external HTTPS load balancer in front of a managed instance group, even though the app responds when you curl it on the VM. How do you troubleshoot it?

FLOW DIAGRAM



- 1 Request reaches the Google front end
- ! 502 no healthy backend or backend closed the connection
- 2 Check backend health status first
- 3 Probes come from Google's health-check ranges
- 4 Allow them on the serving port, then the check passes

EXPLANATION

Run `gcloud compute backend-services get-health`; if backends are UNHEALTHY, the load balancer has nowhere to send traffic. The usual cause is a missing ingress firewall rule for the health-check ranges 35.191.0.0/16 and 130.211.0.0/22 on the app port, followed by a health check on the wrong port or path, or a named port on the instance group that does not match. If backends are healthy but 502s continue, check the load balancer logs for `statusDetails` such as `backend_connection_closed_before_data_sent_to_client`, which often means the app's keepalive timeout is shorter than the load balancer's (set it above 600 seconds). Curl on the VM only proves the app runs, not that the path from Google's front ends is open.

KEY POINTS

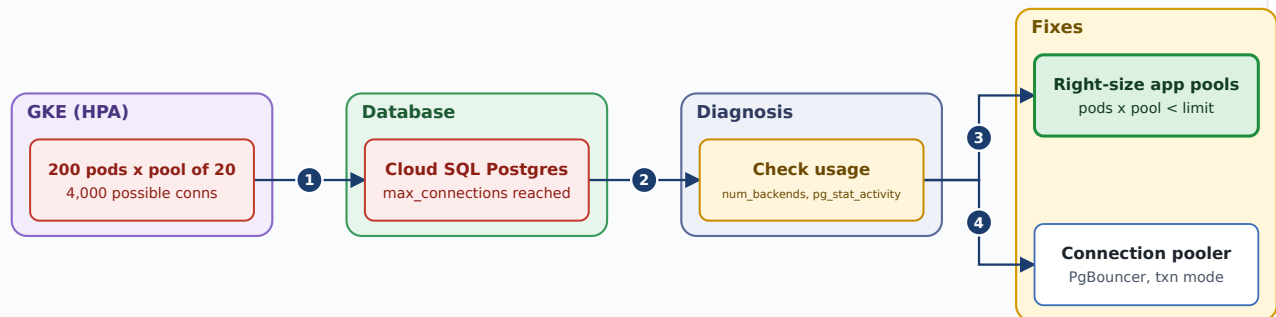
- Health checks for the backend and for MIG autohealing can differ; check both.
- The load balancer logs' `statusDetails` field names the exact failure.

CONFIGURATION EXAMPLE

```
gcloud compute backend-services get-health web-bs --global
gcloud compute firewall-rules create allow-lb-health-checks \
  --network=prod-vpc --direction=INGRESS --action=ALLOW --rules=tcp:8080 \
  --source-ranges=35.191.0.0/16,130.211.0.0/22 --target-tags=web
gcloud compute instance-groups set-named-ports web-mig --region=us-central1 \
  --named-ports=http:8080
```

Q376 After the HPA scaled a service to 200 pods, new requests fail with `FATAL: sorry, too many clients already` from Cloud SQL for PostgreSQL. How do you fix it properly?

FLOW DIAGRAM



- 1 Each pod opens its own pool
- 2 See who holds connections and how many idle
- 3 Smaller pools, cap HPA max replicas
- 4 Many clients share few server connections

EXPLANATION

Check the `database/postgresql/num_backends` metric and `pg_stat_activity` to see which apps hold connections and how many are idle. The real problem is multiplication: pods times pool size exceeds `max_connections`, which is sized from instance memory. Reduce the per-pod pool (most pods need a few connections), cap HPA maximum replicas to what the database can serve, and add a pooler such as PgBouncer in transaction mode (or Cloud SQL managed connection pooling where available) so many clients share fewer server connections. Raise `max_connections` only if the instance has memory for it, and remember the flag change restarts the instance.

KEY POINTS

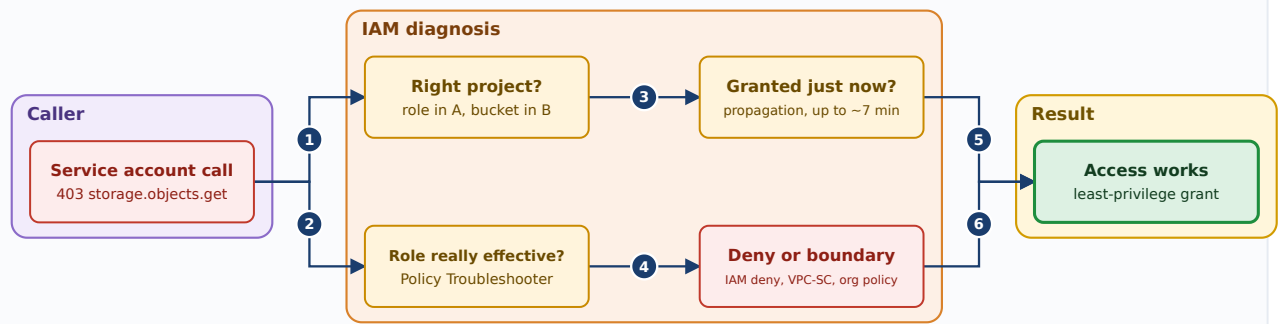
- `gcloud sql instances patch --database-flags` replaces all flags: include the existing ones.
- Idle-in-transaction sessions hold locks too; set idle timeouts.

CONFIGURATION EXAMPLE

```
SELECT username, application_name, state, count(*)
FROM pg_stat_activity
GROUP BY 1, 2, 3 ORDER BY 4 DESC;
SHOW max_connections;
```

Q377 A pipeline's service account gets `403 PERMISSION_DENIED` reading a bucket, although the team says they granted the role. How do you work out why?

FLOW DIAGRAM



- ❶ Error names the permission and resource
- ❷ Troubleshooter shows which binding applies
- ❸ Correct binding added recently?
- ❹ Allowed by IAM but blocked elsewhere
- ❺ Fix the deny rule or perimeter, not the role

EXPLANATION

Read the error: it names the missing permission (for example `storage.objects.get`) and the resource. Then check the obvious mismatches: the role granted in a different project than the bucket, to a different service account (the default compute SA instead of the pipeline SA), or a role that lacks that permission. Policy Troubleshooter shows whether any binding grants the permission to that principal and which policy decides. If IAM allows it, look for IAM deny policies, a VPC Service Controls perimeter (the error mentions it), or conditions on the binding; and if the grant is fresh, wait a few minutes, since IAM changes usually apply within about a minute but can take up to 7.

KEY POINTS

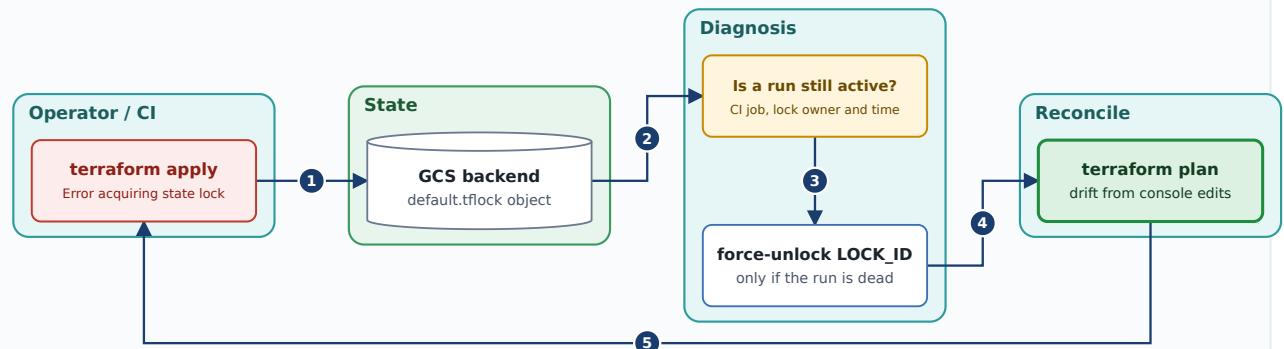
- Check which identity is really calling: `gcloud auth list` or the audit log's `principalEmail`.
- Audit logs for the denied call show the caller and the permission.

CONFIGURATION EXAMPLE

```
gcloud policy-intelligence troubleshoot-policy iam \
  //storage.googleapis.com/projects/_/buckets/proj-b-data \
  --principal-email=etl@proj-a.iam.gserviceaccount.com \
  --permission=storage.objects.get
```

Q378 terraform apply fails with "Error acquiring the state lock" on a GCS backend, and when it finally runs the plan shows unexpected changes. How do you handle both problems?

FLOW DIAGRAM



- ❶ Lock is an object next to the state file
- ❷ Error shows who holds the lock and since when
- ❸ Crashed or cancelled job left the lock
- ❹ Refresh-only plan shows drift
- ❺ Import or revert manual changes, then apply

EXPLANATION

The GCS backend locks by creating a `.tflock` object next to the state; the error shows the lock ID, who holds it and since when. First make sure no pipeline is still running (a second apply in parallel is exactly what the lock prevents); if the holder crashed or was cancelled, run `terraform force-unlock LOCK_ID`. For the unexpected plan, run `terraform plan -refresh-only` to see drift from changes made in the console or by other tools, then either bring the code in line (and `import` resources created by hand) or let Terraform revert them. Prevent repeats by running applies only from CI with a concurrency limit and restricting write IAM in production.

KEY POINTS

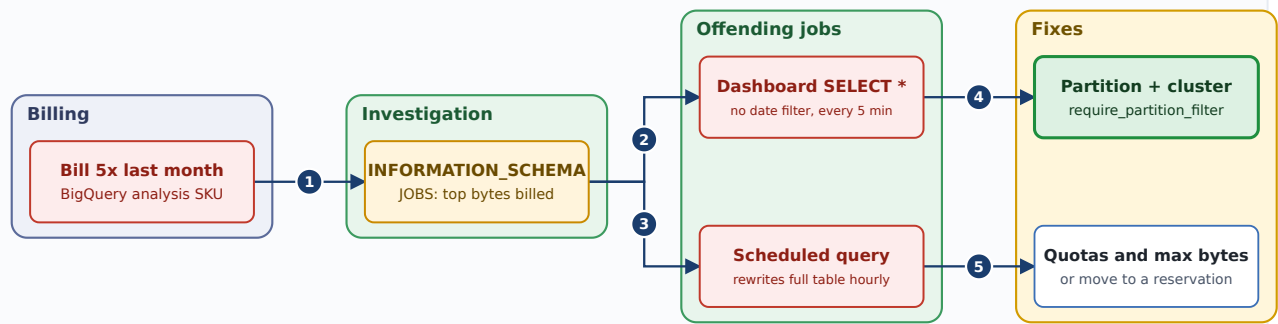
- Never delete the state object; enable versioning on the state bucket so you can roll back.
- Schedule a daily plan to detect drift early.

CONFIGURATION EXAMPLE

```
terraform force-unlock 1727512345678901
terraform plan -refresh-only
# adopt a resource created by hand (Terraform 1.5+)
import {
  to = google_storage_bucket.logs
  id = "prod-app-logs"
}
```


Q379 Finance reports that the BigQuery bill is five times last month's. How do you find the cause and stop it?

FLOW DIAGRAM



- 1 Billing export shows which project and SKU
- 2 Group jobs by user and query text
- 3 Look for repeated and scheduled jobs
- 4 Scan only the partitions and columns needed
- 5 Incremental MERGE, caps on bytes billed

EXPLANATION

Use the billing export to find the project and SKU, then query `INFORMATION_SCHEMA.JOBS` for that region to list the jobs with the most bytes billed, grouped by user, service account and query text. Typical culprits are a dashboard running `SELECT *` over an unpartitioned table every few minutes, a scheduled query that rewrites a whole table instead of merging new data, or a new pipeline running on-demand. Fix the queries (select only needed columns, partition and cluster the tables, require partition filters, use incremental loads), and add guardrails: custom quotas on bytes per day, `maximum_bytes_billed` on scheduled jobs, and budget alerts. If the workload is steady and large, compare with capacity pricing.

KEY POINTS

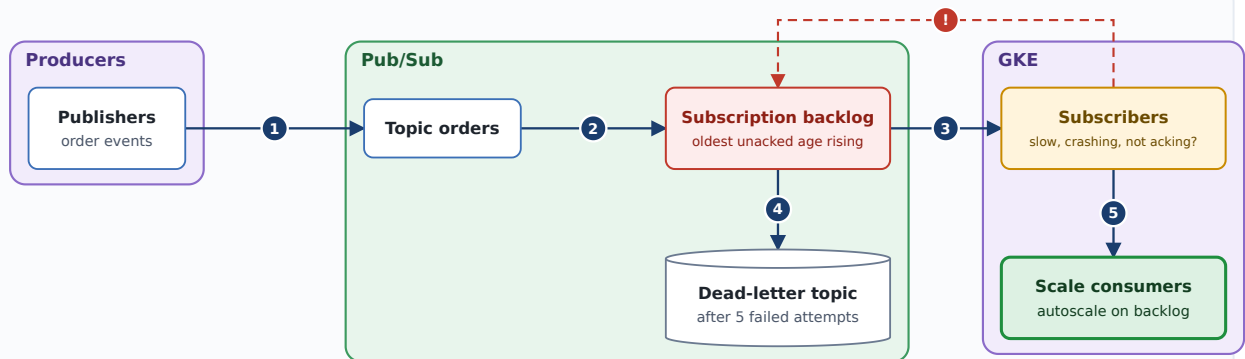
- `LIMIT` does not reduce bytes scanned on a normal table.
- BI Engine or materialized views can serve repeated dashboard queries cheaply.

CONFIGURATION EXAMPLE

```
SELECT user_email, query, COUNT(*) AS runs,
       SUM(total_bytes_billed) / POW(1024, 4) AS tib_billed
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
      AND job_type = 'QUERY'
GROUP BY 1, 2 ORDER BY tib_billed DESC LIMIT 20;
```

Q380 The `oldest_unacked_message_age` of an orders subscription keeps growing and customers see delayed confirmations. How do you troubleshoot it?

FLOW DIAGRAM



- ❶ Publish rate higher than usual?
- ❸ Delivery to subscribers
- ❹ Nack or ack deadline missed, so messages are redelivered
- ❺ Poison messages moved out of the way
- ❻ Add consumers if they are healthy but too few

EXPLANATION

Compare publish rate with ack rate and check `num_undelivered_messages` and `oldest_unacked_message_age`: a growing oldest age with a steady count often means a few messages keep failing, while a growing count means consumers are too slow overall. Look at consumer logs and the `expired` ack or nack metrics; a message that always throws or takes longer than the ack deadline is redelivered forever, so set a dead-letter topic with a maximum number of delivery attempts. If consumers are healthy but too few, scale them on the backlog metric and tune flow control. Also check for a slow downstream dependency, such as a database, that limits all consumers.

KEY POINTS

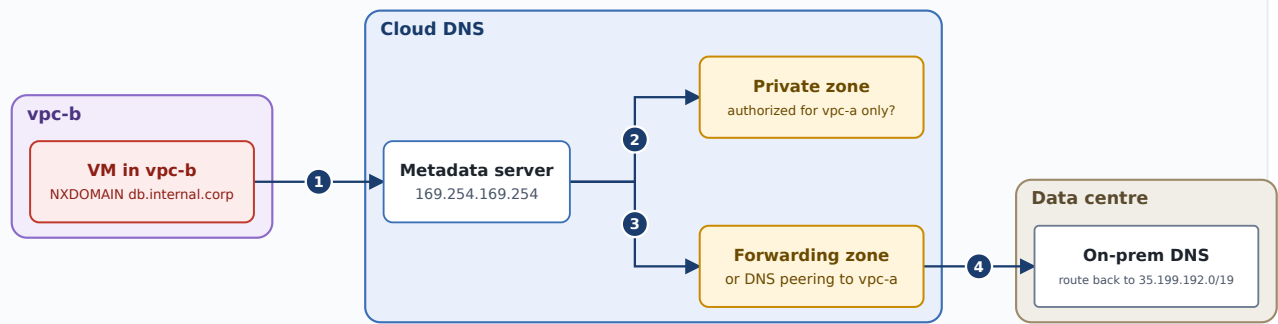
- Dead-lettering needs the Pub/Sub service agent to have publisher and subscriber roles.
- Exactly-once delivery and ordering keys reduce parallelism: check whether they are enabled.

CONFIGURATION EXAMPLE

```
gcloud pubsub subscriptions update orders-sub \
  --ack-deadline=60 --dead-letter-topic=orders-dlq --max-delivery-attempts=5
gcloud pubsub subscriptions describe orders-sub \
  --format='yaml(ackDeadlineSeconds,deadLetterPolicy,retryPolicy)'
```

Q381 VMs in a new VPC cannot resolve `db.internal.corp`, which works from the original VPC and comes partly from on-prem DNS. What do you check?

FLOW DIAGRAM



- ❶ All VM lookups go to Cloud DNS via the metadata server
- ❷ Private zones answer only for authorized networks
- ❸ Names hosted on-prem need a forwarding zone
- ❹ Queries come from 35.199.192.0/19; route and firewall must allow them

EXPLANATION

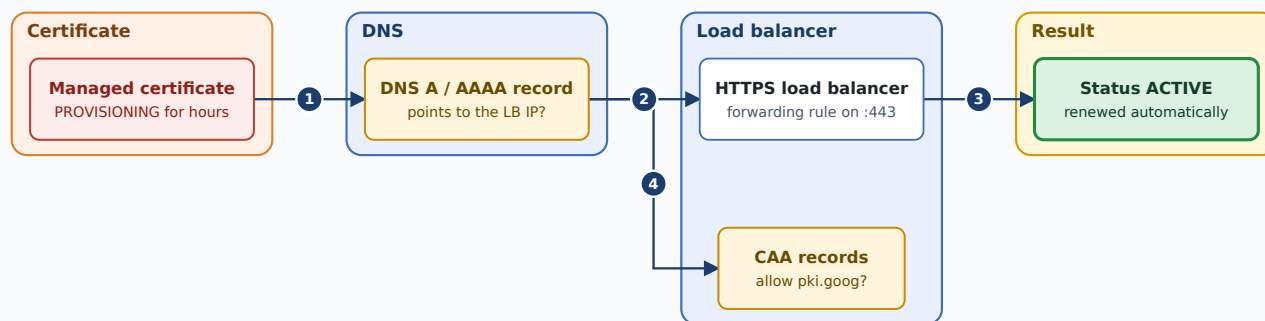
VMs resolve through the metadata server (169.254.169.254), which asks Cloud DNS, and a private zone only answers for networks listed in its visibility. Add vpc-b to the private zone, or create a DNS peering zone from vpc-b to vpc-a so it uses vpc-a's resolution. For names that live on-prem, vpc-b needs its own forwarding zone (or peering to the VPC that has one), and the on-prem DNS servers must be reachable: Cloud DNS sends forwarded queries from 35.199.192.0/19, so on-prem routing (via Cloud Router advertisements over VPN or Interconnect) and firewalls must allow that range. Test with `dig` from the VM against the metadata server.

KEY POINTS

- A custom `/etc/resolv.conf` pointing elsewhere bypasses Cloud DNS entirely.
- DNS server policies with inbound forwarding let on-prem resolve Google Cloud private zones.

Q382 A Google-managed certificate for `www.example.com` on a new HTTPS load balancer has been **PROVISIONING** for hours. How do you get it active?

FLOW DIAGRAM



- 1 Domain status shows `FAILED_NOT_VISIBLE` or pending
- 2 Domain must resolve to this load balancer's IP
- 3 Validation succeeds through the LB, cert goes **ACTIVE**
- 4 CAA records must not block Google's CAs

EXPLANATION

Describe the certificate and read the per-domain status. `FAILED_NOT_VISIBLE` means the domain does not resolve to the load balancer's IP yet (still pointing at the old host, a CDN, or a record not propagated), and `FAILED_CAA_CHECKING` or `FAILED_CAA_FORBIDDEN` means CAA records block Google's certificate authorities. Also confirm the certificate is attached to the target HTTPS proxy and the forwarding rule on port 443 exists. Provisioning can take up to about an hour after DNS is right. To avoid downtime on migrations, use Certificate Manager with DNS authorization, which issues the certificate before you move traffic.

KEY POINTS

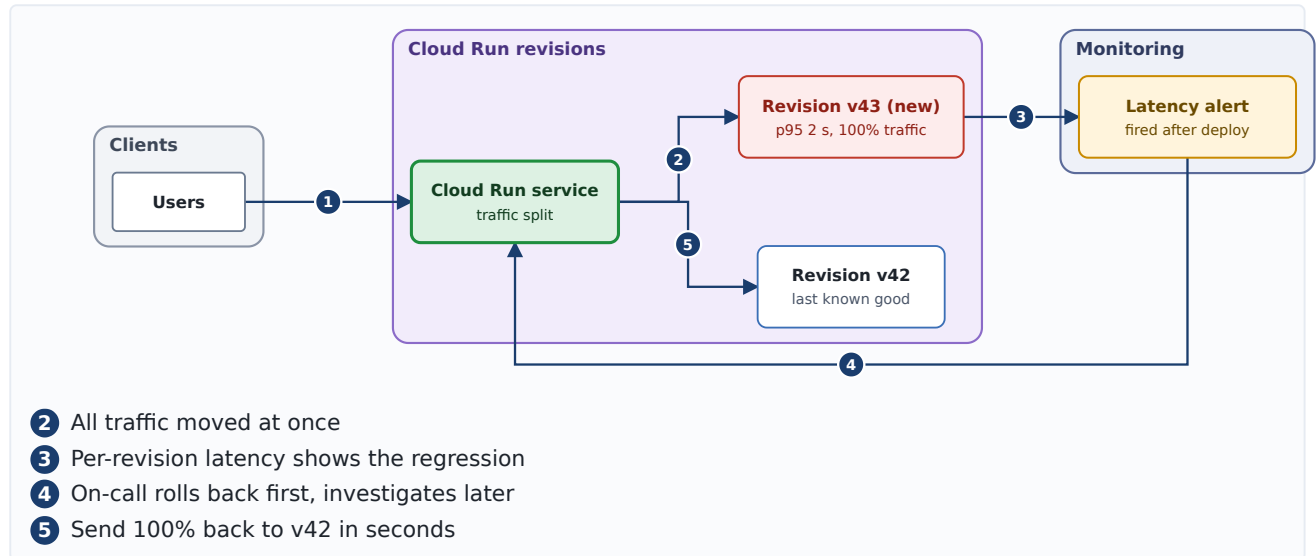
- Each domain on the certificate must pass; one bad name keeps the whole cert pending.
- Check both IPv4 and IPv6 records if the LB has both.

CONFIGURATION EXAMPLE

```
gcloud compute ssl-certificates describe www-cert --global \
  --format='yaml(managed.status,managed.domainStatus)'
dig +short www.example.com A
gcloud compute forwarding-rules list --global \
  --format='table(name,IPAddress,portRange,target)'
```

Q383 Ten minutes after deploying revision v43 of a Cloud Run service, p95 latency doubles and an alert fires. What do you do first, and how do you avoid this next time?

FLOW DIAGRAM



EXPLANATION

Mitigate first: shift all traffic back to the last good revision with `gcloud run services update-traffic`, which takes effect in seconds because the old revision still exists. Confirm latency recovers, then investigate v43 using per-revision metrics, traces and logs (a new dependency call, a missing index, smaller CPU or memory settings). Next time deploy with `--no-traffic` and a tag, send a small share such as 5% to the new revision, and compare its latency and errors against the old one before moving more. Cloud Deploy can automate these canary phases with verification.

KEY POINTS

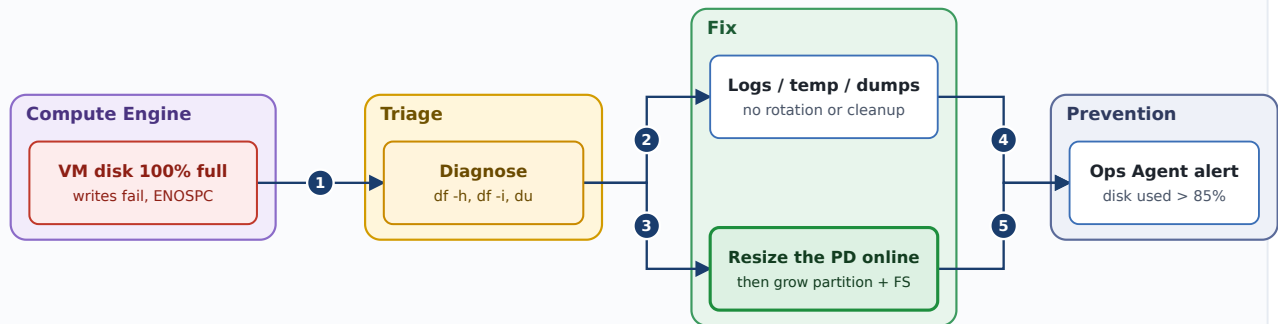
- Roll back first, debug second: the old revision is the fastest fix.
- Keep rollback a one-line command in the runbook.

CONFIGURATION EXAMPLE

```
gcloud run services update-traffic orders-api --region=us-central1 \
  --to-revisions=orders-api-00042-abc=100
# next release: canary
gcloud run deploy orders-api --image=IMAGE --region=us-central1 --no-traffic --tag=canary
gcloud run services update-traffic orders-api --region=us-central1 --to-tags=canary=5
```

Q384 An application on a Compute Engine VM starts failing with `No space left on device`. How do you recover and stop it from happening again?

FLOW DIAGRAM



- 1 Space or inodes? Which directory?
- 2 Clean up and add logrotate or retention
- 3 Need more space: snapshot, then resize
- 4 Alert before it fills again

EXPLANATION

Check whether blocks or inodes are exhausted (`df -h` and `df -i`) and find the largest directories with `du`; common causes are application logs without rotation, temp files, core dumps, or old container images. Free space quickly by removing or compressing what is safe to delete, and remember a deleted file held open by a process still uses space until that process restarts (`ls -of +L1`). If the data is legitimate, take a snapshot, resize the persistent disk online, then grow the partition and file system. Install the Ops Agent and alert on disk utilisation above about 85%, and ship logs to Cloud Logging instead of keeping them locally.

KEY POINTS

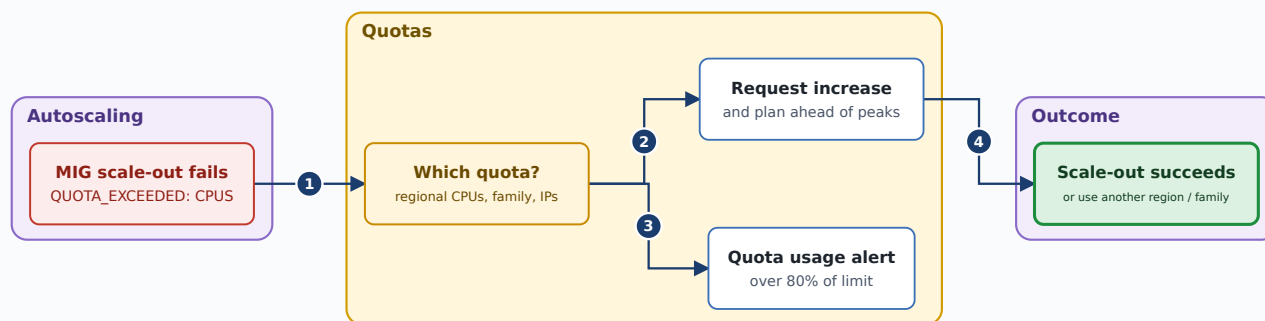
- Persistent disks can grow online but never shrink.
- Separate data disks from the boot disk so the OS keeps working when data fills up.

CONFIGURATION EXAMPLE

```
df -h; df -i
sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail -15
gcloud compute disks snapshot app-1 --zone=us-central1-a --snapshot-names=app-1-pre-resize
gcloud compute disks resize app-1 --size=200GB --zone=us-central1-a
sudo growpart /dev/sda 1 && sudo resize2fs /dev/sda1
```

Q385 During a traffic spike a managed instance group stops scaling and logs `QUOTA_EXCEEDED` for CPUS in `us-central1`. How do you handle it now and in future?

FLOW DIAGRAM



- ❶ Error names the metric and region
- ❷ Short term: request more, or free unused VMs
- ❸ Long term: alert well before the limit
- ❹ Also check stockout errors, which need other zones

EXPLANATION

Read the error and the Quotas page to see which quota is hit: regional CPUs, a machine-family quota such as N2 CPUs, in-use external IPs, persistent disk or GPUs. For immediate relief, delete idle VMs, move part of the load to another region or machine family if the architecture allows it, and submit a quota increase request (small increases are often approved quickly, large ones can take longer). Then create alerts on quota usage (Cloud Monitoring has quota metrics) at around 80%, and review quotas before planned peaks. Note that `ZONE_RESOURCE_POOL_EXHAUSTED` is a capacity stockout, not a quota: spread the MIG across zones or use reservations.

KEY POINTS

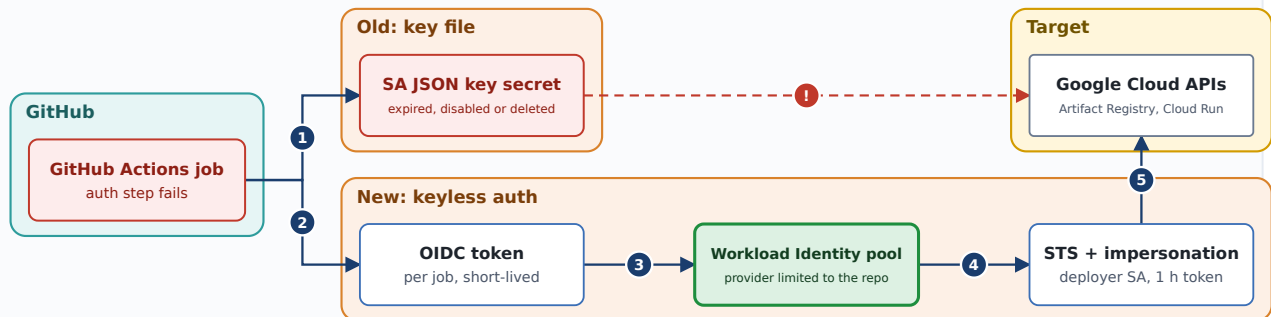
- Quota limits are per project and region; a new project starts with low defaults.
- Reservations guarantee capacity in a zone; quotas only allow usage.

CONFIGURATION EXAMPLE

```
gcloud compute regions describe us-central1 --flatten=quotas \
  --format='table(quotas.metric,quotas.usage,quotas.limit)' \
  | grep -E 'CPUS|IN_USE_ADDRESSES|SSD'
```

Q386 All GitHub Actions deployments to Google Cloud suddenly fail with `invalid_grant` errors. They use a service account JSON key stored as a repository secret. What happened and what is the long-term fix?

FLOW DIAGRAM



- 1 Old: long-lived key in repository secrets
- ! invalid_grant key expired by org policy or rotated
- 2 New: job requests an OIDC token (id-token: write)
- 3 Provider checks issuer and attribute condition
- 4 Exchanged for a short-lived Google token
- 5 Deploy with the service account's permissions

EXPLANATION

Check the key in IAM: it was probably deleted, disabled, or expired by an org policy such as `iam.serviceAccountKeyExpiryHours`, or the service account was disabled; audit logs show who changed it. You can unblock by issuing a new key, but the long-term fix is to remove keys and use Workload Identity Federation: GitHub issues an OIDC token per job, a workload identity pool provider trusts GitHub's issuer with an attribute condition limiting it to your repository (and branch), and the job exchanges the token for short-lived credentials that impersonate the deployer service account. Grant `roles/iam.workloadIdentityUser` on that service account to the matching principal set, then delete the old key and enforce `iam.disableServiceAccountKeyCreation`.

KEY POINTS

- Always set an attribute condition on the provider; without it any GitHub repo could try to use the pool.
- No secret to leak or rotate: tokens live about an hour.

CONFIGURATION EXAMPLE

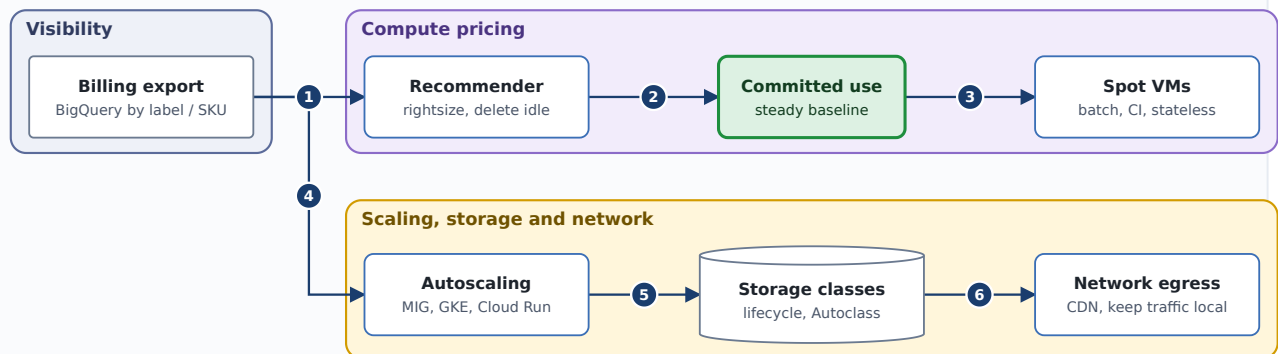
```
permissions:
  id-token: write
  contents: read
steps:
  - uses: google-github-actions/auth@v2
    with:
      workload_identity_provider:
        projects/123456789/locations/global/workloadIdentityPools/github/providers/my-repo
        service_account: deployer@prod-app.iam.gserviceaccount.com
```


15. Cost Optimization, DR & Architecture Design

System-design questions that tie together cost, resilience and migration choices, with the numbers interviewers expect you to reason about.

Q387 The CFO asks you to cut a growing Google Cloud bill by 25% without hurting reliability. Where do you start and which levers do you pull?

FLOW DIAGRAM



- 1 Find the top SKUs, idle VMs and unattached disks
- 2 Rightsize first, then commit only the steady baseline
- 3 Run fault-tolerant work on Spot capacity
- 4 Scale with demand; schedule non-prod to shut down
- 5 Move cold data to cheaper classes
- 6 Cut cross-region and internet egress

EXPLANATION

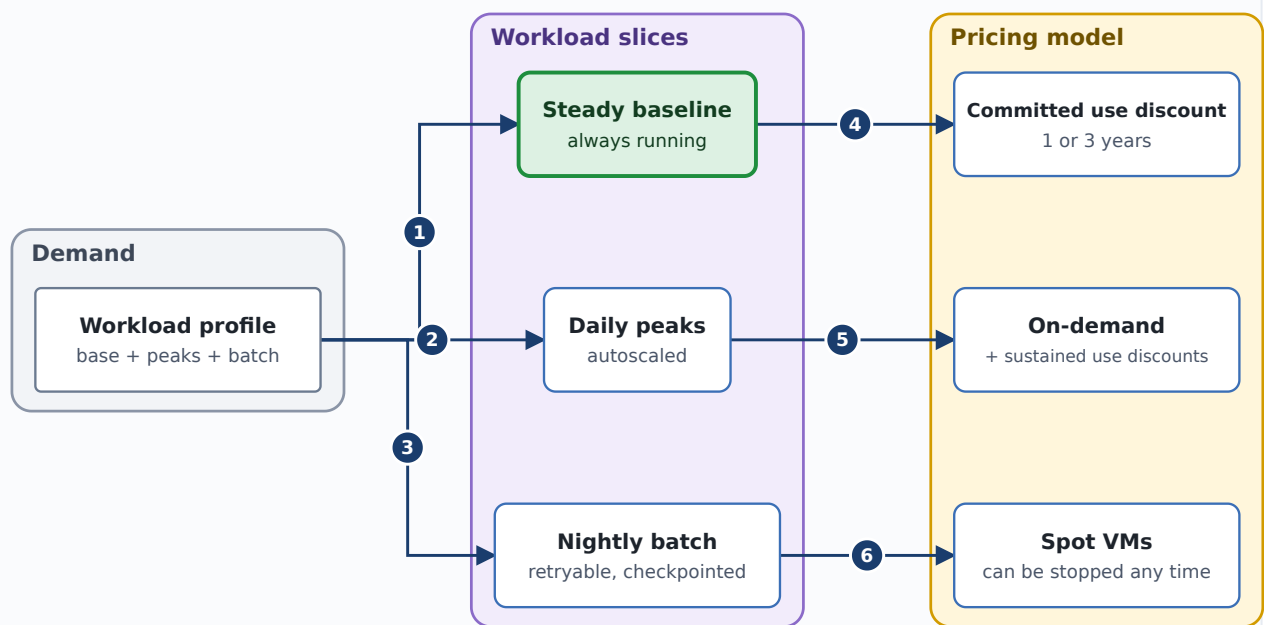
Start with data: export billing to BigQuery, group by project, label and SKU, and pick the few services that make up most of the spend. Then apply the levers in order: remove idle resources and rightsize using Recommender, scale with demand and switch off non-prod at night, and only then buy committed use discounts for the remaining steady baseline so you do not commit to waste. Move batch and CI to Spot VMs, apply storage lifecycle rules or Autoclass, and reduce egress with CDN and same-region placement. Track savings per change so the 25% is measurable.

KEY POINTS

- Order matters: clean up, rightsize, then commit.
- Make cost a team metric with labels and showback.

Q388 A platform runs a 24x7 API fleet, daily traffic peaks, and a large nightly batch. How do you combine committed use discounts, on-demand and Spot VMs?

FLOW DIAGRAM



- ❶ Size the minimum that runs all month
- ❷ Resource-based or spend-based commitments
- ❸ Peaks stay flexible; SUDs apply on eligible families
- ❹ Large discounts; design for preemption

EXPLANATION

Commit only the baseline that runs all month, using resource-based CUDs for specific machine families in a region, or spend-based (flexible) CUDs when usage moves across families and regions. Leave the daily peaks on on-demand capacity behind autoscaling; sustained use discounts apply automatically on eligible machine families. Run the nightly batch on Spot VMs, which are much cheaper but can be reclaimed at any time, so jobs must checkpoint and retry. Review commitment coverage and utilisation monthly, because unused commitments are still billed.

KEY POINTS

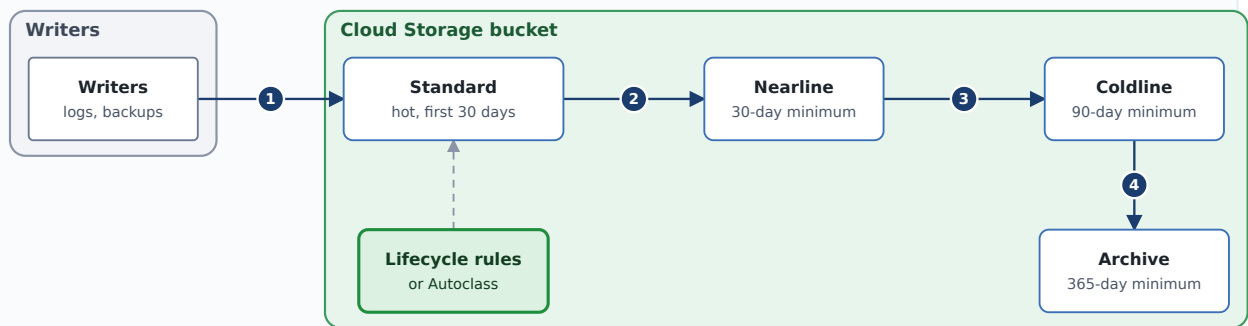
- CUDs are billed whether used or not.
- Spot suits stateless, retryable, time-flexible work.

CONFIGURATION EXAMPLE

```
gcloud compute instance-templates create batch-spot \
  --machine-type=n2-standard-8 --provisioning-model=SPOT \
  --instance-termination-action=DELETE
gcloud compute commitments create api-baseline --region=us-central1 \
  --plan=36-month --resources=vcpu=64,memory=256GB
```

Q389 A bucket holds 800 TB of logs and backups that are rarely read after the first month. How do you cut its cost?

FLOW DIAGRAM



- 1 New objects land in Standard
- 2 Age 30 days: SetStorageClass NEARLINE
- 3 Age 90 days: COLDLINE
- 4 Age 365 days: ARCHIVE, or delete after retention
- Rules run daily on the whole bucket

EXPLANATION

Add lifecycle rules that move objects to Nearline, Coldline and Archive by age and delete them when the retention period ends, or enable Autoclass when access patterns are unpredictable. Colder classes have lower storage prices but higher retrieval costs and minimum storage durations (30, 90 and 365 days), so early deletion or frequent reads can cost more than they save. Check whether objects are read (Storage Insights or access logs) before choosing classes. Also consider object versioning settings and soft delete retention, which can quietly double stored bytes.

KEY POINTS

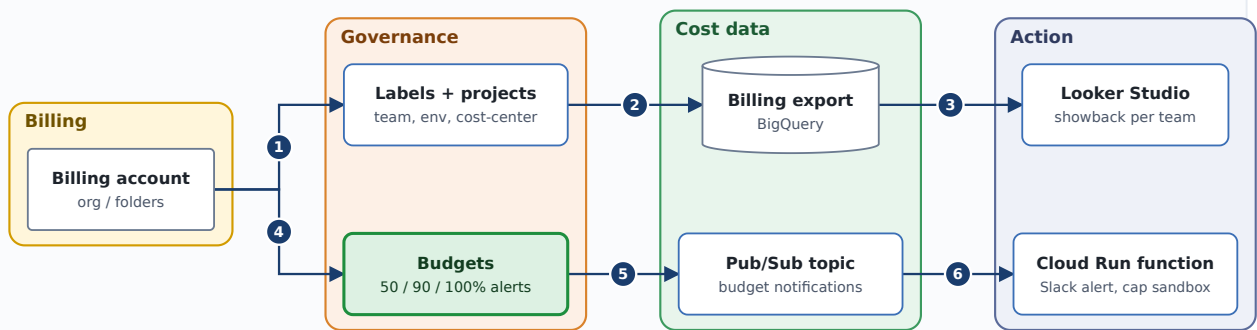
- Colder class = cheaper storage, dearer reads.
- Watch noncurrent versions and soft-deleted objects.

CONFIGURATION EXAMPLE

```
cat > lifecycle.json <<'EOF'
{"rule": [
  {"action": {"type": "SetStorageClass", "storageClass": "NEARLINE"}, "condition": {"age": 30}},
  {"action": {"type": "SetStorageClass", "storageClass": "COLDLINE"}, "condition": {"age": 90}},
  {"action": {"type": "Delete"}, "condition": {"age": 730}}
]}
EOF
gcloud storage buckets update gs://acme-logs --lifecycle-file=lifecycle.json
```

Q390 Forty teams share one billing account and nobody knows who spends what. How do you set up a FinOps practice on Google Cloud?

FLOW DIAGRAM



- 1 Project per team/env; labels enforced in Terraform
- 2 Detailed export carries labels on every line item
- 3 Showback or chargeback reports
- 4 Budgets per project or label, including forecasts
- 5 Programmatic notifications
- 6 Notify owners or stop spend in sandboxes

EXPLANATION

Make ownership visible first: one project per team and environment, required labels (team, env, cost-center) enforced in Terraform and checked in CI, and a folder structure that mirrors the organisation. Export detailed billing data to BigQuery and build showback dashboards so each team sees its own spend and trends. Create budgets per project or label with actual and forecasted thresholds, and send notifications to Pub/Sub so automation can alert owners or disable billing on sandbox projects. Run a regular review where engineering, finance and product agree on commitments and optimisation actions.

KEY POINTS

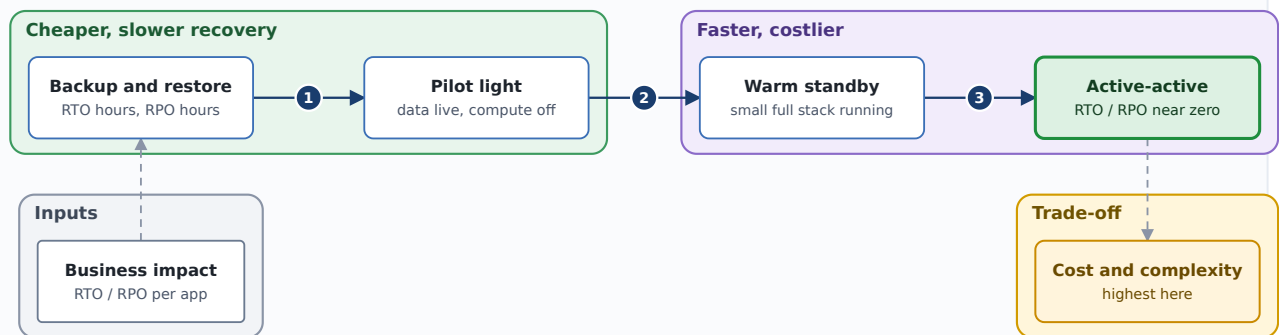
- Budgets alert; they do not cap spend on their own.
- Unlabelled spend is the first metric to drive down.

CONFIGURATION EXAMPLE

```
gcloud billing budgets create --billing-account=0X0X0X-0X0X0X-0X0X0X \
  --display-name="team-a monthly" --budget-amount=5000USD \
  --filter-projects=projects/team-a-prod \
  --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 \
  --threshold-rule=percent=1.0,basis=forecasted-spend \
  --notifications-rule-pubsub-topic=projects/finops/topics/budgets
```

Q391 Explain the main disaster recovery patterns, their typical RTO/RPO, and how you would pick one for a given application on Google Cloud.

FLOW DIAGRAM



- 1 Add continuous data replication to another region
- 2 Keep a scaled-down copy of the app running
- 3 Serve production traffic from both regions
- Choose the cheapest pattern that meets the targets

EXPLANATION

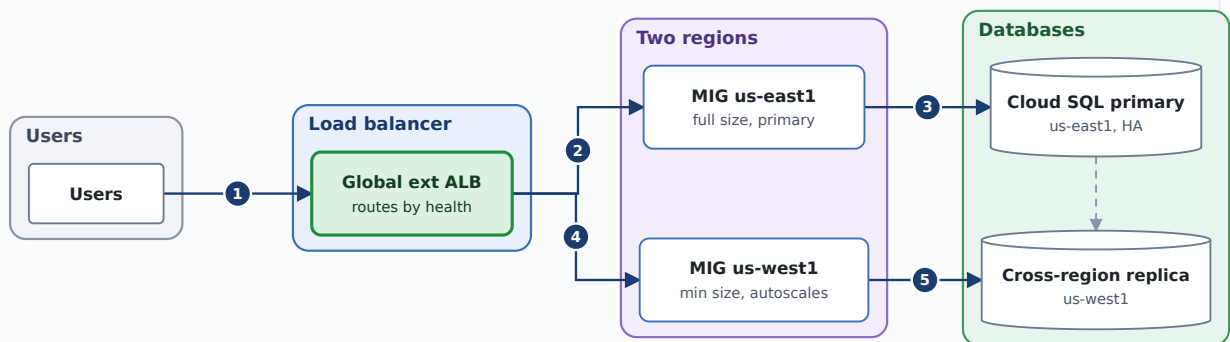
Backup and restore copies data (Backup and DR, snapshots, exports to another region) and rebuilds with Terraform, giving RTO and RPO in hours. Pilot light keeps data replicated continuously, for example a Cloud SQL cross-region replica, with compute off or at zero, so RTO drops to tens of minutes. Warm standby runs a small but complete copy of the stack that scales up on failover, and active-active serves traffic from several regions with data such as Spanner multi-region, giving near-zero RTO and RPO. Pick per application from a business impact analysis, and test the chosen plan regularly.

KEY POINTS

- RTO = time to recover; RPO = data you can lose.
- An untested DR plan is an assumption.

Q392 A regional order system (MIG + Cloud SQL) needs RTO under 30 minutes and RPO under 5 minutes if us-east1 is lost. Design a warm standby.

FLOW DIAGRAM



- 1 One IP; no DNS change during failover
- 2 Normal traffic
- Asynchronous replication, lag in seconds
- 4 On failure traffic shifts and the autoscaler grows the standby
- 5 Promote the replica; standby writes to the new primary

EXPLANATION

Run the app in a full-size MIG in us-east1 and a small MIG in us-west1, both behind one global external Application Load Balancer, so failover is automatic for the web tier. Keep a Cloud SQL cross-region read replica in us-west1; replication lag is normally seconds, which meets the 5-minute RPO. A runbook (ideally automated with Workflows or a script) promotes the replica and points the app at it, and the standby MIG scales up; make sure regional quotas allow full size. Rehearse the failover, and plan fail-back, because after promotion the old primary cannot simply rejoin.

KEY POINTS

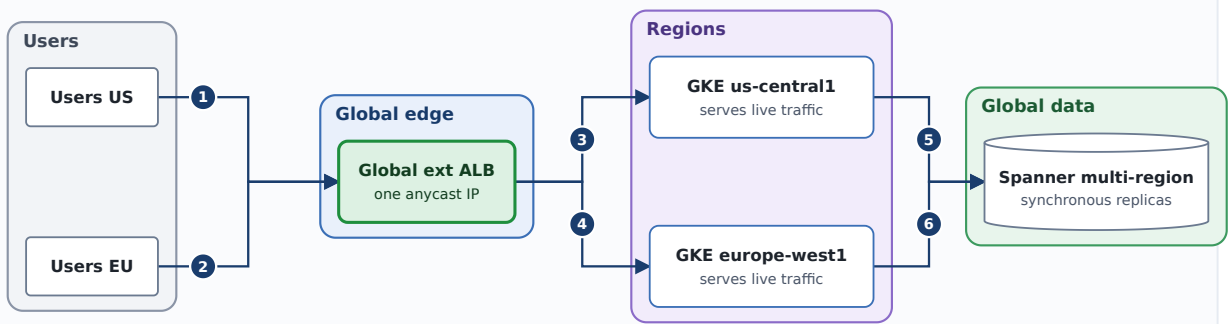
- The LB handles compute failover; database promotion is the critical path.
- Cloud SQL Enterprise Plus adds managed DR switchover; check current features.

CONFIGURATION EXAMPLE

```
gcloud sql instances create orders-dr --master-instance-name=orders-db \
  --region=us-west1 --tier=db-custom-4-16384
# during a disaster
gcloud sql instances promote-replica orders-dr
```

Q393 A global SaaS must stay available if a whole region fails, with no data loss.
How do you design active-active across regions?

FLOW DIAGRAM



- ③ Nearest healthy region
- ④ Either region can take all traffic
- ⑤ Strongly consistent reads and writes from both regions

EXPLANATION

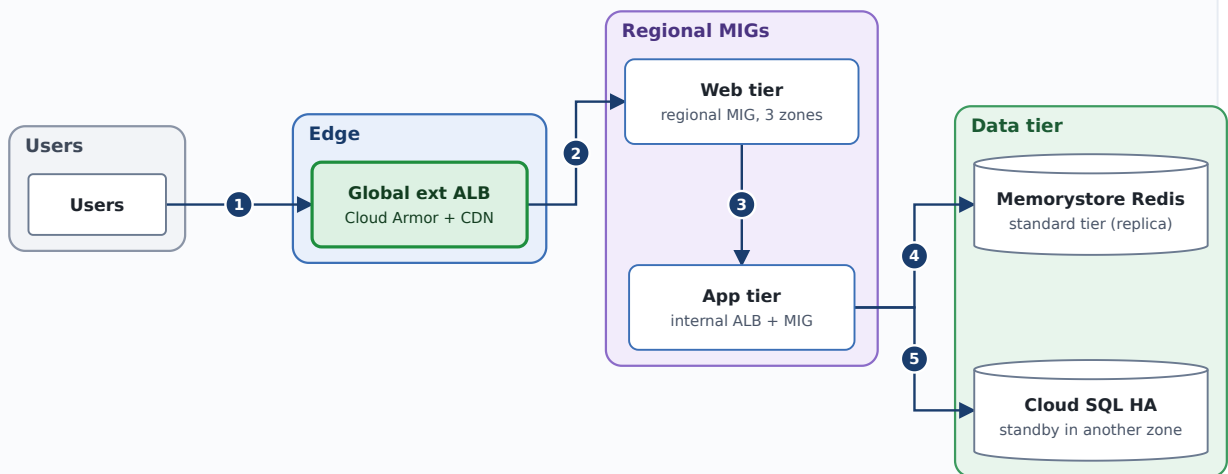
Serve traffic from two or more regions behind a global external Application Load Balancer (or multi-cluster Gateway for GKE), keeping services stateless and deploying the same versions everywhere through one pipeline. Use a data store that replicates synchronously across regions, such as Spanner in a multi-region configuration, so a regional loss causes no data loss and no manual promotion. Size each region to absorb the others' load (or use three regions with N+1), and keep caches and sessions region-local or rebuildable. Active-active costs the most and needs careful handling of latency for writes, so reserve it for systems that need it.

KEY POINTS

- Stateless compute + globally consistent data.
- Capacity for a lost region must already exist or scale in time.

Q394 Design a highly available three-tier web application (web, app, database) in a single Google Cloud region.

FLOW DIAGRAM



- 1 TLS ends at Google's edge; WAF and caching applied
- 2 Spread across three zones; autohealing on
- 3 Internal ALB spreads calls across zones
- 4 Sessions and cache survive a zone loss
- 5 Synchronous standby; automatic failover in about a minute

EXPLANATION

Put an external Application Load Balancer with Cloud Armor in front of a regional managed instance group (or GKE) spread across three zones for the web tier, and an internal Application Load Balancer in front of the app tier, also regional. Use health checks for load balancing and separate autohealing checks, and autoscale on CPU or requests with a minimum that survives one zone failing. For data, use Cloud SQL with high availability (regional), which keeps a synchronous standby in another zone, and Memorystore standard tier for cache. Keep instances stateless and put static assets in Cloud Storage behind Cloud CDN.

KEY POINTS

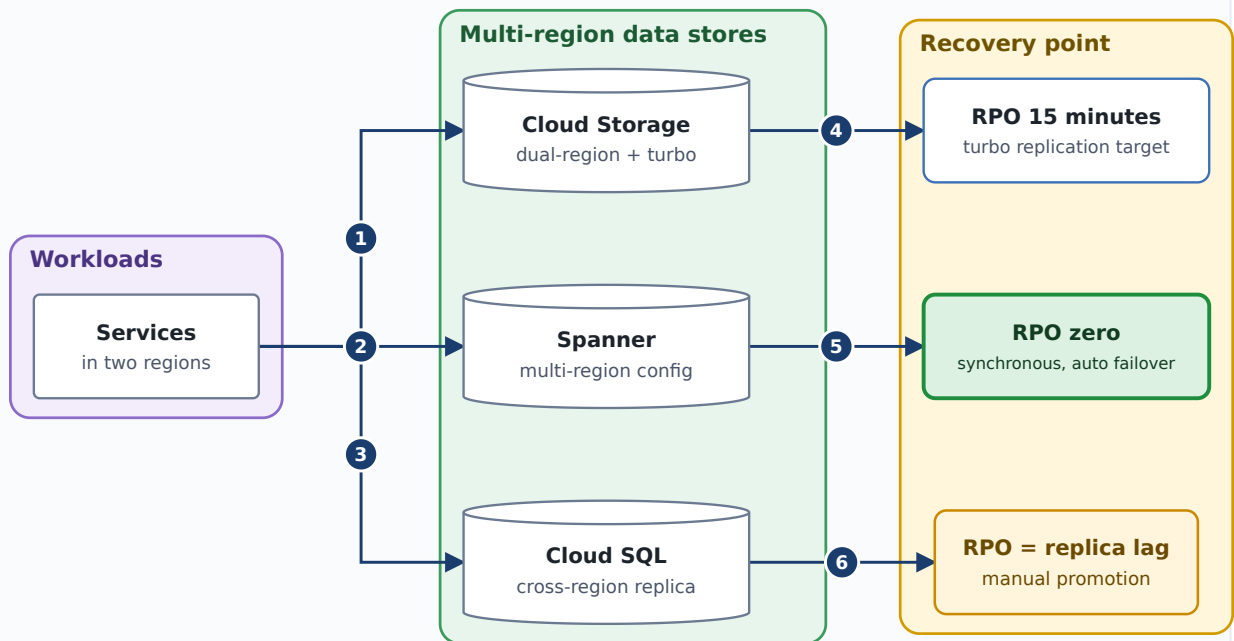
- Regional resources everywhere: MIG, SQL HA, Redis standard tier.
- Plan N+1 zones of capacity.

CONFIGURATION EXAMPLE

```
gcloud compute instance-groups managed create web-mig --region=us-central1 \
  --template=web-tmpl --size=3 --health-check=hc-web --initial-delay=120
gcloud compute instance-groups managed set-autoscaling web-mig \
  --region=us-central1 --min-num-replicas=3 --max-num-replicas=12 \
  --target-cpu-utilization=0.6
gcloud sql instances create app-db --database-version=POSTGRES_16 \
  --region=us-central1 --availability-type=REGIONAL --tier=db-custom-4-16384
```


Q395 Services run in two regions. How do you choose a multi-region strategy for object storage, relational data and analytics?

FLOW DIAGRAM



- 1 One bucket name in both regions
- 2 Global schema, strong consistency
- 3 Writes in one region, reads anywhere

EXPLANATION

For objects, dual-region or multi-region buckets keep one namespace across regions; default replication is asynchronous, and turbo replication on dual-region buckets targets an RPO of 15 minutes. For relational data that needs zero RPO and writes in both regions, use Spanner multi-region; if the app must stay on MySQL or PostgreSQL, Cloud SQL cross-region replicas give read scaling and DR with RPO equal to replication lag and a manual promotion. For analytics, choose BigQuery dataset locations deliberately, since location is fixed at creation, and use cross-region dataset replication if needed. Apply the `gcp.resourceLocations` org policy for residency rules.

KEY POINTS

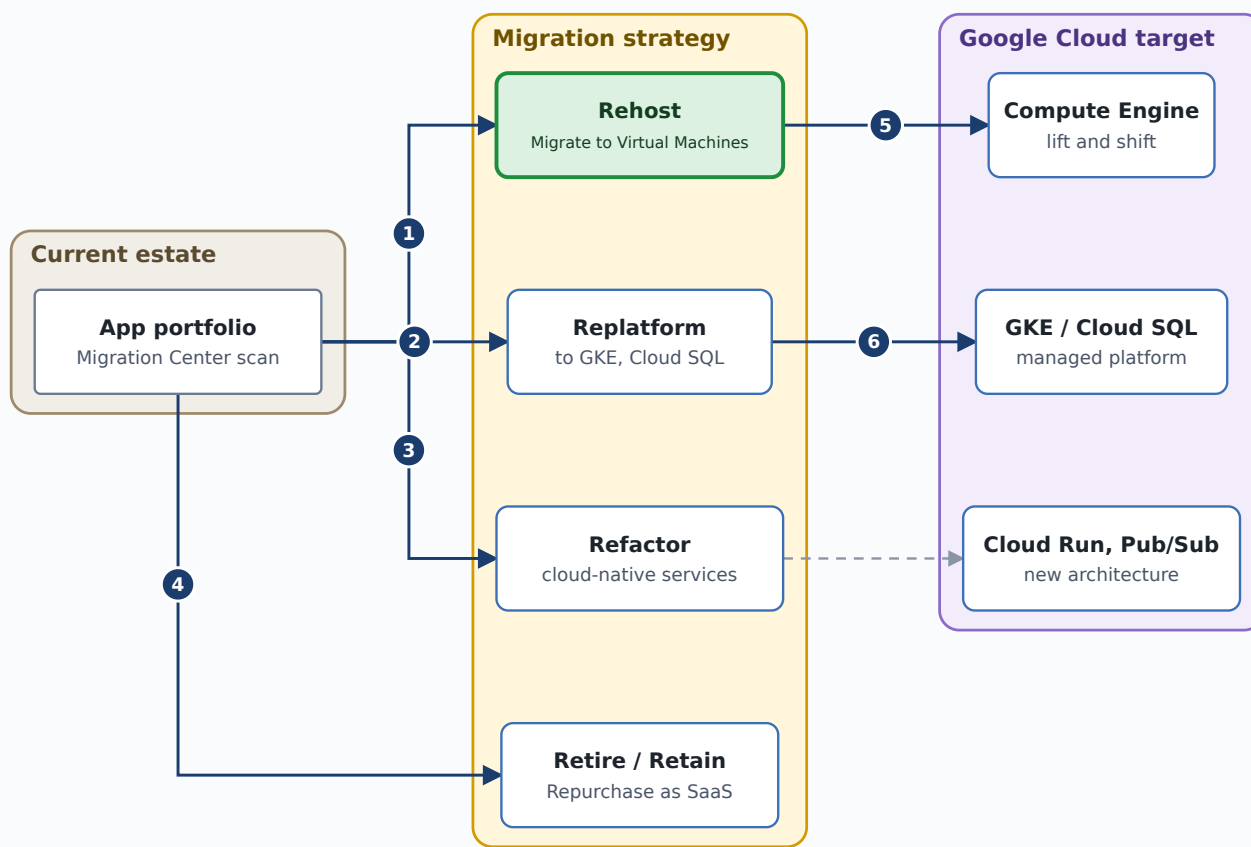
- Match each data store's RPO to the business need.
- Data location choices are hard to change later.

CONFIGURATION EXAMPLE

```
gcloud storage buckets create gs://acme-media --location=NAM4 --rpo=ASYNC_TURBO
gcloud spanner instances create orders --config=nam-eur-asia1 \
  --processing-units=1000 --description=Orders
```

Q396 A company wants to move 300 on-prem applications to Google Cloud in 18 months. How do you apply the 6 Rs and plan the migration?

FLOW DIAGRAM



- 1 Fastest path for most VMs
- 2 Small changes: managed database, containers
- 3 Rewrite where business value justifies it
- 4 Switch off, keep on-prem, or buy SaaS

EXPLANATION

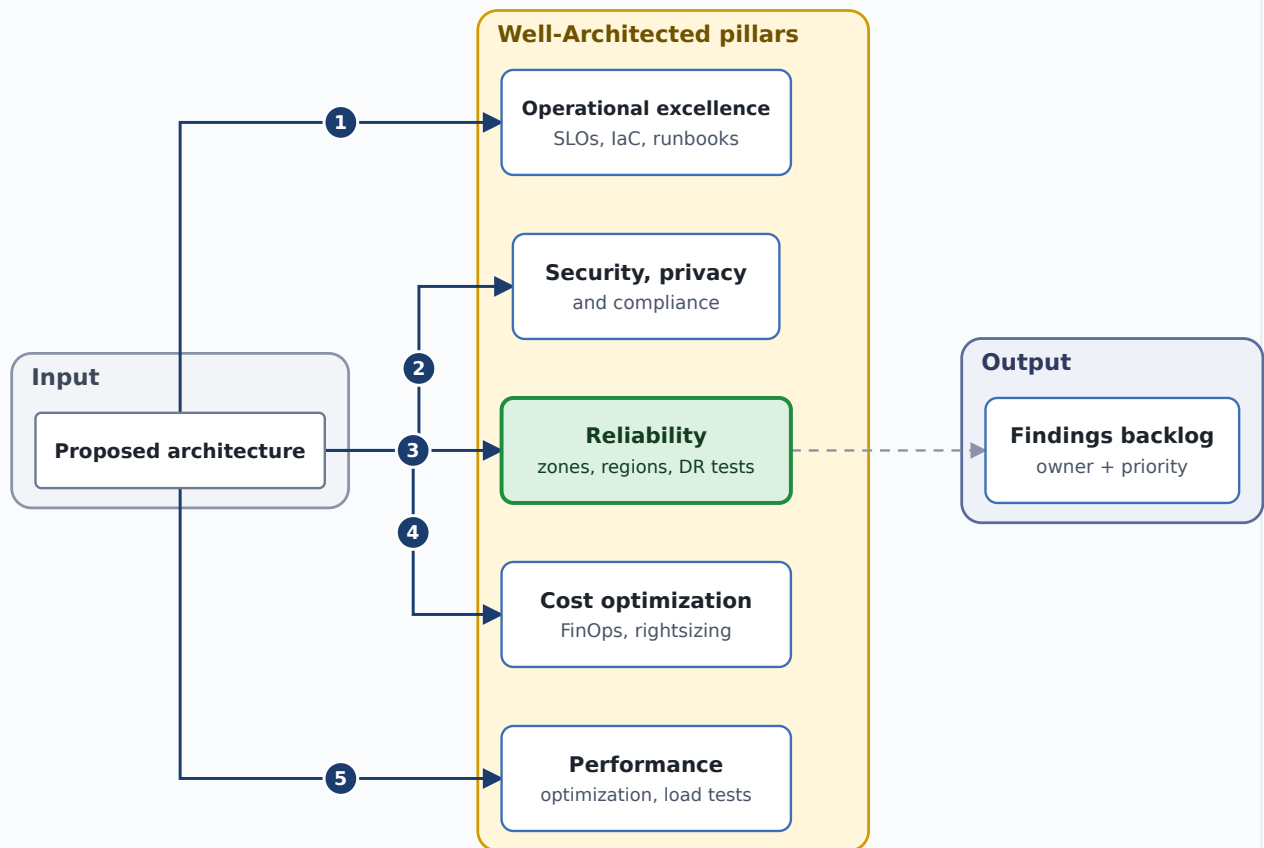
Start with discovery and assessment in Migration Center to build an inventory, dependency map and TCO estimate, then classify each app: rehost (lift and shift with Migrate to Virtual Machines), replatform (for example move the database to Cloud SQL with Database Migration Service or containerise onto GKE), refactor, repurchase as SaaS, retire, or retain on-prem. Group apps into waves by dependency and risk, and build the landing zone (org structure, IAM, networking, logging) before the first wave. Start with low-risk apps to prove the process, then move faster. Most large programmes rehost first and modernise later.

KEY POINTS

- Landing zone before wave 1.
- Dependency mapping prevents broken cut-overs.

Q397 You are asked to review a new architecture against the Google Cloud Well-Architected Framework. What pillars do you cover and what questions do you ask?

FLOW DIAGRAM



- 1 How is it deployed, monitored and operated?
 - 2 Least privilege, encryption, data residency, audit
 - 3 What fails when a zone or region goes down?
 - 4 Who owns the cost and how is it tracked?
 - 5 Does it scale and meet latency targets?
- Findings from every pillar go into one tracked backlog
 - Cross-pillar guidance also covers sustainability and AI/ML workloads (check current list)

EXPLANATION

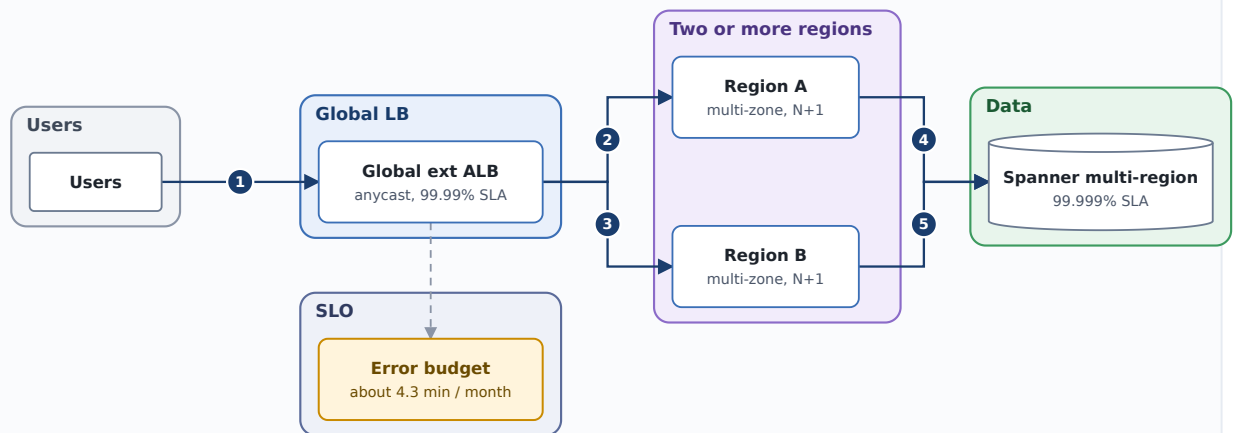
The framework's core pillars are operational excellence, security (with privacy and compliance), reliability, cost optimization and performance optimization, with cross-cutting guidance such as sustainability. For each pillar ask concrete questions: how is it deployed and rolled back, what are the SLOs and alerts; who can access what and is data encrypted and in the right location; what happens when a zone or region fails and when was DR last tested; what does it cost per unit of business value; and has it been load-tested. Record findings with owners and priorities rather than a pass/fail score.

KEY POINTS

- Pillars trade off: more reliability usually costs more.
- Turn findings into a tracked backlog.

Q398 The business wants 99.99% availability for a customer API. What does that mean in practice and how do you design for it on Google Cloud?

FLOW DIAGRAM



- ② Health checks remove a failed region in seconds
- ⑤ No manual database failover in the recovery path
- Measure availability at the LB, burn rate alerts

EXPLANATION

99.99% allows about 4.3 minutes of downtime per month, which is too short for any manual recovery, so failover must be automatic. Availability of components in series multiplies (two 99.95% services in series give about 99.9%), while redundant copies in parallel raise it, so a single region with one database will not reach 99.99%. Use a global load balancer over at least two regions, each multi-zone with N+1 capacity, and a data layer with automatic failover such as Spanner multi-region. Most outages come from changes, so add canary releases, fast rollback, SLO burn-rate alerting and regular failure testing.

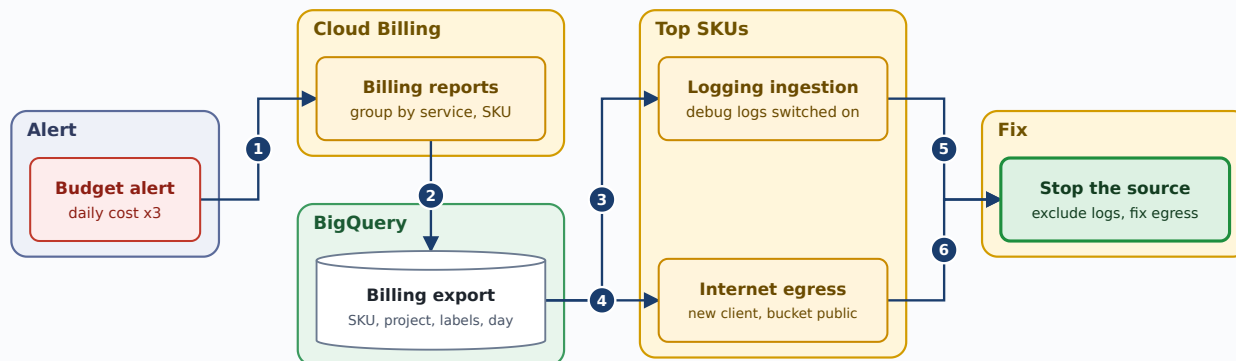
KEY POINTS

- Series multiplies; parallel redundancy adds nines.
- Change management matters as much as architecture.

Q399**Troubleshooting**

A budget alert fires: the project's daily cost jumped from about \$1,200 to \$4,000 in three days, and it is not BigQuery. How do you find the cause and stop the spend?

FLOW DIAGRAM



- 1 Find which service and SKU grew, and from when
- 2 Drill down to project, label and resource
- 3 Cloud Logging volume jumped after a release
- 4 Network egress grew with a new consumer
- 5 Exclusion filter, lower log level
- 6 CDN, same-region reads, access controls

EXPLANATION

In Billing reports, group by service and then by SKU and compare with the previous period to see exactly what grew and on which day. Then query the billing export in BigQuery by project, labels and resource to find the owner, and match the start time against deployments and audit logs. Two frequent non-BigQuery causes are a release that turned on debug logging (Cloud Logging ingestion SKU) and a new heavy consumer of internet or cross-region egress, such as a public bucket or a misplaced client. Stop the source first (log exclusion filter, revert the change, lock down the bucket), then add budget alerts per project and anomaly alerts so the next spike is caught in hours, not days.

KEY POINTS

- Label every resource with team and service so costs can be traced to an owner.
- Use Cloud Billing anomaly detection or a daily scheduled query on the export.

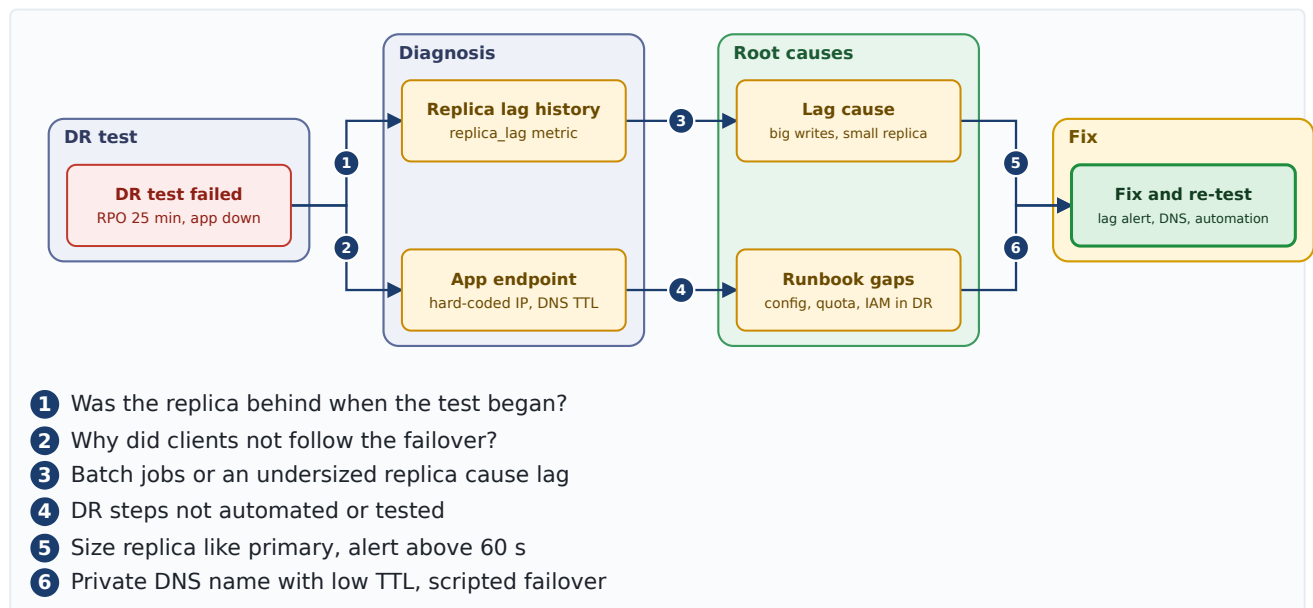
CONFIGURATION EXAMPLE

```

SELECT service.description AS service, sku.description AS sku,
       DATE(usage_start_time) AS day, ROUND(SUM(cost), 2) AS cost
FROM `billing-admin.billing.gcp_billing_export_v1_01A2B3_C4D5E6_F7G8H9`
WHERE usage_start_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
      AND project.id = 'shop-prod'
GROUP BY 1, 2, 3 ORDER BY cost DESC LIMIT 20;
-- stop a noisy log source
-- gcloud logging sinks update _Default --add-exclusion=name=debug,filter='severity<=DEBUG'
  
```

Q400**Troubleshooting**

In a DR test you promoted the Cloud SQL cross-region replica in us-west1, but 25 minutes of orders were missing (RPO target 5 minutes) and the app still pointed at the old region. How do you investigate and fix it?

FLOW DIAGRAM**EXPLANATION**

Asynchronous cross-region replication means your real RPO equals the replica lag at the moment of failover, so check the `database/replication/replica_lag` metric for the test window. Lag usually comes from large batch writes or long transactions, a replica with fewer vCPUs or less disk throughput than the primary, or replication that had stopped without anyone alerting. The app failure is a separate runbook gap: clients used a hard-coded IP or a long DNS TTL, and quotas, secrets or IAM were missing in the DR region. Size the replica like the primary, alert when lag exceeds the RPO budget, point apps at a DNS name you switch during failover, and automate and rehearse the whole failover regularly.

KEY POINTS

- Measure RPO and RTO in every DR test and track them like SLOs.
- Check that replication is running before starting a planned failover.

CONFIGURATION EXAMPLE

```

gcloud sql instances describe orders-replica-usw1 --format='value(replicaConfiguration,state)'
# lag in seconds (Monitoring metric)
# cloudsql.googleapis.com/database/replication/replica_lag
gcloud sql instances patch orders-replica-usw1 --tier=db-custom-8-32768
gcloud sql instances promote-replica orders-replica-usw1
gcloud dns record-sets update db.prod.internal. --type=A --ttl=30 \
  --rrdatas=10.20.0.5 --zone=prod-private
  
```

Answers reflect Google Cloud and DevOps practice as generally documented. Product names, limits, defaults and features change frequently; confirm current details in the official documentation.